

Thomas Richter
Henry von Wahl
Thomas Wick

Einführung in die Numerische Mathematik

Begriffe, Konzepte und
zahlreiche Anwendungs-
beispiele

2. Auflage



MOREMEDIA



Springer Spektrum

Einführung in die Numerische Mathematik

Thomas Richter · Henry von Wahl · Thomas Wick

Einführung in die Numerische Mathematik

Begriffe, Konzepte und zahlreiche
Anwendungsbeispiele

2., überarbeitete und erweiterte Auflage

Thomas Richter
Fakultät für Mathematik
Universität Magdeburg
Magdeburg, Deutschland

Henry von Wahl
Institut für Mathematik
Friedrich-Schiller-Universität Jena
Jena, Deutschland

Thomas Wick
Institut für Angewandte Mathematik
Leibniz Universität Hannover
Hannover, Deutschland

ISBN 978-3-662-69581-4 ISBN 978-3-662-69582-1 (eBook)
<https://doi.org/10.1007/978-3-662-69582-1>

Die Deutsche Nationalbibliothek verzeichnet diese Publikation in der Deutschen Nationalbibliografie; detaillierte bibliografische Daten sind im Internet über <https://portal.dnb.de> abrufbar.

© Der/die Herausgeber bzw. der/die Autor(en), exklusiv lizenziert an Springer-Verlag GmbH, DE, ein Teil von Springer Nature 2017, 2024

Das Werk einschließlich aller seiner Teile ist urheberrechtlich geschützt. Jede Verwertung, die nicht ausdrücklich vom Urheberrechtsgesetz zugelassen ist, bedarf der vorherigen Zustimmung des Verlags. Das gilt insbesondere für Vervielfältigungen, Bearbeitungen, Übersetzungen, Mikroverfilmungen und die Einspeicherung und Verarbeitung in elektronischen Systemen.

Die Wiedergabe von allgemein beschreibenden Bezeichnungen, Marken, Unternehmensnamen etc. in diesem Werk bedeutet nicht, dass diese frei durch jede Person benutzt werden dürfen. Die Berechtigung zur Benutzung unterliegt, auch ohne gesonderten Hinweis hierzu, den Regeln des Markenrechts. Die Rechte des/der jeweiligen Zeicheninhaber*in sind zu beachten.

Der Verlag, die Autor*innen und die Herausgeber*innen gehen davon aus, dass die Angaben und Informationen in diesem Werk zum Zeitpunkt der Veröffentlichung vollständig und korrekt sind. Weder der Verlag noch die Autor*innen oder die Herausgeber*innen übernehmen, ausdrücklich oder implizit, Gewähr für den Inhalt des Werkes, etwaige Fehler oder Äußerungen. Der Verlag bleibt im Hinblick auf geografische Zuordnungen und Gebietsbezeichnungen in veröffentlichten Karten und Institutionsadressen neutral.

Planung/Lektorat: Iris Ruhmann

Springer Spektrum ist ein Imprint der eingetragenen Gesellschaft Springer-Verlag GmbH, DE und ist ein Teil von Springer Nature.

Die Anschrift der Gesellschaft ist: Heidelberger Platz 3, 14197 Berlin, Germany

Wenn Sie dieses Produkt entsorgen, geben Sie das Papier bitte zum Recycling.

Vorwort

Begleitend zur Vorlesung *Einführung in die numerische Mathematik* im Sommersemester 2012 an der Universität Heidelberg ist die erste Version des Skripts entstanden. Die Vorlesung und der Inhalt des Skripts sind elementar gehalten und können mit grundlegenden Kenntnissen aus Analysis und linearer Algebra verstanden werden. Die numerische Mathematik unterscheidet sich jedoch wesentlich von diesen Vorlesungen. In der Analysis und linearen Algebra werden Strukturuntersuchungen zu verschiedenen Problemen gestellt: Ist eine Funktion integrierbar? Existiert die Lösung zu einem Gleichungssystem? Wie können Vektorräume charakterisiert werden? Und so weiter.

In der numerischen Mathematik steht die Berechnung konkreter Lösungen mittels approximativer Verfahren im Mittelpunkt. Neben dem Entwurf von Approximationsmethoden befasst sich die numerische Mathematik insbesondere mit der Analyse dieser Methoden hinsichtlich Genauigkeit und rechentechnischem Aufwand. Diese Verfahren bilden dann die Grundlagen des Wissenschaftlichen Rechnens, also der *mathematischen Modellierung* echter Problemstellungen aus verschiedenen Disziplinen, deren *Diskretisierung* und *Approximation* mithilfe von numerischen Methoden und die anschließende Umsetzung in *Algorithmen* zur Durchführung von *Computersimulationen*.

Es zeigt sich, dass viele mathematische Probleme nicht – oder nur sehr aufwendig – wirklich gelöst werden können. Und selbst wenn eine einfache Lösung existiert, etwa ist $x = \sqrt{2}$ eine der beiden Nullstellen von $f(x) = x^2 - 2$, so kann diese Lösung nicht exakt auf einem Taschenrechner oder Computer dargestellt werden. Numerische Algorithmen werden im Allgemeinen auf einem Computer implementiert, und Fehler, welche zum Beispiel durch Rundung entstehen, müssen in die Analyse mit einbezogen werden. Natürlich wird auch in der numerischen Mathematik mathematisch exakt vorgegangen. Wir können jedoch nicht davon ausgehen, dass mathematische Aufgaben, wie die Berechnung der Summe zweier Zahlen $x + y$, auch auf dem Computer exakt realisiert werden können. Diese Ungenauigkeit bei der Realisierung muss in der numerischen Analyse berücksichtigt werden.

Das Skript ist in zwei große Abschnitte unterteilt, die numerische Betrachtung von Problemen der linearen Algebra und die Untersuchung von Problemen der Analysis. Wir

versuchen dabei jeweils den Zusammenhang zwischen Problem, mathematischer Analyse und numerischem Verfahren herzustellen.

August 2012, Heidelberg

Thomas Richter, Thomas Wick

Für die Vorlesungen *Einführung in die Numerische Mathematik* im Sommersemester 2014 an der Universität Heidelberg und *Numerische Mathematik* im Wintersemester 2015/2016 an der Universität Erlangen wurde das Skript in einigen Teilen überarbeitet, ergänzt und korrigiert und insbesondere um zahlreiche konkrete Beispiele erweitert. Insbesondere aber konnten durch die intensive Korrektur aller Tutoren zahllose kleine Fehler beseitigt werden.

März 2014 und Oktober 2015, Heidelberg und Erlangen

Thomas Richter

Für die Vorlesung *Numerical Modeling for STEEM (MAP502)* im Wintersemester 2016/2017 an der École Polytechnique, Université Paris-Saclay, wurden Teile des Kapitels *Nullstellenbestimmung* ins Englische übersetzt, weiter ausgearbeitet, und anschließend in die deutsche Version zurückübersetzt.

November 2016, Paris

Thomas Wick

Erste Auflage des Buches

Für die erste Auflage des Buches wurde das Material weiter überarbeitet und vervollständigt. Viele kleinere Fehler konnten beseitigt werden. Ganz neu hinzugekommen und ein besonderes Anliegen sind anwendungsbezogene Beispiele, sogenannte *Exkurse* aus verschiedensten Bereichen der Forschung, Technik und des täglichen Lebens, in denen die numerische Mathematik auf den ersten Blick nicht sichtbar ist, jedoch einen großen Stellenwert einnimmt.

Februar 2017, Magdeburg und Paris

Thomas Richter, Thomas Wick

Zweite Auflage des Buches

Für die zweite Auflage wurden zahlreiche Fehler korrigiert, für deren Hinweise wir sehr dankbar sind, Material zu neuronalen Netzen hinzugefügt sowie ein weiteres Kapitel zu Grundlagen der numerischen Analysis gestaltet. Aufgrund der Aktualität von Numerischer Mathematik in vielen Anwendungen nehmen wir mit großer Freude einen weiteren Hauptautor mit auf, sodass Python Code-Snippets den Weg des wissenschaftlichen Rechnens von mathematischen Aufgaben, über Algorithmen-Design via Pseudo-Code bis hin zur tatsächlichen Implementierung besser veranschaulichen.

☛ Ergänzende Python-Programme in jupyter-Notebooks haben wir auf GitHub unter github.com/sn-code-inside/EinfNumMath-RWW zu den meisten Abschnitten dieses Buchens zusammengefasst. Die entsprechenden Stellen sind mit dem Python-Logo markiert.

Neben dem Gebrauch des Buches an unseren Heimatuniversitäten, wurde das Buch auch international genutzt, beispielsweise im Kurs MAP 502 der Ecole Polytechnique sowie im Rahmen des DAAD geförderten Projektes PeCCC (Peruvian Comptenence Center for Scientific Computing) an verschiedenen Universitäten Lateinamerikas. In der On-line-Lehre der Jahre 2020 – 2021 ist uns das Buch eine wertvolle Stütze gewesen.

Magdeburg

Wien

Jena

Hannover

April 2024

Thomas Richter

Henry von Wahl

Thomas Wick

Inhaltsverzeichnis

1	Einleitung	1
1.1	Konzepte der numerischen Mathematik	3
1.1.1	Approximation	3
1.1.2	Konvergenz	4
1.1.3	Fehler	5
1.1.4	Fehlerabschätzung	5
1.1.5	Konvergenzgeschwindigkeit	7
1.1.6	Effizienz	7
1.1.7	Stabilität	9
1.2	Definition eines Algorithmus	11
1.2.1	Numerischer Aufwand und Landau-Symbole	15
1.3	Fehler, Fehlerverstärkung und Konditionierung	17
1.3.1	Konditionierung der Grundoperationen	22
1.4	Zahldarstellung und deren Bedeutung in der Numerik	23
1.4.1	Zahldarstellungen	23
1.4.2	Maschinengenauigkeit	29
1.4.3	Rechengeschwindigkeit – Floating point operations per second	31
1.5	Rundungsfehleranalyse und Stabilität von numerischen Algorithmen	32
1.6	Exkurs: Die Steuerfunktion in der Einkommensteuerberechnung	35

Teil I Numerische Methoden der linearen Algebra

2	Grundlagen der linearen Algebra	41
3	Lineare Gleichungssysteme	51
3.1	Störungstheorie und Stabilitätsanalyse von linearen Gleichungssystemen	52
3.2	Das Gaußsche Eliminationsverfahren und die LR-Zerlegung	56
3.2.1	LR-Zerlegung mit Pivotisierung	64

3.2.2	LR-Zerlegung für diagonaldominante Matrizen	72
3.3	Die Cholesky-Zerlegung für symmetrisch positiv definite Matrizen	74
3.4	Dünn besetzte Matrizen und Bandmatrizen	77
3.4.1	Dünn besetzte Matrizen in der Praxis	80
3.4.2	Der Cuthill-McKee-Sortieralgorithmus	81
3.5	Exkurs: LR-Zerlegung auf Parallelrechnern	87
3.5.1	Konzepte des parallelen Rechnens	88
3.5.2	Parallele LR-Zerlegung	94
3.6	Nachiteration	99
3.7	Fixpunktverfahren zum Lösen linearer Gleichungssysteme	102
3.7.1	Konvergenzkriterium für Jacobi- und Gauß-Seidel-Iteration	108
3.7.2	Relaxationsverfahren: das SOR-Verfahren	113
3.7.3	Praktische Aspekte	117
3.8	Exkurs: Numerische Approximation des Minimalflächenproblems	121
3.8.1	Modellierung	121
3.8.2	Diskretisierung	127
3.8.3	Beispiele	134
4	Orthogonalisierungsverfahren und die QR-Zerlegung	139
4.1	Die QR-Zerlegung	140
4.2	Das Gram-Schmidt-Verfahren	143
4.3	Householder-Transformationen	151
4.4	Givens-Rotationen	158
4.5	Überbestimmte Gleichungssysteme	162
4.6	Exkurs: Astronomische Hauptachsenbestimmung eines Himmelskörpers	170
5	Das Eigenwertproblem	173
5.1	Konditionierung der Eigenwertaufgabe	174
5.2	Direkte Methode	178
5.3	Iterative Verfahren	179
5.4	Zerlegungsverfahren zur Eigenwertbestimmung	187
5.4.1	Cholesky-Verfahren	189
5.4.2	QR-Verfahren	194
5.5	Reduktionsmethoden	199
5.6	Exkurs: Eigenschwingungen eines Mehrfach-Federpendels	206
5.6.1	Mathematische Modellierung	206
5.6.2	Numerische Approximation der Differentialgleichung	213
5.7	Singulärwertzerlegung	220
5.7.1	Approximation der Singulärwertzerlegung	224
5.7.2	Anwendungen der Singulärwertzerlegung	229
5.8	Exkurs: Singulärwertzerlegung zur Bildkompression	236

6	Krylowraumverfahren für lineare Gleichungssysteme und das Eigenwertproblem.	239
6.1	Galerkin-Gleichung und Krylow-Teilräume	240
6.2	Das CG-Verfahren für symmetrisch positiv definite Gleichungssysteme	242
6.3	Vorkonditioniertes CG-Verfahren	248
6.4	Das Arnoldi-Verfahren	252
6.4.1	Das Eigenwertproblem.	255
6.5	Das GMRES-Verfahren	257
6.5.1	Praktische Aspekte und Implementierung	260
6.5.2	Vorkonditioniertes GMRES-Verfahren	269
 Teil II Numerische Methoden der Analysis		
7	Grundlagen der Analysis	273
8	Nullstellenbestimmung	283
8.1	Stabilität und Kondition	284
8.2	Intervallschachtelung	291
8.3	Das Newton-Verfahren	294
8.3.1	Varianten des Newton-Verfahrens	301
8.3.2	Konvergenzbegriffe	313
8.4	Nullstellensuche im $\mathbb{R}^n$	316
8.4.1	Newton-Verfahren im $\mathbb{R}^n$	317
8.4.2	Globalisierung des Newton-Verfahrens	326
8.5	Exkurs: Nullstellensuche im Komplexen	334
8.5.1	Das Verfahren von Bairstow	338
8.5.2	Das komplexe Newton-Verfahren	341
8.6	Exkurs: Fast Inverse Square Root	346
8.6.1	Shading – Schnelle Berechnung von Normalvektoren	347
8.6.2	Newton-Verfahren zur Berechnung der inversen Wurzel	348
8.6.3	Bestimmung des Startwerts	350
9	Interpolation und Approximation	359
9.1	Polynominterpolation	361
9.1.1	Lagrangesche Basispolynome	362
9.1.2	Newtonsche Basispolynome	364
9.1.3	Interpolation von Funktionen und Fehlerabschätzungen	370
9.1.4	Hermiteinterpolation	376
9.2	Stückweise Interpolation	376
9.3	Numerische Differentiation	384
9.3.1	Approximation der ersten Ableitung	384
9.3.2	Approximation der zweiten Ableitung	386

9.3.3	Stabilität der numerischen Differentiation	388
9.4	Richardson-Extrapolation zum Limes	389
9.5	Bestapproximation	396
9.5.1	Gauß-Approximation – beste Approximation in der L^2 -Norm	397
9.5.2	Tschebyscheff-Approximation	411
9.5.3	Optimale Lagrange-Interpolation	420
9.6	Trigonometrische Interpolation	422
9.7	Diskrete Fourier-Transformation	430
9.8	Exkurs: Das JPEG-Format zur Komprimierung von Bildern	433
9.9	Exkurs: Numerische Bausteine zur Lösung von Anfangswertproblemen	441
9.9.1	Herleitung des Anfangswertproblems	442
9.9.2	Numerische Behandlung	442
9.9.3	Ein lineares Anfangswertproblem	444
9.9.4	Ein nichtlineares Anfangswertproblem	448
10	Numerische Integration	451
10.1	Interpolatorische Quadratur	453
10.2	Stückweise interpolatorische Quadraturformeln	459
10.3	Romberg-Quadratur	462
10.4	Gauß-Quadratur	469
10.4.1	Gauß-Legendre-Quadratur	472
10.4.2	Gauß-Tschebyscheff-Quadratur	480
10.5	Exkurs: Keplersche Fassregel	484
11	Gradientenverfahren und künstliche neuronale Netze	487
11.1	Nichtlineare Optimierung	488
11.1.1	Das Gradientenverfahren	489
11.1.2	Das Newton-Verfahren zur Minimierung	492
11.2	Abstiegs- und Gradientenverfahren für lineare Gleichungssysteme	495
11.3	Konvergenzbeweis des CG-Verfahrens	503
11.4	Exkurs: Preisbildung für Honigverkauf	507
11.5	Künstliche neuronale Netze	512
11.5.1	Aufbau künstlicher neuronaler Netze	513
11.5.2	Approximation mit neuronalen Netzen	517
11.5.3	Trainieren von künstlichen neuronalen Netzen	522
11.6	Exkurs: Eigenwertbestimmung mit künstlichen neuronalen Netzen	535
11.6.1	Training des Netzes	536
11.6.2	Einfluss der Netzwerkgröße	537
11.6.3	Einfluss der Trainingsdaten	538
11.6.4	Generalisierung	539

Verzeichnis der Exkurse	543
Python-Programme und jupyter-Notebooks	545
Literatur	547
Stichwortverzeichnis	551



In der numerischen Mathematik werden Verfahren zum „Lösen“ von mathematischen Problemen und Aufgaben entworfen und analysiert. Dabei ist die numerische Mathematik eng mit anderen Zweigen der Mathematik und der Informatik verbunden und kann oft nicht von Anwendungen in Chemie, Physik oder Medizin, den Ingenieurwissenschaften oder Ökonomie getrennt werden. Die Aufgaben, die wir zu lösen versuchen, stellen wir uns meist nicht selbst, sondern sie kommen aus den unterschiedlichsten Bereichen. Auf diese Veranschaulichungen legen wir in diesem Buch besonders Wert und kennzeichnen die Abschnitte als *Exkurse*.

Der übliche Weg vom Problem zur Lösung ist lang und kann in folgende Schritte unterteilt werden:

1. *Mathematische Modellierung*. Das zugrundeliegende Problem wird mathematisch erfasst, es werden Gleichungen (z. B. basierend auf physikalischen Gesetzmäßigkeiten) entwickelt, die das Problem beschreiben. Ergebnis des mathematischen Modells kann ein lineares Gleichungssystem sein, eine Differentialgleichung oder eine Funktion, deren Nullstellen gesucht sind.
2. *Analyse des Modells*. Das mathematische Modell muss auf seine Eigenschaften untersucht werden: Existiert eine Lösung, ist diese Lösung eindeutig? Hängt die Lösung stetig von den Daten ab? Hier kommen alle Teilgebiete der Mathematik zum Einsatz, von der Analysis, linearen Algebra über Statistik, Gruppentheorie, Funktionalanalysis zur Theorie von partiellen Differentialgleichungen.
3. *Numerische Verfahrensentwicklung und deren Analyse*. Ein numerisches Lösungs- oder Approximationsverfahren wird für das Modell entwickelt. Die mathematische Aufgabe wird dazu algorithmisch in mathematischer Notation aufbereitet, anschließend in Pseudo-Code formuliert, und danach in einer Programmiersprache implementiert. Viele mathematische Modelle (etwa Differentialgleichungen) können nicht exakt gelöst werden und oft kann eine Lösung, auch wenn sie existiert, nicht angegeben werden. Die Lösung einer

Differentialgleichung ist eine Funktion $f : [a, b] \rightarrow \mathbb{R}$ und zur genauen Beschreibung müsste der Funktionswert an den unendlich vielen Punkten des Intervalls $[a, b]$ bestimmt werden. Jede durchführbare Verfahrensvorschrift kann allerdings nur aus endlich vielen Schritten bestehen. Das Problem muss zunächst *diskretisiert* werden, also auf eine endlich-dimensionale Aufgabe reduziert werden. Die numerische Mathematik befasst sich neben der Algorithmenentwicklung mit der Analyse der numerischen Verfahren, also mit Untersuchung von Konvergenz und Approximationsfehlern, wenn sich die diskretisierte Lösung der unendlich-dimensionalen Lösung immer weiter annähert (durch Hinzunahme weiterer Punkte in $[a, b]$).

4. *Implementierung*. Das numerische Verfahren muss auf einem Computer implementiert werden. Eine *effiziente Implementierung* erfordert gute Kenntnisse in einer Programmiersprache (zum Beispiel C++, MATLAB, Python, Octave, Java, Julia, Fortran, etc.). Ein *effizientes Programm* erfordert die Entwicklung spezieller Algorithmen. Um moderne Computerarchitekturen nutzen zu können, muss das Verfahren z.B. für die Verwendung von Parallelrechnern modifiziert werden. Gleichzeitig ist man an der Entwicklung *robuster Algorithmen* interessiert. Das heißt, für Änderungen in der Geometrie, der Intervalllänge oder verschiedener Problem-Parameter sollte der Algorithmus immer ähnlich zuverlässig funktionieren.
5. *Auswertung*. Die numerischen Ergebnisse müssen ausgewertet werden. Dies beinhaltet eine grafische oder tabellarische Darstellung der Ergebnisse sowie in technischen Anwendungen z.B. die Auswertung von Kräften, die nur indirekt gegeben sind.
6. *Interpretation*. Anhand von Plausibilitätsanalysen muss die Qualität der Ergebnisse beurteilt werden. Solche Analysen erfolgen entweder in Vergleichen zu bekannten (analytischen) Lösungen oder zu experimentellen Daten. Rein numerische Analysen bei fehlenden analytischen oder experimentellen Daten mittels rechengestützten Konvergenzanalysen sind ebenfalls möglich und entsprechende Werkzeuge geben wir in diesem Buch an die Hand.

Die oben genannten Teilaspekte dürfen nicht getrennt voneinander gesehen werden. Ein effizienter Algorithmus ist nichts wert, wenn das zugrundeliegende Problem überhaupt keine wohldefinierte Lösung hat, und ein numerisches Verfahren für ein Anwendungsproblem ist wertlos, wenn der Computer in absehbarer Zeit zu keiner Lösung kommt.

Des Weiteren wird die obige Liste in der Regel nicht sequenziell abgearbeitet, die Auswertung der Ergebnisse kann Anlass zur Überprüfung der zugrundeliegenden mathematischen Modelle geben und zu modifizierten numerischen Verfahren führen. Wir haben es somit mit einem ständigen Zusammenspiel von Modellierung, Analyse, numerischer Approximation und Anwendung zu tun.

Letztendlich legen wir mit den oben diskutierten Schritten die Basis des *Wissenschaftlichen Rechnens*, welches sich als dritte Säule neben Experiment und Theorie etabliert hat. Die numerische Mathematik hat Züge einer experimentellen Wissenschaft,

in der wir mit Algorithmen und Computerprogrammen, unter Ausnutzung klar definierter mathematischer Strukturen experimentieren.

1.1 Konzepte der numerischen Mathematik

Da sich die Ziele und das Vorgehen in der numerischen Mathematik zum Teil stark von anderen Disziplinen unterscheiden, führen wir in diesem Abschnitt die wesentlichen Konzepte ein, welche charakteristisch sind und uns in allen weiteren Kapiteln dieses Buches immer wieder begegnen. Im Folgenden diskutieren wir mit Hilfe von Beispielen und Definitionen sieben zentrale Konzepte.

1.1.1 Approximation

Typischerweise geht es in der numerischen Mathematik um die Berechnung von mathematischen Problemen und die Angabe von konkreten Lösungen. Von vielen Problemen wissen wir, dass eine Lösung existiert, kennen aber kein Verfahren, um diese Lösung auszurechnen. Ein Beispiel ist die Suche nach Nullstellen eines allgemeinen Polynoms

$$p(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + \dots + a_nx^n.$$

Der Fundamentalsatz der Algebra [60] besagt, dass jedes (nichtkonstante) Polynom mindestens eine (gegebenenfalls komplexe) Nullstelle besitzt. Für Polynome von Grad $n \geq 5$ existiert jedoch keine analytische Lösungsformel. Wir wissen, dass es eine Lösung gibt, kennen jedoch keine Methode, um diese zu berechnen. Hier kommt die numerische Mathematik ins Spiel. Auch wir werden nicht in der Lage sein, dieses Problem *exakt* zu lösen, können aber Methoden entwickeln, um eine Lösung zu *approximieren*.

Approximationen sind natürlich auch zentral in anderen Bereichen der Mathematik. Der *Approximationssatz von Weierstraß* besagt, dass jede stetige Funktion auf einem Intervall $I = [a, b]$ beliebig gut durch ein Polynom approximiert werden kann. Dieser Begriff der Approximation ist eng mit dem Begriff der *Konvergenz* verbunden: Es gibt eine Folge von Polynomen $p_n \in P_n$, welche gegen eine gegebene Funktion $f \in C[a, b]$ konvergiert:

$$\max_{x \in [a, b]} \|f(x) - p_n(x)\| \rightarrow 0 \quad (n \rightarrow \infty)$$

In der numerischen Mathematik werden wir den Lösungsbegriff weitestgehend durch den Begriff einer hinreichend genauen Approximation ersetzen. Wir betrachten ein Problem – zum Beispiel die Approximation einer stetigen Funktion durch ein Polynom – als gelöst, wenn es uns gelingt, ein Polynom $p_n \in P_n$ zu bestimmen, sodass der Fehler zwischen $f(x)$ und $p_n(x)$ auf $I = [a, b]$ hinreichend klein ist. Was bedeutet hinreichend? Hierauf können wir keine Antwort geben. Der Schritt vom Weierstraßschen Approximationssatz zur

konkreten Angabe eines approximierenden Polynoms ist jedoch noch sehr weit. Muss die Lösung konkret angegeben werden, so genügt es bei Weitem nicht zu wissen, dass eine solche Lösung existiert. Vielmehr müssen wir Wege finden, diese Lösung zu konstruieren.

Können mathematische Probleme nicht analytisch gelöst werden oder kann die analytische Lösung nicht in endlicher Zeit berechnet werden, so muss die Lösung **approximiert** werden. In vielen Fällen werden wir eine **numerische Approximation** als Lösung akzeptieren, wenn sie hinreichend genau ist.

Ein einfaches Beispiel ist die Berechnung der Zahl π . Es existieren zahlreiche Formeln zu deren Approximation. Leibniz hat die folgende Reihe zur Approximation von π veröffentlicht (die Reihe war schon weit früher bekannt und wurde vom indischen Mathematiker Madhava verwendet):

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} \pm \dots \quad (1.1)$$

Sie beruht auf einer Reihenentwicklung von $y = \arctan(x)$ sowie der Beziehung $\arctan(1) = \pi/4$. Wollen wir diese Reihe benutzen, um π zu approximieren, so müssen wir die Berechnung an irgendeinem Punkt abbrechen. Wir definieren die endliche Summe Π_n als Approximation

$$\pi \approx \Pi_n := 4 \left(\sum_{k=1}^n \frac{(-1)^{k+1}}{2k-1} \right). \quad (1.2)$$

1.1.2 Konvergenz

Die Analysis liefert uns Methoden zur Untersuchung der *Konvergenz* der Folge (1.2). Es gilt

$$\Pi_n \rightarrow \pi \quad (n \rightarrow \infty),$$

da es sich um eine alternierende Reihe handelt, deren Glieder eine Nullfolge bilden. Konvergenz ist einer der zentralen Begriffe der Analysis, der auch in der numerischen Mathematik eine wichtige Rolle einnimmt.

Konvergenz ist ein qualitativer Begriff. Die Konvergenz einer Folge $(a_n)_{n \in \mathbb{N}}$ gegen einen Grenzwert $a_n \rightarrow a$ für $n \rightarrow \infty$ besagt, dass für jeden positiven Wert $\epsilon > 0$ einen Index $n_0 \in \mathbb{N}$ gibt, so dass alle weiteren Folgenglieder a_n für $n \geq n_0$ näher als ϵ am Grenzwert a liegen, es gilt also $|a - a_n| < \epsilon$ für alle $n \geq n_0$. Der Grenzwert a ist im Rahmen der numerischen Mathematik oft die gesuchte Lösung des Problems. Die Folge $(a_n)_{n \in \mathbb{N}}$ ist das Ergebnis eines numerischen Approximationsverfahren.

Die Leibniz-Reihe konvergiert gegen die gesuchte Lösung, die Zahl π . So wichtig diese Aussage ist, so wenig hilft sie uns in der praktischen numerischen Mathematik.

1.1.3 Fehler

Wie können wir die Zahl π mit einem Fehler von 1 % bestimmen? An welcher Stelle dürfen wir die Berechnung der Reihe abbrechen? Es gilt

$$\Pi_1 = 4, \Pi_2 \approx 2.67, \Pi_3 \approx 3.47, \Pi_4 \approx 2.90, \dots, \Pi_{31} \approx 3.17, \Pi_{32} \approx 3.11, \dots$$

und schließlich gilt für den relativen Fehler die Abschätzung

$$\frac{|\Pi_{32} - \pi|}{\pi} \approx 0.99 \%.$$

Dieses Ergebnis war so nicht vorherzusehen und die *Fehlerabschätzung* gelingt uns hier nur, weil wir die gesuchte Zahl π bereits kennen.

Die numerische Mathematik kann als die Mathematik der **Fehler** betrachtet werden. Numerische Mathematik ist nicht falsch, inexakt oder unpräzise. In der Analyse numerischer Verfahren müssen wir jedoch **Fehler** berücksichtigen. Lösen wir Probleme aus der Anwendung, so treten Messfehler auf. Lösen wir Probleme mit dem Computer, so treten Rechenfehler z. B. durch eine endliche Darstellung von Zahlen auf. Approximieren wir Lösungen, so tritt ein Fehler bei endlichem Abbruch eines eigentlich unendlichen Prozesses auf.

1.1.4 Fehlerabschätzung

Im Allgemeinen versuchen wir in der numerischen Mathematik aber natürlich Probleme zu lösen, deren Lösung wir vorher noch nicht kennen. Der Begriff der *Fehlerabschätzung* erklärt ein weiteres wichtiges Konzept der numerischen Mathematik. Wenn wir eine numerische Approximation Π_n zu einem Problem mit (unbekannter) Lösung π erstellt haben, müssen wir sicherstellen, dass der Fehler zwischen Näherung und exakter Lösung klein ist. So eine *Fehlerabschätzung* sollte berechenbar sein. Wir suchen eine berechenbare Schranke für den Fehler, d. h. eine Formel, die garantiert, dass

$$|\Pi_n - \pi| \leq E_n.$$

Unter einer **Fehlerabschätzung** verstehen wir eine (berechenbare) Formel, die eine Abschätzung für den Fehler angibt, welcher zum Beispiel durch den Abbruch eines unendlichen Prozesses nach einer endlichen Anzahl von Schritten entsteht. Eine **Fehlerabschätzung** kann aber auch eine berechenbare Schranke für den Fehler sein, der durch Rundung auf dem Computer entsteht oder durch fehlerhafte Daten gegeben ist. Ein Ziel von **Fehlerabschätzungen** ist es stets, die Qualität von numerischen Approximationen zu beurteilen. **Fehlerabschätzungen** und **Fehlerkontrolle** sind wesentliche Merkmale der numerischen Mathematik.

Am Beispiel der Leibniz-Reihe können wir zur Herleitung einer entsprechenden Fehlerabschätzung wieder eine Idee in der Analysis finden: Die Leibniz-Reihe ist eine sogenannte *alternierende Reihe*, deren Summanden eine *monoton fallende Nullfolge* bilden. Es gilt:

$$\Pi_n = 4 \sum_{k=1}^n (-1)^{n+1} a_k, \quad a_k := \frac{1}{2k-1}$$

Für solche Reihen gilt stets, dass der Grenzwert zwischen zwei aufeinanderfolgenden Partialsummen liegt. Angewendet auf dieses Beispiel bedeutet dies

$$\Pi_{2n} \leq \pi \leq \Pi_{2n+1}.$$

Aus dieser Abschätzung können wir eine Fehlerabschätzung herleiten. Es gilt

$$0 \leq \pi - \Pi_{2n} \leq \Pi_{2n+1} - \Pi_{2n} = \frac{4}{2n+1} \quad (1.3)$$

und um einen Fehler von 1 % sicherzustellen, muss für $n \in \mathbb{N}$ gelten:

$$\frac{4}{2n+1} < \frac{1}{100} \quad \Leftrightarrow \quad n > 100$$

Für $\Pi_{2n} = \Pi_{200}$ können wir also garantieren, dass wir einen Fehler von 0.01 nicht übersteigen (ohne dass wir π kennen). Es gilt

$$\Pi_{200} - \pi \approx 0.0050 < 0.01$$

Diese Fehlerabschätzung ist nicht scharf. Das bedeutet, dass die gewünschte Genauigkeit schon viel früher erreicht wird, schon für $n = 50$ gilt

$$\Pi_{101} - \pi \approx 0.0099 < 0.01$$

Aber die Abschätzung ist *robust*, d. h., sie garantiert uns eine entsprechende Approximationsgüte. Auch dann, wenn wir das gesuchte Ergebnis noch gar nicht kennen.

1.1.5 Konvergenzgeschwindigkeit

Eine abgewandelte Reihe zur Approximation von π ist

$$\pi \approx \Pi_n := \sqrt{12} \sum_{k=1}^n \frac{(-3)^{-k+1}}{2k-1}. \quad (1.4)$$

Für die entsprechenden Partialsummen Π_n gilt:

$$\Pi_1 \approx 3.46, \quad \Pi_2 \approx 3.08, \quad \Pi_3 \approx 3.16, \quad \Pi_4 \approx 3.14, \dots$$

Nach nur vier Schritten ist bereits ein Fehler von weniger als 1 % erreicht. Diese Approximation scheint viel leistungsfähiger und schneller zu sein als die vorherige in (1.2) gegebene und zur numerischen Approximation besser geeignet.

Zur Beurteilung und zum Vergleich verschiedener Methoden wird die **Konvergenzgeschwindigkeit** eingeführt: Konvergiert eine Folge $(a_n)_{n \in \mathbb{N}}$ schneller oder langsamer als eine zweite Folge $(b_n)_{n \in \mathbb{N}}$ gegen denselben Grenzwert? Oft verwenden wir den Begriff der **Konvergenzordnung** zur Einordnung und Klassifikation verschiedener Methoden: Halbiert sich der Fehler in jedem Schritt der Approximation? Konvergiert eine Methode linear oder quadratisch?

1.1.6 Effizienz

In der numerischen Mathematik werde nicht nur Lösungswege zur Approximation von Problemen entworfen, diese müssen auch *effizient* sein. Dies ist eine Forderung, die sich in vielen Bereichen der Mathematik nicht stellt. Aber wir stellen uns nun die Aufgabe, die Zahl π auf 10 Stellen Genauigkeit möglichst effizient zu approximieren. Mithilfe der Fehlerabschätzung (1.3) können wir für die erste Leibniz-Reihe einen Zusammenhang zwischen dem gewünschten relativen Fehler $\epsilon = 10^{-10}$ und der Anzahl an Schritten $2n_\epsilon$ herleiten:

$$\frac{|\pi - \Pi_{2n_\epsilon}|}{\pi} < \frac{1}{(2n_\epsilon - 1)\pi} \stackrel{!}{<} \epsilon \Leftrightarrow n_\epsilon > \frac{1 + \pi\epsilon}{2\epsilon\pi}$$

Grob gesprochen: Um eine Stelle zu gewinnen, müssen wir 10-mal mehr Schritte durchführen. Um die ersten 10 Stellen von π zu bestimmen, müssen wir mit der Leibniz-Reihe etwa $10^{10} = 10\,000\,000\,000$ Schritte durchführen. Auch moderne Computer sind damit eine Weile beschäftigt. Dieser Zugang mag also *robust* sein, er ist aber sicher nicht *effizient*. Auch die modifizierte Leibniz-Reihe ist die Reihe einer alternierenden Nullfolge und wir

leiten eine entsprechende Fehlerabschätzung her:

$$0 \leq \pi - \Pi_{2n} \leq \Pi_{2n+1} - \Pi_{2n} = \sqrt{12} \frac{(-3)^{-2n}}{4n+1}$$

Jetzt gilt in starker Vereinfachung

$$\frac{|\pi - \Pi_{2n_\epsilon}|}{\pi} < \sqrt{12} \frac{(-3)^{-2n}}{(4n+1)\pi} \stackrel{!}{<} \epsilon \Leftrightarrow n_\epsilon > \frac{\log\left(\frac{4\pi\epsilon}{\sqrt{12}}\right)}{2\log\left(\frac{1}{3}\right)}. \quad (1.5)$$

Für $\epsilon = 10^{-10}$ erhalten wir

$$n_\epsilon > 10.$$

Die modifizierte Folge scheint daher bei Weitem schneller zu konvergieren.

Wir nennen ein numerisches Verfahren **effizient**, wenn es ein mathematisches Problem „ressourcenschonend“ löst oder approximiert. Zu den wichtigen „Ressourcen“ zählt dabei neben der *Rechenzeit* auch der *Speicheraufwand*. Der Begriff der **Effizienz** ist meist ein relativer Begriff. Ein Verfahren ist **effizienter** oder **weniger effizient** als ein alternatives Verfahren. Bei den meisten mathematischen Problemen ist nicht bekannt, welches Verfahren das **effizienteste** ist.

Von Srinivasa Ramanujan existiert zur Approximation von π die Reihe

$$\Pi_n := \frac{9801}{2\sqrt{2}} \left(\sum_{k=0}^n \frac{(4k)!(1103 + 26390k)}{(k!)^4 396^{4k}} \right)^{-1} \rightarrow \pi \quad (n \rightarrow \infty).$$

Für die ersten drei Approximationen gilt:

$$\Pi_0 \approx \underline{\mathbf{3.141592}}730013305660313996$$

$$\Pi_1 \approx \underline{\mathbf{3.141592653589793}}877998905$$

$$\Pi_2 \approx \underline{\mathbf{3.141592653589793238462649}}$$

Die richtigen Stellen sind unterstrichen. In jeder Iteration kommen etwa acht richtige Stellen hinzu. Statt 10 000 000 000 Schritten sind hier nur drei Schritte notwendig. Jeder Schritt in diesem Verfahren ist natürlich viel aufwendiger. Statt einer Division und einer Addition bei der Leibniz-Reihe sind zwei Fakultäten, eine Potenz und einige Multiplikationen durchzuführen. Aber selbst wenn wir diesen größeren Aufwand berücksichtigen, so ist dieses Verfahren bei Weitem *effizienter*.

1.1.7 Stabilität

Um das Konzept der *Stabilität* zu erklären wenden wir uns einem anderen wichtigen Problem der numerischen Mathematik zu, nämlich der Berechnung von Eigenwerten einer Matrix $A \in \mathbb{R}^{n \times n}$. Diese sind als Nullstellen des charakteristischen Polynoms gegeben:

$$\det(\lambda I - A) = 0$$

Beispielsweise habe die Matrix $A \in \mathbb{R}^{5 \times 5}$ die Eigenwerte 1, 2, 3, 4, 5. Das charakteristische Polynom hat demnach die Form

$$p(\lambda) = \det(\lambda I - A) = \prod_{i=1}^5 (\lambda - i) = \lambda^5 - 15\lambda^4 + 85\lambda^3 - 225\lambda^2 + 274\lambda - 120.$$

Wir müssen davon ausgehen, dass es einem Computer nicht gelingen wird, das Polynom exakt zu bestimmen. Die Koeffizienten werden mit einem (kleinen) Fehler versehen sein. Wir betrachten eine kleine Störung ϵ und das Polynom

$$p_\epsilon(\lambda) = \lambda^5 - 15(1 + \epsilon)\lambda^4 + 85(1 - \epsilon)\lambda^3 - 225(1 + \epsilon)\lambda^2 + 274(1 - \epsilon)\lambda - 120(1 + \epsilon).$$

Zu verschiedenen Störungen bestimmen wir in Tab. 1.1 die Nullstellen dieses Polynoms (wir geben nur die ersten drei Stellen an) und bestimmen jeweils auch den maximalen relativen Fehler.

Bei $\epsilon = 10^{-4}$, also einem relativen Fehler der Koeffizienten von nur 0.01 %, beträgt der Fehler in den Eigenwerten bereits fast 5 %. Der Fehler wird um den Faktor 500 verstärkt. Bei einem Fehler von nur 0.03 % treten bereits komplexe Eigenwerte auf. Eine wichtige Struktureigenschaft von z. B. symmetrischen Matrizen bleibt nicht mehr erhalten. Die Berechnung von Eigenwerten mit dem Umweg über das charakteristische Polynom ist kein *numerisch stabiler* Algorithmus.

Die *Stabilität* ist vermutlich der wichtigste und auch schwierigste Begriff der numerischen Mathematik. Es gibt hierfür keine einheitliche Definition. Allerdings gibt Hadamard 1921

Tab. 1.1 Die Nullstellen eines gestörten Polynoms zur Berechnung der Eigenwerte einer Matrix. Schon bei einem relativen Fehler von 0.01 % beträgt der Fehler in den Eigenwerten fast 5 %. Bei einer Störung des Polynomkoeffizienten von nur 0.08 % hat das Polynom bereits komplexe Nullstellen

ϵ	λ_1	λ_2	λ_3	λ_4	λ_5	rel. Fehler
0	1	2	3	4	5	0
$1 \cdot 10^{-4}$	1.00	1.97	3.13	3.82	5.08	4.5 %
$2 \cdot 10^{-4}$	1.00	1.95	3.38	3.52	5.14	12 %
$3 \cdot 10^{-4}$	1.00	1.93	$3.43 + 0.3i$	$3.43 - 0.3i$	5.21	18 %

eine gängige Erklärung zur Wohlgestelltheit einer mathematischen Aufgabe und fordert: Existenz, Eindeutigkeit, stetige Abhängigkeit der Lösung von den Problem Daten. Letzterer Punkt ist eine Form der Stabilität: wie stark verändert sich die Lösung wenn Eingangsdaten verändert werden? Mathematische Problemstellungen, die eines der drei Hadamard-Kriterien verletzen werden *schlecht gestellt* genannt. Aber wir müssen dies gleich wieder einschränken: Die Nullstellen eines Polynoms hängen stetig von den Koeffizienten ab, erfüllen also das Stabilitätskriterium nach Hadamard. Dieser scheinbare Widerspruch ist wieder ein Beispiel für das Zusammenspiel aus qualitativen und quantitativen Aussagen, die in der Numerik so wichtig sind. Die Stabilität nach Hadamard ist ein qualitativer Begriff. In der Numerik sind wir aber vor allem an quantitativen Zusammenhängen interessiert. Es genügt uns nicht zu wissen, dass eine stetige Abhängigkeit existiert, wir wollen wissen, wie stark sich die Störung auf den Fehler auswirkt. Das Konzept der *Stabilität* kommt immer dann zur Anwendung, wenn in der numerischen Mathematik die allgemeinen Denkmuster nicht mehr gelten:

- Wir weisen in der Analysis Konvergenz von Folgen gegen ihren Grenzwert $a_n \rightarrow a$ nach. Aber wie gut ist die Approximation für *kleine* n ? Wir werden numerische Verfahren *stabil* nennen, wenn sie auch für kleine n , also weit weg von der Konvergenz *sinnvolle Ergebnisse* liefern. Was *sinnvoll* ist, muss von Fall zu Fall unterschieden werden.
- Einfache mathematische Gesetze wie das Distributivgesetz gelten auf dem Computer nicht mehr. Unvermeidbare Rundungsfehler führen – je nach Rechenweg – zu unterschiedlichen Ergebnissen. Wir werden numerische Verfahren *stabil* nennen, wenn sie trotz fehlerhafter Daten und fehlerhafter Computerrealisierung robust sind.

Der Begriff der **Stabilität** bezeichnet das Verhalten von numerischen Verfahren bei ihrer praktischen Umsetzung. Ist die Methode **robust** gegenüber Fehlern? Wie verhalten sich konvergente Prozesse, wenn sie nach endlicher Anzahl von Iterationen abgebrochen werden?

Die hier als *Konzepte* eingeführten Begriffe haben alle gemein, dass diese sich, ganz anders als in den meisten Bereichen der Mathematik üblich, nicht klar definieren lassen. Wir sprechen von großen und kleinen Fehlern, können aber keine klaren Grenzen angeben, wann ein Fehler groß oder klein ist. Eine gewisse Ausnahme bildet die Konvergenz, die aus der Analysis wohlbekannt ist und dort bereits eines der wesentlichen Konzepte darstellt. Deren Definition und Idee erfüllt den identischen Zweck in der numerischen Mathematik. Die Wagheit vieler Aussagen in der numerischen Mathematik macht letztere aber nicht einfacher als andere Disziplinen, nur weil gegebenenfalls ungenauere Ergebnisse akzeptabel sind. Im Gegenteil: Da wir wissen, dass Fehler ständig auftreten, gilt es, diese in Definitionen, Sätzen und Beweisen klar zu beschreiben. Darüber hinaus ist es in der Praxis (bei

Computersimulationen) sogar essenziell, die auftretenden Fehler zu kontrollieren, um einen unerwünschten Abbruch des Algorithmus oder falsche Ergebnisse zu vermeiden oder die Effizienz zu verbessern. Zahlreiche Beispiele in diesem Buch belegen die Notwendigkeit zur sorgfältigen Untersuchung der *Fehler* in der numerischen Mathematik.

1.2 Definition eines Algorithmus

Das Ergebnis einer numerischen Aufgabe ist im Allgemeinen eine Zahl oder eine endliche Menge von Zahlen. Beispiele für numerische Aufgaben sind das Berechnen der Nullstellen einer Funktion, die Berechnung von Integralen, die Berechnung der Ableitung einer Funktion in einem Punkt, aber auch komplexere Aufgaben wie das Lösen einer Differentialgleichung.

Zum Lösen von numerischen Aufgaben werden wir unterschiedliche Verfahren kennenlernen. Wir grenzen zunächst ein:

► **Definition 1.1 (Numerisches Verfahren, Algorithmus)** Ein numerisches Verfahren ist eine Vorschrift zum schematischen Lösen oder zur Approximation einer mathematischen Aufgabe. Ein numerisches Verfahren heißt *direkt*, falls die Lösung bis auf Rundungsfehler exakt berechnet werden kann. Ein Verfahren heißt *approximativ*, falls die Lösung nur angenähert werden kann. Ein Verfahren heißt *iterativ*, falls die Näherung durch mehrfache Ausführung einer Vorschrift schrittweise verbessert wird.

Beispiele für direkte Lösungsverfahren sind die p/q -Formel zur Berechnung von Nullstellen quadratischer Polynome (siehe Kap. 8) oder der Gaußsche Eliminationsalgorithmus (Kap. 3) zum Lösen von linearen Gleichungssystemen. Approximative Verfahren müssen z. B. zum Bestimmen von komplizierten Integralen oder auch zum Berechnen von Nullstellen allgemeiner Funktionen eingesetzt werden. Oft ist ein exaktes Lösen von Aufgaben dem Problem nicht angemessen. Wenn Eingabedaten zum Beispiel mit großen Messfehlern behaftet sind, so ist ein exaktes Lösen nicht notwendig.

Polynomauswertung und Hornerschema

Wir betrachten ein Beispiel und dessen Analyse eines direkten Verfahrens im Folgenden im Detail:

Beispiel 1.2 (Polynomauswertung)

Es sei durch

$$p(x) = a_0 + a_1x + \cdots + a_nx^n$$

ein Polynom gegeben. Dabei sei $n \in \mathbb{N}$ sehr groß. Wir werten $p(x)$ in einem Punkt $\xi \in \mathbb{R}$ mit dem trivialen Verfahren aus:

Algorithmus 1.1: Polynomauswertung

Eingabe: Gegeben sei ein Polynom $p(x) = a_0 + a_1x + \dots + a_nx^n$ mit Auswertungsstelle $\xi \in \mathbb{R}$.

¹ **Für** $i = 1$ **bis** n **tue**

² $y_i = a_i \xi^i$

³ $p = p + y_i$

Ergebnis: Das ausgewertete Polynom $p = p(\xi)$.

Wir berechnen den *Aufwand* zur Polynomauswertung: In Schritt 3 des Algorithmus sind zur Berechnung der y_i i Multiplikationen notwendig, insgesamt

$$0 + 1 + 2 + \dots + n = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2}.$$

In Schritt 4 sind weitere n Additionen notwendig. Der Gesamtaufwand des Algorithmus beträgt demnach $n^2/2 + n/2$ Multiplikationen sowie n Additionen. Wir fassen eine Addition und eine Multiplikation zu einer *elementaren Operation* zusammen und erhalten als Aufwand der trivialen Polynomauswertung

$$A_1(n) = \frac{n^2}{2} + \frac{n}{2}$$

elementare Operationen. Wir schreiben das Polynom nun unter Verwendung des Distributivgesetzes um

$$p(x) = a_0 + x(a_1 + x(a_2 + \dots + x(a_{n-1} + xa_n) \dots)) \quad (1.6)$$

und leiten hieraus einen zweiten Algorithmus her:

Algorithmus 1.2: Horner-Schema

Eingabe: Gegeben seien das Polynom $p(x) = a_0 + a_1x + \dots + a_nx^n$ und die Auswertungsstelle $\xi \in \mathbb{R}$.

¹ $p = a_n$

² **Für** $i = n - 1$ **bis** 0 **tue**

³ $p = a_i + \xi \cdot p$

Ergebnis: Das ausgewertete Polynom $p = p(\xi)$.

Jeder der n Schritte des Verfahrens benötigt eine Multiplikation sowie eine Addition, also ergibt sich ein Aufwand von

$$A_2(n) = n$$

elementaren Operationen. Das *Horner-Schema* benötigt für die gleiche Aufgabe wesentlich weniger Operationen, man denke an ein Polynom von Grad $n = 1000$ und vergleiche $A_1(1000) \approx 500\,000$ und $A_2(1000) = 1000$. ◀

Beispiel 1.3 (Horner-Schema)

Wir berechnen den Wert des Polynoms $p(x) = 3x^5 - 2x^4 + 1$ an der Stelle $\xi = 2$. Die Koeffizienten lauten $a_5 = 3, a_4 = -2, a_3 = a_2 = a_1 = 0, a_0 = 1$. Dann erhalten wir das Schema:

$$\begin{array}{r|rrrrrr} i & 5 & 4 & 3 & 2 & 1 & 0 \\ a_i & 3 & -2 & 0 & 0 & 0 & 1 \\ \hline p & 3 & 4 & 8 & 16 & 32 & 65 \end{array}$$

Somit ist $p(2) = 65$. In Python können wir das Horner-Schema dann folgendermaßen implementieren:

Implementierung 1.1: Horner-Schema

```
1 def horner_schema(koeffizienten, x):
2     n = len(koeffizienten)
3     p = koeffizienten[-1]
4     for i in range(n - 1, 0, -1):
5         p = koeffizienten[i - 1] + x * p
6     return p
```

Dieser Code steht in den jupyter-Notebooks¹ zur Verfügung. Wir können den Code dann auf die oben genannten Koeffizienten anwenden

```
1 horner_schema([1, 0, 0, 0, -2, 3], 2)
```

und erhalten damit die Ausgabe

```
1 65
```

Das Horner-Schema kann auf natürliche Weise verallgemeinert werden, um auch die Ableitungen des Polynoms zu berechnen. Somit erhalten wir zur Berechnung der Ableitungen $p^{(k)}(\xi)$ eines Polynoms den folgenden Algorithmus:

¹ github.com/sn-code-inside/EinfNumMath-RWW

Algorithmus 1.3: Vollständiges Horner-Schema

Eingabe: Gegeben sei eine Auswertungsstelle $\xi \in \mathbb{R}$ und das Polynom

$$p(x) = a_0^{(0)} + a_1^{(0)}x + \cdots + a_n^{(0)}x^n.$$

$$1 \quad a_n^{(1)} = a_n^{(0)}, a_n^{(2)} = a_n^{(0)}, \dots, a_n^{(n+1)} = a_n^{(0)}$$

2 **Für** $k = 0$ **bis** $n - 1$ **tue**

3 **Für** $j = n - 1$ **bis** k **tue**

$$4 \quad a_j^{(k+1)} = a_j^{(k)} + a_{j+1}^{(k+1)}\xi$$

$$5 \quad p^{(k)}(\xi) = k! \cdot a_k^{(k+1)}$$

Ergebnis: Die Auswertungen des Polynoms und dessen Ableitungen $p^{(k)}(\xi)$, für $k = 0, \dots, n - 1$

Dieser Algorithmus kann anhand des folgenden Schemas veranschaulicht werden:

i	n	$n - 1$	$n - 2$	$\dots$	2	1	0
	$a_n^{(0)}$	$a_{n-1}^{(0)}$	$a_{n-2}^{(0)}$	$\dots$	$a_2^{(0)}$	$a_1^{(0)}$	$a_0^{(0)}$
$p_i(\xi)$	$a_n^{(1)}$	$a_{n-1}^{(1)}$	$a_{n-2}^{(1)}$	$\dots$	$a_2^{(1)}$	$a_1^{(1)}$	$a_0^{(1)}$
$p_i'(\xi)$	$a_n^{(2)}$	$a_{n-1}^{(2)}$	$a_{n-2}^{(2)}$	$\dots$	$a_2^{(2)}$	$a_1^{(2)}$	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$			
$p_i^{(n-2)}(\xi)$	$a_n^{(n-1)}$	$a_{n-1}^{(n-1)}$	$a_{n-2}^{(n-1)}$				
$p_i^{(n-1)}(\xi)$	$a_n^{(n)}$	$a_{n-1}^{(n)}$					
$p_i^{(n)}(\xi)$	$a_n^{(n+1)}$						

Wir führen das obige Beispiel fort:

Beispiel 1.4 (Vollständiges Horner-Schema)

Wir berechnen die Werte der Ableitungen des Polynoms $p(x) = 3x^5 - 2x^4 + 1$ an der Stelle $\xi = 2$. Dazu erhalten wir das folgende Schema:

i	5	4	3	2	1	0	
a_i	3	-2	0	0	0	1	
p_i	3	4	8	16	32	65	$\Rightarrow p(\xi) = 0! \cdot 65 = 65$
p_i'	3	10	28	72	176		$\Rightarrow p'(\xi) = 1! \cdot 176 = 176$
p_i''	3	16	60	192			$\Rightarrow p''(\xi) = 2! \cdot 192 = 386$
$p_i^{(3)}$	3	22	104				$\Rightarrow p^{(3)}(\xi) = 3! \cdot 104 = 624$
$p_i^{(4)}$	3	28					$\Rightarrow p^{(4)}(\xi) = 4! \cdot 28 = 672$
$p_i^{(5)}$	3						$\Rightarrow p^{(5)}(\xi) = 5! \cdot 3 = 360$

In Python können wir das vollständige Horner-Schema folgendermaßen implementieren:

♣ Implementierung 1.2: Vollständiges Horner-Schema

```

1 import numpy as np
2
3 def vollstaendiges_horner_schema(koeffizienten, x):
4     n = len(koeffizienten)
5     p = np.zeros(n)
6     a = np.zeros((n + 1, n))
7     a[:, -1] = koeffizienten[-1]
8     a[0, :] = koeffizienten
9
10    for k in range(n):
11        for j in range(n - 1, k, -1):
12            a[k + 1, j - 1] = a[k, j - 1] + a[k + 1, j] * x
13            p[k] = np.math.factorial(k) * a[k + 1, k]
14    return p

```

Die Ausgabe ist jetzt ein array der Bibliothek `numpy`, in dem die Werte der i -ten Ableitung im i -ten Eintrag des Arrays gespeichert sind. Dabei ist wichtig zu bemerken, dass in Python Indizes immer bei 0 anfangen.

Wenn wir diesen Code jetzt auf die bereits oben genutzten Koeffizienten anwenden

```
1 vollstaendiges_horner_schema([1, 0, 0, 0, -2, 3], 2)
```

bekommen wir die Ausgabe

```
1 array([ 65., 176., 384., 624., 672., 360.])
```

also ein array mit dem Funktionswert und den Werten aller Ableitungen. ◀

1.2.1 Numerischer Aufwand und Landau-Symbole

Im Folgenden führen wir eine allgemeine Notation zur Beschreibung des Aufwands eines Verfahrens ein.

► **Definition 1.5 (Numerischer Aufwand in elementaren Operationen)** Der *Aufwand* eines numerischen Verfahrens ist die Anzahl der notwendigen elementaren Operationen. Eine *elementare Operation* ist eine Addition sowie eine Multiplikation.

Meist hängt der Aufwand eines Verfahrens von der Problemgröße ab. Die Problemgröße $N \in \mathbb{N}$ wird von Problem zu Problem definiert, beim Lösen eines linearen Gleichungssystems $Ax = b$ mit einer Matrix $A \in \mathbb{R}^{N \times N}$ ist die Größe der Matrix die Problemgröße. Beim

Auswerten eines Polynoms $p(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0$ in einem Punkt x ist die Problemgröße der Grad n des Polynoms. Zur einfachen Schreibweise definieren wir:

► **Definition 1.6 (Landau-Symbole)** (i) Es sei $g(n)$ eine Funktion mit $g \rightarrow \infty$ für $n \rightarrow \infty$. Dann ist $f \in O(g)$ genau dann, wenn

$$\limsup_{n \rightarrow \infty} \left| \frac{f(n)}{g(n)} \right| < \infty,$$

sowie $f \in o(g)$ genau dann, wenn

$$\lim_{n \rightarrow \infty} \left| \frac{f(n)}{g(n)} \right| = 0.$$

(ii) Sei $g(h)$ eine Funktion mit $g(h) \rightarrow 0$ für $h \rightarrow 0$. Wir definieren wie oben $f \in O(g)$ sowie $f \in o(g)$.

Einfach gesprochen: $f \in O(g)$, falls f höchstens so schnell gegen ∞ konvergiert wie g , und $f \in o(g)$, falls g schneller als f gegen ∞ geht. Entsprechend gilt für $g \rightarrow 0$, dass $f \in O(g)$, falls f mindestens so schnell gegen null konvergiert wie g , und $f \in o(g)$, falls f schneller als g gegen null konvergiert. Mithilfe der Landau-Symbole lässt sich der Aufwand eines Verfahrens einfacher charakterisieren.

Beispiel 1.7 (Landau-Symbole)

(i) Im Fall der trivialen Polynomauswertung in Algorithmus 1.1 gilt für den Aufwand $A_1(n)$ in Abhängigkeit der Polynomgröße n

$$A_1(n) \in O(n^2)$$

und im Fall des Horner-Schemas von Algorithmus 1.2

$$A_2(n) \in O(n).$$

Wir sagen: Der Aufwand der trivialen Polynomauswertung wächst quadratisch, der Aufwand des Horner-Schemas linear. Weiter können mit den Landau-Symbolen Konvergenzbegriffe quantifiziert und verglichen werden. In den entsprechenden Kapiteln kommen wir auf diesen Punkt zurück.

(ii) Ein numerisches Verfahren liefert für den Parameter $h > 0$ die Approximation $\bar{a} \approx a(h)$ mit $a(h) \rightarrow \bar{a}$ für $h \rightarrow 0$. Wir sagen, dass das Verfahren *linear* konvergiert, falls

$$\|\bar{a} - a(h)\| = O(h)$$

und quadratisch konvergiert im Fall

$$\|\bar{a} - a(h)\| = O(h^2).$$

Allgemein sprechen wir von Konvergenz der Ordnung α , falls

$$\|\bar{a} - a(h)\| = O(h^\alpha).$$

Im Fall $\|\bar{a} - a(h)\| = o(h)$ nennen wir die Konvergenz *superlinear*. ◀

Bemerkung 1.8 (Landau-Symbole) Das Symbol $O(g)$ beschreibt eine Menge von Funktionen. Es ist

$$f \in O(g),$$

wenn $f(n)/g(n) \leq C$ für $n \geq n_0$ beschränkt bleibt. Oft werden wir jedoch bei der Verwendung der Landau-Symbole das Gleichheitszeichen nutzen, also

$$f = O(g)$$

schreiben. Dies bedeutet keine echte Gleichheit. Der Schluss

$$f_1 = O(g), \quad f_2 = O(g) \quad \Rightarrow \quad f_1 = f_2$$

ist im Allgemeinen falsch. $f = O(g)$ ist nur eine Schreibweise für $f \in O(g)$. In vielen Fällen ist diese jedoch praktisch. Wir können dann mit den Landau-Symbolen einfach rechnen und schreiben zum Beispiel

$$f(n) = \frac{1}{1 - \frac{1}{n}} = 1 + \frac{1}{n} + O\left(\frac{1}{n^2}\right).$$

Die Gleichheit gilt bis auf einen Fehler der maximalen Größenordnung $O(n^{-2})$. Das dies wirklich der Fall ist, ist einfach mit Taylor-Entwicklung von $1/(1-x)$ für $x = 1/n$ nachzurechnen. ♦

1.3 Fehler, Fehlerverstärkung und Konditionierung

Numerische Lösungen sind oft mit Fehlern behaftet. Fehlerquellen sind zahlreich: Die Eingabe kann mit einem Messfehler versehen sein, ein approximatives Verfahren wird die Lösung nicht exakt berechnen, sondern nur annähern, manche Aufgaben können nur mithilfe eines Computers oder Taschenrechners gelöst werden und so ist die Lösung mit *Rundungsfehlern* versehen. Weitere Fehlerquellen sind Diskretisierungsfehler, Interpolationsfehler, Iterationsfehler, Regularisierungsfehler, Implementierungsfehler (bugs), Modellfehler, Datenfehler und Ungewissheiten (uncertainties). In der Numerik müssen wir immer davon ausgehen, dass Eingaben, Zwischenergebnisse und Ergebnisse mit Fehlern behaftet sind. Eine „fehlerfreie Rechnung“ ist eigentlich nur mit viel Zufall möglich. Das macht die

Numerische Mathematik nicht zu einer „falschen Mathematik“. Im Gegenteil ist es Teil der Numerik, die Fehler zu identifizieren und deren Fortpflanzung und Verstärkung soweit wie möglich zu kontrollieren. Wir definieren:

► **Definition 1.9 (Fehler)** Sei $\tilde{x} \in \mathbb{R}$ die Approximation einer Größe $x \in \mathbb{R}$. Mit $|\delta x| = |\tilde{x} - x|$ bezeichnen wir den *absoluten Fehler* und mit $|\delta x|/|x|$ den *relativen Fehler*.

Üblicherweise ist die Betrachtung des relativen Fehlers von größerer Bedeutung: Denn ein absoluter Messfehler von 100 m ist klein, wenn man den Abstand zwischen Erde und Sonne zu bestimmen versucht, jedoch groß, wenn der Abstand zwischen Mensa und Mathematikgebäude gemessen werden soll.

Bei der Analyse von numerischen Verfahren spielen Fehler, insbesondere die Fortpflanzung von Fehlern, eine entscheidende Rolle. Wir betrachten ein Beispiel:

Beispiel 1.10 (Größenbestimmung)

Thomas will seine Größe h bestimmen, hat allerdings kein Maßband zur Verfügung. Dafür hat er eine Uhr, einen Ball und im Physikunterricht gut aufgepasst. Zur Lösung der Aufgabe hat er zwei Ideen:

Verfahren 1: Thomas lässt den Ball aus Kopfhöhe fallen und misst die Zeit t_0 , bis der Ball auf dem Boden ankommt. Die Höhe berechnet er aus der Formel für den freien Fall,

$$y(t) = h - \frac{1}{2}gt^2 \quad \Rightarrow \quad h = \frac{1}{2}gt_0^2,$$

mit der Gravitationskonstanten $g = 9.81 \text{ m/s}^2$.

Verfahren 2: Der Ball wird 2 m über den Kopf geworfen und wir messen die Zeit bis der Ball wieder auf dem Boden angekommen ist. Die Strecke $s = 2 \text{ m}$ wird in $t' = \sqrt{2s/g}$ Sekunden zurückgelegt, hierfür benötigt der Ball eine Startgeschwindigkeit von $v_0 = gt' = \sqrt{2sg} \approx 6.26 \text{ m/s}$. Es gilt für die Flugbahn:

$$y(t) = h + v_0t - \frac{1}{2}gt^2 \quad \Rightarrow \quad h = \frac{1}{2}gt_0^2 - v_0t_0$$

Das zweite Verfahren wird gewählt, weil die Zeit t_0 , die der Ball in Verfahren 1 zum Fallen benötigt, sehr klein ist. Große Messfehler werden vermutet. In Abb. 1.1 skizzieren wir das Vorgehen in beiden Varianten.

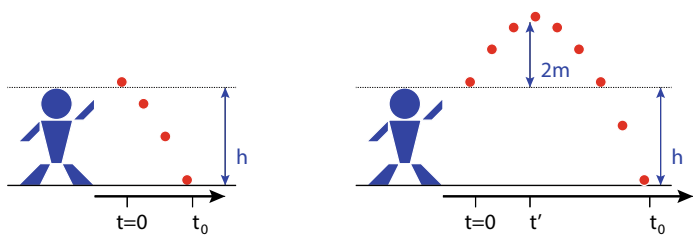


Abb. 1.1 Beispiel 1.10: Experimente zur Bestimmung der eigenen Größe. Links: Ein Ball wird aus Kopfhöhe fallengelassen und wir messen die Zeit t_0 bis er auf dem Boden landet. Rechts: Um die Messfehler zu minimieren wird der Ball zunächst 2m in die Höhe geworfen. Wir messen dann wieder die jetzt längere Zeit t_0 bis der Ball auf dem Boden aufkommt. Die Höhe h kann jeweils aus der Formel für den freien Fall, bzw. für die Wurfparabel hergeleitet werden

Wir führen zunächst Algorithmus 1 durch und messen 5-mal (exakte Lösung $h = 1.80\text{ m}$ und $t_0 \approx 0.606\text{ s}$).

Messung	0.58 s	0.61 s	0.62 s	0.54 s	0.64 s	0.598 s
Messfehler (rel.)	4 %	0.5 %	2 %	10 %	6 %	1 %
Größe	1.65 m	1.82 m	1.89 m	1.43 m	2.01 m	1.76 m
Fehler (abs.)	0.15 m	0.02 m	0.09 m	0.37 m	0.21 m	0.04 m
Fehler (rel.)	8 %	1 %	5 %	21 %	12 %	2 %

In der letzten Spalte wurde als Zeit der Mittelwert aller Messergebnisse gewählt. Wir führen nun Algorithmus 2 durch und messen 5-mal (exakte Lösung $h = 1.80\text{ m}$ und $t_0 \approx 1.519\text{ s}$). In der letzten Spalte wird wieder der Mittelwert aller Messergebnisse betrachtet:

Messung	1.60 s	1.48 s	1.35 s	1.53 s	1.45 s	1.482 s
Messfehler (rel.)	5 %	2.5 %	11 %	< 1 %	5 %	2 %
Größe	2.53 m	1.47 m	0.48 m	1.90 m	1.23 m	1.49 m
Fehler (abs.)	0.73 m	0.33 m	1.32 m	0.10 m	0.57 m	0.31 m
Fehler (rel.)	40 %	18 %	73 %	6 %	32 %	17 %

Trotz anfänglicher Zweifel erweist sich Algorithmus 1 als besser, da kleinere Fehler im Ergebnis erreicht werden. Mögliche Fehlerquellen sind der Messfehler bei der Bestimmung der Zeit t_0 sowie bei Algorithmus 2 die Genauigkeit beim Erreichen der Höhe von 2 m . In Algorithmus 1 führt der Messfehler zu etwa dem doppelten relativen Fehler in der Größe. Bei Algorithmus 2 führen selbst kleine Fehler $\leq 1\%$ in den Messungen zu wesentlich größeren Fehlern im Ergebnis. Der immer noch kleine Fehler von 5% bei der ersten Messung führt zu einem Ergebnisfehler von etwa 40% . Auch bei Betrachtung des

Mittelwerts über alle Messwerte ist die ermittelte Größe von 1.49 m keine gute Näherung. Die Ergebnisse der beiden Verfahren zeigen aber auch, dass der relative Messfehler jeweils um einen festen Faktor verstärkt in den relativen Fehler im Ergebnis eingeht. Bei Verfahren 1 wird der Fehler etwa verdoppelt, bei Verfahren 2 etwa versechsfacht. ◀

Wir werden diesen Zusammenhang nun analytisch untersuchen. Wir betrachten zunächst eine skalare Aufgabe, also eine Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$ mit $f(x) = y$ mit der Eingabe x und der Ausgabe y . Die gestörte Eingabe $\tilde{x} = x + \delta x$ liefert ein gestörtes Ergebnis $\tilde{y} = y + \delta y$, also $\tilde{y} = f(\tilde{x})$. Es gilt

$$\delta y = \tilde{y} - y = f(\tilde{x}) - f(x) = f(x + \delta x) - f(x) \quad (1.7)$$

und nach Erweitern mit $\delta x \cdot \delta x^{-1}$ und Taylor-Entwicklung (wir fordern hierfür, dass die Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$ differenzierbar ist) gilt

$$\delta y = \frac{f(x + \delta x) - f(x)}{\delta x} \cdot \delta x = f'(x)\delta x + o(|\delta x|^2). \quad (1.8)$$

Der Ausdruck $o(|\delta x|^2)$ ist eines der Landau-Symbole aus Definition 1.6 und bedeutet, dass der Fehler schneller als quadratisch in δx gegen Null geht. Die gestörte Eingabe δx wird demnach im Wesentlichen durch den Faktor $f'(\xi)$ verstärkt bzw. abgeschwächt, je nachdem ob $|f'(\xi)| > 1$ bzw. $|f'(\xi)| < 1$ vorliegt. Die relative Fehlerverstärkung erhalten wir mit Teilen durch $y = f(x)$ und Erweitern mit $x \cdot x^{-1}$

$$\frac{\delta y}{y} = \frac{f'(x)}{f(x)}\delta x + o(|\delta x|^2) = f'(x)\frac{x}{f(x)} \cdot \frac{\delta x}{x} + o(|\delta x|^2)$$

Wir nennen

$$\kappa := \left| f'(x) \frac{x}{f(x)} \right|$$

die *Konditionszahl* der Aufgabe und sie beschreibt die relative Fehlerverstärkung in Bezug auf eine Eingabegröße.

► **Definition 1.11 (Konditionszahl)** Für die stetig differenzierbare Funktion $f : x \mapsto y$ ist die *Konditionszahl* gegeben durch

$$\kappa(x) := \left| f'(x) \frac{x}{f(x)} \right|.$$

Eine Aufgabe heißt für eine Eingabe x *schlecht konditioniert*, falls $|\kappa(x)| \gg 1$, ansonsten *gut konditioniert*. Im Fall $|\kappa(x)| < 1$ spricht man von *Fehlerdämpfung*, ansonsten von *Fehlerverstärkung*.

Wir setzen das Beispiel zur experimentellen Größenbestimmung fort:

Beispiel 1.12 (Größenbestimmung, Fortsetzung von Beispiel 1.10)

Verfahren 1 Zum Durchführen des Verfahrens $h(t_0)$ ist als Eingabe die gemessene Zeit t_0 erforderlich. Für die Konditionszahl gilt:

$$\kappa_{h,t_0} := \frac{\partial h(t_0)}{\partial t_0} \frac{t_0}{h(t_0)} = gt_0 \frac{t_0}{\frac{1}{2}gt_0^2} = 2$$

Ein relativer Fehler in der Eingabe $\delta t_0/t_0$ kann demnach einen doppelt so großen relativen Fehler in der Ausgabe verursachen.

Verfahren 2 Wir bestimmen die Konditionszahl in Bezug auf die Zeit t_0 :

$$\kappa_{h,t_0} = (gt_0 - v_0) \frac{t_0}{\frac{1}{2}gt_0^2 - v_0t_0} = 2 \frac{gt_0 - v_0}{gt_0 - 2v_0}$$

Wir wissen, dass der Ball bei exaktem Wurf und ohne Messfehler $t_0 \approx 1.519$ s unterwegs ist bei einer Startgeschwindigkeit von $v_0 \approx 6.26$ m/s. Dies ergibt:

$$\kappa_{h,t_0} \approx \kappa_{h,v_0} \approx 8$$

Fehler in der Eingaben $\delta t_0/t_0$ sowie $\delta v_0/v_0$ werden um den Faktor 8 verstärkt. Die durch die Konditionszahlen vorhergesagten Fehlerverstärkungen lassen sich in den Tabellen zu Beispiel 1.10 gut wiederfinden. ◀

Die Definition der Konditionszahl kann auf höherdimensionale Aufgaben $A : x \mapsto y$ mit $x \in \mathbb{R}^n$ und $y \in \mathbb{R}^m$ erweitert werden. Die Eingabe x sei mit einem Fehler

$$\delta x = \delta_j \cdot e_j \in \mathbb{R}^n$$

behaftet, wobei $e_j = (0, \dots, 0, 1, 0, \dots, 0) \in \mathbb{R}^n$ der j -te Einheitsvektor ist und $\delta_j \geq 0$. Für die gestörte Lösung $\tilde{y} = A(\tilde{x})$ gilt dann in Komponente $i = 1, \dots, m$:

$$\delta y_i = \tilde{y}_i - y_i = A_i(\tilde{x}) - A_i(x) = A_i(x + \delta x_j) - A_i(x)$$

Wir oben erweitern wir mit $\delta x_j \cdot \delta x_j^{-1}$ und teilen durch $y_i = A_i(x)$

$$\frac{\delta y_i}{y_i} = \frac{A_i(x + \delta x_j) - A_i(x)}{\delta x_j} \frac{x_j}{A_i(x)} \frac{\delta x_j}{x_j}.$$

Mit Taylor-Entwicklung von $A_i(\cdot)$ in Richtung x_j folgt

$$\left| \frac{\delta y_i}{y_i} \right| = \left| \frac{\partial A_i(x)}{\partial x_j} \frac{x_j}{A_i(x)} \right| \cdot \left| \frac{\delta x_j}{x_j} \right| + o(\|\delta x\|^2).$$

Wir definieren:

► **Definition 1.13 (Konditionszahl vektorwertiger Abbildungen)** Es sei $A : \mathbb{R}^n \rightarrow \mathbb{R}^m$ stetig differenzierbar. Wir nennen

$$\kappa_{i,j} := \left| \frac{\partial A_i(x)}{\partial x_j} \frac{x_j}{A_i(x)} \right|, \quad i = 1, \dots, n, \quad j = 1, \dots, m$$

die *Konditionszahlen* der Funktion $A(\cdot)$.

1.3.1 Konditionierung der Grundoperationen

Rundungsfehler treten bei jeder elementaren Operation auf. Daher kommt der Konditionierung der Grundoperationen, aus denen alle Algorithmen aufgebaut sind, eine entscheidende Bedeutung zu:

Beispiel 1.14 (Konditionierung der Grundoperationen)

1. Addition, Subtraktion: $A(x, y) = x + y$:

$$\kappa_{A,x} = \left| \frac{x}{x+y} \right| = \left| \frac{1}{1 + \frac{y}{x}} \right|$$

Im Fall $x \approx -y$ kann die Konditionszahl der Addition ($x \approx y$ bei der Subtraktion) beliebig groß werden. Ein Beispiel mit vierstelliger Rechnung:

$$x = 1.021, \quad y = -1.019 \quad \Rightarrow \quad x + y = 0.002$$

Jetzt sei $\tilde{y} = 1.020$ gestört. Der relative Fehler in y ist sehr klein: $|1.019 - 1.020|/|1.019| \leq 0.1\%$. Wir erhalten das gestörte Ergebnis

$$x + \tilde{y} = 0.001$$

und einen Fehler von 100 %. Die enorme Fehlerverstärkung bei der Addition von Zahlen mit etwa gleichem Betrag wird *Auslöschung* genannt. Denn stimmen bei der Berechnung von $x - y$ die ersten wesentlichen Stellen von x und y überein, so werden diese im Ergebnis ausgelöscht: alle ersten Ziffern sind Null und die Zahl wird im Betrag kleiner. Es verbleiben nur noch sehr weniger von Null verschiedene Stellen zur Darstellung der Genauigkeit. Auslöschung tritt üblicherweise dann auf, wenn das Ergebnis einer numerischen Operation verglichen mit den Eingabewerten einen sehr kleinen Betrag hat. Kleine relative Fehler in der Eingabe verstärken sich zu großen relativen Fehlern im Ergebnis.

2. Multiplikation: $A(x, y) = x \cdot y$. Die Multiplikation zweier Zahlen ist stets gut konditioniert:

$$\kappa_{A,x} = \left| y \frac{x}{xy} \right| = 1$$

3. Division: $A(x, y) = x/y$. Dasselbe gilt für die Division:

$$\kappa_{A,x} = \left| \frac{1}{y} \frac{x}{\frac{x}{y}} \right| = 1, \quad \kappa_{A,y} = \left| \frac{x}{y^2 \frac{x}{y}} \right| = 1$$

4. Wurzelziehen: $A(x) = \sqrt{x}$:

$$\kappa_{A,x} = \left| \frac{1}{2\sqrt{x}} \frac{x}{\sqrt{x}} \right| = \frac{1}{2}$$

Ein Fehler in der Eingabe wird im Ergebnis sogar reduziert.



1.4 Zahldarstellung und deren Bedeutung in der Numerik

1.4.1 Zahldarstellungen

Die Analyse von Fehlern und Fehlerfortpflanzungen spielt eine zentrale Rolle in der numerischen Mathematik. Fehler treten vielfältig auf, auch ohne ungenaue Eingabewerte. Oft sind numerische Algorithmen sehr komplex, setzen sich aus vielen Operationen zusammen. Bei der Approximation mit dem Computer treten unweigerlich *Rundungsfehler* auf. Da der Speicherplatz im Computer bzw. die Anzahl der Ziffern auf dem Taschenrechner beschränkt ist, treten Rundungsfehler schon bei der bloßen Darstellung einer Zahl auf. Auch wenn es für die numerische Aufgabe

$$x^2 = 2, \quad \Leftrightarrow x = \pm\sqrt{2},$$

mit $x = \pm\sqrt{2}$ eine einfach anzugebende Lösung gibt, kann diese auf einem Computer nicht exakt dargestellt werden:

$$\sqrt{2} = 1.41421356237309504880 \dots$$

Der naheliegende Grund für einen zwingenden Darstellungsfehler ist der beschränkte Speicher eines Computers. Ein weiterer Grund liegt in der Effizienz. Ein Computer kann effizient nicht mit beliebig langen Zahlen rechnen. Grund ist die beschränkte Datenbandbreite (das sind die 8 Bit, 16 Bit, 32 Bit oder 64 Bit der Prozessoren). Operationen mit Zahlen in längerer Darstellung müssen zusammengesetzt werden, ähnlich dem schriftlichen Multiplizieren oder Addieren aus der Schule. Computer speichern Zahlen gerundet in Binärdarstellung, also zur Basis 2.

$$\text{rd}(x) = \pm \sum_{i=-n_1}^{n_2} a_i 2^i, \quad a_i \in \{0, 1\}. \quad (1.9)$$

Die Genauigkeit der Darstellung und der Rundungsfehler hängen von der Anzahl der Ziffern, also von n_1 und n_2 ab. Die *Fixkommadarstellung* der Zahl im Binärsystem lautet:

$$\text{rd}(x) = [a_{n_2} a_{n_2-1} \dots a_1 a_0 . a_{-1} a_{-2} \dots a_{-n_1}]_2 \quad (1.10)$$

Praktischer ist die *Gleitkommadarstellung* von Zahlen. Hierzu wird die Binärdarstellung normiert und ein gemeinsamer Exponent eingeführt:

$$\text{rd}(x) = \pm \left(\sum_{i=-n_1-n_2}^0 a_{i+n_2} 2^i \right) 2^{n_2}, \quad a_i \in \{0, 1\} \quad (1.11)$$

Der führende Term (die a_i) heißt die *Mantisse* und wird von uns mit M bezeichnet, den *Exponenten* bezeichnen wir mit E . Der Exponent kann dabei eine positive oder negative Zahl sein. Zur Vereinfachung wird der Exponent E als $E = e - b$ mit einer positiven Zahl $e \in \mathbb{N}$ und dem Bias $b \in \mathbb{N}$ geschrieben. Der Biaswert b wird in einer konkreten Zahldarstellung fest gewählt und bestimmt somit den größtmöglichen negativen Exponenten. Der variable Exponentenanteil e wird selbst im Binärformat gespeichert. Es bleibt, die Anzahl der Binärstellen für Mantisse und Exponent zu wählen. Hinzu kommt ein Bit für das Vorzeichen $S \in \{+, -\}$. Die Gleitkommadarstellung im Binärsystem lautet:

$$\text{rd}(x) = S \cdot [m_0 . m_{-1} m_{-2} \dots m_{-m}]_2 \cdot 2^{[e_{\#e} \dots e_1 e_0]_2 - b} \quad (1.12)$$

Die Mantisse wird üblicherweise mit $m_0 = 1$ normiert zu $M \in [1, 2)$. Das heißt, die führende Stelle muss nicht explizit gespeichert werden. Auf dem Computer bleibt die Bitfolge

$$\text{rd}(x) = [S e_{\#e} \dots e_1 e_0 m_{-1} m_{-2} \dots m_{-m}]_2 \quad (1.13)$$

und zur Interpretation müssen wir wissen, wie viele Bits zur Mantisse und zum Exponenten gehören und was der Bias ist.

Auf modernen Computern hat sich das *IEEE-754-Format* [55]² zum Speichern von Zahlen etabliert. Die Grundidee des Formates ist die binäre Gleitkommadarstellung gemäß (1.12), allerdings werden noch etliche Tricks genutzt, um die zur Verfügung stehenden Bits für Mantisse und Exponent möglichst optimal zu nutzen. Zum Verständnis der numerische Mathematik sind diese nicht wesentlich. Wichtig ist aber die Erkenntnis, dass nur eine beschränkte Genauigkeit zur Verfügung steht (gegeben durch die Anzahl der Ziffern in der Mantisse) sowie dass es eine größte und eine kleinste Zahl gibt, die dargestellt werden kann (bestimmt durch die Anzahl der Ziffern im Exponenten sowie durch den Bias b).

² Siehe auch die Wikipedia-Seite zum IEEE-754 Format https://de.wikipedia.org/wiki/IEEE_754.

Tab. 1.2 IEEE 754-Format in einfacher und doppelter Genauigkeit sowie Gleitkommaformate in aktueller und historischer Hardware. Moderne Beschleunigerhardware hat verschiedene Modi und kann (teilweise gleichzeitig) mit verschiedenen Zahlenmodellen rechnen

	Größe	Vorzeichen	Exponent	Mantisse	Bias
einfache Genauigkeit (single)	32 Bit	1 Bit	8 Bit	23+1 Bit	127
doppelte Genauigkeit (double)	64 Bit	1 Bit	11 Bit	52+1 Bit	1023
Zuse Z1 (1938)	24 Bit	1 Bit	7 Bit	15 Bit	–
IBM 704 (1954)	36 Bit	1 Bit	8 Bit	27 Bit	128
i8087 Coprozessor (1980)	Erste Verwendung von IEEE (single + double)				
Intel 486 (1989)	Erste integrierte FPU in Standard-PC				
Nvidia Tesla A100 (2020) FP 32	32 Bit	1 Bit	8 Bit	23 Bit	127
Nvidia Tesla A100 (2020) TF 32	19 Bit	1 Bit	8 Bit	10 Bit	127
Nvidia Tesla A100 (2020) FP 16	16 Bit	1 Bit	5 Bit	10 Bit	15

In Tab. 1.2 sind verschiedene Gleitkommadarstellungen zusammengefasst, die entweder historisch sind oder sich auf aktuelle Formate beziehen. Derzeit wird fast ausschließlich das IEEE-Format verwendet. Hier waren bisher zwei Darstellungen üblich, *single-precision* (in C++ float) und *double-precision* (in C++ double). Die Recheneinheiten moderner Computer nutzen intern eine erhöhte Genauigkeit zum Durchführen von elementaren Operationen (80 Bit bei modernen Intel-CPU's). Gerundet wird erst nach Berechnung des Ergebnisses.

Historisch wurden in Rechensystemen jeweils unterschiedliche Zahlendarstellungen gewählt. Während die ersten Computer noch im Prozessor integrierte Recheneinheiten für Gleitkomma-Arithmetik, die eine sogenannte *floating-point processing unit* (FPU) hatten, verschwand diese zunächst wieder aus den üblichen Computern und war nur in speziellen Rechnern vorhanden. In Form von *Coprozessoren* konnte eine FPU nachgerüstet werden (z. B. der Intel 8087 zum Intel 8086). Der „486er“ war der erste Prozessor für Heimcomputer mit integrierter FPU.

Heute können Gleitkommaberechnungen effizient auf Grafikkarten ausgelagert werden. Die Prozessoren der Grafikkarten, die *graphics processing units* (GPU) sind speziell für solche Berechnungen ausgelegt (z. B. schnelle Berechnung von Lichtbrechungen und Spiegelungen, Abbilden von Mustern auf 3D-Oberflächen). Spezielle Steckkarten, die gleich mehrere GPUs enthalten, werden in Höchstleistungssystemen eingesetzt. Diese Beschleunigerkarten wurden zunächst für Anwendungen der Computergraphik entwickelt und heute spielen sie im *maschinellen Lernen* sowie bei der *Künstlichen Intelligenz (KI)* eine große Rolle. Bei beidem kommt es weniger auf die Genauigkeit als auf die Geschwindigkeit an. Daher hat sich die Berechnung mit Zahlen vom Typ *half-precision* etabliert.

Beispiel 1.15 (Gleitkommadarstellung)

Wir gehen von vierstelliger Mantisse und vier Stellen im Exponenten aus mit Bias $2^{4-1} - 1 = 7$, also ist $\#m = \#e = 4$.

- Die Zahl $x = -96$ hat negatives Vorzeichen, also $S = 1$. Die Binärdarstellung von 96 gemäß (1.9) ist

$$96_{10} = 1 \cdot 64 + 1 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 0 \cdot 1 = 1100000_2,$$

und normalisiert gemäß (1.11) und (1.12) ergibt sich

$$96_{10} = 1.1000_2 \cdot 2^{6_{10}} = 1.1_2 \cdot 2^{13_{10}-7_{10}} = 1.1_2 \cdot 2^{11_{10}-b}.$$

Als Gleitkommadarstellung im Format gemäß (1.13) erhalten wir

$$x = 111011000_2,$$

wobei $s = 1$, $e = 1101_2$ und $m = 1000_2$.

- Die Zahl $x = -384$ hat wieder negatives Vorzeichen und $S = 1$. Die Binärdarstellung von 384 ist:

$$\begin{aligned} 384_{10} &= 1 \cdot 256 + 1 \cdot 128 + 0 \cdot 64 + 0 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 0 \cdot 1 \\ &= 110000000_2 \end{aligned}$$

Also normalisiert

$$384_{10} = 1.1000 \cdot 2^{8_{10}} = 1.1000 \cdot 2^{15_{10}-7_{10}} = 1.1000_2 \cdot 2^{11_{10}-b}.$$

Alle Bits im Exponenten sind 1. Im IEEE-754 Format ist dieser spezielle Exponent $e = 1111_2$ jedoch zur Speicherung von *NaN* (not a number) vorgesehen. Die Zahl 384 ist zu groß, um in diesem Zahlenformat gespeichert zu werden. Stattdessen wird $-\infty$ oder binär 111110000_2 gespeichert.

- Die Zahl $x = 1/3 = 0.33333 \dots$ ist positiv, also $S = 0$. Die Binärdarstellung von $1/3$ ist

$$\frac{1}{3} = 0.01010101 \dots_2.$$

Normalisiert mit vierstelliger Mantisse erhalten wir

$$\frac{1}{3} \approx 1.0101_2 \cdot 2^{-2} = 1.0101_2 \cdot 2^{5_{10}-7_{10}} = 1.0101_2 \cdot 2^{0_{10}-b},$$

also die Binärdarstellung 001010101_2 . Wir mussten bei der Darstellung runden und erhalten rückwärts

$$1.0101_2 \cdot 2^{0101_2-7} = 1.3125 \cdot 2^{-2} = 0.328125$$

mit dem relativen Fehler

$$\left| \frac{\frac{1}{3} - 1.0101_2 \cdot 2^{0101_2-b}}{\frac{1}{3}} \right| \approx 0.016.$$

- Die Zahl $x = \sqrt{2} \approx 1.4142135623 \dots$ ist positiv, also $S = 0$. Gerundet gilt:

$$\sqrt{2} \approx 1.0111_2$$

Diese Zahl liegt bereits normalisiert vor. Mit Bias $b = 7$ gilt für den Exponenten $e = 0 = 7 - 7$

$$\sqrt{2} \approx 1.0111_2 \cdot 2^{0111_2-b},$$

also 001110111 . Rückwärts in Dezimaldarstellung erhalten wir

$$1.0111_2 \cdot 2^{0111_2-b} = 1.4375$$

mit dem relativen Darstellungsfehler

$$\left| \frac{\sqrt{2} - 1.4375}{\sqrt{2}} \right| \approx 0.016.$$

- Schließlich betrachten wir die Zahl $x = -0.003$. Mit $S = 1$ gilt:

$$0.003_{10} \approx 0.00000000110001_2$$

und normalisiert

$$0.003_{10} \approx 1.1001_2 \cdot 2^{-9} = 1.1001_2 \cdot 2^{-2-7}.$$

Der Exponent $-9 = -2 - b$ ist zu klein und kann in diesem Format (also vier Stellen für den Exponenten) nicht dargestellt werden. Das IEEE-754 sieht als Erweiterung die *denormalisierten Zahlen* für Zahlen vor, die besonders nahe bei Null liegen. Hier werden alle Bits (bis auf das Vorzeichen) zur Speicherung der Mantisse verwendet und der Exponent ist immer 2^{1-b} . Details finden sich in [55].



Beispiel 1.15 hat gezeigt, dass eine Zahl zu groß (oder zu negativ) sein kann, um in einer gegebenen Gleitkommakonvention dargestellt werden zu können. Gleichzeitig kann eine Zahl zu nahe an der Null liegen, als dass sie in der Gleitkommadarstellung von Null unterschieden werden kann. Diese wichtigen Fälle erhalten ihre eigene Definition.

► **Definition 1.16 (overflow und underflow)** Als *overflow* wird die Situation bezeichnet, wenn der Absolutwert einer Zahl größer als die größte darstellbare Maschinenzahl ist. Als *underflow* wird die Situation bezeichnet, wenn eine von Null verschiedene Zahl auf Null abgerundet wird. Das Intervall $[-u_{\min}, u_{\min}] \subset \mathbb{R}$ von Zahlen, die zu klein, als dass sie als von Null verschiedene Gleitkommazahl dargestellt werden können, wird der *underflow gap* genannt.

Wann ein *overflow* oder ein *underflow* auftritt hängt im Wesentlichen von den Anzahl der Stellen im Exponenten ab.

Die Ungenauigkeit in der Zahldarstellung muss in der numerischen Mathematik immer berücksichtigt werden. Denn sie führt dazu, dass auch einfachste Rechnungen nicht exakt durchgeführt werden können, bzw., dass das Ergebnis nicht exakt gespeichert werden kann. Die genaue Definition des IEEE-754 Formats ist allerdings zumeist unerheblich. Auch ist die Wahl des Binärsystems nicht entscheidend. Wir definieren daher eine vereinfachte Interpretation:

► **Definition 1.17 (Zahldarstellung mit n -stelliger Genauigkeit)** Für eine Zahl $x \in \mathbb{R}$ bezeichnen wir mit der *Zahldarstellung mit n Stellen Genauigkeit* die Approximation

$$\text{rd}(x) = \text{sgn}(x)0.x_1x_2 \dots x_n \cdot 10^E,$$

wobei $x_1, \dots, x_n \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ die n wesentlichen Stellen sind, $\text{sgn}(x)$ das Vorzeichen von x ist und der Exponent $E \in \mathbb{N}$ gegeben ist als

$$E = \begin{cases} \lceil \log_{10}(|x|) \rceil & x \neq 0 \\ 0 & x = 0 \end{cases}$$

Alternativ schreiben wir eine Zahl mit n Stellen Genauigkeit als eine Abfolge von n zusammenhängenden Ziffern mit beliebig vielen führenden oder abschließenden Nullen. Dies bedeutet beispielsweise, dass die Zahl $x = 100/3$ bei drei Stellen Genauigkeit die vereinfachte Darstellung

$$\text{rd}\left(\frac{100}{3}\right) = 33.3 = 0.333 \cdot 10^2$$

hat. Die Lichtgeschwindigkeit ist bei 5 Stellen Genauigkeit gegeben als

$$\text{rd}(299\,792\,458) \text{ ms}^{-1} = 299\,790\,000 \text{ ms}^{-1} = 0.29979 \cdot 10^9 \text{ ms}^{-1}$$

und schließlich ist das Plancksche Wirkungsquantum bei 4-stelliger Genauigkeit gegeben als

$$\begin{aligned} \text{rd}(6.626\,070\,15 \cdot 10^{-34}) \text{ Js} \\ = 000\,000\,000\,000\,000\,000\,000\,000\,000\,000\,662\,6 \text{ Js} \\ = 0.6626 \cdot 10^{-35} \text{ Js.} \end{aligned}$$

1.4.2 Maschinengenauigkeit

Aus der begrenzten Anzahl von Ziffern der Mantisse ergibt sich zwangsläufig ein Fehler bei der Durchführung von numerischen Algorithmen. Der relative Fehler, der bei der Computerdarstellung $\tilde{x}$ einer Zahl $x \in \mathbb{R}$ entstehen kann,

$$\left| \frac{\text{rd}(x) - x}{x} \right|,$$

ist durch die sogenannte *Maschinengenauigkeit* beschränkt:

► **Definition 1.18 (Maschinengenauigkeit)** Die *Maschinengenauigkeit* eps ist der maximale relative Rundungsfehler der Zahldarstellung und wird bestimmt als

$$\text{eps} := \inf\{x > 0 : \text{rd}(1 + x) > 1\}.$$

Der Maschinengenauigkeit kommt in der numerischen Mathematik eine große Bedeutung zu. Sie ist weniger eine Abschätzung für den maximalen relativen Fehler, der in jeder Rechnung auftreten kann, sondern vielmehr ein Maß für den Fehler, den wir in jedem Rechenschritt erwarten müssen. Die Maschinengenauigkeit gilt dabei nicht gleichmäßig im ganzen darstellbaren Zahlenraum. Sind z. B. denormalisierte Zahlen vorgesehen, so verschlechtert sich die Genauigkeit für x nahe bei Null. Definition 1.18 reicht aber im Allgemeinen zur Beschreibung der wesentlichen Effekte.

Da Rundungsfehler zwangsläufig auftreten, gelten grundlegende mathematische Gesetze wie das Assoziativgesetz

$$(a + b) + c = a + (b + c), \quad (a \cdot b) \cdot c = a \cdot (b \cdot c)$$

oder das Distributivgesetz

$$a \cdot (b + c) = ab + ac, \quad (a + b) \cdot c = ac + bc$$

auf Computern nicht mehr. D. h., aufgrund von Rundungsfehlern, die immer auftreten, spielt die Reihenfolge, in der Operationen ausgeführt werden, eine wichtige Rolle und verändert das Ergebnis. Auch ein einfacher Vergleich von Zahlen ist oft nicht möglich, die Abfrage

```
1 if 3.8/10 == 0.38:
```

kann durch Rundung das falsche Ergebnis liefern und muss durch Abfragen der Art

```
1 if np.abs(3.8/10 - 0.38) < eps:
```

ersetzt werden (der Befehl `np.abs(...)` steht in der Python-Bibliothek `numpy` für den Betrag einer Zahl). Die Maschinengenauigkeit kann in Python mit

```
1 import numpy
2 print(numpy.finfo(float).eps)
```

ermittelt werden. Die kleinsten und größten darstellbaren Zahlen erhalten wir mit

```
1 import sys
2 sys.float_info.min
3 sys.float_info.max
```

Diese Werte hängen vom Betriebssystem ab und nicht der verwendeten Software. Auf unserem Rechner erhalten wir

```
2.220446049250313e-16    # eps (Maschinengenauigkeit)
2.2250738585072014e-308  # min (kleinste darstellbare Zahl)
1.7976931348623157e+308  # max (größte darstellbare Zahl)
```

Die Maschinengenauigkeit spielt auch bei der Bestimmung von Abbruchkriterien von iterativen Algorithmen eine wichtige Rolle. Wir betrachten z. B. die Fehlerabschätzung zur Leibniz-Reihe aus Abschn. 1.1.6, siehe (1.5). Wir wollen die Iteration abbrechen, wenn der Fehler kleiner ist als eine von uns gewählte Toleranz $\epsilon > 0$. D. h., im Falle der Leibniz-Reihe brechen wir die Iteration ab, wenn

$$\frac{|\Pi_{2n+1} - \Pi_{2n}|}{\pi} < \epsilon$$

gilt. Wir müssen stets $\epsilon \gg \text{eps}$ wählen. Denn die Forderung $\epsilon < \text{eps}$ werden wir nie erfüllen können, da ja schon der Darstellungsfehler der Lösung größer sein kann.

Bemerkung 1.19 Die Maschinengenauigkeit hat nichts mit der kleinsten Zahl zu tun, die auf einem Computer darstellbar ist. Diese ist durch die Anzahl der Stellen im Exponenten bestimmt. Die Maschinengenauigkeit wird durch die Anzahl der Stellen in der Mantisse bestimmt. ♦

1.4.3 Rechengeschwindigkeit – Floating point operations per second

Die meisten numerischen Algorithmen bestehen im Kern aus der wiederholten Ausführung der Grundoperationen. Der Aufwand von Algorithmen wird in der Anzahl der notwendigen elementaren Operationen (in Abhängigkeit von der Problemgröße) gemessen. Die Laufzeit eines Algorithmus hängt von der Leistungsfähigkeit des Rechners ab. Diese wird in *FLOPS*, also *floating point operations per second* gemessen.

In Tab. 1.3 fassen wir die erreichbaren FLOPS für verschiedene Computersysteme zusammen. Die Leistung ist im Laufe der Jahre rapide gestiegen, alle zehn Jahre wird etwa der Faktor 1000 erreicht. Die Leistungssteigerung beruht zum einen auf effizienterer Hardware. Erste Computer hatten Register mit 8 Bit Breite, die in jedem Takt (das ist die MHz-Angabe) verarbeitet werden konnten. Auf aktueller Hardware stehen Register mit 64 Bit zur Verfügung. Hinzu kommt eine effizientere Abarbeitung von Befehlen durch sogenanntes *Pipelining*: Die übliche Abfolge im Prozessor beinhaltet 1. *Speicher lesen*, 2. *Daten bearbeiten* und 3. *Ergebnis speichern*. Das Pipelining erlaubt es dem Prozessor, schon den Speicher für die nächste Operation auszulesen, während noch die aktuelle bearbeitet wird. So konnte die Anzahl der notwendigen Prozessortakte pro Rechenoperation erheblich reduziert werden. Die Kombination aus Intel 8086 mit FPU 8087 hat bei einem Takt von 8 Mhz und 50000 FLOPS etwa 150 Takte pro Fließkommaoperation benötigt. Der 486er braucht bei 66 MHz und etwa 1 000 000 FLOPS nur 50 Takte pro Operation. Der Pentium III liegt bei etwa 5 Takten pro Fließkommaoperation.

In den letzten Jahren beruht die Effizienzsteigerung wesentlich auf einem sehr hohen Grad an Parallelisierung. Ein Core i7-Prozessor kann mehrere Operationen gleichzeitig durchführen. Eine Nvidia L40 GPU erreicht ihre Leistung mit fast 20000 Rechenkernen. Um diese Leistung effizient nutzen zu können, müssen die Algorithmen entsprechend angepasst werden, sodass auch alle Kerne ausgelastet werden. Kann ein Algorithmus diese spezielle Architektur nicht ausnutzen, so fällt die Leistung auf etwa ein GigaFLOP zurück, also auf das Niveau des Pentium III aus dem Jahr 2001. Werden die 20000 Kerne hingegen optimal ausgenutzt, so erreicht eine einzige Grafikkarte vom Typ L40 die gleiche Leistungsklasse wie der K Computer, der im Jahr 2011 der schnellste Computer der Welt war. Noch beeindruckender wird dieser Vergleich, wenn die Anschaffungskosten pro FLOP oder der Energieverbrauch pro FLOP betrachtet werden.

Modernste Supercomputer wie der *Frontier* (schnellster Computer der Welt in 2023) vernetzen fast 10 000 000 Kerne (die meisten auf Beschleunigerkarten). Einen aktuellen Überblick gibt die *Top 500 Liste der Supercomputer*³. Bei parallelen Höchstleistungsrechnern spielt auch der Stromverbrauch eine bedeutende Rolle. Der *Frontier* verbraucht im Jahr so viel Strom wie 100 000 Privatpersonen. Jede Nutzungsstunde schlägt mit einer Stromrechnung von einigen Tausend Euro zu Buche. Durch neue Bauweisen, insbesondere durch kleineres Layout der Platinen, kann der Stromverbrauch pro FLOP ständig gesenkt werden.

³ <https://www.top500.org> Liste der jeweils 500 schnellsten Computer der Welt.

Tab. 1.3 Gleitkommageschwindigkeit sowie Anschaffungskosten einiger aktueller und historischer Computer (HD = Heidelberg). Nicht alle Computer oder Beschleunigerkarten erreichen ihre Performance bei der gleichen Genauigkeit

CPU	Jahr	FLOPS	Preis	Energieverbrauch
Zuse Z3	1941	0.3	Einzelstücke	4 kW
IBM 704	1955	$5 \cdot 10^3$	>10 000 000 €	75 kW
Intel 8086 + 8087	1980	$50 \cdot 10^3$	2000 €	200 W
i486	1991	$1.4 \cdot 10^6$	2000 €	200 W
iPhone 4s	2011	$100 \cdot 10^6$	500 €	10 W
2 x Pentium III	2001	$800 \cdot 10^6$	2000 €	200 W
Core i7	2011	$100 \cdot 10^9$	1000 €	200 W
Helics I (Parallelrechner in HD)	2002	$1 \cdot 10^{12}$	1 000 000 €	40 kW
NVIDIA GTX 580 (GPU)	2010	$1 \cdot 10^{12}$	500 €	250 W
K Computer	2011	$10 \cdot 10^{15}$	500 000 000 €	12 000 kW
Frontier	2022	$1.1 \cdot 10^{18}$	600 000 000 €	22 300 kW
NVIDIA L40 (GPU)	2022	$1.2 \cdot 10^{15}$	8000 €	300 W

1.5 Rundungsfehleranalyse und Stabilität von numerischen Algorithmen

Beim Entwurf von numerischen Algorithmen für eine Aufgabe sind oft unterschiedliche Wege möglich, die sich z. B. in der Reihenfolge der Verfahrensschritte unterscheiden. Unterschiedliche Algorithmen zu ein und derselben Aufgabe können dabei rundungsfehlerbedingt zu unterschiedlichen Ergebnissen führen.

Beispiel 1.20 (Distributivgesetz)

Wir betrachten die Aufgabe $A(x, y, z) = x \cdot z - y \cdot z = (x - y)z$ und zur Berechnung zwei Vorschriften:

$$\begin{aligned} a_1^{(1)} &:= x \cdot z, & a_1^{(2)} &:= x - y, \\ a_2^{(1)} &:= y \cdot z, & a^{(2)} &:= a_1^{(2)} \cdot z, \\ a^{(1)} &:= a_1^{(1)} - a_2^{(1)} \end{aligned}$$

Es sei $x = 0.519$, $y = 0.521$, $z = 0.941$. Bei vierstelliger Arithmetik erhalten wir:

$$\begin{aligned} a_1^{(1)} &:= \text{rd}(x \cdot z) = 0.4884, & a_1^{(2)} &:= \text{rd}(x - y) = -0.002, \\ a_2^{(1)} &:= \text{rd}(y \cdot z) = 0.4903, & a_2^{(2)} &:= \text{rd}(a_1^{(2)} \cdot z) = -0.001882, \\ a_3^{(1)} &:= \text{rd}(a_1^{(1)} - a_2^{(1)}) = -0.0019 \end{aligned}$$

Mit $A(x, y, z) = -0.001882$ ergeben sich die relativen Fehler:

$$\left| \frac{-0.0001882 - a_3^{(1)}}{0.0001882} \right| \approx 0.01, \quad \left| \frac{-0.0001882 - a_2^{(2)}}{0.0001882} \right| = 0$$

Das Distributivgesetz gilt auf dem Computer nicht! ◀

Die Stabilität hängt entscheidend vom Design des Algorithmus ab. Eingabefehler oder auch Rundungsfehler, die in einzelnen Schritten entstehen, werden in darauffolgenden Schritten des Algorithmus verstärkt. Wir analysieren nun beide Verfahren im Detail und gehen davon aus, dass in jedem der elementaren Schritte (wir haben hier nur Addition und Multiplikation) ein relativer Rundungsfehler ϵ mit $|\epsilon| \leq \text{eps}$ entsteht, also

$$\text{rd}(x + y) = (x + y)(1 + \epsilon), \quad \text{rd}(x \cdot y) = (x \cdot y)(1 + \epsilon).$$

Zur Analyse eines gegebenen Algorithmus verfolgen wir die Rundungsfehler, welche in jedem Schritt entstehen, und deren Akkumulation:

Beispiel 1.21 (Stabilität des Distributivgesetzes)

Wir berechnen zunächst die Konditionszahlen der Aufgabe:

$$\kappa_{A,x} = \left| \frac{1}{1 - \frac{y}{x}} \right|, \quad \kappa_{A,y} = \left| \frac{1}{1 - \frac{x}{y}} \right|, \quad \kappa_{A,z} = 1$$

Für $x \approx y$ ist die Aufgabe schlecht konditioniert. Wir starten mit Algorithmus 1 und schätzen in jedem Schritt den Rundungsfehler ab. Zusätzlich betrachten wir Eingabefehler (oder Darstellungsfehler) von x , y und z . Wir berücksichtigen stets nur Fehlerterme erster Ordnung und fassen alle weiteren Terme mit den Landau-Symbolen (für kleine ϵ) zusammen:

$$\begin{aligned} a_1 &= x(1 + \epsilon_x)z(1 + \epsilon_z)(1 + \epsilon_1) = xz(1 + \epsilon_x + \epsilon_z + \epsilon_1 + O(\epsilon^2)) \\ &= xz(1 + 3\epsilon_1 + O(\epsilon^2)) \\ a_2 &= y(1 + \epsilon_y)z(1 + \epsilon_z)(1 + \epsilon_2) = yz(1 + \epsilon_y + \epsilon_z + \epsilon_2 + O(\epsilon^2)) \\ &= yz(1 + 3\epsilon_2 + O(\epsilon^2)) \\ a_3 &= (xz(1 + 3\epsilon_1) - yz(1 + 3\epsilon_2) + O(\epsilon^2))(1 + \epsilon_3) \\ &= (xz - yz)(1 + \epsilon_3) + 3xz\epsilon_1 - 3yz\epsilon_2 + O(\epsilon^2) \end{aligned}$$

Wir bestimmen den relativen Fehler:

$$\left| \frac{a_3 - (xz - yz)}{xz - yz} \right| = \frac{|(xz - yz)\epsilon_3 + 3xz\epsilon_1 - 3yz\epsilon_2|}{|xz - yz|} \leq \epsilon + 3 \frac{|x| + |y|}{|x - y|} \epsilon$$

Die Fehlerverstärkung dieses Algorithmus kann für $x \approx y$ groß werden und entspricht etwa (Faktor 3) der Konditionierung der Aufgabe. Wir nennen den Algorithmus daher stabil.

Wir betrachten nun Algorithmus 2:

$$\begin{aligned} a_1 &= (x(1 + \epsilon_x) - y(1 + \epsilon_y))(1 + \epsilon_1) \\ &= (x - y)(1 + \epsilon_1) + x\epsilon_x - y\epsilon_y + O(\epsilon^2) \\ a_2 &= z(1 + \epsilon_z)((x - y)(1 + \epsilon_1) + x\epsilon_x - y\epsilon_y + O(\epsilon^2))(1 + \epsilon_2) \\ &= z(x - y)(1 + \epsilon_1 + \epsilon_2 + \epsilon_z + O(\epsilon)^2) + zx\epsilon_x - zy\epsilon_y + O(\epsilon^2) \end{aligned}$$

Für den relativen Fehler gilt in erster Ordnung:

$$\begin{aligned} \left| \frac{a_2 - (xz - yz)}{xz - yz} \right| &= \frac{|z(x - y)(\epsilon_1 + \epsilon_2 + \epsilon_z) + zx\epsilon_x - zy\epsilon_y|}{|xz - yz|} \\ &\leq 3\epsilon + \frac{|x| + |y|}{|x - y|}\epsilon \end{aligned}$$

Die Fehlerverstärkung kann für $x \approx y$ wieder groß werden. Der Verstärkungsfaktor ist jedoch geringer als bei Algorithmus 1. Insbesondere fällt auf, dass dieser zweite Algorithmus bei fehlerfreien Eingabedaten keine Fehlerverstärkung aufweist. (Das ist der Fall $\epsilon_x = \epsilon_y = \epsilon_z = 0$.)

Beide Algorithmen sind stabil, der zweite hat bessere Stabilitätseigenschaften als der erste. ◀

Wir nennen die beiden Algorithmen stabil, obwohl der eine wesentlich bessere Stabilitätseigenschaften hat. Der Stabilitätsbegriff dient daher oft zum Vergleich verschiedener Algorithmen. Wir definieren:

► **Definition 1.22 (Stabilität)** Ein numerischer Algorithmus zum Lösen einer Aufgabe heißt *stabil*, falls die bei der Durchführung akkumulierten Rundungsfehler den durch die Kondition der Aufgabe gegebenen unvermeidlichen Fehler nicht übersteigen.

Wir halten fest: Für ein schlecht konditioniertes Problem existiert kein stabiler Algorithmus mit einer geringeren Fehlerfortpflanzung als durch die Konditionierung bestimmt. Für gut konditionierte Probleme können jedoch beliebig instabile Verfahren existieren.

Der wesentliche Unterschied zwischen beiden Algorithmen aus dem Beispiel ist die Reihenfolge der Operationen. In Algorithmus 1 ist der letzte Schritt eine Subtraktion, deren schlechte Kondition wir unter dem Begriff *Auslöschung* kennengelernt haben. Bereits akkumulierte Rundungsfehler zu Beginn des Verfahrens werden hier noch einmal wesentlich verstärkt. Bei Algorithmus 2 werden durch die abschließende Multiplikation die Rundungsfehler, die zu Beginn auftreten, nicht weiter verstärkt. Aus dem analysierten Beispiel leiten wir die Regel her:

Bei dem Entwurf von numerischen Algorithmen sollen schlecht konditionierte Operationen zu Beginn durchgeführt werden.

1.6 Exkurs: Die Steuerfunktion in der Einkommensteuerberechnung

Wir präsentieren im Folgenden einen ersten Exkurs, welcher die durchaus überraschende Einsicht bereit hält, in welchen alltäglichen Bereichen Numerik zu finden ist. Wir veranschaulichen das Horner-Schema als auch die konkrete Anwendung von Rundungsfehlerrechnungen (Abschn. 1.3) anhand eines praktischen Beispiels aus der Steuergesetzgebung. Die Einkommensteuern können durch Polynome beschrieben werden, um diese anschließend in Steuertabellen anzugeben. In den bis 2009 gültigen Versionen des Einkommensteuergesetzes (EstG §32a) wird explizit vermerkt (siehe Absatz (3) in EstG §32a), dass die Polynome mit dem Horner-Schema auszuwerten sind⁴. Seit dem Jahr 2010 sind allerdings die Absätze (2) – (4) im Gesetz weggefallen. Nichtsdestotrotz schauen wir uns die aktuellen Werte (Jahr 2023) an. Der amtliche Einkommensteuertarif ist bereits in der Form (1.6) angegeben [23] (EstG §32a Einkommensteuertarif, Absatz 1, Stand 2023)^{5,6}:

$$E(x) = \begin{cases} 0 \text{ €} & x \leq 10\,908 \text{ €} \\ (979.18y + 1\,400)y \text{ €} & 10\,909 \text{ €} \leq x - 10\,908 \leq 15\,999 \text{ €} \\ (192.59z + 2\,397)z \text{ €} + 966.53 \text{ €} & 16\,000 \text{ €} \leq x - 15\,999 \leq 62\,809 \text{ €} \\ 0.42x \text{ €} - 9\,972.98 \text{ €} & 62\,810 \text{ €} \leq x \leq 277\,825 \text{ €} \\ 0.45x \text{ €} - 18\,307.73 \text{ €} & 277\,826 \text{ €} \leq x, \end{cases} \quad (1.14)$$

wobei $y = (x - 10\,908)/10\,000$ und $z = (x - 15\,999)/10\,000$.

⁴ EstG §32a, Absatz 3 [23] (Jahr 2002): „Die zur Berechnung der tariflichen Einkommensteuer erforderlichen Rechenschritte sind in der Reihenfolge auszuführen, die sich nach dem Horner-Schema ergibt. Dabei sind die sich aus den Multiplikationen ergebenden Zwischenergebnisse für jeden weiteren Rechenschritt mit drei Dezimalstellen anzusetzen; die nachfolgenden Dezimalstellen sind fortzulassen. Der sich ergebende Steuerbetrag ist auf den nächsten vollen Euro-Betrag abzurunden.“

⁵ EstG §32a Einkommensteuertarif, Absatz 1, Fussnote 1: Die tarifliche Einkommensteuer bemisst sich nach dem auf volle Euro abgerundeten zu versteuernden Einkommen.

⁶ 3) Die Größe „y“ ist ein Zehntausendstel des den Grundfreibetrag übersteigenden Teils des auf einen vollen Euro-Betrag abgerundeten zu versteuernden Einkommens. 4) Die Größe „z“ ist ein Zehntausendstel des 15 999 EUR übersteigenden Teils des auf einen vollen Euro-Betrag abgerundeten zu versteuernden Einkommens. 5) Die Größe „x“ ist das auf einen vollen Euro-Betrag abgerundete zu versteuernde Einkommen. 6) Der sich ergebende Steuerbetrag ist auf den nächsten vollen Euro-Betrag abzurunden.

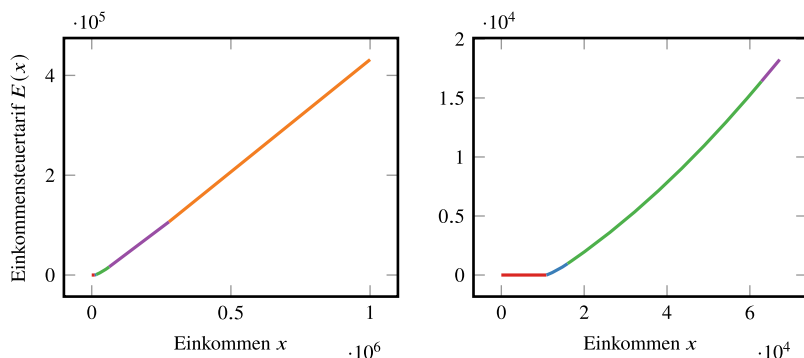


Abb. 1.2 Steuerfunktion $E(x)$. Links: gesamte Steuerfunktion. Rechts: Einschränkung auf Einkommen bis 70 000 €

In den Fußnoten werden mit y und z andere Bezeichnungen für die Variablen verwendet, je nachdem, in welchem Bereich das Einkommen liegt. Diese stückweise definierte Funktion $E(x)$ wird Steuerfunktion genannt, siehe Abb. 1.2.

Wir erkennen sofort die Form der Horner-Polynome (1.6):

$$p(x) = a_0 + x(a_1 + xa_2)$$

z. B. mit $a_0 = 0$, $a_1 = 1\,400$, $a_2 = 979.18$.

Wir nehmen als Einstiegsgehalt einer Ingenieurin $x = 53\,750$ € (Master (Uni/TH), Stand Jahr 2022⁷) Brutto-Jahreseinkommen an. Das heißt, wir werten die Steuerfunktion an der Stelle $\xi = 53\,750$ € aus. Damit fällt diese in Gruppe 3. Zunächst ist $z = \frac{53\,750 - 15\,999}{10\,000} = 3.775$ (in der amtlichen Rechnung werden drei Dezimalstellen angesetzt). Das Horner-Schema liest sich nun:

i	2	1	0
a_i	192.59 2 397.00	966.53	
E_i	192.53 3 125.027	12 759.731	$= E(\xi)$

Im Jahr 2023 muss die Ingenieurin damit $E(53\,750) = 12\,759.00$ € an Einkommensteuer bezahlen.

Wir berechnen nun den Spitzensteuersatz an der Stelle $\xi = 53\,750$ €. Der Spitzensteuersatz zum Einkommen x gibt an, welcher Anteil jedes zusätzlich verdienten Euros an Steuern abgeführt werden muss.

Die Spitzensteuersatzkurve ist ein Grenzsteuersatz und mathematisch durch die erste Ableitung $E'(x)$ der Steuerfunktion $E(x)$ gegeben. Wie wir leicht in (1.14) sehen, ergibt

⁷ <https://www.academics.de/ratgeber/ingenieur-gehalt>

Tab. 1.4 Vollständiges Horner-Schema zur Berechnung der Steuerbetragsfunktion und dessen Ableitung im Einkommensbereich (3) aus (1.14)

i	2	1	0
a_i	192.59	2 397.00	939.57
E	192.59	3 125.027	12 759.731
E'_i	192.59	3 852.054	

sich für den 5. Einkommensbereich der Spitzensteuersatz von $E'(x) = 0.45$, d. h., 45 %, da $E(x) = 0.45x - 18\,307.73$. Zur Berechnung des Spitzensteuersatzes für unsere Ingenieurin mit $\xi = 53\,750\text{€}$ Einkommen ist Vorsicht bei der Ableitung geboten, da die Funktion $E(z)$ in z gegeben ist, wir aber an x interessiert sind. Dementsprechend ist die Kettenregel anzuwenden, da $E(z) := E(z(x))$ und $E'(z(x)) \cdot z'(x)$ gilt.

Dann gilt für den mittleren Einkommensbereich (3), siehe (1.14), das um die 1. Ableitung erweiterte Horner-Schema mit der Darstellung aus Tab. 1.4. Um nun den Spitzensteuersatz zu bekommen, müssen wir noch die Ableitung der inneren Funktion $z'(x)$ miteinbeziehen:

$$E'(\xi) = a_1^{(2)} \cdot z'(\xi) = 3852.054 \cdot \frac{1}{10\,000},$$

da $z(x) = \frac{x-15\,999}{10\,000}$. Der Spitzensteuersatz bei einem Einkommen von 53 750€ liegt also bei 0.385, sprich 38.5 %. Im Vergleich dazu liegt der Durchschnittssteuersatz, welcher durch $d(x) = s(x)/x$ definiert ist, bei $d(x) = \frac{12\,759}{53\,750} = 0.237$, sprich 23.7 %.

Numerische Methoden der linearen Algebra

In der linearen Algebra wird die Struktur von linearen Abbildungen $T : V \rightarrow W$ zwischen endlich-dimensionalen Vektorräumen V und W untersucht. In der *numerischen linearen Algebra* befassen wir uns mit einigen praktischen Fragestellungen der linearen Algebra. Diese können grob in drei Schwerpunkte eingeteilt werden: *lineare Gleichungssysteme*, *Orthogonalisierungsverfahren* und *Eigenwertaufgaben*. Zunächst widmen wir uns dem Lösen von linearen Gleichungssystemen $Ax = b$. Fokus der Untersuchung ist auf der Lösung von linearen Gleichungssystemen mit reellen quadratischen Matrizen $A \in \mathbb{R}^{n \times n}$. Das bekannte Gaußsche Eliminationsverfahren ist der Einstiegspunkt. Wir werden jedoch schnell feststellen, dass dieses einfache Verfahren weder sehr stabil noch sehr effizient ist. Werden die Gleichungssysteme groß und bestehen z. B. aus $n \gg 1\,000\,000$ Gleichungen, so ist das Gaußsche Eliminationsverfahren nicht mehr durchführbar und wir müssen Alternativen untersuchen.

Des Weiteren untersuchen wir das Problem der Orthogonalisierung: Ein Erzeugendensystem $\{v_1, \dots, v_n\} \subset V$ soll bezüglich eines Skalarprodukts $(\cdot, \cdot)_V : V \times V \rightarrow \mathbb{R}$ in ein Orthogonalsystem überführt werden:

$$(v_i, v_j) = 0, \quad \forall i \neq j.$$

In der linearen Algebra wurde hierzu das *Orthogonalisierungsverfahren von Gram-Schmidt* eingeführt. Wieder sehen wir, dass dieses Verfahren zwar sehr einfach und effizient ist, jedoch numerische Stabilitätsprobleme aufweist. Ein numerisch berechnetes *Orthogonalsystem* wird durch Rundungsfehler im Ergebnis nicht mehr orthogonal sein.

Schließlich werden wir Verfahren zur Berechnung von Eigenwerten linearer Abbildungen untersuchen, also dem Finden von komplexen Zahlen $\lambda \in \mathbb{C}$ mit

$$A\omega = \lambda\omega,$$

wobei $\omega \in \mathbb{C}^n$ ein zugehöriger Eigenvektor ist. Wir werden sehen, dass die numerische Berechnung der Eigenwerte als Nullstellen des charakteristischen Polynoms $\chi(\lambda) := \det(A - \lambda I) = 0$ nicht stabil ist. Stattdessen werden wir iterative Verfahren kennenlernen, um die Eigenwerte zu approximieren.

Die meisten Probleme der linearen Algebra treten als Teilprobleme anderer Verfahren auf. Große lineare Gleichungssysteme müssen zur Diskretisierung von Differentialgleichungen, aber z. B. auch bei der Approximation von Funktionen gelöst werden.

Orthogonalisierungsverfahren werden bei der Lösung linearer Gleichungssysteme mit Fixpunktiterationen, aber auch beim Entwurf von Verfahren zur numerischen Quadratur (der Approximation von Integralen) benötigt.

In unseren Entwicklungen werden wir regelmäßig auf die in der Einleitung eingeführten *Konzepte* zurückgreifen und anhand derer unsere Resultate charakterisieren.

Wir sammeln zunächst einige Definitionen und grundlegende Resultate. Für eine ausführliche Einführung in die lineare Algebra sei beispielsweise auf [32] verwiesen. Es sei V stets ein endlich dimensionaler Vektorraum über dem Körper $\mathbb{K}$. Üblicherweise betrachten wir den Raum der reellwertigen Vektoren $V = \mathbb{R}^n$.

► **Definition 2.1 (Basis & Dimension)** Eine Teilmenge $B = \{v_1, \dots, v_n\} \subset V$ eines Vektorraums über $\mathbb{K}$ heißt *Basis*, falls sich jedes Element $v \in V$ eindeutig als Linearkombination der *Basisvektoren* B darstellen lässt:

$$v = \sum_{i=1}^n \alpha_i v_i, \quad \alpha_i \in \mathbb{K}.$$

Die Anzahl der Vektoren n in der Basis heißt die *Dimension des Vektorraumes*.

Die eindeutige Darstellbarkeit jedes $v \in V$ durch Basisvektoren erlaubt es, den Vektorraum V mit dem Vektorraum der Koeffizientenvektoren $\alpha \in \mathbb{K}^n$ zu identifizieren. Daher können wir uns in diesem Abschnitt im Wesentlichen auf diesen Raum (bzw. auf $\mathbb{R}^n$) beschränken. Alle Eigenschaften und Resultate übertragen sich auf V .

► **Definition 2.2 (Norm)** Eine Abbildung $\|\cdot\| : V \rightarrow \mathbb{R}_+$ heißt *Norm*, falls sie die folgenden drei Eigenschaften besitzt:

1. positive Definitheit: $\|x\| \geq 0 \quad \forall x \in V, \quad \|x\| = 0 \Rightarrow x = 0,$
2. Linearität: $\|\alpha x\| = |\alpha| \|x\| \quad \forall x \in V, \alpha \in \mathbb{K},$
3. Dreiecksungleichung: $\|x + y\| \leq \|x\| + \|y\| \quad \forall x, y \in V.$

Ein Vektorraum mit einer Norm heißt *normierter Raum*. Im $\mathbb{K}^n$ häufig verwendete Normen sind die *Maximumsnorm* $\|\cdot\|_\infty$, die *euklidische Norm* $\|\cdot\|_2$ sowie die *l_1 -Norm* $\|\cdot\|_1$

$$\|x\|_\infty := \max_{i=1,\dots,n} |x_i|, \quad \|x\|_2 := \left(\sum_{i=1}^n |x_i|^2 \right)^{\frac{1}{2}}, \quad \|x\|_1 := \sum_{i=1}^n |x_i|.$$

Im Vektorraum $\mathbb{R}^n$ sowie in allen endlich-dimensionalen Vektorräumen gilt die Äquivalenz aller Normen.

Satz 2.3 (Normäquivalenz) Zu zwei beliebigen Normen $\|\cdot\|$ sowie $\|\cdot\|'$ im endlich-dimensionalen Vektorraum V existiert eine Konstante $c > 0$, sodass gilt:

$$\frac{1}{c} \|x\| \leq \|x\|' \leq c \|x\| \quad \forall x \in V.$$

Normen sind wichtig für den Begriff der Konvergenz und der Satz besagt, dass eine Folge $x_n \rightarrow x$, welche bzgl. einer Norm $\|\cdot\|$ konvergiert, auch bzgl. jeder anderen Norm $\|\cdot\|'$ konvergiert. Dieser Zusammenhang ist typisch für endlich-dimensionale Räume und gilt z. B. nicht in (unendlich-dimensionalen) Funktionenräumen. So gilt z. B. für die Funktionenfolge $f_n(x) = \exp(-nx^2)$ in der L^2 -Norm

$$\|f_n - 0\|_{L^2([-1,1])} := \left(\int_{-1}^1 |f_n(x) - 0|^2 dx \right)^{\frac{1}{2}} \xrightarrow{n \rightarrow \infty} 0,$$

in der Maximumsnorm jedoch

$$\|f_n - 0\|_\infty := \max_{x \in [-1,1]} |f_n(x) - 0| \xrightarrow{n \rightarrow \infty} 0.$$

In endlich dimensionalen Vektorräumen sind zwar alle Normen äquivalent, aber die hierbei auftretenden Konstanten c können von der Dimension n des Vektorraumes abhängen. Das wird wichtig, wenn Konvergenzaussagen für $n \rightarrow \infty$ betrachtet werden. Im Grenzübergang gilt die Äquivalenz nicht mehr.

► **Definition 2.4 (Skalarprodukt)** Eine Abbildung $(\cdot, \cdot) : V \times V \rightarrow \mathbb{K}$ heißt *Skalarprodukt*, falls sie die folgenden Eigenschaften besitzt:

1. Definitheit: $(x, x) > 0 \quad \forall x \in V, x \neq 0,$
 $(x, x) = 0 \Rightarrow x = 0,$
2. Linearität: $(x, \alpha y + z) = \alpha(x, y) + (x, z) \quad \forall x, y, z \in V, \alpha \in \mathbb{K},$
3. Hermesch: $(x, y) = \overline{(y, x)} \quad \forall x, y \in V,$

dabei ist $\bar{z}$ die komplexe Konjugation von $z \in \mathbb{C}$.

In reellen Räumen gilt die echte Symmetrie $(x, y) = (y, x)$. Weiter folgt aus 2. und 3. dass $(\alpha x, y) = \alpha(x, y)$. Ein Beispiel ist das *euklidische Skalarprodukt* für Vektoren $x, y \in \mathbb{R}^n$

$$(x, y)_2 = x^T y = \sum_{i=1}^n x_i y_i.$$

Vektorräume mit Skalarprodukt werden *Prä-Hilbert-Räume* genannt. Komplexe Vektorräume mit Skalarprodukt nennt man auch *unitäre Räume*, im Reellen spricht man von *euklidischen Räumen*.

Skalarprodukte sind eng mit Normen verknüpft:

Satz 2.5 (Induzierte Norm) Es sei V ein Vektorraum mit Skalarprodukt. Dann ist durch

$$\|x\| = \sqrt{(x, x)}, \quad x \in V,$$

auf V die *induzierte Norm* gegeben.

Ein Prä-Hilbert-Raum ist somit immer auch ein normierter Raum. Falls der Prä-Hilbert-Raum V bezüglich der vom Skalarprodukt induzierten Norm vollständig ist, also ein Banach-Raum (siehe z. B. [88]) ist, so heißt V *Hilbert-Raum*.

Die euklidische Norm ist die vom euklidischen Skalarprodukt induzierte Norm:

$$\|x\|_2 = (x, x)_2^{\frac{1}{2}} = \left(\sum_{i=1}^n x_i^2 \right)^{\frac{1}{2}}$$

Einige wichtige Sätze gelten für Paare aus Skalarprodukt und induzierter Norm:

Satz 2.6 Es sei V ein Vektorraum mit Skalarprodukt $(\cdot, \cdot)$ und induzierter Norm $\|\cdot\|$. Dann gilt die Cauchy-Schwarzsche Ungleichung:

$$|(x, y)| \leq \|x\| \|y\| \quad \forall x, y \in V,$$

sowie die Parallelogrammidentität:

$$\|x + y\|^2 + \|x - y\|^2 = 2\|x\|^2 + 2\|y\|^2 \quad \forall x, y \in V.$$

Wir definieren weiter:

► **Definition 2.7 (Orthogonalität)** Zwei Vektoren $x, y \in V$ eines Prä-Hilbert-Raums heißen *orthogonal*, falls $(x, y) = 0$.

Die Einführung einer Normabbildung auf V ermöglicht es, den Abstand $\|x - y\|$ zu definieren. Die Existenz eines Skalarprodukts ermöglicht es, zwei Elementen einen Winkel zuzuordnen. Es gilt im euklidischen Skalarprodukt

$$\cos(\angle(x, y)) = \frac{(x, y)_2}{\|x\|_2 \|y\|_2},$$

und hieraus folgt $x \perp y$, falls $(x, y)_2 = 0$. Dieser euklidische Winkelbegriff und Orthogonalitätsbegriff lässt sich auf beliebige Räume mit Skalarprodukt übertragen.

Eine der Aufgaben der numerischen linearen Algebra ist die Orthogonalisierung (oder auch Orthonormalisierung) von gegebenen Systemen von Vektoren:

► **Definition 2.8 (Orthonormalbasis)** Eine Basis $B = \{v_1, \dots, v_n\}$ von V heißt *Orthogonalbasis* bezüglich des Skalarprodukts $(\cdot, \cdot)$, falls gilt:

$$(\phi_i, \phi_j) = 0 \quad \forall i \neq j,$$

und *Orthonormalbasis*, falls gilt:

$$(\phi_i, \phi_j) = \delta_{ij}$$

mit dem Kronecker-Symbol

$$\delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}.$$

Mit unterschiedlichen Skalarprodukten existieren unterschiedliche Orthonormalbasen zu ein und demselben Vektorraum V . Orthogonalität stimmt dann im Allgemeinen nicht mit dem geometrischen Orthogonalitätsbegriff des euklidischen Raums überein:

Beispiel 2.9 (Skalarprodukte und Orthonormalbasis)

Es sei $V = \mathbb{R}^2$. Durch

$$(x, y)_2 := x_1 y_1 + x_2 y_2, \quad (x, y)_\omega := 2 x_1 y_1 + x_2 y_2,$$

sind zwei verschiedene Skalarprodukte gegeben. Das erste ist das euklidische Skalarprodukt. Die Skalarprodukteigenschaften des zweiten sind einfach zu überprüfen. Durch

$$x_1 = \frac{1}{\sqrt{2}}(1, 1)^T, \quad x_2 = \frac{1}{\sqrt{2}}(-1, 1)^T,$$

ist eine Orthonormalbasis bezüglich $(\cdot, \cdot)_2$ gegeben. Es gilt jedoch:

$$(x_1, x_2)_\omega = \frac{1}{2}(-2 + 1) = \frac{-1}{2} \neq 0.$$

Eine Orthonormalbasis bzgl. $(\cdot, \cdot)_\omega$ erhalten wir z. B. durch

$$x_1 = \frac{1}{\sqrt{3}}(1, 1)^T, \quad x_2 = \frac{1}{2}(-1, 2)^T.$$



Orthonormalbasen werden für zahlreiche numerische Verfahren benötigt, bei der Gaußschen Quadratur (Abschn. 10.4), bei der Gauß-Approximation von Funktionen (Abschn. 9.5.1) und z. B. für die QR-Zerlegung (Kap. 4) einer Matrix.

Ist durch $\{a_1, \dots, a_n\}$ eine linear unabhängige Menge von Vektoren eines Vektorraums gegeben, so lässt sich mit dem Gram-Schmidt-Verfahren ein Orthogonal- oder sogar Orthonormalsystem erstellen:

Satz 2.10 (Gram-Schmidt-Orthonormalisierungsverfahren) Es sei durch $\{a_1, \dots, a_n\}$ eine linear unabhängige Teilmenge eines Vektorraums V gegeben, durch $(\cdot, \cdot)$ ein Skalarprodukt auf V mit induzierter Norm $\|\cdot\|$. Die Vorschrift

$$(i) \quad q_1 := \frac{a_1}{\|a_1\|},$$

$$(ii) \quad i = 2, \dots, n : \quad \tilde{q}_i := a_i - \sum_{j=1}^{i-1} (a_i, q_j) q_j, \quad q_i := \frac{\tilde{q}_i}{\|\tilde{q}_i\|},$$

erzeugt ein Orthonormalsystem $\{q_1, \dots, q_n\}$ in V . Es gilt ferner

$$(q_i, a_j) = 0 \quad \forall 1 \leq j < i \leq n.$$

Ist $\{a_1, \dots, a_n\}$ eine Basis von V , so ist durch $\{q_1, \dots, q_n\}$ eine Orthonormalbasis von V gegeben.

Beweis Wir führen den Beweis per Induktion. Für $i = 1$ ist $a_1 \neq 0$, da durch $a_1, \dots, a_n$ der ganze V aufgespannt wird. Für $i = 2$ gilt

$$(\tilde{q}_2, q_1) = (a_2, q_1) - (a_2, q_1) \underbrace{(q_1, q_1)}_{=1} = 0.$$

Aus der linearen Unabhängigkeit von a_2 und q_1 folgt, dass $\tilde{q}_2 \neq 0$. Es sei nun also $(q_j, q_k) = \delta_{jk}$ für $k, j < i$. Dann gilt für $k < i$ beliebig

$$(\tilde{q}_i, q_k) = (a_i, q_k) - \sum_{j=1}^{i-1} (a_i, q_j) \underbrace{(q_j, q_k)}_{=\delta_{jk}} = (a_i, q_k) - (a_i, q_k) = 0.$$

Da $\text{span}\{q_1, \dots, q_j\} = \text{span}\{a_1, \dots, a_j\}$, folgt auch $(q_i, a_j) = 0$ für $j < i$. □

Das Verfahren ist nach den beiden Mathematikern Gram und Schmidt benannt, war aber bereits Laplace und Cauchy bekannt.

Wir betrachten im Folgenden den Vektorraum aller $\mathbb{R}^{n \times m}$ -Matrizen. Auch dieser Vektorraum ist endlich-dimensional und prinzipiell können wir den Vektorraum der $n \times m$ -Matrizen mit dem Vektorraum der (nm) -Vektoren identifizieren. Von besonderem Interesse ist für uns der Vektorraum der quadratischen $\mathbb{R}^{n \times n}$ -Matrizen. Hier definieren wir:

► **Definition 2.11 (Eigenwerte, Eigenvektoren)** Die *Eigenwerte* $\lambda \in \mathbb{C}$ einer Matrix $A \in \mathbb{R}^{n \times n}$ sind definiert als Nullstellen des charakteristischen Polynoms:

$$\det(A - \lambda I) = 0$$

Die Menge aller Eigenwerte einer Matrix A heißt das *Spektrum* von A

$$\sigma(A) := \{\lambda \in \mathbb{C}, \lambda \text{ Eigenwert von } A\}.$$

Der *Spektralradius* $\text{spr} : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}_+$ ist der betragsmäßig größte Eigenwert:

$$\text{spr}(A) := \max\{|\lambda|, \lambda \in \sigma(A)\}$$

Ein Element $w \in \mathbb{R}^n \setminus \{0\}$ heißt *Eigenvektor* zum Eigenwert λ , falls gilt:

$$Aw = \lambda w.$$

Bei der Untersuchung von linearen Gleichungssystemen $Ax = b$ stellt sich zunächst die Frage, ob ein solches Gleichungssystem überhaupt lösbar und ob die Lösung eindeutig ist. Wir fassen zusammen:

Satz 2.12 (Reguläre Matrix) Für eine quadratische Matrix $A \in \mathbb{R}^{n \times n}$ sind die folgenden Aussagen äquivalent:

1. Die Matrix A ist regulär.
2. Die transponierte Matrix A^T ist regulär.
3. Die Inverse A^{-1} ist regulär.
4. Das lineare Gleichungssystem $Ax = b$ ist für jedes $b \in \mathbb{R}^n$ eindeutig lösbar.
5. Es gilt $\det(A) \neq 0$.
6. Alle Eigenwerte von A sind ungleich null.

Wir definieren weiter:

► **Definition 2.13 (Positive Definitheit)** Eine Matrix $A \in \mathbb{K}^{n \times n}$ heißt *positiv definit*, falls

$$(Ax, x) > 0 \quad \forall x \neq 0, \quad x \in \mathbb{K}^n.$$

Sie heißt *positiv semidefinit*, falls

$$(Ax, x) \geq 0 \quad \forall x \in \mathbb{K}^n.$$

Entsprechend heißt die Matrix *negativ definit*, falls

$$(Ax, x) < 0 \quad \forall x \neq 0, \quad x \in \mathbb{K}^n,$$

und *negativ semidefinit*, falls

$$(Ax, x) \leq 0 \quad \forall x \in \mathbb{K}^n.$$

Trifft keine dieser Ungleichungen zu, so heißt die Matrix *indefinit*.

Beispiel 2.14 (Definitheit von Matrizen)

Die Matrix

$$A = \begin{pmatrix} 1 & 4 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 3 \end{pmatrix}$$

ist positiv definit. Die Matrix

$$A = \begin{pmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

ist indefinit. ◀

Umgekehrt können wir aus der definierenden Eigenschaft der positiven Definitheit einer Matrix ablesen: Falls A positiv definit und hermitesch ist, so ist durch $(A \cdot, \cdot)$ ein Skalarprodukt gegeben. Es gilt:

Satz 2.15 (Positiv definite Matrizen) Es sei $A \in \mathbb{R}^{n \times n}$ eine symmetrische Matrix. Dann ist A genau dann positiv definit, falls alle (reellen) Eigenwerte von A positiv sind. Ist A symmetrisch positiv definit, so sind alle Diagonalelemente von A positiv, d. h. echt größer null, und das betragsmäßig größte Element steht auf der Diagonalen.

Beweis (i) Es sei A eine symmetrische Matrix mit einer Orthonormalbasis aus Eigenvektoren $w_1, \dots, w_n$. A sei positiv definit. Dann gilt für einen beliebigen Eigenvektor w_i mit Eigenwert λ_i :

$$0 < (Aw_i, w_i) = \lambda_i(w_i, w_i) = \lambda_i$$

Umgekehrt seien alle λ_i positiv. Für $x = \sum_{i=1}^n \alpha_i w_i$ mit $x \neq 0$ gilt:

$$(Ax, x) = \sum_{i,j} (\lambda_i \alpha_i \omega_i, \alpha_j \omega_j) = \sum_i \lambda_i \alpha_i^2 > 0$$

(ii) A sei nun eine reelle, positiv definite Matrix. Es sei e_i der i -te Einheitsvektor. Dann gilt:

$$0 < (Ae_i, e_i) = a_{ii}.$$

Das heißt, alle Diagonalelemente sind positiv.

(iii) Entsprechend wählen wir nun $x = e_i - \text{sign}(a_{ij})e_j$. Wir nehmen an, dass $a_{ji} = a_{ij}$ das betragsmäßig größte Element der Matrix sei. Dann gilt:

$$0 < (Ax, x) = a_{ii} - \text{sign}(a_{ij})(a_{ij} + a_{ji}) + a_{jj} = a_{ii} + a_{jj} - 2|a_{ij}| \leq 0.$$

Aus diesem Widerspruch folgt die letzte Aussage des Satzes: Das betragsmäßig größte Element muss ein Diagonalelement sein. $\square$

Für Normen auf dem Raum der Matrizen definieren wir weitere Struktureigenschaften.

► **Definition 2.16 (Matrixnormen)** Eine Norm $\|\cdot\| : \mathbb{R}^{n \times m} \rightarrow \mathbb{R}_+$, für alle $m, n \in \mathbb{N}$ heißt *Matrixnorm*, falls sie submultiplikativ ist:

$$\|AB\| \leq \|A\| \|B\| \quad \forall A \in \mathbb{R}^{k \times l}, B \in \mathbb{R}^{l \times m}.$$

Sie heißt *verträglich* mit einer Vektornorm $\|\cdot\| : \mathbb{R}^m \rightarrow \mathbb{R}_+$, falls

$$\|Ax\| \leq \|A\| \|x\| \quad \forall A \in \mathbb{R}^{n \times m}, x \in \mathbb{R}^m$$

gilt. Eine Matrixnorm $\|\cdot\| : \mathbb{R}^{n \times m} \rightarrow \mathbb{R}_+$ heißt von einer Vektornorm $\|\cdot\| : \mathbb{R}^m \rightarrow \mathbb{R}_+$ *induziert*, falls gilt, dass

$$\|A\| := \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|}.$$

Von einer Vektornorm induzierte Matrixnormen heißen auch *natürliche Matrixnormen*.

Für eine induzierte Matrixnorm gilt stets:

$$\|A\| := \sup_{x \neq 0} \frac{\|Ax\|}{\|x\|} = \sup_{\|x\|=1} \|Ax\| = \sup_{\|x\| \leq 1} \|Ax\|$$

Es ist leicht nachzuweisen, dass jede von einer Vektornorm induzierte Matrixnorm mit dieser auch verträglich ist. Verträglich mit der euklidischen Norm ist aber auch die *Frobenius-Norm*

$$\|A\|_F := \left(\sum_{i,j=1}^n a_{ij}^2 \right)^{\frac{1}{2}}, \quad (2.1)$$

welche nicht von einer Vektornorm induziert ist. Für allgemeine Normen auf dem Vektorraum der Matrizen gilt nicht notwendigerweise $\|I\| = 1$, wobei $I \in \mathbb{R}^{n \times n}$ die Einheitsmatrix ist. Dieser Zusammenhang gilt aber für jede von einer Vektornorm induzierte Matrixnorm. Wir fassen im folgenden Satz die wesentlichen induzierten Matrixnormen zusammen.

Satz 2.17 (Induzierte Matrixnormen) Sei $A \in \mathbb{R}^{n \times m}$. Die aus der euklidischen Vektornorm, der Maximumnorm sowie der l_1 -Norm induzierten Matrixnormen sind die *Spektralnorm* $\|\cdot\|_2$, die maximale Zeilensumme $\|\cdot\|_\infty$, sowie die maximale Spaltensumme $\|\cdot\|_1$:

$$\|A\|_2 = \sqrt{\text{spr}(A^T A)}, \quad \|A\|_\infty = \max_{i=1, \dots, n} \sum_{j=1}^m |a_{ij}|,$$

$$\|A\|_1 = \max_{j=1, \dots, m} \sum_{i=1}^n |a_{ij}|.$$

Beweis (i) Es gilt:

$$\|A\|_2^2 = \sup_{x \neq 0} \frac{\|Ax\|_2^2}{\|x\|_2^2} = \sup_{x \neq 0} \frac{(Ax, Ax)}{\|x\|_2^2} = \sup_{x \neq 0} \frac{(A^T Ax, x)}{\|x\|_2^2}.$$

Die Matrix $A^T A$ ist symmetrisch und hat als solche nur reelle Eigenwerte. Sie besitzt eine Orthonormalbasis $\omega_i \in \mathbb{R}^m$, $i = 1, \dots, m$, von Eigenvektoren mit Eigenwerten $\lambda_i \geq 0$. Alle Eigenwerte λ_i sind größer gleich null, denn:

$$\lambda_i = \lambda_i(\omega_i, \omega_i) = (A^T A \omega_i, \omega_i) = (A \omega_i, A \omega_i) = \|A \omega_i\|^2 \geq 0 \quad (2.2)$$

Es sei $x \in \mathbb{R}^m$ beliebig mit Basisdarstellung $x = \sum_i \alpha_i \omega_i$. Es gilt dann wegen $(\omega_i, \omega_j)_2 = \delta_{ij}$ die Beziehung $\|x\|_2^2 = \sum_i \alpha_i^2$ mit $\alpha_i \in \mathbb{R}$:

$$\begin{aligned} \|A\|_2^2 &= \sup_{|\alpha| \neq 0} \frac{(\sum_i \alpha_i A^T A \omega_i, \sum_i \alpha_i \omega_i)}{\sum_i \alpha_i^2} = \sup_{|\alpha| \neq 0} \frac{(\sum_i \alpha_i \lambda_i \omega_i, \sum_i \alpha_i \omega_i)}{\sum_i \alpha_i^2} \\ &= \sup_{|\alpha| \neq 0} \frac{\sum_i \lambda_i \alpha_i^2}{\sum_i \alpha_i^2} \leq \max_i \lambda_i, \end{aligned}$$

wobei

$$|\alpha| = \max_i |\alpha_i|.$$

Es sei nun umgekehrt durch λ_k der größte Eigenwert gegeben. Dann gilt wegen (2.2) mit $\alpha_i = \delta_{ki}$:

$$0 \leq \max_i \lambda_i = \lambda_k = \sum_i \lambda_i \alpha_i^2 = \sum_{i,j} (\lambda_i \alpha_i \omega_i, \alpha_j \omega_j) = (A^T Ax, x) = \|Ax\|_2^2. \quad (2.3)$$

Also gilt $\max_i \lambda_i \leq \|A\|_2^2$.

(ii) Wir zeigen das Ergebnis exemplarisch für die Maximumsnorm:

$$\|Ax\|_\infty = \sup_{\|x\|_\infty=1} \left(\max_i \sum_{j=1}^m a_{ij} x_j \right)$$

Diese Summe mit $\|x\|_\infty = 1$ nimmt ihr Maximum an, falls $|x_j| = 1$ und falls das Vorzeichen x_j so gewählt wird, dass $a_{ij}x_j \geq 0$ für alle $j = 1, \dots, m$. Dann gilt:

$$\|Ax\|_\infty = \max_i \sum_{j=1}^m |a_{ij}|$$

□

Als Nebenresultat erhalten wir aus (2.3), dass jeder Eigenwert betragsmäßig durch die Spektralnorm der Matrix A beschränkt ist. Es gilt sogar mit beliebiger Matrixnorm und verträglicher Vektornorm für einen Eigenwert λ mit zugehörigem Eigenvektor $w \in \mathbb{R}^n$ von A , dass

$$|\lambda| = \frac{\|\lambda w\|}{\|w\|} = \frac{\|Aw\|}{\|w\|} \leq \frac{\|A\| \|w\|}{\|w\|} = \|A\|.$$

Eine einfache Schranke für den betragsmäßig größten Eigenwert erhält man also durch Analyse beliebiger (verträglicher) Matrixnormen.

Aus Satz 2.17 folgern wir weiter, dass für symmetrische Matrizen die $\|\cdot\|_2$ -Norm mit dem Spektralradius der Matrix selbst übereinstimmt, daher der Name Spektralnorm.

Satz 2.18 *Es sei $A \in \mathbb{R}^{m \times n}$ eine beliebige rechteckige Matrix. Dann gilt:*

$$\text{Rang}(A) = \text{Rang}(A^T) = \text{Rang}(AA^T) = \text{Rang}(A^T A),$$

wobei $\text{Rang}(A) := \dim(\text{Bild}(A))$ den (Spalten-)Rang der Matrix A bezeichnet, also die maximale Anzahl linear unabhängiger Spaltenvektoren.

Der Beweis findet sich beispielsweise bei [32].

Das Lösen von linearen Gleichungssystemen (LGS) ist eine der wichtigsten numerischen Aufgaben. Oft sind Probleme nicht direkt in Form eines linearen Gleichungssystems gegeben, sondern ergeben sich im Zuge der Modellierung oder der Diskretisierung und Approximation von Problemen, z. B. von Differentialgleichungen oder Optimierungsproblemen. Solche linearen Gleichungssysteme sind unter Umständen sehr groß. Groß bedeutet, dass Gleichungssysteme mit vielen Millionen¹ bis Milliarden² Unbekannten gelöst werden müssen. Wir werden uns in diesem Abschnitt ausschließlich mit reellwertigen Matrizen $A \in \mathbb{R}^{n \times n}$ befassen. Methoden zum Lösen von linearen Gleichungssystemen klassifiziert man als *direkte Verfahren*, welche die Lösung des Gleichungssystems unmittelbar und bis auf Rundungsfehlereinflüsse exakt berechnen und *iterative Verfahren*, welche die Lösung durch eine Fixpunktiteration approximieren. In diesem Kapitel befassen wir uns zunächst mit direkten Methoden.

Als einführendes Beispiel betrachten wir ein Gleichungssystem mit Gleitkommazahlen

$$\begin{pmatrix} 0.988 & 0.960 \\ 0.992 & 0.963 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0.084 \\ 0.087 \end{pmatrix}$$

mit der Lösung $(x, y)^T = (3, -3)^T$. Wir bestimmen die Lösung wieder durch Gauß-Elimination und runden nach jeder Rechnung auf dreistellige Genauigkeit:

¹ Beispielsweise muss im Zuge der europäischen Raumfahrtmission Gaia ein Gleichungssystem mit 600 Mio. (also $6 \cdot 10^8$) Unbekannten gelöst werden [8].

² Im Jahr 2016 wurden auf einem sogenannten *Petascale-Computer*, also einem Parallelrechner mit mehr als 10^{15} FLOPS, lineare Gleichungssysteme mit bis zu $6 \cdot 10^{11}$ (0.6 Trillionen) Unbekannten gelöst [53], und in [62] mit 1.036×10^{12} Unbekannte pro Sekunde.

$$\left(\begin{array}{cc|c} 0.988 & 0.960 & 0.084 \\ 0.992 & 0.963 & 0.087 \end{array} \right) \times 0.988/0.992$$

$$\left(\begin{array}{cc|c} 0.988 & 0.960 & 0.084 \\ 0.988 & 0.959 & 0.0866 \end{array} \right) \downarrow -$$

$$\left(\begin{array}{cc|c} 0.988 & 0.959 & 0.084 \\ 0 & 0.001 & -0.0026 \end{array} \right)$$

Mithilfe der Gauß-Elimination haben wir die Matrix A auf eine Dreiecksgestalt transformiert. Die rechte Seite b wurde entsprechend modifiziert. Das resultierende Dreieckssystem kann nun sehr einfach durch *Rückwärtseinsetzen* gelöst werden (bei dreistelliger Rechnung):

$$0.001y = -0.0026 \quad \Rightarrow \quad y = -2.6$$

$$0.988x = 0.087 - 0.959 \cdot (-2.6) \approx 2.58 \quad \Rightarrow \quad x = 2.61.$$

Wir erhalten also $(x, y) = (2.61, -2.60)$. Der relative Fehler der numerischen Lösung beträgt somit mehr als 10 %. Die numerische Aufgabe, ein Gleichungssystem zu lösen, scheint also entweder generell sehr schlecht konditioniert zu sein (siehe Kap. 1), oder aber das Eliminationsverfahren zur Lösung eines linearen Gleichungssystems ist numerisch sehr instabil und nicht gut geeignet. Der Frage nach der Konditionierung und Stabilität gehen wir im folgenden Abschnitt auf den Grund.

3.1 Störungstheorie und Stabilitätsanalyse von linearen Gleichungssystemen

Zu einer quadratischen regulären Matrix $A \in \mathbb{R}^{n \times n}$ sowie einem Vektor $b \in \mathbb{R}^n$ betrachten wir das lineare Gleichungssystem

$$Ax = b.$$

Durch numerische Fehler, Rundungsfehler, Eingabefehler oder durch Messungenauigkeiten liegen sowohl A als auch b nur gestört vor:

$$\tilde{A}\tilde{x} = \tilde{b}$$

Dabei sei $\tilde{A} = A + \delta A$ sowie $\tilde{b} = b + \delta b$ mit den Störungen δA sowie δb .

Wir kommen nun zur Kernaussage dieses Abschnitts und wollen die Fehlerverstärkung beim Lösen von linearen Gleichungssystemen betrachten. Fehler können dabei sowohl in der Matrix A als auch in der rechten Seite b auftauchen. Wir betrachten zunächst Störungen der rechten Seite.

Satz 3.1 (Störung der rechten Seite) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix, $b \in \mathbb{R}^n$. Durch $x \in \mathbb{R}^n$ sei die Lösung des linearen Gleichungssystems $Ax = b$ gegeben. Es sei δb eine Störung der rechten Seite $\tilde{b} = b + \delta b$ und $\tilde{x}$ die Lösung des gestörten Gleichungssystems $A\tilde{x} = \tilde{b}$. Weiter sei $\|\cdot\|$ eine mit einer Vektornorm $\|\cdot\|$ verträgliche Matrixnorm. Dann gilt:

$$\frac{\|\delta x\|}{\|x\|} \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|},$$

mit der *Konditionszahl* der Matrix

$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|.$$

Beweis Es sei $\|\cdot\|$ eine beliebige Matrixnorm mit verträglicher Vektornorm $\|\cdot\|$. Für die Lösung $x \in \mathbb{R}^n$ und gestörte Lösung $\tilde{x} \in \mathbb{R}^n$ gilt:

$$\tilde{x} - x = A^{-1}(A\tilde{x} - Ax) = A^{-1}(\tilde{b} - b) = A^{-1}\delta b$$

Also:

$$\frac{\|\delta x\|}{\|x\|} \leq \|A^{-1}\| \frac{\|\delta b\|}{\|x\|} \cdot \frac{\|x\|}{\|b\|} = \|A^{-1}\| \frac{\|\delta b\|}{\|b\|} \cdot \frac{\|Ax\|}{\|x\|} \leq \underbrace{\|A\| \cdot \|A^{-1}\|}_{=: \text{cond}(A)} \frac{\|\delta b\|}{\|b\|}$$

□

Bemerkung 3.2 (Konditionszahl einer Matrix) Die *Konditionszahl* einer Matrix spielt die entscheidende Rolle in der numerischen linearen Algebra. Betrachten wir etwa die Konditionierung der Matrix-Vektor-Multiplikation $y = Ax$, so erhalten wir bei gestörter Eingabe $\tilde{x} = x + \delta x$ wieder

$$\frac{\|\delta y\|}{\|y\|} \leq \text{cond}(A) \frac{\|\delta x\|}{\|x\|}.$$

Die Konditionszahl einer Matrix hängt von der gewählten Norm ab. Da jedoch alle Matrixnormen im $\mathbb{R}^{n \times n}$ äquivalent sind, überträgt sich diese Eigenschaft auf die entsprechenden Konditionsbegriffe. Mit Satz 2.17 folgern wir für symmetrische Matrizen für den Spezialfall $\text{cond}_2(A)$:

$$\text{cond}_2(A) = \|A\|_2 \cdot \|A^{-1}\|_2 = \frac{\max\{|\lambda|, \lambda \text{ Eigenwert von } A\}}{\min\{|\lambda|, \lambda \text{ Eigenwert von } A\}}.$$

◆

Wir betrachten nun den Fall, dass die Matrix A eines linearen Gleichungssystems mit einer Störung δA versehen ist. Es stellt sich zunächst die Frage, ob die gestörte Matrix $\tilde{A} = A + \delta A$ überhaupt noch regulär ist.

Hilfssatz 3.3 Es sei durch $\|\cdot\|$ eine von der Vektornorm induzierte Matrixnorm gegeben. Weiter sei $B \in \mathbb{R}^{n \times n}$ eine Matrix mit $\|B\| < 1$. Dann ist die Matrix $I + B$ regulär und es gilt die Abschätzung

$$\|(I + B)^{-1}\| \leq \frac{1}{1 - \|B\|}.$$

Beweis Es gilt

$$\|(I + B)x\| \geq \|x\| - \|Bx\| \geq (1 - \|B\|)\|x\|.$$

Da $1 - \|B\| > 0$, ist durch $I + B$ eine injektive Abbildung gegeben. Also ist $I + B$ eine reguläre Matrix. Weiter gilt:

$$\begin{aligned} 1 &= \|I\| = \|(I + B)(I + B)^{-1}\| = \|(I + B)^{-1} + B(I + B)^{-1}\| \\ &\geq \|(I + B)^{-1}\| - \|B\| \|(I + B)^{-1}\| = \|(I + B)^{-1}\| (1 - \|B\|) > 0. \end{aligned}$$

□

Mit diesem Hilfssatz können wir im Folgenden auch auf die Störung der Matrix eingehen.

Satz 3.4 (Störung der Matrix) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix, $b \in \mathbb{R}^n$. Weiter sei $x \in \mathbb{R}^n$ die Lösung des linearen Gleichungssystems $Ax = b$ und sei $\tilde{A} = A + \delta A$ eine gestörte Matrix mit $\|\delta A\| \leq \|A^{-1}\|^{-1}$. Für die gestörte Lösung $\tilde{x} = x + \delta x$ von $\tilde{A}\tilde{x} = b$ gilt:

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \text{cond}(A)\|\delta A\|/\|A\|} \frac{\|\delta A\|}{\|A\|}.$$

Beweis Für die Lösung x sowie die gestörte Lösung $\tilde{x}$ und den Fehler $\delta x := \tilde{x} - x$ gilt

$$\begin{aligned} (A + \delta A)\tilde{x} &= b \\ (A + \delta A)x &= b + \delta Ax \end{aligned} \quad \Rightarrow \quad \delta x = -[A + \delta A]^{-1} \delta Ax.$$

Da laut Voraussetzung $\|A^{-1}\delta A\| \leq \|A^{-1}\| \|\delta A\| < 1$, folgt mit Hilfssatz 3.3:

$$\begin{aligned} \|\delta x\| &\leq \|[I + A^{-1}\delta A]^{-1}A^{-1}\delta A\| \|x\| \leq \frac{\|A^{-1}\|}{1 - \|A^{-1}\delta A\|} \|\delta A\| \|x\| \\ &\leq \frac{\text{cond}_2(A)}{1 - \|A^{-1}\| \|\delta A\|} \frac{\|\delta A\|}{\|A\|} \|x\| \end{aligned}$$

Das Ergebnis erhalten wir durch Erweitern mit $\|A\|/\|A\|$. □

Diese beiden Störungssätze können einfach kombiniert werden, um gleichzeitig die Störung durch rechte Seite und Matrix abschätzen zu können.

Satz 3.5 (Störungssatz für lineare Gleichungssysteme) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix, $b \in \mathbb{R}^n$ die rechte Seite. Weiter sei $x \in \mathbb{R}^n$ die Lösung des linearen Gleichungssystems $Ax = b$. Für die Lösung $\tilde{x} \in \mathbb{R}^n$ des gestörten Systems $\tilde{A}\tilde{x} = \tilde{b}$ mit Störungen $\delta b = \tilde{b} - b$ und $\delta A = \tilde{A} - A$ gilt unter der Voraussetzung

$$\|\delta A\| < \frac{1}{\|A^{-1}\|}$$

die Abschätzung

$$\frac{\|\delta x\|}{\|x\|} \leq \frac{\text{cond}(A)}{1 - \text{cond}(A)\|\delta A\|/\|A\|} \left(\frac{\|\delta b\|}{\|b\|} + \frac{\|\delta A\|}{\|A\|} \right)$$

mit der *Konditionszahl*

$$\text{cond}(A) = \|A\| \|A^{-1}\|.$$

Beweis Wir kombinieren die Aussagen von Satz 3.1 und 3.4. Hierzu sei x die Lösung von $Ax = b$, $\tilde{x}$ die gestörte Lösung $\tilde{A}\tilde{x} = \tilde{b}$ und $\hat{x}$ die Lösung zu gestörter rechter Seite $A\hat{x} = \tilde{b}$. Dann gilt:

$$\|x - \tilde{x}\| \leq \|x - \hat{x}\| + \|\hat{x} - \tilde{x}\| \leq \text{cond}(A) \frac{\|\delta b\|}{\|b\|} \|x\| + \frac{\text{cond}(A)}{1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}} \frac{\|\delta A\|}{\|A\|} \|x\|.$$

Beachte $\|\delta A\| < \|A^{-1}\|^{-1}$, dann

$$0 \leq \text{cond}(A) \frac{\|\delta A\|}{\|A\|} < \|A\| \|A^{-1}\| \frac{1}{\|A\| \|A^{-1}\|} = 1.$$

Also gilt

$$\text{cond}(A) \leq \frac{\text{cond}(A)}{1 - \text{cond}(A) \frac{\|\delta A\|}{\|A\|}}.$$

Damit folgt die Aussage. □

Mit diesem Ergebnis kehren wir zum einführenden Beispiel aus Kap. 3 zurück:

$$A = \begin{pmatrix} 0.988 & 0.959 \\ 0.992 & 0.963 \end{pmatrix} \Rightarrow A^{-1} \approx \begin{pmatrix} 8302 & -8267 \\ -8552 & 8517 \end{pmatrix}$$

In der maximalen Zeilensummennorm $\|\cdot\|_\infty$ gilt:

$$\|A\|_\infty = 1.955, \quad \|A^{-1}\|_\infty \approx 17069, \quad \text{cond}_\infty(A) \approx 33370.$$

Das Lösen eines linearen Gleichungssystems mit der Matrix A ist also äußerst schlecht konditioniert, falls die Matrix eine große Konditionszahl hat. Hieraus resultiert der enorme Rundungsfehler im Beispiel zu Beginn des Kapitels. Wir halten fest: Bei großer Kondi-

tionszahl ist der Einfluss von Rundungsfehlern sehr groß. Der große Fehler ist immanent mit der Aufgabe verbunden und nicht unbedingt auf ein Stabilitätsproblem des Verfahrens zurückzuführen.

3.2 Das Gaußsche Eliminationsverfahren und die LR-Zerlegung

Das wichtigste Verfahren zum Lösen eines linearen Gleichungssystems $Ax = b$ mit quadratischer Matrix $A \in \mathbb{R}^{n \times n}$ ist das Gaußsche Eliminationsverfahren. Die um die rechte Seite b erweiterte Matrix $(A|b)$ wird durch schrittweise Elimination zunächst auf Dreiecksgestalt gebracht. Hierzu wird im i -ten Schritt das Vielfache der i -ten Zeile zu den Zeilen $j > i$ addiert mit dem Ziel, dass die Elemente unterhalb der Diagonale, also a_{ji} zu Null eliminiert werden:

$$\left(\begin{array}{cccc|ccc} * & * & * & \cdots & * & * & * \\ * & * & * & & * & * & * \\ * & * & * & & * & * & * \\ \vdots & & & \ddots & \vdots & \vdots & \vdots \\ * & * & * & \cdots & * & * & * \end{array} \right) \rightarrow \left(\begin{array}{cccc|ccc} * & * & * & \cdots & * & * & * \\ 0 & * & * & & * & * & * \\ 0 & * & * & & * & * & * \\ \vdots & & & \ddots & \vdots & \vdots & \vdots \\ 0 & * & * & \cdots & * & * & * \end{array} \right) \rightarrow \dots \rightarrow \left(\begin{array}{cccc|ccc} * & * & * & \cdots & * & * & * \\ 0 & * & * & & * & * & * \\ 0 & 0 & * & & * & * & * \\ \vdots & & & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & \cdots & 0 & * & * & * \end{array} \right)$$

Im Anschluss folgt die Rückwärts-Elimination. Beginnend mit $i = n, n-1, \dots$ addieren wir Vielfache der i -ten Zeile zu allen Zeilen $j < i$, diesmal mit dem Ziel die Elemente oberhalb der Diagonalen a_{ji} auf Null zu bringen:

$$\left(\begin{array}{cccc|ccc} * & * & * & \cdots & * & * & * \\ 0 & * & * & & * & * & * \\ \vdots & \ddots & \ddots & & \vdots & \vdots & \vdots \\ 0 & & 0 & * & * & * & * \\ 0 & \cdots & 0 & 0 & * & * & * \end{array} \right) \rightarrow \left(\begin{array}{cccc|ccc} * & * & \cdots & 0 & * & * & * \\ 0 & * & \cdots & 0 & * & * & * \\ \vdots & \ddots & \ddots & & \vdots & \vdots & \vdots \\ 0 & & 0 & 0 & * & * & * \\ 0 & \cdots & 0 & 0 & * & * & * \end{array} \right) \rightarrow \dots \rightarrow \left(\begin{array}{cccc|ccc} * & 0 & \cdots & 0 & * & * & * \\ 0 & * & 0 & & 0 & * & * \\ \vdots & \ddots & \ddots & \ddots & \vdots & \vdots & \vdots \\ 0 & & & * & 0 & * & * \\ 0 & 0 & \cdots & 0 & * & * & * \end{array} \right)$$

Es resultiert im linken $n \times n$ -Block eine Diagonalmatrix und das Ergebnis kann nach Division durch die Diagonale abgelesen werden.

Die LR -Zerlegung baut auf der Gauss-Elimination auf. Der wesentliche Unterschied ist, dass die Transformation der Matrix und das Lösen des linearen Gleichungssystems in zwei Schritte geteilt werden. Zunächst erstellen wir eine multiplikative Zerlegung $A = LR$ und erst im Anschluss wird das eigentliche lineare Gleichungssystem gelöst.

Wir beginnen mit der Zerlegung der Matrix in die beiden Faktoren L und R . Der Algorithmus ist iterativ aufgebaut. Im i -ten Schritt werden die Elemente a_{ki} für $k > i$ eliminiert und die Einträge $a_{kl}^{(i-1)}$ von $A^{(i-1)}$ werden zu den Einträgen $a_{kl}^{(i)}$ von $A^{(i)}$ transformiert mittels

$$a_{kl}^{(i)} = a_{kl}^{(i-1)} - \frac{a_{ki}^{(i-1)}}{a_{ii}^{(i-1)}} a_{il}^{(i-1)} \quad k, l = i+1, \dots, n.$$

Wir schreiben diesen Schritt kompakt in Form einer Matrix-Vektor Multiplikation.

Satz 3.6 (Eliminationsschritt der LR-Zerlegung) Es sei $A^{(i-1)} \in \mathbb{R}^{n \times n}$ eine Matrix mit teilweiser Diagonalgestalt von Stufe $i - 1$, d. h.

$$a_{kl}^{(i-1)} = 0 \text{ für } l = 1, \dots, i - 1 \text{ und } k = l + 1, \dots, n,$$

sowie mit *Pivot-Element* $a_{ii}^{(i-1)} \neq 0$. Dann hat die Matrix

$$A^{(i)} = F^{(i)} A^{(i-1)}$$

teilweise Diagonalgestalt mit Stufe i , d. h.

$$a_{kl}^{(i)} = 0 \text{ für } l = 1, \dots, i \text{ und } k = l + 1, \dots, n,$$

wobei $F^{(i)}$ die *Frobenius-Matrix* mit

$$F^{(i)} := \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -g_{i+1}^{(i)} & \ddots & \\ & & \vdots & & \ddots \\ & & -g_n^{(i)} & & 1 \end{pmatrix}, \quad g_k^{(i)} := \frac{a_{ki}^{(i-1)}}{a_{ii}^{(i-1)}}$$

ist.

Beweis Der Beweis erfolgt im Wesentlichen durch Nachrechnen. Zunächst gilt $a_{ii}^{(i-1)} \neq 0$, so dass die $g_k^{(i-1)}$ und damit ganz $F^{(i)}$ wohl definiert sind. Rechnen wir das Matrix-Matrix Produkt elementweise aus und nutzen die spezielle Form der Frobenius-Matrix, so erhalten wir

$$\begin{aligned} [F^{(i)} A^{(i-1)}]_{kl} &= \sum_{j=1}^n F_{kj}^{(i)} a_{jl}^{(i-1)} \\ &= a_{kl}^{(i-1)} - \begin{cases} 0 & k \leq i \\ g_k^{(i)} a_{il}^{(i-1)} & k > i \end{cases} \\ &= a_{kl}^{(i-1)} - \begin{cases} 0 & k \leq i \\ \frac{a_{ki}^{(i-1)}}{a_{ii}^{(i-1)}} a_{il}^{(i-1)} & k > i \end{cases} \\ &= \begin{cases} a_{kl}^{(i-1)} & k \leq i \\ 0 & k > i, l \leq i \\ a_{kl}^{(i-1)} - \frac{a_{ki}^{(i-1)} a_{il}^{(i-1)}}{a_{ii}^{(i-1)}} & k > i, l > i. \end{cases} \end{aligned}$$

Die Multiplikation mit $F^{(i)}$ von links lässt die ersten i Zeilen unverändert. Ebenso bleiben die ersten $i - 1$ Spalten unverändert. Die i -te Spalte unterhalb der Diagonalen wird zu Null gesetzt, so dass die resultierende Matrix teilweise Diagonalform in den ersten i Zeilen und Spalten hat. Der verbleibende Block $k, l = i + 1, \dots, n$ erhält neue Werte. $\square$

Die Transformation der Matrix A auf obere rechte Dreiecksgestalt erfolgt jetzt einfach durch wiederholte Anwendung dieses Satzes, d. h. nach Multiplikation mit $F^{(1)}, F^{(2)}, \dots, F^{(n-1)}$

$$R := A^{(n-1)} = F^{(n-1)} F^{(n-2)} \dots F^{(1)} A. \quad (3.1)$$

Falls alle *Pivot-Elemente* der im Laufe der Sequenz auftretenden Matrizen $A^{(i)} = F^{(i)} F^{(i-1)} \dots F^{(1)} A$ ungleich Null sind, d. h. $a_{ii}^{(i-1)} \neq 0$, so sind die Frobenius-Matrizen $F^{(i)}$ wohl definiert und Satz 3.6 ist anwendbar und zeigt, dass R rechte obere Dreiecksgestalt hat. Zur Identifikation der Matrix L betrachten wir die Frobenius-Matrizen genauer.

Satz 3.7 (Frobenius-Matrix) Es sei $F^{(i)} \in \mathbb{R}^{n \times n}$ eine Frobenius-Matrix der Form

$$F^{(i)} := \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & -g_{i+1} & \ddots & & \\ & & \vdots & & \ddots & \\ & & -g_n & & & 1 \end{pmatrix},$$

wobei $g_i \in \mathbb{R}$. Jede Frobenius-Matrix $F^{(i)} \in \mathbb{R}^{n \times n}$ ist regulär und für ihre Inverse gilt:

$$[F^{(i)}]^{-1} := \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & g_{i+1} & \ddots & & \\ & & \vdots & & \ddots & \\ & & g_n & & & 1 \end{pmatrix}.$$

Für zwei Frobenius-Matrizen $F^{(i_1)}$ und $F^{(i_2)}$ mit $i_1 < i_2$ gilt:

$$F^{(i_1)} F^{(i_2)} = F^{(i_1)} + F^{(i_2)} - I = \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & -g_{i_1+1}^{(i_1)} & 1 & & \\ & & \vdots & & \ddots & \\ & & \vdots & & & 1 \\ & & \vdots & & & & -g_{i_2+1}^{(i_2)} & \ddots & \\ & & \vdots & & & & \vdots & & \ddots \\ & & -g_n^{(i_1)} & & & & -g_n^{(i_2)} & & 1 \end{pmatrix}.$$

Beweis Dies folgt durch komponentenweises Nachrechnen der Produkte $F^{(i)}[F^{(i)}]^{-1}$ sowie $F^{(i_1)}F^{(i_2)}$. Wir zeigen exemplarisch den zweiten Teil. Hierzu führen wir zunächst die Matrizen $\hat{F}^{(i)} = F^{(i)} - I$ ein und schreiben das Produkt als

$$F^{(i_1)}F^{(i_2)} = I + \hat{F}^{(i_1)} + \hat{F}^{(i_2)} + \hat{F}^{(i_1)}\hat{F}^{(i_2)}.$$

Für das Produkt folgt komponentenweise mit der speziellen Struktur der $\hat{F}^{(i_1)}$ und $\hat{F}^{(i_2)}$ und wegen $i_1 < i_2$

$$[\hat{F}^{(i_1)}\hat{F}^{(i_2)}]_{kl} = \sum_{r=1}^n \hat{F}_{kr}^{(i_1)}\hat{F}_{rl}^{(i_2)} = 0,$$

da $F_{kr}^{(i_1)} \neq 0$ nur für $r = i_1$ und $F_{rl}^{(i_2)} \neq 0$ nur für $r > l = i_2$ möglich ist. $\square$

Das Produkt von Frobenius-Matrizen ist nicht kommutativ und auch die Form bleibt nicht erhalten. D.h., der zweite Teil des Satzes gilt nicht im Fall $i_1 > i_2$. Allerdings kann der Satz einfach auf die Situation $1 \leq i_1 < i_2 < \dots < i_r < n$ verallgemeinert werden und es gilt

$$\prod_{j=1}^r F^{(i_j)} = \sum_{j=1}^r F^{(i_j)} - (r-1)I. \quad (3.2)$$

Wir führen nun (3.1) fort und multiplizieren nacheinander mit den Inversen von $F^{(n-1)}, F^{(n-2)}, \dots$ und erhalten

$$\begin{aligned} R &:= A^{(n-1)} = F^{(n-1)}F^{(n-2)} \dots F^{(1)}A \\ \Leftrightarrow [F^{(1)}]^{-1}[F^{(2)}]^{-1} \dots [F^{(n-1)}]^{-1}R &= A. \end{aligned} \quad (3.3)$$

Die Faktoren $[F^{(i)}]^{-1}$ sind laut Satz 3.7 selbst alle Frobenius-Matrizen und mit (3.2) ergibt sich, dass das Produkt

$$L := \prod_{j=1}^{n-1} [F^{(j)}]^{-1} = \sum_{j=1}^{n-1} [F^{(j)}]^{-1} - (n-2)I \quad (3.4)$$

eine reguläre Matrix und genauer eine linke untere Dreiecksmatrix ist mit Einsen auf der Diagonale ist. Es gilt somit $A = LR$ und wir haben die gesuchte Faktorisierung einer Matrix A in eine untere linke Dreiecksmatrix (mit Einsen auf der Diagonale) und eine obere rechte Dreiecksmatrix hergeleitet.

Satz 3.8 (LR-Zerlegung) Es sei $A \in \mathbb{R}^{n \times n}$ eine quadratische, reguläre Matrix. Angenommen, alle bei der Elimination auftretenden Pivot-Elemente $a_{ii}^{(i-1)}$ seien ungleich null. Dann existiert die eindeutig bestimmte LR-Zerlegung in eine rechte obere reguläre Dreiecksmatrix $R \in \mathbb{R}^{n \times n}$ sowie in eine linke untere reguläre Dreiecksmatrix $L \in \mathbb{R}^{n \times n}$ mit Diagonaleinträgen 1. Der Aufwand zur Durchführung der LR-Zerlegung beträgt

$$\frac{1}{3}n^3 + O(n)$$

elementare Operationen, d. h. Multiplikationen und Additionen.

Beweis (i) *Eindeutigkeit.* Angenommen, es existieren zwei LR-Zerlegungen

$$A = L_1 R_1 = L_2 R_2 \quad \Leftrightarrow \quad L_2^{-1} L_1 = R_2 R_1^{-1}.$$

Das Produkt von Dreiecksmatrizen ist wieder eine Dreiecksmatrix, also müssen beide Produkte Diagonalmatrizen sein. Das Produkt $L_2^{-1} L_1$ hat nur Einsen auf der Diagonale, also folgt

$$L_2^{-1} L_1 = R_2 R_1^{-1} = I,$$

und somit $L_1 = L_2$ und $R_1 = R_2$.

(ii) *Durchführbarkeit.* Jeder Schritt der Elimination ist durchführbar, solange nicht durch $a_{ii}^{(i-1)} = 0$ geteilt werden muss. Die Matrix F ist per Konstruktion regulär und somit existiert auch die Inverse L .

(iii) *Aufwand.* Im i -ten Eliminationsschritt

$$A^{(i)} = F^{(i)} A^{(i-1)}$$

sind zunächst $n - i$ arithmetische Operationen zur Berechnung der $g_j^{(i)}$ für $j = i + 1, \dots, n$ notwendig. Die Matrix-Matrix-Multiplikation betrifft nur alle Elemente a_{kl} mit $k > i$ sowie $l > i$. Es gilt

$$a_{kl}^{(i)} = a_{kl}^{(i-1)} - g_k^{(i)} a_{il}^{(i-1)}, \quad k, l = i + 1, \dots, n.$$

Hierfür sind $(n - i)^2$ arithmetische in $g_k^{(i)} a_{il}^{(i-1)}$ Operationen notwendig und $(n - i)$ anschließende Subtraktionen zwischen $a_{kl}^{(i-1)} - g_k^{(i)} a_{il}^{(i-1)}$. Insgesamt summiert sich der Aufwand

in den $n - 1$ Schritten zu:

$$N_{LR}(n) = \sum_{i=1}^{n-1} (n - i + (n - i)^2) = \sum_{i=1}^{n-1} i + i^2,$$

und mit den bekannten Summenformeln folgt:

$$N_{LR}(n) = \frac{n^3}{3} - \frac{n}{3}$$

□

Die Komplexität der LR-Zerlegung ist also $O(n^3)$. Verdoppeln wir die Matrixgröße n , so steigt der Aufwand asymptotisch um den Faktor 8. Verzehnfachen wir die Größe der Matrix, so steigt der Aufwand um den Faktor 1000. Ist die Matrix A einmal zerlegt, so können lineare Gleichungssysteme $Ax = b$ im Nachgang einfach gelöst werden:

Algorithmus 3.1: Lösen von linearen Gleichungssystemen mit der LR-Zerlegung

Eingabe: Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix, für welche die LR-Zerlegung existiert und $b \in \mathbb{R}^n$ eine gegebene rechte Seite.

1 Erstelle LR-Zerlegung $A = LR$

2 Vorwärtseinsetzen: $Ly = b$

3 Rückwärtseinsetzen: $Rx = y$

Ergebnis: Die Lösung $x \in \mathbb{R}^n$ des System $Ax = b$.

Setzen wir $y = Rx$ in die Gleichung $Ly = b$ ein, so zeigt sich mit $Ly = LRx = Ax = b$, dass die beiden Schritte wirklich die Lösung des linearen Gleichungssystems ergeben. Die Lösungsschritte in Zeilen 2 und 3 des Algorithmus werden *Vorwärtseinsetzen* und *Rückwärtseinsetzen* genannt, dass Gleichungssysteme mit Dreiecksmatrizen direkt aufgelöst werden können.

Algorithmus 3.2: Rückwärtseinsetzen

Eingabe: Es sei $R \in \mathbb{R}^{n \times n}$ eine reguläre rechte obere Dreiecksmatrix, d. h. $r_{ii} \neq 0$ und $b \in \mathbb{R}^n$ eine gegebene rechte Seite.

1 Setze $x_n = r_{nn}^{-1} b_n$.

2 **Für** $i = n - 1$ **bis** 1 **tue**

3 $x_i = r_{ii}^{-1} \left(b_i - \sum_{j=i+1}^n r_{ij} x_j \right)$

Ergebnis: Die Lösung $x \in \mathbb{R}^n$ des oberen Dreieckssystems $Rx = b$.

In Python kann das Rückwärtseinsetzen dann zusammen mit der Bibliothek `numpy` folgendermaßen implementiert werden und steht in den jupyter-Notebooks³ zur Verfügung. Dabei ist `R` ein `numpy.array` der Größe $n \times n$ und `b` ein `numpy.array` der Länge n .

³ github.com/sn-code-inside/EinfNumMath-RWW

♣ Implementierung 3.1: Rückwärtseinsetzen

```

1 def rueckwaerts_einsetzen(R, b):
2     n = b.shape[0]
3     x = np.empty_like(b)
4     x[n - 1] = b[n - 1] / R[n - 1, n - 1]
5     for i in range(n - 2, -1, -1):
6         xr = 0
7         for j in range(i + 1, n):
8             xr += R[i, j] * x[j]
9         x[i] = (b[i] - xr) / R[i, i]
10    return x

```

Es gilt:

Satz 3.9 (Rückwärtseinsetzen) Es sei $R \in \mathbb{R}^{n \times n}$ eine rechte obere Dreiecksmatrix mit $r_{ii} \neq 0$. Dann ist die Matrix R regulär und das Rückwärtseinsetzen erfordert

$$N_R(n) = \frac{n^2}{2} + O(n)$$

elementare Operationen.

Beweis Es gilt $\det(R) = \prod r_{ii} \neq 0$. Also ist die Matrix R regulär.

Jeder Schritt des Rückwärtseinsetzens besteht aus Additionen, Multiplikationen und Division durch die Diagonalelemente. Bei $r_{ii} \neq 0$ ist jeder Schritt durchführbar.

Zur Berechnung von x_i sind $n - i$ Multiplikationen und Additionen notwendig. Hinzu kommt eine Division pro Schritt. Dies ergibt

$$n + \sum_{i=1}^{n-1} (n - i) = n + (n - 1)n - \frac{(n - 1)n}{2} = \frac{n^2}{2} + \frac{n}{2}. \quad \square$$

Die Vorwärtselimination läuft entsprechend des Rückwärtseinsetzens in Algorithmus 3.2 und gemäß Satz 3.9 benötigt sie $O(n^2)$ Operationen. Das eigentliche Lösen eines linearen Gleichungssystems ist also weit weniger aufwendig als das Erstellen der Zerlegung. In vielen Anwendungsproblemen, etwa bei der Diskretisierung von parabolischen Differentialgleichungen, müssen sehr viele Gleichungssysteme mit unterschiedlichen rechten Seiten, aber identischen Matrizen hintereinander gelöst werden. Hier bietet es sich an, die Zerlegung nur einmal zu erstellen und dann wiederholt anzuwenden.

Bemerkung 3.10 (Praktische Aspekte) Die Matrix L ist eine linke untere Dreiecksmatrix mit Einsen auf der Diagonale. Die bekannten Diagonalelemente müssen demnach nicht

gespeichert werden. Ebenso müssen die Nullelemente der Matrizen $A^{(i)}$ unterhalb der Diagonale nicht gespeichert werden. Es bietet sich an, die Matrizen L und R in der gleichen quadratischen Matrix zu speichern. In Schritt i gilt dann:

$$\tilde{A}^{(i)} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & \cdots & \cdots & a_{1n} \\ \textbf{l}_{21} & a_{22}^{(1)} & a_{23}^{(1)} & & & & \vdots \\ \textbf{l}_{31} & \textbf{l}_{32} & a_{33}^{(2)} & & \ddots & & \vdots \\ \vdots & & \ddots & \ddots & & & \vdots \\ \vdots & & & \textbf{l}_{i+1,i} & a_{i+1,i+1}^{(i)} & \cdots & a_{i+1,n}^{(i)} \\ \vdots & & & \vdots & & \ddots & \vdots \\ \textbf{l}_{n1} & \textbf{l}_{n2} & \cdots & \textbf{l}_{n,i} & a_{n,i+1}^{(i)} & \cdots & a_{nn}^{(i)} \end{pmatrix}$$

Dabei sind die fett gedruckten Werte die Einträge von L . Die Werte oberhalb der Linie ändern sich im Verlaufe des Verfahrens nicht mehr und bilden bereits die Einträge L sowie R . ♦

Da die LR-Zerlegung *an Ort und Stelle* eine besondere Bedeutung einnimmt, geben wir hier den entsprechenden Algorithmus an.

Algorithmus 3.3: LR-Zerlegung einer Matrix A ohne Zusatzspeicher

Eingabe: Es sei $A = (a_{ij})_{ij} \in \mathbb{R}^{n \times n}$ eine reguläre Matrix, deren LR-Zerlegung existiert.

```

1 Für  $i = 1$  bis  $n$  tue
2   Für  $k = i + 1$  bis  $n$  tue
3      $a_{ki} = a_{ki} / a_{ii}$ 
4   Für  $j = i + 1$  bis  $n$  tue
5      $a_{kj} = a_{kj} - a_{ki} \cdot a_{ij}$ ;
```

Ergebnis: Die Matrix A überschrieben mit ihrer LR-Zerlegung.

Eine Python-Implementierung der LR-Zerlegung ohne Zusatzspeicher, mit `numpy.array`s zum Speichern der Matrizen, kann folgendermaßen aussehen:

♣ Implementierung 3.2: LR-Zerlegung einer Matrix A ohne Zusatzspeicher

```

1 def LR_zerlegung_einfach(A):
2     for i in range(0, n):
3         for k in range(i + 1, n):
4             A[k, i] = A[k, i] / A[i, i]
5         for j in range(i + 1, n):
6             A[k, j] = A[k, j] - A[k, i] * A[i, j]
7     return None
```

Wir merken nochmal an, dass wir jetzt keinen Zugriff mehr auf die ursprüngliche Matrix haben, da diese überschrieben wurde. Zudem müssen wir uns merken, dass die L -Matrix nur Einsen auf der Diagonalen hat, die wir nicht abspeichern.

3.2.1 LR-Zerlegung mit Pivotisierung

Das Element $a_{ii}^{(i-1)}$ wird das *Pivot-Element* genannt. Um die LR-Zerlegung durchführen zu können muss stets $a_{ii}^{(i-1)} \neq 0$ gelten. Dies ist jedoch für reguläre Matrizen nicht zwingend notwendig. Wir betrachten als Beispiel die Matrix

$$A := \begin{pmatrix} 1 & 4 & 2 \\ 2 & 8 & 1 \\ 1 & 2 & 1 \end{pmatrix}.$$

Im ersten Schritt zur Erstellung der LR-Zerlegung ist $a_{11}^{(0)} = 1$ und es gilt:

$$A^{(1)} = F^{(1)}A = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 4 & 2 \\ 2 & 8 & 1 \\ 1 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 4 & 2 \\ 0 & 0 & -3 \\ 0 & -2 & -1 \end{pmatrix}$$

An dieser Stelle bricht der Algorithmus ab, denn es gilt $a_{22}^{(1)} = 0$. Wir könnten den Algorithmus jedoch mit der Wahl $a_{32}^{(1)} = -2$ als neues Pivot-Element weiterführen. Dies geschieht systematisch durch Einführen einer *Pivotisierung*. Im i -ten Schritt des Verfahrens wird zunächst ein geeignetes Pivot-Element a_{ki} in der i -ten Spalte mit $k \geq i$ gesucht. Die k -te und i -te Zeile werden getauscht und die LR-Zerlegung kann weiter durchgeführt werden. Das Tauschen von k -ter und i -ter Zeile erfolgt durch Multiplikation mit einer Pivot-Matrix:

$$P^{ki} := \begin{pmatrix} 1 & & & & & & & & & & \\ & \ddots & & & & & & & & & \\ & & 1 & & & & & & & & \\ & & & 0 & 0 & \dots & 0 & 1 & & & \\ & & & 0 & 1 & & & 0 & & & \\ & & & \vdots & \ddots & & \vdots & & & & \\ & & & 0 & & & 1 & 0 & & & \\ & & & 1 & 0 & \dots & 0 & 0 & & & \\ & & & & & & & & 1 & & \\ & & & & & & & & & \ddots & \\ & & & & & & & & & & 1 \end{pmatrix}$$

Es gilt $P_{jj}^{ki} = 1$ für $j \neq k$ und $j \neq i$ sowie $P_{ki}^{ki} = P_{ik}^{ki} = 1$, alle anderen Elemente sind null. Wir fassen einige Eigenschaften von P zusammen:

Satz 3.11 (Pivot-Matrizen) Es sei $P = P^{ki}$ die Pivot-Matrix mit $P_{jj}^{ki} = 1$ für $j \neq k, i$ und $P_{ki}^{ki} = P_{ik}^{ki} = 1$. Die Anwendung $P^{ki} A$ von links tauscht k -te und i -te Zeile von A , die Anwendung $A P^{ki}$ von rechts tauscht k -te und i -te Spalte. Es gilt:

$$P^2 = I \text{ und somit } P^{-1} = P.$$

Beweis Übung. □

Algorithmus 3.4: Pivot-Suche in Schritt $(i-1) \mapsto (i)$

Eingabe: Es sei $A^{(i-1)} = (a_{kl}^{(i-1)})_{kl} \in \mathbb{R}^{n \times n}$ eine reguläre Matrix.

- 1 Suche Index $j \geq i$, sodass $|a_{ji}| = \max_{l \geq i} |a_{li}|$.
- 2 Setze $P^{(i)} := P^{ji}$.

Ergebnis: Pivot-Matrix $P^{(i)}$

Im Anschluss bestimmen wir $A^{(i)}$ als

$$A^{(i)} = F^{(i)} P^{(i)} A^{(i-1)}.$$

Die Pivotalisierung sorgt dafür, dass alle Elemente $g_k^{(i)} = a_{ki}^{(i-1)} / a_{ii}^{(i-1)}$ von $F^{(i)}$ im Betrag durch 1 beschränkt sind. Insgesamt erhalten wir die Zerlegung:

$$R = A^{(n-1)} = F^{(n-1)} P^{(n-1)} \dots F^{(1)} P^{(1)} A = \left(\prod_{i=n-1}^1 F^{(i)} P^{(i)} \right) A. \quad (3.5)$$

Die Pivot-Matrizen kommutieren nicht mit A oder den $F^{(i)}$. Daher ist ein Übergang zur LR-Zerlegung nicht ohne Weiteres möglich. Wir definieren die Matrizen

$$\tilde{P}^{(i+1)} := P^{(n-1)} \dots P^{(i+1)}, \quad \tilde{P}^{(n)} := I,$$

mit $\tilde{P}^{(i)} \tilde{P}^{(i)} = 1$ und fügen dieses Produkt in (3.5) ein:

$$\begin{aligned} R &= \left(\prod_{i=n-1}^1 F^{(i)} \tilde{P}^{(i+1)} \tilde{P}^{(i+1)} P^{(i)} \right) A \\ &= \left(\prod_{i=n-1}^1 F^{(i)} \tilde{P}^{(i+1)} \tilde{P}^{(i)} \right) A = \underbrace{\tilde{P}^{(n)}}_{=I} \left(\prod_{i=n-1}^1 \tilde{P}^{(i+1)} F^{(i)} \tilde{P}^{(i+1)} \right) \tilde{P}^{(1)} A \end{aligned} \quad (3.6)$$

Wir definieren

$$\tilde{F}^{(i)} := \tilde{P}^{(i+1)} F^{(i)} \tilde{P}^{(i+1)} = P^{(n-1)} \dots P^{(i+1)} F^{(i)} P^{(i+1)} \dots P^{(n-1)}. \quad (3.7)$$

Die Matrix $\tilde{F}^{(i)}$ entsteht durch mehrfache Zeilen- und Spaltenvertauschung von $F^{(i)}$. Dabei werden nur Zeilen und Spalten $j > i$ vertauscht. Die Matrix $\tilde{F}^{(i)}$ hat die gleiche Besetzungsstruktur wie $F^{(i)}$ und insbesondere nur Einsen auf der Diagonale. Das heißt, sie ist wieder eine Frobenius-Matrix und Satz 3.7 gilt weiter. Es sind lediglich die Einträge in der i -ten Spalte unterhalb der Diagonale permutiert. Das gleiche gilt für ihre Inverse

$$\tilde{L}^{(i)} := [\tilde{F}^{(i)}]^{-1} = P^{(n-1)} \dots P^{(i+1)} [F^{(i)}]^{-1} P^{(i+1)} \dots P^{(n-1)}. \quad (3.8)$$

Wir können die LR-Zerlegung mit Pivotisierung demnach schreiben als

$$R = \underbrace{\tilde{F}^{(n-1)} \tilde{F}^{(n-2)} \dots \tilde{F}^{(1)}}_{=: \tilde{F}} \underbrace{\tilde{P}^{(1)}}_{=: \tilde{P}} A.$$

Da alle $\tilde{F}^{(i)}$ Frobenius-Matrizen sind ist $\tilde{F}$ eine reguläre untere Dreiecksmatrix mit Einsen auf der Diagonale und mit $\tilde{L} = \tilde{F}^{-1}$ ergibt sich als

$$\tilde{L}R = \tilde{P}A.$$

Beim Erstellen der LR-Zerlegung müssen also nicht nur die $A^{(i)}$, sondern auch die bisher berechneten $L^{(i)}$ permutiert werden.

Satz 3.12 (LR-Zerlegung mit Pivotisierung) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix. Dann existiert immer eine LR-Zerlegung mit Pivotisierung

$$PA = LR,$$

wobei P ein Produkt von Pivot-Matrizen ist, L eine untere Dreiecksmatrix mit Diagonaleinträgen 1 und R eine rechte obere Dreiecksmatrix. Die LR-Zerlegung ohne Pivotisierung $P = I$ ist eindeutig, falls sie existiert.

Beweis Übung. □

Mit der Pivotisierung stellen wir sicher, dass die LR-Zerlegung für jede reguläre Matrix A existiert. Auf der anderen Seite kann durch geeignete Pivotisierung die Stabilität der Zerlegung verbessert werden. Durch Wahl eines Pivot-Elements a_{ki} mit maximalem relativen Betrag (bezogen auf die Zeile) kann die Gefahr der Auslöschung verringert werden.

Beispiel 3.13 (LR-Zerlegung ohne Pivotisierung)

Es sei:

$$A = \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ 1.4 & 1.1 & -0.7 \\ 0.8 & 4.3 & 2.1 \end{pmatrix}, \quad b = \begin{pmatrix} 1.2 \\ -2.1 \\ 0.6 \end{pmatrix},$$

und die Lösung des linearen Gleichungssystems $Ax = b$ ist gegeben durch (Angabe mit fünfstelliger Genauigkeit):

$$x \approx \begin{pmatrix} 0.34995 \\ -0.98023 \\ 2.1595 \end{pmatrix}$$

Für die Matrix A gilt $\text{cond}_\infty(A) = \|A\|_\infty \|A^{-1}\|_\infty \approx 7.2 \cdot 1.2 \approx 8.7$. Die Aufgabe ist gut konditioniert, die Konditionszahl 8.7 lässt eine Verstärkung des Fehlers um etwa eine Stelle erwarten. Wir erstellen zunächst die LR-Zerlegung (dreistellige Rechnung). Dabei schreiben wir die Einträge von L fett gedruckt in die Ergebnismatrix:

$$F^{(1)} = \begin{pmatrix} 1 & 0 & 0 \\ -\frac{1.4}{2.3} & 1 & 0 \\ -\frac{0.8}{2.3} & 0 & 1 \end{pmatrix} \approx \begin{pmatrix} 1 & 0 & 0 \\ -0.609 & 1 & 0 \\ -0.348 & 0 & 1 \end{pmatrix},$$

$$[L^{(1)}, A^{(1)}] \approx \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ \mathbf{0.609} & 0.0038 & -1.31 \\ \mathbf{0.348} & 3.67 & 1.75 \end{pmatrix}.$$

Im zweiten Schritt gilt:

$$F^{(2)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{3.67}{0.0038} & 1 \end{pmatrix} \approx \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -966 & 1 \end{pmatrix},$$

$$[L^{(2)}L^{(1)}, A^{(2)}] \approx \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ \mathbf{0.609} & 0.0038 & -1.31 \\ \mathbf{0.348} & \mathbf{966} & 1270 \end{pmatrix}.$$

Die LR-Zerlegung ergibt sich als

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.609 & 1 & 0 \\ 0.348 & 966 & 1 \end{pmatrix}, \quad R := \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ 0 & 0.0038 & -1.31 \\ 0 & 0 & 1270 \end{pmatrix}.$$

Wir lösen das Gleichungssystem nun durch Vorwärts- und Rückwärtseinsetzen:

$$A\tilde{x} = L \underbrace{R\tilde{x}}_{=y} = b$$

Zunächst gilt

$$\begin{aligned}y_1 &= 1.2, & y_2 &= -2.1 - 0.609 \cdot 1.2 \approx -2.83, \\y_3 &= 0.6 - 0.348 \cdot 1.2 + 966 \cdot 2.83 \approx 2730.\end{aligned}$$

Und schließlich:

$$\begin{aligned}\tilde{x}_3 &= \frac{2730}{1270} \approx 2.15, \\ \tilde{x}_2 &= \frac{-2.83 + 1.31 \cdot 2.15}{0.0038} \approx -3.55, \\ \tilde{x}_1 &= \frac{1.2 + 1.8 \cdot 3.55 - 1 \cdot 2.15}{2.3} \approx 2.37.\end{aligned}$$

Für die Lösung $\tilde{x}$ gilt:

$$\tilde{x} = \begin{pmatrix} 2.37 \\ -3.55 \\ 2.15 \end{pmatrix}, \quad \frac{\|\tilde{x} - x\|_2}{\|x\|_2} \approx 1.4,$$

d. h. einen relativen Fehler von 140 %, obwohl wir nur Rundungsfehler und noch keine gestörte Eingabe betrachtet haben.

Um die Implementierung der LR-Zerlegung zum Lösen des linearen Gleichungssystems in Python nutzen zu können, müssen wir noch das Vorwärtseinsetzen implementieren. Wir gehen vom Fall aus, dass die LR-Zerlegung anstelle der ursprünglichen Matrix A gespeichert wurde und dass die Einsen auf der Diagonale separat berücksichtigt werden müssen.

🔧 Implementierung 3.3: Vorwärtseinsetzen

```
1 def vorwaerts_einsetzen_ohne_diag(L, b):
2     x = np.zeros_like(b)
3     for i in range(0, b.shape[0]):
4         xr = 0
5         for j in range(0, i):
6             xr += L[i, j] * x[j]
7         x[i] = (b[i] - xr)
8     return x
```

Um das Gleichungssystem zu lösen, legen wir die Matrix und die rechte Seite als `numpy.array` an. Um Rundungseffekte deutlich zu sehen, speichern wir die Einträge als `half` Gleitkommazahlen.

```
1 import numpy as np
2 A = np.array([[2.3, 1.8, 1],[1.4, 1.1, -0.7],[0.8, 4.3, 2.1]],
3             ↪ dtype=np.half)
3 b = np.array([1.2, -2.1, 0.6], dtype=np.half)
```

Die einfache LR-Zerlegung ohne Zusatzspeicher ergibt dann

```
1 LR_zerlegung_einfach(A)
2 print('Modifiziertes A =\n', A)
```

```
Modifiziertes A =
[[ 2.301e+00  1.800e+00  1.000e+00]
 [ 6.089e-01  3.906e-03 -1.309e+00]
 [ 3.477e-01  9.410e+02  1.233e+03]]
```

Durch Vorwärts- und Rückwärtseinsetzen erhalten wir

```
1 y = vorwaerts_einsetzen_ohne_diag(A, b)
2 print('y = ', y)
3 x = rueckwaerts_einsetzen(A, y)
4 print('x = ', x)
```

```
y = [ 1.200e+00 -2.830e+00  2.664e+03]
x = [ 0.365 -1.      2.16 ]
```

Zur Kontrolle des Fehlers vergleichen wir die Lösung mit dem Algorithmus, der in der Lineare-Algebra-Bibliothek NumPy [49] implementiert unter Verwendung von double Gleitkommazahlen berechnet:

```
1 A = np.array([[2.3, 1.8, 1],[1.4, 1.1, -0.7],[0.8, 4.3, 2.1]],
    ↪ dtype=np.double)
2 b = np.array([1.2, -2.1, 0.6], dtype=np.double)
3 x_np = np.linalg.solve(A, b)
4
5 print('x_np = ', x_np)
6 print('Relativer Fehler:', np.linalg.norm(x - x_np) /
    ↪ np.linalg.norm(x_np))
```

```
x_np = [ 0.34994583 -0.98022752  2.15953413]
Relativer Fehler: 0.0103672211412733
```

Also auch bei der höheren Genauigkeit von half im Vergleich zu drei signifikanten Stellen, haben wir dennoch einen relativen Fehler von 1 %. ◀

Dieses Negativbeispiel zeigt einmal wieder die schlechte Stabilität der LR-Zerlegung zum Lösen von linearen Gleichungssystemen. Im zweiten Schritt wurde als Pivot-Element mit $a_{22}^{(1)} = 0.0038$ ein Wert nahe bei 0 gewählt. Hierdurch entstehen Werte von sehr unterschied-

licher Größenordnung in den Matrizen L und R . Dies wirkt sich ungünstig auf die weitere Stabilität aus.

Beispiel 3.14 (LR-Zerlegung mit Pivotisierung)

Wir setzen das Beispiel in Schritt 2 fort und suchen zunächst das Pivot-Element:

$$[L^{(1)}, A^{(1)}] = \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ \mathbf{0.609} & 0.0038 & -1.31 \\ \mathbf{0.348} & \boxed{3.67} & 1.75 \end{pmatrix}, \quad P^{(2)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

Also, pivotisiert:

$$[\tilde{L}^{(1)}, \tilde{A}^{(1)}] = \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ \mathbf{0.348} & 3.67 & 1.75 \\ \mathbf{0.609} & 0.0038 & -1.31 \end{pmatrix}.$$

Weiter folgt nun

$$F^{(2)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{0.0038}{3.67} & 1 \end{pmatrix} \approx \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -0.00104 & 1 \end{pmatrix},$$

$$[L^{(2)}\tilde{L}^{(1)}, \tilde{A}^{(2)}] \approx \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ \mathbf{0.348} & 3.67 & 1.75 \\ \mathbf{0.609} & \mathbf{0.00104} & -1.31 \end{pmatrix}.$$

Wir erhalten die Zerlegung $\tilde{L}R = PA$ als

$$\tilde{L} := \begin{pmatrix} 1 & 0 & 0 \\ 0.348 & 1 & 0 \\ 0.609 & 0.00104 & 1 \end{pmatrix}, \quad R := \begin{pmatrix} 2.3 & 1.8 & 1.0 \\ 0 & 3.67 & 1.75 \\ 0 & 0 & -1.31 \end{pmatrix}, \quad P := \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}.$$

Das lineare Gleichungssystem lösen wir in der Form:

$$PAx = \tilde{L} \underbrace{Rx}_{=y} = Pb.$$

Zunächst gilt für die rechte Seite: $\tilde{b} = Pb = (1.2, 0.6, -2.1)^T$ und Vorwärtseinsetzen in $\tilde{L}y = \tilde{b}$ ergibt

$$y_1 = 1.2,$$

$$y_2 = 0.6 - 0.348 \cdot 1.2 \approx 0.182,$$

$$y_3 = -2.1 - 0.609 \cdot 1.2 - 0.00104 \cdot 0.182 \approx -2.83.$$

Als Näherung $\tilde{x}$ erhalten wir

$$\tilde{x} = \begin{pmatrix} 0.350 \\ -0.980 \\ 2.160 \end{pmatrix}$$

mit einem relativen Fehler

$$\frac{\|\tilde{x} - x\|_2}{\|x\|_2} \approx 0.0002,$$

also von nur 0.02 % statt 140 %.

In Python werden wir die Pivot-Matrizen nicht speichern, sondern sowohl die A -Anteile als auch die L -Anteile der im Laufe des Verfahrens auftretenden Matrizen direkt tauschen. Um das System danach per Vorwärts und Rückwärtseinsetzen zu lösen, muss dann noch die rechte Seite b permutiert werden. Hierzu genügt es, in jedem Schritt die Indizes der Pivotisierung zu speichern:

♣ Implementierung 3.4: LR-Zerlegung mit Pivotisierung

```

1 def LR_zerlegung_mit_pivot(A):
2     n = A.shape[0]
3     pivot = []
4     for i in range(0, n):
5         k = i                                # Pivot-Suche
6         for j in range(i, n):
7             if abs(A[j, i]) > abs(A[k, i]):
8                 k = j
9         A[[i, k], :] = A[[k, i], :]          # Permutierung
10        pivot.append([i, k])                 # Pivot merken
11        for k in range(i + 1, n):
12            A[k, i] = A[k, i] / A[i, i]
13            for j in range(i + 1, n):
14                A[k, j] = A[k, j] - A[k, i] * A[i, j]
15    return pivot

```

Wenden wir dies auf das vorherige Beispiel an, erhalten wir bei `half` Gleitkommazahlen

```

1 A = np.array([[2.3, 1.8, 1],[1.4, 1.1, -0.7],[0.8, 4.3, 2.1]],
2              dtype=np.half)
3
4 b = np.array([1.2, -2.1, 0.6], dtype=np.half)
5
6
7 pivot = LR_zerlegung_mit_pivot(A)
8 print('Modifiziertes A =\n', A)
9
10
11 for p in pivot:
12     b[p] = b[[p[1], p[0]]]
13
14 print('P b = ', b)
15
16
17 y = vorwaerts_einsetzen_ohne_diag(A, b)
18 print('y = ', y)

```

```

13 x = ruckwaerts_einsetzen(A, y)
14 print('x = ', x)

```

```

Modifiziertes A =
[[ 2.301e+00  1.800e+00  1.000e+00]
 [ 3.477e-01  3.676e+00  1.752e+00]
 [ 6.089e-01  1.062e-03 -1.311e+00]]
P b = [ 1.2  0.6 -2.1]
y = [ 1.2      0.1829 -2.83 ]
x = [ 0.3494 -0.98    2.16 ]

```

Die LR-Zerlegung mit Pivotisierung ergibt dann den relativen Fehler

```

1 np.linalg.norm(x - x_np) / np.linalg.norm(x_np)

1 0.0003696258527581158

```

nur 0.037 % anstelle von 1 %.



Die Beispiele zeigen, dass die berechnete LR-Zerlegung in praktischer Anwendung natürlich keine echte Zerlegung, sondern aufgrund von Rundungsfehlern nur eine Näherung der Matrix $A \approx LR$ ist. Man kann durch Berechnung von LR leicht die Probe machen und den Fehler $A - LR$ bestimmen.

Die LR-Zerlegung ist eines der wichtigsten direkten Verfahren zum Lösen von linearen Gleichungssystemen. Der Aufwand zur Berechnung der LR-Zerlegung steigt allerdings mit dritter Ordnung $O(n^3)$ sehr schnell. Selbst auf modernen Computern übersteigt die Laufzeit für große Gleichungssysteme schnell eine sinnvolle Grenze, siehe Tab. 3.1. In den folgenden Abschnitten diskutieren wir, wie spezielle Struktureigenschaften der Matrix A ausgenutzt werden können, um die LR-Zerlegung in Spezialfällen stark zu beschleunigen oder um die Stabilität der Zerlegung zu verbessern.

3.2.2 LR-Zerlegung für diagonaldominante Matrizen

Satz 3.12 besagt, dass die LR-Zerlegung für beliebige reguläre Matrizen mit Pivotisierung möglich ist. Es gibt allerdings auch viele Matrizen, bei denen die LR-Zerlegung ohne Pivotisierung stabil durchführbar ist. Im Allgemeinen ist es der Matrix A nicht anzusehen, ob die Zerlegung ohne Pivotisierung möglich ist. Beispiele hierfür sind positiv definite oder *diagonaldominante Matrizen*.

Tab. 3.1 Rechenzeit zum Erstellen der LR-Zerlegung einer Matrix $A \in \mathbb{R}^{n \times n}$ auf einem hypothetischen Rechner mit 10 GigaFLOPS bei optimaler Auslastung

n	Operationen ($\approx \frac{1}{3}n^3$)	Zeit LR-Zerlegung
100	$300 \cdot 10^3$	30 μ s
1000	$300 \cdot 10^6$	30 ms
10000	$300 \cdot 10^9$	30 s
100000	$300 \cdot 10^{12}$	10h
1000000	$300 \cdot 10^{15}$	1 Jahr

► **Definition 3.15 (Diagonaldominanz)** Eine Matrix $A \in \mathbb{R}^{n \times n}$ heißt *diagonaldominant*, falls

$$|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|, \quad i = 1, \dots, n.$$

Eine diagonaldominante Matrix hat das betragsmäßig größte Element auf der Diagonalen, bei regulären Matrizen sind die Diagonalelemente zudem ungleich null.

Satz 3.16 (LR-Zerlegung diagonaldominanter Matrizen) Sei $A \in \mathbb{R}^{n \times n}$ eine reguläre und diagonaldominante Matrix. Dann ist die LR-Zerlegung ohne Pivotisierung durchführbar und alle auftretenden Pivot-Elemente $a_{ii}^{(i-1)}$ sind von null verschieden.

Beweis Wir führen den Beweis über Induktion und zeigen, dass alle Untermatrizen $A_{k,l>i}^{(i)}$ wieder diagonaldominant sind. Für eine diagonaldominante Matrix gilt

$$|a_{11}| \geq \sum_{j>1} |a_{1j}| \geq 0,$$

und, da A regulär ist, auch zwingend $|a_{11}| > 0$. Der erste Schritt der LR-Zerlegung ist somit durchführbar.

Wir zeigen jetzt, dass die Matrix $\tilde{A}$ nach einem Eliminationsschritt eine diagonaldominante Untermatrix $\tilde{A}_{i,j>1}$ hat. Für deren Einträge $\tilde{a}_{ij}$ gilt:

$$\tilde{a}_{ij} = a_{ij} - \frac{a_{i1}a_{1j}}{a_{11}}, \quad i, j = 2, \dots, n$$

Also gilt für die Untermatrix:

$$\begin{aligned}
i = 2, \dots, n : \quad \sum_{j=2, j \neq i}^n |\tilde{a}_{ij}| &\leq \underbrace{\sum_{j=1, j \neq i}^n |a_{ij}| - |a_{i1}|}_{\leq |a_{ii}|} + \underbrace{\frac{|a_{i1}|}{|a_{11}|} \sum_{j=2}^n |a_{1j}| - \frac{|a_{i1}|}{|a_{11}|} |a_{1i}|}_{\leq |a_{11}|} \\
&\leq |a_{ii}| - \frac{|a_{i1}|}{|a_{11}|} |a_{1i}| \leq |a_{ii}| - \frac{a_{i1}}{a_{11}} a_{1i} = |\tilde{a}_{ii}|
\end{aligned}$$

Die resultierende Matrix ist wieder diagonaldominant. $\square$

Die Diagonaldominanz einer Matrix sehr einfach zu überprüfen und daher ein gutes Kriterium, um die Notwendigkeit der Pivotisierung abzuschätzen.

3.3 Die Cholesky-Zerlegung für symmetrisch positiv definite Matrizen

Eine wichtige Klasse von Matrizen sind die positiv definiten Matrizen, siehe Definition 2.13 sowie Satz 2.15. Es zeigt sich, dass für symmetrisch positiv definite Matrizen $A \in \mathbb{R}^{n \times n}$ eine symmetrische Zerlegung $A = \tilde{L}\tilde{L}^T$ in eine untere Dreiecksmatrix $\tilde{L}$ erstellt werden kann. Diese ist immer ohne Pivotisierung durchführbar und wird *Cholesky-Zerlegung* genannt. Der Aufwand zum Erstellen der Cholesky-Zerlegung ist erwartungsgemäß nur halb so groß wie der Aufwand zum Erstellen der LR-Zerlegung.

Satz 3.17 (LR-Zerlegung einer positiv definiten Matrix) Die Matrix $A \in \mathbb{R}^{n \times n}$ sei symmetrisch positiv definit. Dann existiert eine eindeutige LR-Zerlegung ohne Pivotisierung.

Beweis Wir gehen ähnlich vor wie im Beweis zum Satz über die LR-Zerlegung diagonaldominanter Matrizen, Satz 3.16, und führen den Beweis per Induktion. Dazu sei A eine symmetrisch positiv definite Matrix. Ein Schritt der LR-Zerlegung ist durchführbar, da laut Satz 2.15 $a_{11} > 0$ gilt. Wir zeigen, dass die Teilmatrix $\tilde{A}_{i,j>1}$ nach einem Eliminationsschritt wieder symmetrisch positiv definit ist. Es gilt aufgrund der Symmetrie von A :

$$\tilde{a}_{ij} = a_{ij} - \frac{a_{1j}a_{i1}}{a_{11}} = a_{ji} - \frac{a_{1i}a_{j1}}{a_{11}} = \tilde{a}_{ji},$$

d. h., $\tilde{A}_{i,j>1}$ ist symmetrisch.

Nun sei $x \in \mathbb{R}^n$ ein Vektor $x = (x_1, \tilde{x}) \in \mathbb{R}^n$ mit $\tilde{x} = (x_2, \dots, x_n) \in \mathbb{R}^{n-1}$ beliebig. Den Eintrag x_1 werden wir im Laufe des Beweises spezifizieren. Es gilt wegen der positiven Definitheit von A :

$$\begin{aligned}
0 < (Ax, x) &= \sum_{ij} a_{ij} x_i x_j = a_{11} x_1^2 + 2x_1 \sum_{j=2}^n a_{1j} x_j + \sum_{i,j=2}^n a_{ij} x_i x_j \\
&= a_{11} x_1^2 + 2x_1 \sum_{j=2}^n a_{1j} x_j + \sum_{i,j=2}^n \underbrace{\left(a_{ij} - \frac{a_{1j} a_{i1}}{a_{11}} \right)}_{=\tilde{a}_{ij}} x_i x_j + \sum_{i,j=2}^n \frac{a_{1j} a_{i1}}{a_{11}} x_i x_j \\
&= a_{11} \left(x_1^2 + 2x_1 \frac{1}{a_{11}} \sum_{j=2}^n a_{1j} x_j + \frac{1}{a_{11}^2} \sum_{i,j=2}^n a_{1j} a_{i1} x_i x_j \right) + \sum_{i,j=2}^n \tilde{a}_{ij} x_i x_j \\
&= a_{11} \left(x_1 + \frac{1}{a_{11}} \sum_{j=2}^n a_{1j} x_j \right)^2 + (\tilde{A}_{i,j>1} \tilde{x}, \tilde{x})
\end{aligned}$$

Die positive Definitheit von $\tilde{A}_{i,j>1}$ folgt somit bei der Wahl

$$x_1 = -\frac{1}{a_{11}} \sum_{j=2}^n a_{1j} x_j.$$

□

Für eine symmetrisch positiv definite Matrix A ist die LR-Zerlegung immer ohne Pivotisierung durchführbar. Dabei treten nur positive Pivot-Elemente $a_{ii}^{(i-1)}$ auf. Das heißt, die Matrix R hat nur positive Diagonalelemente $r_{ii} > 0$. Es sei $D \in \mathbb{R}^{n \times n}$ die Diagonalmatrix mit $d_{ii} = r_{ii} > 0$. Dann gilt $A = LR = LD\tilde{R}$ mit einer rechten oberen Dreiecksmatrix $\tilde{R} = D^{-1}R$, welche nur Einsen auf der Diagonalen hat. Da A symmetrisch ist, folgt:

$$A = LR = LD\tilde{R} = \tilde{R}^T DL^T = A^T$$

Aufgrund der Eindeutigkeit der LR-Zerlegung gilt $L = \tilde{R}^T$ und $R = DL^T$. Da D nur positive Diagonaleinträge hat, existiert die Matrix $\sqrt{D}$ und wir schreiben:

$$A = LR = LD\tilde{R} = LD^{\frac{1}{2}} D^{\frac{1}{2}} \tilde{R} = \underbrace{LD^{\frac{1}{2}}}_{=: \tilde{L}} D^{-\frac{1}{2}} R.$$

Wegen $L = \tilde{R}^T$ folgt $\tilde{L}^T = D^{\frac{1}{2}} \tilde{R} = D^{-\frac{1}{2}} R$, also $A = \tilde{L} \tilde{L}^T$.

Satz 3.18 (Cholesky-Zerlegung) Es sei $A \in \mathbb{R}^{n \times n}$ eine symmetrisch positiv definite Matrix. Dann existiert die Cholesky-Zerlegung

$$A = \tilde{L} \tilde{L}^T$$

in eine untere linke Dreiecksmatrix $\tilde{L}$. Sie kann ohne Pivotisierung in

$$\frac{n^3}{6} + O(n^2)$$

elementaren Operationen durchgeführt werden.

Anstelle eines Beweises geben wir einen effizienten Algorithmus zur direkten Berechnung der Cholesky-Zerlegung an. Hier kann der notwendige Aufwand leicht aus dem Koeffizientenvergleich bei $A = \tilde{L}^T \tilde{L}$ abgelesen werden:

Algorithmus 3.5: Direkte Berechnung der Cholesky-Zerlegung

Eingabe: Gegeben sei eine symmetrisch positiv definite Matrix $A \in \mathbb{R}^{n \times n}$.

1 **Für** $j = 1$ **bis** n **tue**

2 $l_{11} = \sqrt{a_{11}}$, bzw. $l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$.

3 **Für** $i = j + 1$ **bis** n **tue**

4 $l_{ij} = l_{jj}^{-1} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right)$.

Ergebnis: Die untere Dreiecksmatrix der Cholesky Zerlegung $L = (l_{ij})$, $j \leq i$.

Die Python-Implementierung lautet

♣ Implementierung 3.5: Direkte Berechnung der Cholesky-Zerlegung

```

1 def cholesky(A):
2     n = A.shape[0]
3     L = np.zeros((n, n))
4
5     for j in range(0, n):
6         l = 0
7         for k in range(0, j):
8             l += L[j, k]**2
9         L[j, j] = np.sqrt(A[j, j] - l)
10        for i in range(j + 1, n):
11            l = 0
12            for k in range(j):
13                l += L[i, k] * L[j, k]
14            L[i, j] = (A[i, j] - l) / L[j, j]
15    return L

```

Der Algorithmus kann iterativ aus der Beziehung $\tilde{L} \tilde{L}^T = A$ hergeleitet werden. Es gilt:

$$a_{ij} = \sum_{k=1}^{\min\{i,j\}} l_{ik} l_{jk}$$

Wir gehen Spaltenweise für $j = 1, 2, \dots, n$ vor. Das heißt, $\tilde{L}$ sei für alle Spalten bis $j - 1$ bekannt. Dann gilt in Spalte j zunächst für das Diagonalelement:

$$a_{jj} = \sum_{k=1}^j l_{jk}^2 \Rightarrow l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$$

Ist das j -te Diagonalelement l_{jj} bekannt, so gilt für $i > j$, dass

$$\begin{aligned} a_{ij} &= \sum_{k=1}^j l_{ik} l_{jk} \Rightarrow l_{ij} l_{jj} = a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \\ &\Rightarrow l_{ij} = l_{jj}^{-1} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right). \end{aligned}$$

Zum Lösen des linearen Gleichungssystems mit der Cholesky-Zerlegung muss der Algorithmus zum Vorwärts- und Rückwärtseinsetzen angepasst werden, da $\tilde{L}$ auf der Diagonalen beliebige (positive) Einträge hat und nicht notwendigerweise Einsen.

3.4 Dünn besetzte Matrizen und Bandmatrizen

Der Aufwand zum Erstellen der LR- oder Cholesky-Zerlegung wächst kubisch mit der Größe der Matrix. In vielen Anwendungsproblemen treten *dünn besetzte Matrizen* auf:

► **Definition 3.19 (Dünn besetzte Matrix)** Eine Matrix $A \in \mathbb{R}^{n \times n}$ heißt *dünn besetzt*, falls sie nur $O(n)$ von null verschiedene Einträge besitzt. Das *Besetzungsmuster* (im Englischen *sparsity pattern*) $B \subset \{1, \dots, n\}^2$ von A ist die Menge aller Indexpaare (i, j) mit $a_{ij} \neq 0$.

Andere, weniger strenge Definitionen von dünn besetzten Matrizen verlangen, dass die Matrix $O(n \log(n))$ oder auch $O(n\sqrt{n})$ beziehungsweise einfach $o(n^2)$ von null verschiedene Einträge besitzt.

Ein Beispiel für dünn besetzte Matrizen sind *Tridiagonalmatrizen* der Form

$$A \in \mathbb{R}^{n \times n}, \quad a_{ij} = 0 \quad \forall |i - j| > 1.$$

Eine Verallgemeinerung von Tridiagonalsystemen sind die Bandmatrizen:

► **Definition 3.20 (Bandmatrix)** Eine Matrix $A \in \mathbb{R}^{n \times n}$ heißt *Bandmatrix* mit *Bandbreite* $m \in \mathbb{N}$, falls

$$a_{ij} = 0 \quad \forall |i - j| > m.$$

Eine Bandmatrix hat höchstens $n(2m + 1)$ von null verschiedene Einträge.

Es zeigt sich, dass die LR-Zerlegung einer Bandmatrix wieder eine Bandmatrix ist und daher effizient durchgeführt werden kann.

Satz 3.21 (LR-Zerlegung einer Bandmatrix) Es sei $A \in \mathbb{R}^{n \times n}$ eine Bandmatrix mit Bandbreite m . Die LR-Zerlegung (ohne Permutierung)

$$LR = A$$

ist wieder eine Bandmatrix, d. h.

$$L_{ij} = R_{ij} = 0 \quad \forall |i - j| > m,$$

und kann in

$$O(nm^2)$$

Operationen durchgeführt werden.

Beweis Wir zeigen induktiv, dass die entstehenden Eliminationsmatrizen $\tilde{A}_{i,j>1}$ wieder Bandmatrizen sind. Es gilt:

$$\tilde{a}_{ij} = a_{ij} - \frac{a_{1j}a_{i1}}{a_{11}}, \quad i, j = 2, \dots, n.$$

Es sei nun $|i - j| > m$. Dann ist $a_{ij} = 0$. Ebenso müssen $a_{1j} = a_{i1} = 0$ sein, da $1 \leq i, j \leq n$.

Zur Aufwandsberechnung vergleiche den Beweis zu Satz 3.8. Im Fall einer Bandmatrix müssen in Schritt i der Elimination nicht mehr $(n - i)^2$ elementare Operationen, sondern höchstens m^2 elementare Operationen durchgeführt werden. Ebenso müssen nur m Elemente der Frobenius-Matrix zur Reduktion bestimmt werden. Insgesamt ergibt sich:

$$\sum_{i=1}^n (m + m^2) = nm^2 + O(nm).$$

□

Für Tridiagonalmatrizen kann die LR-Zerlegung sehr effizient durchgeführt werden und der Algorithmus hat einen eigenen Namen.

Satz 3.22 (Thomas-Algorithmus) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Tridiagonalmatrix. Die Matrizen L und R der LR-Zerlegung von A sind wieder Tridiagonalmatrizen und können in $O(n)$ Operationen erstellt werden.

Beweis Folgt direkt aus Satz 3.21.

□

Der Unterschied, eine LR-Zerlegung für eine voll besetzte Matrix und eine Bandmatrix durchzuführen, ist enorm. Zur Diskretisierung der Laplace-Gleichung mit finiten Differenzen müssen lineare Gleichungssysteme mit der sogenannten *Modellmatrix* gelöst werden:

Tab. 3.2 Rechenzeit zum Erstellen der LR-Zerlegung einer Bandmatrix $A \in \mathbb{R}^{n \times n}$ mit Bandbreite $m = \sqrt{n}$ auf einem Rechner mit 10 GigaFLOPS

n	Operationen	Zeit (Bandstruktur)	Zeit (voll besetzt)
100	10^5	1 μ s	30 μ s
1000	10^6	100 μ s	30 ms
10000	10^8	10 ms	30 s
100000	10^{10}	1 s	10 h
1000000	10^{12}	2 min	1 Jahr

$$A = \begin{pmatrix} A_m & -I_m & 0 & \cdots & 0 \\ -I_m & A_m & -I_m & & \vdots \\ 0 & -I_m & \ddots & \ddots & 0 \\ \vdots & & \ddots & \ddots & -I_m \\ 0 & \cdots & 0 & -I_m & A_m \end{pmatrix}, \quad A_m = \begin{pmatrix} 4 & -1 & 0 & \cdots & 0 \\ -1 & 4 & -1 & & \vdots \\ 0 & -1 & \ddots & \ddots & 0 \\ \vdots & & \ddots & \ddots & -1 \\ 0 & \cdots & 0 & -1 & 4 \end{pmatrix} \Bigg\} m,$$

wobei $I_m \in \mathbb{R}^{m \times m}$ die Einheitsmatrix ist. Die Matrix $A \in \mathbb{R}^{n \times n}$ (für eine Quadratzahl n) hat die Bandbreite $m = \sqrt{n}$. In Tab. 3.2 geben wir die notwendigen Rechenzeiten zum Erstellen der LR-Zerlegung auf einem hypothetischen Rechner mit 10 GigaFLOPS an. Man vergleiche mit Tab. 3.1.

Die Modellmatrix ist eine Bandmatrix mit Bandbreite $m = \sqrt{n}$, hat aber in jeder Zeile neben dem Diagonalelement höchstens vier von Null verschiedene Einträge $a_{i,i \pm 1}$ und $a_{i,i \pm m}$. Bei dünn besetzten Matrizen stellt sich die Frage, ob die LR-Zerlegung, also die Matrizen L und R , das gleiche dünne Besetzungsmuster haben. Dies ist im Allgemeinen nicht der Fall und die Matrizen L und R können dicht besetzt sein. Im Fall der Modellmatrix sind L und R dicht besetzte Bandmatrizen mit Bandbreite m .

Der Aufwand zur Berechnung der LR-Zerlegung einer dünn besetzten Matrix hängt wesentlich von der Sortierung, also der Pivotisierung der Matrix ab. Wir betrachten hierzu ein einfaches Beispiel:

Beispiel 3.23 (LR-Zerlegung einer dünn besetzten Matrix)

Wir betrachten die beiden Matrizen:

$$A_1 := \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 0 & 0 \\ 3 & 0 & 1 & 0 \\ 4 & 0 & 0 & 1 \end{pmatrix}, \quad A_2 := \begin{pmatrix} 1 & 0 & 0 & 4 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 2 \\ 4 & 3 & 2 & 1 \end{pmatrix}.$$

Die beiden Matrizen gehen durch simultanes Vertauschen der ersten und vierten sowie der zweiten und dritten Zeile und Spalte auseinander hervor. Es gilt:

$$P^T A_1 P = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 1 & 0 & 0 \\ 3 & 0 & 1 & 0 \\ 4 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 4 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 2 \\ 4 & 3 & 2 & 1 \end{pmatrix} = A_2$$

Die LR-Zerlegung der Matrix A_1 (ohne Pivotisierung) lautet:

$$L_1 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 3 & 2 & 1 & 0 \\ 4 & \frac{8}{3} & 1 & 1 \end{pmatrix}, \quad R_1 = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 0 & -3 & -6 & -8 \\ 0 & 0 & 4 & 4 \\ 0 & 0 & 0 & \frac{7}{3} \end{pmatrix}$$

und für die Matrix A_2 erhalten wir (wieder ohne Pivotisierung)

$$L_2 = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 4 & 3 & 2 & 1 \end{pmatrix}, \quad R_2 = \begin{pmatrix} 1 & 0 & 0 & 4 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & -28 \end{pmatrix}$$

Obwohl beide Matrizen bis auf Zeilen- und Spaltentausch das gleiche lineare Gleichungssystem beschreiben, haben die LR-Zerlegungen ein gänzlich unterschiedliches Besetzungsmuster: Die LR-Zerlegung von Matrix A_1 ist voll besetzt, während die LR-Zerlegung zu Matrix A_2 mit dem gleichen dünnen Besetzungsmuster auskommt wie die Matrix A_2 selbst.

Dieses Beispiel lässt sich auf die entsprechende $n \times n$ -Matrix verallgemeinern. In Fall 1 sind n^2 Einträge zum Speichern der LR-Zerlegung notwendig, in Fall 2 nur $3n$. Für $n \gg 1000$ ist dieser Unterschied entscheidend. Die Anzahl der benötigten Operationen unterscheidet sich in beiden Fällen noch wesentlicher. ◀

3.4.1 Dünn besetzte Matrizen in der Praxis

Dünn besetzte Matrizen treten in der Anwendung oft auf, etwa bei der Diskretisierung von Differentialgleichungen, aber auch bei der Berechnung von kubischen Splines. In den Exkursen 3.8 und 5.6 werden wir auf dünn besetzte Matrizen von Art der Modellmatrix stoßen. Damit die LR-Zerlegung aus der dünnen Besetzungsstruktur Nutzen ziehen kann, muss die Matrix entsprechend permutiert (man sagt in diesem Zusammenhang auch „sortiert“) sein. Werden Nulleinträge in A in der LR-Zerlegung überschrieben, so spricht man von *fill-ins*. Verbesserung der LR-Zerlegung beruhen weniger auf der Zerlegung selbst sondern auf der Entwicklung effizienter Sortierv Verfahren zur Reduktion der *fill-ins*. Wir wissen, dass die LR-

Zerlegung einer Bandmatrix mit Bandbreite m wieder eine Bandmatrix mit Bandbreite m ist und in $O(nm^2)$ Operationen durchgeführt werden kann. Eine Idee zur Sortierung der Matrix A besteht nun darin, die Einträge so anzuordnen, dass die sortierte Matrix eine Bandmatrix mit möglichst dünner Bandbreite ist.

3.4.2 Der Cuthill-McKee-Sortieralgorithmus

Ein klassisches Verfahren ist der *Cuthill-McKee-Algorithmus* benannt nach Elizabeth Cuthill und James McKee. Dieser erstellt eine Sortierung der n Indizes, also eine Bijektion

$$f : \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\},$$

sodass miteinander verbundene Indizes i und j , also Indizes zu Matrixeinträgen $a_{ij} \neq 0$, nahe beieinanderstehen. Zur Herleitung des Verfahrens benötigen wir einige Begriffe. Es sei $A \in \mathbb{R}^{n \times n}$ eine dünn besetzte Matrix mit Besetzungsmuster

$$B(A) = \{(i, j) \in \{1, \dots, n\} \times \{1, \dots, n\} : a_{ij} \neq 0\}.$$

Dabei gehen wir der Einfachheit halber davon aus, dass B symmetrisch ist. Aus $(i, j) \in B$ folgt $(j, i) \in B$. Die Matrix selbst muss aber nicht symmetrisch sein. Zu einem Index $i \in \{1, \dots, n\}$ sei $\mathcal{N}(i)$ die Anzahl aller mit i verbundenen Indizes:

$$\mathcal{N}(i) := \{j \in \{1, \dots, n\} : (i, j) \in B\}.$$

Und für eine beliebige Menge $N \subset \{1, \dots, n\}$ bezeichnen wir mit $\#N$ die Menge der Elemente in N , d. h. mit $\#\mathcal{N}(i)$ die Anzahl der Nachbarn von i .

Der Algorithmus füllt schrittweise eine Indexliste $I = (i_1, i_2, \dots)$, bis alle Indizes $1, \dots, n$ einsortiert sind. Wir starten mit dem Index $I = (i_1)$, welcher die geringste Zahl von Nachbarn $\#\mathcal{N}(i_1) \leq \#\mathcal{N}(j), \forall j$ besitzt. Im Anschluss fügen wir die Nachbarn $j \in \mathcal{N}(i_1)$ von i_1 hinzu, in der Reihenfolge der jeweiligen Zahl von Nachbarn. Auf diese Weise handelt sich der Algorithmus von Nachbar zu Nachbar und arbeitet die Besetzungsstruktur der Matrix ab, bis alle Indizes zur Liste hinzugefügt wurden.

Bei der genauen Definition des Algorithmus sind noch einige Sonderfälle zu betrachten, sodass dieser auch wirklich terminiert und alle Indizes findet:

Algorithmus 3.6: Cuthill-McKee-Algorithmus

Eingabe: Es sei $A \in \mathbb{R}^{n \times n}$ eine dünn besetzte Matrix mit symmetrischem Besetzungsmuster $B \in (i, j)$

1 Initialisiere leere Indexliste $I = []$ und eine leere Warteschlange Q .

2 **Für** $k = 1$ **bis** n **tue**

3 **Wenn** $\#I = n$ **dann**

4 Stopp

5 **Sonst wenn** $\#I = k - 1 < n$ **dann**

6 Bestimme Indexelement $i_k \notin I: \#\mathcal{N}(i_k) \leq \#\mathcal{N}(j) \forall j \in \{1, \dots, n\} \setminus I$.

7 Nehme i_k hinten in der Liste I auf.

8 Füge i_k der Warteschlange Q zu.

9 Wähle den ersten Index i_k in der Warteschlange I .

10 $N_k := \mathcal{N}(i_k) \setminus I$

11 **Wenn** $\#N_k > 0$ **dann**

12 Sortiere $N_k = [s_1^k, s_2^k, \dots]$ gemäß $\#\mathcal{N}(s_i^k) \leq \#\mathcal{N}(s_{i+1}^k)$

13 Nehme $[s_1^k, s_2^k, \dots]$ sortiert hinten in der Liste I auf.

14 Füge $s_1, s_2, \dots$ hinten an die Warteschlange Q an.

15 Entferne i_k aus der Warteschlange.

Ergebnis: Eine Sortierung der Indizes der Variablen mit der die sortierte Matrix eine dünnere Bandbreite hat.

In Python kann man dies mit Hilfe der Bibliothek `scipy`, in der Datenstrukturen für dünn-besetzte Matrizen implementiert sind, folgendermaßen implementieren

♣ Implementierung 3.6: Cuthill-McKee-Algorithmus

```

1 def cuthill_mckee(A):
2     n = A.shape[0]
3     row, col = A.nonzero()
4     N = [(l, len(l)) for l in [list(row[col == i]) for i in range(n)]]
5
6     I, Q = [], []
7     R = [i for i in range(n)]
8
9     for k in range(n):
10        if len(I) == n:
11            break
12        elif len(I) == k:
13            i = R[np.argmin(np.array([N[i][1] for i in R]))]
14            I.append(i)
15            Q.append(i)
16            R.remove(i)
17
18        i = Q[0]
19        nachbarn = [n for n in N[i][0] if n not in I]
20        nachbarn_sort = sorted(nachbarn, key=lambda i: N[i][1])
21        for ik in nachbarn_sort:

```

```

22         I.append(ik)
23         Q.append(ik)
24         R.remove(ik)
25     Q.pop(0)
26
27     data, row, col = [], [], []
28     for key in A.todok().keys():
29         data.append(A[key])
30         row.append(I.index(key[0]))
31         col.append(I.index(key[1]))
32
33     return sp.sparse.csr_matrix((data, (row, col)), shape=(n, n))

```

Wir betrachten zur Veranschaulichung ein Beispiel:

Beispiel 3.24 (Cuthill-McKee)

Es sei eine Matrix $A \in \mathbb{R}^{8 \times 8}$ mit folgendem Besetzungsmuster gegeben:

$$A := \begin{pmatrix} * & * & & & & & & * \\ * & * & & * & * & & & * \\ & & * & * & * & * & & \\ & & & * & & & & \\ * & * & & * & & * & & \\ * & & & & * & & & \\ & & * & * & * & & & \\ * & * & * & & & & & * \end{pmatrix}$$

Index	$N(i)$	$\#N(i)$
1	1,2,8	3
2	1,2,5,6,8	5
3	3,5,7,8	4
4	4	1
5	2,3,5,7	4
6	2,6	2
7	3,5,7	3
8	1,2,3,8	4

- *Schritt $k = 1$:* (1) Die Indexliste I ist leer. Der Index 4 hat nur einen Nachbarn (sich selbst), d. h.

$$i_1 = 4, \quad I = (4).$$

(2) Für den Index 4 gilt $N_1 = \mathcal{N}(4) \setminus I = \emptyset$, d. h. weiter mit $k = 2$.

- *Schritt $k = 2$:* (1) Die Indexliste hat nur $k - 1 = 1$ Element. Von den verbliebenden Indizes hat Index 6 die minimale Zahl von zwei Nachbarn, d. h.

$$i_2 = 6, \quad I = (4, 6).$$

(2) Es gilt $N_2 = \mathcal{N}(6) \setminus I = \{2, 6\} \setminus \{4, 6\} = \{2\}$.

(3) Ein einzelnes Element ist natürlich sortiert, d. h.

$$i_3 = 2, \quad I = (4, 6, 2).$$

- *Schritt $k = 3$:* (1) Dieser Schritt greift nicht, da I bereits drei Elemente besitzt, es ist $i_3 = 2$.

(2) Es gilt $N_3 = \mathcal{N}(2) \setminus I = \{1, 2, 5, 6, 8\} \setminus \{4, 6, 2\} = \{1, 5, 8\}$.

(3) Es gilt $\#N(1) = 3$, $\#N(5) = 4$ und $\#N(8) = 4$, d. h., wir fügen sortiert hinzu:

$$i_4 = 1, \quad i_5 = 5, \quad i_6 = 8, \quad I = (4, 6, 2, 1, 5, 8).$$

- *Schritt $k = 4$:* (1) I hat mehr als drei Elemente.

(2) $i_4 = 1$. Es ist $N_4 = \mathcal{N}(1) \setminus I = \{1, 2, 8\} \setminus \{4, 6, 2, 1, 5, 8\} = \emptyset$. Daher weiter mit $k = 5$

- *Schritt $k = 5$:* (1) I hat genug Elemente.

(2) $i_5 = 5$. Es ist $N_5 = \mathcal{N}(5) \setminus I = \{2, 3, 5, 7\} \setminus \{4, 6, 2, 1, 5, 8\} = \{3, 7\}$.

(3) Es gilt $\#N(3) = 4$ und $\#N(7) = 3$, d. h., Index 7 wird zuerst angefügt:

$$i_7 = 7, \quad i_8 = 3, \quad I = (4, 6, 2, 1, 5, 8, 7, 3)$$

- *Schritt $k = 6$:* (0) Stopp, da $\#I = 8$.

Wir erhalten mit $I_0 = (1, 2, 3, 4, 5, 6, 7, 8)$, $(I_0)_j \mapsto I_j$ die Sortierung:

$$1 \rightarrow 4, \quad 2 \rightarrow 6, \quad 3 \rightarrow 2, \quad 4 \rightarrow 1, \quad 5 \rightarrow 5, \quad 6 \rightarrow 8, \quad 7 \rightarrow 7, \quad 8 \rightarrow 3$$

Wir erstellen die sortierte Matrix $\tilde{A}$ gemäß $\tilde{a}_{kj} = a_{i_k i_j}$:

$$A := \begin{pmatrix} * & & & & & & & \\ & * & * & & & & & \\ & * & * & * & * & * & & \\ & & * & * & & * & & \\ & & * & & * & & * & * \\ & & & * & * & & * & * \\ & & & & * & * & * & * \\ & & & & & * & * & * \\ & & & & & & * & * & * \end{pmatrix} \begin{matrix} 4 \\ 6 \\ 2 \\ 1 \\ 5 \\ 8 \\ 7 \\ 3 \end{matrix}$$

Die so sortierte Matrix ist eine Bandmatrix mit Bandbreite $m = 3$. ◀

Viele Methoden zur Sortierung einer Matrix basieren auf der Graphentheorie. Der Cuthill-McKee-Algorithmus sucht eine Permutierung des Besetzungsmusters, sodass eng benachbarte Indizes auch in der Reihenfolge nahe beieinanderstehen. Andere Methoden versuchen den durch das Besetzungsmuster aufgespannten Graphen möglichst in Teilgraphen zu zerlegen. Diese Teilgraphen entsprechen Blöcken in der Matrix A . Die anschließende LR-Zerlegung erzeugt dann voll besetzte, dafür kleine Blöcke. Die einzelnen Blöcke sind nur schwach gekoppelt. Ein Vorteil dieser Sortierung ist die Möglichkeit, effiziente Parallelisierungsverfahren für die einzelnen Blöcke verwenden zu können.

In Abb. 3.1 zeigen wir die Besetzungsstruktur einer Matrix $A \in \mathbb{R}^{1089 \times 1089}$ vor und nach entsprechender Sortierung. In einem Raster von $1089 \cdot 1089$ Punkten (i, j) ist ein Punkt jeweils schwarz gefärbt, falls a_{ij} zu einem Matrixeintrag ungleich null gehört. Auch wenn der erste Eindruck etwas anderes vermittelt, so ist in allen drei Abbildungen die gleiche Anzahl von schwarzen Punkten vorhanden. Die Matrix hier hat eine symmetrische Besetzungsstruktur, d.h. falls (i, j) schwarz ist, so ist auch (j, i) schwarz und es sind jeweils 9409 Einträge ungleich null (das sind weniger als 1 % aller Einträge). Die LR-Zerlegung der unsortierten Matrix ist nahezu voll belegt mit etwa 1 000 000 Einträgen und deren Berechnung erfordert etwa $400 \cdot 10^6$ Operationen. Der Cuthill-McKee-Algorithmus erzeugt eine Bandmatrix mit sehr dünner Bandbreite $m \approx 70$. Zur Speicherung der LR-Zerlegung sind dann nur noch 150 000 Einträge notwendig und die Berechnung erfordert etwa $5 \cdot 10^6$ Operationen, also nur $1/80$ des Aufwands. Schließlich zeigen wir zum Vergleich die Sortierung mit einem sogenannten *Multi-Fronten-Verfahren*. Hier wird die Matrix in einzelne Blöcke geteilt. Zur Berechnung der LR-Zerlegung sind hier $3 \cdot 10^6$ Operationen notwendig ($1/133$ des Aufwands). Details hierzu finden sich in [44].

Beispiel 3.25 (Cholesky-Zerlegung für Bandmatrizen)

Aus Satz 3.21 wissen wir, dass die LR-Zerlegung einer Bandmatrix wieder zwei Bandmatrizen mit derselben Bandbreite produziert. Es folgt sofort, dass dieselbe Aussage für die Cholesky-Zerlegung gilt. Dies können wir in einer Implementierung ausnutzen, um

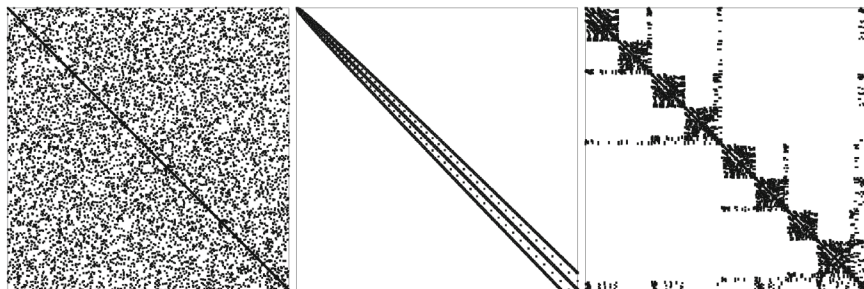
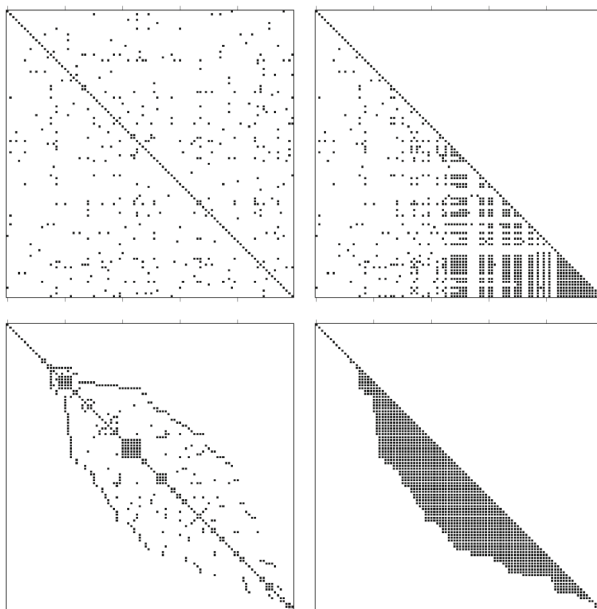


Abb. 3.1 Besetzungsstruktur einer dünn besetzten Matrix $A \in \mathbb{R}^{1089 \times 1089}$ mit insgesamt 9409 von null verschiedenen Einträgen. Links vor Sortierung, Mitte Cuthill-McKee-Algorithmus und rechts nach Sortierung mit einem Multi-Fronten-Verfahren aus [44]

Rechenaufwand an Stellen zu vermeiden, an denen wir wissen, dass die L Matrix einen Nulleintrag haben muss.

Mit der Bibliothek `scipy`, lassen sich dünnbesetzte mit zufälliger Besetzungsstruktur generieren. Wenn wir eine solche Matrix B konstruieren, dann wissen wir dass $A = I + BB^T$ symmetrisch und positiv definit ist. Die Besetzungsstruktur einer solchen Matrix ist oben rechts in Abb. 3.2 gegeben. Wenden wir die Cholesky-Zerlegung direkt darauf an, dann erhalten wir eine quasi vollbesetzte Matrix L . Die Besetzungsstruktur hiervon ist oben rechts in Abb. 3.2 gezeigt.

Abb. 3.2 Oben links:
Besetzungsstruktur einer
dünnbesetzten symmetrisch
positiv definiten Matrix. Oben
rechts: Besetzungsstruktur der
Cholesky Zerlegung. Unten
Links: Besetzungsstruktur der
Matrix nach Anwendung des
Cuthill-McKee-Algorithmus.
Unten rechts:
Besetzungsstruktur der
Cholesky-Zerlegung der
umsortierten Matrix



Mit dem Cuthill-McKee-Algorithmus, können wir die Matrix A umsortieren um eine Bandmatrix in mit einer möglichst geringe Bandbreite zu erhalten. Die Besetzungsstruktur die sich für unsere Matrix ergibt können wir in unten links in Abb. 3.2 sehen. Wenn wir die Cholesky-Zerlegung hierauf an, sehen wir, dass wir wieder eine Bandmatrix haben. Die Anzahl der Nicht-Null-Einträge dieser unteren linken Dreiecksmatrix ist zwar größer als die der Cholesky-Zerlegung der ursprünglichen Matrix, aber die Zerlegung ist effizienter zu berechnen und die Bandstruktur kann jetzt auch im Vor- und Rückwärts-einsetzen ausgenutzt werden, sodass diese auch nur den Aufwand $O(pn)$ haben, wobei p die Bandbreite ist. ◀

3.5 Exkurs: LR-Zerlegung auf Parallelrechnern

In Abschn. 1.4 der Einleitung sind wir kurz auf die Leistungsfähigkeit moderner Computer eingegangen. Der Zuwachs an Leistung ergibt sich aus drei Faktoren:

- Steigerung der Taktrate, also der Anzahl an Operationen, die ein Computer pro Sekunde durchführen kann,
- Steigerung der Effizienz, d. h. eine Reduktion von Takten, die der Prozessor pro Operation (z. B. der Multiplikation zweier Zahlen) benötigt,
- Einsatz vieler paralleler Kerne.

In den 20 Jahren vom Intel 486 in 1991 bis zum Intel Core i7 in 2011 hat sich die Taktrate von 50 MHz auf etwa 3 GHz um den Faktor 60 erhöht. Der Leistungszuwachs in *FLOPS* (*floating point operations per second*) beträgt jedoch 70000, siehe Tab. 1.3. Ein großer Teil dieser Leistungssteigerung kann auf ein besseres Chipdesign zurückgeführt werden, es werden weniger Takte pro Operation benötigt. Darüber hinaus verfügen moderne Prozessoren jedoch über mehrere *Kerne*, können also mehrere Operationen gleichzeitig *parallel* durchführen. Der Core i7 kann bis zu acht Rechnungen gleichzeitig durchführen. Dies geschieht jedoch nicht automatisch, also von Seiten der Hardware. Jedes numerische Verfahren muss speziell für solche Parallelrechner implementiert werden. Ein üblicher *sequenzieller* Algorithmus wird dieses Potenzial nicht nutzen, d. h., der Prozessor kann nur zu etwa 10 % ausgelastet werden. Bei Grafik-Chips (GPU) mit über 500 Kernen ist dieses Ungleichgewicht noch größer. Werden die numerischen Methoden nicht an die neue Hardware angepasst, so geht ein großer Teil des möglichen Leistungsspektrums verloren.

Wir wollen hier am Beispiel der LR-Zerlegung einer voll besetzten Matrix einige, grundsätzliche Konzepte des *parallelen Rechnens* einführen und verweisen ansonsten auf die Literatur [75].

3.5.1 Konzepte des parallelen Rechnens

Zunächst klären wir, was unter parallelem Rechnen bzw. unter einem Parallelrechner zu verstehen ist. Aus [3] entnehmen wir die sehr vage Definition:

► **Definition 3.26 (Parallelrechner)** Ein *Parallelrechner* ist eine Ansammlung von Berechnungseinheiten (Prozessoren), die durch koordinierte Zusammenarbeit große Probleme schnell lösen können.

Parallelität kann sich auf die Daten und auf die Algorithmen beziehen:

- Moderne Prozessoren haben mehrere (üblicherweise 2–8) *Kerne*, also unabhängige Betriebseinheiten. Unabhängige Algorithmen laufen parallel und greifen dabei auf einen gemeinsamen Speicher zu. Dieses Modell wird *multiple instruction, single data (MISD)* genannt und die parallele Nutzung mehrerer Kerne wird als *Multi-Core-Parallelisierung* bezeichnet.
- Einige Computer haben mehrere Prozessoren, jeder von letzteren hat mehrere Kerne. Die Prozessoren greifen weiterhin auf einen gemeinsamen Speicherbereich zu, sodass diese Modelle wieder in den Bereich *multiple instruction, single data* fallen. Die Prozessoren haben jedoch jeweils einen eigenen Zwischenspeicher, den sogenannten *Cache*. Der Zugriff auf Daten aus dem Cache ist sehr schnell. Dies führt dazu, dass der Austausch von Daten zwischen mehreren Prozessoren langsamer sein kann als die Arbeit mit den eigenen Daten.
- Große Parallelrechner, sogenannte *Cluster*, bestehen aus Netzwerken von einzelnen Computern. Auf jedem Computer laufen parallel Algorithmen, die jeweils nur auf den lokalen Speicher zugreifen können. Hier liegt also auch eine Parallelisierung der Daten vor. Dieses Modell wird *multiple instruction, multiple data (MIMD)* genannt. Jeder der Computer selbst kann wieder vom MISD-Typ sein und mehrere Prozessoren mit mehreren Kernen haben. Um auf Daten von anderen Computern zugreifen zu können, müssen diese ausgetauscht werden. Dieser Datenaustausch ist weit langsamer als der Zugriff auf Daten im eigenen Speicher.
- Moderne Grafikkarten und Prozessoren verwenden eine spezielle Form der Parallelisierung. Es läuft ein und derselbe Algorithmus, der parallel auf verschiedenen Daten arbeitet. Dieses Modell heißt *single instruction, multiple data (SIMD)*.

Eine effiziente Nutzung von Parallelrechnern unterscheidet sich je nach Typ von Algorithmus zu Algorithmus. Bei der Parallelisierung von Algorithmen genügt es nicht, nur die Rechenoperationen zu betrachten. Der Computer braucht zum Rechnen auch Daten. Wenn diese nicht schnell genug aus dem Speicher gelesen bzw. die Ergebnisse nicht schnell genug in den Speicher geschrieben werden können, dann hat der Prozessor nichts zu tun.

Bemerkung 3.27 (Rechengeschwindigkeit und Datengeschwindigkeit) Eine moderne Mehrkern-CPU erreicht theoretisch eine Leistung von mehr als 100 GigaFLOPS, also mehr als $100\,000\,000\,000 = 10^{11}$ Berechnungen pro Sekunde. Wir betrachten die Summe zweier Vektoren

$$z = x + y, \quad z_i = x_i + y_i \text{ für } i = 1, \dots, N,$$

mit $N = 10^{10}$. Theoretisch sollte der Prozessor in der Lage sein, diese Aufgabe in 0.1 s zu bewältigen. Um die Einträge $x_i + y_i$ zu addieren, müssen x_i, y_i aus dem Speicher zur CPU und z_i zurück in den Speicher kopiert werden. Insgesamt müssen $3 \cdot 10^{10}$ Fließkommazahlen vom Speicher in die CPU oder zurückgeschrieben werden. Bei doppelter Genauigkeit, siehe Abschn. 1.4, sind dies $8 \cdot 3 \cdot 10^{10} = 24 \cdot 10^{10}$ Bytes. Ein moderner Computer erreicht eine Datenrate von etwa 30 Gigabyte pro Sekunde, kann also $30 \cdot 10^9$ Daten zwischen CPU und Speicher übertragen. Zu den 0.1 s Rechenzeit kommen bei diesem Beispiel somit noch etwa 8 s zum Transfer der Daten hinzu. Nutzt man Parallelrechner mit verteiltem Speicher, so kann die Transferrate noch erheblich größer ausfallen. ♦

In Anbetracht dieser Diskussion werden wir im Laufe dieses Abschnitts immer davon ausgehen, dass der Transfer von Daten einen Aufwand bedeutet, der unter Umständen viel größer ist, als der eigentliche Rechenaufwand.

Ziel der Parallelisierung ist die Reduktion von Laufzeiten durch gleichzeitige Verwendung von $P \in \mathbb{N}$ parallelen *Prozessen*. Wir definieren als Erweiterung von Definition 1.5:

► **Definition 3.28 (Paralleler Aufwand, Speedup, Overhead und Effizienz)** Zur Bearbeitung eines Problems von Problemgröße $n \in \mathbb{N}$ sei ein *optimaler sequenzieller Algorithmus* mit dem Aufwand $T(n)$ gegeben. (Dabei kann der Aufwand entweder in elementaren Operationen oder in Laufzeit gemessen werden.) Den Aufwand des parallelen Algorithmus mit $P \in \mathbb{N}$ parallelen Prozessen bezeichnen wir mit $T_P(n)$. Der *parallele Speedup* $S_P(n)$, die *parallele Effizienz* $E_P(n)$ und der *parallele Overhead* $O_P(n)$ sind gegeben als

$$S_P(n) = \frac{T(n)}{T_P(n)}, \quad E_P(n) = \frac{S_P(n)}{P}, \quad O_P(n) = PT_P(n) - T(n).$$

Wir betrachten hierzu ein Beispiel.

Beispiel 3.29 (Parallele Addition zweier Vektoren)

Die Aufgabe besteht in der Berechnung von $x = y + z$ mit $x, y, z \in \mathbb{R}^n$. Der sequenzielle Algorithmus ist gegeben als

Algorithmus 3.7: Sequentielle Addition zweier Vektoren

```

1 Für  $i = 1$  bis  $n$  tue
2    $x_i = y_i + z_i$ 
```

und hat die Laufzeit $T(n) = n$, gemessen in elementaren Operationen. Wir betrachten nun eine parallele Variante des Verfahrens, verteilt auf $P \in \mathbb{N}$ Prozesse, die auf den gleichen Speicherbereich zugreifen können. Wir gehen davon aus, dass P ein Teiler von n ist.

Algorithmus 3.8: Parallele Addition zweier Vektoren

```

1  $n_p = n/P$ 
  /* Auf jeden Prozess  $p=1, \dots, P$ :
2 Für  $i = p \cdot n_p + 1$  bis  $(p+1) \cdot n_p$  tue
3    $x_i = y_i + z_i$ 
  */

```

Die eigentliche Arbeit, also das Addieren der Vektoren, wird optimal auf die P Prozesse aufgeteilt. Dennoch besteht bei diesem Algorithmus ein kleiner paralleler Zusatzaufwand (also *Overhead*), die Berechnung der Elemente pro Prozess, $n_p = n/P$, sowie die Berechnung der Grenzen für jeden Prozess, $p \cdot n_p$ sowie $(p+1) \cdot n_p$. Somit gilt:

$$T_P(n) = \frac{n}{P} + 3, \quad O_P(n) = P T_P(n) - T(n) = (n - n) + 3P = 3P.$$

Der parallele Speedup ergibt sich als

$$S_P(n) = \frac{T(n)}{T_P(n)} = \frac{n}{\frac{n}{P} + 3} = \frac{P}{1 + 3\frac{P}{n}},$$

die parallele Effizienz als

$$E_P(n) = \frac{S_P(n)}{P} = \frac{1}{1 + 3\frac{P}{n}}. \quad (3.9)$$

Für die Effizienz gilt $0 < E_P(n) < 1$. Bei P fest und $n \rightarrow \infty$ folgt $E_P(n) \rightarrow 1$. Umgekehrt für n fest und $P \rightarrow n$ folgt $E_P(n) \rightarrow 1/4$. In Abb. 3.3 ist für $n = 10\,000$ und $n = 10\,000$ der parallele Speedup für $P = 1, \dots, 500$ Prozesse dargestellt. Je nach Problemgröße nimmt der Speedup und damit die parallele Effizienz stark ab. Für eine kleine Anzahl von parallelen Prozessen wird dieses parallele Verfahren sehr effizient sein. Auf den acht Kernen eines Core i7 kann fast optimaler Speedup erwartet werden. ◀

Dieses einfache Beispiel zeigt, dass die Analyse von parallelen Algorithmen nicht trivial ist. Die parallele Effizienz hängt von der Anzahl an Prozessoren und der Problemgröße ab. Zur genaueren Analyse von parallelen Verfahren werden zwei weitere Begriffe eingeführt:

► **Definition 3.30 (Skalierbarkeit)** Unter der *starken Skalierbarkeit* eines parallelen Verfahrens versteht man das Verhalten der parallelen Laufzeit bei steigender Anzahl von Prozessen P und fester Problemgröße n . Unter der *schwachen Skalierbarkeit* eines parallelen Verfahrens versteht man das Verhalten der parallelen Laufzeit bei steigender Anzahl von Prozessen P bei fester Problemgröße pro Prozess n/P .

Wir haben bereits gesehen, dass die *starke Skalierbarkeit* schlechter wird, wenn die Anzahl der Prozesse vergrößert wird. Zur Analyse der *schwachen Skalierbarkeit* führen wir die Größe $c = n/P$ ein und erhalten aus (3.9)

$$E_P(n) = \frac{1}{1 + \frac{3}{c}} \text{ bei } c = \frac{n}{P} \text{ fest.}$$

Für $n \rightarrow \infty$ folgt mit $P = c/n$ für die Effizienz bei gleichzeitig steigender Zahl an Prozessen $E_P(n) \rightarrow 1/(1 + 3/c)$. Halten wir die Problemgröße pro Prozess fest, etwa bei $c = 100$, so bleibt die parallele Effizienz stets $E_P(n) \approx 0.97$.

Die Addition zweier Vektoren ist ein sehr einfach zu parallelisierendes Verfahren, da die einzelnen Threads unabhängig voneinander, jeweils auf eigenen Daten arbeiten können. Probleme ergeben sich meist dann, wenn Daten zwischen den Threads ausgetauscht werden müssen. Hierzu betrachten wir ein zweites einfaches Beispiel:

Beispiel 3.31 (Paralleles Skalarprodukt)

Es seien $x, y \in \mathbb{R}^n$. Zu berechnen ist das Skalarprodukt $r = (x, y)$. Ein sequenzieller Algorithmus ist gegeben durch

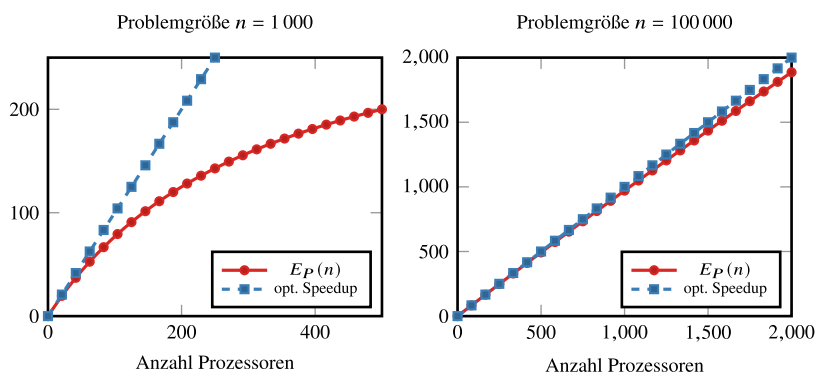


Abb. 3.3 Theoretischer paralleler Speedup für die Addition zweier Vektoren für eine steigende Anzahl von Prozessen. Die erreichte Effizienzsteigerung hängt stark von der Problemgröße ab

```

1  $r = 0$ 
2 Für  $i = 1$  bis  $n$  tue
3    $r = r + x_i \cdot y_i$ 

```

und hat wieder lineare Laufzeit $T(n) = n$. Für einen parallelen Algorithmus stellen wir uns nun vor, dass die Vektoren $x, y \in \mathbb{R}^n$ bereits auf P Prozesse aufgeteilt sind. Auf jedem Prozess das Teilprodukt berechnet und im Anschluss wird das Ergebnis auf dem Prozess mit Nummer $p = 1$ eingesammelt.

Algorithmus 3.9: Paralleles Skalarprodukt

```

/* Auf dem Prozess mit Nummer  $p \in \{1, \dots, P\}$  */
1  $n_p = n/P$ 
2  $r_p = 0$ 
3 Für  $i = p \cdot n_p + 1$  bis  $(p+1) \cdot n_p$  tue
4    $r_p = r_p + x_i \cdot y_i$ 
5 Wenn  $p = 1$  dann
6   Setze  $r = r_1$ 
7   Für  $i = 2$  bis  $P$  tue
8     Empfange  $r_i$  von Prozess  $i$ 
9      $r = r + r_i$ 
10  Für  $i = 2$  bis  $P$  tue
11    Sende Ergebnis  $r$  an Prozess  $i$ 
12 Sonst
13   Sende Teilergebnis  $r_p$  an Prozess 1
14   Empfange Gesamtergebnis  $r$  von Prozess 1

```

Die erste Phase des parallelen Algorithmus ist vergleichbar mit der Addition zweier Vektoren. Die parallele Laufzeit ist wieder n/P . In der zweiten Phase des Algorithmus werden die Ergebnisse addiert. Dies geschieht beim Prozess mit Nummer 1. Dieser empfängt der Reihe nach die Ergebnisse von den Prozessen $p = 2, \dots, P$, addiert und sendet diese dann. Der Aufwand für diesen zweiten Schritt ist

$$T_{\text{sum}}(n, P) = 2c_k P + P,$$

wobei c_k beschreibt, wie lange die Kommunikation, also der Austausch eines Werts zwischen den Prozessen dauert. Als Gesamtaufwand ergibt sich

$$T(n, P) = \frac{n}{P} + 2c_k P + P,$$

und eine *parallele Effizienz* von

$$E_P(n) = \frac{T(n)}{PT_P(n)} = \frac{n}{n + 2c_k P^2 + P^2} = \frac{1}{1 + \frac{2c_k + 1}{n} P^2}.$$

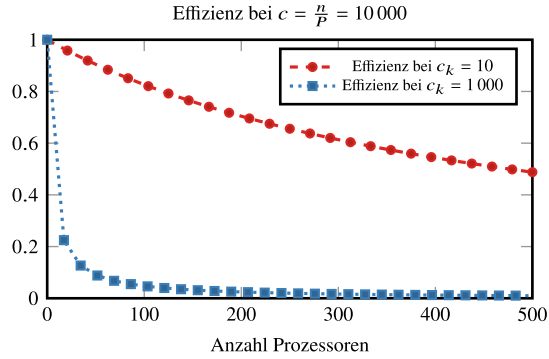


Abb. 3.4 Schwache Skalierbarkeit für das parallele Skalarprodukt (nicht optimaler Algorithmus) bei fester Problemgröße pro Thread $c = n/P = 10\,000$. Der Fall $c_k = 10$ passt zu einem System mit sehr schnellem Zugriff auf den Speicher, z. B. ein *Shared-Memory-System* mit vielen Kernen und gemeinsamem Hauptspeicher. Der Fall $c_k = 1000$ passt zu einem System mit langsamer Speichieranbindung, z. B. ein *Cluster* mit einem üblichen Netzwerk

Für die Analyse untersuchen wir wieder die schwache Skalierbarkeit, wählen also $c = n/P$ fest. Dann gilt

$$E_P(n) = \frac{1}{1 + \frac{2c_k + 1}{c} P}. \quad (3.10)$$

In Abb. 3.4 zeigen wir den Verlauf der Effizienz für $c = 10\,000$. Wir gehen also davon aus, dass jeder Prozess über ein Teilproblem der Größe 10 000 verfügt. Angenommen, die Parallelisierung folgt bei Nutzung des gleichen Speichers, so wählen wir $c_k = 10$: Der Zugriff auf eine Zahl aus dem Speicher sei somit 10-mal so teuer wie eine einzelne Berechnung. Nehmen wir an, die Prozesse sind auf einem großen Cluster verteilt, so wählen wir $c_k = 1000$, die Transferrate sei also wesentlich langsamer. ◀

Dieses Beispiel zeigt, dass das Skalarprodukt eine für die Parallelisierung sehr ungünstige Operation ist. Pro Speichereinheit ist nur genau eine Rechenoperation notwendig. Auf Systemen mit vielen parallelen Prozessen bei schlechtem Netzwerk ist das Skalarprodukt kaum effizient parallelisierbar und muss, soweit möglich, vermieden werden.

Algorithmus 3.9 ist nicht optimal. Das Einsammeln von verteilten Daten, wie es im zweiten Schritt der Berechnung geschieht, ist eine parallele Operation, die in vielen Algorithmen auftaucht. Sie wird *all-to-one reduction* genannt und kann besser durchgeführt werden. Die Idee ist, nicht nur einen Thread zu beschäftigen: Zunächst bildet jeder Thread mit Nummer $p = 2k$ die Summe von sich und seinem Nachbarn, also $r_{2k} = r_{2k} + r_{2k+1}$. Im Anschluss bilden alle Threads mit Nummer $p = 4k$ die Summen $r_{4k} = r_{4k} + r_{4k+2}$. Dies wird rekursiv in $\log_2(P)$ Schritten durchgeführt, bis der erste Thread das Ergebnis hat, siehe [75].

Algorithmus 3.10: Kommunikation – alle an Prozess 1

```

/* Für  $p = 1, \dots, P = 2^k$  */
1 Für  $i = 1$  bis  $k - 1$  tue
2    $r = \lceil p/2^i \rceil$ 
3    $s = p \bmod s2^i$ 
4   Wenn  $s = 1$  dann
5     Empfange von  $p + 2^{i-1}$ 
6   Sonst
7     Sende an  $s$ 

```

Prozess 1 sammelt hier die Informationen von allen anderen Prozessen in $k = \log_2(P)$ Schritten ein. Bei Verwendung dieser Methode kann die parallele Laufzeit des Skalarprodukts reduziert werden auf

$$T(n, P) = \frac{n}{P} + 2c_k \log_2(P) + \log_2(P).$$

Bei fester Problemgröße pro Thread führt dies statt (3.10) zu

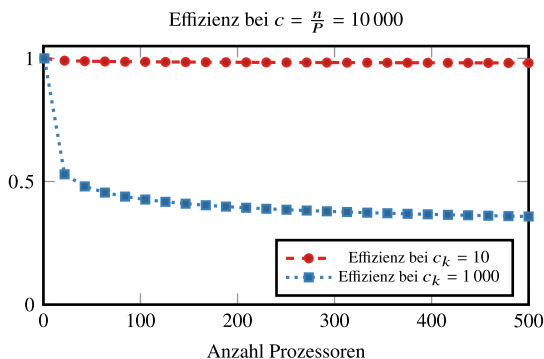
$$E_P(n) = \frac{1}{1 + \frac{(2c_k+1)}{c} \log_2(P)},$$

und wesentlich besseren parallelen Effizienzen. Im Fall $c_k = 1000$ sinkt die parallele Effizienz für $c = 10\,000$ jedoch wieder sehr schnell, siehe Abb. 3.5.

3.5.2 Parallele LR-Zerlegung

Diese beiden einfachen Beispiele zeigen, dass die Analyse von parallelen Algorithmen sehr aufwendig ist und auch bereits kleine Unachtsamkeiten (wie eine zu einfache Summenbildung) eine effiziente Parallelisierung verhindern. Im Folgenden wollen wir uns nun der

Abb. 3.5 Schwache Skalierbarkeit für das parallele Skalarprodukt (optimaler logarithmischer Algorithmus) bei fester Problemgröße pro Thread $c = n/P = 10\,000$ sowie für $c_k = 10$ (schneller Speicherzugriff) und $c_k = 1000$ (langsamer Speicherzugriff)



eigentlichen Aufgabe, der Entwicklung einer parallelen LR-Zerlegung, zuwenden. Es sei also durch $A \in \mathbb{R}^{n \times n}$ eine (voll besetzte) reguläre Matrix gegeben. Wir gehen wieder davon aus, dass die Kommunikation von Werten zwischen den Prozessen „teuer“ ist. Wir betrachten die LR-Zerlegung $A = LR$ ohne Pivotisierung (siehe Abschn. 3.2.2) und wollen den Algorithmus der LR-Zerlegung *an Ort und Stelle* parallelisieren, siehe Algorithmus 3.3:

Algorithmus 3.11: Sequentielle LR-Zerlegung einer Matrix A

Eingabe: Es sei $A = (a_{ij})_{ij} \in \mathbb{R}^{n \times n}$ eine reguläre Matrix, deren LR-Zerlegung existiert:

```

1 Für  $i = 1$  bis  $n$  tue
2   Für  $k = i + 1$  bis  $n$  tue
3      $a_{ki} = a_{ki} / a_{ii}$ 
4   Für  $j = i + 1$  bis  $n$  tue
5      $a_{kj} = a_{kj} - a_{ki} \cdot a_{ij}$ 
```

In Satz 3.8 haben wir den sequenziellen Aufwand der LR-Zerlegung als

$$T(n) = \frac{1}{3}n^3 + O(n^2) \quad (3.11)$$

bestimmt. Für die Parallelisierung gehen wir davon aus, dass die Matrix $A \in \mathbb{R}^{n \times n}$ zeilenweise auf die P Prozesse verteilt ist. Dafür sei $n_p = n/P \in \mathbb{N}$ eine ganze Zahl. Wir wählen die Zerlegung zunächst so, dass Prozess 1 über die Zeilen $1, \dots, n_p$ verfügt, Prozess 2 über $n_p + 1, \dots, 2n_p$ und so weiter. Wir definieren hierfür die Indexmengen

$$p_s := (p - 1) \cdot n_p + 1, \quad p_e := p \cdot n_p, \quad \mathcal{N}(p) = \{p_s, \dots, p_e\}, \quad p = 1, \dots, P.$$

Eine Zeile $i \in \{1, \dots, N\}$ liegt in Indexmenge $\mathcal{N}(\lceil i/n_p \rceil)$:

$$A := \left(\begin{array}{cccc} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ \vdots & \vdots & & \vdots \\ a_{n_p,1} & a_{n_p,2} & \cdots & a_{n_p,n} \\ \hline a_{n_p+1,1} & a_{n_p+1,2} & \cdots & a_{n_p+1,n} \\ \vdots & \vdots & & \vdots \\ a_{2n_p,1} & a_{2n_p,2} & \cdots & a_{2n_p,n} \\ \hline \vdots & \vdots & & \vdots \\ \vdots & \vdots & & \vdots \\ \hline a_{(p-1)n_p+1,1} & a_{(p-1)n_p+1,2} & \cdots & a_{(p-1)n_p+1,n} \\ \vdots & \vdots & & \vdots \\ a_{n,1} & a_{n,2} & \cdots & a_{n,n} \end{array} \right)$$

Die Zeilen unterhalb der Diagonale können nun in jedem Prozess parallel verarbeitet werden. Um die Schritte 3 und 5 von Algorithmus 3.11 durchführen zu können, müssen die Prozesse die Pivot-Zeile i kennen. Falls diese Zeile nicht im entsprechenden Prozess gespeichert ist, müssen die Elemente $a_{i,j}$ für $j \geq i$ zunächst übertragen werden.

Algorithmus 3.12: Parallele LR-Zerlegung einer Matrix A

```

Eingabe: Es sei  $A = (a_{ij})_{ij} \in \mathbb{R}^{n \times n}$  eine reguläre Matrix, deren LR-Zerlegung existiert.
          Prozess  $p$  verfüge über Zeilen  $a_{i,j}$  mit  $i \in \mathcal{N}(p)$ .

/* Auf Prozess  $p$ : */
1  $p_s = (p - 1) \cdot n_p + 1$ ,  $p_e = p \cdot n_p$ 
2 Für  $i = 1$  bis  $n$  tue
    /* Austausch Pivot-Zeile */
3 Wenn  $\lceil i/n_p \rceil = p$  dann
4     Sende  $a_{i,i}, \dots, a_{i,n}$  an Prozesse  $p + 1, \dots, P$ 
5 Sonst wenn  $\lceil i/n_p \rceil < p$  dann
6     Empfange  $a_{i,i}, \dots, a_{i,n}$  von Prozess  $p$ 
    /* Eliminieren */
7 Für  $k = \min\{i + 1, p_s\}$  bis  $\max\{n, p_e\}$  tue
8      $a_{ki} = a_{ki}/a_{ii}$ 
9     Für  $j = i + 1$  bis  $n$  tue
10         $a_{kj} = a_{kj} - a_{ki} \cdot a_{ij}$ 

```

Die eigentliche Eliminationsschleife 7–10 wird auf jedem Prozess p nur noch für die Zeilen $i \in \mathcal{N}(i)$ durchgeführt. Es fallen in Schritt i in jedem Prozess maximal $(n - i) \cdot n_p$ Operationen an. Als zusätzliche parallele Arbeit fällt die Übertragung der Pivot-Zeile, also die Übertragung von $n - i$ Einträgen an die Prozesse $q = p + 1, \dots, P$ an. Dabei hat der aktive Prozess p die meiste Arbeit, da dieser die Zeile an alle anderen Prozesse verschickt (welche die meiste Zeit hierauf warten). Das Austeilen der Werte an die $P - p$ Prozesse kann mit dem optimalen Verfahren aus Algorithmus 3.10 in $\lceil \log_2(P - p) \rceil$ Schritten erfolgen. Wir fassen den Aufwand des parallelen Verfahrens in Schritt i zusammen:

$$T_i(n, P) = c_k \log_2(P - \lceil i/n_p \rceil) \cdot (n - i) + (n - i) \min\{n_p, n - i\}$$

Das Minimum kommt ins Spiel, da in den letzten n_p Schritten kein Prozess mehr einen vollen Block zu verarbeiten hat. Der logarithmische Term beschreibt den Aufwand bei der Kommunikation. Wir approximieren ihn grob als $\log_2(P - \lceil i/n_p \rceil) \approx \log_2(P)$, summieren über i von 1 bis n und erhalten mit den bekannten Summenformeln

$$\begin{aligned}
 T(n, P) &= \sum_{i=1}^n T_i(n, P) \approx \sum_{i=1}^n \left(c_k(n - i) \log_2(P) + (n - i) \min\{n_p, n - i\} \right) \\
 &= c_k \frac{n^2 \log_2(P)}{2} + \frac{n^3}{2P} + O(n^2) + O(n^3/P^2)
 \end{aligned}$$

und den *parallelen Speedup*

$$S_P(n) = \frac{T(n)}{T(n, P)} \approx \frac{1}{\frac{3}{2P} + \frac{3c_k \log_2(P)}{2n}},$$

sowie die *parallele Effizienz*

$$E_P(n) = \frac{S_P(n)}{P} = \frac{1}{\frac{3}{2} + \frac{3c_k P \log_2(P)}{2n}} = \frac{1}{\frac{3}{2} + \frac{3c_k \log_2(P)}{2n_p}} \quad (3.12)$$

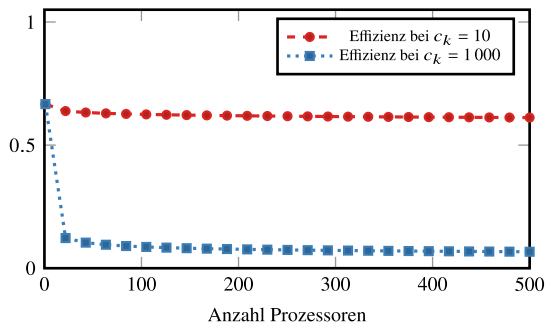
mit der Problemgröße pro Prozess $n_p = n/P$. Für die Fälle $c_k = 10$ und $c_k = 1000$ bei $n_p = 1000$ zeigen wir wieder die erreichbare parallele Effizienz in Abb. 3.6. Bei $c_k = 10$ ist die LR-Zerlegung sehr gut parallelisierbar. Bei $c_k = 1000$ sinkt die Effizienz allerdings sehr schnell. Man beachte, dass $n_p = 1000$ bei $p = 500$ Prozessen bedeutet, dass $n = p \cdot n_p = 500\,000$. Es geht also bereits um eine Matrix der Größe $A \in \mathbb{R}^{500\,000 \times 500\,000}$ mit $2.5 \cdot 10^{11}$ Einträgen. Bei doppelter Genauigkeit sind zur Speicherung bereits 2000 Gigabyte Hauptspeicher nötig. Auf einem *Cluster* aus 500 einzelnen Computern mit jeweils nur 4 Gigabyte Speicher ist dies möglich, die langsame Kommunikation zwischen den einzelnen Knoten macht die Ausführung jedoch ineffizient.

Das Problem bei dem bisherigen Algorithmus ist die schlechte Verteilung der Arbeit auf die P Prozesse. Für alle Schritte $i = n_p + 1, \dots, n$ hat Prozess 1 bereits keine Arbeit mehr. Dies liegt an der zeilenweisen Partitionierung der Daten. Man spricht von einer schlechten *Lastverteilung* (Englisch load balancing). Eine bessere Verteilung ergibt sich, wenn die Zeilen den P Prozessen alternierend zugeteilt werden. Hieraus ergeben sich die Indexmengen

$$\mathcal{N}(p) = \{p, p + P, p + 2P, \dots, p + (n_p - 1)P\}, \quad p = 1, \dots, P.$$

Kommuniziert wird nun stets mit allen anderen Prozessen.

Abb. 3.6 Schwache Skalierbarkeit für die parallele LR-Zerlegung bei logarithmischer Kommunikation und fester Problemgröße pro Prozess von $n_p = 1000$



Algorithmus 3.13: Parallele LR-Zerlegung einer Matrix A bei alternierender Verteilung

```

Eingabe: Es sei  $A = (a_{ij})_{ij} \in \mathbb{R}^{n \times n}$  eine reguläre Matrix, deren LR-Zerlegung existiert.
          Prozess  $p$  verfüge über Zeilen  $a_{i,j}$  mit  $i \in \mathcal{N}(p)$ 
/* Auf Prozess  $p$ :
1 Für  $i = 1$  bis  $n$  tue
    /* Austausch Pivot-Zeile
2 Wenn  $i \bmod P = p$  dann
    Sende  $a_{i,i}, \dots, a_{i,n}$  an andere Prozesse
3 Sonst wenn  $i \bmod P \neq p$  dann
    Empfange  $a_{i,i}, \dots, a_{i,n}$  von Prozess  $i \bmod P$ 
4 /* Eliminieren
5 Für  $k = i + 1$  bis  $n$  tue
    Wenn  $k \bmod P = p$  dann
6        $a_{ki} = a_{ki} / a_{ii}$ 
7       Für  $j = i + 1$  bis  $n$  tue
8          $a_{kj} = a_{kj} - a_{ki} \cdot a_{ij}$ 
9
10

```

Zur Analyse der Laufzeit trennen wir wieder den Aufwand für die Kommunikation und den eigentlichen Rechenaufwand. Da jeder Prozess in Schritt i noch $n_p - \lceil i/P \rceil$ Zeilen der Länge $n - i$ eliminieren muss, gilt in Approximation

$$T_i(n, P) \approx c_k \log_2(P)(n - i) + \left(n_p - \frac{i}{P}\right)(n - i),$$

und summiert

$$\begin{aligned}
 T(n, P) &= \sum_{i=1}^n \left(c_k \log_2(P)(n - i) + \frac{(n - i)^2}{P} \right) \\
 &= \frac{c_k n(n + 1) \log_2(P)}{2} + \frac{n(n + 1)(2n + 1)}{6P}.
 \end{aligned}$$

Dies ergibt die *parallele Effizienz* (wie überspringen den *Speedup*)

$$E(n, P) = \frac{1}{\frac{c_k P \log_2(P)}{2n} + 1} = \frac{1}{1 + c_k \frac{\log_2(P)}{n_p}}.$$

Der Vergleich mit (3.12) zeigt eine um 50 % bessere Effizienz. Insbesondere erreichen wir für kleine P die Effizienz des sequenziellen Algorithmus. Wir können nicht vermeiden, dass bei langsamem Datenaustausch die Effizienz schnell sinkt.

Eine gute Parallelisierung von numerischen Verfahren ist sehr aufwendig. Es müssen Probleme betrachtet werden, die bei einer rein mathematischen Sichtweise keine Rolle spielen. Ein Verfahren kann gut sein auf einem bestimmten Computertyp, auf einem anderen jedoch vollkommen versagen. Für weiter gehende Studien verweisen wir auf die reichhaltige

Literatur, sowohl über theoretische Konzepte des parallelen Rechnens als auch über praktische Anwendungen und Tipps zur Umsetzung [75, 81].

3.6 Nachiteration

Die numerisch durchgeführte LR-Zerlegung $A = LR$ stellt aufgrund von Rundungsfehlern nur eine Näherung dar und die so approximierte Lösung $LR\tilde{x} = b$ ist mit einem Fehler $x - \tilde{x}$ behaftet. Der Defekt gibt einen Anhaltspunkt für den Fehler.

► **Definition 3.32 (Defekt eines linearen Gleichungssystems)** Es sei $\tilde{x} \in \mathbb{R}^n$ die Näherung zur Lösung $x \in \mathbb{R}^n$ von $Ax = b$. Die Größe

$$d(\tilde{x}) := b - A\tilde{x},$$

bezeichnet den Defekt.

Für die exakte Lösung $x \in \mathbb{R}^n$ von $Ax = b$ gilt $d(x) = 0$. Je genauer unsere Lösung ist, umso kleiner ist der Defekt. Für allgemeine Approximationen erhalten wir die folgende *A-posteriori-Fehlerabschätzung*:

Satz 3.33 (Fehlerabschätzung für lineare Gleichungssysteme) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix und $\tilde{x} \in \mathbb{R}^n$ die Approximation zur Lösung $x \in \mathbb{R}^n$ von $Ax = b$. Dann gilt

$$\frac{\|x - \tilde{x}\|}{\|x\|} \leq \text{cond}(A) \frac{\|d(\tilde{x})\|}{\|b\|}.$$

Beweis Es gilt

$$x - \tilde{x} = A^{-1}(b - A\tilde{x}) = A^{-1}d(\tilde{x}) \Rightarrow \|x - \tilde{x}\| \leq \|A^{-1}\| \|d(\tilde{x})\|.$$

Teilen durch $\|b\| = \|Ax\| \leq \|A\| \|x\|$ liefert das gewünschte Ergebnis

$$\frac{\|x - \tilde{x}\|}{\|x\|} \leq \|A\| \|A^{-1}\| \frac{\|d(\tilde{x})\|}{\|b\|}.$$

□

Neben seiner Rolle zur Fehlerabschätzung kommt dem Defekt eine weitere wichtige Bedeutung zu. Wir gehen davon aus, dass wir den Defekt $d(\tilde{x})$ ohne Rundungsfehler berechnen können. Weiter nehmen wir an, dass wir auch die *Defektgleichung*

$$Aw = d(\tilde{x}) = b - A\tilde{x}$$

exakt nach $w \in \mathbb{R}^n$ lösen können. Dann gilt ist $\tilde{x} + w$

$$\tilde{x} + w = \tilde{x} + A^{-1}(b - A\tilde{x}) = \tilde{x} + x - \tilde{x} = x$$

die exakte Lösung des Gleichungssystems. Dieser Vorgang wird *Defektkorrektur* oder *Nachiteration* genannt. Die Annahme, dass Defekt und Defektgleichung ohne Rundungsfehler gelöst werden können, ist natürlich nicht realistisch (dann könnte auch das Originalsystem exakt gelöst werden). Das Prinzip der Defektkorrektur kann aber ausgenutzt werden um die aufwändigen Teile der Rechnung, d. h. die Zerlegung der Matrix $A = LR$, mit reduzierter Genauigkeit durchzuführen. Dies erfordert weniger Speicher und ist auf vielen Computersystemen (wie z. B. GPU Beschleunigern) weit schneller. Die Berechnung des Defektes $d(\tilde{x})$ sowie das Vorwärts- und Rückwärtseinsetzen erfolgt dann mit hoher Genauigkeit.

Die Nachiteration kann in mehreren Schritten als iterativer Prozess durchgeführt werden.

Algorithmus 3.14: Nachiteration

Eingabe: Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix und $b \in \mathbb{R}^n$. Zur Lösung von $Ax = b$ sind die folgenden Schritte durchzuführen:

```

1  Erstelle  $LR = PA$                                      // einfache Genauigkeit
2   $d^{(1)} = b$ 
3   $x^{(1)} = 0$ 
4  Für  $i = 1, \dots, \text{tue}$ 
5     $Ly^{(i)} = Pd^{(i)}$                                      // doppelte Genauigkeit
6     $Rw^{(i)} = y^{(i)}$                                      // doppelte Genauigkeit
7     $x^{(i+1)} = x^{(i)} + w^{(i)}$                              // doppelte Genauigkeit
8     $d^{(i+1)} = b - Ax^{(i+1)}$                              // doppelte Genauigkeit
Ergebnis: Approximierte Lösung  $x$  von  $Ax = b$ .
```

Der Vorteil der Nachiteration liegt in der mehrfachen Verwendung der erstellten LR-Zerlegung. Zur Berechnung der LR-Zerlegung in der ersten Zeile des Verfahrens sind $O(n^3)$ Operationen notwendig, Vorwärts- und Rückwärtseinsetzen sowie die Berechnung des Defektes erfordern nur $O(n^2)$ Operationen. Das heißt, selbst bei Verwenden höherer Genauigkeit ist der Aufwand in Schritt 4–8 des Verfahrens klein im Vergleich zu Schritt 1.

Satz 3.34 (Nachiteration) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix und $b \in \mathbb{R}^n$. Weiter sei $\epsilon > 0$ die hinreichend kleine Fehlertoleranz einfacher Genauigkeit und $0 < \alpha < \epsilon$ die Fehlertoleranz hoher Genauigkeit (etwa $\alpha = \epsilon^2$). Wir definieren die Iteration

$$x^{(0)} = 0, \quad x^{(i)} = x^{(i-1)} + (LR)^{-1}(b - Ax^{(i-1)}),$$

wobei die LR -Zerlegung der Matrix A mit nur einfacher Genauigkeit $O(\epsilon)$ berechnet wird und alle weiteren Berechnungen mit hoher Genauigkeit $O(\alpha)$ durchgeführt werden. Dann gilt mit der exakten Lösung $x = A^{-1}b$ die asymptotische Fehlerabschätzung

$$\frac{\|x^{(i)} - x\|}{\|x\|} = c(A)O(\epsilon) \frac{\|x^{(i-1)} - x\|}{\|x\|} + O(\alpha)$$

mit einer Konstante $c(A) > 0$ die von der Konditionszahl der Matrix A abhängt.

Beweis Wir schreiben einen Schritt der Nachiteration in kompakter Form und geben die jeweiligen Genauigkeiten an:

$$x^{(i)} = \underbrace{x^{(i-1)} + (LR)^{-1}}_{\text{hoch}} \underbrace{(b - Ax^{(i-1)})}_{\text{niedrig}}$$

Mit den Abschätzungen für die relativen Fehler der LR-Zerlegung, der Matrix-Vektor Multiplikation und des Lösen von linearen Gleichungssystemen erhalten wir mit der niedrigen Genauigkeit $\epsilon > 0$ und der hohen Genauigkeit $\epsilon > \alpha > 0$

$$x^{(i)} = (1 + O(\alpha)) \left(x^{(i-1)} + (I + c(A)O(\epsilon)A^{-1})(1 + c(A)O(\alpha))(b - Ax^{(i-1)}) \right),$$

wobei $c(A)$ von der Konditionszahl der Matrix A abhängt, siehe Satz 3.5. Ausmultipliziert und mit $b = Ax$ ergibt sich

$$x^{(i)} = x^{(i-1)} + (1 + c(A)O(\epsilon) + c(A)^2O(\epsilon\alpha))(x - x^{(i-1)}) + O(\alpha)x.$$

Hiermit erhalten wir für den Fehler $x - x^{(i)}$ den Zusammenhang

$$x - x^{(i)} = (c(A)O(\epsilon) + c(A)^2O(\epsilon\alpha))(x - x^{(i-1)}) + O(\alpha)x.$$

Das Ergebnis folgt indem wir den Betrag nehmen und durch $\|x\|$ teilen. Im Fall $c(A)\epsilon < 1$ ist der quadratische Term kleiner und kann asymptotisch vernachlässigt werden. $\square$

Jeder Schritt der Nachiteration verbessert den Fehler um den Faktor $c(A)O(\epsilon)$ bis die Schwelle $O(\alpha)$ erreicht wird. Der Nutzen der Nachiteration liegt einerseits im reduzierten Speicherbedarf, wenn L und R nur mit geringer Genauigkeit berechnet werden, jedoch auch in der schnelleren Ausführung. Dies ist gerade auf moderner Beschleuniger-Hardware relevant, wo Rechnungen in doppelter Genauigkeit, in einfacher und auf sogenannten *Tensor-Cores* auch in halber Genauigkeit durchgeführt werden können. Bei sehr schlecht konditionierten Matrizen stößt die Nachiteration aber an ihre Grenzen, da für Konvergenz auf jeden Fall $\epsilon c(A) < 1$ gelten muss.

Beispiel 3.35 (Nachiteration)

Wir führen den Pythonteil von Beispiel 3.14 fort und verwenden dabei `np.half` Genauigkeit für die LR-Zerlegung und `np.single` Genauigkeit für die gespeicherten Vektoren.

Um die Nachiteration mit `numpy.arrays` zu implementieren, müssen wir vorsichtig sein, damit wir die Einträge der Vektoren ändern und nicht die Vektoren mit neuen überschreiben.

🔗 Implementierung 3.7: Nachiteration

```

1 def nachiteration(A, b, n=3, precision2=np.half):
2     A_2 = A.astype(precision2)
3     pivot = LR_zerlegung_mit_pivot(A_2)
4     d = b.copy()
5     x = np.zeros_like(b)
6     for i in range(n):
7         for p in pivot:
8             d[p] = d[[p[1], p[0]]]
9         y = vorwaerts_einsetzen_ohne_diag(A_2, d)
10        w = rueckwaerts_einsetzen(A_2, y)
11        x[:] += w
12        d[:] = b - A.dot(x)
13    return x

```

Wenden wir diese Implementierung mit einer Nachiteration auf das Problem aus Beispiel 3.14 an, erhalten wir

```

1 A = np.array([[2.3, 1.8, 1],[1.4, 1.1, -0.7],[0.8, 4.3, 2.1]],
2             dtype=np.single)
3 b = np.array([1.2, -2.1, 0.6], dtype=np.single)
4 x = nachiteration(A_single, b_single, n=2, precision2=np.half)
5 np.linalg.norm(x - x_np) / np.linalg.norm(x_np)

```

```
1 2.4034927140331146e-07
```

einen relativen Fehler von 0.000024 %. Also ist die Lösung in etwa drei Größenordnungen besser im Vergleich zu der einfachen Anwendung der LR-Zerlegung mit Pivotisierung mit half Gleitkommazahlen. ◀

3.7 Fixpunktverfahren zum Lösen linearer Gleichungssysteme

Mit der Nachiteration haben wir in Abschn. 3.6 eine Methode kennengelernt, um den Fehleinfluss durch sukzessive Iteration zu verringern. Mit der gestörten LR-Zerlegung $A \approx \tilde{L}\tilde{R}$ haben wir die Iteration

$$x^{k+1} = x^k + \tilde{R}^{-1}\tilde{L}^{-1}(b - Ax^k), \quad k = 0, 1, 2, \dots$$

definiert. Obwohl $\tilde{L}$ und $\tilde{R}$ nicht exakt sind, konvergiert diese Iteration (falls das Residuum $d^k := b - Ax^k$ exakt berechnet werden kann), siehe Satz 3.34. Mit der gestörten LR-Zerlegung $\tilde{L}\tilde{R} \approx A$ lässt sich ein Nachiterationsschritt kompakt schreiben als

$$x^{(i+1)} = x^{(i)} + Cd(\tilde{x}^{(i)}), \quad C := \tilde{R}^{-1}\tilde{L}^{-1}.$$

Dabei ist die Matrix C eine Approximation zur Inversen von A . Es stellt sich die Frage, ob die Nachiteration auch dann ein konvergentes Verfahren bildet, wenn $\tilde{C}$ eine noch größere Approximation der Inversen $\tilde{C} \approx A^{-1}$ ist. Dabei könnte man an Approximationen denken, die auf der einen Seite weiter von A^{-1} entfernt sind, dafür aber wesentlich effizienter z. B. in $O(n^2)$ Operationen, zu erstellen sind.

► **Definition 3.36 (Fixpunktverfahren zum Lösen linearer Gleichungssysteme)** Es sei $A \in \mathbb{R}^{n \times n}$ sowie $b \in \mathbb{R}^n$ und $C \in \mathbb{R}^{n \times n}$. Für einen beliebigen Startwert $x^0 \in \mathbb{R}^n$ iteriere für $k = 1, 2, \dots$

$$x^k = x^{k-1} + C(b - Ax^{k-1}). \quad (3.13)$$

Alternativ führen wir die Bezeichnungen $B := I - CA$ und $c := Cb$ ein. Dann gilt

$$x^k = (I - CA)x^{k-1} + Cb = Bx^{k-1} + c. \quad (3.14)$$

Aufgrund der Konstruktion kann man sich leicht klarmachen, dass durch die Vorschrift $g(x) = Bx + c = x + C(b - Ax)$ wirklich eine Fixpunktiteration mit der Lösung von $Ax = b$ als Fixpunkt gegeben ist, denn für $Ax = b$ ist

$$g(x) = x + \underbrace{C(b - Ax)}_{=0} = x.$$

Fixpunktverfahren sind aus der Analysis bekannt und werden in Kap. 7 kurz rekapituliert. Verfahren von Typ (3.14) werden *stationäre Iterationen* genannt, da die Verfahrensvorschrift $x^k = Bx^{k-1} + c$ immer gleich bleibt und nicht von der aktuellen Näherung x^k abhängt.

Die Konvergenzuntersuchung von Fixpunktiterationen von Typ (3.13) ist recht einfach. Ziehen wir auf beiden Seiten die exakte Lösung $x = A^{-1}b$ ab, so ergibt sich

$$\begin{aligned} x^k - x &= x^{k-1} - x + C(b - Ax^{k-1}) = (I - CA)(x - x^{k-1}) \\ \Rightarrow \|x^k - x\| &\leq \|I - CA\| \cdot \|x - x^{k-1}\| \end{aligned}$$

und die Methode konvergiert, wenn für die *stationäre Matrix* $I - CA$ in der Norm $\|I - CA\| = \rho < 1$ gilt. Es ergibt sich jedoch das Dilemma, dass je nach verwendeter Matrixnorm $\|\cdot\|$ unterschiedliche Konvergenzresultate vorhergesagt werden. Z.B. gilt für die Matrix

$$B = \begin{pmatrix} 0.4 & -0.6 \\ 0.5 & 0.5 \end{pmatrix},$$

dass $\|B\|_2 \approx 0.78$ aber $\|B\|_1 = 1.1$ und $\|B\|_\infty = 1$, vergleiche Satz 2.17. Dies scheint ein Widerspruch zu sein. Die *stationäre Iteration* ist ja immer die gleiche, unabhängig von betrachteter Norm und die Folge x^k konvergiert entweder oder sie konvergiert nicht. Mit einer konkreten Norm $\|I - CA\|$ können wir die Konvergenzrate nur abschätzen. Abhilfe schafft der folgende Satz.

Hilfssatz 3.37 (Matrixnorm und Spektralradius) Zu jeder beliebigen Matrix $B \in \mathbb{R}^{n \times n}$ und zu jedem $\epsilon > 0$ existiert eine natürliche Matrixnorm (also eine von einer Vektornorm induzierte Matrixnorm) $\|\cdot\|_\epsilon$, sodass gilt:

$$\text{spr}(B) \leq \|B\|_\epsilon \leq \text{spr}(B) + \epsilon$$

Beweis Für symmetrische Matrizen ist die Aussage klar, da

$$\text{spr}(B) = \|B\|_2$$

mit der von der euklidischen Norm induzierten Matrixnorm gilt. Für den allgemeinen Fall verweisen wir auf [71]. $\square$

Im nun Folgenden werden wir daher stets mit dem Spektralradius als Norm operieren. Mit diesem Hilfssatz zeigen wir das fundamentale Resultat über allgemeine lineare Fixpunktiterationen:

Satz 3.38 (Fixpunktverfahren zum Lösen linearer Gleichungssysteme) Die Iteration (3.13) konvergiert für jeden Startwert $x^0 \in \mathbb{R}^n$ genau dann gegen die Lösung $x \in \mathbb{R}^n$ von $Ax = b$, falls $\rho := \text{spr}(B) < 1$. Dann gilt das asymptotische Konvergenzverhalten

$$\lim_{k \rightarrow \infty} \left(\frac{\|x^k - x\|}{\|x^0 - x\|} \right)^{1/k} = \text{spr}(B).$$

Beweis Nach einigen Vorbereitungen zeigen wir in Schritt (i), dass aus $\text{spr}(B) < 1$ die Konvergenz $x^k \rightarrow x$ folgt. In Schritt (ii) zeigen wir die Rückrichtung und schließlich in Schritt (iii) die Konvergenzaussage.

Wir definieren den Fehler $e^k := x^k - x$ und erhalten bei Ausnutzen der Fixpunkteigenschaft $x = Bx + c$ die Iterationsvorschrift

$$e^k = x^k - x = Bx^{k-1} + c - (Bx + c) = Be^{k-1}.$$

Entsprechend gilt

$$e^k = B^k e^0 = B^k (x^0 - x). \quad (3.15)$$

(i) Hilfssatz 3.37 besagt, dass zu jedem $\epsilon > 0$ eine natürliche Matrixnorm $\|\cdot\|_\epsilon$ existiert mit

$$\text{spr}(B) \leq \|B\|_\epsilon \leq \text{spr}(B) + \epsilon.$$

Es sei nach Voraussetzung $\text{spr}(B) < 1$, dann existiert ein $\epsilon > 0$ mit

$$\|B\|_\epsilon \leq \text{spr}(B) + \epsilon < 1,$$

und aus (3.15) erhalten wir sofort bei $k \rightarrow \infty$ in der entsprechenden induzierten Vektornorm $\|\cdot\|_\epsilon$

$$\|e^k\|_\epsilon = \|B^k e^0\|_\epsilon \leq \|B^k\|_\epsilon \|e^0\|_\epsilon \leq \|B\|_\epsilon^k \|e^0\|_\epsilon \rightarrow 0 \quad (k \rightarrow \infty).$$

Da im $\mathbb{R}^n$ alle Normen äquivalent sind, konvergiert also $x^k \rightarrow x$ für $k \rightarrow \infty$.

(ii) Die Iteration sei konvergent $x^k \rightarrow x$. Es sei $w \in \mathbb{R}^n$ ein Eigenvektor zum betragsmäßig größten Eigenwert $\lambda_{\max}$ von $B = I - CA$. Dann gilt für $x^0 = x + w$, also $w = x^0 - x = -e^0$ mit (3.15):

$$\lambda_{\max}^k w = B^k w = -B^k e^0 = -e^k$$

Da die Iteration nach Voraussetzung konvergent ist (für jeden Startwert), folgt

$$-e^k = \lambda_{\max}^k w \rightarrow 0 \quad (k \rightarrow \infty),$$

also notwendigerweise

$$\text{spr}(B) = |\lambda_{\max}| < 1.$$

(iii) Fehlerabschätzung: Aufgrund der Äquivalenz aller Normen existieren Konstanten m, M mit $m \leq M$, sodass

$$m\|x\| \leq \|x\|_\epsilon \leq M\|x\|, \quad x \in \mathbb{R}^n.$$

Damit gilt

$$\|e^k\| \leq \frac{1}{m} \|e^k\|_\epsilon = \frac{1}{m} \|B^k e^0\|_\epsilon \leq \frac{1}{m} \|B\|_\epsilon^k \|e^0\|_\epsilon \leq \frac{M}{m} (\text{spr}(B) + \epsilon)^k \|e^0\|.$$

Wegen

$$\left(\frac{M}{m}\right)^{1/k} \rightarrow 1, \quad (k \rightarrow \infty)$$

folgt damit

$$\lim_{k \rightarrow \infty} \left(\frac{\|e^k\|}{\|e^0\|} \right)^{1/k} \leq \text{spr}(B) + \epsilon.$$

Da $\epsilon > 0$ beliebig klein gewählt wird, folgt hiermit die Behauptung. $\square$

Satz 3.38 legt das theoretische Fundament für allgemeine Fixpunktiterationen. Für den Spektralradius $\rho := \text{spr}(B) = \text{spr}(I - CA)$ muss $\rho < 1$ gelten. Ziel dieses Abschnitts ist die Konstruktion von Iterationsmatrizen C , welche

1. möglichst nahe an der Inversen $C \approx A^{-1}$ liegen, damit $\text{spr}(I - CA) \ll 1$,
2. eine möglichst einfache Berechnung der Iteration $x^k = Bx^{k-1} + c$ ermöglichen.

Die erste Forderung ist einleuchtend und $C = A^{-1}$ stellt in diesem Sinne die optimale Matrix dar. Die zweite Bedingung beschreibt den Aufwand zum Durchführen der Fixpunktiteration. Wählen wir etwa $C = \tilde{R}^{-1}\tilde{L}^{-1}$ so bedeutet jeder Schritt ein Vorwärts- und ein Rückwärtseinsetzen, d.h. einen Aufwand der Größenordnung $O(n^2)$. Hinzu kommen die $O(n^3)$ Operationen zum einmaligen Erstellen der Zerlegung. Die Wahl $C = I$ erlaubt eine sehr effiziente Iteration mit $O(n)$ Operationen in jedem Schritt, ohne zusätzlichen Aufwand zum Aufstellen der Iterationsmatrix C . Die Einheitsmatrix I ist jedoch im Allgemeinen eine sehr schlechte Approximation von A^{-1} . Beide Forderungen sind Maximalforderungen und scheinen sich zu widersprechen. Die Wahl von $C = \omega I$ führt auf die *Richardson-Iteration*.

► **Definition 3.39 (Richardson-Iteration)** Zur Lösung von $Ax = b$ sei $x^0 \in \mathbb{R}^n$ ein beliebiger Startwert. Iteriere für $k = 1, 2, \dots$

$$x^k = x^{k-1} + \omega(b - Ax^{k-1})$$

mit einem Relaxationsparameter $\omega > 0$.

Zur Konstruktion weiterer einfacher iterativer Verfahren spalten wir die Matrix A additiv auf zu $A = L + D + R$ mit

$$A = \underbrace{\begin{pmatrix} 0 & & \dots & 0 \\ a_{21} & \ddots & & \\ \vdots & \ddots & \ddots & \\ a_{n1} & \dots & a_{n,n-1} & 0 \end{pmatrix}}_{=:L} + \underbrace{\begin{pmatrix} a_{11} & \dots & 0 \\ & \ddots & \\ 0 & \dots & a_{nn} \end{pmatrix}}_{=:D} + \underbrace{\begin{pmatrix} 0 & a_{12} & \dots & a_{1n} \\ & \ddots & \ddots & \vdots \\ & & \ddots & a_{n-1,n} \\ 0 & \dots & & 0 \end{pmatrix}}_{=:R}.$$

Diese additive Zerlegung ist nicht mit der multiplikativen LR-Zerlegung zu verwechseln. Ist die Matrix A bekannt, so kann die additive Zerlegung in L , D , R ohne weitere Berechnungen angegeben werden. Wir definieren die zwei wichtigsten Iterationsverfahren.

► **Definition 3.40 (Jacobi-Verfahren)** Zur Lösung von $Ax = b$ sei $x^0 \in \mathbb{R}^n$ ein beliebiger Startwert. Iteriere für $k = 1, 2, \dots$

$$x^k = x^{k-1} + D^{-1}(b - Ax^{k-1}),$$

bzw. mit der *Jacobi-Iterationsmatrix* $J := -D^{-1}(L + R)$

$$x^k = Jx^{k-1} + D^{-1}b.$$

► **Definition 3.41 (Gauß-Seidel-Verfahren)** Zur Lösung von $Ax = b$ sei $x^0 \in \mathbb{R}^n$ ein beliebiger Startwert. Iteriere für $k = 1, 2, \dots$

$$x^k = x^{k-1} + (D + L)^{-1}(b - Ax^{k-1}),$$

bzw. mit der *Gauß-Seidel-Iterationsmatrix* $H := -(D + L)^{-1}R$

$$x^k = Hx^{k-1} + (D + L)^{-1}b.$$

Diese beiden Fixpunktverfahren sind einfach, aber dennoch sehr gebräuchlich.

Satz 3.42 (Durchführung des Jacobi- und Gauß-Seidel-Verfahrens) Ein Schritt des Jacobi- bzw. Gauß-Seidel-Verfahrens ist jeweils in $n^2 + O(n)$ Operationen durchführbar. Mit skalaren Operationen kann das Jacobi-Verfahren durch Algorithmus 3.15 beschrieben werden. Das Gauß-Seidel-Verfahren kann in skalaren Operationen mit Algorithmus 3.16 durchgeführt werden.

Algorithmus 3.15: Jacobi-Verfahren

Eingabe: Gegeben seien $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, $x^0 \in \mathbb{R}^n$ sowie eine Toleranz $\epsilon > 0$.

1 **Für** $k = 0, 1, \dots$ **tue**

2 **Für** $i = 1, \dots, n$ **tue**

3 $x_i^{k+1} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij}x_j^k \right)$

4 **Wenn** $\|b - Ax^{k+1}\| < \epsilon$ **dann**

5 Stop

Ergebnis: Approximative Lösung $x = x^{k+1}$ von $Ax = b$.

Algorithmus 3.16: Gauß-Seidel-Verfahren

Eingabe: Gegeben seien $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$, $x^0 \in \mathbb{R}^n$ sowie eine Toleranz $\epsilon > 0$.

1 **Für** $k = 0, 1, \dots$ **tue**

2 **Für** $i = 1, \dots, n$ **tue**

3 $x_i^{k+1} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{k+1} - \sum_{j > i} a_{ij}x_j^k \right)$

4 **Wenn** $\|b - Ax^{k+1}\| < \epsilon$ **dann**

5 Stop

Ergebnis: Approximative Lösung $x = x^{k+1}$ von $Ax = b$.

Beweis Übung. □

Jeder Schritt dieser Verfahren benötigt mit $O(n^2)$ größenordnungsmäßig genauso viele Operationen wie die Nachiteration mit der LR-Zerlegung. Bei Jacobi und Gauß-Seidel sind jedoch weit schlechtere Konvergenzraten zu erwarten (wenn die Verfahren überhaupt kon-

vergieren, dieser Nachweis steht hier noch aus!) Der Vorteil der einfachen Iterationsverfahren zeigt sich erst bei dünn besetzten Matrizen. Hat eine Matrix $A \in \mathbb{R}^{n \times n}$ nur $O(n)$ Einträge, so benötigt jeder Iterationsschritt nur $O(n)$ Operationen. Für eine dünn besetzte Matrix kann die LR-Zerlegung $A = LR$ aus voll besetzten Matrizen L und R bestehen. Dann benötigt das Vorwärts- und Rückwärtseinsetzen weiterhin $O(n^2)$ arithmetische Operationen.

3.7.1 Konvergenzkriterium für Jacobi- und Gauß-Seidel-Iteration

Zur Untersuchung der Konvergenz muss gemäß Satz 3.38 der Spektralradius der Iterationsmatrizen $J = -D^{-1}(L+R)$ sowie $H := -(D+L)^{-1}R$ untersucht werden. Im Einzelfall ist diese Untersuchung nicht ohne Weiteres möglich und einer Matrix A kann der entsprechende Spektralradius nur schwer angesehen werden. Daher leiten wir in diesem Abschnitt einfach überprüfbare Kriterien her, um eine Aussage über die Konvergenz der beiden Verfahren treffen zu können.

Satz 3.43 (Starkes Zeilensummenkriterium) Falls die Zeilensummen der Matrix $A \in \mathbb{R}^{n \times n}$ der strikten *Diagonaldominanz* genügen,

$$|a_{jj}| > \sum_{k=1, k \neq j}^n |a_{jk}|, \quad j = 1, \dots, n,$$

so gilt $\text{spr}(J) < 1$ bzw. $\text{spr}(H) < 1$. Jacobi- und Gauß-Seidel-Verfahren konvergieren.

Beweis Wir beweisen den Satz für beide Verfahren gleichzeitig. Es sei λ_J ein Eigenwert von J und λ_H ein Eigenwert von H . Mit v_J und v_H bezeichnen wir zugehörige normierte Eigenvektoren $\|v_J\|_\infty = \|v_H\|_\infty = 1$. Dann gilt

$$\lambda_J v_J = J v_J = -D^{-1}(L+R)v_J$$

und für das Gauß-Seidel-Verfahren

$$\lambda_H v_H = H v_H = -(D+L)^{-1}R v_H \Leftrightarrow (D+L)\lambda_H v_H = -R v_H,$$

also

$$\lambda_H v_H = -D^{-1}(R + L\lambda_H)v_H.$$

Für das Jacobi-Verfahren folgt mit $\|v_J\|_\infty = 1$, dass

$$|\lambda_J| \leq \|D^{-1}(L+R)v_J\|_\infty \leq \|D^{-1}(L+R)\|_\infty \leq \max_{j=1, \dots, n} \left\{ \frac{1}{|a_{jj}|} \sum_{k=1, k \neq j}^n |a_{jk}| \right\} < 1,$$

d. h., alle Eigenwerte sind betragsmäßig kleiner 1 und das Jacobi-Verfahren konvergiert. Für das Gauß-Seidel-Verfahren gilt

$$\begin{aligned} |\lambda_H| &\leq \|D^{-1}(\lambda_H L + R)\|_\infty \|v_H\|_\infty = \|D^{-1}(\lambda_H L + R)\|_\infty \\ &\leq \max_{j=1,\dots,n} \left\{ \frac{1}{|a_{jj}|} \left(\sum_{k<j} |\lambda_H| |a_{jk}| + \sum_{k>j} |a_{jk}| \right) \right\}. \end{aligned} \quad (3.16)$$

Wir zeigen $|\lambda_H| < 1$ über ein Widerspruchsargument. Angenommen $|\lambda_H| \geq 1$. Dann gilt

$$\begin{aligned} &\max_{j=1,\dots,n} \left\{ \frac{1}{|a_{jj}|} \left(\sum_{k<j} |\lambda_H| |a_{jk}| + \sum_{k>j} |a_{jk}| \right) \right\} \\ &= |\lambda_H| \max_{j=1,\dots,n} \left\{ \frac{1}{|a_{jj}|} \left(\sum_{k<j} |a_{jk}| + \frac{1}{|\lambda_H|} \sum_{k>j} |a_{jk}| \right) \right\} \\ &\leq |\lambda_H| \max_{j=1,\dots,n} \left\{ \frac{1}{|a_{jj}|} \left(\sum_{k<j} |a_{jk}| + \sum_{k>j} |a_{jk}| \right) \right\} = |\lambda_H| \underbrace{\|D^{-1}(L + R)\|_\infty}_{<1}. \end{aligned}$$

Jetzt führt (3.16) und die strikte Diagonaldominanz zum Widerspruch:

$$|\lambda_H| \leq |\lambda_H| \|D^{-1}(L + R)\|_\infty < |\lambda_H|$$

Es gilt notwendigerweise $|\lambda_H| < 1$. □

Dieses Kriterium ist einfach zu überprüfen und erlaubt sofort eine Einschätzung, ob Jacobi- und Gauß-Seidel-Verfahren bei einer gegebenen Matrix konvergieren. Es zeigt sich jedoch, dass die strikte Diagonaldominanz eine zu starke Forderung darstellt. Als Beispiel nehmen wir die Modellmatrix des Laplace-Operators, welche im Zuge einer Minimalflächenapproximation in Abschn. 3.8 auftritt. Die Matrix hat die Form

$$A = \begin{pmatrix} B & -I & & \\ -I & B & \ddots & \\ & \ddots & \ddots & -I \\ & & -I & B \end{pmatrix}, \quad B = \begin{pmatrix} 4 & -1 & & \\ -1 & 4 & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 4 \end{pmatrix}, \quad I = \begin{pmatrix} 1 & & & \\ & \ddots & & \\ & & \ddots & \\ & & & 1 \end{pmatrix}$$

und ist nur diagonaldominant (siehe Definition 3.15), jedoch nicht strikt diagonaldominant. Es zeigt sich aber, dass sowohl Jacobi- als auch Gauß-Seidel-Verfahren dennoch konvergieren. Zur Abschwächung der Voraussetzungen definieren wir wie folgt:

► **Definition 3.44 (Irreduzibilität)** Eine Matrix $A \in \mathbb{R}^{n \times n}$ heißt irreduzibel, wenn es keine Permutationsmatrix P gibt, sodass die Matrix A durch Spalten und Zeilen in eine Block-Dreiecksmatrix zerfällt

$$PAP^T = \begin{pmatrix} \tilde{A}_{11} & 0 \\ \tilde{A}_{21} & \tilde{A}_{22} \end{pmatrix}$$

mit den Matrizen $\tilde{A}_{11} \in \mathbb{R}^{p \times p}$, $\tilde{A}_{22} \in \mathbb{R}^{q \times q}$, $\tilde{A}_{21} \in \mathbb{R}^{q \times p}$, mit $p, q > 0$, $p + q = n$.

Ob eine gegebene Matrix $A \in \mathbb{R}^{n \times n}$ irreduzibel ist, lässt sich nicht unmittelbar bestimmen. Oft hilft das folgende äquivalente Kriterium, welches einfacher zu überprüfen ist:

Satz 3.45 (Irreduzibilität) Eine Matrix $A \in \mathbb{R}^{n \times n}$ ist genau dann irreduzibel, falls der zugehörige *gerichtete Graph*

$$\mathcal{G}(A) = \{\text{Knoten: } \{1, 2, \dots, n\}, \text{Kanten: } \{(i, j), a_{ij} \neq 0\}\}$$

zusammenhängend ist. Das heißt: Zu zwei beliebigen *Knoten* i und j existiert ein Pfad $(i, i_1) = (i_0, i_1), (i_1, i_2), \dots, (i_{m-1}, i_m) = (i_{m-1}, j)$ mit Kante $(i_{k-1}, i_k) \in \mathcal{G}(A)$.

Für den Beweis verweisen wir auf [71]. Nach dieser alternativen Charakterisierung bedeutet die Irreduzibilität, dass zu je zwei Indizes i, j ein Pfad $i = i_0 \mapsto i_1 \mapsto \dots \mapsto i_m = j$ besteht, sodass $a_{i_{k-1}, i_k} \neq 0$ ist. Anschaulich entspricht dies einem abwechselnden Springen in Zeilen und Spalten der Matrix A , wobei nur Einträge ungleich null getroffen werden dürfen.

Satz 3.46 (Schwachtes Zeilensummenkriterium) Die Matrix $A \in \mathbb{R}^{n \times n}$ sei irreduzibel und es gelte das *schwache Zeilensummenkriterium*, d. h., die Matrix A sei diagonaldominant

$$|a_{jj}| \geq \sum_{k=1, k \neq j}^n |a_{jk}|, \quad j = 1, \dots, n,$$

und in mindestens einer Zeile $r \in \{1, \dots, n\}$ gelte starke Diagonaldominanz

$$|a_{rr}| > \sum_{k=1, k \neq r}^n |a_{rk}|.$$

Dann ist A regulär und es gilt $\text{spr}(J) < 1$ bzw. $\text{spr}(H) < 1$. Das heißt, Jacobi- und Gauß-Seidel-Verfahren konvergieren.

Beweis (i) Durchführbarkeit der Verfahren: Aufgrund der Irreduzibilität von A gilt notwendigerweise

$$\sum_{k=1}^n |a_{jk}| > 0, \quad j = 1, \dots, n.$$

Wegen der vorausgesetzten Diagonaldominanz folgt dann hieraus $a_{jj} \neq 0$ für $j = 1, 2, \dots, n$.

(ii) Zeige $\text{spr}(J) \leq 1$ und $\text{spr}(H) \leq 1$: Diese Aussage erhalten wir entsprechend zum Vorgehen im Beweis zu Satz 3.43. Es bleibt zu zeigen, dass kein Eigenwert den Betrag eins hat.

(iii) Nachweis, dass $|\lambda| < 1$: Angenommen, es gebe einen Eigenwert $\lambda \in \sigma(J)$ mit $|\lambda| = 1$. Es sei $v \in \mathbb{C}^n$ der zugehörige normierte Eigenvektor mit $\|v\|_\infty = 1$. Insbesondere gelte $|v_s| = \|v\|_\infty = 1$ für ein $s \in \{1, \dots, n\}$. Dann erhalten wir aufgrund der Struktur der Iterationsmatrix (hier nur explizit für das Jacobi-Verfahren)

$$J = -D^{-1}(L + R) = \begin{pmatrix} 0 & \frac{-a_{12}}{a_{11}} & \frac{-a_{13}}{a_{11}} & \dots & \frac{-a_{1n}}{a_{11}} \\ \frac{-a_{21}}{a_{22}} & 0 & \frac{-a_{23}}{a_{22}} & \dots & \frac{-a_{2n}}{a_{22}} \\ \vdots & & \ddots & & \vdots \\ \frac{-a_{n1}}{a_{nn}} & \dots & \dots & & 0 \end{pmatrix}$$

die folgende Abschätzung

$$|v_i| = \underbrace{|\lambda|}_{=1} |v_i| = |(Av)_i| \leq \sum_{k \neq i} \frac{|a_{ik}|}{|a_{ii}|} |v_k| \leq \sum_{k \neq i} \frac{|a_{ik}|}{|a_{ii}|} \leq 1, \quad i = 1, 2, \dots, n, \quad (3.17)$$

wobei wir die Struktur von J in der ersten Ungleichung, $|v_i| \leq \|v\|_\infty = 1$ in der zweiten und die schwache Diagonaldominanz in der letzten Abschätzung verwendet haben.

Aufgrund der Irreduzibilität gibt es zu je zwei Indizes s, r stets eine Kette von Indizes $i_1 \mapsto i_2 \mapsto \dots \mapsto i_m$ mit $i_m \in \{1, \dots, n\}$, sodass

$$a_{s,i_1} \neq 0, a_{i_1,i_2} \neq 0, \dots, a_{i_m,r} \neq 0.$$

Durch mehrfache Anwendung von (3.17) folgt der Widerspruch (nämlich dass $|\lambda| = 1$):

$$\begin{aligned} |v_r| &= |\lambda v_r| \leq \frac{1}{|a_{rr}|} \sum_{k \neq r} |a_{rk}| \|v\|_\infty < \|v\|_\infty \quad (\text{strikte DD in einer Zeile}), \\ |v_{i_m}| &= |\lambda v_{i_m}| \leq \frac{1}{|a_{i_m,i_m}|} \left[\sum_{k \neq i_m,r} |a_{i_m,k}| \|v\|_\infty + |a_{i_m,r}| |v_r| \right] < \|v\|_\infty, \\ &\vdots \end{aligned}$$

$$|v_{i_1}| = |\lambda v_{i_1}| \leq \frac{1}{|a_{i_1, i_1}|} \left[\sum_{k \neq i_1, i_1} |a_{i_1, k}| \|v\|_\infty + |a_{i_1, i_2}| |v_{i_2}| \right] < \|v\|_\infty,$$

$$\|v\|_\infty = |\lambda v_s| \leq \frac{1}{|a_{ss}|} \left[\sum_{k \neq s, i_1} |a_{s, k}| \|v\|_\infty + |a_{s, i_1}| |v_{i_1}| \right] < \|v\|_\infty.$$

Daher muss $\text{spr}(J) < 1$. (In den Abschätzungen gilt: DD = Diagonaldominanz.)

Mit analoger Schlussfolgerung wird $\text{spr}(H) < 1$ bewiesen. Hierzu nutzt man ebenfalls die spezielle Struktur der Iterationsmatrix H sowie die umgekehrte Dreiecksungleichung ($|a - b| \geq ||a| - |b||$), um (3.17) zu erhalten. Die restliche Argumentation erfolgt dann auf analogem Wege. $\square$

Mit diesem Satz kann die Konvergenz von Jacobi- sowie Gauß-Seidel-Verfahren für die Modellmatrix nachgewiesen werden. Denn diese ist strikt diagonaldominant.

Beispiel 3.47 (Jacobi- und Gauß-Seidel-Verfahren bei der Modellmatrix)

Wir betrachten das lineare Gleichungssystem $Ax = b$ mit der Modellmatrix $A \in \mathbb{R}^{n \times n}$

$$A = \begin{pmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{pmatrix} \quad (3.18)$$

sowie der rechten Seite $b \in \mathbb{R}^n$ mit $b = (1, \dots, 1)^T$. Zu i, j mit $i < j$ gilt

$$a_{i,i} \neq 0 \rightarrow a_{i,i+1} \neq 0 \rightarrow a_{i+1,i+2} \neq 0 \rightarrow a_{i+2,i+3} \neq 0 \rightarrow \dots \rightarrow a_{j-1,j} \neq 0.$$

Die Matrix ist also irreduzibel, des Weiteren diagonaldominant und in erster und letzter Zeile auch stark diagonaldominant. Jacobi- und Gauß-Seidel-Verfahren konvergieren. Experimentell bestimmen wir für die Problemgröße $n = 10 \cdot 2^k$ für $k = 2, 3, \dots$ die Anzahl der notwendigen Iterationsschritte sowie die Rechenzeit (Core i7-Prozessor 2011) zur Approximation des Gleichungssystems mit einer Fehlertoleranz $\|x^k - x\| < 10^{-4}$.

Matrixgröße	Jacobi		Gauß-Seidel	
	Schritte	Zeit (s)	Schritte	Zeit (s)
80	9453	0.02	4727	0.01
160	37 232	0.13	18 617	0.06
320	147 775	0.92	73 888	0.43
640	588 794	7.35	294 398	3.55
1 280	2 149 551	58	1 074 776	29
2 560	8 590 461	466	4 295 231	233

Es zeigt sich, dass für das Gauß-Seidel-Verfahren stets halb so viele Iterationsschritte notwendig sind wie für das Jacobi-Verfahren. Weiter steigt die Anzahl der notwendigen Iterationsschritte mit der Matrixgröße n . Bei doppelter Matrixgröße steigt die Anzahl der Iterationen etwa um den Faktor 4. Der Zeitaufwand steigt noch stärker mit einem Faktor von etwa 8, da jeder einzelne Schritt einen größeren Aufwand bedeutet. Diesen Zusammenhang zwischen Matriceigenschaft und Konvergenzgeschwindigkeit werden wir später genauer untersuchen.

Wir merken hier noch an, dass Jacobi- und Gauß-Seidel-Verfahren effizient unter Ausnutzung der dünnen Besetzungsstruktur programmiert wurden. Dennoch steigt der zeitliche Aufwand zur Approximation mit vorgegebener Genauigkeit mit dritter Ordnung in der Problemgröße n . ◀

3.7.2 Relaxationsverfahren: das SOR-Verfahren

Das vorangehende Beispiel zeigt, dass Jacobi- sowie Gauß-Seidel-Verfahren sehr langsam konvergieren. Für die Modellmatrix $A \in \mathbb{R}^{n \times n}$ steigt die Anzahl der notwendigen Iterationsschritte (zum Erreichen einer vorgegebenen Genauigkeit) quadratisch mit $O(n^2)$. Obwohl jeder einzelne Schritt sehr einfach ist und äußerst effizient in $O(n)$ Operationen durchgeführt werden kann, sind diese Verfahren den direkten nicht überlegen. Oft, etwa bei Tridiagonalsystemen, sind direkte Löser mit einem Aufwand von $O(n)$ sogar unschlagbar schnell.

Das SOR-Verfahren ist eine Weiterentwicklung der Gauß-Seidel-Iteration durch die Einführung eines *Relaxationsparameters* $\omega > 0$. Das i -te Element berechnet sich laut Satz 3.42 als

$$x_i^{k,GS} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{k,GS} - \sum_{j > i} a_{ij} x_j^{k-1} \right), \quad i = 1, \dots, n.$$

Zur Bestimmung der SOR-Lösung verwenden wir nicht unmittelbar diese Approximation $x_i^{k,GS}$, sondern führen einen Relaxationsparameter $\omega > 0$ ein und definieren

$$x_i^{k,SOR} = \omega x_i^{k,GS} + (1 - \omega) x_i^{k-1}, \quad i = 1, \dots, n$$

als einen gewichteten Mittelwert zwischen Gauß-Seidel-Iteration und alter Approximation. Dieser Relaxationsparameter ω kann nun verwendet werden, um die Konvergenzeigenschaften der Iteration wesentlich zu beeinflussen. Im Fall $\omega = 1$ ergibt sich gerade das Gauß-Seidel-Verfahren. Im Fall $\omega < 1$ spricht man von *Unterrelaxation*, im Fall $\omega > 1$ von *Überrelaxation*. „SOR“ steht für *successive over relaxation*, verwendet also Relaxationsparameter $\omega > 1$. „Successive“ (also schrittweise) bedeutet, dass die Relaxation für jeden einzelnen Index angewendet wird. Die Relaxation kann effizient in Indexschreibweise für $i = 1, \dots, n$ in die Iteration integriert werden

$$x_i^{k,\text{SOR}} = \omega \frac{1}{a_{ii}} \left(b_i - \sum_{j<i} a_{ij} x_j^{k,\text{SOR}} - \sum_{j>i} a_{ij} x_j^{k-1} \right) + (1 - \omega) x_i^{k-1}.$$

In Vektorschreibweise gilt

$$x^{k,\text{SOR}} = \omega D^{-1} (b - Lx^{k,\text{SOR}} - Rx^{k-1}) + (1 - \omega)x^{k-1}.$$

Trennen der Terme nach $x^{k,\text{SOR}}$ sowie x^{k-1} ergibt

$$(D + \omega L)x^{k,\text{SOR}} = \omega b + [(1 - \omega)D - \omega R]x^{k-1},$$

also die Iteration

$$x^{k,\text{SOR}} = H_\omega x^{k-1} + \omega [D + \omega L]^{-1} b, \quad H_\omega := [D + \omega L]^{-1} [(1 - \omega)D - \omega R].$$

Dieses Verfahren mit der Iterationsmatrix H_ω passt wieder in das Schema der allgemeinen Fixpunktiterationen und gemäß Satz 3.38 hängt die Konvergenz des Verfahrens von der Bedingung $\rho_\omega := \text{spr}(H_\omega) < 1$ ab.

Die Schwierigkeit bei der Realisierung des SOR-Verfahrens ist die Bestimmung von guten Relaxationsparametern, sodass die Matrix H_ω einen möglichst kleinen Spektralradius besitzt.

Satz 3.48 (Relaxationsparameter des SOR-Verfahrens) Es sei $A \in \mathbb{R}^{n \times n}$ mit regulärem Diagonalteil $D \in \mathbb{R}^{n \times n}$. Dann gilt

$$\text{spr}(H_\omega) \geq |\omega - 1|, \quad \omega \in \mathbb{R}.$$

Für $\text{spr}(H_\omega) < 1$ muss deshalb $\omega \in (0, 2)$ gelten.

Beweis Wir nutzen die Matrixdarstellung der Iteration

$$H_\omega = [D + \omega L]^{-1} [(1 - \omega)D - \omega R] = (I + \underbrace{\omega D^{-1}L}_{=: \tilde{L}})^{-1} \underbrace{D^{-1}D}_{=: I} [(1 - \omega)I - \omega \underbrace{D^{-1}R}_{=: \tilde{R}}].$$

Die Matrizen $\tilde{L}$ sowie $\tilde{R}$ sind echte Dreiecksmatrizen mit Nullen auf der Diagonale. Das heißt, es gilt $\det(I + \omega\tilde{L}) = 1$ sowie $\det((1 - \omega)I - \omega\tilde{R}) = (1 - \omega)^n$. Nun gilt für die Determinante von H_ω

$$\det(H_\omega) = (1 - \omega)^n.$$

Für die Eigenwerte λ_i von H_ω gilt folglich

$$\begin{aligned} \prod_{i=1}^n \lambda_i &= \det(H_\omega) = (1 - \omega)^n \\ \Rightarrow \operatorname{spr}(H_\omega) &= \max_{1 \leq i \leq n} |\lambda_i| \geq \left(\prod_{i=1}^n |\lambda_i| \right)^{\frac{1}{n}} = |1 - \omega|. \end{aligned}$$

Die letzte Abschätzung nutzt, dass das geometrische Mittel von n Zahlen kleiner ist, als das Maximum. $\square$

Dieser Satz liefert eine erste Abschätzung für die Wahl des Relaxationsparameters, hilft jedoch noch nicht beim Bestimmen eines optimalen Werts. Für die wichtige Klasse von positiv definiten Matrizen kann gezeigt werden, dass das SOR-Verfahren für alle Werte $\omega \in (0, 2)$ konvergiert.

Für die oben angegebene Modellmatrix ist die Konvergenz von Jacobi- sowie Gauß-Seidel-Iteration auch theoretisch abgesichert. Für symmetrisch positiv definite Matrizen kann für die Matrix des Jacobi- J und Gauß-Seidel-Verfahrens H_1 der folgende Zusammenhang gezeigt werden:

$$\operatorname{spr}(J)^2 = \operatorname{spr}(H_1) \quad (3.19)$$

Ein Schritt der Gauß-Seidel-Iteration führt zu der gleichen Fehlerreduktion wie zwei Schritte der Jacobi-Iteration. Dieses Resultat findet sich in Beispiel 3.47 exakt wieder. Weiter kann für diese Matrizen ein Zusammenhang zwischen Eigenwerten der Matrix H_ω sowie den Eigenwerten der Matrix J hergeleitet werden. Angenommen, es gilt $\rho_J := \operatorname{spr}(J) < 1$. Dann gilt für den Spektralradius der SOR-Matrix

$$\operatorname{spr}(H_\omega) = \begin{cases} \omega - 1 & \omega \geq \omega_{\text{opt}}, \\ \frac{1}{4}(\rho_J \omega + \sqrt{\rho_J^2 \omega^2 - 4(\omega - 1)})^2 & \omega \leq \omega_{\text{opt}}. \end{cases}$$

Ist der Spektralradius der Matrix J bekannt, so kann der optimale Parameter ω_{opt} als Schnittpunkt dieser beiden Funktionen gefunden werden, siehe Abb. 3.7. Es gilt

$$\omega_{\text{opt}} = \frac{2 \left(1 - \sqrt{1 - \rho_J^2} \right)}{\rho_J^2}. \quad (3.20)$$

Beispiel 3.49 (Modellmatrix mit SOR-Verfahren)

Wir betrachten wieder die vereinfachte Modellmatrix aus Beispiel 3.47. Für die Matrix $J = -D^{-1}(L + R)$ gilt

$$J = \begin{pmatrix} 0 & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ & \ddots & \ddots & \ddots & \\ & & \frac{1}{2} & 0 & \frac{1}{2} \\ & & & \frac{1}{2} & 0 \end{pmatrix}.$$

Zunächst bestimmen wir die Eigenwerte λ_i und Eigenvektoren w_i für $i = 1, \dots, n$ dieser Matrix. Hierzu machen wir den Ansatz

$$w^i = (w_k^i)_{k=1,\dots,n}, \quad w_k^i = \sin\left(\frac{\pi i k}{n+1}\right).$$

Dann gilt mit dem Additionstheorem $\sin(x \pm y) = \sin(x) \cos(y) \pm \cos(x) \sin(y)$

$$\begin{aligned} (Jw^i)_k &= \frac{1}{2}w_{k-1}^i + \frac{1}{2}w_{k+1}^i = \frac{1}{2} \left(\sin\left(\frac{\pi i(k-1)}{n+1}\right) + \sin\left(\frac{\pi i(k+1)}{n+1}\right) \right) \\ &= \frac{1}{2} \sin\left(\frac{\pi i k}{n+1}\right) \left(\cos\left(\frac{\pi i}{n+1}\right) + \cos\left(\frac{\pi i}{n+1}\right) \right) = w_k^i \cos\left(\frac{\pi i}{n+1}\right). \end{aligned}$$

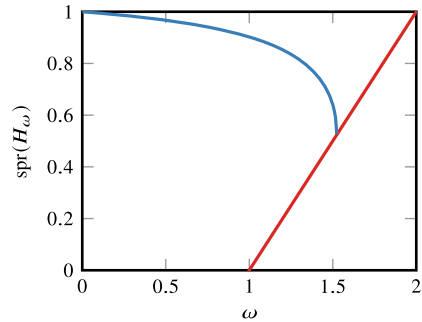
Man beachte, dass diese Gleichung wegen $w_0^i = w_{n+1}^i = 0$ auch für die erste und letzte Zeile, d.h. für $i = 1$ sowie $i = n$ gültig ist. Es gilt $\lambda_i = \cos(\pi i/(n+1))$ und der betragsmäßig größte Eigenwert von J wird für $i = 1$ sowie $i = n$ angenommen. Hier gilt mit der Reihenentwicklung des Kosinus

$$\lambda_{\max} = \lambda_1 = \cos\left(\frac{\pi}{n+1}\right) = 1 - \frac{\pi^2}{2(n+1)^2} + O\left(\frac{1}{(n+1)^4}\right).$$

Der größte Eigenwert geht mit $n \rightarrow \infty$ quadratisch gegen 1. Hieraus bestimmen wir mit (3.20) für einige Schrittweiten aus Beispiel 3.47 die optimalen Relaxationsparameter:

n	$\lambda_{\max}(J)$	ω_{opt}
320	0.9999521084	1.980616162
640	0.9999879897	1.990245664
1280	0.9999969927	1.995107064

Abb. 3.7 Optimaler Relaxationsparameter ω_{opt} in Abhängigkeit des Spektralradius



Schließlich führen wir für diese Parameter das SOR-Verfahren mit optimalem Relaxationsparameter durch und fassen die Ergebnisse in folgender Tabelle zusammen:

Matrixgröße	Jacobi		Gauß-Seidel		SOR	
	Schritte	Zeit (s)	Schritte	Zeit (s)	Schritte	Zeit (s)
320	147 775	0.92	73 888	0.43	486	$\ll 1$
640	588 794	7.35	294 398	3.55	1034	0.02
1280	2 149 551	58	1 074 776	29	1937	0.05
2560	zu aufwendig				4127	0.22
5120					8251	0.90
10240					16 500	3.56

Die Anzahl der notwendigen Schritte steigt beim SOR-Verfahren nur linear in der Problemgröße. Dies ist im Gegensatz zum quadratischen Anstieg beim Jacobi- sowie beim Gauß-Seidel-Verfahren ein wesentlicher Fortschritt. Da der Aufwand eines Schrittes des SOR-Verfahrens mit dem von Jacobi und Gauß-Seidel vergleichbar ist, führt das SOR-Verfahren zu einem Gesamtaufwand von nur $O(n^2)$ Operationen. Dieses positive Resultat gilt jedoch nur dann, wenn der optimale SOR-Parameter bekannt ist. ◀

3.7.3 Praktische Aspekte

Bei der Herleitung der Fixpunktiterationen sind wir von der Darstellung

$$x^{k+1} = x^k + C^{-1}(b - Ax^k) \quad (3.21)$$

ausgegangen. Hier wird klar, dass es sich in der Tat um Fixpunktabbildungen handelt, da das Residuum $d(x) = b - Ax$ im Fall $Ax^k = b$ verschwindet und es somit zu keinem weiteren Update der Näherung kommt. Konvergenz liegt vor, wenn $\rho = \text{spr}(I - C^{-1}A) < 1$ gilt.

Allgemein gilt

$$\|x - x^k\| \leq \rho \|x - x^{k-1}\| \leq \rho^k \|x - x^0\|. \quad (3.22)$$

Für $k \rightarrow \infty$ konvergiert x^k gegen die exakte Lösung des linearen Gleichungssystems. Als exakte Löser werden diese Verfahren jedoch nicht eingesetzt, stattdessen wird die Iteration nach hinreichender Reduktion des Fehlers abgebrochen. Prinzipiell lässt sich auf Basis der Abschätzung (3.22) die Anzahl der notwendigen Schritte zum Erreichen einer Toleranz ϵ_{tol} bestimmen, jedoch ist die Rate ρ im Allgemeinen nicht verfügbar.

Ein gutes Kriterium zum Abbruch der Iteration liefert die Abschätzung aus Satz 3.33 für den Defekt $d^k := b - Ax^k$

$$\frac{\|x^k - x\|}{\|x\|} \leq \text{cond}(A) \frac{\|b - Ax^k\|}{\|b\|}.$$

Hier entsteht jedoch ein ähnliches Problem: Die Konditionszahl der Matrix A ist im Allgemeinen nicht bekannt. Damit kann ebenfalls keine quantitativ korrekte Abschätzung hergeleitet werden. Dieser einfache Zusammenhang zwischen Defekt und Fehler kann jedoch genutzt werden um eine *Fehlerreduktion* zu erreichen:

Bemerkung 3.50 (Fehlerreduktion) Bei der Durchführung von iterativen Lösungsverfahren werden als Abbruchkriterium oft Toleranzen relativ zum Startfehler gefordert. Die Iteration wird gestoppt, falls gilt:

$$\|x^k - x\| \leq \epsilon_{\text{tol}} \|x^0 - x\|$$

Als praktisch durchführbares Kriterium werden die unbekannten Fehler durch die Defekte ersetzt:

$$\|b - Ax^k\| \leq \epsilon_{\text{tol}} \|b - Ax^0\|$$

Wegen $b - Ax^k = A(x - x^k)$ gilt dann allerdings

$$\begin{aligned} \|x^k - x\| &\leq \|A^{-1}\| \|b - Ax^k\| \leq \epsilon_{\text{tol}} \|A^{-1}\| \|b - Ax^0\| \\ &\leq \epsilon_{\text{tol}} \cdot \text{cond}(A) \|x^0 - x\|. \end{aligned}$$

Es kann also eine deutliche Fehleinschätzung des Fehlers vorliegen. ◆

Abschließend diskutieren wir anhand der Modellmatrix die verschiedenen Verfahren im Vergleich:

Beispiel 3.51 (LGS mit der Modellmatrix)

Es sei $A \in \mathbb{R}^{n \times n}$ mit $n = m^2$ die Modellmatrix

$$A = \begin{pmatrix} B & -I & & \\ -I & B & \ddots & \\ & \ddots & \ddots & -I \\ & & -I & B \end{pmatrix}, \quad B = \begin{pmatrix} 4 & -1 & & \\ -1 & 4 & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 4 \end{pmatrix}, \quad I = \begin{pmatrix} 1 & & & \\ & \ddots & & \\ & & \ddots & \\ & & & 1 \end{pmatrix}$$

mit $B, I \in \mathbb{R}^{m \times m}$. Die Matrix A ist eine Bandmatrix mit Bandbreite m . Weiter ist die Matrix A symmetrisch positiv definit. Sie ist irreduzibel und diagonaldominant und erfüllt in den Rändern der Blöcke das starke Zeilensummenkriterium. Alle bisher diskutierten Verfahren können auf die Matrix A angewendet werden. Größter sowie kleinster Eigenwert und Spektralkondition von A berechnen sich zu

$$\lambda_{\min} = \frac{2\pi^2}{n} + O(n^{-2}), \quad \lambda_{\max} = 8 - \frac{2\pi^2}{n} + O(n^{-2}), \quad \kappa \approx \frac{4}{\pi^2} n \approx n.$$

Direkte Verfahren

Die LR-Zerlegung ist (da A positiv definit) ohne Permutierung durchzuführen. Gemäß Satz 3.21 beträgt der Aufwand hierzu $N_{LR} = nm^2 = 4n^2$ Operationen. Die Lösung ist dann bis auf Rundungsfehlereinflüsse exakt gegeben. Alternativ kann die Cholesky-Zerlegung von A erstellt werden. Hierzu sind $N_{LL} = 2n^2$ Operationen notwendig. LR- bzw. Cholesky-Zerlegung sind dicht besetzte Bandmatrizen. Das anschließende Lösen mit Vorwärts- und Rückwärtselimination benötigt weitere $2nm$ Operationen. Wir fassen die notwendigen Operationen in folgender Tabelle zusammen:

$n = m^2$	N_{LR}	N_{LL}
100	$5 \cdot 10^4$	$2 \cdot 10^4$
10000	$5 \cdot 10^9$	$2 \cdot 10^9$
1000000	$5 \cdot 10^{12}$	$2 \cdot 10^{12}$

Bei effizienter Implementierung auf moderner Hardware ist das Gleichungssystem mit 10000 Unbekannten in wenigen Sekunden, das größte Gleichungssystem in wenigen Stunden lösbar. Grundlegend für die effiziente Anwendung direkter Verfahren ist eine Vorsortierung der dünn besetzten Systeme, siehe Abschn. 3.4.

Einfache Fixpunktiterationen

Wir schätzen zunächst für Jacobi- sowie Gauß-Seidel-Verfahren die Spektralradien ab. Mit einem Eigenwert λ und Eigenvektor w von A gilt

$$\begin{aligned} Aw = \lambda w &\Rightarrow Dw + (L + R)w = \lambda w \\ &\Rightarrow -D^{-1}(L + R)w = -D^{-1}(\lambda I - D)w. \end{aligned}$$

Das heißt, wegen $D_{ii} = 4$ gilt

$$Jw = \frac{\lambda - 4}{4}w$$

und die Eigenwerte von J liegen zwischen

$$\begin{aligned} \lambda_{\min}(J) &= \frac{1}{4}(\lambda_{\min}(A) - 4) \approx -1 + \frac{\pi^2}{2n}, \\ \lambda_{\max}(J) &= \frac{1}{4}(\lambda_{\max}(A) - 4) \approx 1 - \frac{\pi^2}{2n}. \end{aligned}$$

Minimaler und maximaler Eigenwert liegen jeweils nahe an -1 bzw. an 1 . Für die Konvergenzrate gilt

$$\rho_J := \text{spr}(J) = 1 - \frac{\pi^2}{2n}$$

und mit (3.19) folgt für das Gauß-Seidel-Verfahren

$$\rho_H := \rho_J^2 \approx 1 - \frac{\pi^2}{n}.$$

Zur Reduktion des Fehlers um einen gegebenen Faktor ϵ sind t Schritte erforderlich:

$$\rho_J^{t_J} = \epsilon \Rightarrow t_J = \frac{\log(\epsilon)}{\log(\rho)} \approx \frac{2}{\pi^2} \log(\epsilon)n, \quad t_H \approx \frac{1}{\pi^2} \log(\epsilon)n.$$

Jeder Schritt hat den Aufwand eines Matrix-Vektor-Produkts, d. h. im gegebenen Fall $5n$. Schließlich bestimmen wir gemäß (3.20) den optimalen SOR-Parameter zu

$$\omega_{\text{opt}} \approx 2 - 2 \frac{\pi}{\sqrt{n}}.$$

Dann gilt

$$\rho_\omega = \omega_{\text{opt}} - 1 \approx 1 - 2 \frac{\pi}{\sqrt{n}}.$$

Hieraus erhalten wir

$$t_\omega \approx \frac{\log(\epsilon)}{2\pi} \sqrt{n}.$$

Der Aufwand des SOR-Verfahrens entspricht dem des Gauß-Seidel-Verfahrens mit einer zusätzlichen Relaxation, d. h. $6n$ Operationen pro Schritt. Wir fassen für die drei Verfahren Konvergenzrate, Anzahl der notwendigen Schritte und Gesamtaufwand zusammen. Dabei ist stets $\epsilon = 10^{-4}$ gewählt:

$n = m^2$	Jacobi	Gauß-Seidel	SOR	
	$t_J \quad N_J$	$t_H \quad N_H$	ω_{opt}	$t_J \quad N_J$
100	180 10^5	90 $5 \cdot 10^4$	1.53	9 10^5
10 000	18 500 10^9	9300 $5 \cdot 10^8$	1.94	142 10^7
1 000 000	1 866 600 10^{13}	933 000 $5 \cdot 10^{12}$	1.99	1460 10^9

Während das größte Gleichungssystem mit dem optimalen SOR-Verfahren in wenigen Sekunden gelöst werden kann, benötigen Jacobi- und Gauß-Seidel-Verfahren etliche Stunden.

Wir werden dieses Beispiel in Abschn. 6.2 als Beispiele 6.7 und 6.10 fortsetzen und noch weitere iterative Verfahren einbeziehen, die auf eine gänzlich andere Art hergeleitet sind.



3.8 Exkurs: Numerische Approximation des Minimalflächenproblems

Das *Minimalflächenproblem* ist eines der klassischen Probleme der Analysis. Gegeben sei eine geschlossene Kurve $\Gamma \subset \mathbb{R}^3$ und gesucht ist eine glatte Fläche $\mathcal{F} \subset \mathbb{R}^3$, welche durch die Kurve Γ begrenzt ist und unter allen Kurven die minimale Fläche hat. Minimalflächen haben viele Anwendungen und entsprechen physikalisch oft dem Prinzip der *Energieminimierung*. Seifenblasen zwischen einem Drahttring sind Näherungen von Minimalflächen. Auch in der Architektur sind für die Dachkonstruktionen des Münchener Olympiastadions (Abb. 3.8) oder des im Bau befindlichen Hauptbahnhofs in Stuttgart Minimalflächen Vorbilder. Für Details zur Theorie und für genauere Definitionen verweisen wir auf die Literatur [6, 58].

3.8.1 Modellierung

Zur *Modellierung*, also mathematischen Beschreibung des Problems suchen wir zunächst einen einfachen Weg zur Beschreibung von Flächen im Raum. Wir beschränken uns zunächst auf Kurven Γ und Flächen $\mathcal{F}$, die sich als Graph einer Funktion $f : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ über dem Bereich $[0, 1]^2$ schreiben lassen:

$$\mathcal{F}(f) = \left\{ \begin{pmatrix} x \\ y \\ f(x, y) \end{pmatrix}, 0 \leq x, y \leq 1 \right\}$$



Abb. 3.8 Das Münchener Olympiastadium

Man nennt dies eine *Parametrisierung* der Fläche. Dies ist eine sehr starke Einschränkung. Zunächst ist die Projektion der Fläche auf die Grundfläche $\mathbb{R}^2$ stets ein einfaches Quadrat. Da die Fläche als Graph gegeben ist, sind darüber hinaus viele Formen ausgeschlossen.

► **Definition 3.52 (Vereinfachtes Minimalflächenproblem)** Es sei $\Omega = (0, 1)^2$ und $\Gamma = \partial\Omega$ der Rand vom Gebiet. Weiter sei $g : \Gamma \rightarrow \mathbb{R}$ eine glatte Funktion zur Definition der Randkurve

$$\Gamma(g) = \left\{ \begin{pmatrix} x \\ y \\ g(x, y) \end{pmatrix}, (x, y) \in \Gamma \right\}.$$

Den Raum der zulässigen Parametrisierungen bezeichnen wir mit

$$\mathcal{X}^g := \{\phi : \bar{\Omega} \rightarrow \mathbb{R}, \phi = g \text{ auf } \Gamma\}.$$

Gesucht ist die Parametrisierung $f \in X^g$, sodass die zugehörige Fläche $\mathcal{F}(f)$ unter allen Flächen mit einer Parametrisierung in X^g den kleinsten Flächeninhalt hat:

$$A(\mathcal{F}(f)) \leq A(\mathcal{F}(h)) \quad \forall h \in \mathcal{X}^g$$

Um das Minimalflächenproblem lösen zu können, müssen wir zunächst zu einer gegebenen Parametrisierung den Flächeninhalt berechnen.

Satz 3.53 (Flächeninhalt einer parametrisierten Fläche) Es sei $f \in C^1(\bar{\Omega})$ die Parametrisierung einer Fläche $\mathcal{F}$. Dann gilt für den Flächeninhalt von $\mathcal{F}$

$$A(\mathcal{F}) = \int_0^1 \int_0^1 \sqrt{1 + f_x(x, y)^2 + f_y(x, y)^2} \, dx \, dy$$

mit den partiellen Ableitungen $f_x(x, y) = \frac{\partial f}{\partial x}(x, y)$ und $f_y(x, y) = \frac{\partial f}{\partial y}(x, y)$.

Beweis Für eine formale Herleitung verweisen wir auf die Literatur [6]. Hier verweisen wir auf Abb. 3.9. Es sei $(x, y) \in \Omega$ ein Punkt auf der Grundfläche und durch dx und dy ein infinitesimales rechteckiges Flächenelement $\Delta_2 := (x, x + dx) \times (y, y + dy) \subset \Omega$ gegeben. Dieses Flächenelement Δ_2 wird mittels f auf eine Fläche $\Delta(f) \subset \Omega$ mit den beiden Tangentialvektoren

$$\mathbf{t}_x(x, y) = \begin{pmatrix} 1 \\ 0 \\ f_x(x, y) \end{pmatrix}, \quad \mathbf{t}_y(x, y) = \begin{pmatrix} 0 \\ 1 \\ f_y(x, y) \end{pmatrix}$$

abgebildet. Dann gilt

$$\frac{|\Delta(f)|}{|\Delta_2|} \rightarrow \|\mathbf{t}_x \times \mathbf{t}_y\| \quad (|\Delta_2| \rightarrow 0).$$

Mit

$$\|\mathbf{t}_x \times \mathbf{t}_y\| = \left\| \begin{pmatrix} -f_x(x, y) \\ -f_y(x, y) \\ 1 \end{pmatrix} \right\| = \sqrt{1 + f_x(x, y)^2 + f_y(x, y)^2}$$

folgt das Ergebnis. □

Damit wir diesen Satz anwenden können, müssen die Funktionen im Raum $\mathcal{X}^g$ mindestens einmal stetig differenzierbar sein.

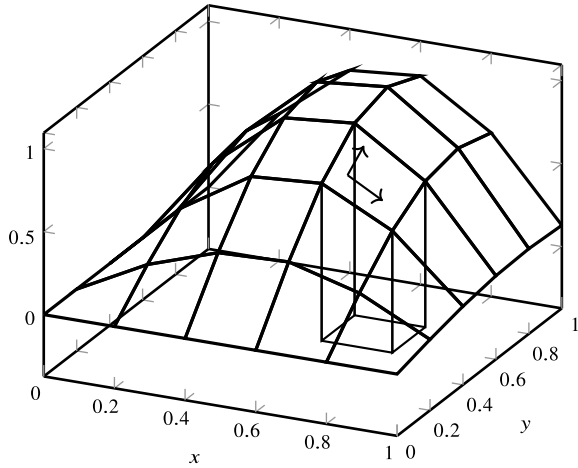
In einem ersten Schritt formulieren wir das Minimalflächenproblem als ein *Variationsproblem*, siehe [58]. Angenommen durch $f \in \mathcal{X}^g$ sei die Parametrisierung einer Minimalfläche gegeben. Dann gilt für jede Funktion $h \in \mathcal{X}^g$

$$A(\mathcal{F}(f)) \leq A(\mathcal{F}(h)) \quad \forall h \in \mathcal{X}^g. \quad (3.23)$$

Für die Differenz gilt

$$\delta f := h - f \in \mathcal{X}^0 := \{\phi \in [0, 1]^2 \rightarrow \mathbb{R}, \phi = 0 \text{ auf } \Gamma\},$$

Abb. 3.9 Berechnung des Flächeninhalts einer parametrisch gegebenen Fläche $f : \Omega \mapsto \mathcal{F}$. Die Tangentialvektoren $\mathbf{t}_x$ und $\mathbf{t}_y$ spannen die infinitesimalen Flächenelemente auf



da alle Funktionen in $\mathcal{X}^g$ die Randwerte erfüllen. Wir können also (3.23) auch schreiben als

$$A(\mathcal{F}(f)) \leq A(\mathcal{F}(f + \delta f)) \quad \forall \delta f \in \mathcal{X}^0.$$

Von der Struktur her ganz ähnlich zu Satz 11.4 gilt nun:

Satz 3.54 (Minimalfläche als Variationsproblem) Es sei $f \in \mathcal{X}^g$ eine Lösung des Minimalflächenproblems, welche zweimal stetig differenzierbar ist, also $f \in C^2(\Omega)$. Dann ist f als Lösung der *partiellen Differentialgleichung*

$$\begin{aligned} & -f_{xx}(x, y) (1 + f_y(x, y)^2) - f_{yy}(x, y) (1 + f_x(x, y)^2) \\ & + 2f_{xy}(x, y) f_x(x, y) f_y(x, y) = 0 \end{aligned}$$

in dem Gebiet $\Omega = (0, 1)^2$ und mit den Randwerten

$$f(x, y) = g(x, y)$$

auf dem Rand $\Gamma = \partial\Omega$ gegeben.

Beweis Es sei $f \in \mathcal{X}^g$ die Parametrisierung einer Minimalfläche. Weiter sei $\phi \in \mathcal{X}^0$ beliebig, aber fest. Dann gilt

$$A(\mathcal{F}(f)) \leq A(\mathcal{F}(f + s\phi)) =: a(s) \quad \forall s \in \mathbb{R}.$$

Die Funktion $a(s)$ hat also im Punkt $s = 0$ ein (lokales) Minimum. Es gilt somit

$$a'(s) \Big|_{s=0} = 0 \quad \forall \phi \in \mathcal{X}^g.$$

Wir berechnen die Ableitung mithilfe von der Formel aus Satz 3.53:

$$\begin{aligned}
 a'(0) &= \frac{\partial}{\partial f} A(\mathcal{F}(f + s\phi)) \Big|_{s=0} \\
 &= \int_0^1 \int_0^1 \frac{2(f_x + s\phi_x)\phi_x + 2(f_y + s\phi_y)f_y\phi_y}{2\sqrt{1 + f_x^2 + f_y^2}} dx dy \Big|_{s=0} \\
 &= \int_0^1 \int_0^1 \frac{f_x\phi_x + f_y\phi_y}{\sqrt{1 + f_x^2 + f_y^2}} dx dy
 \end{aligned}$$

Zur vereinfachten Schreibweise haben wir hier die Argumente „ x “ und „ y “ der Funktionen $f(x, y)$ und $\phi(x, y)$ weggelassen. Mit

$$\gamma(x, y) := \sqrt{1 + f_x(x, y)^2 + f_y(x, y)^2}$$

gilt bei partieller Integration

$$a'(0) = - \int_0^1 \int_0^1 \left(\frac{\partial}{\partial x} \frac{f_x}{\gamma} + \frac{\partial}{\partial y} \frac{f_y}{\gamma} \right) \cdot \phi dx dy + \underbrace{\int_0^1 \frac{f_x\phi}{\gamma} \Big|_0^1 dy + \int_0^1 \frac{f_y\phi}{\gamma} \Big|_0^1 dx}_{=0}.$$

Die Randterme fallen weg, da die Funktionen $\phi \in \mathcal{X}^0$ auf dem Rand Γ alle null sind. Weiter gilt (entsprechend für die y -Ableitung)

$$\frac{\partial}{\partial x} \frac{f_x}{\gamma} = \frac{f_{xx}\gamma - f_x\gamma_x}{\gamma^2} \text{ mit } \gamma_x = \frac{f_x f_{xx} + f_y f_{xy}}{\gamma},$$

d. h. mit $\gamma^2 = 1 + f_x^2 + f_y^2$

$$\frac{\partial}{\partial x} \frac{f_x}{\gamma} + \frac{\partial}{\partial y} \frac{f_y}{\gamma} = \frac{f_{xx}}{\gamma^3} (1 + f_y^2) + \frac{f_{yy}}{\gamma^3} (1 + f_x^2) - \frac{2f_x f_y f_{xy}}{\gamma^3}.$$

Insgesamt ergibt sich für die Ableitung $a'(0)$ die Bedingung

$$a'(0) = \int_0^1 \int_0^1 \left(-f_{xx}(1 + f_y^2) - f_{yy}(1 + f_x^2) + 2f_{xy}f_xf_y \right) \cdot \frac{\phi}{\gamma^3} dx dy \stackrel{!}{=} 0.$$

Die Funktion

$$\tilde{\phi} := \frac{\phi}{\gamma^3}$$

kann beliebig in $C_0^\infty(\Omega) \subset \mathcal{X}^0$ gewählt werden. Durch die Wahl von geeigneten Dirac-Folgen $(\tilde{\phi}_n)_{n \geq 0}$ mit $\phi_n \in C_0^\infty$ gilt der punktweise Zusammenhang

$$-f_{xx}(1 + f_y^2) - f_{yy}(1 + f_x^2) + 2f_{xy}f_xf_y = 0 \text{ in } \Omega \quad (3.24)$$

und $f = g$ auf dem Rand $\Gamma = \partial\Omega$ nach Konstruktion. Dieses Prinzip wird *Hauptsatz der Variationsrechnung* genannt [50, 58]. $\square$

Bemerkung 3.55 (Partielle Differentialgleichungen) Gleichungen vom Typ (3.24) werden *partielle Differentialgleichungen* genannt. Die gesuchte Lösung ist eine Funktion $f : \Omega \rightarrow \mathbb{R}$ und es wird ein funktioneller Zusammenhang zwischen den verschiedenen partiellen Ableitungen beschrieben. Partielle Differentialgleichungen sind oft *Randwertprobleme*. In diesem Exkurs handelt es sich um das *Dirichlet-Problem*, bei dem die Funktionswerte der gesuchten Funktion f auf dem Rand des Gebietes vorgegeben sind. Die Analyse von partiellen Differentialgleichungen, also die Untersuchung von Fragestellungen wie Existenz oder Eindeutigkeit von Lösungen, ist ein großer Bereich der Mathematik [30, 91]. Die numerische Approximation partieller Differentialgleichungen stellt einen Schwerpunkt innerhalb der numerischen Mathematik dar [45]. $\blacklozenge$

Die Differentialgleichung (3.24), welche die Lösung des Minimalflächenproblems beschreibt, ist aufgrund der Nichtlinearität bereits sehr aufwendig in ihrer Analyse und auch in der numerischen Behandlung. Wir werden die Gleichung daher vereinfachen und annehmen, dass die Steigung der Funktion f klein ist, dass also $|\nabla f| \ll 1$ und damit erst recht $|\nabla f|^2 \ll 1$ gilt, d. h., $f_x^2 = f_y^2 \approx 0$ und $f_{xy} \approx 0$. Sehr vereinfacht beschreiben wir daher die Lösung des Minimalflächenproblems durch das folgende lineare Problem:

► **Definition 3.56 (Laplace-Gleichung)** Es sei $\Omega = (0, 1)^2 \subset \mathbb{R}^2$ und g eine auf dem Rand $\Gamma = \partial\Omega$ vorgegebene stetige Funktion. Gesucht ist eine zweimal stetig differenzierbare Funktion $f \in C^2(\bar{\Omega})$ mit

$$-\Delta f(x, y) := -f_{xx}(x, y) - f_{yy}(x, y) = 0 \text{ in } \Omega$$

mit $f(x, y) = g(x, y)$ auf dem Rand Γ . Der *Laplace-Operator*

$$\Delta := \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

ist die Summe der zweiten partiellen Ableitungen.

Die Laplace-Gleichung ist die wichtigste *elliptische partielle Differentialgleichung*. Lösungen der Laplace-Gleichung können (meist vereinfacht) viele Prozesse wie die Temperaturausbreitung, Diffusionsprozesse oder eben das Minimalflächenproblem beschreiben.

3.8.2 Diskretisierung

Das Hauptproblem bei der Lösung oder Approximation von partiellen Differentialgleichungen ist die Anzahl der Unbekannten: Gesucht ist eine Funktion $f : \Omega \rightarrow \mathbb{R}$, d.h., es sind Funktionswerte in den unendlich vielen Punkten des Gebietes Ω gesucht. Um das Problem für eine algorithmische Approximation greifbar zu machen, wird es im ersten Schritt diskretisiert, also durch ein endlich-dimensionales Problem ersetzt. Anstelle einer Funktion $f : \Omega \rightarrow \mathbb{R}$ werden wir die Lösung nur in einigen diskreten Punkten suchen. Wir definieren:

► **Definition 3.57 (Punktgitter)** Es sei $\Omega \subset \mathbb{R}^2$ ein zweidimensionales Gebiet (eine offene, zusammenhängende Teilmenge). Unter einem *Punktgitter*

$$\Omega_h := \{\mathbf{x}_i = (x_i^1, x_i^2) \in \Omega, \quad i = 1, \dots, N\}$$

verstehen wir eine Menge von paarweise verschiedenen Punkten im Gebiet. Die Menge der *Randpunkte* wird mit

$$\Gamma_h := \{\mathbf{x}_i \in \Omega_h, \quad \mathbf{x}_i \in \Gamma := \partial\Omega\}$$

bezeichnet. Unter der *Gitterweite*

$$h_i := \min_{j \neq i} \|\mathbf{x}_i - \mathbf{x}_j\|$$

verstehen wir den minimalen Abstand zum nächstgelegenen Punkt. Die *maximale Gitterweite* ist gegeben als

$$h := \max_{1 \leq i \leq N} h_i.$$

Da wir das einfache Gebiet $\Omega = (0, 1)^2$ betrachten, führen wir mit einer uniformen Gitterweite $h = 1/M$ ein *uniformes Punktgitter* mit $N = (M + 1)^2$ Punkten ein:

$$\Omega_h = \{\mathbf{x}_{ij} := (i \cdot h, j \cdot h), \quad 0 \leq i, j \leq M\} \quad (3.25)$$

Anstelle einer kontinuierlichen Funktion $f : \Omega \rightarrow \mathbb{R}$ suchen wir nun eine Gitterfunktion $f_h : \Omega_h \rightarrow \mathbb{R}$:

► **Definition 3.58 (Gitterfunktion)** Es sei Ω_h ein Punktgitter mit $N \in \mathbb{N}$ Punkten. Unter einer Gitterfunktion $\mathbf{f}_h : \Omega_h \rightarrow \mathbb{R}$ verstehen wir den Vektor

$$(\mathbf{f}_h)_i = f_i, \quad 1 \leq i \leq N.$$

Gesucht sind nun die Werte des Vektors $\mathbf{f}_h$, sodass $f_i \approx f(\mathbf{x}_i)$ eine möglichst gute Näherungslösung des vereinfachten Minimalflächenproblems ist.

Finite-Differenzen-Approximation

Gesucht ist eine Approximation der Differentialgleichung

$$-\Delta f(x, y) = 0 \text{ in } \Omega, \quad f \equiv g \text{ auf } \Gamma. \quad (3.26)$$

Zur Approximation müssen wir die Ableitungen der gesuchten Funktion $f : \Omega \rightarrow \mathbb{R}$ näherungsweise durch die diskreten Werte $\mathbf{f}_h$ ausdrücken. In Abschn. 9.3 werden wir Techniken zur Approximation von Ableitungen näher untersuchen. Auf einem regelmäßigen Punktgitter können Ableitungen einfach durch Differenzenquotienten ausgedrückt werden:

Satz 3.59 (Differenzenquotienten) Es sei $f \in C^4$. Es gilt

$$\begin{aligned} -f_{xx}(x, y) &= \frac{2f(x, y) - f(x+h, y) - f(x-h, y)}{h^2} + O(h^2), \\ -f_{yy}(x, y) &= \frac{2f(x, y) - f(x, y+h) - f(x, y-h)}{h^2} + O(h^2). \end{aligned}$$

Hieraus folgt sofort

$$\begin{aligned} -\Delta f(x, y) &= \frac{4f(x, y) - f(x+h, y) - f(x-h, y) - f(x, y+h) - f(x, y-h)}{h^2} \\ &\quad + O(h^2). \end{aligned}$$

Beweis Der Beweis folgt mithilfe der Taylor-Entwicklung, siehe auch Abschn. 9.3. Es gilt (hierbei genügt es, den skalaren Fall zu betrachten)

$$f(x \pm h) = f(x) \pm hf'(x) + \frac{h^2}{2} f''(x) \pm \frac{h^3}{6} f'''(x) + \frac{h^4}{24} f^{(iv)}(\xi)$$

mit einem Zwischenwert $\xi \in [x-h, x+h]$. Also folgt

$$\frac{-2f(x) + f(x+h) + f(x-h)}{h^2} = f''(x) + O(h^2).$$

Entsprechendes gilt für y und die Approximation des Laplace-Operators setzt sich aus den beiden Richtungsableitungen zusammen. $\square$

Auf dem regelmäßigen Gitter (3.25) können die Ableitungen der partiellen Differentialgleichung (3.26) mithilfe der Differenzenquotienten und der Gitterfunktion $\mathbf{f}_h$ approximiert werden. In jedem der inneren Gitterpunkte (d. h. für $1 \leq i, j \leq M-1$, gilt die Bedingung

$$\frac{1}{h^2} (4f_{ij} - f_{i+1,j} - f_{i-1,j} - f_{i,j+1} - f_{i,j-1}) = 0. \quad (3.27)$$

Auf dem Rand gilt die Gleichung

$$f_{ij} = g(\mathbf{x}_{ij}), \quad i \in \{0, M\} \text{ oder } j \in \{0, M\}. \quad (3.28)$$

Die Gl. (3.27) und (3.28) bilden zusammen ein lineares Gleichungssystem aus $N = (M+1)^2$ Gleichungen und auch $N = (M+1)^2$ Variablen. Die äußeren Punkte, d.h. bei $i = 0$ oder $i = M$, $j = 0$ oder $j = M$ auf dem Rand, sind bereits bekannt. Hier muss kein Gleichungssystem mehr gelöst werden. Bei entsprechender Sortierung der Unbekannten f_{ij} können wir das lineare Gleichungssystem in der kompakten Form

$$\mathbf{A}_h(\mathbf{f}_h) = 0 \quad (3.29)$$

schreiben. Dabei ist $\mathbf{A}_h \in \mathbb{R}^{(M-1)^2 \times (M-1)^2}$ (da wir nur noch die inneren Punkte betrachten). In Abb. 3.10 zeigen wir die sogenannte *lexikografische* Sortierung der inneren Freiheitsgrade.

Bei dieser Wahl der Anordnung der Freiheitsgrade ergibt sich die folgende Blockgestalt der Matrix, die wir schon in den Kap. 3.4 und 3.7.1 als *Modellmatrix* kennengelernt haben:

$$\mathbf{A}_h = \begin{pmatrix} \mathbf{B} & -\mathbf{I} & & \\ -\mathbf{I} & \mathbf{B} & \ddots & \\ & \ddots & \ddots & -\mathbf{I} \\ & & -\mathbf{I} & \mathbf{B} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 4 & -1 & & \\ -1 & 4 & \ddots & \\ & \ddots & \ddots & -1 \\ & & -1 & 4 \end{pmatrix}, \quad \mathbf{I} = \begin{pmatrix} 1 & & & \\ & \ddots & & \\ & & \ddots & \\ & & & 1 \end{pmatrix}$$

mit $\mathbf{B}, \mathbf{I} \in \mathbb{R}^{(M-1) \times (M-1)}$ und somit $\mathbf{A}_h \in \mathbb{R}^{N \times N}$ wobei $N := (M-1)^2$. Es gilt:

Satz 3.60 (Modellmatrix) Die Modellmatrix $\mathbf{A}_h \in \mathbb{R}^{N \times N}$ mit $N = (M-1)^2$ ist eine Bandmatrix mit Bandbreite $M-1$. Sie ist irreduzibel, diagonaldominant, erfüllt das starke

					$x_{(M-1)^2}$
x_M					$x_{2(M-1)}$
x_1	x_2				x_{M-1}

Abb. 3.10 Lexikografische Anordnung der Freiheitsgrade auf einem uniformen Punktgitter. Die Anordnung der Freiheitsgrade ändert nicht die Lösung des linearen Gleichungssystems, sie hat aber eine große Auswirkung auf die Matrixstruktur. Unterschiedliche Sortierungen der Freiheitsgrade entsprechen einer gleichzeitigen Zeilen- und Spaltenpermutierung der resultierenden Matrix

Zeilensummenkriterium (siehe Satz 3.43), ist symmetrisch positiv definit. Für ihre Eigenwerte und Eigenvektoren gilt für $k = (r, s)$ mit $r, s \in \{1, \dots, M-1\}$

$$\begin{aligned} (\omega^{r,s})_{i,j} &= \sin\left(\frac{\pi r i}{M}\right) \sin\left(\frac{\pi s j}{M}\right), \quad i, j = 1, \dots, M-1 \\ \lambda^{r,s} &= \frac{1}{h^2} \left(4 - 2 \cos\left(\frac{\pi r}{M}\right) - 2 \cos\left(\frac{\pi s}{M}\right)\right). \end{aligned} \quad (3.30)$$

Für die Konditionszahl der Matrix gilt

$$\text{cond}_2(\mathbf{A}_h) = O(N) = O(M^2) = O\left(\frac{1}{h^2}\right).$$

Die Eigenwerte der Matrix können einfach nachgerechnet werden. Man vergleiche mit Beispiel 3.49.

Bemerkung 3.61 (Kondition der Modellmatrix) Die Konditionszahl der Modellmatrix verhält sich wie

$$\text{cond}_2(\mathbf{A}_h) = O\left(\frac{1}{h^2}\right) = O\left(\frac{1}{M^2}\right),$$

d. h., sie wächst quadratisch mit kleiner werdender Gitterweite. In Beispiel 3.42 haben wir die entsprechende eindimensionale Modellmatrix untersucht. Diese entsteht zum Beispiel durch Diskretisierung des eindimensionalen Laplace-Problems

$$-\partial_{xx} f(x) = 0$$

mithilfe der Finite-Differenzen-Methode. Hier ergeben sich – siehe Beispiel 3.49 – sehr ähnliche Eigenwerte und Eigenvektoren. Auch im eindimensionalen Fall hat die Modellmatrix eine Kondition der Ordnung $O(h^{-2})$, d. h. quadratisch in der Gitterweite. Allgemein zeigt sich, dass die quadratische Ordnung nicht etwa mit der quadratischen Approximationsgüte der zentralen Differenzenquotienten oder mit der Dimension des Gebietes Ω zusammenhängt, sondern durch die *Ordnung des Differentialoperators* gegeben ist: Die zweite Ableitung ∂_{xx} und der Laplace-Operator Δ sind Differentialoperatoren zweiter Ordnung. ♦

Numerische Lösung

Wir untersuchen nun Methoden zum Lösen linearer Gleichungssysteme der Art

$$\mathbf{A}_h \mathbf{x}_h = \mathbf{b}_h,$$

wobei $\mathbf{A}_h \in \mathbb{R}^{N \times N}$ mit $N = (M-1)^2$ die Modellmatrix ist, $\mathbf{x}_h \in \mathbb{R}^N$ der Vektor der inneren Freiheitsgrade, also der Unbekannten, und $\mathbf{b}_h \in \mathbb{R}^N$ die rechte Seite. Dieser Vektor kann wieder in einer Blockform angegeben werden:

$$\mathbf{b}_h = \begin{pmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \vdots \\ \mathbf{b}_{M-2} \\ \mathbf{b}_{M-1} \end{pmatrix}, \quad \mathbf{b}_i \in \mathbb{R}^{M-1},$$

wobei für den ersten und letzten Block die Form

$$\mathbf{b}_1 = \frac{1}{h^2} \begin{pmatrix} g_{0,1} + g_{1,0} \\ g_{2,0} \\ \vdots \\ g_{M-2,0} \\ g_{M,1} + g_{M-1,0} \end{pmatrix}, \quad \mathbf{b}_{M-1} = \frac{1}{h^2} \begin{pmatrix} g_{0,M-1} + g_{1,M} \\ g_{2,M} \\ \vdots \\ g_{M-2,M} \\ g_{M,M-1} + g_{M-1,M} \end{pmatrix},$$

und für die mittleren Blöcke die Form

$$\mathbf{b}_i = \frac{1}{h^2} \begin{pmatrix} g_{0,i} \\ 0 \\ \vdots \\ 0 \\ g_{M,i} \end{pmatrix}, \quad i = 1, \dots, M-1,$$

angenommen wird.

Das Lösen des linearen Gleichungssystems kann nun entweder mit direkten Verfahren oder mit iterativen Methoden erfolgen.

Wir betrachten zunächst direkte Methoden. Aufgrund der Bandstruktur und der Symmetrie der Matrix $\mathbf{A}_h$ eignet sich insbesondere die Cholesky-Zerlegung aus Abschn. 3.3. Das lineare Gleichungssystem kann in $O(N^2) = O(h^{-2})$ Operationen gelöst werden.

Wir wissen bereits aus den bisherigen Abschnitten, dass die einfachen iterativen Verfahren wie Gauß-Seidel oder Jacobi nicht konkurrenzfähig sind. Stattdessen untersuchen wir gleich das – für symmetrische Verfahren geeignete – CG-Verfahren aus Abschn. 6.2. Das CG-Verfahren *löst* das Gleichungssystem nicht, sondern erstellt in k Schritten eine Approximation $\mathbf{x}_h^{(k)} \approx \mathbf{x}_h$. Für diese gilt laut Satz 11.10 die Abschätzung

$$\|\mathbf{x}^{(k)} - \mathbf{x}_h\|_{\mathbf{A}_h} \leq 2 \left(\frac{1 - 1/\sqrt{\kappa}}{1 + 1/\sqrt{\kappa}} \right)^k \|\mathbf{x}_h^{(0)} - \mathbf{x}_h\|_{\mathbf{A}_h}.$$

Mit der oben hergeleiteten Konditionszahl $\kappa = O(h^{-2})$ gilt

$$\|\mathbf{x}^{(k)} - \mathbf{x}_h\|_{A_h} \leq |1 - h|^k \|\mathbf{x}^{(0)} - \mathbf{x}_h\|_{A_h}. \quad (3.31)$$

Jede Iteration des CG-Verfahrens erfordert zur Durchführung die Berechnung von einem Matrix-Vektor-Produkt, zwei Skalarprodukten und drei Vektor-Additionen. Der Aufwand pro Schritt beträgt bei der Modellmatrix (mit maximal fünf Einträgen pro Zeile) somit

$$E_{CG} = 1 \cdot 5N + 2 \cdot N + 3 \cdot N = 10N. \quad (3.32)$$

Der Gesamtaufwand hängt nun von der benötigten Anzahl von Schritten ab.

Wenden wir das CG-Verfahren zur Approximation der linearen Gleichungssysteme beim Minimalflächenproblem an, so kombinieren wir zwei numerische Approximationen: die Finite-Differenzen-Methode zur Approximation der exakten Fläche $f(x)$ durch die numerische Approximation $\mathbf{x}_h$ und das CG-Verfahren zur Approximation von $\mathbf{x}_h$ durch $\mathbf{x}_h^{(k)}$. Der Gesamtfehler lässt sich abschätzen als

$$\|f - \mathbf{x}_h^{(k)}\| \leq \|f - \mathbf{x}_h\| + \|\mathbf{x}_h - \mathbf{x}_h^{(k)}\|. \quad (3.33)$$

Um ein gutes Kriterium für die Genauigkeit der CG-Approximation $\|\mathbf{x}_h - \mathbf{x}_h^{(k)}\|$ herzuleiten, benötigen wir somit eine Abschätzung des Finite-Differenzen-Fehlers $\|f - \mathbf{x}_h\|$. Ist dieser sehr groß, so muss auch das lineare Gleichungssystem nicht genau approximiert werden.

Fehlerabschätzung für die Finite-Differenzen-Methode

Wir wollen nun den Fehler zwischen der Lösung des vereinfachten Minimalflächenproblems

$$-\Delta f(x, y) = 0 \text{ in } \Omega, \quad f(x, y) = g(x, y) \text{ auf } \Gamma,$$

und der Finite-Differenzen-Approximation $\mathbf{x}_h$ mit $\mathbf{x}_{i,j} \approx f(x_{i,j})$ analysieren. Zunächst stellt sich die Frage, wie überhaupt der *Fehler* zwischen einer Funktion $f(x, y)$ und einigen diskreten Werten $\mathbf{x}_h$ in den Gitterpunkten gemessen werden kann. Hierzu definieren wir uns die diskrete Gitterfunktion der exakten Werte

$$\mathbf{f}_h \in \mathbb{R}^N, \quad \mathbf{f}_{ij} = f(x_{i,j}),$$

und den Fehler $\mathbf{e}_h \in \mathbb{R}^N$

$$\mathbf{e}_h = \mathbf{f}_h - \mathbf{x}_h.$$

Für diesen erhalten wir sofort

$$\mathbf{e}_h = \mathbf{A}_h^{-1} \mathbf{A}_h \mathbf{e}_h = \mathbf{A}_h^{-1} (\mathbf{A}_h \mathbf{f}_h - \mathbf{b}_h),$$

da $\mathbf{A}_h \mathbf{x}_h = \mathbf{b}_h$ gilt. Wir definieren:

► **Definition 3.62 (Abschneidefehler)** Der *Abschneidefehler* $\tau_h \in \mathbb{R}^N$, auch *Konsistenzfehler* genannt, ist definiert als

$$\tau_h = \frac{1}{h^2} \mathbf{A}_h (\mathbf{f}_h - \mathbf{b}_h).$$

Im Fall

$$\|\tau_h\| \rightarrow 0 \quad (h \rightarrow 0)$$

heißt die Finite-Differenzen-Methode *konsistent* mit der Differentialgleichung.

Der Abschneidefehler misst, ob die exakte Lösung der Differentialgleichung auch die diskrete Gleichung erfüllt. Der Abschneidefehler ist einfach zu berechnen. Es gilt:

Satz 3.63 (Abschneidefehler) Es sei $f \in C^4(\Omega)$ die Lösung der Gleichung

$$-\Delta f(x, y) = b(x, y) \text{ in } \Omega, \quad f(x, y) = 0 \text{ auf } \Gamma.$$

Dann gilt für den Abschneidefehler

$$\max \|\tau_h\| = O(h^2).$$

Beweis Das Ergebnis folgt unmittelbar aus Satz 3.59. Denn mit $\mathbf{f}_{i,j} = f(x_{i,j})$ und $\mathbf{b}_{i,j} = b(x_{i,j})$ ist

$$\begin{aligned} \frac{1}{h^2} (\mathbf{A}_h \mathbf{f}_h)_{ij} &= \frac{4\mathbf{f}_{ij} - \mathbf{f}_{i+1,j} - \mathbf{f}_{i-1,j} - \mathbf{f}_{i,j+1} - \mathbf{f}_{i,j-1}}{h^2} \\ &= -\Delta f(x_{i,j}) + O(h^2) = \mathbf{b}_{i,j} + O(h^2). \end{aligned}$$

Hiermit gilt

$$\tau_{i,j} = O(h^2).$$

□

Man sagt: Die Differenzenmethode ist *konsistent mit Ordnung 2*. Der Konsistenzfehler misst den *lokalen Diskretisierungsfehler*, also den Fehler in einem Punkt $x_{i,j}$. Es wird der Fehler des Differenzenquotienten berücksichtigt, nicht jedoch, dass die verwendeten Funktionswerte, z. B. $f_{i+1,j}$ oder $f_{i,j-1}$, selbst mit einem Fehler behaftet sind. Wir kehren zur Fehlerabschätzung zurück und erhalten z. B. in der Maximumsnorm

$$\|\mathbf{f}_h - \mathbf{x}_h\|_\infty = \|\mathbf{e}_h\|_\infty \leq \|\mathbf{A}_h^{-1}\|_\infty \|\tau_h\|_\infty \leq \|\mathbf{A}_h^{-1}\|_\infty \|\tau_h\|_\infty \leq C \|\mathbf{A}_h^{-1}\|_\infty h^2.$$

Um eine echte Fehlerabschätzung zu erhalten, benötigen wir eine Abschätzung für die Inverse der Matrix $\mathbf{A}_h$. Für unsere Modellmatrix kennen wir die Eigenwerte, vergleiche (3.30), und aus $\lambda_{\min} \geq c > 0$ (unabhängig von $h > 0$) folgt, dass die Eigenwerten der Inversen nach oben beschränkt sind. Es gilt also

$$\|\mathbf{A}_h^{-1}\| \leq C \quad (h \rightarrow 0). \quad (3.34)$$

Wir setzen wieder bei der Abschätzung des Gesamtfehlers (3.33) ein. Mit der eben hergeleiteten Abschätzung für den Fehler der Differenzenapproximation gilt

$$\|f - \mathbf{x}_h^{(k)}\|_\infty \leq Ch^2 + \|\mathbf{x}_h - \mathbf{x}_h^{(k)}\|_\infty \quad (3.35)$$

mit einer unbekannten Konstante $C > 0$. Ein effizienter Einsatz des CG-Verfahrens zur Approximation des Gleichungssystems muss nun dafür sorgen, dass der CG-Fehler nicht den Gesamtfehler dominiert. Auf der anderen Seite lohnt es nicht, das Gleichungssystem zu genau zu lösen, da dann der Diskretisierungsfehler dominiert. Ein sinnvolles Kriterium entspricht einer Äquilibration der beiden Fehleranteile

$$\|\mathbf{x}_h - \mathbf{x}_h^{(k)}\| \leq Ch^2,$$

wobei die Konstante $C > 0$ natürlich nicht bekannt ist. Wir wählen $C = 1$. Entsprechend (3.31) gilt es nun die Iterationszahl $k \in \mathbb{N}$ so zu bestimmen, dass

$$(1 - h)^k \approx h^2 \quad \Leftrightarrow \quad k = \frac{\log(h^2)}{\log(1 - h)} = \frac{1}{h} \log\left(\frac{1}{h^2}\right) + O(\log(h)).$$

Bei $N = 1/h^2$ folgt für die benötigte Anzahl an Iterationen

$$k = \log(N)N^{\frac{1}{2}} + O(\log(h)).$$

Zusammen mit dem Aufwand pro Schritt des CG-Verfahrens (siehe (3.32)) ergibt sich als Gesamtaufwand zur Approximation des Problems

$$E_{CG}(total) = 10N^{\frac{3}{2}} \log\left(N^{-\frac{1}{2}}\right),$$

verglichen mit der direkten Lösung des Cholesky-Verfahrens

$$E_{Cholesky}(total) = N^2.$$

Dieser Unterschied wirkt auf den ersten Blick nicht groß, macht aber bei einer Problemgröße von $M = 1000$, also $N = 10^6$, bereits einen Faktor 150 aus, damit z. B. eine Laufzeit von etwa 10 min statt einem Tag.

3.8.3 Beispiele

Wir betrachten abschließend zwei Beispiele für numerische Realisationen von Minimalflächen. Dabei werden wir untersuchen, wie sich der Fehler der Finite-Differenzen-Diskretisierung verhält, ob wir also Satz 3.62 und die Fehlerabschätzung (3.35) verifizieren können. Weiter überprüfen wir, ob die lineare Approximation, d. h. die Vereinfachung der Minimalflächengleichung (3.54) zur Laplace-Gleichung (Definition 3.56), zulässig ist.

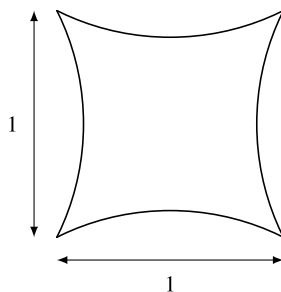


Abb. 3.11 Das Grundgebiet für das Minimalflächenproblem ist ein Quadrat mit abgerundeten Seiten. Die Seiten sind Kreissektoren von Kreisen mit Radius $r = \sqrt{5}/4$ und den Mittelpunkten $x_m^1 = (-1, 1/2)$, $x_m^2 = (2, 1/2)$, $x_m^3 = (1/2, -1)$, $x_m^4 = (1/2, 2)$

Beispiel 1: Musikpavillon

Das erste Beispiel ist dem *Musikpavillon* nachempfunden, welcher 1955 zur Bundesgartenschau in Kassel errichtet wurde,⁴ siehe auch [5]. Dieser ist eines der ersten Beispiele für den Einsatz einer Minimalfläche in der Architektur. Hier betrachten wir vereinfacht als Grundfläche ein Quadrat mit ausgeschnittenen runden Seiten, siehe auch Abb. 3.11:

$$\Omega = (0, 1) \times (0, 1) \setminus \left(\overline{K_1(2, 1/2)} \cup \overline{K_1(1/2, -1)} \cup \overline{K_1(-1, 1/2)} \cup \overline{K_1(1/2, 2)} \right),$$

und schreiben auf dem Rand die gesuchte Auslenkung $g(x, y)$ vor als

$$g(x, y) = \frac{1}{2}|1 - x - y|.$$

In Abb. 3.12 zeigen wir die Lösung. Links in der Abbildung wird das volle nichtlineare Modell, rechts die Laplace-Gleichung betrachtet. Bei diesem Beispiel können keine Unterschiede erkannt werden, die resultierenden Minimalflächen sehen jeweils gleich aus.

Ein Blick auf die erreichte *Minimalfläche* $A(\mathcal{F}(f))$ in Tab. 3.3 zeigt jedoch einen kleinen Unterschied. Betrachten wir das nichtlineare Modell, so ist die resultierende Fläche mit $A \approx 0.73421$ im Gegensatz zu $A \approx 0.73424$ etwas kleiner, die Abweichung beträgt lediglich 0.05 %. Das vereinfachte Modell ist hier eine sehr gute Näherung.

In der Tabelle wird die Lösung des Minimalflächenproblems jeweils auf einer ganzen Gittersequenz für steigende Problemgrößen angegeben. So kann die Konvergenz der Methode, also die Abschätzung (3.35), geprüft werden. Da wir für das Problem die exakte Lösung, also die Funktion $f(x, y)$ nicht kennen, erstellen wir uns zunächst einen *Referenzwert*. Dieser kann auf Basis der numerischen Ergebnisse selbst und mithilfe von numerischer Extrapolation erreicht werden, vergleiche Abschn. 9.4 in diesem Buch. Nach unseren Überlegungen

⁴ Der Musikpavillon in Kassel wurde 1955 von Frei Otto zur Bundesgartenschau entworfen und ist eine der ersten *Minimalflächen* in der Architektur. Später hat Frei Otto die Überdachung des Olympiaparks in München entworfen.

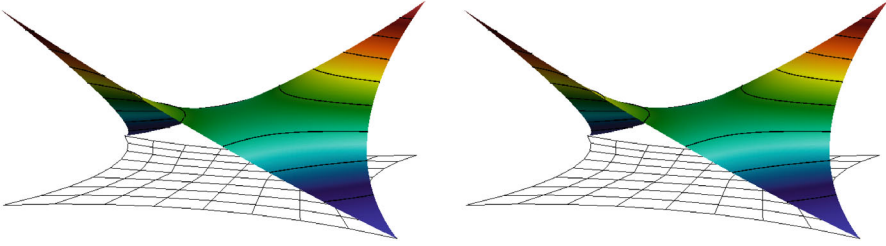


Abb. 3.12 Minimalfläche des Musikpavillons. Links: volles, nichtlineares Modell. Rechts: reduzierte Laplace-Gleichung. Die Linien geben die Niveaulinien zu den Höhen $h_i = i/20$ für $i = 0, \dots, 10$ an. Auf den ersten Blick ist kein Unterschied zwischen den beiden Modellen zu erkennen

gehen wir davon aus, dass der Flächeninhalt der diskreten Näherung $A(\mathcal{F}(f_h))$ gegen den echten Flächeninhalt konvergiert, und machen hierzu den Ansatz

$$A(\mathcal{F}(f)) = A(\mathcal{F}(f_h)) + Ch^\alpha + \mathcal{R} \quad (3.36)$$

mit einer unbekannten Konstanten C , der unbekannten (wir erwarten $\alpha = 2$) Konvergenzordnung und dem unbekannten Flächeninhalt $A(\mathcal{F}(f))$ der exakten Parametrisierung der Fläche $f(x, y)$. Mit $\mathcal{R}$ bezeichnen wir Restterme höherer Ordnung. Die drei Unbekannten C , $A(\mathcal{F}(f))$ und α können wir aus drei aufeinanderfolgenden Werten der Tab. 3.3 ermitteln, wenn wir das Restglied $\mathcal{R}$ vernachlässigen. Da wir nicht an dem Wert der Konstante, sondern nur an Konvergenzordnung α und Flächeninhalt $A(\mathcal{F}(f))$ interessiert sind, können wir die Gitterweiten als $h = 1$, $h = 1/2$ und $h = 1/4$ wählen. Dann ergibt sich

$$\begin{aligned} \alpha &= \log_2 \left(\frac{A(\mathcal{F}(f_h)) - A(\mathcal{F}(f_{h/2}))}{A(\mathcal{F}(f_{h/2})) - A(\mathcal{F}(f_{h/4}))} \right), \\ \tilde{A}(\mathcal{F}(f)) &= \frac{A(\mathcal{F}(f_h))A(\mathcal{F}(f_{h/4})) - A(\mathcal{F}(f_{h/2}))^2}{A(\mathcal{F}(f_h)) - 2A(\mathcal{F}(f_{h/2})) + A(\mathcal{F}(f_{h/4}))}. \end{aligned} \quad (3.37)$$

Diese Formeln können einfach aus Gl. (3.36) hergeleitet werden. Bei entsprechender Regularität der Funktion $A(\mathcal{F}(f_h))$ kann gezeigt werden, dass die *Extrapolation* $\tilde{A}(\mathcal{F}(f))$ wirklich eine bessere Näherung als der beste numerische Wert ist (d. h. als die Näherung zu kleinster Gitterweite). Die theoretischen Grundlagen werden in Abschn. 9.4 erklärt. Hier nutzen wir die Formel zum Erzeugen eines Referenzwerts und zur Angabe von Fehlern in Tab. 3.3. An diesem ersten Beispiel zeigt sich nahezu perfekt die quadratische Konvergenz der numerischen Werte.

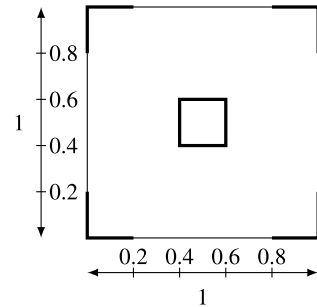
Beispiel 2: Minimalfläche mit reduzierter Regularität

Als zweites Beispiel betrachten wir die Konfiguration aus Abb. 3.13. Die „Minimalfläche“ ist in der Mitte hoch eingespannt und an den vier Ecken eines Gebietes am Boden befestigt. Dazwischen sucht sie sich ihre Position frei.

Tab. 3.3 Berechnung der Minimalfläche des Musikpavillons. Links: Berechnung mit dem vollen, nichtlinearen Modell. Rechts: reduziertes Modell auf Basis der Laplace-Gleichung. Schon bei sehr kleiner Problemgröße können sehr gute Approximationen erreicht werden. In beiden Fällen zeigt sich die theoretisch erwartete quadratische Konvergenz in der Gitterweite $O(h^2)$

N	$A(\mathcal{F}(f_h))$	Fehler	N	$A(\mathcal{F}(f_h))$	Fehler
9	0.82393	0.08972	9	0.82393	0.08969
25	0.75703	0.02282	25	0.75705	0.02281
81	0.73993	0.00572	81	0.73996	0.00572
289	0.73564	0.00143	289	0.73567	0.00143
1 089	0.73457	0.00036	1 089	0.73460	0.00036
4 225	0.73430	0.00009	4 225	0.73433	0.00009
Exakt	0.73421 (2.00)		Exakt	0.73424 (1.98)	

Abb. 3.13 Die Minimalfläche ist in der Mitte in der Höhe $H = 0.5$ eingespannt. In den vier Ecken ist sie am Boden bei $H = 0$ eingespannt. An den übrigen Rändern ist die Fläche frei



Das Ergebnis zeigen wir in Abb. 3.14. Hier zeigt sich ein großer Unterschied zwischen dem vollen nichtlinearen Modell (links) und der vereinfachten Laplace-Gleichung (rechts). Diesmal unterscheidet sich auch der Wert des Flächenfunktional. Beim vollen Modell

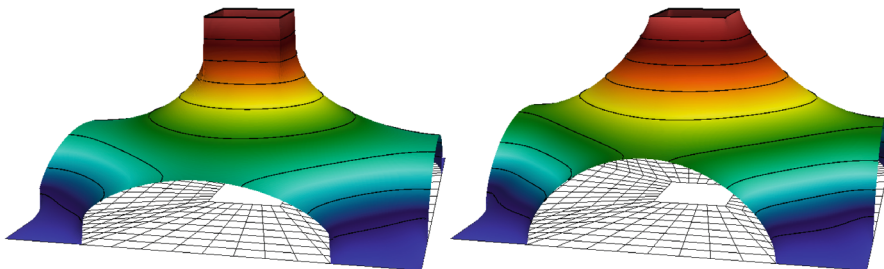


Abb. 3.14 Minimalfläche zu Beispiel 2. Links wurde das volle nichtlineare Modell, rechts die Laplace-Gleichung betrachtet. Gerade in der Nähe der *einspringenden Ecken* in der Mitte des Gebietes zeigen sich sehr große Unterschiede

Tab. 3.4 Konvergenzverhalten bei Beispiel 2. Links das volle, rechts das reduzierte Modell. Der Flächeninhalt beim vollen Modell ist um fast 2 % geringer. Hier liegt nicht mehr die theoretisch erwartete quadratische Konvergenz vor. Beide Verfahren konvergieren nur noch etwa linear in der Gitterweite $O(h)$. Die „exakten“ Werte wurden als Referenzwerte durch Extrapolation ermittelt

N	$A(\mathcal{F}(f_h))$	Fehler	N	$A(\mathcal{F}(f_h))$	Fehler
48	1.3211	0.0951	48	1.3244	0.0734
160	1.2755	0.0495	160	1.2823	0.0312
576	1.2532	0.0272	576	1.2650	0.0140
2 176	1.2410	0.0150	2 176	1.2574	0.0063
8 448	1.2342	0.0082	8 448	1.2539	0.0029
33 280	1.2304	0.0044	33 280	1.2524	0.0013
132 096	1.2284	0.0024	132 096	1.2517	0.0006
Exakt	1.2260 (0.89)		Exakt	1.2511 (1.13)	

ergibt sich die Fläche $A \approx 1.2260$ gegenüber $A \approx 1.2517$ bei der Laplace-Gleichung – eine Abweichung von fast 2 %.

Der größte Unterschied zum ersten Beispiel zeigt sich allerdings in Tab. 3.4. Wir ermitteln wieder Extrapolation $\tilde{A}(\mathcal{F}(f))$ und Konvergenzordnung α mit Formel (3.37). Hier erhalten wir – entgegen der theoretischen Vorhersage – nur noch lineare Konvergenz in der Gitterweite $O(h)$. Es liegt nicht mehr die erwartete quadratische Konvergenzordnung vor. Die Fehler sind weitaus größer als beim ersten Beispiel.

Woran liegt dieses schlechte Abschneiden? Satz 3.62 haben wir bewiesen, die Abschätzung (3.34) können wir hier nicht beweisen, sie ist aber korrekt und das Ergebnis hängt nur von der Verteilung der Gitterpunkte ab, nicht von den Randdaten g . Auch die Gleichungssysteme, linear wie nichtlinear, werden hier genau genug gelöst. Der Grund für das schlechtere Abschneiden bei diesem Beispiel liegt in der Theorie partieller Differentialgleichungen. Es zeigt sich, dass zu diesem Beispiel überhaupt keine Funktion $f \in C^4(\Omega)$ existiert, die entweder die Laplace-Gleichung oder die volle Gleichung der Minimalflächen erfüllt. Analysiert man das Beispiel genauer, so zeigt sich, dass die Ableitungen der Lösung an den vier inneren Ecken nicht beschränkt sind. Man nennt Ecken, die in das Gebiet hineinragen, *einspringende Ecken*. Und an solchen einspringenden Ecken haben Lösungen der Laplace-Gleichung nur eine sehr eingeschränkte Regularität. Auf diesem Gebiet existiert nicht mal eine Lösung, die wenigstens einmal stetig differenzierbar wäre. Satz 3.62 ist nur korrekt für $f \in C^4(\Omega)$. Daher können wir diesen Satz hier nicht anwenden.

Orthogonalisierungsverfahren und die QR-Zerlegung

4

Die LR-Zerlegung einer regulären Matrix $A \in \mathbb{R}^{n \times n}$ in die beiden Dreiecksmatrizen L und R basiert auf der Elimination mit Frobenius-Matrizen, d. h. $R = FA$, mit $L = F^{-1}$. Es gilt:

$$\text{cond}(R) = \text{cond}(FA) \leq \text{cond}(F) \text{cond}(A) = \|F\| \|L\| \text{cond}(A)$$

Bei entsprechender Pivotisierung gilt für alle Einträge der Matrizen F und L die Abschätzung $|f_{ij}| \leq 1$ sowie $|l_{ij}| \leq 1$ (siehe Satz 3.12). Dennoch können die Normen von L und F im Allgemeinen nicht günstiger als $\|F\|_\infty \leq n$ und $\|L\|_\infty \leq n$ abgeschätzt werden. Es gilt damit die pessimistische Abschätzung

$$\text{cond}_\infty(R) \leq n^2 \text{cond}_\infty(A).$$

Die Matrix R , welche zur Rückwärtselimination invertiert werden muss, hat eine unter Umständen weitaus schlechtere Konditionierung als die Matrix A selbst (welche auch schon sehr schlecht konditioniert sein kann).

In diesem Kapitel werden wir mit der QR-Zerlegung einer Matrix A eine stabile Zerlegung einer Matrix $A \in \mathbb{R}^{n \times n}$ in eine rechte obere Dreiecksmatrix $R \in \mathbb{R}^{n \times n}$ und in eine orthogonale Matrix Q analysieren. Die QR-Zerlegung ist stabiler als die LR-Zerlegung und kann zum Lösen linearer Gleichungssysteme herangezogen werden. Darüber hinaus werden wir die QR-Zerlegung auch auf nicht-quadratische Matrizen $A \in \mathbb{R}^{n \times m}$ mit $m > n$ erweitern und sie in Abschn. 4.5 zum Lösen überbestimmter linearer Gleichungssysteme verwenden. Schließlich ist die QR-Zerlegung aber auch die Grundlage von effizienten Algorithmen zur Berechnung von Eigenwerten. Dies behandeln wir in Abschn. 5.4.2.

Über die verschiedenen Einsatzmöglichkeiten der QR-Zerlegung hinaus werden wir die Orthogonalisierung von Vektoren als ein eigenständiges numerisches Problem betrachten.

Der einfachste Algorithmus ist das Gram-Schmidtsche Orthogonalisierungsverfahren, siehe Satz 2.10 in Kap. 2, er wird sich jedoch als numerisch instabil erweisen.

4.1 Die QR-Zerlegung

Wir suchen zu einer Matrix $A \in \mathbb{R}^{n \times n}$ einen Zerlegungsprozess $A = QR$ in eine Dreiecksmatrix $R \in \mathbb{R}^{n \times n}$, welcher numerisch stabil ist, indem die Zerlegung nur mithilfe von Matrizen durchgeführt wird, die gut konditioniert sind. Hierzu bieten sich orthogonale Matrizen an.

► **Definition 4.1 (Orthogonale Matrix)** Eine Matrix $Q \in \mathbb{R}^{n \times n}$ heißt *orthogonal*, falls ihre Zeilen- und Spaltenvektoren eine Orthonormalbasis des $\mathbb{R}^n$ bilden. Es gilt $Q^T Q = I$, also $Q^{-1} = Q^T$.

Orthogonale Matrizen haben die Spektralkondition $\text{cond}_2(Q) = 1$, denn für alle Eigenwerte λ einer orthogonalen Matrix Q zu Eigenvektor ω gilt

$$\|\omega\|_2^2 = (Q^T Q \omega, \omega)_2 = (Q \omega, Q \omega)_2 = (\lambda \omega, \lambda \omega)_2 = |\lambda|^2 \|\omega\|_2^2 \Rightarrow |\lambda| = 1.$$

Bei einer Zerlegung der Form $A = QR$ mit einer orthogonalen Matrix Q gilt also $R = Q^T A$ und somit

$$\text{cond}_2(R) = \text{cond}_2(Q^T A) \leq \text{cond}_2(Q^T) \text{cond}_2(A) = \text{cond}_2(A).$$

Die Dreiecksmatrix R hat höchstens die Kondition der ursprünglichen Matrix A . Wir fassen zunächst einige Eigenschaften orthogonaler Matrizen zusammen.

Satz 4.2 (Orthogonale Matrix) Es sei $Q \in \mathbb{R}^{n \times n}$ eine orthogonale Matrix. Dann ist Q regulär und es gilt

$$Q^{-1} = Q^T, \quad Q^T Q = I, \quad \|Q\|_2 = 1, \quad \text{cond}_2(Q) = 1.$$

1. Es gilt $\det(Q) = 1$ oder $\det(Q) = -1$.
2. Für zwei orthogonale Matrizen $Q_1, Q_2 \in \mathbb{R}^{n \times n}$ ist auch das Produkt $Q_1 Q_2$ eine orthogonale Matrix. Für eine beliebige Matrix $A \in \mathbb{R}^{n \times n}$ gilt $\|Q_1 Q_2 A\|_2 = \|A\|_2$.
3. Für beliebige Vektoren $x, y \in \mathbb{R}^n$ gilt

$$\|Qx\|_2 = \|x\|_2, \quad (Qx, Qy)_2 = (x, y)_2.$$

Beweis Wir beweisen hier nur die im Kontext der numerischen Stabilität wesentliche Eigenschaft, dass die Multiplikation mit orthogonalen Matrizen die Spektralnrm einer Matrix

nicht ändert. Es gilt

$$\|QAx\|_2^2 = (QAx, QAx)_2 = (Q^T QAx, Ax)_2 = (Ax, Ax)_2 = \|Ax\|_2^2.$$

Und es folgt

$$\|QA\|_2 = \sup_{x \neq 0} \frac{\|QAx\|_2}{\|x\|_2} = \sup_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2} = \|A\|_2.$$

□

Wir definieren:

► **Definition 4.3 (QR-Zerlegung)** Die Zerlegung einer Matrix $A \in \mathbb{R}^{n \times n}$ in eine orthogonale Matrix $Q \in \mathbb{R}^{n \times n}$ sowie eine rechte obere Dreiecksmatrix $R \in \mathbb{R}^{n \times n}$ gemäß

$$A = QR$$

heißt *QR-Zerlegung*.

Wir definieren die QR-Zerlegung für allgemeine rechteckige Matrizen. Einmal erstellt, kann die QR-Zerlegung genutzt werden, um lineare Gleichungssysteme mit der Matrix A effizient zu lösen. Die Matrix Q ist zwar - im Gegensatz zur Matrix L der LR-Zerlegung - im Allgemeinen voll besetzt und keine Dreiecksmatrix, ihre Inverse kann jedoch trivial angegeben werden, da

$$Ax = b \Leftrightarrow Q^T Ax = Q^T b \Leftrightarrow Rx = Q^T b.$$

Satz 4.4 (QR-Zerlegung) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix. Dann existiert eine Zerlegung $A = QR$ in eine orthogonale Matrix $Q \in \mathbb{R}^{n \times n}$ und eine rechte obere Dreiecksmatrix $R \in \mathbb{R}^{n \times n}$.

Beweis Es seien $A = (a_1, \dots, a_n)$ die Spaltenvektoren der Matrix A . Da A regulär ist, sind die Vektoren a_i linear unabhängig. Jetzt sei $q_1, \dots, q_n \in \mathbb{R}^n$ ein Orthonormalsystem mit den Eigenschaften

$$(q_i, q_j)_2 = \delta_{ij} \text{ für } 1 \leq i, j \leq n, \text{ sowie } (q_i, a_j)_2 = 0 \text{ für } 1 \leq j < i \leq n.$$

Ein solches System aus Vektoren kann mit dem Gram-Schmidt-Verfahren erzeugt werden, siehe Satz 2.10.

Die Matrix $Q = [q_1, \dots, q_n] \in \mathbb{R}^{n \times n}$ ist orthogonal und für die Einträge der Matrix $R = Q^T A \in \mathbb{R}^{n \times n}$ mit $R = (r_{ij})_{ij}$ für $i, j = 1, \dots, n$ gilt

$$r_{ij} = (q_i, a_j)_2 = 0 \text{ für } i > j. \quad (4.1)$$

Also ist R eine rechte obere Dreiecksmatrix und mit $(Q^T)^{-1} = (Q^{-1})^T = (Q^T)^T = Q$ folgt

$$A = QR.$$

□

Die QR-Zerlegung ist nicht eindeutig. Denn angenommen, durch $q_1, \dots, q_n$ sei das Orthonormalsystem aus Eigenvektoren gegeben, dann entsteht durch Vorzeichenwechsel $q_1, \dots, q_{i-1}, -q_i, q_{i+1}, \dots, q_n$ ein weiteres Orthonormalsystem aus Eigenvektoren. Diese Wahl führt gemäß (4.1) zu $\tilde{r}_{ij} = -r_{ij}$ und eine zweite QR-Zerlegung von A ist gegeben.

Diese Überlegung kann zu einer einfachen Normierung der QR-Zerlegung herangezogen werden. In jedem Schritt wird das Vorzeichen des Vektors q_i so gewählt, dass $r_{ii} = (q_i, a_i) > 0$, dass also die Matrix R nur positive Diagonalelemente besitzt.

Satz 4.5 (Eindeutigkeit der QR-Zerlegung) Die QR-Zerlegung $A = QR$ einer regulären Matrix $A \in \mathbb{R}^{n \times n}$ ist bei Wahl von $r_{ii} > 0$ eindeutig.

Beweis Es seien $A = Q_1 R_1 = Q_2 R_2$ zwei QR-Zerlegungen zu A . Dann gilt:

$$Q := Q_2^T Q_1 = R_2 R_1^{-1} \in \mathbb{R}^{m \times m}, \quad Q^T := Q_1^T Q_2 = R_1 R_2^{-1} \in \mathbb{R}^{n \times n}. \quad (4.2)$$

Die Matrizen Q und Q^T sind beide rechte obere Dreiecksmatrizen, also muss Q eine Diagonalmatrix sein. Weiter gilt

$$Q^T Q = Q_1^T Q_2 Q_2^T Q_1 = R_1 R_2^{-1} R_2 R_1^{-1} = I,$$

d. h., Q ist orthogonal. Aus (4.2) folgt

$$QR_1 = R_2$$

und für jeden Einheitsvektor e_i gilt

$$q_{ii} r_{ii}^{(1)} = e_i^T (QR_1) e_i = e_i^T (R_2) e_i = r_{ii}^{(2)} > 0. \quad (4.3)$$

Die Diagonalelemente $q_{ii} > 0$ sind positiv. Da die Diagonalelemente einer Diagonalmatrix gerade die Eigenwerte der Matrix sind, folgt aus $|\lambda| = 1$ (da Q orthogonal), dass $\lambda = 1$ und somit $Q = I$. Nun folgt $Q_1 = Q_2$ und somit ebenso $R_1 = R_2$. □

Der entscheidende Schritt zum Erstellen einer QR-Zerlegung ist die Orthogonalisierung eines Systems von n unabhängigen Vektoren $a_1, \dots, a_n \in \mathbb{R}^n$. Diese Aufgabe wird in den folgenden Abschnitten detailliert untersucht. Insbesondere entwickeln wir Orthogonalisierungsverfahren her, die weitaus stabiler sind als das einfache Gram-Schmidt-Verfahren.

Bemerkung 4.6 (QR-Zerlegung für rechteckige Matrizen) Wir haben die QR-Zerlegung für reguläre Matrizen $A \in \mathbb{R}^{n \times n}$ eingeführt. Das Verfahren lässt sich jedoch direkt auf rechteckige Matrizen $A \in \mathbb{R}^{n \times m}$ mit $n > m$ und vollem Rang $\text{rang}(A) = m$ übertragen. Die Spaltenvektoren $a_1, \dots, a_m \in \mathbb{R}^n$ sind weiterhin linear unabhängig und können, z. B. mit dem Gram-Schmidt-Verfahren zu einem Orthonormalsystem $q_1, \dots, q_m \in \mathbb{R}^n$ mit der Eigenschaft

$$(q_i, q_j)_2 = \delta_{ij}, \quad i, j = 1, \dots, m, \quad (q_i, a_j)_2 = 0 \quad 1 \leq j, i \leq m. \quad (4.4)$$

Die Matrix $Q = (q_1, \dots, q_m) \in \mathbb{R}^{n \times m}$ ist als nicht-quadratische Matrix natürlich nicht orthogonal, aber es gilt

$$Q^T Q = I \in \mathbb{R}^{m \times m}.$$

Ebenso folgt mit der Orthonormalität (4.4), dass $R = Q^T A \in \mathbb{R}^{m \times m}$ eine rechte obere Dreiecksmatrix ist.

Wenn wir die Vektoren $q_1, \dots, q_m$ durch $\tilde{q}_{m+1}, \dots, \tilde{q}_n$ zu einer Orthonormalbasis des $\mathbb{R}^n$ ergänzen, und die Matrix R mit $n - m$ Nullzeilen zur Matrix $\tilde{R} \in \mathbb{R}^{n \times m}$ erweitern, so gilt die übliche Schreibweise

$$\tilde{Q} \tilde{R} = A \quad \Leftrightarrow \quad \left(\begin{array}{c|c} Q & \tilde{Q} \end{array} \right) \begin{pmatrix} R \\ \hline 0 \end{pmatrix} = \begin{pmatrix} A \\ \hline 0 \end{pmatrix}.$$

Die Anreicherung der Vektoren $q_1, \dots, q_m$ um $\tilde{q}_{m+1}, \dots, \tilde{q}_n$ dient lediglich zur Angleichung der Schreibweise. Diese $n - m$ Vektoren müssen jedoch nicht bestimmt werden um die Matrix $R = Q^T A$ zu erstellen. $\blacklozenge$

4.2 Das Gram-Schmidt-Verfahren

Das Gram-Schmidt-Verfahren, siehe Satz 2.10, ist einfach aufgebaut und beruht jeweils auf Projektionen eines Vektors auf bereits orthogonale Anteile. Die grundlegende Iterationsformel zur Orthonormalisierung von Vektoren $a_1, a_2, \dots, a_m$ ist

$$q_1 := \frac{a_1}{\|a_1\|}, \quad \tilde{q}_i = a_i - \sum_{j=1}^{i-1} (a_i, q_j) q_j, \quad q_i := \frac{\tilde{q}_i}{\|\tilde{q}_i\|}, \quad i = 2, 3, \dots, m.$$

Dabei ist (x, y) ein Skalarprodukt und $\|x\| = (x, x)^{\frac{1}{2}}$ die zugehörige Norm. Der Aufwand des Verfahrens steigt mit der Anzahl der Vektoren in der Basis. In Schritt i des Verfahrens ist die Berechnung von $i - 1$ Skalarprodukten, von $i - 1$ Vektoradditionen sowie eine

Normberechnung notwendig. Im Raum $V = \mathbb{R}^n$ beträgt die Anzahl der Operationen im Schritt n somit $O(n^2)$. Der Gesamtaufwand zur Orthogonalisierung von n Vektoren des $\mathbb{R}^n$ verhält sich wie $O(n^3)$.

Beispiel 4.7 (Numerische Instabilität des Gram-Schmidt-Verfahrens)

Als Beispiel betrachten wir die drei Vektoren

$$a_1 = \begin{pmatrix} 1 \\ \frac{1}{2} \\ \frac{1}{3} \end{pmatrix}, \quad a_2 = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{3} \\ \frac{1}{4} \end{pmatrix}, \quad a_3 = \begin{pmatrix} \frac{1}{3} \\ \frac{1}{4} \\ \frac{1}{5} \end{pmatrix}.$$

und führen die Rechnung direkt in Python durch. Bei Verwendung von `half` als Gleitkommazahlen zeigt sich schon bei dieser kleinen Aufgabe eine numerische Instabilität. Das Gram-Schmidt Verfahren ist folgendermaßen implementiert.

🔧 Implementierung 4.1: Gram-Schmidt Verfahren

```
1 def gram_schmidt(A):
2     n = A.shape[0]
3     Q = np.zeros_like(A)
4
5     for i in range(n):
6         Q[:, i] = A[:, i]
7         for j in range(i):
8             Q[:, i] -= np.inner(A[:, i], Q[:, j]) * Q[:, j]
9         Q[:, i] /= np.linalg.norm(Q[:, i])
10    return Q
```

Wenden wir dies auf die 3×3 Hilbert-Matrix in `half` Genauigkeit an

```
1 A = np.array([[1,      1 / 2, 1 / 3],
2              [1 / 2, 1 / 3, 1 / 4],
3              [1 / 3, 1 / 4, 1 / 5]], dtype=np.half)
4 Q = gram_schmidt(A)
5 print(Q)
```

erhalten wir

```
[[ 0.857 -0.4995 0.274 ]
 [ 0.4285 0.569 -0.7905]
 [ 0.2856 0.653 0.548 ]]
```

Wir testen hiervon die Orthogonalität, d. h. wir berechnen $Q^T Q - I$ und erhalten

```

1 print(Q.T @ Q)
2 err = np.linalg.norm(Q.T @ Q - np.identity(Q.shape[0]), ord=2)
3 print(f' ||Q * Q^T - I||_2 = {err}')
```

```

[[ 0.9995    0.002161  0.05252 ]
 [ 0.002161  0.9995   -0.2289 ]
 [ 0.05252  -0.2289    1.      ]]
||Q * Q^T - I||_2 = 0.33211513864862696
```

mit einem Fehler von 33 % nur eine sehr ungenaue Approximation der Einheitsmatrix. ◀

Wir setzen das Beispiel nun fort und verwenden das Gram-Schmidt-Verfahren zur Berechnung der QR-Zerlegung:

Beispiel 4.8 (QR-Zerlegung mit Gram-Schmidt)

Es sei das LGS $Ax = b$ mit

$$A := \begin{pmatrix} 1 & 1 & 1 \\ 0.01 & 0 & 0.01 \\ 0 & 0.01 & 0.01 \end{pmatrix}, \quad b := \begin{pmatrix} 1 \\ 0 \\ 0.02 \end{pmatrix},$$

und exakter Lösung

$$x = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}$$

gegeben. Wir bestimmen mit dem Gram-Schmidt-Verfahren aus den Spaltenvektoren $A = (a_1, a_2, a_3)$ die entsprechende Orthonormalbasis. Bei dreistelliger Genauigkeit erhalten wir

$$q_1 = \frac{q_1}{\|q_1\|} \approx \begin{pmatrix} 1 \\ 0.01 \\ 0 \end{pmatrix}.$$

Weiter:

$$\tilde{q}_2 = a_2 - (a_2, q_1)q_1 \approx a_2 - q_1 = \begin{pmatrix} 0 \\ -0.01 \\ 0.01 \end{pmatrix}, \quad q_2 = \frac{\tilde{q}_2}{\|\tilde{q}_2\|} \approx \begin{pmatrix} 0 \\ -0.709 \\ 0.709 \end{pmatrix}$$

Schließlich:

$$\tilde{q}_3 = a_3 - (a_3, q_1)q_1 - (a_3, q_2)q_2 \approx a_3 - q_1 - 0 = \begin{pmatrix} 0 \\ 0 \\ 0.01 \end{pmatrix}, \quad q_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Die QR-Zerlegung ergibt sich mit der „orthogonalen Matrix“

$$\tilde{Q} = \begin{pmatrix} 1 & 0 & 0 \\ 0.01 & -0.709 & 0 \\ 0 & 0.709 & 1 \end{pmatrix}$$

sowie der rechten oberen Dreiecksmatrix $\tilde{R}$ mit $\tilde{r}_{ij} = (q_i, a_j)$ für $j \geq i$

$$\tilde{R} := \begin{pmatrix} (q_1, a_1) & (q_1, a_2) & (q_1, a_3) \\ 0 & (q_2, a_2) & (q_2, a_3) \\ 0 & 0 & (q_3, a_3) \end{pmatrix} \approx \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0.00709 & 0 \\ 0 & 0 & 0.01 \end{pmatrix}.$$

Wir lösen mithilfe der QR-Zerlegung $\tilde{R}\tilde{x} = \tilde{Q}^T b$, also

$$\begin{pmatrix} 1 & 1 & 1 \\ 0 & 0.00709 & 0 \\ 0 & 0 & 0.01 \end{pmatrix} \begin{pmatrix} \tilde{x}_1 \\ \tilde{x}_2 \\ \tilde{x}_3 \end{pmatrix} = \tilde{b} = \tilde{Q}^T b \approx \begin{pmatrix} 1 \\ 0.0142 \\ 0.02 \end{pmatrix}.$$

Es folgt

$$\tilde{x} = \begin{pmatrix} -3 \\ 2 \\ 2 \end{pmatrix}$$

mit dem relativen Fehler von 140 %, der sich aus

$$\frac{\|\tilde{x} - x\|_2}{\|x\|_2} \approx 1.41$$

ergibt. Bei der Lösung des Gleichungssystems mit dieser gestörten Matrix entsteht ein sehr großer Fehler. Dies liegt daran, dass die Matrix Q nur eine sehr gestörte Orthogonalität aufweist:

$$I \stackrel{!}{=} \tilde{Q}^T \tilde{Q} \approx \begin{pmatrix} 1 & -0.00709 & 0 \\ -0.00709 & 1.01 & 0.709 \\ 0 & 0.709 & 1 \end{pmatrix}, \quad \|\tilde{Q}^T \tilde{Q} - I\|_2 \approx 0.7$$

Wir passen die Python-Implementierung des Gram-Schmidt Verfahrens an, um die QR-Zerlegung zu erstellen:

♣ Implementierung 4.2: QR-Zerlegung mit Gram-Schmidt Verfahren

```

1 def qr_gram_schmidt(A):
2     n = A.shape[0]
3     Q, R = np.zeros_like(A), np.zeros_like(A)
4
5     for i in range(n):
6         Q[:, i] = A[:, i]
7         for j in range(i):
8             Q[:, i] -= np.inner(A[:, i], Q[:, j]) * Q[:, j]
9         Q[:, i] /= np.linalg.norm(Q[:, i])
10        for j in range(i, n):
11            R[i, j] = np.inner(Q[:, i], A[:, j])
12    return Q, R

```

Mit dem in Kap. 3 implementierten `rueckwaerts_einsetzen` können wir das Gleichungssystem lösen. Mit `half` Gleitkommazahlen erhalten wir

```

1 A = np.array([[1, 1, 1],
2               [0.01, 0, 0.01],
3               [0, 0.01, 0.01]], dtype=np.half)
4 b = np.array([1, 0, 0.02], dtype=np.half)
5 x_ex = np.array([-1, 1, 1])
6
7 Q, R = qr_gram_schmidt(A)
8 b2 = np.dot(Q.transpose(), b)
9 x = rueckwaerts_einsetzen(R, b2)
10
11 print('x =', x)
12 print('x_ex = ', x_ex)

```

```

x = [-3.  2.  2.]
x_ex = [-1  1  1]

```

```

1 rel_err = np.linalg.norm(x - x_ex) / np.linalg.norm(x_ex)
2 print(f'||x - x_ex|| / ||x|| = {rel_err}')

```

```
||x - x_ex|| / ||x_ex|| = 1.414213562373095
```

Hier ist die Orthogonalität ähnlich zu der per Hand berechneten QR-Zerlegung:

```

1 err = np.linalg.norm(Q @ Q.transpose() - np.identity(Q.shape[0]),
2                       ord=2)
3 print(f'||Q * Q^T - I||_2 = {err}')

```

$$\|Q * Q^T - I\|_2 = 0.7072275245944134$$



Die QR-Zerlegung hat prinzipiell bessere Stabilitätseigenschaften als die LR-Zerlegung, da die Matrix R höchstens die Konditionszahl der Matrix A trägt. Diese Überlegung gilt jedoch nur dann, wenn Q und R fehlerfrei erzeugt werden können. Das einfache Gram-Schmidt-Verfahren eignet sich nicht, um die orthogonale Matrix Q zu erstellen. In den folgenden Abschnitten werden wir alternative Orthogonalisierungsverfahren vorstellen, die selbst auf der Anwendung gut konditionierter Operationen beruhen. Zunächst gehen wir noch kurz auf eine Variante des Gram-Schmidt-Verfahrens ein. Der Orthogonalisierungsschritt

$$\tilde{q}_i := a_i - \sum_{j=1}^{i-1} (a_i, q_j) q_j,$$

wird durch eine iterative Vorschrift ersetzt:

$$\tilde{q}_i^0 := a_i, \quad k = 1, \dots, i-1: \quad \tilde{q}_i^{(k)} = \tilde{q}_i^{(k-1)} - (\tilde{q}_i^{(k-1)}, q_k) q_k, \quad \tilde{q}_i := \tilde{q}_i^{(i-1)}$$

Auf diese Weise werden neu entstehende Fehler stets bei der Projektion berücksichtigt.

Satz 4.9 (Modifiziertes Gram-Schmidt-Orthonormalisierungsverfahren) Es sei durch $\{a_1, \dots, a_n\}$ eine Basis eines Vektorraums V gegeben, durch $(\cdot, \cdot)$ ein Skalarprodukt mit induzierter Norm $\|\cdot\|$. Die Vorschrift

$$\begin{aligned} (i) \quad q_1 &:= \frac{a_1}{\|a_1\|}, \\ i = 2, \dots, n: \quad (ii) \quad \tilde{q}_i^{(0)} &:= a_i \\ \tilde{q}_i^{(k)} &:= \tilde{q}_i^{(k-1)} - (\tilde{q}_i^{(k-1)}, q_k) q_k, \quad k = 1, \dots, i-1, \\ q_i &:= \frac{\tilde{q}_i^{(i-1)}}{\|\tilde{q}_i^{(i-1)}\|}, \end{aligned}$$

erzeugt eine Orthonormalbasis $\{q_1, \dots, q_n\}$ von V . Es gilt ferner:

$$(q_i, a_j) = 0 \quad \forall 1 \leq j < i \leq n.$$

Der Beweis kann entsprechend zu Satz 2.10 durchgeführt werden. Wir führen nun mit dieser Modifikation Beispiel 4.7 erneut durch.

Beispiel 4.10 (Modifiziertes Gram-Schmidt-Verfahren)

Wir betrachten wieder die Vektoren

$$a_1 = \begin{pmatrix} 1 \\ \frac{1}{2} \\ \frac{1}{3} \end{pmatrix}, \quad a_2 = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{3} \\ \frac{1}{4} \end{pmatrix}, \quad a_3 = \begin{pmatrix} \frac{1}{3} \\ \frac{1}{4} \\ \frac{1}{5} \end{pmatrix}$$

und berechnen die Orthogonalisierung mit dem modifizierten Algorithmus, den wir in Python implementieren:

🔗 Implementierung 4.3: Modifiziertes Gram-Schmidt Verfahren

```

1 def mod_gram_schmidt(A):
2     n = A.shape[0]
3     Q = np.zeros_like(A)
4
5     for i in range(n):
6         Q[:, i] = A[:, i]
7         for j in range(i):
8             Q[:, i] -= np.inner(Q[:, i], Q[:, j]) * Q[:, j]
9         Q[:, i] /= np.linalg.norm(Q[:, i])
10    return Q

```

Da wir im Schritt k von Teil (ii) des Verfahrens immer nur auf $\tilde{q}_i^{(k-1)}$ zugreifen müssen, können wir dies jeweils mit $\tilde{q}_i^{(k)}$ überschreiben. Wir wenden das Verfahren erneut auf die 3×3 Hilbert-Matrix mit half Gleitkommazahlen an

```

1 Q = mod_gram_schmidt(A)
2 err = np.linalg.norm(Q @ Q.transpose() - np.identity(Q.shape[0]),
3     ↪ ord=2)
4 print(f'Q =\n{Q}')
4 print(f'||Q * Q^T - I||_2 = {err}')

```

und erhalten

```

Q =
[[ 0.857 -0.4995  0.1514]
 [ 0.4285  0.569 -0.661 ]
 [ 0.2856  0.653  0.733 ]]
||Q * Q^T - I||_2 = 0.06270300839082639

```

Im Vergleich zu Beispiel 4.7 erreichen wir den (immer noch großen) relativen Fehler von 6.3 %, der aber weit kleiner ist als die 33 %, die wir mit dem ursprünglichen Verfahren erreicht hatten. ◀

Entsprechend wiederholen wir die QR-Zerlegung der Matrix A aus Beispiel 4.8 nun mithilfe des modifizierten Gram-Schmidt-Verfahrens:

Beispiel 4.11 (QR-Zerlegung mit modifiziertem Gram-Schmidt)

Es sei wieder das LGS

$$A := \begin{pmatrix} 1 & 1 & 1 \\ 0.01 & 0 & 0.01 \\ 0 & 0.01 & 0.01 \end{pmatrix}, \quad b := \begin{pmatrix} 1 \\ 0 \\ 0.02 \end{pmatrix},$$

mit Lösung

$$x = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}$$

gegeben. Wir verzichten hier auf die detaillierte Angabe der dreistelligen Rechnung und springen gleich zur Python-Implementierung. Um die QR Zerlegung mit dem modifizierten Gram-Schmidt-Verfahren zu berechnen, müssen wir unseren bisherigen Code nur um die Erstellung der Matrix R in Zeilen 10–11 erweitern:

🔗 Implementierung 4.4: QR-Zerlegung mit modifiziertem Gram-Schmidt

```

1 def qr_mod_gram_schmidt(A):
2     n = A.shape[0]
3     Q, R = np.zeros_like(A), np.zeros_like(A)
4
5     for i in range(n):
6         Q[:, i] = A[:, i]
7         for j in range(i):
8             Q[:, i] -= np.inner(Q[:, i], Q[:, j]) * Q[:, j]
9         Q[:, i] /= np.linalg.norm(Q[:, i])
10        for j in range(i, n):
11            R[i, j] = np.inner(Q[:, i], A[:, j])
12    return Q, R

```

Angewandt auf die Matrix

```

1 Q, R = qr_mod_gram_schmidt(A)
2 b3 = np.dot(Q.transpose(), b)
3 x2 = rueckwaerts_einsetzen(R, b3)
4 print(f'x = {x}')

```

ergibt dies

$$x = [-2. \quad 2. \quad 1.]$$

Hieraus ergibt sich der (relative) Fehler in der Lösung und Orthogonalität:

```
1 rel_err = np.linalg.norm(x2 - x_ex) / np.linalg.norm(x_ex)
2 print(f' ||x - x_ex|| / ||x_ex|| = {rel_err} ')
3
4 err = np.linalg.norm(Q @ Q.transpose() - np.identity(Q.shape[0]),
   ↪   ord=2)
5 print(f' ||Q * Q^T - I||_2 = {err} ')
```

```
||x - x_ex|| / ||x_ex|| = 0.8164965809277261
||Q * Q^T - I||_2 = 0.01000213623046875
```

Obwohl die Orthogonalität um einen Faktor 70 besser geworden ist, hat sich der Fehler der Lösung nicht einmal halbiert. Der Grund für das schlechte Ergebnis liegt im Aufbau des Verfahrens: der modifizierte Algorithmus orthogonalisiert in Iteration i immer bzgl. der bereits berechneten Vektoren q_j , $j < i$. Dies führt zu einer verbesserten Orthogonalität $(q_i, q_j) \approx 0$, nicht aber zu einer besseren Orthogonalität in Bezug auf die Spalten der Matrix A , d. h., $R = Q^T A$ hat im Allgemeinen keine Dreiecksgestalt. ◀

Aufgrund der Orthogonalität der Matrix Q hat die QR-Zerlegung das Potenzial zu einer sehr stabilen Methode. Bisher ist es jedoch nicht gelungen, den Zerlegungsprozess selbst hinreichend stabil durchzuführen. Als Problem stellt sich die Berechnung der Einträge der Matrix R heraus.

4.3 Householder-Transformationen

Das geometrische Prinzip hinter dem Gram-Schmidt-Verfahren ist die Projektion der Vektoren a_i auf die bereits erstellte Orthogonalbasis $q_1, \dots, q_{i-1}$. Diese Projektion ist schlecht konditioniert, falls a_i fast parallel zu den q_j mit $j < i$, etwa $a_i \approx q_j$ ist. Dann droht Auslöschung. Wir können einen Schritt des Gram-Schmidt-Verfahrens kompakt mithilfe eines Matrix-Vektor-Produkts schreiben:

$$\tilde{q}_i = [I - G^{(i)}]a_i, \quad G^{(i)} = \sum_{l=1}^{i-1} q_l q_l^T$$

Dabei ist $vv^T \in \mathbb{R}^{n \times n}$ das *dyadische Produkt* zweier Vektoren, d. h.

$$a, b \in \mathbb{R}^n \quad \Rightarrow \quad (ab^T)_{ij} = a_i b_j, \quad i, j = 1, \dots, n.$$

Die Matrix $[I - G^{(i)}]$ ist eine Projektion auf $\mathbb{R}^n \rightarrow \mathbb{R}^n$, denn

$$[I - G^{(i)}]^2 = I - 2 \sum_{l=1}^{i-1} q_l q_l^T + \sum_{k,l=1}^{i-1} q_l \underbrace{q_l^T q_k}_{=\delta_{lk}} q_k^T = [I - G^{(i)}].$$

Weiter gilt:

$$[I - G^{(i)}]q_k = q_k - \sum_{l=1}^{i-1} q_l \underbrace{q_l^T q_k}_{=\delta_{lk}} = 0, \quad k < i$$

Die Matrix $I - G^{(i)}$ ist also nicht regulär. Falls in Schritt i der Vektor a_i fast parallel zu den bereits orthogonalen Vektoren ist, also $\tilde{a}_i \in \delta a_i + \text{span}\{q_1, \dots, q_{i-1}\}$, so gilt

$$\tilde{q}_i = [I - G^{(i)}]\tilde{a}_i = [I - G^{(i)}]\delta a_i \Rightarrow \frac{\|\delta q_i\|}{\|\tilde{q}_i\|} = \frac{\|\delta q_i\|}{\|[I - G^{(i)}]\delta a_i\|}.$$

Bei $\delta a_i \rightarrow 0$ ist eine beliebig große Fehlerverstärkung möglich.

Im Folgenden suchen wir eine Transformation von A zu einer Dreiecksmatrix R , die selbst auf orthogonalen und somit stabilen Operationen aufbaut. Eine orthogonale Matrix $Q \in \mathbb{R}^{n \times n}$ mit $\det(Q) = 1$ stellt eine Drehung dar und bei $\det(Q) = -1$ eine Spiegelung (oder eine Drehspiegelung). Die Householder-Transformationen nutzen das Prinzip der Spiegelung, um eine Matrix $A \in \mathbb{R}^{n \times n}$ in eine rechte obere Dreiecksmatrix zu transformieren.

► **Definition 4.12 (Householder-Transformation)** Für einen Vektor $v \in \mathbb{R}^n$ mit $\|v\|_2 = 1$ ist durch vv^T das *dyadische Produkt* definiert und die Matrix

$$S := I - 2vv^T \in \mathbb{R}^{n \times n}$$

heißt *Householder-Transformation*.

Es gilt:

Satz 4.13 (Householder-Transformation) Jede Householder-Transformation $S = I - 2vv^T$ mit $\|v\|_2 = 1$ ist symmetrisch und orthogonal. Das Produkt zweier Householder-Transformationen $S_1 S_2$ ist wieder eine orthogonale Matrix.

Beweis (i) Es gilt Symmetrie

$$S^T = [I - 2vv^T]^T = I - 2(vv^T)^T = I - 2vv^T = S$$

und weiter

$$S^T S = I - 4vv^T + 4v \underbrace{v^T v}_{=1} = I,$$

d. h., $S^{-1} = S^T$ und S stehen orthogonal zueinander.

(ii) Mit zwei symmetrischen orthogonalen Matrizen S_1 sowie S_2 gilt

$$(S_1 S_2)^T = S_2^T S_1^T = S_2^{-1} S_1^{-1} = (S_1 S_2)^{-1},$$

sodass das Produkt $S_1 S_2$ eine orthogonale Matrix ist. Wir haben hierfür nur die Orthogonalität von S_1 und S_2 genutzt. $\square$

Wir schließen die grundlegende Untersuchung der Householder-Transformationen mit einer geometrischen Charakterisierung ab:

Bemerkung 4.14 (Householder-Transformation als Spiegelung) Es sei $v \in \mathbb{R}^n$ ein beliebiger normierter Vektor mit $\|v\|_2 = 1$. Weiter sei $x \in \mathbb{R}^n$ gegeben mit $x = \alpha v + w^\perp$, wobei $w^\perp \in \mathbb{R}^n$ ein Vektor mit $v^T w^\perp = 0$ im orthogonalen Komplement zu v ist. Dann gilt für die Householder-Transformation $S = I - 2vv^T$:

$$S(\alpha v + w^\perp) = [I - 2vv^T](\alpha v + w^\perp) = \alpha(v - 2v \underbrace{v^T v}_{=1}) + w^\perp - 2v \underbrace{v^T w^\perp}_{=0} = -\alpha v + w^\perp$$

Das heißt, die Householder-Transformation beschreibt eine Spiegelung an der auf v senkrecht stehenden Ebene. $\blacklozenge$

Mithilfe der Householder-Transformationen soll eine Matrix $A \in \mathbb{R}^{n \times n}$ Schritt für Schritt in eine rechte obere Dreiecksmatrix transformiert werden:

$$A^{(0)} = A, \quad A^{(i)} = S^{(i)} A^{(i-1)}, \quad S^{(i)} = I - 2v^{(i)}(v^{(i)})^T,$$

wobei $a_{kl}^{(i)} = 0$ für $l \leq i$ und $k > l$ teilweise Diagonalgestalt hat. Schließlich ist $R := A^{(n-1)}$.

Wir beschreiben den ersten Schritt des Verfahrens: Gegeben sei die reguläre Matrix $A^{(0)} := A \in \mathbb{R}^{n \times n}$, geschrieben in ihren Spaltenvektoren a_i für $i = 1, \dots, n$. Wir suchen eine orthogonale Householder-Transformation $S^{(1)}$, so dass deren Multiplikation von links, d. h. $A^{(1)} = S^{(1)} A^{(0)}$ die erste Spalte von $A^{(0)}$ auf ein Vielfaches des ersten Einheitsvektor e_1 spiegelt. Die Spaltenvektoren von $A^{(1)}$ nennen wir $a_i^{(1)}$ für $i = 1, \dots, n$, und erwarten

$$a_1^{(1)} = S^{(1)} a_1^{(0)} \in \text{span}\{e_1\}.$$

Wir müssen an der Ebene senkrecht zu $a_1 \pm \|a_1\| e_1$ spiegeln, siehe Abb. 4.1. Zur Bestimmung des Spiegelungsvektors haben wir durch die Wahl des Vorzeichens zwei Möglichkeiten. Aus Stabilitätsgründen wählen wir

$$v^{(1)} := \frac{a_1 + \text{sign}(a_{11}) \|a_1\| e_1}{\|a_1 + \text{sign}(a_{11}) \|a_1\| e_1\|}, \quad (4.5)$$

wobei wir hier $\text{sign}(0) = 1$ definieren, um durch optimale Wahl des Vorzeichens die Gefahr von Auslöschung zu vermeiden. Mit dieser Wahl gilt:

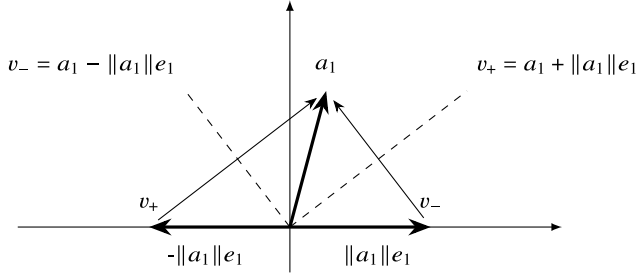


Abb. 4.1 Spiegelungsachsen (gestrichelt) und Normalen zur Spiegelung v_+ und v_- zur Spiegelung von a_1 auf $\text{span}\{e_1\}$

$$\begin{aligned} a_i^{(1)} &= S^{(1)} a_i = a_i - 2(v^{(1)}, a_i)v^{(1)}, \quad i = 2, \dots, n \\ \Rightarrow a_1^{(1)} &= -\text{sign}(a_{11})||a_1||e_1 \end{aligned} \quad (4.6)$$

Die resultierende Matrix $A^{(1)}$ ist wieder regulär und es gilt $a_1^{(1)} \in \text{span}(e_1)$:

$$A := \begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & \ddots & & \vdots \\ a_{31} & a_{32} & \ddots & \ddots & \vdots \\ \vdots & & & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & \cdots & a_{nn} \end{pmatrix} \Rightarrow A^{(1)} := S^{(1)} A = \left(\begin{array}{c|cccc} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} & \cdots & a_{1n}^{(1)} \\ 0 & a_{22}^{(1)} & \ddots & & \vdots \\ 0 & a_{32}^{(1)} & \ddots & \ddots & \vdots \\ \vdots & & & \ddots & \vdots \\ 0 & a_{n2}^{(1)} & \cdots & \cdots & a_{nn}^{(1)} \end{array} \right)$$

Wir setzen das Verfahren mit der Teilmatrix $\tilde{A}^{(1)} := A_{kl>1}^{(1)} \in \mathbb{R}^{(n-1) \times (n-1)}$ fort. Hierzu sei der verkürzte zweite Spaltenvektor definiert als $\tilde{a}^{(1)} = (a_{22}^{(1)}, a_{32}^{(1)}, \dots, a_{n2}^{(1)})^T \in \mathbb{R}^{n-1}$. Dann wählen wir:

$$\tilde{v}^{(2)} := \frac{\tilde{a}^{(1)} + \text{sign}(a_{22}^{(1)})||\tilde{a}^{(1)}||e_1}{||\tilde{a}^{(1)} + \text{sign}(a_{22}^{(1)})||\tilde{a}^{(1)}||e_1}, \quad S^{(2)} = \left(\begin{array}{c|cccc} 1 & 0 & \cdots & \cdots & 0 \\ 0 & & & & \\ 0 & & I - 2\tilde{v}^{(2)}(\tilde{v}^{(2)})^T & & \\ \vdots & & & & \\ 0 & & & & \end{array} \right) \in \mathbb{R}^{n \times n}$$

Multiplikation mit $S^{(2)}$ von links, also $A^{(2)} := S^{(2)} A^{(1)}$, lässt die erste Zeile unverändert. Eliminiert wird der Block $A_{kl>1}^{(1)}$. Für die resultierende Matrix $A^{(2)}$ gilt $a_{kl}^{(2)} = 0$ für $l = 1, 2$ bei $k > l$. Nach $n - 1$ Schritten gilt

$$R := \underbrace{S^{(n-1)} S^{(n-2)} \dots S^{(1)}}_{=: Q^T} A.$$

Alle Transformationen sind orthogonal, somit ist auch $Q \in \mathbb{R}^{n \times n}$ eine orthogonale (und natürlich reguläre) Matrix (siehe Satz 4.2).

Satz 4.15 (QR-Zerlegung mit Householder-Transformationen) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix. Dann lässt sich die QR-Zerlegung von A nach Householder in

$$\frac{2}{3}n^3 + O(n^2)$$

elementaren Operationen (also jeweils eine Multiplikationen oder Divisionen sowie eine Addition) numerisch stabil durchführen.

Beweis (i) Die Durchführbarkeit der QR-Zerlegung geht aus dem Konstruktionsprinzip hervor. Aus der Regularität der Matrizen $A^{(i)}$ folgt, dass der Teilvektor $\tilde{a}^{(i)} \in \mathbb{R}^{n-i}$ ungleich Null sein muss. Dann ist $\tilde{v}^{(i)}$ gemäß (4.5) wohldefiniert. Die folgende Elimination wird mit (4.6) spaltenweise durchgeführt:

$$\tilde{a}_k^{(i+1)} = \underbrace{[I - 2\tilde{v}^{(i)}(\tilde{v}^{(i)})^T]}_{=\tilde{S}^{(i)}} \tilde{a}_k^{(i)} = \tilde{a}_k^{(i)} - 2(\tilde{v}^{(i)}, \tilde{a}_k^{(i)})\tilde{v}^{(i)}$$

Die numerische Stabilität folgt aus $\text{cond}_2(S^{(i)}) = 1$, siehe Bemerkung 3.2.

(ii) Wir kommen nun zur Abschätzung des Aufwands. Im Schritt $A^{(i)} \rightarrow A^{(i+1)}$ muss zunächst der Vektor $\tilde{v}^{(i)} \in \mathbb{R}^{n-i}$ berechnet werden. Hierzu sind $2(n-i) + 2$ elementare Operationen notwendig. Im Anschluss erfolgt die spaltenweise Elimination. Für jeden der $n-i$ Spaltenvektoren ist ein Skalarprodukt $(\tilde{v}^{(i)}, \tilde{a}^{(i)})$ (das sind $n-i$ elementare Operationen) sowie eine Vektoraddition und die Multiplikation von $(\tilde{v}^{(i)}, \tilde{a}^{(i)})$ mit $\tilde{v}^{(i)}$ (weitere $n-i$ elementare Operationen) durchzuführen. Insgesamt ergibt sich so der Aufwand:

$$N_{QR} = 2 \sum_{i=1}^{n-1} (n-i) + (n-i)^2 = n(n-1) + 2 \frac{n(n-1)(2n-1)}{6} = \frac{2n^3}{3} + O(n^2)$$

□

Bemerkung 4.16 (QR-Zerlegung mit Householder-Transformationen) Die Householder-Matrizen $\tilde{S}^{(i)} = I - 2\tilde{v}^{(i)}(\tilde{v}^{(i)})^T$ werden nicht explizit aufgestellt und gespeichert. In einer effizienten Umsetzung merkt man sich nur die Vektoren $\tilde{v}^{(i)} \in \mathbb{R}^{n-i+1}$. Auch die orthogonale Matrix

$$Q := (S^{(1)})^T \dots (S^{(n-1)})^T$$

wird nicht explizit berechnet und gespeichert. Wird die Matrix benötigt, um z. B. die rechte Seite $\tilde{b} = Q^T b$ zu berechnen, so muss dies Schritt für Schritt erfolgen. Es gilt

$$\tilde{b} = Q^T b = S^{(n-1)} \dots S^{(1)} b$$

und das Produkt wird mithilfe einer einfachen Iteration auf Basis der Vektoren $v^{(i)}$ bestimmt:

Eingabe: Vektor b und Householder-Vektoren $v_1, \dots, v_{n-1}$

1 $b^{(0)} = b$

2 **Für** $i = 1$ **bis** $n - 1$ **tue**

3 $b^{(i+1)} = b^{(i)} - 2(v^{(i)}, b^{(i)})v^{(i)}$

Ergebnis: $\tilde{b} = b^{(n)}$

Neben der oberen Dreiecksmatrix R müssen die Vektoren $\tilde{v}^{(i)} \in \mathbb{R}^{n+1-i}$ gespeichert werden. Insgesamt sind damit $(n+1)n$ Werte zu speichern. Im Gegensatz zur LR-Zerlegung kann dies nicht alleine im Speicherplatz der Matrix A geschehen, da sowohl R als auch die $\tilde{v}^{(i)}$ die Diagonale besetzen. Bei der praktischen Realisierung muss ein weiterer Diagonalvektor angelegt werden. ♦

Abschließend berechnen wir die QR-Zerlegung zu Beispiel 4.8 mithilfe der Householder-Transformationen:

Beispiel 4.17 (QR-Zerlegung mit Householder-Transformationen)

Es sei wieder das LGS $Ax = b$ mit

$$A := \begin{pmatrix} 1 & 1 & 1 \\ 0.01 & 0 & 0.01 \\ 0 & 0.01 & 0.01 \end{pmatrix}, \quad b := \begin{pmatrix} 1 \\ 0 \\ 0.02 \end{pmatrix} \quad \text{und} \quad x = \begin{pmatrix} -1 \\ 1 \\ 1 \end{pmatrix}$$

gegeben. Wir verzichten wieder auf eine Rechnung per Hand bei dreistelliger Genauigkeit und betrachten sofort die Python-Implementierung der Erstellung der Householder-Vektoren:

♣ Implementierung 4.5: QR-Zerlegung mit Householder-Transformationen

```

1 def qr_householder(A):
2     n, m = A.shape
3     V = np.zeros_like(A)
4
5     for i in range(m):
6         V[i:, i] = A[i:, i]
7         ei = np.zeros(n - i, dtype=A.dtype)
8         ei[0] = 1.0
9         V[i:, i] += np.sign(A[i, i]) * np.linalg.norm(V[i:, i]) * ei
10        V[i:, i] /= np.linalg.norm(V[i:, i])

```

```

11         for k in range(i, m):
12             A[i:, k] -= 2 * np.inner(V[i:, i], A[i:, k]) * V[i:, i]
13     return V

```

Hierzu müssen wir noch die Anwendung der transponierten Matrix Q^T auf die rechte Seite implementieren:

♣ Implementierung 4.6: Anwenden der Householder-Transformationen

```

1 def QT_anwenden(V, b):
2     for i in range(b.shape[0]):
3         b[i:] -= 2 * np.inner(V[i], b[i:]) * V[i]
4     return None

```

Dies wenden wir wieder auf unsere Matrix A unter Verwendung von `half` Gleitkommazahlen an:

```

1 V = qr_householder(A)

```

Berechnen wir die Matrix Q^T , die aus diesen Householder-Vektoren bekommen, dann haben wir

$$Q^T = \begin{pmatrix} -1.0 & -0.01 & 0.0 \\ 0.007053 & -0.705 & 0.7065 \\ 0.00707 & -0.7065 & -0.707 \end{pmatrix}$$

und

$$Q^T Q \approx \begin{pmatrix} 1.0 & -1.073 \cdot 10^{-6} & -1.669 \cdot 10^{-6} \\ -1.073 \cdot 10^{-6} & 0.9966 & -1.330 \cdot 10^{-3} \\ -1.669 \cdot 10^{-6} & -1.33 \cdot 10^{-3} & 0.9990 \end{pmatrix}.$$

Dies entspricht einem relativen Fehler $\|Q^T Q - I\| \leq 4 \cdot 10^{-3}$. Wenden wir die Vektoren nun an um das Gleichungssystem zu lösen, erhalten wir die Lösung

```

1 QT_anwenden(V, b)
2 x = rueckwaerts_einsetzen(A, b)
3 print(f'x = {x}')

```

```
[-1.      0.999  1.001]
```

Diese hat den relativen Fehler

```

1 rel_err = np.linalg.norm(x - x_ex) / np.linalg.norm(x_ex)
2 print(f'||x - x_ex|| / ||x_ex|| = {rel_err}')

```


$$||x - x_{\text{ex}}|| / ||x_{\text{ex}}|| = 0.0007973599423122326$$

Wir vergleichen das Resultat mit den vorherigen Ergebnissen aus den Beispielen 4.8 und 4.11. Im Vergleich zur QR-Zerlegung mit dem modifizierten Gram-Schmidt-Verfahren haben wir beispielsweise bei halber (half) Genauigkeit den Fehler der Lösung um einen Faktor von 17 verbessert. Darüber hinaus haben wir die Lösung auf vier Nachkommastellen bestimmt. Richtig durchgeführt ist die QR-Zerlegung ein numerisch sehr stabiles Verfahren. ◀

4.4 Givens-Rotationen

Das Gram-Schmidt-Verfahren basiert auf Projektionen, die Householder-Transformationen sind Spiegelungen. Schließlich stellen wir kurz eine Methode vor, die auf Rotationen beruht. Wir definieren:

► **Definition 4.18 (Givens-Rotation)** Unter einer *Givens-Rotation* im $\mathbb{R}^n$ versteht man die Drehung um den Winkel θ in der durch zwei Einheitsvektoren e_i und e_j aufgespannten Ebene. Die Transformationsmatrix ist gegeben durch:

$$G(i, j, \theta) = \begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & c & & -s & \\ & & & 1 & & \\ & & & & \ddots & \\ & & & & & 1 & \\ & s & & & c & & \\ & & & & & 1 & \\ & & & & & & \ddots & \\ & & & & & & & 1 \end{pmatrix}, \quad \begin{aligned} c &= \cos(\theta) \\ s &= \sin(\theta) \end{aligned}$$

Es gilt:

Satz 4.19 (Givens-Rotation) Die Givens-Rotation $G(i, j, \theta)$ ist eine orthogonale Matrix mit $\det(G) = 1$. Es ist $G(i, j, \theta)^{-1} = G(i, j, -\theta)$.

Beweis Nachrechnen. □

Wie die Householder-Transformationen sind die Givens-Rotationen orthogonale Matrizen. Die Multiplikation von links an eine Matrix, also GA , oder an einen Vektor, also Gx , ändert nur die i -te und j -te Zeile von A bzw. von x :

$$G(i, j, \theta)A = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{i-1,1} & \cdots & a_{i-1,n} \\ ca_{i1} - sa_{j1} & \cdots & ca_{in} - sa_{jn} \\ a_{i+1,1} & \cdots & a_{i+1,n} \\ \vdots & \ddots & \vdots \\ a_{j-1,1} & \cdots & a_{j-1,n} \\ sa_{i1} + ca_{j1} & \cdots & sa_{in} + ca_{jn} \\ a_{j+1,1} & \cdots & a_{j+1,n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix}$$

Die QR-Zerlegung auf der Basis von Givens-Rotationen transformiert die Matrix A wieder schrittweise in eine obere rechte Dreiecksmatrix R . Durch Anwenden einer Givens-Rotation kann jedoch nur ein einzelnes Unterdiagonalelement eliminiert werden und nicht eine ganze Spalte:

$$\begin{pmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix} \rightarrow \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ * & * & * & * \\ * & * & * & * \end{pmatrix} \rightarrow \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \\ * & * & * & * \end{pmatrix} \rightarrow \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \\ 0 & * & * & * \end{pmatrix}$$

$$\rightarrow \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & * & * & * \end{pmatrix} \rightarrow \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & * & * \end{pmatrix} \rightarrow \begin{pmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{pmatrix}$$

Wir betrachten einen Schritt des Verfahrens. Dazu sei die Matrix A gegeben in der Form:

$$A = \begin{pmatrix} a_{11} & \cdots & \cdots & \cdots & \cdots & a_{1n} \\ 0 & \ddots & & \ddots & & \vdots \\ \vdots & \ddots & a_{ii} & & & \vdots \\ \vdots & & 0 & a_{i+1,i+1} & & \vdots \\ \vdots & & \vdots & \vdots & & \vdots \\ \vdots & & 0 & & & \vdots \\ \vdots & & a_{ji} & & \ddots & \vdots \\ \vdots & & \vdots & & & \vdots \\ 0 & \cdots & a_{ni} & a_{n,i+1} & \cdots & a_{nn} \end{pmatrix}$$

Wir suchen die Givens-Rotation $G(i, j, \theta)$ zur Elimination von a_{ji} . Für GA gilt

$$(G(i, j, \theta)A)_{ji} = sa_{ii} + ca_{ji}, \quad c := \cos(\theta), \quad s := \sin(\theta).$$

Anstelle den Winkel θ zu finden, bestimmen wir gleich die Werte c und s mit dem Ansatz

$$sa_{ii} + ca_{ji} = 0, \quad c^2 + s^2 = 1 \quad \Rightarrow \quad c := \frac{a_{ii}}{\sqrt{a_{ii}^2 + a_{ji}^2}}, \quad s := -\frac{a_{ji}}{\sqrt{a_{ii}^2 + a_{ji}^2}}.$$

Anwendung von $G(i, j, \theta)$ auf A ergibt:

$$\begin{aligned} (G(i, j, \theta)A)_{ji} &= \frac{-a_{ji}a_{ii} + a_{ii}a_{ji}}{\sqrt{a_{ii}^2 + a_{ji}^2}} = 0, \\ (G(i, j, \theta)A)_{ii} &= \frac{a_{ii}^2 + a_{ji}^2}{\sqrt{a_{ii}^2 + a_{ji}^2}} = \sqrt{a_{ii}^2 + a_{ji}^2} \end{aligned}$$

Das Element a_{ji} wird eliminiert. Zur Elimination der i -ten Spalte (unterhalb der Diagonale) sind $(n-i)$ Givens-Rotationen notwendig. Eine dieser Rotationen ändert die $(n-i)$ Matrix-Einträge von jeweils zwei Zeilen. D.h., zur Elimination der i -ten Spalte sind $4(n-i)^2$ elementare Operationen notwendig. Hinzu kommen $O(n-i)$ Operationen zur Berechnung der Koeffizienten, die wir vernachlässigen können. Insgesamt ergeben sich

$$\sum_{i=1}^{n-1} 4(n-i)^2 = 4 \sum_{i=1}^{n-1} i^2 = \frac{2}{3}(n-1)n(2n-1) = \frac{4}{3}n^3 + O(n^2)$$

elementare Operationen. Das ist etwas der doppelte Aufwand verglichen mit der Householder-Methode.

Die QR-Zerlegung nach Givens gewinnt aber an Bedeutung, wenn die Matrix A bereits dünn besetzt ist. Nur Unterdiagonalelemente mit $a_{ji} \neq 0$ müssen gezielt eliminiert

werden. Bei sogenannten *Hessenberg-Matrizen* (das sind rechte obere Dreiecksmatrizen, die zusätzlich noch eine linke Nebendiagonale besitzen) kann die QR-Zerlegung mit Givens-Rotationen in nur $O(n^2)$ Operationen durchgeführt werden. Die QR-Zerlegung von Hessenberg-Matrizen spielt eine entscheidende Rolle bei dem wichtigsten Verfahren zur Berechnung von Eigenwerten einer Matrix, siehe Kap. 5.

In Python können wir die QR-Zerlegung mit Givens-Rotationen dann folgendermaßen implementieren. Hierbei erlauben wir auch den Fall, dass die Matrix mehr Zeilen als Spalten hat.

Implementierung 4.7: QR-Zerlegung mit Givens-Rotationen

```

1 def qr_givens(A):
2     n, m = A.shape
3     QT = np.identity(n, dtype=A.dtype)
4
5     for i in range(m):
6         for j in range(i + 1, n):
7             c, s = A[i, i], -A[j, i]
8             nrm = np.sqrt(c**2 + s**2)
9             c, s = c / nrm, s / nrm
10            for k in range(i, m):
11                t1, t2 = A[i, k], A[j, k]
12                A[i, k] = c * t1 - s * t2
13                A[j, k] = s * t1 + c * t2
14            for k in range(n):
15                t1, t2 = QT[i, k], QT[j, k]
16                QT[i, k] = c * t1 - s * t2
17                QT[j, k] = s * t1 + c * t2
18     return QT

```

Beispiel 4.20 (QR-Zerlegung nach Givens)

Wie in den Beispielen 4.8 und 4.17 sei wieder die folgende Matrix und rechte Seite gegeben:

$$A := \begin{pmatrix} 1 & 1 & 1 \\ 0.01 & 0 & 0.01 \\ 0 & 0.01 & 0.01 \end{pmatrix}, \quad b := \begin{pmatrix} 1 \\ 0 \\ 0.02 \end{pmatrix}.$$

Wenden wir unsere Python-Implementierung an, erhalten wir bei halber Genauigkeit (half) die Lösung mit relativen Fehler

```

1 QT = qr_givens(A)
2 QTb = np.dot(QT, b)
3 x = rueckwaerts_einsetzen(A, QTb)

```

$$||x - x_{\text{ex}}|| / ||x|| = 0.0003986799711561163$$

Betrachten wir nun die Matrix Q^T genauer, sehen wir

$$Q^T = \begin{pmatrix} 1.0 & 0.01 & 0.0 \\ 0.007072 & -0.707 & 0.707 \\ 0.007072 & -0.707 & -0.707 \end{pmatrix}.$$

Dieses Ergebnis ist fast identisch zur Matrix in Beispiel 4.17. Die Approximationseigenschaften sind daher auch sehr gut und wir beobachten, dass $QR \approx A$ und $QQ^T \approx I$ gelten:

```
1 err = np.linalg.norm(A2 - QT.transpose() @ A, ord=2) /
  ↳ np.linalg.norm(A2, ord=2)
2 print(f' ||A - QR||_2 / ||A||_2 = {err}')
```

$$||A - QR||_2 / ||A||_2 = 7.22635781314447\text{e-}07$$

```
1 Id = np.identity(QT.shape[0], dtype=np.single)
2 err = np.linalg.norm(Id - QT @ QT.T, ord=2)
3 print(f' ||I - Q^T*Q||_2 = {err}')
```

$$||I - Q^T Q||_2 = 5.002249963581562\text{e-}05$$

Die Matrix A2 ist dabei eine Kopie von A in single Gleitkommadarstellung, damit wir vom Fehler die 2-Norm bestimmen können. Wir sehen somit, dass die QR-Zerlegung bis auf die Maschinengenauigkeit von half, die bei etwa $\text{eps} \approx 4 \cdot 10^{-4}$ liegt, genau bestimmt haben und die Matrix Q auf die verwendete Maschinengenauigkeit orthogonal ist. ◀

4.5 Überbestimmte Gleichungssysteme

In vielen Anwendungen treten lineare Gleichungssysteme auf, die eine unterschiedliche Anzahl von Gleichungen und Unbekannten besitzen:

$$Ax = b, \quad A \in \mathbb{R}^{n \times m}, \quad x \in \mathbb{R}^m, \quad b \in \mathbb{R}^n, \quad n \neq m.$$

Im Fall $n > m$ (also mehr Gleichungen als Unbekannte) sprechen wir von *überbestimmten* Gleichungssystemen, im Fall $n < m$ von *unterbestimmten* Gleichungssystemen. Ein effizi-

entes Verfahren zur Bestimmung von Lösungen von überbestimmten Gleichungssystemen beruht auf der QR-Zerlegung.

Die Untersuchung der eindeutigen Lösbarkeit von allgemeinen Gleichungssystemen mit rechteckiger Matrix A kann nicht mehr an deren Regularität ausgemacht werden. Stattdessen wissen wir, dass ein Gleichungssystem genau dann lösbar ist, falls der Rang der Matrix A gleich dem Rang der erweiterten Matrix $[A|b]$ ist. Gilt zusätzlich $\text{rang}(A) = m$, so ist die Lösung eindeutig. Ein unterbestimmtes lineares Gleichungssystem mit $n < m$ kann daher nie eindeutig lösbar sein. Wir betrachten in diesem Abschnitt ausschließlich überbestimmte Gleichungssysteme, d.h. den Fall $n > m$. In der Praxis treten solche überbestimmte Gleichungssysteme in der Ausgleichsrechnung und in Optimierungsaufgaben sehr häufig auf. Ein überbestimmtes Gleichungssystem ist im allgemeinen Fall nicht lösbar.

Beispiel 4.21 (Überbestimmtes Gleichungssystem)

Wir suchen das quadratische Interpolationspolynom

$$p(x) = a_0 + a_1x + a_2x^2,$$

gegeben durch die Vorschrift:

$$p\left(-\frac{1}{4}\right) = 0, \quad p\left(\frac{1}{2}\right) = 1, \quad p(2) = 0, \quad p\left(\frac{5}{2}\right) = 1$$

Dies ergibt vier Gleichungen für nur drei Unbekannte:

$$\begin{aligned} a_0 - \frac{1}{4}a_1 + \frac{1}{16}a_2 &= 0 \\ a_0 + \frac{1}{2}a_1 + \frac{1}{4}a_2 &= 1 \\ a_0 + 2a_1 + 4a_2 &= 0 \\ a_0 + \frac{5}{2}a_1 + \frac{25}{4}a_2 &= 1 \end{aligned}$$

Wir versuchen das lineare Gleichungssystem mit Gauß-Elimination zu lösen:

$$\left(\begin{array}{ccc|c} 1 & -\frac{1}{4} & \frac{1}{16} & 0 \\ 1 & \frac{1}{2} & \frac{1}{4} & 1 \\ 1 & 2 & 4 & 0 \\ 1 & \frac{5}{2} & \frac{25}{4} & 1 \end{array}\right) \rightarrow \left(\begin{array}{ccc|c} 1 & -\frac{1}{4} & \frac{1}{16} & 0 \\ 0 & 3 & \frac{3}{4} & 4 \\ 0 & 9 & \frac{63}{4} & 0 \\ 0 & 11 & \frac{99}{4} & 4 \end{array}\right) \rightarrow \left(\begin{array}{ccc|c} 1 & -\frac{1}{4} & \frac{1}{16} & 0 \\ 0 & 3 & 3 & 4 \\ 0 & 0 & \frac{27}{2} & -12 \\ 0 & 0 & 22 & -\frac{32}{4} \end{array}\right)$$

Das heißt, es müsste $a_2 = -8/9 \approx -0.89$ sowie $a_2 = -4/11 \approx -0.36$ gelten. ◀

In Kap. 9 betrachten wir das Interpolationsproblem im Detail. Wir werden sehen, dass dieses Ergebnis zu erwarten war: Zur eindeutigen Interpolation von $n + 1$ Punkten durch ein Polynom ist dieses vom Grad n zu wählen.

Bei überbestimmten Gleichungssystemen müssen wir die Zielstellung ändern. Wir können nicht mehr von der Lösung des Gleichungssystems sprechen. Stattdessen werden wir

einen Vektor $x \in \mathbb{R}^m$ als *beste Approximation* der Lösung charakterisieren. Wie gut die Approximation ist, können wir am Defekt festmachen:

$$d(x) = b - Ax \in \mathbb{R}^n$$

Eine beste Approximation $x \in \mathbb{R}^m$ ist diejenige, die den Defekt in einer Norm minimiert:

► **Definition 4.22 (Bestapproximation)** Es sei $A \in \mathbb{R}^{n \times m}$ mit $n > m$ und $b \in \mathbb{R}^n$. Durch $\|\cdot\|$ sei eine Norm auf $\mathbb{R}^n$ gegeben. Ein Vektor $x \in \mathbb{R}^m$ heißt *Bestapproximation* zu $Ax = b$, falls

$$\|b - Ax\| \leq \|b - Ay\| \quad \forall y \in \mathbb{R}^m.$$

Unterschiedliche Normen werden unterschiedliche Bestapproximationen definieren. Für den folgenden Satz ist es wesentlich, die euklidische Norm, induziert vom euklidischen Skalarprodukt, zu betrachten.

► **Definition 4.23 (Methode der kleinsten Fehlerquadrate)** Es sei $A \in \mathbb{R}^{n \times m}$ mit $n > m$ und $b \in \mathbb{R}^n$. Die *Methode der kleinsten Fehlerquadrate* definiert die Bestapproximation $x \in \mathbb{R}^m$ als diejenige Näherung an $Ax = b$, deren Defekt die kleinste euklidische Norm annimmt:

$$\|b - Ax\|_2 = \min_{y \in \mathbb{R}^m} \|b - Ay\|_2 \quad (4.7)$$

Die Lösung $x \in \mathbb{R}^m$ wird auch *Least-Squares-Lösung* genannt.

Die Suche nach der Bestapproximation in der euklidischen Norm ist eng mit der Bestapproximation von Funktionen verwandt, also der Approximation von Messwerten mit einfachen Funktionen (dem „fitten“), eine Aufgabe, die wir in Kap. 9 betrachten werden. Es gilt:

Satz 4.24 (Kleinste Fehlerquadrate) Angenommen, für die Matrix $A \in \mathbb{R}^{n \times m}$ mit $n > m$ gilt $\text{rang}(A) = m$. Dann ist die Matrix $A^T A \in \mathbb{R}^{m \times m}$ positiv definit und die Bestapproximation $x \in \mathbb{R}^m$ ist eindeutig bestimmt als Lösung des *Normalgleichungssystems*

$$A^T A x = A^T b.$$

Beweis (i) Es gilt $\text{rang}(A) = m$. Das heißt, $Ax = 0$ gilt nur dann, wenn $x = 0$. Hieraus folgt die positive Definitheit der Matrix $A^T A$:

$$(A^T A x, x)_2 = (Ax, Ax)_2 = \|Ax\|_2^2 > 0 \quad \forall x \neq 0,$$

und das Gleichungssystem $A^T A x = A^T b$ ist für jede rechte Seite $b \in \mathbb{R}^n$ eindeutig lösbar.

(ii) Es sei $x \in \mathbb{R}^m$ die Lösung des Normalgleichungssystems. Dann gilt für beliebiges $y \in \mathbb{R}^m$

$$\begin{aligned}
\|b - A(x + y)\|_2^2 &= \|b - Ax\|_2^2 + \|Ay\|_2^2 - 2(b - Ax, Ay)_2 \\
&= \|b - Ax\|_2^2 + \|Ay\|_2^2 - \underbrace{2(A^T b - A^T Ax, y)_2}_{=0} \geq \|b - Ax\|_2^2.
\end{aligned}$$

(iii) Nun sei x das Minimum von (4.7). Das heißt, es gilt für beliebigen Vektor y :

$$\begin{aligned}
\|b - Ax\|_2^2 &\leq \|b - A(x + y)\|_2^2 \\
&= \|b - Ax\|_2^2 + \|Ay\|_2^2 - 2(b - Ax, Ay)_2 \quad \forall y \in \mathbb{R}^m
\end{aligned}$$

Hieraus folgt:

$$2(A^T b - A^T Ax, y)_2 \leq \|Ay\|_2^2$$

Wir wählen y als $y = se_i$ mit dem i -ten Einheitsvektor und $s \in \mathbb{R}$. Dann folgt

$$2s(A^T b - A^T Ax)_i \leq |s|^2 \|A\|^2 \Rightarrow |(A^T b - A^T Ax)_i| \leq |s| \|A\|^2$$

und für $s \rightarrow 0$ gilt $A^T Ax = A^T b$. □

Die beste Approximation (in der euklidischen Norm) eines überbestimmten Gleichungssystems kann durch Lösen des Normalgleichungssystems gefunden werden. Der naive Ansatz, die Matrix $A^T A$ zu bestimmen und dann das Normalgleichungssystem etwa mit dem Cholesky-Verfahren zu lösen, ist numerisch nicht ratsam. Zunächst ist der Aufwand zur Berechnung von $A^T A$ sehr groß und die Berechnung der Matrix-Matrix-Multiplikation schlecht konditioniert. Weiter gilt die Abschätzung

$$\text{cond}(A^T A) = \|A^T A\| \cdot \|(A^T A)^{-1}\|,$$

was auf eine potentiell große Konditionszahl von $A^T A$ hinweist. Bei einer regulären Matrix folgt die Abschätzung $\text{cond}(A^T A) \leq \text{cond}(A)^2$. In einem Beispiel untersuchen wir dennoch diese - vermutlich nicht stabile - Methode. Wir betrachten wieder das überbestimmte Gleichungssystem aus Beispiel 4.21.

Beispiel 4.25 (Lösen der Normalgleichung)

Die exakte Lösung des Normalgleichungssystems $A^T Ax = A^T b$ ist gegeben durch

$$x \approx \begin{pmatrix} 0.3425 \\ 0.3840 \\ -0.1131 \end{pmatrix} \Rightarrow p(x) = 0.3425 + 0.3740x - 0.1131x^2. \quad (4.8)$$

Es gilt

$$\|b - Ax\|_2 \approx 0.947.$$

Wir stellen das Normalgleichungssystem mit dreistelliger Rechnung auf:

$$A^T A = \begin{pmatrix} 4 & 4.75 & 10.6 \\ 4.75 & 10.6 & 23.7 \\ 10.6 & 23.7 & 55.1 \end{pmatrix}, \quad A^T b = \begin{pmatrix} 2 \\ 3 \\ 6.5 \end{pmatrix}$$

Bei dreistelliger Arithmetik bestimmen wir die Cholesky-Zerlegung mit dem direkten Verfahren aus Algorithmus 3.5.

$$\begin{aligned} l_{11} &= \sqrt{4} = 2 \\ l_{21} &= 4.75/2 \approx 2.38 \\ l_{31} &= 10.6/2 = 5.3 \\ l_{22} &= \sqrt{10.6 - 2.38^2} \approx 2.22 \\ l_{32} &= (23.7 - 2.38 \cdot 5.3)/2.22 \approx 5 \\ l_{33} &= \sqrt{55.1 - 5.3^2 - 5^2} \approx 1.42 \end{aligned} \quad , \quad L := \begin{pmatrix} 2 & 0 & 0 \\ 2.38 & 2.22 & 0 \\ 5.3 & 5 & 1.42 \end{pmatrix}$$

Wir lösen

$$L \underbrace{L^T x}_{=y} = A^T b \Rightarrow y \approx \begin{pmatrix} 1 \\ 0.279 \\ -0.137 \end{pmatrix} \Rightarrow \tilde{x} \approx \begin{pmatrix} 0.348 \\ 0.345 \\ -0.0972 \end{pmatrix}$$

mit dem Polynom

$$p(x) = 0.348 + 0.345x - 0.0972x^2$$

und dem Defekt $\|b - A\tilde{x}\| \approx 0.95$ sowie dem relativen Fehler zur exakten Lösung x aus (4.8)

$$\frac{\|\tilde{x} - x\|}{\|x\|} \approx 0.08.$$

Eine Abweichung von 8 % bei einer 4×3 -Matrix weist bereits auf eine wesentliche Fehlerverstärkung hin. ◀

Wir entwickeln ein alternatives Verfahren, welches das Aufstellen des Normalgleichungssystems $A^T A x = A^T b$ umgeht, und nutzen hierfür die QR-Zerlegung für nicht-quadratische Matrizen, die wir in Bemerkung 4.6 beschrieben haben. Für $A \in \mathbb{R}^{n \times m}$ ist

$$A = QR$$

mit $Q \in \mathbb{R}^{n \times m}$ und $R \in \mathbb{R}^{m \times m}$. Alternativ kann R zu $\tilde{R} \in \mathbb{R}^{n \times m}$ um $n - m$ Nullzeilen und Q zu $\tilde{Q} \in \mathbb{R}^{n \times n}$ um weitere $n - m$ orthogonale Vektoren erweitert werden.

Mit dieser verallgemeinerten QR-Zerlegung gilt nun:

$$A^T A x = A^T b \Leftrightarrow (QR)^T QR x = (QR)^T b \Leftrightarrow R^T R x = R^T Q^T b$$

Die (kleine) Matrix $R \in \mathbb{R}^{m \times m}$ ist regulär, also ergibt sich die Lösung des Normalgleichungssystems als

$$Rx = Q^T b.$$

Beispiel 4.26 (Bestapproximation eines überbestimmten Gleichungssystems mit erweiterter QR-Zerlegung)

Für das überbestimmte lineare Gleichungssystem aus Beispiel 4.21 gilt

$$A = \begin{pmatrix} 1 & -\frac{1}{4} & \frac{1}{16} \\ 1 & \frac{1}{2} & \frac{1}{4} \\ 1 & 2 & 4 \\ 1 & \frac{5}{2} & \frac{25}{4} \end{pmatrix}, \quad b = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}.$$

Wir wählen im ersten Schritt der Householder-Transformation (dreistellige Rechnung)

$$v^{(1)} = \frac{a_1 + \|a_1\|e_1}{\|a_1 + \|a_1\|e_1\|} \approx \begin{pmatrix} 0.866 \\ 0.289 \\ 0.289 \\ 0.289 \end{pmatrix}.$$

Dann gilt mit $a_i^{(1)} = a_i - 2(a_i, v^{(1)})v^{(1)}$

$$\tilde{A}^{(1)} \approx \begin{pmatrix} -2 & -2.38 & -5.29 \\ 0 & -0.210 & -1.54 \\ 0 & 1.29 & 2.21 \\ 0 & 1.79 & 4.46 \end{pmatrix} =: (a_1^{(1)}, a_2^{(1)}, a_3^{(1)}).$$

Mit dem reduzierten Vektor $\tilde{a}_2^{(1)} = (-0.21, 1.29, 1.79)^T$ gilt weiter

$$\tilde{v}^{(2)} = \frac{\tilde{a}_2^{(1)} - \|\tilde{a}_2^{(1)}\|\tilde{e}_2}{\|\tilde{a}_2^{(1)} - \|\tilde{a}_2^{(1)}\|\tilde{e}_2\|} \approx \begin{pmatrix} -0.740 \\ 0.393 \\ 0.546 \end{pmatrix}.$$

Und hiermit

$$\tilde{A}^{(2)} \approx \begin{pmatrix} -2 & -2.38 & -5.29 \\ 0 & 2.22 & 5.04 \\ 0 & 0 & -1.28 \\ 0 & 0 & -0.392 \end{pmatrix} =: (a_1^{(2)}, a_2^{(2)}, a_3^{(2)}).$$

Wir müssen einen dritten Schritt durchführen und mit $\tilde{a}_3^{(2)} = (-1.28, -0.392)$ ergibt sich nach gleichem Prinzip

$$\tilde{v}^{(3)} = \begin{pmatrix} -0.989 \\ -0.148 \end{pmatrix}.$$

Schließlich erhalten wir

$$\tilde{R} = \tilde{A}^{(3)} \approx \begin{pmatrix} -2 & -2.38 & -5.29 \\ 0 & 2.22 & 5.04 \\ 0 & 0 & 1.34 \\ 0 & 0 & 0 \end{pmatrix}.$$

Zum Lösen des Ausgleichsystems

$$A^T A x = A^T b \Leftrightarrow R x = \tilde{b}$$

bestimmen wir zunächst die rechte Seite nach der Vorschrift $b^{(i)} = b^{(i-1)} - 2(b^{(i-1)}, v^{(i)}) v^{(i)}$:

$$\tilde{b} = \begin{pmatrix} -1 \\ 0.28 \\ -0.155 \end{pmatrix}$$

Abschließend lösen wir durch Rückwärtseinsetzen $R x = \tilde{b}^{(3)}$

$$\tilde{x} \approx \begin{pmatrix} 0.343 \\ 0.389 \\ -0.116 \end{pmatrix}$$

und erhalten das Interpolationspolynom

$$p(x) = 0.343 + 0.389x - 0.116x^2$$

mit Defekt

$$\|b - A\tilde{x}\|_2 \approx 0.948$$

und relativem Fehler zur exakten Least-Squares-Lösung

$$\frac{\|\tilde{x} - x\|}{\|x\|} \approx 0.01,$$

d. h. ein Fehler von etwa 1 % anstelle von fast 8 % beim direkten Lösen des Normalsystems. Wir fassen die Ergebnisse noch einmal zusammen: Die exakte Bestapproximation x , die Cholesky-Lösung des Normalgleichungssystems x_{LL} und die Lösung mit erweiterter QR-Zerlegung x_{QR} sind gegeben durch

$$x \approx \begin{pmatrix} 0.343 \\ 0.384 \\ -0.113 \end{pmatrix}, \quad x_{LL} \approx \begin{pmatrix} 0.348 \\ 0.345 \\ -0.0972 \end{pmatrix}, \quad x_{QR} \approx \begin{pmatrix} 0.343 \\ 0.389 \\ -0.116 \end{pmatrix}.$$

Der Defektvektor ist jeweils gegeben als

$$b - Ax \approx \begin{pmatrix} -0.240 \\ 0.493 \\ -0.659 \\ 0.403 \end{pmatrix}, \quad b - Ax_{LL} \approx \begin{pmatrix} -0.256 \\ 0.503 \\ -0.649 \\ 0.397 \end{pmatrix}, \quad b - Ax_{QR} \approx \begin{pmatrix} -0.238 \\ 0.492 \\ -0.657 \\ 0.410 \end{pmatrix}.$$

Eine Lösung des Gleichungssystems ist in keinem der Fälle gegeben. Es kann nur um die Bestimmung einer besten Approximation im Sinne des Defekts gehen.

In Python können wir unsere Implementierung der QR-Zerlegung nach Householder wiederverwenden. Da wir aber eine andere rechte Seite $\tilde{b}$ brauchen, müssen wir die Anwendung der Matrix Q^T ebenfalls anpassen.

```
1 def b_tilde(V, b):
2     b = b.copy()
3     for i in range(len(V)):
4         b[i:] -= 2 * np.inner(V[i], b[i:]) * V[i]
5     return np.array(b[:len(V)])
```

Wenden wir dies auf das obige Beispiel an, erhalten wir

```
1 V = qr_householder(A)
2 bt = b_tilde(V, b)
3 x_h = rueckwaerts_einsetzen(A[:len(V),:], bt)
4 print(f'x = {x_h}')
```

```
x = [ 0.3423  0.3804 -0.1113]
```

Damit ergibt sich der Defekt

```
1 print('||Ax - b|| = ', np.linalg.norm(np.inner(A2, x) - b))
```

```
||Ax - b|| = 0.9473
```

Alternativ können wir unsere QR-Zerlegung mit Givens-Rotationen verwenden. Damit ergibt sich die Lösung

```
1 QT = qr_givens(A)
2 bt = np.dot(QT, b)[:A.shape[1]]
3 x_g = rueckwaerts_einsetzen(A[:A.shape[1],:], bt)
4 print(f'x = {x_g}')
```

```
x = [ 0.3425  0.384  -0.113 ]
```

mit dem Defekt

```
1 print('||Ax - b|| = ', np.linalg.norm(np.inner(A_neu, x_g) - b))
```

```
||Ax - b|| = 0.9473
```



In Abschn. 9.5.1 werden wir die diskrete Gauß-Approximation als Methode zur Näherung von Funktionen an Messwerte betrachten. Diese ist eine Anwendung der Methode der kleinsten Fehlerquadrate. Die lineare Ausgleichsrechnung, auch *lineare Regression* genannt, ist ein Spezialfall der diskreten Gauß-Approximation. Hier gilt es, Paare von Messwerten (x_i, y_i) für $i = 1, \dots, n$ möglichst gut durch einen linearen Zusammenhang $y(x) = ax + b$ darzustellen (siehe auch Exkurs 11.4). Als Anwendung der Bestapproximation überbestimmter Gleichungssysteme führt dies auf Matrizen der Größe $A \in \mathbb{R}^{n \times 2}$.

Eine weitere Anwendung der QR-Zerlegung werden wir in Kap. 5 kennenlernen. Eines der leistungsfähigsten Verfahren zur Approximation der Eigenwerte einer Matrix $A \in \mathbb{R}^{n \times n}$ basiert auf der wiederholten Anwendung der QR-Zerlegung.

4.6 Exkurs: Astronomische Hauptachsenbestimmung eines Himmelskörpers

In diesem Exkurs stellen wir eine historische Anwendung von überbestimmten Gleichungssystemen vor. Die ursprüngliche Entwicklung der Methode von Carl Friedrich Gauß war genau durch diese Problemstellung gegeben. Der Italiener Piazzi beobachtete 40 Tage lang im Jahr 1801 den Planeten Ceres und verlor ihn dann aus dem Blick [36]. Gauß nahm die vorhandenen Daten, benutzte die Methode der kleinsten Quadrate, um dann numerisch (natürlich noch ohne Computer) die Planetenlaufbahn zu berechnen. Hierdurch konnten die Astronomen den Planeten tatsächlich wiederfinden. Ein Himmelskörper bewegt sich auf einer Ellipsenbahn um den Ursprung

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1.$$

Wir können die Position des Körpers zu verschiedenen Zeiten messen:

t	1	2	3	4	5	6
x	0.2	-0.7	-0.3	1.2	2.0	-2.4
y	1.8	1.7	-1.8	-1.6	1.1	0.5

Aus diesen Daten wollen wir die Längen der beiden Hauptachsen a und b bestimmen. Die unbekannten Größen a und b tauchen in einem nichtlinearen Zusammenhang auf. Daher

definieren wir zunächst die beiden Hilfsgrößen

$$\alpha = \frac{1}{a^2}, \quad \beta = \frac{1}{b^2}.$$

Es entsteht das überbestimmte Gleichungssystem

$$\begin{pmatrix} 0.04 & 3.24 \\ 0.49 & 2.56 \\ 0.09 & 3.24 \\ 1.44 & 2.56 \\ 4.00 & 1.21 \\ 5.76 & 0.25 \end{pmatrix} \begin{pmatrix} \alpha \\ \beta \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{pmatrix}.$$

Zum Lösen dieses überbestimmten Problems könnten wir einerseits unmittelbar die Normalgleichung aufstellen und dann lösen. Es gilt

$$A^T A \approx \begin{pmatrix} 51.50 & 11.64 \\ 11.64 & 35.63 \end{pmatrix}, \quad A^T b \approx \begin{pmatrix} 11.82 \\ 13.06 \end{pmatrix}.$$

Als Lösung von $A^T A x = A^T b$ folgt

$$x = \begin{pmatrix} \alpha \\ \beta \end{pmatrix} \approx \begin{pmatrix} 0.158 \\ 0.315 \end{pmatrix}, \quad a = \frac{1}{\sqrt{\alpha}} \approx 2.51, \quad b = \frac{1}{\sqrt{\beta}} \approx 1.78.$$

Ein alternativer Lösungsweg besteht im Erstellen der QR-Zerlegung von A . Für $A \in \mathbb{R}^{6 \times 2}$ müssen wir zwei Schritte durchführen. Wir erhalten

$$A = QR, \quad R \approx \begin{pmatrix} -7.18 & -1.62 \\ 0 & -5.74 \\ \hline 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}$$

und für die rechte Seite

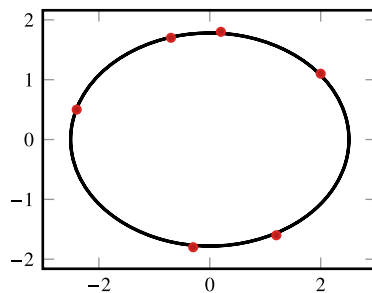
$$\tilde{b} = Q^T b \approx \begin{pmatrix} -1.65 \\ -1.81 \end{pmatrix}.$$

Die Lösung des Gleichungssystems ergibt sich wie oben zu

$$\tilde{R}x = \tilde{b} \Rightarrow x \approx \begin{pmatrix} 0.158 \\ 0.315 \end{pmatrix}.$$

In Abb. 4.2 ist das Ergebnis grafisch dargestellt. Für den Defekt der Least-Squares-Lösung gilt

Abb. 4.2 Positionen des Himmelskörpers auf einer Ellipse zu unterschiedlichen Zeiten



$$\|b - Ax\|_2 \approx 0.13,$$

die Lösung $x \in \mathbb{R}^2$ ist Bestapproximation, jedoch nicht Lösung des überbestimmten linearen Gleichungssystems. Der Defekt von 0.13 sagt nichts über die Qualität der numerischen Lösung aus, sondern eher über die Qualität der Daten. Falls das postulierte elliptische Gesetz für die Planetenbahn exakt ist, gibt der Defekt ein Maß für den Messfehler der Beobachtungen an.

In diesem Abschnitt befassen wir uns mit dem Eigenwertproblem: Zu einer Matrix $A \in \mathbb{R}^{n \times n}$ sind die Eigenwerte (und gegebenenfalls Eigenvektoren) gesucht. Wir erinnern an Definition 2.11, d. h.

$$Aw = \lambda w \quad (5.1)$$

mit $A \in \mathbb{C}^{n \times n}$, Eigenvektor $w \in \mathbb{C}^n$ und Eigenwert $\lambda \in \mathbb{C}$. Im Folgenden schränken wir uns auf den reellwertigen Fall und formulieren

Satz 5.1 (Eigenwerte) Es sei $A \in \mathbb{R}^{n \times n}$. Dann gilt:

1. Die Matrix A hat genau n Eigenwerte, ihrer Vielfachheit nach gezählt.
2. Die Eigenwerte sind Nullstellen des *charakteristischen Polynoms*

$$\det(A - \lambda I) = 0.$$

3. Reelle Matrizen können komplexe Eigenwerte haben. Komplexe Eigenwerte reeller Matrizen treten stets als konjugierte Paare $\lambda, \bar{\lambda}$ auf.
4. Eigenvektoren zu unterschiedlichen Eigenwerten sind linear unabhängig.
5. Falls n linear unabhängige Eigenvektoren existieren, so existiert eine reguläre Matrix $S \in \mathbb{R}^{n \times n}$, sodass $S^{-1}AS = D$ eine Diagonalmatrix ist. Die Spaltenvektoren von S sind die Eigenvektoren und die Diagonaleinträge von D die Eigenwerte.
6. Reelle symmetrische Matrizen $A \in \mathbb{R}^{n \times n}$ haben nur reelle Eigenwerte. Es existiert eine Orthonormalbasis von Eigenvektoren und eine Diagonalisierung $Q^T A Q = D$ mit einer orthogonalen Matrix bestehend aus den Eigenvektoren.
7. Bei Dreiecksmatrizen stehen die Eigenwerte auf der Diagonalen.

Beweise für die einzelnen Aussagen finden sich zum Beispiel in [32]. Wir betrachten hier nur die reelle Situation, d. h., reelle Matrizen $A \in \mathbb{R}^{n \times n}$.

Zu einem Eigenwert $\lambda \in \mathbb{C}$ sind die Eigenvektoren als Lösung des homogenen linearen Gleichungssystems bestimmt:

$$(A - \lambda I)w = 0$$

Umgekehrt ergibt sich sofort

Satz 5.2 (Rayleigh-Quotient) Sei $A \in \mathbb{R}^{n \times n}$ sowie $w \in \mathbb{R}^n$ ein Eigenvektor. Dann ist durch den *Rayleigh-Quotienten* der zugehörige Eigenwert gegeben:

$$\lambda = \frac{(Aw, w)_2}{\|w\|_2^2}$$

Mithilfe dieser Definition folgt mit der Cauchy-Schwarzschen Ungleichung eine einfache Schranke für die Eigenwerte einer Matrix, denn

$$|\lambda| \leq \sup_{w \neq 0} \frac{(Aw, w)_2}{\|w\|_2^2} \leq \frac{\|A\|_2 \|w\|_2^2}{\|w\|_2^2} = \|A\|_2.$$

Wir haben in Kap. 2 bereits gesehen, dass diese Schranke für die Beträge der Eigenwerte in jeder Matrixnorm mit verträglicher Vektornorm gilt. Dies ist die erste und grösste Eingrenzung von Eigenwerten.

5.1 Konditionierung der Eigenwertaufgabe

Bevor wir auf konkrete Verfahren zur Eigenwertberechnung eingehen, analysieren wir die Kondition der Aufgabe, d. h. die Abhängigkeit der Eigenwerte von der Störung der Matrix. Hierfür benötigen wir zunächst einen Hilfssatz:

Hilfssatz 5.3 Es seien $A, B \in \mathbb{R}^{n \times n}$ beliebige Matrizen. Dann gilt für jeden Eigenwert λ von A , der nicht zugleich Eigenwert von B ist, in jeder natürlichen Matrixnorm die Abschätzung

$$\|(\lambda I - B)^{-1}(A - B)\| \geq 1.$$

Beweis Sei λ Eigenwert von A mit zugehörigem Eigenvektor $w \neq 0$. Dann gilt

$$(A - B)w = (\lambda I - B)w.$$

Wenn λ kein Eigenwert von B ist, so ist die Matrix $(\lambda I - B)$ regulär, d. h., es folgt

$$(\lambda I - B)^{-1}(A - B)w = w.$$

Und schließlich durch Normbildern

$$\begin{aligned} 1 &= \frac{\|(\lambda I - B)^{-1}(A - B)w\|}{\|w\|} \leq \sup_{x \neq 0} \frac{\|(\lambda I - B)^{-1}(A - B)x\|}{\|x\|} \\ &= \|(\lambda I - B)^{-1}(A - B)\|. \end{aligned} \quad \square$$

Auf dieser Basis erhalten wir ein zweites, verfeinertes, einfaches Kriterium zur Eingrenzung der Eigenwerte einer Matrix:

Satz 5.4 (Gerschgorin-Kreise) Es sei $A \in \mathbb{R}^{n \times n}$. Alle Eigenwerte λ von A liegen in der Vereinigung der *Gerschgorin-Kreise*:

$$K_i = \left\{ z \in \mathbb{C} : |z - a_{ii}| \leq \sum_{k=1, k \neq i}^n |a_{ik}| \right\}, \quad i = 1, \dots, n$$

Angenommen, zu den Indexmengen $I_m = \{i_1, \dots, i_m\}$ und $I'_m = \{1, \dots, n\} \setminus I_m$ seien die Vereinigungen $U_m = \bigcup_{i \in I_m} K_i$ und $U'_m = \bigcup_{i \in I'_m} K_i$ disjunkt. Dann liegen genau m Eigenwerte (ihrer algebraischen Vielfachheit nach gezählt) in U_m und $n - m$ Eigenwerte in U'_m .

Beweis (i) Es sei $D \in \mathbb{R}^{n \times n}$ die Diagonalmatrix $D = \text{diag}(a_{ii})$. Weiter sei λ ein Eigenwert von A mit $\lambda \neq a_{ii}$. In diesem Fall wäre die Aussage des Satzes trivialerweise erfüllt. Hilfssatz 5.3 besagt bei Wahl der maximalen Zeilensummennorm:

$$\begin{aligned} 1 \leq \|(\lambda I - D)^{-1}(A - D)\|_\infty &= \max_i \left\{ \sum_{j=1, j \neq i}^n |(\lambda - a_{ii})^{-1} a_{ij}| \right\} \\ &= \max_i \left\{ |\lambda - a_{ii}|^{-1} \sum_{j=1, j \neq i}^n |a_{ij}| \right\} \end{aligned}$$

Das heißt, es existiert zu jedem λ ein Index $i \in \{1, \dots, n\}$, sodass gilt:

$$|\lambda - a_{ii}| \leq \sum_{j=1, j \neq i}^n |a_{ij}|$$

Jeder Eigenwert λ liegt also in mindestens einem der Gerschgorin-Kreise.

(ii) Es seien durch I_m und I'_m disjunkte Indexmengen mit $I_m \cup I'_m = \{1, \dots, n\}$ gegeben so dass die Vereinigungen U_m und U'_m ihrer Kreise auch disjunkt sind. Wir definieren für $s \in \mathbb{R}$ die Matrix

$$A_s := D + s(A - D)$$

mit $A_0 = D$ und $A_1 = A$. Die Eigenwerte von A_s bezeichnen wir mit $\lambda_{s,i}$. Entsprechend definieren wir die Vereinigungen:

$$U_{m,s} := \bigcup_{i \in I_m} K_i(A_s), \quad U'_{m,s} := \bigcup_{i \in I'_m} K_i(A_s),$$

$$K_i(A_s) = \left\{ z \in \mathbb{C}, |z - a_{ii}| \leq s \sum_{j \neq i} |a_{ij}| \right\}$$

Es ist $U_{m,1} = U_m$ und $U'_{m,1} = U'_m$ und diese beiden Mengen sind disjunkt. Für $s = 0$ bestehen die Kreise alle nur aus einem Punkt und enthalten jeweils genau den einen Eigenwert $\lambda_{i,0} = a_{ii}$. Die Kreisradien hängen linear von s ab und daher bleiben für $s \in [0, 1]$ die Kreisvereinigungen disjunkt. Da schließlich die Eigenwerte der Matrix stetig von den Matrixeinträgen abhängen, folgt die Aussage des Satzes. $\square$

Durch die Wahl anderer Matrixnormen können ähnliche Kriterien hergeleitet werden:

Bemerkung 5.5 (Eigenwerte von A^T) Bei Wahl der 1-Norm, also $\|\cdot\|_1$ der maximalen Spaltensumme folgt entsprechend

$$|\lambda - a_{ii}| \leq \sum_{i=1, j \neq i}^n |a_{ji}|,$$

d. h., die Spaltensummen bilden den Radius der entsprechenden Kreise. Entsprechend könnte man das Gerschgorin-Kriterium auf die transponierte Matrix A^T anwenden, denn aus $\det(A) = \det(A^T)$ folgt, dass A und A^T die gleichen Eigenwerte haben. Wir können die Kreisradien somit entweder anhand der Zeilen- oder der Spaltensumme bestimmen. $\blacklozenge$

Bemerkung 5.6 (Stetige Abhängigkeit der Eigenwerte von den Matrixeinträgen) Die Eigenwerte sind Nullstellen des charakteristischen Polynoms

$$p_A(\lambda) = \det(A - \lambda I) = \alpha_0 + \alpha_1 \lambda + \cdots + \alpha_n \lambda^n.$$

Die Koeffizienten $\alpha_0, \dots, \alpha_n$ hängen stetig von den Einträgen a_{ij} der Matrix A ab. Schwieriger zu zeigen ist, dass die (komplexen) Nullstellen eines Polynoms auch stetig von den Koeffizienten abhängen. Wir verweisen auf die Literatur, beispielsweise Abschn. 3.9 in [86]. $\blacklozenge$

Beispiel 5.7 (Gerschgorin-Kreise)

Wir betrachten die Matrix

$$A = \begin{pmatrix} 2 & 0.1 & -0.5 \\ 0.2 & 3 & 0.5 \\ -0.4 & 0.1 & 5 \end{pmatrix}$$

mit den Eigenwerten $\Lambda(A) \approx \{1.91, 3.01, 5.08\}$. Eine erste Schranke liefern verschiedene, mit Vektornormen verträgliche Matrixnormen von A , z. B.

$$\|A\|_\infty = 5.5, \quad \|A\|_1 = 6, \quad \|A\|_2 \approx 5.1.$$

Die Gerschgorin-Kreise von A sind

$$K_1 = \{z \in \mathbb{C}, |z - 2| \leq 0.6\},$$

$$K_2 = \{z \in \mathbb{C}, |z - 3| \leq 0.7\},$$

$$K_3 = \{z \in \mathbb{C}, |z - 5| \leq 0.5\}.$$

Diese Abschätzung kann verfeinert werden, da A und A^T die gleichen Eigenwerte besitzen. Die Radien der Gerschgorin-Kreise können auch als Summe der Spaltenbeträge berechnet werden. Zusammen ergibt sich

$$K_1 = \{z \in \mathbb{C}, |z - 2| \leq 0.6\},$$

$$K_2 = \{z \in \mathbb{C}, |z - 3| \leq 0.2\},$$

$$K_3 = \{z \in \mathbb{C}, |z - 5| \leq 0.5\}.$$

Alle drei Kreise sind disjunkt, d. h., in jedem der Kreise liegt genau ein Eigenwert. ◀

Bemerkung 5.8 (Gerschgorin-Kreise) Man könnte aus dem obigen Beispiel leicht die folgende Schlussfolgerung ziehen:

„Alle Eigenwerte einer Matrix A liegen in den Gerschgorin-Kreisen. Diese sind als Kreise um die Diagonalelemente a_{ii} mit Radius $\min\{\sum_{j \neq i} |a_{ij}|, \sum_{j \neq i} |a_{ji}|\}$ gegeben, also jeweils dem Minimum aus Zeilen- und Spaltensumme.“

Diese Verschärfung des Satzes ist im Allgemeinen jedoch falsch. Ebenso wäre der Schluss falsch, dass in jedem der Kreise mindestens ein Eigenwert liegen muss, auch wenn diese nicht disjunkt sind. Für beide Fälle lassen sich leicht Gegenbeispiele angeben. ♦

Wir können nun den allgemeinen Stabilitätssatz für das Eigenwertproblem beweisen.

Satz 5.9 (Stabilität des Eigenwertproblems) Es sei $A \in \mathbb{R}^{n \times n}$ eine Matrix mit n linear unabhängigen Eigenvektoren $w_1, \dots, w_n$. Durch $\tilde{A} = A + \delta A$ sei eine beliebig gestörte

Matrix gegeben. Dann existiert zu jedem Eigenwert $\lambda(\tilde{A})$ von $\tilde{A} = A + \delta A$ ein Eigenwert $\lambda(A)$ von A , sodass mit der Matrix $W := (w_1, \dots, w_n)$ gilt:

$$|\lambda(A) - \lambda(\tilde{A})| \leq \text{cond}_2(W) \|\delta A\|$$

Beweis Es gilt für $i = 1, \dots, n$ die Beziehung $Aw_i = \lambda_i(A)w_i$ oder in Matrixschreibweise $AW = W \text{diag}(\lambda_i(A))$, also

$$W^{-1}AW = \text{diag}(\lambda_i(A)).$$

Da die w_i linear unabhängig sind, ist W regulär. Wir betrachten nun einen Eigenwert $\tilde{\lambda} = \lambda(\tilde{A})$. Falls $\tilde{\lambda}$ auch Eigenwert von A ist, so gilt die Behauptung. Also sei $\tilde{\lambda}$ nun kein Eigenwert von A . Dann folgt

$$\begin{aligned} \|(\tilde{\lambda}I - A)^{-1}\|_2 &= \|W^{-1}[\tilde{\lambda}I - \text{diag}(\tilde{\lambda}_i(A))]\|_2 \\ &\leq \text{cond}_2(W) \|[\tilde{\lambda}I - \text{diag}(\lambda_i(A))]\|_2^{-1}. \end{aligned}$$

Für die (symmetrische) Diagonalmatrix $\tilde{\lambda}I - \text{diag}(\lambda_i(A))$ gilt

$$\|[\tilde{\lambda}I - \text{diag}(\lambda_i(A))]\|_2^{-1} = \max_{i=1, \dots, n} |\tilde{\lambda} - \lambda_i(A)|^{-1}.$$

Da $\delta A = \tilde{A} - A$, folgt mit Hilfssatz 5.3 das gewünschte Ergebnis:

$$\begin{aligned} 1 \leq \|[\tilde{\lambda}I - A]^{-1}\delta A\|_2 &\leq \|[\tilde{\lambda}I - A]^{-1}\|_2 \|\delta A\|_2 \\ &\leq \text{cond}_2(W) \max_{i=1, \dots, n} |\tilde{\lambda} - \lambda_i(A)|^{-1} \|\delta A\|_2 \quad \square \end{aligned}$$

Die Konditionierung des Eigenwertproblems einer Matrix A hängt von der Konditionszahl der Matrix der Eigenvektoren w_i ab. Für symmetrische (hermitesche) Matrizen existiert eine Orthonormalbasis von Eigenvektoren mit $\text{cond}_2(W) = 1$. Für solche Matrizen ist das Eigenwertproblem gut konditioniert. Für allgemeine Matrizen kann das Eigenwertproblem beliebig schlecht konditioniert sein.

5.2 Direkte Methode

Die Eigenwerte einer Matrix A können prinzipiell als Nullstellen des charakteristischen Polynoms $\chi_A(z) = \det(zI - A)$ berechnet werden. Die Berechnung der Nullstellen kann zum Beispiel mit dem Newton-Verfahren (Abschn. 8.3) geschehen und die erforderlichen Startwerte können mithilfe der Gerschgorin-Kreise bestimmt werden.

In Kap. 8 werden wir jedoch feststellen, dass die Berechnung von Nullstellen eines Polynoms ein sehr schlecht konditioniertes Problem ist. Wir betrachten hierzu ein Beispiel.

Beispiel 5.10 (Direkte Berechnung von Eigenwerten)

Es sei $A \in \mathbb{R}^{5 \times 5}$ eine Matrix mit den Eigenwerten $\lambda_i = i$, $i = 1, \dots, 5$. Dann gilt für das charakteristische Polynom $\chi_A(z) := \det(A - zI)$:

$$\chi_A(z) = \prod_{i=1}^5 (z - i) = z^5 - 15z^4 + 85z^3 - 225z^2 + 274z - 120$$

Der Koeffizient -15 vor z^4 sei mit einem relativen Fehler von 0.1% gestört:

$$\tilde{\chi}_A(z) = z^5 - 0.999 \cdot 15z^4 + 85z^3 - 225z^2 + 274z - 120$$

Dieses gestörte Polynom hat die Nullstellen (d. h. Eigenwerte)

$$\lambda_1 \approx 0.999, \quad \lambda_2 \approx 2.05, \quad \lambda_3 \approx 2.76, \quad \lambda_{4/5} \approx 4.59 \pm 0.430i.$$

Die Eigenwerte können also nur mit einem (ab λ_3) wesentlichen Fehler bestimmt werden. Es gilt etwa

$$\frac{|4.59 + 0.43i - 5|}{5} \approx 0.1,$$

d. h., der Fehler in den Eigenwerten beträgt 10%, was eine Fehlerverstärkung um den Faktor 100 bedeutet. ◀

Das Aufstellen des charakteristischen Polynoms erfordert eine Vielzahl schlecht konditionierter Additionen sowie Multiplikationen. Für allgemeine Matrizen verbietet sich dieses direkte Verfahren. Lediglich für spezielle Matrizen wie Tridiagonalsysteme lassen die Eigenwerte bestimmen, ohne dass zunächst die Koeffizienten des charakteristischen Polynoms explizit berechnet werden müssen.

5.3 Iterative Verfahren

Das vorangegangene Beispiel widerspricht scheinbar zunächst der (bei hermiteschen Matrizen) guten Konditionierung des Eigenwertproblems. Hier müssen wir jedoch zwischen der Konditionierung der Aufgabe und der Stabilität eines bestimmten Verfahrens unterscheiden. Auch wenn die Aufgabe gut konditioniert sein kann, so ist die Suche nach Nullstellen des charakteristischen Polynoms kein stabiles Verfahren (man vergleiche insbesondere Abschn. 1.3 und 1.5).

Als erstes iteratives Verfahren zur Approximation von Eigenwerten betrachten wir die *Potenzmethode nach von Mises*. Angenommen, zu gegebener Matrix $A \in \mathbb{R}^{n \times n}$ existiere eine Basis aus Eigenvektoren $\{w_1, \dots, w_n\}$, d. h., die Matrix sei diagonalisierbar. Weiter gelte $|\lambda_n| > |\lambda_{n-1}| \geq \dots \geq |\lambda_1| > 0$. Dann sei $x^{(0)} \in \mathbb{R}^n$ in Basisdarstellung

$$x^{(0)} = \sum_{j=1}^n \alpha_j w_j, \quad \alpha_j \in \mathbb{R} \text{ für } j = 1, \dots, n.$$

Wir nehmen an, dass $\alpha_n \neq 0$, dass also die Komponente des Vektors $\alpha = (\alpha_1, \dots, \alpha_n)$ zum Eigenvektor des betragsmäßig größten Eigenwerts nichttrivial ist. Wir definieren die Iteration

$$x^{(i+1)} = \frac{Ax^{(i)}}{\|Ax^{(i)}\|}, \quad i = 1, 2, \dots$$

Dann gilt

$$x^{(i+1)} = \frac{A \frac{Ax^{(i-1)}}{\|Ax^{(i-1)}\|}}{\|A \frac{Ax^{(i-1)}}{\|Ax^{(i-1)}\|}\|} = \frac{A^2 x^{(i-1)}}{\|A^2 x^{(i-1)}\|} \frac{\|Ax^{(i-1)}\|}{\|Ax^{(i-1)}\|} = \dots = \frac{A^{i+1} x^{(0)}}{\|A^{i+1} x^{(0)}\|}. \quad (5.2)$$

Weiter gilt mit der Basisdarstellung von $x^{(0)}$

$$A^i x^{(0)} = \sum_{j=1}^n \alpha_j \lambda_j^i w_j = \alpha_n \lambda_n^i \left(w_n + \sum_{j=1}^{n-1} \frac{\alpha_j}{\alpha_n} \frac{\lambda_j^i}{\lambda_n^i} w_j \right). \quad (5.3)$$

Es gilt $|\lambda_j/\lambda_n| < 1$ für $j < n$, daher folgt

$$A^i x^{(0)} = \sum_{j=1}^n \alpha_j \lambda_j^i w_j = \alpha_n \lambda_n^i (w_n + o(1)).$$

Hieraus folgt durch Normierung

$$\frac{A^i x^{(0)}}{\|A^i x^{(0)}\|} = \left(\frac{\alpha_n \lambda_n^i}{|\alpha_n \lambda_n^i|} \right) \frac{w_n}{\|w_n\|} + o(1) \rightarrow \beta w_n \quad (i \rightarrow \infty)$$

mit einem $\beta \in \mathbb{R}$. Die Iteration läuft in den Raum, der durch w_n aufgespannt wird. Für einen Vektor w , der Vielfaches eines Eigenvektors ist, $w = s w_n$, gilt

$$Aw = s Aw_n = s \lambda_n w_n = \lambda_n w.$$

Diese vektorwertige Gleichung gilt in jeder Komponente, kann daher nach dem Eigenwert aufgelöst werden:

$$\lambda_n = \frac{[Aw]_k}{w_k}$$

Wir fassen zusammen:

Satz 5.11 (Potenzmethode nach von Mises) Es sei $A \in \mathbb{R}^{n \times n}$ eine Matrix mit n linear unabhängigen Eigenvektoren $\{w_1, \dots, w_n\}$. Der betragsmäßig größte Eigenwert sei separiert $|\lambda_n| > |\lambda_{n-1}| \geq \dots \geq |\lambda_1|$. Es sei $x^{(0)} \in \mathbb{R}^n$ ein Startwert mit nichttrivialer Kompo-

nente in Bezug auf w_n . Für einen beliebigen Index $k \in \{1, \dots, n\}$ konvergiert die Iteration

$$\tilde{x}^{(i)} = A x^{(i-1)}, \quad x^{(i)} := \frac{\tilde{x}^{(i)}}{\|\tilde{x}^{(i)}\|}, \quad \lambda^{(i)} := \frac{\tilde{x}_k^{(i)}}{x_k^{(i-1)}},$$

gegen den betragsmäßig größten Eigenwert

$$|\lambda^{(i)} - \lambda_n| = O\left(\left|\frac{\lambda_{n-1}}{\lambda_n}\right|^i\right), \quad i \rightarrow \infty$$

Beweis Wir knüpfen an der Vorbereitung (5.2) des Beweises an. Es gilt

$$\lambda^{(i)} = \frac{\tilde{x}_k^{(i)}}{x_k^{(i-1)}} = \frac{[Ax^{(i-1)}]_k}{x_k^{(i-1)}} = \frac{[A^i x^{(0)}]_k}{[A^{i-1} x^{(0)}]_k}.$$

Weiter mit (5.3) gilt

$$\begin{aligned} \lambda^{(i)} &= \frac{a_n \lambda_n^i \left([w_n]_k + \sum_{j=1}^{n-1} \frac{\alpha_j}{\alpha_n} \frac{\lambda_j^i}{\lambda_n^i} [w_j]_k \right)}{a_n \lambda_n^{i-1} \left([w_n]_k + \sum_{j=1}^{n-1} \frac{\alpha_j}{\alpha_n} \frac{\lambda_j^{i-1}}{\lambda_n^{i-1}} [w_j]_k \right)} \\ &= \lambda_n \frac{[w_n]_k + \sum_{j=1}^{n-1} \frac{\alpha_j}{\alpha_n} \frac{\lambda_j^i}{\lambda_n^i} [w_j]_k}{[w_n]_k + \sum_{j=1}^{n-1} \frac{\alpha_j}{\alpha_n} \frac{\lambda_j^{i-1}}{\lambda_n^{i-1}} [w_j]_k} \\ &= \lambda_n \frac{[w_n]_k + \left(\frac{\alpha_{n-1}}{\alpha_n} + o(1) \right) \left| \frac{\lambda_{n-1}}{\lambda_n} \right|^i [w_{n-1}]_k}{[w_n]_k + \left(\frac{\alpha_{n-1}}{\alpha_n} + o(1) \right) \left| \frac{\lambda_{n-1}}{\lambda_n} \right|^{i-1} [w_{n-1}]_k} \\ &= \lambda_n \left(1 + O\left(\left| \frac{\lambda_{n-1}}{\lambda_n} \right|^{i-1} \right) \right). \end{aligned}$$

Die letzte Abschätzung nutzt für $0 < x < 1$ den Zusammenhang

$$\frac{1 + x^{i+1}}{1 + x^i} = 1 + \frac{x^{i+1} - x^i}{1 + x^i} = 1 + x^i \frac{x - 1}{1 + x^i} = 1 + O(|x|^i).$$

□

In Python kann dies zum Beispiel dann implementiert werden durch

♣ Implementierung 5.1: Potenzmethode nach von Mises

```

1 def potenzmethode_mises(A, x, n, k=0):
2     lams = []
3     for i in range(n):
4         x_neu = np.inner(A, x)
5         lams.append(x_neu[k] / x[k])
6         x = x_neu / np.linalg.norm(x_neu)
7     return lams

```

Die Potenzmethode ist eine sehr einfache und numerisch stabile Iteration. Sie kann allerdings nur den betragsmäßig größten Eigenwert ermitteln. Der Konvergenzbeweis kann verallgemeinert werden auf Matrizen, deren r betragsgrößte Eigenwerte mehrfach vorkommen. Konvergenz gilt jedoch nur dann, wenn aus $|\lambda_n| = |\lambda_{n-1}| = \dots = |\lambda_{n-r}| > |\lambda_{n-r-1}|$ auch $\lambda_n = \lambda_{n-1} = \dots = \lambda_{n-r}$ folgt. Es darf also nicht zwei verschiedene Eigenwerte geben, die beide den gleichen größten Betrag annehmen, in der komplexen Ebene aber unterschiedliche Punkte bezeichnen. Dies schließt zum Beispiel den Fall zweier komplex konjugierter Eigenwerte $\lambda \neq \bar{\lambda}$ als betragsgrößte aus.

Bemerkung 5.12 (Potenzmethode bei symmetrischen Matrizen) Die Potenzmethode kann im Fall symmetrischer Matrizen durch Verwenden des Rayleigh-Quotienten verbessert werden. Die Iteration

$$\lambda^{(i)} = \frac{(Ax^{(i-1)}, x^{(i-1)})_2}{(x^{(i-1)}, x^{(i-1)})_2} = (\tilde{x}^{(i)}, x^{(i-1)})_2$$

liefert die Fehlerabschätzung

$$\lambda^{(i)} = \lambda_n + O\left(\left|\frac{\lambda_{n-1}}{\lambda_n}\right|^{2i}\right).$$



Beispiel 5.13 (Potenzmethode nach von Mises)

Es sei

$$A = \begin{pmatrix} 2 & 1 & 2 \\ -1 & 2 & 1 \\ 1 & 2 & 4 \end{pmatrix}$$

mit den Eigenwerten

$$\lambda_1 \approx 0.80131, \quad \lambda_2 = 2.2865, \quad \lambda_3 = 4.9122.$$

Wir starten die Potenzmethode mit $x^{(0)} = (1, 1, 1)^T$ und wählen zur Normierung die Maximumsnorm. Weiter wählen wir als Index $k = 1$. Dann erhalten wir

$$\begin{aligned}
 i = 1 : \tilde{x}^{(1)} = Ax^{(0)} &= \begin{pmatrix} 5 \\ 2 \\ 7 \end{pmatrix}, & x^{(1)} &\approx \begin{pmatrix} 0.714 \\ 0.286 \\ 1 \end{pmatrix}, & \lambda^{(1)} &\approx 5, \\
 i = 2 : \tilde{x}^{(2)} = Ax^{(1)} &= \begin{pmatrix} 3.71 \\ 0.857 \\ 5.29 \end{pmatrix}, & x^{(2)} &\approx \begin{pmatrix} 0.702 \\ 0.162 \\ 1 \end{pmatrix}, & \lambda^{(2)} &\approx 5.19, \\
 i = 3 : \tilde{x}^{(3)} = Ax^{(2)} &= \begin{pmatrix} 3.57 \\ 0.621 \\ 5.03 \end{pmatrix}, & x^{(3)} &\approx \begin{pmatrix} 0.710 \\ 0.124 \\ 1 \end{pmatrix}, & \lambda^{(3)} &\approx 5.077, \\
 i = 4 : \tilde{x}^{(4)} = Ax^{(3)} &= \begin{pmatrix} 3.54 \\ 0.538 \\ 4.96 \end{pmatrix}, & x^{(4)} &\approx \begin{pmatrix} 0.715 \\ 0.108 \\ 1 \end{pmatrix}, & \lambda^{(4)} &\approx 4.999, \\
 i = 5 : \tilde{x}^{(5)} = Ax^{(4)} &= \begin{pmatrix} 3.54 \\ 0.502 \\ 4.93 \end{pmatrix}, & x^{(5)} &\approx \begin{pmatrix} 0.717 \\ 0.102 \\ 1 \end{pmatrix}, & \lambda^{(5)} &\approx 4.950, \\
 i = 6 : \tilde{x}^{(6)} = Ax^{(5)} &= \begin{pmatrix} 3.54 \\ 0.486 \\ 4.92 \end{pmatrix}, & x^{(6)} &\approx \begin{pmatrix} 0.719 \\ 0.0988 \\ 1 \end{pmatrix}, & \lambda^{(6)} &\approx 4.930.
 \end{aligned}$$

Nehmen wir unsere Python-Implementierung

```

1 A = np.array([[2, 1, 2],
2               [-1, 2, 1],
3               [1, 2, 4]], dtype=np.double)
4 x = np.array([1, 1, 1], dtype=np.double)
5
6 l = potenzmethode_mieses(A, x, 6, k=0)
7 print(l)

```

erhalten wir

```

[5.0, 5.2, 5.0769230769230775, 4.992424242424242, 4.949924127465857,
↪ 4.929797670141017]

```

Nach sechs Schritten haben wir einen Eigenwert auf die Genauigkeit von einer Nachkommastelle bestimmt. ◀

Neben der Festlegung auf den betragsgrößten Eigenwert hat die Potenzmethode den Nachteil sehr langsamer Konvergenz, falls die Eigenwerte nicht hinreichend separiert sind. Eine

einfache Erweiterung der Potenzmethode ist die *inverse Iteration nach Wieland* zur Berechnung des kleinsten Eigenwerts einer Matrix. Zur Herleitung verwenden wir die Tatsache, dass zu einem Eigenwert λ einer regulären Matrix $A \in \mathbb{R}^{n \times n}$ durch $\mu := \lambda^{-1}$ ein Eigenwert der inversen Matrix $A^{-1} \in \mathbb{R}^{n \times n}$ gegeben ist:

$$Aw = \lambda(A)w \Leftrightarrow A^{-1}w = \lambda^{-1}w =: \mu w$$

Die Potenzmethode, angewendet auf die inverse Matrix, liefert den betragsgrößten Eigenwert $\mu_{\max}$ von A^{-1} . Der Kehrwert dieses Eigenwerts ist der betragskleinste Eigenwert der Matrix A selbst $\lambda_{\min} = \mu_{\max}^{-1}$. Dieses Prinzip kann weiter verfeinert werden. Dazu sei λ ein Eigenwert von A mit zugehörigem Eigenvektor $w \in \mathbb{R}^n$, $\sigma \in \mathbb{C}$ sei eine beliebige komplexe Zahl (jedoch kein Eigenwert von A). Dann gilt

$$(A - \sigma I)w = (\lambda - \sigma)w =: \lambda_{\sigma} w \Leftrightarrow [A - \sigma I]^{-1}w = \lambda_{\sigma}^{-1}w =: \mu_{\sigma} w.$$

Die Anwendung der Potenzmethode auf die Matrix $[A - \sigma I]^{-1}$ liefert nach vorangestellter Überlegung den betragskleinsten Eigenwert der Matrix $[A - \sigma I]$, d.h. den Eigenwert von A , der σ am nächsten liegt. Liegen Schätzungen für die Eigenwerte der Matrix A vor, so können die genauen Werte mit der inversen Iteration mit Shift bestimmt werden.

Satz 5.14 (Inverse Iteration mit Shift) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix mit Eigenwerten $\lambda_1, \dots, \lambda_n \in \mathbb{C}$. Mit einem $\sigma \approx \lambda_l \in \mathbb{C}$ für ein $l \in \{1, \dots, n\}$ gelte

$$\alpha_l := \min_{j \neq l} |\lambda_j - \sigma| > |\lambda_l - \sigma| > 0.$$

Es sei $x^{(0)} \in \mathbb{R}^n$ ein geeigneter normierter Startwert. Für $i = 1, 2, \dots$ konvergiert die Iteration

$$[A - \sigma I]\tilde{x}^{(i)} = x^{(i-1)}, \quad x^{(i)} := \frac{\tilde{x}^{(i)}}{\|\tilde{x}^{(i)}\|}, \quad \mu^{(i)} := \frac{\tilde{x}_k^{(i)}}{x_k^{(i-1)}}, \quad \lambda^{(i)} := \sigma + (\mu^{(i)})^{-1},$$

für jeden Index $k \in \{1, \dots, n\}$ gegen den Eigenwert λ_l von A :

$$|\lambda_l - \lambda^{(i)}| = O\left(\left|\frac{\lambda_l - \sigma}{\alpha_l}\right|^i\right)$$

Beweis Der Beweis ist eine einfache Folgerung aus Satz 5.11. Falls σ kein Eigenwert von A ist, so ist

$$B := A - \sigma I$$

invertierbar. Die Matrix B^{-1} hat die Eigenwerte $\mu_1, \dots, \mu_n$ mit

$$\mu_i = (\lambda_i - \sigma)^{-1} \Leftrightarrow \lambda_i = \mu_i^{-1} + \sigma. \quad (5.4)$$

Für diese gilt nach Voraussetzung

$$|\mu_l| > |\mu_j| > 0, \quad j \neq i.$$

Die Potenzmethode, Satz 5.11, angewandt auf die Matrix B^{-1} liefert eine Approximation für μ_l :

$$|\mu^{(i)} - \mu_l| = O\left(\left|\frac{\max_{j \neq l} |\mu_j|}{\mu_l}\right|^i\right)$$

Die gewünschte Aussage folgt mit (5.4). □

In jedem Schritt der Iteration muss ein lineares Gleichungssystem

$$(A - \sigma I)\tilde{x} = x$$

gelöst werden. Dies geschieht am besten mit einer Zerlegungsmethode, etwa der LR-Zerlegung der Matrix. Die Zerlegung kann einmal in $O(n^3)$ Operationen erstellt werden, anschließend sind in jedem Schritt der Iteration weitere $O(n^2)$ Operationen für das Vorwärts- und Rückwärtseinsetzen notwendig. Die Konvergenzgeschwindigkeit der Inversen Iteration mit Shift kann durch eine gute Schätzung σ gesteigert werden. Eine weitere Verbesserung der Konvergenzgeschwindigkeit kann durch ständiges Anpassen der Schätzung σ erreicht werden. Wird in jedem Schritt die beste Approximation $\lambda^{(i)} = \sigma + 1/\mu^{(i)}$ als neue Schätzung verwendet, so kann mindestens superlineare Konvergenz erreicht werden. Hierzu wählen wir $\sigma^{(0)} = \sigma$ und iterieren

$$\sigma^{(i)} = \sigma^{(i-1)} + \frac{1}{\mu^{(i)}}.$$

Jede Modifikation des Shifts ändert allerdings das lineare Gleichungssystem und erfordert die erneute (teure) Erstellung der LR-Zerlegung.

Beispiel 5.15 (Inverse Iteration mit Shift nach Wieland)

Es sei

$$A = \begin{pmatrix} 2 & -0.1 & 0.4 \\ 0.3 & -1 & 0.4 \\ 0.2 & -0.1 & 4 \end{pmatrix}$$

mit den Eigenwerten

$$\lambda_1 = 1.954, \quad \lambda_2 \approx -0.983, \quad \lambda_3 = 4.029.$$

Wir wollen alle Eigenwerte mit der inversen Iteration mit Shift bestimmen. Startwerte erhalten wir durch Analyse der Gerschgorin-Kreise:

$$K_1 = K_{0.5}(2), \quad K_2 = K_{0.2}(-1), \quad K_3 := K_{0.3}(4).$$

Die drei Kreise sind disjunkt und wir wählen als Shift in der Inversen Iteration $\sigma_1 = 2$, $\sigma_2 = -1$ sowie $\sigma_3 = 4$. Die Iteration wird stets mit $v = (1, 1, 1)^T$ und Normierung bezüglich der Maximumsnorm gestartet. Wir erhalten bei Wahl der ersten Komponente zur Berechnung der μ die folgenden Näherungen:

σ_i	$\mu_i^{(1)}$	$\mu_i^{(2)}$	$\mu_i^{(3)}$	$\mu_i^{(4)}$
2	-16.571	-21.744	-21.619	-21.622
-1	1.840	49.743	59.360	59.422
4	6.533	36.004	33.962	33.990

Diese Approximationen ergeben die folgenden Eigenwertnäherungen:

$$\lambda_1 = \sigma_1 + 1/\mu_1^{(4)} \approx 1.954$$

$$\lambda_2 = \sigma_2 + 1/\mu_2^{(4)} \approx -0.983$$

$$\lambda_3 = \sigma_3 + 1/\mu_3^{(4)} \approx 4.029$$

Alle drei Näherungen sind in den ersten vier Stellen exakt.

In Python können wir dies zum Beispiel mit unserer LR-Zerlegung mit Pivotisierung implementieren. Hierbei müssen wir nur vorsichtig sein und die Zahl $x_k^{(i-1)}$ speichern bevor wir die Pivotisierung anwenden.

🔧 Implementierung 5.2: Inverse Iteration mit Shift nach Wieland

```

1 def inverse_iteration_wieland(A, x, sigma, n, k=0):
2     n1, n2 = A.shape
3     n3, = x.shape
4     assert (n1==n2), 'Matrix nicht quadratisch'
5     assert (n1==n3), 'Matrix und Vektor Dimensionen passen nicht'
6
7     B = np.array(A - sigma * np.identity(n1), dtype=A.dtype)
8     pivot = LR_zerlegung_mit_pivot(B)
9
10    lams = []
11    for i in range(n):
12        xk = x[k]
13        for p in pivot:
14            x[p] = x[[p[1], p[0]]]
15        y = vorwaerts_einsetzen_ohne_diag(B, x)
16        x_neu = rueckwaerts_einsetzen(B, y)
17        mu = x_neu[k] / xk
18        lams.append(sigma + 1 / mu)
19        x = np.array(x_neu / np.linalg.norm(x_neu, ord=np.inf))
20    return lams

```

Wenden wir dies auf das obige Beispiel an, erhalten wir

```

1 A = np.array([[2, -0.1, 0.4], [0.3, -1, 0.4], [0.2, -0.1, 4]])
2 v = np.array([1, 1, 1], dtype=np.double)
3
4 lam1 = inverse_iteration_wieland(A, v, 2, 4, k=0)
5 lam2 = inverse_iteration_wieland(A, v, -1, 4, k=0)
6 lam3 = inverse_iteration_wieland(A, v, 4, 4, k=0)
7
8 print(f'lam_1^{len(lam1)} = {lam1[-1]}')
9 print(f'lam_2^{len(lam2)} = {lam2[-1]}')
10 print(f'lam_3^{len(lam3)} = {lam3[-1]}')

lam_1^4 = 1.9537505736728318
lam_2^4 = -0.9831712938368212
lam_3^4 = 4.02942062252305

```

Die Eigenwerte sind damit in den ersten sechs Nachkommastellen exakt. ◀

Das Beispiel demonstriert, dass die inverse Iteration mit Shift zu einer wesentlichen Beschleunigung der Konvergenz führt, falls gute Schätzungen der Eigenwerte vorliegen.

Bemerkung 5.16 (Komplexe Eigenwerte) Reelle Matrizen $A \in \mathbb{R}^{n \times n}$ können komplexe Eigenwerte haben. Die einfachen Iterationsverfahren bauen jedoch vollständig auf reeller Arithmetik auf und werden nie komplexe Eigenwerte finden können. Dies ist kein Widerspruch, denn laut Satz 5.1 tauchen komplexe Eigenwerte reeller Matrizen stets als komplex konjugiertes Paar $\lambda, \bar{\lambda} \in \mathbb{C}$ auf. Für betragsgrößte Eigenwerte mit $\lambda \neq \bar{\lambda} \in \mathbb{C} \setminus \mathbb{R}$ konvergiert die Methode nicht. Dies kann einfach anhand der Matrix

$$A = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$$

erprobt werden. Durch die Wahl eines komplexen Shifts $\sigma \in \mathbb{C}$ können mit der inversen Iteration auch komplexe Eigenwerte gefunden werden. Die Methode erfordert dann jedoch komplexe Arithmetik, auch beim Lösen der linearen Gleichungssysteme. ♦

5.4 Zerlegungsverfahren zur Eigenwertbestimmung

Ein ganz anderer Ansatz zur Suche von Eigenwerten einer Matrix $A \in \mathbb{R}^{n \times n}$ ergibt sich durch die sogenannten *Zerlegungsverfahren*. Wir machen die folgende Beobachtung:

Satz 5.17 (Eigenwerte bei multiplikativen Zerlegungen) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix. Es existiere die Zerlegung

$$A = BC$$

mit zwei regulären Matrizen $B, C \in \mathbb{R}^{n \times n}$. Dann haben die Matrizen $A = BC$ und $A' = CB$ die gleichen Eigenwerte. Mit den n Eigenwerten (ihrer Vielfachheit nach) $\lambda_1, \dots, \lambda_n \in \mathbb{C}$ von A bzw. von A' gilt

$$\text{spur}(A) = \text{spur}(A') = \sum_{i=1}^n \lambda_i.$$

Beweis (i) Es sei $\lambda \in \mathbb{C}$ eine Eigenwert von A mit zugehörigem Eigenvektor $w \in \mathbb{C}^n$. Dann gilt

$$\lambda w = Aw = BCw = B(CB)B^{-1}w \Leftrightarrow \lambda(B^{-1}w) = CB(B^{-1}w) = A'(B^{-1}w)$$

also haben BC und CB den gleichen Eigenwert λ . Jedem Eigenvektor w von A ist ein entsprechender Eigenvektor $w' = B^{-1}w$ von A' zugeordnet.

(ii) Die Spur einer Matrix ist eine *Invariante* unter Ähnlichkeitstransformationen, siehe z. B. [32]. Wir führen den Beweis, da der Beziehung zwischen Spur und Eigenwerten im Rahmen der Zerlegungsverfahren eine besondere Bedeutung zukommt. Mit den Eigenwerten λ_i als Nullstellen des charakteristischen Polynoms gilt

$$\begin{aligned} \det(\lambda I - A) &= \det(\lambda I - A') = \prod_{i=1}^n (\lambda - \lambda_i) \\ &= \lambda^n - \lambda^{n-1} \sum_{i=1}^n \lambda_i + \dots + \prod_{i=1}^n (-\lambda_i). \end{aligned} \quad (5.5)$$

Die Summe der Eigenwerte taucht im Koeffizienten vor dem zweithöchsten Monom λ^{n-1} auf. Umgekehrt gilt mit der Leibniz-Formel

$$\det(\lambda I - A) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n [\lambda I - A]_{i, \sigma(i)},$$

wobei die Summe über alle Permutationen σ der symmetrischen Gruppe S_n gebildet wird. Mit $\text{sgn}(\sigma)$ wird das *Signum* der Permutation bezeichnet, also $+1$ bei einer geraden, -1 bei einer ungeraden Permutation. Zum Vergleich mit (5.5) sind nur die Terme von Interesse, bei denen der Exponent λ^{n-1} auftaucht. Dies kann nur der Fall sein, wenn σ die Identität ist, denn ansonsten müssen mindestens zwei Diagonalterme fehlen. Hiermit können wir die Determinante schreiben als

$$\det(\lambda I - A) = \prod_{i=1}^n (\lambda - a_{ii}) + p_{n-2}(\lambda),$$

wobei $p_{n-2}(\lambda)$ ein Polynom von Grad $n - 2$ ist. Es folgt

$$\det(\lambda I - A) = \lambda^n - \lambda^{n-1} \sum_{i=1}^n a_{ii} + q_{n-2}(\lambda) + p_{n-2}(\lambda)$$

mit einem weiteren Polynom $q_{n-2} \in P_{n-2}$. Der Vergleich mit (5.5) zeigt

$$\text{spur}(A) = \sum_{i=1}^n a_{ii} = \sum_{i=1}^n \lambda_i = \sum_{i=1}^n a'_{ii} = \text{spur}(A').$$

□

Die Iteration

$$A_k = B_k C_k \mapsto C_k B_k =: A_{k+1}$$

definiert also eine Ähnlichkeitstransformation. Alle Matrizen A_k haben die gleichen Eigenwerte und auch die gleiche Spur. Für viele uns bekannte Zerlegungen (LR-Zerlegung, Cholesky-Zerlegung, QR-Zerlegung) zeigt sich ein zunächst sehr erstaunlicher Zusammenhang: Die Folge von Matrizen $(A_k)_k \in \mathbb{R}^{n \times n}$ konvergiert gegen eine Dreiecksmatrix. Da die Matrizen ähnlich sind, können die Eigenwerte der anfänglichen Matrix $A_0 = A$ von der Diagonale abgelesen werden.

5.4.1 Cholesky-Verfahren

Falls mehrere oder sogar alle Eigenwerte gesucht sind, so sind Zerlegungsmethoden die effizientesten Verfahren zur Eigenwertbestimmung. Die mathematische Analyse ist weit aufwendiger als die Analyse von Potenzmethode und inverser Iteration. Wir betrachten zunächst ein Verfahren basierend auf der Cholesky-Zerlegung.

Algorithmus 5.1: Cholesky-Verfahren

Eingabe: Es sei $A \in \mathbb{R}^{n \times n}$ eine symmetrisch positiv definite Matrix

- 1 Setze $A_0 := A$
- 2 **Für** $i = 0, 1, 2, \dots$ **tue**
- 3 Erstelle die Cholesky-Zerlegung $A_i =: L_i L_i^T$
- 4 Berechne $A_{i+1} := L_i^T L_i$

In dem wir Implementierung 3.5 der Cholesky-Zerlegung verwenden, können wir das Cholesky-Verfahren folgendermaßen implementieren. Dabei geben wir darauf acht, dass wir aufgrund der unteren Dreiecksstruktur von L nicht die vollständige Matrix-Multiplikation ausführen müssen.

♣ Implementierung 5.3: Cholesky-Verfahren

```

1 def cholesky_verfahren(A, k):
2     A = A.copy()
3     n, m = A.shape
4     for i in range(k):
5         L = cholesky(A)
6         for i in range(n):
7             for j in range(n):
8                 a = 0
9                 for k in range(max(i, j), n):
10                     a += L[k, i] * L[k, j]
11                 A[i, j] = a
12     return A

```

Das spezielle Matrix-Matrix-Produkt, dass wir in Zeilen 6–11 implementiert haben, kann auch mit dem vollen Matrix-Matrix-Produkt von `numpy`

```

6 A = L.transpose() @ L

```

ersetzt werden. Da hiermit die Matrizen-Multiplikation effizient intern implementiert ist, und keine verhältnismäßig langsamen Python-Schleifen verwendet werden oder Zugriffe auf einzelne Matrix-Einträge stattfinden, ist dies auch unter Umständen schneller, auch wenn viele Null-Einträge berechnet werden.

Wir zeigen zunächst, dass das Cholesky-Verfahren durchführbar ist, dass also alle Matrizen $A_{(i)}$ wieder symmetrisch positiv definit sind, somit eine Cholesky-Zerlegung existiert.

Satz 5.18 (Durchführbarkeit des Cholesky-Verfahrens) Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit und $A = LL^T$ die Cholesky-Zerlegung. Die Matrix $A' = L^T L$ ist symmetrisch positiv definit.

Beweis Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit. Dann existiert die Cholesky-Zerlegung

$$A = LL^T$$

in eine reguläre Dreiecksmatrix $L \in \mathbb{R}^{n \times n}$ und die Matrix $A' = L^T L$ ist wieder symmetrisch, denn

$$A'^T = (L^T L)^T = L^T L.$$

Weiter ist die Matrix A' positiv definit, denn

$$(A'x, x) = (L^T Lx, x) = (Lx, Lx) > 0 \quad \forall x \neq 0,$$

da L regulär ist. □

Die Konvergenz des Cholesky-Verfahrens kann verhältnismäßig einfach nachgewiesen werden. Es konvergiert immer, d. h. für jede symmetrisch positiv definite Matrix.

Satz 5.19 (Konvergenz des Cholesky-Verfahrens) Es sei $A \in \mathbb{R}^{n \times n}$ eine symmetrisch positiv definite Matrix. Die Folge von Matrizen A_k des Cholesky-Verfahrens aus Algorithmus 5.1 konvergiert gegen eine Diagonalmatrix, deren Einträge gerade die Eigenwerte von A sind.

Beweis (i) Zum Nachweis der Konvergenz betrachten wir zwei aufeinanderfolgende Matrizen A_k und A_{k+1} . Es sei

$$A_k = L_k L_k^T, \quad A_{k+1} = L_k^T L_k,$$

wobei L_k die linke untere Dreiecksmatrix der Cholesky-Zerlegung ist. Für die Einträge der Matrizen $A_k = (a_{ij}^{(k)})_{ij}$ und $A_{k+1} = (a_{ij}^{(k+1)})_{ij}$ gilt

$$\begin{aligned} a_{ij}^{(k)} &= \sum_{k=1}^n l_{ik}^{(k)} l_{jk}^{(k)} = \sum_{k=1}^{\min\{i,j\}} l_{ik}^{(k)} l_{jk}^{(k)}, \\ a_{ij}^{(k+1)} &= \sum_{k=1}^n l_{ki}^{(k)} l_{kj}^{(k)} = \sum_{k=\max\{i,j\}}^n l_{ki}^{(k)} l_{kj}^{(k)}. \end{aligned} \quad (5.6)$$

Mit $s_m^{(k)}$ bezeichnen wir die partielle Spur, also

$$s_m^{(k)} := \sum_{i=1}^m a_{ii}^{(k)}, \quad s_m^{(k+1)} := \sum_{i=1}^m a_{ii}^{(k+1)}.$$

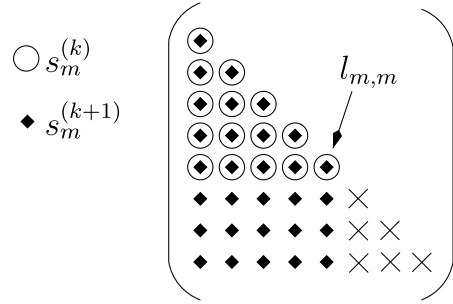
Für diese gilt mit (5.6)

$$s_m^{(k)} = \sum_{i=1}^m \sum_{k=1}^i (l_{ik}^{(k)})^2, \quad s_m^{(k+1)} = \sum_{i=1}^m \sum_{k=i}^n (l_{ki}^{(k)})^2.$$

In Abb. 5.1 stellen wir grafisch dar, welche Einträge von L in den beiden Partialspuren $s_m^{(k)}$ und $s_m^{(k+1)}$ vorkommen: Bei $s_m^{(k)}$ ist es die Teilmatrix links oberhalb von $l_{mm}^{(k)}$, bei $s_m^{(k+1)}$ ist es die ganze Teilmatrix links von $l_{mm}^{(k)}$. Der Unterschied beider Partialspuren ist somit gegeben als

$$s_m^{(k+1)} - s_m^{(k)} = \sum_{i=m+1}^n \sum_{j=1}^m (l_{ij}^{(k)})^2 \geq 0. \quad (5.7)$$

Abb. 5.1 Elemente von $L = (l_{ij})_{ij}$, die in den Partialsummen $s_m^{(k)}$ (Kreise) und $s_m^{(k+1)}$ (Rauten) auftauchen. Die Differenz ist gegeben durch alle Indizes, die mit Rauten markiert sind, jedoch keine Kreise haben



Die Partialspuren $s_m^{(k)}$ sind jeweils (für jedes m) eine monoton steigende Folge. Diese Folgen sind allerdings beschränkt, denn positiv definite Matrizen haben nur positive Diagonaleinträge, siehe Satz 2.15, und es gilt

$$s_m^{(k)} \leq s_n^{(k)} = \text{spur}(A_k) = \text{spur}(A) = \sum_{i=1}^n \lambda_i.$$

Als monoton steigende und beschränkte Folge konvergiert

$$s_m^{(k)} \rightarrow s_m \leq \sum_{i=1}^n \lambda_i.$$

Mit (5.7) folgt dann

$$\sum_{i=m+1}^n \sum_{j=1}^m (l_{ij}^{(k)})^2 = s_m^{(k+1)} - s_m^{(k)} \rightarrow 0 \quad (k \rightarrow \infty),$$

also für alle $m \in \{1, \dots, n\}$ und alle $i > m$ und $j \leq m$:

$$l_{ij}^{(k)} \rightarrow 0 \text{ für } k \rightarrow \infty, \quad \forall i > m, j \leq m$$

Alle Nebendiagonalen konvergieren gegen null, sodass

$$L_k \rightarrow \text{diag}(l_{11}, \dots, l_{nn}) \quad (k \rightarrow \infty).$$

(ii) Wir haben bereits gezeigt, dass die Matrizen A und A_k für $k \geq 0$ die gleichen Eigenwerte besitzen. Falls $A_k \rightarrow \Lambda = \text{diag}(l_1, \dots, l_n)$, so müssen die Diagonalelemente $l_1, \dots, l_n$ gerade die Eigenwerte von A sein. Wir erhalten keine Aussage über die Reihenfolge der Eigenwerte. $\square$

Beispiel 5.20 (Cholesky-Verfahren)

Wir betrachten die Matrix

$$A = \begin{pmatrix} 3 & -1 & 0 & 1 \\ -1 & 3 & 1 & 1 \\ 0 & 1 & 3 & 0 \\ 1 & 1 & 0 & 3 \end{pmatrix}.$$

Mit $A_0 := A$ führen wir einige Schritte des Cholesky-Verfahrens mit unserem Python-Code durch. Dabei geben wir ein paar der Iterationsmatrizen mit aus.

```
1 A = np.array([[3, -1, 0, 1],
2              [-1, 3, 1, 1],
3              [0, 1, 3, 0],
4              [1, 1, 0, 3]], dtype=np.double)
5 A_chol = cholesky_verfahren(A, 50)
```

```
A_0 =
[[ 3.66666667 -0.47140452 -0.17817416  0.79681907]
 [-0.47140452  3.70833333  0.74018043  1.12687234]
 [-0.17817416  0.74018043  2.7202381  -0.42591771]
 [ 0.79681907  1.12687234 -0.42591771  1.9047619 ]]
```

```
A_1 =
[[ 3.90909091 -0.23728987 -0.31241772  0.44884361]
 [-0.23728987  4.20305862  0.34963659  0.6942616 ]
 [-0.31241772  0.34963659  2.72441556 -0.42311459]
 [ 0.44884361  0.6942616  -0.42311459  1.1634349 ]]
```

```
A_2 =
[[ 4.          -0.19112739 -0.32003501  0.21701249]
 [-0.19112739  4.3390411  0.16839657  0.33700088]
 [-0.32003501  0.16839657  2.74715053 -0.2597052 ]
 [ 0.21701249  0.33700088 -0.2597052  0.91380838]]
```

...

```
A_49 =
[[ 4.47235427e+00 -6.46190915e-02 -4.96860043e-06  3.08058421e-18]
 [-6.46190915e-02  4.00884003e+00 -2.35753313e-05  3.17640652e-17]
 [-4.96860043e-06 -2.35753313e-05  2.68889218e+00 -2.25698099e-13]
 [ 3.08058421e-18  3.17640652e-17 -2.25698099e-13  8.29913513e-01]]
```

Die Eigenwerte lassen sich anhand der `numpy.linalg` Funktion `eig` bestimmen als

```
1 print(np.linalg.eig(A)[0])
```

```
[0.82991351 2.68889218 4.          4.4811943 ]
```

Das Cholesky-Verfahren konvergiert also und es lassen sich alle Eigenwerte bestimmen. Es konvergiert allerdings sehr langsam. Nach 50 Schritten haben wir die Eigenwerte bis auf einen relativen Fehler von 0.2% bestimmt. ◀

Das Cholesky-Verfahren kann natürlich nur für symmetrisch positiv definite Matrizen angewendet werden. Eine Verallgemeinerung ist ein entsprechendes Verfahren, basierend auf der LR-Zerlegung, d. h. eine Iteration der Art

$$A_0 := A, \quad A_k =: L_k R_k, \quad A_{k+1} := R_k L_k.$$

Auch dieses Verfahren konvergiert gegen eine Matrix, deren Diagonalelemente die Eigenwerte von A sind. Das Verfahren kann jedoch nur dann angewendet werden, wenn die LR-Zerlegung in jedem Schritt ohne Pivotisierung (vergleiche Satz 3.12) durchgeführt werden kann. Angenommen, es ist

$$PA = LR$$

mit einer Pivot-Matrix $P \neq I$, dann sind die Matrizen A und $A' := RL$ nicht mehr ähnlich.

5.4.2 QR-Verfahren

Da die LR-Zerlegung bei allgemeinen regulären Matrizen ausscheidet, betrachten wir als Alternative die QR-Zerlegung. Diese, man vergleiche Abschn. 4.1, existiert für beliebige reguläre Matrizen und wir kennen stabile Verfahren zu deren Berechnung, beispielsweise die Householder-Transformationen 4.3 oder die Givens Rotationen 4.4. Nachteil der QR-Zerlegung ist der, im Vergleich zur LR-Zerlegung, doppelt so hoher Aufwand.

Algorithmus 5.2: QR-Verfahren

Eingabe: Es sei $A \in \mathbb{R}^{n \times n}$

- 1 Setze $A_0 := A$
- 2 **Für** $k = 0, 1, 2, \dots$ **tue**
- 3 Erstelle die QR-Zerlegung $A_k =: Q_k R_k$
- 4 Berechne $A_{k+1} := R_k Q_k$

Mit unserer Implementierung der QR-Zerlegung nach Givens können wir dies dann in Python folgendermaßen implementieren:

♣ Implementierung 5.4: QR-Verfahren mit Givens-Rotationen

```

1 def qr_verfahren(A, k):
2     A = A.copy()
3     for i in range(k):
4         QT = qr_givens(A)
5         A = A @ QT.transpose()
6     return A

```

Das QR-Verfahren erzeugt eine Folge $A_k, k \geq 0$ von Matrizen, die gemäß Satz 5.17 alle ähnlich zur Matrix A sind. Wir werden sehen, dass die Diagonaleinträge der Folgenglieder A_k gegen die Eigenwerte der Matrix A laufen. Die Analyse des Verfahrens ist aufwendiger und wir geben nur eine kurze Beweisskizze.

Satz 5.21 (QR-Verfahren zur Eigenwertberechnung) Es sei $A \in \mathbb{R}^{n \times n}$ eine Matrix mit separierten Eigenwerten

$$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n|.$$

Dann gilt für die Diagonalelemente $a_{ii}^{(t)}$ der durch das QR-Verfahren erzeugten Matrizen $A^{(t)}$

$$\{\alpha_{11}^{(t)}, \dots, \alpha_{nn}^{(t)}\} \rightarrow \{\lambda_1, \dots, \lambda_n\} \quad (t \rightarrow \infty).$$

Beweis Der ausführliche Beweis findet sich bei [21, 68]. Ein wichtiger Baustein im Beweis ist der Zusammenhang

$$A^{k+1} = \underbrace{Q_0 Q_1 \dots Q_k}_{=\tilde{Q}_k} \underbrace{R_k R_{k-1} \dots R_0}_{=\tilde{R}_k}$$

den wir mit Induktion beweisen. Zunächst gilt wie behauptet $A^1 = Q_0 R_0 = \tilde{Q}_0 \tilde{R}_0$. Die Aussage sei somit für A^k korrekt. Dann ist

$$\begin{aligned}
 A^{k+1} &= A^1 A^k = Q_0 R_0 \underbrace{Q_0 Q_1 \dots Q_{k-1}}_{=\tilde{Q}^{k-1}} \underbrace{R_{k-1} R_{k-2} \dots R_0}_{=\tilde{R}^{k-1}} \\
 &= Q_0 \underbrace{R_0 Q_0}_{=A_1=Q_1 R_1} Q_1 \dots Q_{k-1} R_{k-1} \dots R_0 \\
 &= Q_0 Q_1 \underbrace{R_1 Q_1}_{=A_2=Q_2 R_2} Q_2 \dots Q_{k-1} R_{k-1} \dots R_0 \\
 &= \dots \\
 &= Q_0 Q_1 \dots Q_{k-2} Q_{k-1} \underbrace{R_{k-1} Q_{k-1}}_{=A_k=Q_k R_k} R_{k-1} \dots R_0 \\
 &= Q_0 Q_1 \dots Q_{k-2} Q_{k-1} \dots Q_k R_k R_{k-1} \dots R_0 = \tilde{Q}_k \tilde{R}_k.
 \end{aligned}$$

Das Produkt $\tilde{Q}_k$ ist wieder eine orthogonale Matrix und $\tilde{R}_k$, als Produkt von rechten oberen Dreiecksmatrizen, ist selbst eine. Im Laufe des Verfahrens wird also eine Zerlegung der Potenz A^k erstellt. Diese kann genutzt werden, um einen Zusammenhang zur Potenzmethode nach von Mises herzustellen. $\square$

Im Allgemeinen konvergieren die Diagonalelemente der Folgenglieder A_k mindestens linear gegen die Eigenwerte. In speziellen Fällen (z. B. symmetrische Matrizen) wird jedoch mit guter Shift-Strategie sogar quadratische oder kubische Konvergenz erreicht. Die Analyse ist aber sehr aufwändig [90]. Verglichen mit der Potenzmethode und der inversen Iteration weist das QR-Verfahren daher zum einen bessere Konvergenzeigenschaften auf, gleichzeitig werden alle Eigenwerte der Matrix A bestimmt.

Beispiel 5.22 (QR-Verfahren)

Wir betrachten die Matrix aus Beispiel 5.20

$$A = \begin{pmatrix} 3 & -1 & 0 & 1 \\ -1 & 3 & 1 & 1 \\ 0 & 1 & 3 & 0 \\ 1 & 1 & 0 & 3 \end{pmatrix}.$$

Wir wenden unsere Implementierung an und betrachten für die ersten Iterationen $A_{k+1} = R_k Q_k$

```
1 A_neu = qr_verfahren(A, 25)
```

```
A_0 =
[[ 3.90909091 -0.23728987 -0.31241772  0.44884361]
 [-0.23728987  4.20305862  0.34963659  0.6942616 ]
 [-0.31241772  0.34963659  2.72441556 -0.42311459]
 [ 0.44884361  0.6942616  -0.42311459  1.1634349 ]]
```

```
A_1 =
[[ 4.04651163 -0.19251315 -0.280484  0.10005528]
 [-0.19251315  4.36319045  0.09547029  0.15393536]
 [-0.280484  0.09547029  2.74000116 -0.14246604]
 [ 0.10005528  0.15393536 -0.14246604  0.85029676]]
```

```
A_2 =
[[ 4.10147992 -0.20988706 -0.19194259  0.02052083]
 [-0.20988706  4.35310663  0.0354996  0.03135503]
 [-0.19194259  0.0354996  2.71419158 -0.04148131]
 [ 0.02052083  0.03135503 -0.04148131  0.83122187]]
```

...

A₂₄ =

```
[ [ 4.47235427e+00 -6.46190915e-02 -4.96860043e-06  4.84794998e-16]
  [-6.46190915e-02  4.00884003e+00 -2.35753313e-05 -2.25833466e-16]
  [-4.96860043e-06 -2.35753313e-05  2.68889218e+00 -2.25615109e-13]
  [ 3.08058421e-18  3.17640652e-17 -2.25698099e-13  8.29913513e-01]]
```

Es werden wieder alle vier Eigenwerte bestimmt. Nach nur 25 Schritten liegen alle Eigenwerte wieder mit einem relativen Fehler von weniger als 0.2% vor. ◀

Das QR-Verfahren kann erheblich beschleunigt werden. Die Konvergenzgeschwindigkeit hängt wieder von der Separierung der Eigenwerte $|\lambda_i| > |\lambda_{i+1}|$ ab. Durch die Wahl eines *Shifts* kann das Spektrum günstiger verteilt werden. Grundlage hierfür ist der folgende Satz.

Satz 5.23 (Ähnlichkeitstransformation und Shift) Es sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix $\mu \in \mathbb{R}$ so, dass $A - \mu I$ regulär ist. Es sei

$$QR = A - \mu I,$$

die QR-Zerlegung. Dann sind die Matrizen A und

$$A' = RQ + \mu I$$

ähnlich.

Beweis Es gilt

$$A' = RQ + \mu I = Q^T QRQ + \mu I = Q^T (A - \mu I)Q + \mu I = Q^T A Q. \quad \square$$

Bei richtiger Wahl des Shifts ist die QR-Methode äußerst schnell. Wir verweisen auf die Literatur [21].

Beispiel 5.24 (QR-Verfahren mit Shift)

Wir betrachten wieder die Matrix

$$A = \begin{pmatrix} 3 & -1 & 0 & 1 \\ -1 & 3 & 1 & 1 \\ 0 & 1 & 3 & 0 \\ 1 & 1 & 0 & 3 \end{pmatrix}.$$

Zur Anwendung des QR-Verfahrens mit Shift passen wir unsere vorhandene Implementierung des QR-Verfahrens an. Als Shift verwenden wir stets das größte Diagonalelement $\mu_k = \max\{|A_k]_{ii}\}$.

♣ Implementierung 5.5: QR-Verfahren mit Shift

```

1 def qr_verfahren_mit_shift(A, k):
2     A = A.copy()
3     Id = np.identity(A.shape[0], dtype=A.dtype)
4     for l in range(k):
5         mu = np.amax(np.diag(A))
6         A[:, :] -= mu * Id
7         QT = qr_givens(A)
8         A[:, :] = A @ QT.transpose() + mu * Id
9     return A

```

Die ersten Schritte ergeben hierbei

```

mu_0 = 3.0
A_0 =
[[ 2.         -1.34164079  0.67082039  0.5         ]
 [-1.34164079  3.6         0.7         0.2236068 ]
 [ 0.67082039  0.7         3.4         0.4472136 ]
 [ 0.5         0.2236068  0.4472136  3.         ]]

```

```

mu_1 = 3.5999999999999996
A_1 =
[[ 0.91304348 -0.50102009  0.09837421  0.14239908]
 [-0.50102009  4.35563604  0.26225201 -0.03157971]
 [ 0.09837421  0.26225201  3.18483703  0.62151247]
 [ 0.14239908 -0.03157971  0.62151247  3.54648345]]

```

```

mu_2 = 4.355636041564521
A_2 =
[[ 8.30705670e-01 -4.05668796e-02 -3.96742622e-03 -1.68422780e-02]
 [-4.05668796e-02  3.19594681e+00  7.66669931e-01  8.13801166e-02]
 [-3.96742622e-03  7.66669931e-01  3.94422136e+00 -2.37077169e-01]
 [-1.68422780e-02  8.13801166e-02 -2.37077169e-01  4.02912616e+00]]

```

```

mu_3 = 4.029126163790677
A_3 =
[[ 8.29995403e-01 -1.24432457e-02 -3.77520654e-06  1.65271069e-04]
 [-1.24432457e-02  2.75040626e+00  3.26440965e-01 -2.25823280e-03]
 [-3.77520654e-06  3.26440965e-01  4.41933913e+00  1.21169434e-02]
 [ 1.65271069e-04 -2.25823280e-03  1.21169434e-02  4.00025921e+00]]

```

```

mu_4 = 4.419339127724223
A_4 =
[[ 8.29932204e-01 -5.89457686e-03 -3.08739043e-06 -1.90655467e-05]
 [-5.89457686e-03  2.68895970e+00  1.23244007e-02  5.50074407e-04]
 [-3.08739043e-06  1.23244007e-02  4.46881158e+00 -7.59412090e-02]
 [-1.90655467e-05  5.50074407e-04 -7.59412090e-02  4.01229652e+00]]

```

```

mu_5 = 4.468811575985431
A_5 =
[[ 8.29917985e-01 -2.88318998e-03 -2.45584035e-06 4.00544086e-07]
 [-2.88318998e-03 2.68888805e+00 6.66883821e-04 -2.36284725e-05]
 [-2.45584035e-06 6.66883821e-04 4.01246830e+00 7.64485048e-02]
 [ 4.00544087e-07 -2.36284725e-05 7.64485048e-02 4.46872566e+00]]

```

Nach nur sechs Schritten haben wir alle Eigenwerte bis auf einen relativen Fehler von weniger als 1% bestimmt. ◀

In jedem Schritt der Iteration muss jedoch eine QR-Zerlegung der Iterationsmatrix A_i erstellt werden. Bei allgemeiner Matrix $A \in \mathbb{R}^{n \times n}$ sind hierzu $O(n^3)$ arithmetische Operationen notwendig. Um die Eigenwerte mit hinreichender Genauigkeit zu approximieren, sind oft sehr viele Schritte notwendig. In der praktischen Anwendung wird das QR-Verfahren daher immer in Verbindung mit einer *Reduktionsmethode* (siehe folgender Abschnitt) eingesetzt, bei der die Matrix A zunächst in eine „einfache“ ähnliche Form transformiert wird.

5.5 Reduktionsmethoden

Das Erstellen der QR-Zerlegung einer Matrix $A \in \mathbb{R}^{n \times n}$ mithilfe von Householder-Matrizen bedarf $O(n^3)$ arithmetischer Operationen, siehe Satz 4.15. Da in jedem Schritt des QR-Verfahrens diese Zerlegung neu erstellt werden muss, ist die Methode sehr aufwändig. Hat die Matrix A jedoch eine spezielle Struktur, ist sie z. B. eine Bandmatrix, so kann auch die QR-Zerlegung mit weit geringerem Aufwand erstellt werden.

Satz 5.25 (Ähnliche Matrizen) Zwei Matrizen $A, B \in \mathbb{C}^{n \times n}$ heißen *ähnlich*, falls es eine reguläre Matrix $S \in \mathbb{C}^{n \times n}$ gibt, sodass gilt:

$$A = S^{-1}BS.$$

Ähnliche Matrizen haben das gleiche charakteristische Polynom und die gleichen Eigenwerte. Zu einem Eigenwert λ sowie Eigenvektor w von A gilt:

$$B(Sw) = S(Aw) = \lambda Sw.$$

Unser Ziel ist es, die Matrix A zunächst in eine *ähnliche Matrix* mit gleichen Eigenwerten zu transformieren, für welche die QR-Zerlegung dann schneller berechnet werden kann oder bei der die Eigenwerte sogar direkt ablesbar sind. Mögliche *Normalformen*, bei denen die Eigenwerte unmittelbar ablesbar sind, sind die *Jordansche Normalform*, die Diagonalisierung von A , oder die *Schursche Normalform*. Die Aufgabe, eine Matrix A in eine

dieser Normalformen zu transformieren, ist üblicherweise nur bei Kenntnis der Eigenwerte möglich. Ein Kompromiss ist jedoch die sogenannte *Hessenberg-Normalform*. Dies ist eine Dreiecksmatrix mit einer weiteren Nebendiagonale und sie kann verhältnismäßig einfach aufgebaut werden.

Satz 5.26 (Hessenberg-Normalform) Zu jeder Matrix $A \in \mathbb{R}^{n \times n}$ existiert eine orthogonale Matrix $Q \in \mathbb{R}^{n \times n}$, sodass

$$H := Q^T A Q = \begin{pmatrix} * & \cdots & \cdots & \cdots & * \\ * & \ddots & \ddots & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & * & * \end{pmatrix}$$

eine *Hessenberg-Matrix* ist, also eine rechte obere Dreiecksmatrix, die zusätzlich eine untere Nebendiagonale besitzt, $H_{ij} = 0$ für $i > j + 1$. Falls A symmetrisch ist (also $A^T = A$), so ist $Q^T A Q$ eine Tridiagonalmatrix.

Beweis Die Konstruktion der Hessenberg-Matrix erfolgt ähnlich der QR-Zerlegung mit Householder-Transformationen. Hier haben wir die rechte obere Dreiecksmatrix durch sukzessive Multiplikation mit Spiegelungsmatrizen $S^{(k)}$ erstellt, d. h.

$$R = \underbrace{S^{(n-1)} \cdots S^{(2)} S^{(1)}}_{=Q^T} A.$$

Die Matrizen R und A sind aber nicht ähnlich. Um dies zu erreichen, werden wir die Multiplikationen von links und rechts durchführen, d. h. $A^{(i)} = S^{(i)} A^{(i-1)} S^{(i)}$ (die orthogonalen Spiegelungsmatrizen $S^{(i)}$ sind symmetrisch, daher können wir auf das Transponieren verzichten). $A^{(i)}$ und $A^{(i-1)}$ sind dann ähnlich, haben also gleiche Eigenwerte.

Wir beschreiben den ersten Schritt. Bei der QR-Zerlegung ist die Matrix $S^{(1)}$ so gewählt, dass die Multiplikation $S^{(1)} A^{(0)}$ den ersten Spaltenvektor $a_1^{(0)}$ von $A^{(0)}$ auf ein Vielfaches des ersten Einheitsvektors e_1 abbildet, d. h., die erste Subdiagonale von $S^{(1)} A^{(0)}$ ist null. Die Multiplikation von rechts mit $S^{(1)}$ zerstört diese Eigenschaft im allgemeinen wieder und die Einträge der Spalte werden wieder aufgefüllt. Wir erreichen das gewünschte Ergebnis aber mit einem einfachen Trick: $S^{(1)}$ wird so gewählt, dass der erste Spaltenvektor in den Teilraum $\text{span}(e_1, e_2)$ abgebildet wird. Hierzu bestimmen wir $S^{(1)}$ gemäß

$$S^{(1)} = I - 2v^{(i)}(v^{(i)})^T, \quad v^{(i)} = \frac{\tilde{a}_1^{(0)} + \|\tilde{a}_1^{(0)}\|e_2}{\|\tilde{a}_1^{(0)}\| + \|\tilde{a}_1^{(0)}\|e_2\|},$$

wobei $\tilde{a}_1^{(0)}$ der um das erste Element gekürzte Spaltenvektor ist, also

$$\tilde{a}_1^{(0)} = (0, a_{21}^{(0)}, \dots, a_{n1}^{(0)}).$$

Dann hat $S^{(1)}$ die Form

$$S^{(1)} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & * & \cdots & * \\ \vdots & \vdots & \ddots & \vdots \\ 0 & * & \cdots & * \end{pmatrix}$$

und die Multiplikation mit $S^{(1)}$ von links lässt die erste Zeile unverändert und die Multiplikation mit $S^{(1)}$ von rechts lässt die erste Spalte gleich. Wir erreichen hiermit zwar keine Dreiecksgestalt, jedoch die gewünschte Hessenberggestalt:

$$A^{(1)} = S^{(1)} A^{(0)} S^{(1)} = \left(\begin{array}{c|ccc} a_{11} & a_{12} & \cdots & a_{1n} \\ \hline * & * & \cdots & * \\ 0 & \vdots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ 0 & * & \cdots & * \end{array} \right) =: \left(\begin{array}{c|ccc} a_{11} & * & \cdots & * \\ \hline * & & & \\ 0 & & & \\ \vdots & & & \\ 0 & & & \end{array} \right) \tilde{A}_{(1)}$$

mit dem $(n-1) \times (n-1)$ -Matrixblock $\tilde{A}^{(1)} \in \mathbb{R}^{(n-1) \times (n-1)}$. Im zweiten Schritt wird das entsprechende Verfahren auf die Matrix $\tilde{A}^{(1)}$ angewendet. Nach $n-2$ Schritten erhalten wir die Matrix $A^{(n-2)}$, die Hessenberg-Gestalt hat:

$$Q^T A Q := \underbrace{S^{(n-2)} \cdots S^{(1)}}_{=: Q^T} A \underbrace{S^{(1)} \cdots S^{(n-2)}}_{=: Q}$$

Im Fall $A = A^T$ gilt

$$(Q^T A Q)^T = Q^T A^T Q = Q^T A Q.$$

Das heißt, auch $A^{(n-2)}$ ist wieder symmetrisch. Symmetrische Hessenberg-Matrizen sind Tridiagonalmatrizen. $\square$

Eine Python Implementierung der Transformation in die Hessenberg-Gestalt kann dann folgendermaßen aussehen:

🔧 Implementierung 5.6: Transformation auf Hessenberg-Gestalt

```
1 def reduktion_auf_hessenberg(A):
2     n, m = A.shape
3     for i in range(m - 2):
4         v = A[i + 1:, i].copy()
5         ei = np.zeros(n - i - 1, dtype=A.dtype)
6         ei[0] = 1
7         v += np.sign(v[0]) * np.linalg.norm(v) * ei
```

```

8      v /= np.linalg.norm(v)
9      A[i + 1:, i:] -= 2 * np.outer(v, np.inner(v.T, A[i + 1:, i:].T))
10     A[:, i + 1:] -= 2 * np.outer(np.inner(A[:, i + 1:], v), v)

```

Bemerkung 5.27 (Hessenberg-Normalform) Die Transformation einer Matrix $A \in \mathbb{R}^{n \times n}$ in Hessenberg-Normalform erfordert bei Verwendung der Householder-Transformationen $\frac{5}{3}n^3 + O(n^2)$ arithmetische Operationen. Im Fall symmetrischer Matrizen erfordert die Transformation in eine Tridiagonalmatrix $\frac{2}{3}n^3 + O(n^2)$ Operationen. ♦

Die Transformation in Hessenberg-Normalform benötigt demnach etwas mehr arithmetische Operationen als *eine* QR-Zerlegung. Im Anschluss können die QR-Zerlegungen einer Hessenberg-Matrix mit weitaus geringerem Aufwand erstellt werden. Weiter gilt für die QR-Zerlegung einer Hessenberg-Matrix A , dass die Matrix RQ wieder Hessenberg-Gestalt hat. Beides fasst der folgende Satz zusammen:

Satz 5.28 (QR-Zerlegung von Hessenberg-Matrizen) Es sei $A \in \mathbb{R}^{n \times n}$ eine Hessenberg-Matrix. Dann kann die QR-Zerlegung $A = QR$ mit Householder-Transformationen in $2n^2 + O(n)$ arithmetische Operationen durchgeführt werden. Die Matrix RQ hat wieder Hessenberg-Gestalt.

Beweis (i) Es gilt $a_{ij} = 0$ für $i > j + 1$. Wir zeigen induktiv, dass diese Eigenschaft für alle Matrizen $A^{(i)}$ gilt, die im Laufe der QR-Zerlegung entstehen. Zunächst folgt im ersten Schritt für $v^{(1)} = a_1 + \|a_1\|e_1$, dass $v_k^{(1)} = 0$ für alle $k > 2$. Die neuen Spaltenvektoren berechnen sich zu

$$a_i^{(1)} = a_i - (a_i, v^{(1)})v^{(1)}. \quad (5.8)$$

Da $v_k^{(1)} = 0$ für $k > 2$, gilt $(a_i^{(1)})_k = 0$ für $k > i + 1$, d.h., $A^{(1)}$ hat wieder Hessenberg-Normalform. Diese Eigenschaft gilt induktiv für $i = 2, \dots, n - 1$.

(ii) Da der (reduzierte) Vektor $\tilde{v}^{(i)}$ in jedem Schritt nur zwei von null verschiedene Einträge hat, kann dieser in zwei arithmetischen Operationen erstellt werden. Die Berechnung der $n - i$ neuen Spaltenvektoren gemäß (5.8) bedarf je vier arithmetischer Operationen. Insgesamt ergibt sich ein Aufwand von

$$\sum_{i=1}^{n-1} 2 + 4(n - i) = 2n^2 + O(n).$$

□

Eine Python-Implementierung kann dann folgende Gestalt haben. Hierbei berechnen wir auch die Matrix Q explizit, damit wir diese später in der Berechnung der Eigenwerte mit dem QR-Verfahren verwenden können.

♣ Implementierung 5.7: QR-Zerlegung für Hessenberg Matrizen

```

1 def qr_householder_fuer_hessenberg(A):
2     n, m = A.shape
3     dtype = A.dtype
4     Q = np.identity(n, dtype=dtype)
5     for i in range(m - 1):
6         v = A[i: i + 2, i].copy()
7         ei = np.zeros(2, dtype=dtype)
8         ei[0] = 1.0
9         v += np.sign(A[i,i]) * np.linalg.norm(v) * ei
10        v /= np.linalg.norm(v)
11        A[i:i + 2, i:] -= 2 * np.outer(v, np.inner(v.T, A[i:i + 2,
12        ↪ i:].T))
13        Q[:, i:i + 2] -= 2 * np.outer(np.inner(Q[:, i: i + 2], v), v)
14    return Q

```

In Verbindung mit der Reduktion auf Hessenberg, bzw. auf Tridiagonalgestalt ist das QR-Verfahren eines der effizientesten Verfahren zur Berechnung von Eigenwerten einer Matrix $A \in \mathbb{R}^{n \times n}$. Die QR-Zerlegung kann in $O(n^2)$ Operationen durchgeführt werden und im Laufe des QR-Verfahrens entstehen ausschließlich Hessenberg-Matrizen. Abschließend betrachten wir hierzu ein Beispiel:

Beispiel 5.29 (Eigenwertberechnung mit Reduktion und QR-Verfahren)

Wir betrachten die Matrix

$$A = \begin{pmatrix} 338 & -20 & -90 & 32 \\ -20 & 17 & 117 & 70 \\ -90 & 117 & 324 & -252 \\ 32 & 70 & -252 & 131 \end{pmatrix}.$$

Die Matrix A ist symmetrisch und hat die Eigenwerte

$$\lambda_1 \approx 547.407, \quad \lambda_2 \approx 297.255, \quad \lambda_3 \approx -142.407, \quad \lambda_4 \approx 107.745.$$

Schritt 1: Reduktion auf Hessenberg-(Tridiagonal)-Gestalt

Im ersten Reduktionsschritt wählen wir mit $\tilde{a}_1 = (0, -20, -90, 32)$ den Spiegelungsvektor $v^{(1)}$ als

$$v^{(1)} = \frac{\tilde{a}_1 - \|\tilde{a}_1\|e_2}{\|\tilde{a}_1 - \|\tilde{a}_1\|e_2\|}.$$

Im ersten Schritt unseres Codes erhalten wir mit $S_{(1)} := I - 2v^{(1)}(v^{(1)})^T$ die Matrix $A_{(1)} = S_{(1)}AS_{(1)}$:

```
[ [ 3.380000000e+02  9.75909832e+01 -2.84217094e-14  7.10542736e-15]
  [ 9.75909832e+01  4.77579168e+02  2.77974214e+01  1.06979728e+02]
  [-2.84217094e-14  2.77974214e+01 -8.23446072e+01 -1.03493607e+02]
  [ 7.10542736e-15  1.06979728e+02 -1.03493607e+02  7.67654387e+01]]
```

Mit $\tilde{a}_2^{(1)} \approx (0, 0, 27.797, 106.980)^T$,

$$v^{(2)} = \frac{\tilde{a}_2 + \|\tilde{a}_2\|e_3}{\|\tilde{a}_2 + \|\tilde{a}_2\|e_3\|} \quad \text{und} \quad S_{(2)} := I - 2v^{(2)}(v^{(2)})^T$$

die Matrix $H := A_{(2)} = S_{(2)}A_{(1)}S_{(2)}$:

```
[ [ 3.380000000e+02  9.75909832e+01  2.70632072e-16  2.92951775e-14]
  [ 9.75909832e+01  4.77579168e+02 -1.10532162e+02  1.42108547e-14]
  [-2.84217094e-14 -1.10532162e+02  1.63207694e+01 -1.29130642e+02]
  [ 7.10542736e-15  0.00000000e+00 -1.29130642e+02 -2.18999378e+01]]
```

Die Matrix H hat nun (bis auf Rundungsfehler) Tridiagonalgestalt und alle weiteren Berechnungen basieren auf

```
[ [ 3.380000000e+02  9.75909832e+01  0.00000000e+00  0.00000000e+00]
  [ 9.75909832e+01  4.77579168e+02 -1.10532162e+02  0.00000000e+00]
  [ 0.00000000e+00 -1.10532162e+02  1.63207694e+01 -1.29130642e+02]
  [ 0.00000000e+00  0.00000000e+00 -1.29130642e+02 -2.18999378e+01]]
```

Alle Transformationen waren Ähnlichkeitstransformationen. Daher haben H und A die gleichen Eigenwerte. Wenn wir die Eigenwerte von A und H mittels der Funktion `np.linalg.eig` numerisch überprüfen, sehen wir, dass der Unterschied bei 10^{-13} , also in der Größenordnung der Maschinengenauigkeit, liegt.

Schritt 2: QR-Verfahren

Wir führen nun einige Schritte des QR-Verfahrens durch. Hierfür verwenden wir unsere QR-Zerlegung mit Householder Transformationen für den Spezialfall von Hessenberg-Matrizen. Das QR-Verfahren hat dann folgende Gestalt:

🔧 Implementierung 5.8: QR-Verfahren für Hessenberg Matrizen

```
1 def eigenwerte_hessenberg_qr(A, k):
2     reduktion_auf_hessenberg(A)
3     for i in range(k):
4         Q = qr_householder_fuer_hessenberg(A)
5         A = A @ Q
6     return np.diagonal(A)
```

Wir wenden dies auf die obige Matrix an und betrachten die Ergebnisse aus den ersten Iterationen

A_0=

```
[ [ 4.00759162e+02  1.23633693e+02  2.96068783e-14  6.09597216e-15]
 [ 1.23633693e+02  4.41337578e+02  3.21310279e+01 -3.62480544e-14]
 [ 3.12483731e-14  3.21310279e+01 -4.20816954e+01 -1.22497513e+02]
 [-6.82657267e-15 -1.21878868e-14 -1.22497513e+02  9.98495521e+00]]
```

A_1=

```
[ [ 4.73938190e+02  1.13970724e+02  1.91482189e-15  1.80543844e-14]
 [ 1.13970724e+02  3.70410752e+02 -1.08311338e+01 -5.07842115e-15]
 [-2.98597627e-14 -1.08311338e+01 -7.46803080e+01 -1.11052737e+02]
 [ 1.01160781e-14  9.60055188e-15 -1.11052737e+02  4.03313663e+01]]
```

A_2=

```
[ [ 5.20096493e+02  7.80160200e+01  1.23209238e-14 -1.25066927e-14]
 [ 7.80160200e+01  3.24515418e+02  4.34959013e+00 -3.78291085e-14]
 [ 2.94474464e-14  4.34959013e+00 -9.87169105e+01 -9.49422609e+01]
 [-1.20803921e-14 -7.42681841e-15 -9.49422609e+01  6.41050000e+01]]
```

...

A_9=

```
[ [ 5.47401220e+02  1.21920894e+00 -1.20882609e-14  2.00836013e-14]
 [ 1.21920894e+00  2.97261149e+02 -2.16530512e-02  1.38847046e-14]
 [-2.91300457e-14 -2.16530512e-02 -1.41346782e+02 -1.62521231e+01]
 [ 1.38390076e-14  3.05621674e-15 -1.62521231e+01  1.06684413e+02]]
```

Für die Diagonalelemente der Matrix A_9, also nach 10 Iterationen, haben wir

$$a_{11}^{(9)} \approx 547.401, \quad a_{22}^{(9)} \approx 297.261, \quad a_{33}^{(9)} \approx -141.346, \quad a_{44}^{(9)} \approx 106.684. \quad (5.9)$$

Diese Diagonalelemente stellen sehr gute Näherungen an *alle* Eigenwerte der Matrix A dar:

$$\begin{aligned} \frac{|a_{11}^{(10)} - \lambda_1|}{|\lambda_1|} &\approx 1.085 \cdot 10^{-5}, & \frac{|a_{22}^{(10)} - \lambda_2|}{|\lambda_2|} &\approx 1.998 \cdot 10^{-5}, \\ \frac{|a_{33}^{(10)} - \lambda_3|}{|\lambda_3|} &\approx 0.0098, & \frac{|a_{44}^{(10)} - \lambda_4|}{|\lambda_4|} &\approx 0.0074 \end{aligned}$$

Der Fehler für λ_3 und λ_4 ist größer, da diese Eigenwerte weniger separiert sind. Mit einer guten Shift-Strategie kann die Konvergenz weiter beschleunigt werden. ◀

5.6 Exkurs: Eigenschwingungen eines Mehrfach-Federpendels

In diesem Exkurs betrachten wir das Verhalten von Federpendeln, siehe Abb. 5.2. Bei der Beschreibung von mechanischen Systemen wie dem Federpendel spielen die Eigenwerte des Systems oft eine wichtige Rolle. Hier werden sie das Auftreten des Resonanzeffekts, also eine Verstärkung von einwirkenden Kräften, erklären.

Ein (mathematisches) Federpendel ist eine punktförmige Masse m (d. h., sie hat keine Ausdehnung), die an einer masselosen Feder der Länge l hängt. Durch die Einwirkung einer Kraft, z. B. der Gravitationskraft, wird die Masse angezogen und hierdurch die Feder ausgedehnt. Die Feder setzt dieser Ausdehnung u eine Kraft F entgegen und wir gehen vereinfacht davon aus, dass die Kraft proportional zur Ausdehnung (und auch zur Stauchung der Feder) ist, also

$$F(u) = \kappa \cdot u,$$

wobei $\kappa > 0$ die sogenannte *Federkonstante* ist. Dieses Gesetz wird *Hookesches Gesetz* genannt.

5.6.1 Mathematische Modellierung

Um eine Gleichung für die Feder herzuleiten, wenden wir das Prinzip der *Energieerhaltung* an: Die gesamte Energie des Systems bleibt über die Zeit erhalten. Die Energie teilen wir auf in die *kinematische Energie*

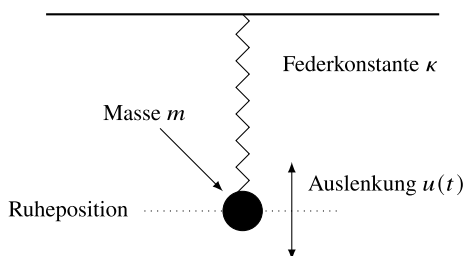
$$E_{kin}(t) = \frac{1}{2} m u'(t)^2,$$

wobei $v(t) := u'(t)$ die Geschwindigkeit der Masse ist, und in die *potentielle Energie*, welche durch die Ausdehnung der Feder gegeben ist:

$$E_{pot}(t) = \frac{1}{2} \kappa u(t)^2$$

Für die Gesamtenergie $E(t) = E_{kin}(t) + E_{pot}(t)$ gilt die Erhaltungsgleichung

Abb. 5.2 Konfiguration eines mathematischen Federpendels. Eine Masse m hängt an einer masselosen Feder. Die Kraft der Feder ist proportional zur Auslenkung $F = \kappa \cdot u$



$$0 \stackrel{!}{=} \frac{d}{dt} E(t) = mu''(t)u'(t) + \kappa u'(t)u(t) = (mu''(t) + \kappa u(t)) \cdot u'(t).$$

Damit diese Bedingung für alle möglichen Werte von $u'(t)$, d. h. für alle Geschwindigkeiten, erfüllt ist, muss überall die *Differentialgleichung*

$$u''(t) + \lambda^2 u(t) = 0, \quad \lambda^2 := \frac{\kappa}{m} > 0, \quad (5.10)$$

erfüllt sein. Gesucht ist eine Funktion $u(t)$, deren zweite Ableitung gerade gleich $-\lambda^2$ -mal die Funktion selbst ist:

$$u''(t) = -\lambda^2 u(t)$$

Infrage kommen die trigonometrischen Funktionen und wir machen den Ansatz:

$$u(t) = c_1 \sin(\omega_1 t) + c_2 \cos(\omega_2 t)$$

mit

$$u''(t) = -c_1 \omega_1^2 \sin(\omega_1 t) - c_2 \omega_2^2 \cos(\omega_2 t).$$

Wie wir leicht sehen, müssen insgesamt vier unbekannte Parameter spezifiziert werden. Dies geschieht mithilfe der Materialeigenschaften sowie der Anfangsdaten. Wir können zunächst ablesen, dass

$$\omega_1 = \omega_2 = \lambda = \sqrt{\frac{\kappa}{m}}$$

gelten muss. Hiermit folgt

$$u\left(t + \frac{2\pi}{\lambda} k\right) = u(t), \quad k \in \mathbb{Z}, \quad (5.11)$$

d. h., $2\pi/\lambda$ ist die Periode der Schwingung und $\lambda/(2\pi)$ die Frequenz des Federpendels.

Es bleibt die Bestimmung der beiden Konstanten c_1 und c_2 . Um diese eindeutig festlegen zu können, fehlen noch Bedingungen. Um ein wohldefiniertes System zu erhalten, müssen wir den Zustand der Feder zu Beginn der Betrachtung definieren. Wir gehen davon aus, dass die Masse am Anfang eine Auslenkung u^0 erfährt, aber im Ruhezustand mit $v(0) = u'(0) = 0$ ist. Hieraus erhalten wir

$$\begin{aligned} u^0 &= u(0) = c_2, \\ 0 &= v(0) = u'(0) = c_1. \end{aligned}$$

Die Lösung der Federgleichung mit diesen *Anfangsbedingungen* ist hiermit

$$u(t) = u^0 \cos\left(\sqrt{\frac{\kappa}{m}} t\right).$$

Modell mit vielen Federn

Wir gehen jetzt davon aus, dass das System aus N Massen m_i besteht, die jeweils eine Auslenkung $u_i(t)$ erfahren. Dabei seien die Massen m_{i-1} und m_i jeweils mit einer Feder mit Federkonstante κ verbunden (zur Vereinfachung seien alle Federn mit der gleichen Konstante κ beschrieben), siehe Abb. 5.3. Wir bestimmen die kinetische Energie als Summe aller Teilenergien

$$E_{kin}(t) = \sum_{i=1}^N \frac{1}{2} m_i u_i'(t)^2$$

und die potentielle Energie als Summe der potentiellen Energien der einzelnen Federn, siehe erneut Abb. 5.3,

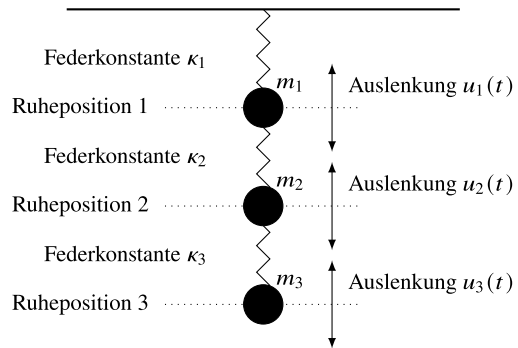
$$E_{pot}(t) = \sum_{i=1}^N \frac{1}{2} \kappa (u_i(t) - u_{i-1}(t))^2,$$

dabei sei $u_0(t) \equiv 0$ nur zur einfachen Schreibweise eingeführt. Wir wenden erneut das Prinzip der Energieerhaltung an. Für die Ableitung der totalen Energie $E(t) = E_{kin}(t) + E_{pot}(t)$ gilt

$$\begin{aligned} E'(t) &= \sum_{i=1}^N (m_i u_i''(t) u_i'(t) + \kappa (u_i'(t) - u_{i-1}'(t)) (u_i(t) - u_{i-1}(t))) \\ &= \sum_{i=1}^{N-1} u_i'(t) (m_i u_i''(t) + \kappa (u_i(t) - u_{i-1}(t)) - \kappa (u_{i+1}(t) - u_i(t))) \\ &\quad + u_N'(t) (m_N u_N''(t) + \kappa (u_N(t) - u_{N-1}(t))). \end{aligned}$$

Aus der Bedingung $E'(t) \equiv 0$ folgt (für $N > 1$) das System aus Differentialgleichungen

Abb. 5.3 Konfiguration eines mathematischen Federpendels mit drei Massen m_1, m_2, m_3 und drei Federn mit Konstanten $\kappa_1, \kappa_2, \kappa_3$. Wir beschreiben die Auslenkung $u_i(t)$ jeder Masse relativ zu ihrer Ruheposition



$$\begin{aligned}
u_1''(t) + \frac{2\kappa}{m_1}u_1(t) - \frac{\kappa}{m_1}u_2(t) &= 0, \\
u_2''(t) - \frac{\kappa}{m_2}u_1(t) + \frac{2\kappa}{m_2}u_2(t) - \frac{\kappa}{m_2}u_3(t) &= 0, \\
&\vdots \\
u_i''(t) - \frac{\kappa}{m_i}u_{i-1}(t) + \frac{2\kappa}{m_i}u_i(t) - \frac{\kappa}{m_i}u_{i+1}(t) &= 0, \\
&\vdots \\
u_{N-1}''(t) - \frac{\kappa}{m_{N-1}}u_{N-2}(t) + \frac{2\kappa}{m_{N-1}}u_{N-1}(t) - \frac{\kappa}{m_{N-1}}u_N(t) &= 0, \\
u_N''(t) - \frac{\kappa}{m_N}u_{N-1}(t) + \frac{\kappa}{m_N}u_N(t) &= 0.
\end{aligned}$$

Falls nur eine Feder existiert, also $N = 1$, so ist wieder

$$u_1''(t) + \frac{\kappa}{m_1}u_1(t) = 0.$$

Wir gehen nun stets von $N > 1$ aus. Durch Einführung des Vektors $u(t) \in \mathbb{R}^N$ können wir die Gleichung kompakt schreiben als

$$u''(t) + Au(t) = 0, \quad A = \kappa \begin{pmatrix} \frac{2}{m_1} & -\frac{1}{m_1} & 0 & \cdots & 0 \\ -\frac{1}{m_2} & \frac{2}{m_2} & -\frac{1}{m_2} & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -\frac{1}{m_{N-1}} & \frac{2}{m_{N-1}} & -\frac{1}{m_{N-1}} \\ 0 & \cdots & 0 & -\frac{1}{m_N} & \frac{1}{m_N} \end{pmatrix}. \quad (5.12)$$

Satz 5.30 (Abschätzung der Eigenwerte der Federmatrix) Die Matrix A , gegeben durch (5.12), ist stets regulär. Alle Eigenwerte μ von A sind reell und positiv und es gilt die Abschätzung

$$0 < \mu \leq \max \left\{ \frac{2\kappa}{m_N}, \max_{i < N} \frac{4\kappa}{m_i} \right\}.$$

Im Fall uniformer Federmassen $m_i \equiv m$ für alle i ist die Matrix A , gegeben durch (5.12), symmetrisch positiv definit.

Beweis (i) Wir zeigen zunächst, dass die Matrix A regulär ist. Hierzu sei $M \in \mathbb{R}^{N \times N}$ die reguläre Diagonalmatrix

$$M = \text{diag}(m_1, \dots, m_N).$$

Dann ist

$$A = M^{-1} \underbrace{\kappa \begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -1 & 2 & -1 \\ 0 & \cdots & 0 & -1 & 1 \end{pmatrix}}_{=: \bar{A}}.$$

Die Matrix $\bar{A}$ ist regulär, denn die Betrachtung des homogenen Systems

$$\bar{A}x = 0$$

liefert (von hinten nach vorne)

$$\begin{aligned} -x_{N_1} + x_N &= 0 & \Leftrightarrow & x_{N-1} = x_N \\ -x_{N_2} + 2x_{N-1} - x_N &= 0 & \Leftrightarrow & x_{N-2} = 2x_{N-1} - x_N = x_N, \\ & \vdots & & \\ -x_1 + 2x_2 - x_3 &= 0 & \Leftrightarrow & x_1 = 2x_2 - x_3 = x_N, \end{aligned}$$

und schließlich wegen

$$2x_1 - x_2 = 0,$$

dass $x_1 = x_2 = \cdots = x_N = 0$ gelten muss. Die Matrix $\bar{A}$ und somit auch A ist also regulär.

(ii) Es sei $\mu \in \mathbb{C}$ ein Eigenwert von A mit zugehörigem Eigenvektor $w \in \mathbb{C}^N$. Dann ist

$$\mu w = Aw = \kappa M^{-1} \bar{A} w. \quad (5.13)$$

Wir definieren die reguläre Diagonalmatrix

$$M^{\frac{1}{2}} = \text{diag}(\sqrt{m_1}, \dots, \sqrt{m_N}).$$

Multiplikation von (5.13) mit $M^{\frac{1}{2}}$ liefert

$$\mu(M^{\frac{1}{2}} w) = M^{-\frac{1}{2}} \kappa \bar{A} w = [M^{-\frac{1}{2}} \kappa \bar{A} M^{-\frac{1}{2}}](M^{\frac{1}{2}} w).$$

Durch μ ist auch ein Eigenwert zur Matrix

$$\kappa M^{-\frac{1}{2}} \bar{A} M^{-\frac{1}{2}}$$

gegeben. Diese Matrix ist symmetrisch, also muss μ reell sein.

(iii) Die Abschätzung der Eigenwerte erfolgt mithilfe des Satzes von Gerschgorin, Satz 5.4. Im Fall $m_i = m$ ist die Matrix symmetrisch. $\square$

Es seien also $\lambda_1^2, \dots, \lambda_N^2 \in \mathbb{R}_+$ die positiven reellen Eigenwerte der Matrix A . Wir können einfach zeigen, dass die Matrix diagonalisierbar ist. Denn es existiert eine orthogonale Matrix $\bar{W}$, sodass für die symmetrisch positiv definite Matrix $\kappa M^{-\frac{1}{2}} \bar{A} M^{-\frac{1}{2}}$ gilt:

$$\bar{W}^T [M^{-\frac{1}{2}} \kappa \bar{A} M^{-\frac{1}{2}}] \bar{W} = \Lambda$$

mit einer Diagonalmatrix

$$\Lambda = \text{diag}(\lambda_1^2, \dots, \lambda_N^2).$$

Wegen $\bar{W}^T = \bar{W}^{-1}$ folgt

$$\Lambda = \bar{W}^{-1} [M^{-\frac{1}{2}} \kappa \bar{A} M^{-\frac{1}{2}}] \bar{W} = \underbrace{\bar{W}^{-1} M^{\frac{1}{2}}}_{=: W^{-1}} \underbrace{M^{-1} \kappa \bar{A}}_{=: A} \underbrace{M^{-\frac{1}{2}} \bar{W}}_{=: W}.$$

Die Matrix A ist also diagonalisierbar mit der regulären Matrix $W = M^{-\frac{1}{2}} \bar{W}$.

Mit dieser Matrix W können wir die Differentialgleichung transformieren. Wir definieren die transformierte Lösung

$$\bar{u}(t) = Wu(t),$$

und erhalten bei Multiplikation der Differentialgleichung von links mit W^{-1} das diagonalisierte System

$$\bar{u}''(t) + \Lambda \bar{u}(t) = 0$$

oder elementweise als

$$\bar{u}_i''(t) + \lambda_i^2 \bar{u}_i(t) = 0, \quad i = 1, \dots, N.$$

Dieses System können wir wieder mit dem Ansatz

$$\bar{u}_i(t) = a_i \sin(\lambda_i t) + b_i \cos(\lambda_i t)$$

lösen. Zum Zeitpunkt $t = 0$ seien wieder Auslenkungen u^0 vorgegeben. Die Geschwindigkeit ist 0. Mit $\bar{u}^0 = Wu^0$ folgt

$$\bar{u}_i(t) = \bar{u}_i^0 \cos(\lambda_i t), \quad u(t) = W^{-1} \bar{u}(t).$$

Die Wurzeln der Eigenwerte bestimmen die Frequenzen der einzelnen Schwingungsanteile.

Bemerkung 5.31 (Bedeutung von Eigenschwingungen) Den Eigenwerten von mechanischen Systemen kommt eine große Bedeutung zu. Wir gehen nun davon aus, dass die Feder (wir betrachten eine einzelne) nicht nur zum Anfangszeitpunkt ausgelenkt ist, sondern dass ständig eine Kraft einwirkt. Die Differentialgleichung (5.10) schreibt sich dann als

$$u''(t) + \lambda^2 u(t) = f(t), \quad u(0) = u^0, \quad u'(0) = 0, \quad (5.14)$$

wobei $f(t)$ die einwirkende Kraft ist. Wir gehen davon aus, dass diese Kraft beliebig klein ist, $|f(t)| \leq \epsilon$, mit einem $\epsilon > 0$, dass sie aber mit der gleichen Frequenz oszilliert:

$$f(t) = \epsilon \sin(\lambda t)$$

Für den Ansatz

$$u(t) = u^0 \cos(\lambda t) + \frac{\epsilon}{2\lambda} t \sin(\lambda t)$$

können wir nachrechnen, dass eine Lösung von Gl. (5.14) vorliegt. Für diese Lösung gilt

$$|u(t)| \rightarrow \infty \quad (t \rightarrow \infty).$$

Egal wie klein die periodisch wirkende Kraft $|f(t)| \leq \epsilon$ ist, die Schwingung wird stets verstärkt. Dieser Effekt wird *Resonanz* genannt und tritt immer dann auf, wenn eine periodische Kraft genau eine Eigenfrequenz trifft. Sie spielt bei vielen Bauwerken eine Rolle, die regelmäßigen Belastungen ausgesetzt sind. Dies sind beispielsweise Brücken mit einer starken Belastung durch Wind, Fußballstadien mit hüpfenden Fans oder Ölplattformen bei starkem, gleichmäßigem Wellengang. In Fahrzeugen sollte die Motorschwingung nicht den Abgasstrang anregen, da die Vibrationen langfristig Risse entstehen lassen würden, die zum Bruch führen. Bei dem Entwurf von Bauwerken und -teilen ist darauf zu achten, dass alle maßgeblichen Eigenfrequenzen in einem Bereich liegen, der durch die zu erwartende Belastung nichtperiodisch angeregt wird, um so Eigenschwingungen zu vermeiden. Aus diesem Grund spielen größte und kleinste Eigenwerte eine besondere Rolle. ♦

Uniformes Mehrfachfederpendel

Wir betrachten zunächst den einfachen Fall

$$m_i = m, \quad \kappa_i = \kappa, \quad i = 1, \dots, N.$$

Dann ist die Matrix gegeben als

$$A = \frac{\kappa}{m} \begin{pmatrix} 2 & -1 & 0 & \dots & 0 \\ -1 & 2 & -1 & \ddots & \vdots \\ 0 & \ddots & \ddots & \ddots & 0 \\ \vdots & \ddots & -1 & 2 & -1 \\ 0 & \dots & 0 & -1 & 1 \end{pmatrix}.$$

Wir wählen $m = 1$ und $\kappa = 5$. Im Folgenden werden wir den größten und kleinsten Eigenwert $\lambda_{\max}^2$ und $\lambda_{\min}^2$ dieser Matrix berechnen. Der Satz von Gerschgorin 5.4 gibt uns die Schranke

$$0 \leq \lambda_i^2 \leq \frac{4\kappa}{m} \leq 20.$$

Wir können somit die höchste im System auftretende Eigenfrequenz abschätzen als

$$F_{\max} = \frac{\lambda_{\max}}{2\pi} \leq \frac{\sqrt{20}}{2\pi} \approx 0.71.$$

Um den kleinsten Eigenwert zu berechnen, verwenden wir die inverse Iteration.

N	1	2	4	8	16
$\lambda_{\min}^2$	5	1.91	0.60	0.17	0.045
$F_{\min}$	0.35	0.22	0.12	0.066	0.034
$\lambda_{\max}^2$	5	13.01	17.58	18.53	18.55
$F_{\max}$	0.35	0.58	0.67	0.69	0.69

Für steigende Anzahlen an Federn sinken kleinster Eigenwert und damit auch kleinste Frequenz. Der größte Eigenwert nähert sich langsam der Schranke 20 an.

5.6.2 Numerische Approximation der Differentialgleichung

Auch wenn Verfahren zur numerischen Approximation von Differentialgleichungen ganze Bücher füllen, wollen wir kurz einen einfachen Weg zur Approximation der Lösung von

$$u''(t) + Au(t) = 0 \text{ für } t > 0 \text{ sowie } u(0) = u^0 \in \mathbb{R}^N \text{ und } v(0) := u'(0) = 0$$

beschreiben. Grundsätzliche Idee einer Approximation ist es, dass wir nicht die ganze Funktion $u(t)$, sondern nur einzelne diskrete Funktionswerte $u(t_n)$ suchen für

$$t_n = n \cdot h, \quad n = 0, 1, 2, \dots$$

mit einem Parameter $h > 0$, der sogenannten *Gitterweite*. Eine Taylor-Entwicklung der Lösung ergibt

$$u(t_{n+1}) = u(t_n + h) = u(t_n) + hu'(t_n) + \frac{h^2}{2}u''(t_n) + O(h^3).$$

Wir können die erste Ableitung durch die Geschwindigkeit $v(t_n) = u'(t_n)$ und die zweite Ableitung durch die Differentialgleichung $u''(t_n) = -Au(t_n)$ ausdrücken. So erhalten wir

$$u(t_{n+1}) = u(t_n) + hv(t_n) - \frac{h^2}{2}Au(t_n) + O(h^3) \quad (5.15)$$

oder für die numerische Approximation $u_n \approx u(t_n)$ und $v_n \approx v(t_n)$ die Vorschrift

$$u_{n+1} = \left(I - \frac{h^2}{2}A \right) u_n + hv_n.$$

Kennen wir u_n und v_n , so lässt sich u_{n+1} leicht berechnen. Im Anschluss benötigen wir eine Approximation von v_{n+1} . Zur Berechnung haben wir v_n , u_n und auch u_{n+1} zur Verfügung. Wir machen den Ansatz

$$v(t_{n+1}) = v(t_n) + \int_{t_n}^{t_{n+1}} v'(s) ds.$$

Das Integral approximieren wir mit der *Trapezregel*, siehe Definition 10.4, die wir in Kap. 10 diskutieren werden:

$$v(t_{n+1}) = v(t_n) + \int_{t_n}^{t_{n+1}} v'(s) ds \approx v(t_n) + \frac{h}{2} v'(t_n) + \frac{h}{2} v'(t_{n+1}) + O(h^3)$$

Mit $v'(t) = u''(t) = -Au(t)$ erhalten wir schließlich

$$v(t_{n+1}) = v(t_n) - \frac{h}{2} Au(t_n) - \frac{h}{2} Au(t_{n+1}) + O(h^3). \quad (5.16)$$

Zusammen erhalten wir die einfache Iterationsvorschrift

$$\left. \begin{aligned} (i) \quad u_{n+1} &= u_n - \frac{h^2}{2} Au_n + hv_n \\ (ii) \quad v_{n+1} &= v_n - \frac{h}{2} Au_n - \frac{h}{2} Au_{n+1} \end{aligned} \right\} \quad n = 1, 2, \dots \quad (5.17)$$

Satz 5.32 (Konvergenz der Approximationsvorschrift) Bei exakten Startwerten $u_0 = u(0)$ und $v_0 = v(0)$ gilt für die von der Iterationsvorschrift (5.17) erzeugte Approximation die Fehlerabschätzung

$$\max_{1 \leq k \leq n} \{\|u(t_k) - u_k\|, \|v(t_k) - v_k\|\} \leq c(t_n)h^2,$$

mit einer von der Intervalllänge t_n abhängigen Konstante.

Beweis Es sei $n \in \mathbb{N}$ und $t_n = hn$. Aus der Herleitung der Vorschrift erhalten wir mit (5.15) und (5.16) für die echte Lösung $u(t)$ und $v(t)$ den Zusammenhang

$$\begin{aligned} u(t_{n+1}) &= u(t_n) + hv(t_n) - \frac{h^2}{2} Au(t_n) + O(h^3), \\ v(t_{n+1}) &= v(t_n) - \frac{h}{2} Au(t_n) - \frac{h}{2} Au(t_{n+1}) + O(h^3). \end{aligned} \quad (5.18)$$

Zur Beschreibung des Fehlers führen wir die Bezeichnungen

$$e_n^u := u(t_n) - u_n, \quad e_n^v := v(t_n) - v_n$$

ein. Dann folgt aus (5.17) und (5.18):

$$\begin{aligned}
 (i) \quad e_{n+1}^u &= e_n^u - \frac{h^2}{2} A e_n^u + h e_n^v + O(h^3) \\
 (ii) \quad e_{n+1}^v &= e_n^v - \frac{h}{2} A e_n^u - \frac{h}{2} A e_{n+1}^u + O(h^3)
 \end{aligned}$$

Setzen wir die erste in die zweite Zeile ein, so können wir e_{n+1}^u eliminieren:

$$e_{n+1}^v = e_n^v - \frac{h}{2} A e_n^u - \frac{h}{2} A e_n^u - \frac{h^2}{2} A e_n^v + \frac{h^3}{4} A e_n^u + O(h^3)$$

Aus (i) und dieser Fehlerformel für e_n^v erhalten wir in der Spektralnorm

$$\begin{aligned}
 \|e_{n+1}^u\|_2 &\leq \left\| I - \frac{h^2}{2} A \right\|_2 \|e_n^u\|_2 + h \|e_n^v\|_2 + O(h^3), \\
 \|e_{n+1}^v\|_2 &\leq \left\| I - \frac{h^2}{2} A \right\|_2 \|e_n^v\|_2 + \left(h + \frac{h^3}{4} \right) \|A\|_2 \|e_n^u\|_2 + O(h^3).
 \end{aligned}$$

Nun sei $E_n := \max\{\|e_n^u\|_2, \|e_n^v\|_2\}$ und wir erhalten für das Maximum

$$\|E_{n+1}\| \leq \left(\left\| I - \frac{h^2}{2} A \right\|_2 + h + \left(h + \frac{h^3}{4} \right) \|A\|_2 \right) \|E_n\| + O(h^3).$$

In Satz 5.30 haben wir gezeigt, dass die Eigenwerte λ^2 der Matrix A alle reell und positiv sind. Für $0 < h < 2/\sqrt{\text{spr}(A)}$ gilt

$$\left\| I - \frac{h^2}{2} A \right\|_2 \leq 1.$$

Hiermit folgt

$$\|E_{n+1}\|_2 \leq \alpha(h) \|E_n\|_2 + O(h^3), \quad \alpha(h) := 1 + h + \left(h + \frac{h^3}{4} \right) \|A\|_2. \quad (5.19)$$

Eine wiederholte Anwendung dieser Abschätzung gibt:

$$\|E_n\|_2 \leq \alpha(h)^n \|E_0\| + O(h^3) \cdot \sum_{k=0}^{n-1} \alpha(h)^k$$

Es ist $E_0 = 0$, da die Startwerte bekannt sind. Weiter ist

$$\alpha(h)^k = \left(1 + h + \left(h + \frac{h^3}{4} \right) \|A\|_2 \right)^k = e^{k \log \left(1 + h + \left(h + \frac{h^3}{4} \right) \|A\|_2 \right)}.$$

Wegen $\log(1+x) \leq x$ für $x > 0$ folgt mit $k = t_k/h$

$$\alpha(h)^k \leq e^{t_k \left(1 + \left(1 + \frac{h^2}{4}\right) \|A\|_2\right)} \leq e^{c(A)t_k} \leq e^{c(A)t_n}.$$

Mit (5.19) erhalten wir schließlich bei Summation das Ergebnis

$$\|E_n\|_2 \leq O(h^3) \cdot n e^{c(A)t_n} \leq O(h^2) t_n e^{c(A)t_n}. \quad \square$$

Bemerkung 5.33 Die hier untersuchte Methode ist eine sogenannte Differenzenmethode zur Diskretisierung von Anfangswertaufgaben, siehe z. B. [25]. Die Untersuchung ist bereits sehr aufwendig, da die Methode von zweiter Ordnung in der Gitterweite h ist.

Auch die Differentialgleichung $u'' + Au = 0$ ist von zweiter Ordnung. Dies bedeutet, dass die zweite Ableitung von $u(t)$ auftaucht. Durch Einführen der Geschwindigkeit $v = u'$ als Hilfsvariable haben wir die Gleichung umgeschrieben in ein System erster Ordnung. Die hier untersuchte Methode gehört zu den Runge-Kutta-Verfahren. Zur Numerik gewöhnlicher Differentialgleichungen existiert vielfältige Literatur [76, 82]. $\blacklozenge$

Wir führen nun einige einfache Simulationen durch. Zunächst zeigen wir die Schwingung eines einfachen Pendels, welches zu Beginn mit $u(0) = -0.1$ und $v(0) = 0$ ausgelenkt ist. In Abb. 5.4 zeigen wir drei verschiedene Situationen. Ganz oben hat die Feder die Federkonstante $\kappa = (2\pi)^2$ und die Masse $m = 1$. In der Mitte ist $\kappa = (3\pi)^2$ und weiterhin $m = 1$, ganz unten ist $\kappa = (3\pi)^2$ und $m = 4$. Es stellen sich jeweils die nach (5.11) zu erwartenden Frequenzen ein. Für den Fall $\kappa = (3\pi)^2$ und $m = 4$ werten wir für verschiedene Gitterweiten $h > 0$, die numerisch berechnete Auslenkung und die Geschwindigkeit zum Zeitpunkt $T = 1/2$ aus und stellen den Fehler dar. Die exakte Lösung ist durch $u(t) = u^0 \cos(\sqrt{\kappa/mt})$ gegeben durch

$$u\left(\frac{1}{2}\right) \approx 0.707107, \quad v\left(\frac{1}{2}\right) \approx 0.333216.$$

h	0.1	0.05	0.025	0.0125
$ u(10) - u_N $	$1.56 \cdot 10^{-3}$	$3.87 \cdot 10^{-4}$	$9.64 \cdot 10^{-5}$	$2.41 \cdot 10^{-5}$
$ v(10) - v_N $	$1.67 \cdot 10^{-2}$	$4.14 \cdot 10^{-3}$	$1.03 \cdot 10^{-3}$	$2.58 \cdot 10^{-4}$

Die quadratische Ordnung, also eine Viertelung des Fehlers bei Halbierung der Schrittweite, ist klar abzulesen.

Als zweites Beispiel betrachten wir ein Vierfach-Federpendel. Wir wählen die uniforme Federkonstante

$$\kappa = (2\pi)^2$$

und zunächst auch eine uniforme Gewichtsverteilung

$$m_1 = m_2 = m_3 = m_4 = 1.$$

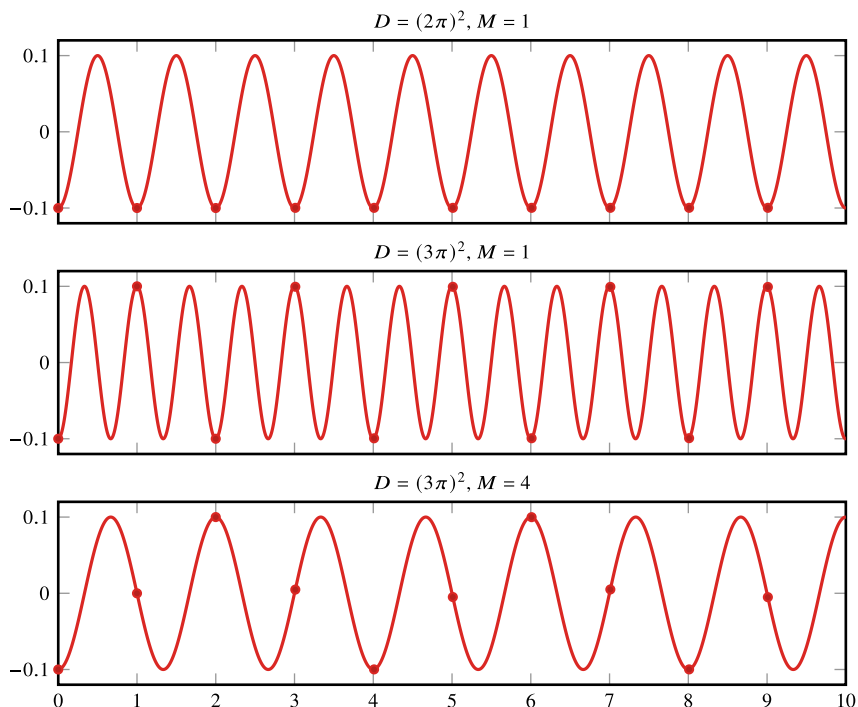


Abb. 5.4 Schwingung eines Einfach-Federpendels bei verschiedener Federkonstante und verschiedener Masse. Neben dem Verlauf der Federposition geben die Punkte die Position der Feder zu jeder festen Sekunde an. Die theoretisch zu erwartenden Frequenzen lassen sich jeweils ablesen. Oben: $F = 2\pi/\sqrt{(2\pi)^2/1} = 1$. Mitte: $F = 2\pi/\sqrt{(3\pi)^2/1} = 2/3$. Unten: $F = 2\pi/\sqrt{(3\pi)^2/4} = 4/3$

In Abb. 5.5 zeigen wir das Schwingungsverhalten der Vierfach-Federpendel. Wir wählen unterschiedliche Startwerte. Links gilt

$$u_1(0) = u_2(0) = u_3(0) = 0, \quad u_4(0) = -0.1.$$

Rechts ist

$$u_1(0) = u_2(0) = u_3(0) = u_4(0) = -1.$$

Das Schwingungsverhalten ist sehr komplex und kaum noch vorherzusagen.

Im nächsten Beispiel bleiben wir bei $\kappa = (2\pi)^2$ für alle Federn. Von den vier Massen wählen wir drei gleich, jeweils die zweite aber entweder größer oder kleiner:

$$m_1 = m_3 = m_4 = 1, \quad m_2 = 10 \text{ oder } m_2 = 0.1.$$

Zu Beginn ist nur die zweite Feder ausgelenkt, es gilt also

$$u_1(0) = u_3(0) = u_4(0) = 0, \quad u_2(0) = -0.1.$$

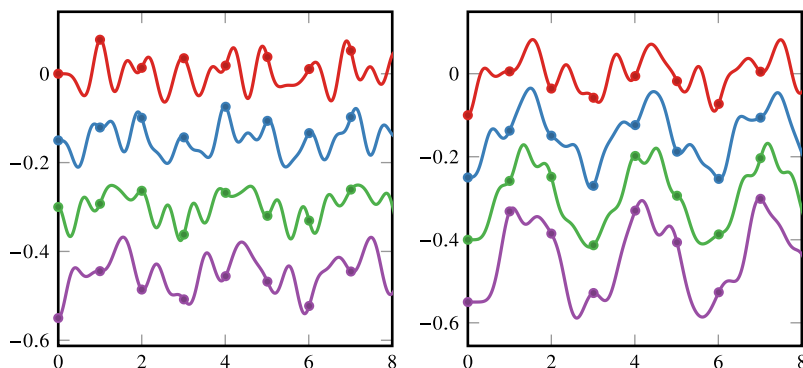


Abb. 5.5 Schwingung eines Vierfach-Federpendels. In beiden Abbildungen gilt $\kappa = (2\pi)^2$ und $m = 1$. Wir untersuchen zwei Startbedingungen: Links ist die unterste Masse um -0.1 ausgelenkt, rechts sind alle Massen zu Beginn um -0.1 ausgelenkt

In Abb. 5.6 zeigen wir das jeweilige Schwingungsverhalten. An diesem Beispiel sehen wir, dass die Eigenwerte der Matrix wesentlich von der Masse abhängen. Die Berechnung der Eigenwerte mit Potenzmethode bzw. mit der inversen Iteration gibt

$$\begin{aligned} m = 0.1 : \quad \lambda^2 &\in [5.63, 831.1], & F &\in [0.37, 5.59], \\ m = 10 : \quad \lambda^2 &\in [1.58, 88.8], & F &\in [0.2, 1.5]. \end{aligned} \quad (5.20)$$

In der rechten Abbildung tritt eine sehr hohe Frequenz auf.

Zum Abschluss wollen wir noch kurz die Anregung von Eigenschwingungen untersuchen. Hierzu betrachten wir weiter das Beispiel mit $\kappa = (2\pi)^2$, $m_1 = m_3 = m_4 = 1$ und $m_2 = 0.1$. Zu Beginn sei die Masse m_4 um $u_4(0) = -0.1$ ausgelenkt. Für die anderen

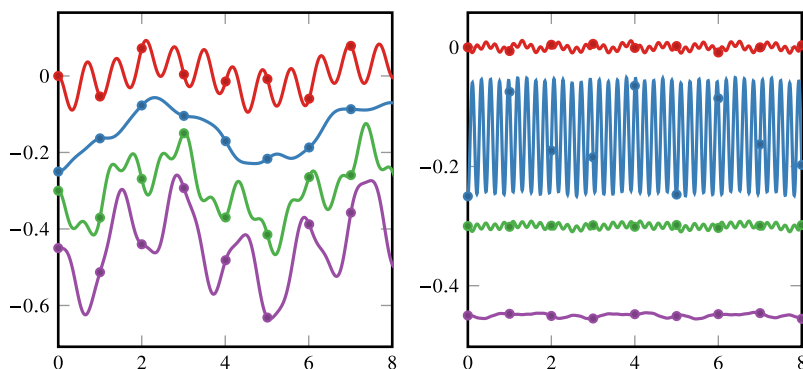


Abb. 5.6 Schwingung eines Vierfach-Federpendels. In beiden Abbildungen gilt $\kappa = (2\pi)^2$. Die zweite Feder ist zu Beginn ausgelenkt. Links gilt $m_2 = 10$, in der rechten Abbildung gilt $m_2 = 0.1$

Auslenkungen gelte $u_1(0) = u_2(0) = u_3(0) = 0$. Wir treiben die Dynamik nun durch eine Kraft und ändern die Differentialgleichung in

$$u''(t) + Au(t) = f(t), \quad u(0) = u^0, \quad u'(0) = 0.$$

Statt den Verlauf der Oszillationen betrachten wir nun die gesamte Energie des Systems, also den Ausdruck

$$E_n = \sum_{i=1}^N \frac{1}{2} m_i |u'_i(t_n)|^2 + \sum_{i=1}^N \frac{1}{2} \kappa |u_i(t_n) - u_i(t_{n-1})|^2,$$

bestehend aus kinetischer und potentieller Energie. Ganz oben in Abb. 5.7 zeigen wir den Verlauf der Energie für die konstante Kraft $f_1 = 0$. Hier wirkt keine externe Kraft ein und das Prinzip der Energieerhaltung besagt, dass $E_n = E_0$ für alle $n \geq 0$ gelten muss. Mit $\kappa = (2\pi)^2$ und den Anfangsauslenkungen $u_1(0) = u_2(0) = u_3(0) = 0$ sowie $u_4(0) = -0.1$ ergibt sich

$$E_0 = \frac{(2\pi)^2}{2} (-0.1)^2 \approx 0.197,$$

wie in der obersten Grafik von Abb. 5.7 abzulesen ist. Die zweite Abbildung gehört zur konstanten Kraft $f_2 = -1$. Hier ist die Energie nichtkonstant, sie scheint jedoch beschränkt zu

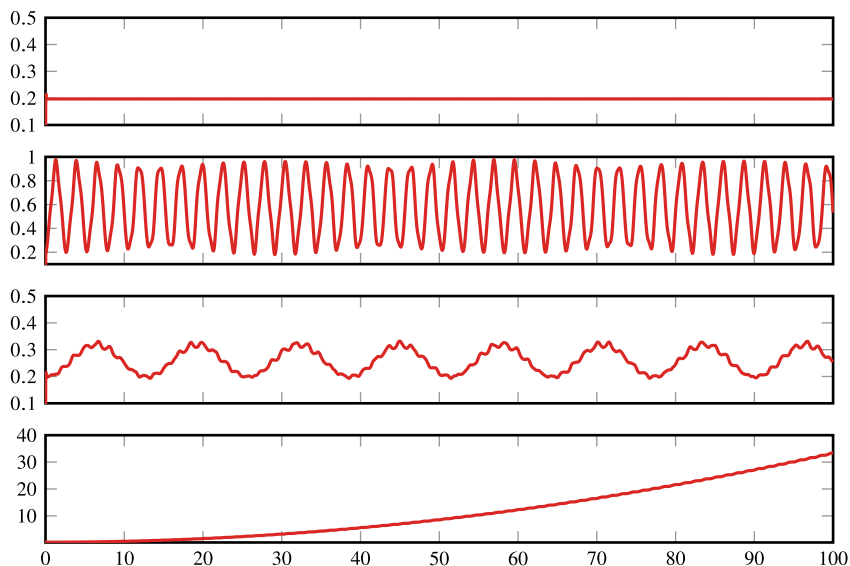


Abb. 5.7 Anregung der Mehrfach-Feder durch einer Kraft f . Von oben nach unten: $f_1 = 0$ und $f_2 = -0.1$ konstante Kräfte, $f_3(t) = 0.1 \cos(2\pi \cdot 0.3t)$ und $f_4(t) = 0.1 \cos(2\pi \cdot 0.3777t)$. Die Oszillation ist stabil, wenn nicht - wie bei $f_4(t)$ - genau eine Eigenfrequenz angeregt wird. Hier entsteht der Resonanzeffekt

sein. Dieser Fall entspricht der Einwirkung der Schwerkraft. Diese lenkt die Kugeln natürlich aus, aber es kommt nicht zu einer ständigen Verstärkung der Oszillationen. Die dritte Darstellung in Abb. 5.7 gehört zur oszillierenden Kraft $f_3(t) = \sin(2\pi \cdot 0.3t)$. Wieder ändert sich die Energie im System, bleibt jedoch beschränkt. Durch 0.3 wird eine Frequenz beschrieben, die außerhalb des Spektrums dieses Mehrfach-Federpendels liegt. Wir haben etwa 0.37 als kleinste und 5.59 als größte Frequenz gefunden, siehe 5.20. Es kommt nicht zum Resonanzeffekt. Schließlich wird durch die oszillierende Kraft $f_4(t) = \sin(2\pi \cdot 0.3777t)$ die unterste Grafik in Abb. 5.7 erzeugt. Diese Frequenz entspricht der kleinsten Eigenfrequenz des Systems. Hier wächst die Energie sehr schnell an.

Falls die anregende Kraft eine der Eigenfrequenzen des Systems trifft, so führt jede noch so kleine Kraft schließlich zu einer beliebig großen Auslenkung der Federn.

5.7 Singulärwertzerlegung

Eigenwerte sind für quadratische Matrizen, also für Abbildungen $f : V \rightarrow V$ von einem Vektorraum in sich selbst, definiert. In diesem Abschnitt werden wir dieses Konzept erweitern und führen für allgemeine rechteckige Matrizen $A \in \mathbb{R}^{n \times m}$ *Singulärwerte* ein, um die Eigenschaften der Matrix zu charakterisieren.

In Abschn. 4.5 haben wir überbestimmte lineare Gleichungssysteme $Ax = b$ mit $A \in \mathbb{R}^{n \times m}$ und $n > m$ behandelt. Zugang zu einem Lösungsbegriff war das Normalgleichungssystem $A^T Ax = A^T b$. Wir betrachten nun wieder die Matrix $A^T A \in \mathbb{R}^{m \times m}$, jedoch unabhängig davon, ob n oder m größer ist, oder ob die Matrix quadratisch ist. Die symmetrisch positiv semi-definite Matrix $A^T A$ besitzt ein System aus reellen und nichtnegativen Eigenwerten sowie eine Orthonormalbasis $\{v_1, \dots, v_m\}$ mit $v_i \in \mathbb{R}^m$ für $i = 1, \dots, m$ mit

$$A^T A v_i = \lambda_i v_i, \quad (v_i, v_j) = \delta_{ij}, \quad i, j = 1, \dots, m. \quad (5.21)$$

Ohne Einschränkung gehen wir davon aus, dass die Eigenwerte sortiert sind, d. h.

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0, \quad (5.22)$$

wobei r für $1 \leq r \leq m$ der Rang der Matrix $A^T A$ und auch der Matrix A ist, siehe Satz 2.18, sowie $\lambda_{r+1} = \dots = \lambda_m = 0$.

► **Definition 5.34 (Singuläre Werte)** Sei $A \in \mathbb{R}^{n \times m}$. Weiter seien

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0$$

die von Null verschiedenen Eigenwerte von $A^T A$. Dann heißen die Werte $\sigma_i = \sqrt{\lambda_i}$ für $1 \leq i \leq r$ die *Singulärwerte* der Matrix A . Im Gegensatz zu den Eigenwerten einer Matrix sind die Singulärwerte stets per Definition positiv.

Satz 5.35 (Singulärwerte) Es sei $A \in \mathbb{R}^{n \times m}$ mit Rang $r \leq \min\{n, m\}$. Dann existiert ein System aus r Singulärwerten $\sigma_1 \geq \dots \geq \sigma_r > 0$ und $\sigma_{r+1} = \dots = \sigma_m = 0$ im Fall $r < m$, sowie zwei Orthonormalsysteme $\{v_1, \dots, v_m\}$ mit $v_i \in \mathbb{R}^m$ sowie $\{u_1, \dots, u_r\}$ mit $u_i \in \mathbb{R}^n$, so dass gilt

$$A^T A v_i = \sigma_i^2 v_i \quad i = 1, \dots, m, \quad (5.23)$$

$$A v_i = \sigma_i u_i \quad i = 1, \dots, r, \quad (5.24)$$

$$A^T u_i = \sigma_i v_i \quad i = 1, \dots, r. \quad (5.25)$$

Die Vektoren $u_1, \dots, u_r$ spannen den Bildraum von A auf.

Beweis Mit den $r = \text{rang}(A)$ positiven Eigenwerten λ_i von $A^T A$ definieren wir die Singulärwerte $\sigma_i = \sqrt{\lambda_i}$, siehe (5.21)–(5.22). Da $A^T A$ symmetrisch positiv semi-definit ist existiert ein Orthonormalsystem aus Eigenvektoren $v_1, \dots, v_m$ mit welchen (5.23) gilt. Für $i = 1, \dots, r$ definieren wir

$$u_i = \frac{1}{\sigma_i} A v_i, \quad (5.26)$$

welche ein Orthonormalsystem bilden, denn

$$(u_i, u_j) = \frac{1}{\sigma_i \sigma_j} (A v_i, A v_j) = \frac{\sigma_i^2}{\sigma_i \sigma_j} (v_i, v_j) = \delta_{ij}.$$

Mit (5.26) ergibt sich direkt (5.24) und Multiplikation mit A^T liefert schließlich (5.25).

Der Rang r von $A^T A$ entspricht dem Rang von A , also spannen die $u_1, \dots, u_r$ den Bildraum von A auf. $\square$

Bemerkung 5.36 (Singulärwerte symmetrischer Matrizen) Für eine quadratische symmetrische Matrix $A \in \mathbb{R}^{n \times n}$ mit $A^T = A$ sind die Singulärwerte gerade die Beträge der Eigenwerte. $\blacklozenge$

Das in Satz 5.35 eingeführte Orthonormalsystem $\{u_1, \dots, u_r\}$ kann zu einer Orthonormalbasis des $\mathbb{R}^n$ erweitert werden mit Vektoren $u_{r+1}, \dots, u_n$. Diese Vektoren spannen den Kern von A^T auf, d.h. $A^T u_j = 0$ für $j = r+1, \dots, n$. Also gilt

$$(u_j, A v_i) = \begin{cases} \sigma_i & 1 \leq i = j \leq r \\ 0 & 1 \leq i \leq m, 1 \leq j \neq i \leq n, \end{cases}$$

oder, wenn wir die beiden orthogonalen Matrizen $V \in \mathbb{R}^{m \times m}$ und $U \in \mathbb{R}^{n \times n}$ einführen

$$U^T A V = \Sigma := \begin{pmatrix} \tilde{\Sigma} & 0 \\ 0 & 0 \end{pmatrix} \in \mathbb{R}^{n \times m}, \quad (5.27)$$

wobei $\tilde{\Sigma} \in \mathbb{R}^{r \times r}$ ist mit $\tilde{\Sigma} = \text{diag}(\sigma_1, \dots, \sigma_r)$.

► **Definition 5.37 (Singularwertzerlegung)** Es sei $A \in \mathbb{R}^{n \times m}$. Eine Zerlegung von A der Form

$$A = U \Sigma V^T$$

in die orthogonalen Matrizen $V \in \mathbb{R}^{m \times m}$ und $U \in \mathbb{R}^{n \times n}$ und $\Sigma \in \mathbb{R}^{n \times m}$ gemäß (5.27) heißt Singularwertzerlegung.

Satz 5.38 (Singularwertzerlegung) Zu jeder Matrix $A \in \mathbb{R}^{n \times m}$ existiert eine Singularwertzerlegung mit zwei orthonormalen Matrizen $V \in \mathbb{R}^{m \times m}$ und $U \in \mathbb{R}^{n \times n}$ sowie der Matrix $\Sigma \in \mathbb{R}^{n \times m}$ mit $\Sigma_{ii} = \sigma_i > 0$ für $i = 1, \dots, r$ und $\Sigma_{ij} = 0$ sonst, so dass

$$U^T A V = \Sigma.$$

Die singulären Werte $\sigma_1 \geq \dots \geq \sigma_r > 0$ sind zwar eindeutig, die Zerlegung selbst ist es im Allgemeinen jedoch nicht, da die Eigenräume von A , aber auch die Vektoren $u_{r+1}, \dots, u_n$ nicht eindeutig bestimmt sind.

Da die Matrizen U und V orthogonal zueinander sind, folgen die Darstellungen

$$A = U \Sigma V^T, \quad A^T = V^T \Sigma U$$

oder, in Summenschreibweise

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T, \quad A^T = \sum_{i=1}^r \sigma_i v_i u_i^T.$$

Die r singulären Werte, zusammen mit den Vektoren $v_1, \dots, v_r$ und $u_1, \dots, u_r$ charakterisieren also die gesamte Matrix A .

Wir schließen diesen Abschnitt mit drei recht einfachen Beispielen ab, die noch nicht effiziente Numerik zeigen, sondern lediglich ein Gefühl für die Singularwertzerlegung herstellen sollen.

Beispiel 5.39

Die Drehmatrix für eine Drehung um den Winkel α ist gegeben durch

$$A = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}$$

und erlaubt die Singularwertzerlegung

$$A = \underbrace{\begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}}_{=U} \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}}_{=\Sigma} \underbrace{\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}}_{=V^T}.$$



Beispiel 5.40

Wir bestimmen die Singulärwertzerlegung der singulären Matrix

$$A = \begin{pmatrix} 1 & 1 \\ 7 & 7 \end{pmatrix}.$$

Offensichtlich hat die Matrix den Rang 1, da die beiden Spalten linear abhängig sind. Dementsprechend gibt es einen Singulärwert. Wir bestimmen zuerst die Matrix $A^T A$ als

$$A^T A = \begin{pmatrix} 50 & 50 \\ 50 & 50 \end{pmatrix}.$$

Diese Matrix hat die Eigenwerte $\lambda_1 = 100$ und $\lambda_2 = 0$. Die entsprechenden Eigenvektoren sind

$$v_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad v_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}.$$

Somit können wir die Matrix V bestimmen:

$$V = [v_1, v_2] = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & -1 \\ 1 & 1 \end{pmatrix}$$

Der Singulärwert ist $\sigma_1 = \lambda(A^T A) = \sqrt{100} = 10$. Die Werte der Matrix U können nun wie oben gezeigt bestimmt werden:

$$u_1 = \frac{Av_1}{10} = \frac{(1, 7)}{\sqrt{50}},$$

$$u_2 = \frac{Av_2}{10} = \frac{(-7, 1)}{\sqrt{50}}.$$

Damit gilt

$$U = [u_1, u_2] = \frac{1}{\sqrt{50}} \begin{pmatrix} 1 & -7 \\ 7 & 1 \end{pmatrix}.$$

Offensichtlich gilt $U^T U = I$. Die finale Zerlegung ist dann gegeben durch

$$A = U \Sigma V^T = \frac{1}{\sqrt{50}} \begin{pmatrix} 1 & -7 \\ 7 & 1 \end{pmatrix} \begin{pmatrix} 10 & 0 \\ 0 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ -1 & 1 \end{pmatrix}.$$



Beispiel 5.41

Schließlich geben wir die Singulärwertzerlegung der Matrix von Beispiel 4.21 aus Abschn. 4.5 an. Hier liegt eine rechteckige Matrix $A \in \mathbb{R}^{4 \times 3}$ vor, was im Folgenden die spezielle Struktur der Matrix Σ verdeutlichen wird. Die Matrix A ist gegeben durch

$$A = \begin{pmatrix} 1 & -\frac{1}{4} & \frac{1}{16} \\ 1 & \frac{1}{2} & \frac{1}{4} \\ 1 & 2 & 4 \\ 1 & \frac{5}{2} & \frac{25}{4} \end{pmatrix}$$

Die Singulärwertzerlegung von A ist gegeben als:

$$A = U \Sigma V^T \approx \begin{pmatrix} -0.017 & 0.696 & 0.671 & -0.253 \\ -0.073 & 0.680 & -0.511 & 0.521 \\ -0.556 & 0.149 & -0.430 & -0.695 \\ -0.827 & -0.175 & 0.321 & 0.426 \end{pmatrix} \cdot \begin{pmatrix} 8.217 & 0 & 0 \\ 0 & 1.380 & 0 \\ 0 & 0 & 0.523 \\ 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} -0.179 & -0.391 & -0.903 \\ 0.979 & 0.0203 & -0.203 \\ 0.0978 & -0.920 & 0.379 \end{pmatrix}$$



5.7.1 Approximation der Singulärwertzerlegung

Die Singulärwertzerlegung kann prinzipiell mit Hilfe der Eigenwerte und Eigenvektoren der Matrix $A^T A$ erstellt werden. Dieses Verfahren ist jedoch numerisch instabil, schon da die Konditionszahl von $A^T A$ quadratisch in Bezug zur Konditionszahl von A steigen kann. Die Singulärwertzerlegung kann jedoch auch direkt auf Basis der Matrix A approximiert werden. Wir skizzieren einen Algorithmus, der 1970 von Golub und Reinsch [41] vorgestellt wurde. Inzwischen gibt es effizientere Varianten, aber das Grundprinzip bleibt erhalten, siehe z. B. [42, 85]. Eine neuere Zusammenfassung von [41] und Erweiterung wurde kürzlich in [77] vorgestellt.

Wir gehen für das Folgende ohne Einschränkung davon aus, dass $A \in \mathbb{R}^{n \times m}$ mit $n \geq m$ gegeben ist. Ansonsten betrachten wir die transponierte Matrix A^T , denn wir wissen, dass zu

$$A = U \Sigma V^T,$$

durch

$$A^T = V \Sigma U^T$$

eine Singulärwertzerlegung der transponierten Matrix gegeben ist. Die Einschränkung dient dazu, Fallunterscheidungen zu vermeiden. Die Methode läuft in zwei Schritten ab:

1. Zunächst wird die Matrix $A \in \mathbb{R}^{n \times m}$ durch orthogonale Transformationen auf obere *Bidiagonalgestalt* gebracht. D.h., mit zwei orthogonalen Matrizen $H_L \in \mathbb{R}^{n \times n}$ und $H_R \in \mathbb{R}^{m \times m}$ mit $H_L^T H_L = I \in \mathbb{R}^{n \times n}$ und $H_R^T H_R = I \in \mathbb{R}^{m \times m}$ zerlegen wir A gemäß

$$H_L A H_R = B \in \mathbb{R}^{n \times m}, \quad (5.28)$$

wobei $B_{ij} = 0$ für $j < i$ sowie $j > i + 1$.

2. Im zweiten Schritt wird die Singulärwertzerlegung $B = R_L \Sigma R_R^T$ der Matrix B approximiert. Dieser zweite Schritt kann im allgemeinen nicht exakt erfolgen, und stattdessen führen wir eine Iteration ein, die gegen die Singulärwertzerlegung konvergiert.

Schließlich ergibt sich die (approximative) Singulärwertzerlegung von A als

$$A = H_L^T R_L \Sigma R_R^T H_R^T.$$

5.7.1.1 Transformation auf Bidiagonalgestalt

Das Vorgehen zur Transformation einer Matrix $A \in \mathbb{R}^{n \times m}$ mit $n \geq m$ auf Bidiagonalgestalt mit Householder-Transformationen, siehe Abschn. 4.3, ist ähnlich zum Vorgehen bei der QR-Zerlegung, bzw. beim Erstellen der Hessenberg-Normalform. Wir multiplizieren die Matrix A von links mit Spiegelungsmatrizen $S_L^{(i)}$ mit dem Ziel, die Unterdiagonale der i -ten Spalte zu eliminieren. Hiervon sind m Schritte notwendig. Nach jedem Schritt multiplizieren wir die entstehende Matrix mit einer weiteren Spiegelungsmatrix $S_r^{(l)}$ von rechts, diesmal mit dem Ziel, die Einträge rechts neben dem ersten Nebendiagonalelement zu eliminieren. In diesem zweiten Schritt müssen wir, wie bei der Hessenberg-Normalform, einen verkürzten Spiegelungsvektor verwenden, so dass die ersten i Spalten der Matrix nicht geändert werden. Wir zeigen die Transformation $A^{(i)} \rightarrow A^{(i+1)}$, lassen dabei den Index (i) , der auf die Iteration hinweist, der Einfachheit weg:

$$\begin{pmatrix}
 b_{11} & b_{12} & 0 & \cdots & \cdots & \cdots & 0 \\
 0 & \ddots & \ddots & \ddots & & & \vdots \\
 \vdots & \ddots & b_{i-1,i-1} & b_{i-1,i} & 0 & \cdots & 0 \\
 \vdots & & 0 & a_{i,i} & a_{i,i+1} & \cdots & a_{i,m} \\
 \vdots & & 0 & a_{i+1,i} & a_{i+1,i+1} & \cdots & a_{i+1,m} \\
 \vdots & & \vdots & \vdots & \vdots & \ddots & \vdots \\
 0 & \cdots & 0 & a_{n,i} & a_{n,i+1} & \cdots & a_{n,m}
 \end{pmatrix} \quad (5.29)$$

$$S_L^{(i)} \cdot \downarrow$$

$$\begin{pmatrix}
 b_{11} & b_{12} & 0 & \cdots & \cdots & \cdots & 0 \\
 0 & \ddots & \ddots & \ddots & & & \vdots \\
 \vdots & \ddots & b_{i-1,i-1} & b_{i-1,i} & 0 & \cdots & 0 \\
 \vdots & & 0 & b_{i,i} & \tilde{a}_{i,i+1} & \cdots & \tilde{a}_{i,m} \\
 \vdots & & 0 & 0 & \tilde{a}_{i+1,i+1} & \cdots & \tilde{a}_{i+1,m} \\
 \vdots & & \vdots & \vdots & \vdots & \ddots & \vdots \\
 0 & \cdots & 0 & 0 & \tilde{a}_{n,i+1} & \cdots & \tilde{a}_{n,m}
 \end{pmatrix}$$

$$\downarrow \cdot S_R^{(i)}$$

$$\begin{pmatrix}
 b_{11} & b_{12} & 0 & \cdots & \cdots & \cdots & 0 \\
 0 & \ddots & \ddots & \ddots & & & \vdots \\
 \vdots & 0 & b_{i-1,i-1} & b_{i-1,i} & 0 & \cdots & 0 \\
 \vdots & & 0 & b_{i,i} & b_{i,i+1} & 0 & 0 \\
 \vdots & & 0 & 0 & \hat{a}_{i+1,i+1} & \cdots & \hat{a}_{i+1,m} \\
 \vdots & & \vdots & \vdots & \vdots & \ddots & \vdots \\
 0 & \cdots & 0 & 0 & \hat{a}_{n,i+1} & \cdots & \hat{a}_{n,m}
 \end{pmatrix}$$

Die Householder-Transformation von links ist gegeben als

$$S_L^{(i)} = I - 2v_L^{(i)}(v_L^{(i)})^T, \quad v_L^{(i)} = \frac{\tilde{a}_i + \text{sign}(\tilde{a}_i)\|\tilde{a}_i\|e_i}{\|\tilde{a}_i + \text{sign}(\tilde{a}_i)\|\tilde{a}_i\|e_i\|}$$

mit dem verkürzten Spaltenvektor

$$\tilde{a}_i = (\underbrace{0, \dots, 0}_{i-1}, \underbrace{a_{i,i}, \dots, a_{n,i}}_{n+1-i})^T.$$

Multiplikation mit $S_L^{(i)}$ lässt die ersten $i - 1$ Zeilen und Spalten unverändert. Im Anschluss wird mit einer Householder-Transformation von rechts multipliziert

$$S_R^{(i)} = I - 2v_R^{(i)}(v_R^{(i)})^T, \quad v_R^{(i)} = \frac{\hat{a}_i + \text{sign}(\hat{a}_i)\|\hat{a}_i\|e_{i+1}}{\|\hat{a}_i + \text{sign}(\hat{a}_i)\|\hat{a}_i\|e_{i+1}\|}.$$

Dabei ist $\hat{a}_i$ ein verkürzter Spaltenvektor, der sich aus den Zeilen-Nebendiagonalen der Zwischenmatrix $S_L^{(i)}A^{(i)}$ bildet, siehe (5.29)

$$\hat{a}_i = (\underbrace{0, \dots, 0}_i, \underbrace{\tilde{a}_{i,i+1}, \dots, \tilde{a}_{i,m}}_{m-i})^T.$$

Es sind m Multiplikationen von links notwendig (bzw. $m - 1$, falls die Matrix A quadratisch ist) und $m - 2$ Multiplikationen von rechts. Insgesamt ergibt sich

$$B = \underbrace{S_L^{(m)} \dots S_L^{(1)}}_{S_L} A \underbrace{S_R^{(1)} \dots S_R^{(m-2)}}_{S_R}.$$

Wir fassen zusammen:

Satz 5.42 (Bidiagonalisierung) Es sei $A \in \mathbb{R}^{n \times m}$ mit $n \geq m$. Dann existieren zwei orthogonale Matrizen $S_L \in \mathbb{R}^{n \times n}$ und $S_R \in \mathbb{R}^{m \times m}$ und eine obere rechte Bidiagonalmatrix $B \in \mathbb{R}^{n \times m}$, so dass gilt

$$B = S_L A S_R.$$

Die Zerlegung kann mit dem Aufwand $4nm^2 - \frac{4}{3}m^3$ bei Verwendung von Householder-Transformationen von links und rechts erstellt werden.

5.7.1.2 Singulärwertzerlegung einer Bidiagonalmatrix

Wir gehen nun davon aus, dass $B \in \mathbb{R}^{n \times m}$ eine Bidiagonalmatrix ist und suchen die Singulärwertzerlegung von B . Ohne Einschränkung nehmen wir an, dass $m = n$ ist, denn wenn zu $B \in \mathbb{R}^{m \times m}$ eine Singulärwertzerlegung $\Sigma = R_L B R_R$ gefunden ist, so können Σ und B einfach um weitere Nullzeilen und R_L um weitere Nullzeilen und -spalten ergänzt werden. Es ist also $B \in \mathbb{R}^{m \times m}$ der Form

$$B = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \dots & 0 \\ 0 & \alpha_2 & \beta_2 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ \vdots & & \ddots & \ddots & \beta_{n-1} \\ 0 & \dots & \dots & 0 & \alpha_n \end{pmatrix}. \quad (5.30)$$

Der Algorithmus nach Golub und Kahan [40] approximiert die Singulärwertzerlegung von B iterativ durch Multiplikation mit Givens-Rotationen von rechts und links

$$B^{(0)} = B, \quad l = 1, 2, \dots, m : \quad B^{(l)} = G_l^{(l)} B^{(l-1)} G_r^{(l)}. \quad (5.31)$$

Dabei sind die Matrizen $G_l^{(l)}$ und $G_r^{(l)}$ selbst Produkte von $m - 1$ elementaren Givensrotationen

$$G_l^{(l)} = G_l^{(l),m-1} G_l^{(l),m-2} \dots G_l^{(l),1}, \quad G_r^{(l)} = G_r^{(l),1} G_r^{(l),2} \dots G_r^{(l),m-1}. \quad (5.32)$$

Das Ziel der Iteration (5.31) ist es, die Matrix B sukzessive in einer Diagonalmatrix mit den Singulärwerten auf der Diagonale zu überführen.

Die erste Multiplikation erfolgt von rechts

$$B^{(l),1} = B^{(l-1)} G_r^{(l),1}$$

mit einer Givens-Rotation $G_r^{(l),1} = G(1, 2, \theta^{(l)})$. Wir verweisen auf Definition 4.18 zum Aufbau der Givens-Matrix $G(i, j, \theta)$. Der Winkel $\theta^{(l)}$ ist zunächst frei und wird später so gewählt, dass wir Konvergenz erreichen.

Bei dieser Multiplikation wird ein neuer nicht-Null Eintrag $B_{2,1}^{(l),1} \neq 0$ unterhalb der Diagonalen erzeugt und die zweite Multiplikation (von links)

$$B^{(l),1'} = G_l^{(l),1} B^{(l),1}$$

mit einer Givens-Rotation $G_l^{(l),1'} = G(1, 2, \theta^{(l),1'})$ dient nun dazu, diesen eben erzeugten Eintrag wieder zu eliminieren. Stattdessen wird ein neuer nicht-Null Eintrag $B_{1,3}^{(l),1'}$ erzeugt. So geht es weiter, bis die Matrix wieder die Ausgangsform hat. Der zu Beginn noch frei wählende Parameter $\theta^{(l)}$ kann so bestimmt werden, dass die letzte Nebendiagonale β_{n-1} möglichst schnell gegen Null konvergiert, alle weiteren Winkel hängen von diesem ersten ab. Sobald eine Nebendiagonale klein genug ist, z. B.

$$|B_{i-1,i}^{(l)}| \leq \epsilon_{\text{tol}} (|B_{i-1,i-1}^{(l)}| + |B_{i,i}^{(l)}|),$$

so wird der Wert auf Null gesetzt $B_{i-1,i}^{(l)} = 0$ und der Wert α_n ist der erste gefundene Singulärwert. Im Anschluss wird der Algorithmus mit der verbleibenden reduzierten Matrix $B_{1 \leq i, j < m}^{(l)}$ fortgeführt.

Nach l Schritten entsteht eine Approximation an die Singulärwertzerlegung von B

$$\Sigma^{(l)} = \underbrace{G_l^{(l)} G_l^{(l-1)} \dots G_l^{(1)}}_{=G_l} B \underbrace{G_r^{(1)} G_r^{(2)} \dots G_r^{(l)}}_{=G_r},$$

wobei $G_l^{(l)}$ und $G_r^{(l)}$ entsprechend (5.32) aufgebaut sind. Wir fassen zum Abschluss noch den Aufwand des Verfahrens zusammen.

Satz 5.43 (Aufwand einer Golub-Kahan Iteration) Es sei $B \in \mathbb{R}^{m \times m}$ eine obere rechte Bidiagonalmatrix. Jede Iteration des Golub-Kahan Algorithmus hat den Aufwand $24m + O(1)$. Werden zusätzlich zu den Singulärwerten auch die orthogonalen Matrizen G_l und G_r berechnet, so steigt der Aufwand auf $4mn + O(m^2)$.

Beweis Pro Iteration werden $m - 1$ Givens-Rotationen, jeweils von links und rechts, auf eine Matrix $B^{(l),i} \in \mathbb{R}^{m \times m}$ angewendet. Dabei ist die Matrix $B^{(l),i}$ eine Matrix mit maximal 3 Einträgen pro Zeile, d. h., eine Givens-Rotation benötigt 12 Operationen. Insgesamt werden für die gesamte Sequenz somit $24(m - 1)$ Operationen benötigt.

Werden zusätzlich die orthogonalen Matrizen $G_l \in \mathbb{R}^{n \times n}$ und $G_r \in \mathbb{R}^{m \times m}$ berechnet, so kommen $(m - 1)4n$ und $(m - 1)4m$ Operationen hinzu. $\square$

5.7.2 Anwendungen der Singulärwertzerlegung

Im Folgenden diskutieren wir beispielhaft einige Anwendungen der Singulärwertzerlegung.

5.7.2.1 Bild und Kern homogener linearer Abbildungen

Es sei $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ eine lineare Abbildung mit der Darstellungsmatrix $A \in \mathbb{R}^{n \times m}$. Wir suchen das Bild und den Kern von A , d. h. die Unterräume

$$\text{Kern}(A) = \text{span}\{x \in \mathbb{R}^m \mid Ax = 0\} \subset \mathbb{R}^m,$$

$$\text{Bild}(A) = \text{span}\{y = Ax \in \mathbb{R}^n \mid x \in \mathbb{R}^m\} \subset \mathbb{R}^n.$$

Durch $A = U\Sigma V^T$ sei die Singulärwertzerlegung von A gegeben mit Singulärwerten $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$ mit $r \leq m$. Die Zahl r , d. h., die Anzahl der von null verschiedenen Singulärwerte gibt den Rang der Matrix A an. Die Singulärwertzerlegung kann stabil erstellt werden [22], d. h., die relative Genauigkeit der ermittelten Singulärwerte hängt nur von der Kondition des Problems und der Rechengenauigkeit ab. Es gilt also für die numerische Approximation $\tilde{\sigma}_i$ zu σ_i

$$\frac{|\tilde{\sigma}_i - \sigma_i|}{|\sigma_i|} \leq \kappa \epsilon \quad \Rightarrow \quad |\tilde{\sigma}_i - \sigma_i| \leq \kappa \epsilon |\sigma_i|.$$

Auch Singulärwerte nahe bei Null können stabil ermittelt werden. Jedoch kann auch die Singulärwertzerlegung nicht entscheiden, ob ein singulärer Wert wirklich exakt Null ist, oder nur nahe bei null. Diese Frage ist jedoch bei der Bestimmung von Rang, Bild und Kern wesentlich. Numerisch wählen wir daher eine Toleranz, z. B. $\epsilon_{\text{Kern}} \sigma_1$ und bestimmen den Rang von A mittels

$$r := \arg \max_{1 \leq i \leq n} \{\sigma_i > \epsilon_{\text{Kern}} \sigma_1\}.$$

Die Dimension des Bildes ist dann r und $n - \text{rang}(A)$ ist die Dimension des Kerns. Wir modifizieren Σ zu $\tilde{\Sigma}$, so dass für alle Diagonalelemente $\tilde{\Sigma}_{ii} = 0$ gilt bei $i > \text{rang}(A)$. Das homogene lineare Gleichungssystem zur Bestimmung des Kerns lautet

$$U \tilde{\Sigma} V^T x = 0 \Rightarrow \tilde{\Sigma} y = 0, \quad x = Vy.$$

Für die Lösung gilt $y_1 = \dots = y_r = 0$ und die y_i mit $i = r + 1, \dots, n$ können frei gewählt werden. Der Kern von A ist dann aufgespannt durch

$$\text{Kern}(A) = \text{span}\{v_{\text{rang}(A)+1}, \dots, v_n\}, \quad (5.33)$$

wobei v_i die Spaltenvektoren der Matrix A sind. Gleichmaßen einfach kann das Bild von A bestimmt werden. Es ist

$$y = Ax = U \tilde{\Sigma} V^T x = U \tilde{\Sigma} z.$$

Da V regulär ist, können $x, z \in \mathbb{R}^m$ gleichbedeutend verwendet werden. Nur die ersten r Singulärwerte sind von Null verschieden, also gilt

$$\text{Bild}(A) = \text{span}\{u_1, \dots, u_r\},$$

wobei u_i die Spaltenvektoren von U sind.

5.7.2.2 Pseudoinverse

Die Singulärwertzerlegung einer Matrix kann verwendet werden, um den Begriff der Inversen einer Matrix zu verallgemeinern.

► **Definition 5.44 (Moore-Penrose Inverse)** Es sei $A \in \mathbb{R}^{n \times m}$. Die *Pseudoinverse* $A^+ \in \mathbb{R}^{m \times n}$ von A ist eine Matrix mit den Eigenschaften

$$\begin{aligned} (i) \quad & AA^+A = A \\ (ii) \quad & A^+AA^+ = A^+ \\ (iii) \quad & (AA^+)^T = AA^+ \\ (iv) \quad & (A^+A)^T = A^+A \end{aligned} \quad (5.34)$$

Die Inverse A^{-1} einer quadratischen regulären Matrix erfüllt alle diese Eigenschaften.

Satz 5.45 (Pseudoinverse) Es sei $A \in \mathbb{R}^{n \times m}$ mit Singulärwertzerlegung $A = U \Sigma V^T$ mit $U \in \mathbb{R}^{n \times n}$, $V \in \mathbb{R}^{m \times m}$, $\Sigma \in \mathbb{R}^{n \times m}$ und Singulärwerten $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > 0$. Wir definieren die Matrix $\Sigma^+ \in \mathbb{R}^{m \times n}$ mit

$$\Sigma^+ = \text{diag}(\sigma_1^{-1}, \dots, \sigma_r^{-1}, 0, \dots, 0).$$

Die Matrizen Σ^+ und $A^+ = V\Sigma^+U^T$ sind Moore-Penrose Inverse zu Σ sowie zu A .

Beweis Die Eigenschaften aus Definition 5.44 können alle elementar nachgerechnet werden. $\square$

Eine mögliche Anwendung der Pseudoinversen ist das Lösen von linearen Gleichungssystemen $Ax = b$ mit nicht-quadratischer Matrix $A \in \mathbb{R}^{n \times m}$, d. h. im Fall $n > m$ von überbestimmten und im Fall $n < m$ von unterbestimmten linearen Gleichungssystemen. Den Fall $n > m$ mit vollem Rang $\text{rang}(A) = m$ haben wir in Abschn. 4.5 kennengelernt. Da im allgemeinen Fall keine exakte Lösung $Ax = b$ zu erwarten ist, haben wir die Bestapproximation eingeführt

$$x \in \mathbb{R}^m : \|Ax - b\|_2 \leq \|Ay - b\|_2 \quad \forall y \in \mathbb{R}^m.$$

Die Lösung ist eindeutig bestimmt durch das Normalgleichungssystem

$$A^T A x = A^T b, \quad (5.35)$$

wenn $A^T A$ regulär ist, d. h., wenn A vollen Rang hat. Hier sehen wir den Bezug zur Singulärwertzerlegung, denn die Singulärwerte sind gerade die Wurzeln der Eigenwerte von $A^T A$ und es gilt mit $A = U\Sigma V^T$

$$A^T A = V\Sigma^2 V^T,$$

also lässt sich (5.35) schreiben als

$$V\Sigma^2 V^T x = V\Sigma U^T b \Leftrightarrow \Sigma V^T x = U^T b \Leftrightarrow x = V\tilde{\Sigma}^+ U^T b = A^+ b.$$

Mit dieser Vorbereitung kann das Verfahren auch Gleichungssysteme mit nicht vollem Rang erweitert werden.

► **Definition 5.46 (Pseudonormallösung)** Es sei $A \in \mathbb{R}^{n \times m}$ und $b \in \mathbb{R}^n$. Weiter sei $\bar{x} \in \mathbb{R}^m$ eine Bestapproximation in der euklidischen Norm, also

$$\|A\bar{x} - b\|_2 \leq \|Ay - b\|_2 \quad \forall y \in \mathbb{R}^m.$$

Dann heißt der Vektor $x^+ \in \mathbb{R}^m$ gemäß

- (i) $\|Ax^+ - b\|_2 \leq \|Ay - b\|_2 \quad \forall y \in \mathbb{R}^m$
- (ii) $\|x^+\|_2 \leq \|x\|_2 \quad \forall x \in \{\bar{x} + \text{Kern}(A)\},$

die *Pseudonormallösung* zum linearen Gleichungssystem $Ax = b$.

Die Bestapproximation $\bar{x} \in \mathbb{R}^m$ hängt noch von der rechten Seite b ab. Erst die zweite Bedingung $\|x^+\|_2 \leq \|x\|_2$ führt zu einer eindeutigen Lösung und somit ist unter allen mög-

lichen Bestapproximationen die Pseudonormallösung diejenige mit der kleinsten Euklidischen Norm.

Satz 5.47 (Pseudonormallösung) Es sei $A \in \mathbb{R}^{n \times m}$ und $b \in \mathbb{R}^n$. Die Pseudonormallösung zu $Ax = b$ ist durch

$$x^+ = A^+b$$

eindeutig bestimmt.

Beweis Durch x^+ ist eine Lösung des Normalgleichungssystems und somit eine Bestapproximation gegeben:

$$A^T Ax^+ = (V \Sigma^2 V^T)(V \Sigma^+ U^T)b = V \Sigma U^T b = A^T b$$

Den Kern der Matrix A schreiben wir mit (5.33) als

$$\text{Kern}(A) = \text{span}\{v_{r+1}, \dots, v_n\},$$

wobei $p = \text{Rang}(A)$ ist. Für $z \in \text{Kern}(A)$ beliebig gilt

$$\|x^+ + z\|_2^2 = \|x^+\|_2^2 + 2(x^+, z) + \|z\|_2^2.$$

Weiter ist mit $z = \sum_{i=r+1}^n \alpha_i v_i$

$$\begin{aligned} (x^+, z) &= \sum_{i=r+1}^n \alpha_i (V \Sigma^+ U^T b, v_i) = \sum_{i=r+1}^n \alpha_i (\Sigma^+ U^T b, V^T v_i) \\ &= \sum_{i=r+1}^n \alpha_i (\Sigma^+ U^T b, e_i) = 0, \end{aligned}$$

da alle $\sigma_i = 0$ sind für $i > p$. Also gilt die orthogonale Zerlegung

$$\|x^+ + z\|_2^2 = \|x^+\|_2^2 + \|z\|_2^2 \quad \forall z \in \text{Kern}(A)$$

was zeigt, dass x^+ in der Tat die eindeutige Pseudonormallösung ist. □

Bemerkung 5.48 Im Fall einer quadratischen Matrix $A \in \mathbb{R}^{n \times n}$ und im Fall einer eindeutigen Lösung $\bar{x} = A^{-1}b$ stimmt die Pseudonormallösung x^+ mit der klassischen Lösung $\bar{x}$ überein. Damit stellt die Pseudonormallösung einen verallgemeinerten Lösungsbegriff bereit. ◆

Bei der Definition der Pseudonormallösung sowie bei Satz 5.47 hat es keine Rolle gespielt, ob $n = m$, $n < m$ oder $n > m$ gilt. Wir haben den Lösungsbegriff von linearen Glei-

chungssystemen auf den allgemeinen Fall erweitert, der unterbestimmte und überbestimmte Gleichungen beinhaltet.

Beispiel 5.49 (Berechnung einer Pseudoinversen A^+)

Aus Beispiel 5.41 kennen wir A , U , Σ und V^T . Wir nutzen

$$B := V \tilde{\Sigma} U^T, \quad \tilde{\Sigma} \in \mathbb{R}^{m \times n}, \quad \tilde{\Sigma}_{ij} := \tau_i \cdot \delta_{ik}, \quad (5.36)$$

mit dem Vektor $\tau \in \mathbb{R}^n$, gegeben als

$$\tau_i := \begin{cases} \sigma_i^{-1} & \sigma_i \neq 0, \\ 0 & \sigma_i = 0. \end{cases} \quad (5.37)$$

um die Pseudoinverse zu $A \in \mathbb{R}^{4 \times 3}$ zu berechnen: $A^+ = V \tilde{\Sigma} U^T$. Zunächst lautet die Matrix $\tilde{\Sigma} \in \mathbb{R}^{3 \times 4}$ mithilfe von (5.36) und (5.37)

$$\tilde{\Sigma} = \begin{pmatrix} 0.121705 & 0 & 0 & 0 \\ 0 & 0.724587 & 0 & 0 \\ 0 & 0 & 1.91011 & 0 \end{pmatrix}.$$

Dann gilt insbesondere die Eigenschaft $\Sigma \cdot \tilde{\Sigma} \in \mathbb{R}^{4 \times 4}$:

$$\Sigma \cdot \tilde{\Sigma} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Für die Pseudoinverse $A^+ \in \mathbb{R}^{3 \times 4}$ erhalten wir nun mithilfe der bereits in Beispiel 5.41 berechneten orthogonalen Matrizen U und V^T und (5.36)

$$A^+ = V \tilde{\Sigma} U^T \approx \begin{pmatrix} 0.620 & 0.388 & 0.0378 & -0.0459 \\ -1.169 & 0.911 & 0.785 & -0.527 \\ 0.386 & -0.462 & -0.273 & 0.349 \end{pmatrix}.$$

Die Probe $A^+ A \in \mathbb{R}^{3 \times 3}$ liefert

$$A^+ A \approx \begin{pmatrix} 0.620 & 0.388 & 0.0378 & -0.0459 \\ -1.169 & 0.911 & 0.785 & -0.527 \\ 0.386 & -0.462 & -0.273 & 0.349 \end{pmatrix} \cdot \begin{pmatrix} 1 & -\frac{1}{4} & \frac{1}{16} \\ 1 & \frac{1}{2} & \frac{1}{4} \\ 1 & 2 & 4 \\ 1 & \frac{5}{2} & \frac{25}{4} \end{pmatrix} \approx \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

Wir bemerken zum Schluss, dass in diesem Beispiel alle Rechnungen mit exakter Arithmetik durchgeführt wurden. Lediglich für die übersichtlichere Darstellung wurde auf drei Stellen gerundet. ◀

5.7.2.3 Niedrig-Rang Approximation

Die Singulärwertzerlegung kann, wie die Analyse der Eigenwerte und Eigenvektoren einer quadratischen Matrix, zur Charakterisierung der Matrix, bzw. der linearen Abbildung $f : x \mapsto Ax$ verwendet werden. Zunächst gilt:

Korollar 5.50 (Singulärwertzerlegung und Matrixnorm) Es sei $A \in \mathbb{R}^{n \times m}$ mit Rang r und Singulärwertzerlegung V, U, Σ . Dann gelten

$$\|A\|_2 = \sigma_1, \quad \|A\|_F = \sqrt{\sum_{i=1}^r \sigma_i^2}.$$

Beweis Die Singulärwerte σ_i (sowie evtl. weitere Nullen) sind gerade die Quadrate der Eigenwerte von $A^T A$. Daher folgt das Ergebnis für die Spektralnorm. Für die Frobenius-Norm gilt

$$\|A\|_F^2 = \text{spur}(A^T A) = \sum_{i=1}^r \sigma_i^2.$$

□

Da die Vektoren u_i, v_i normiert sind, können wir der Sequenz

$$(\sigma_1, v_1, u_1), (\sigma_2, v_2, u_2), \dots$$

eine Rolle in der Bewertung ihrer Wichtigkeit zur Charakterisierung der Matrix A zuschreiben. Wir definieren

► **Definition 5.51 (Niedrig-Rang Approximation)** Es sei $A \in \mathbb{R}^{n \times m}$ eine Matrix mit Rang r und Singulärwertzerlegung V, U, Σ . Dann heißt die Matrix $A_l \in \mathbb{R}^{n \times m}$ für $l \leq r$ gemäß

$$A_l := \sum_{i=1}^l \sigma_i u_i v_i^T$$

die *Rang- l -Approximation* der Matrix A . Mit der Matrix

$$\Sigma_l = \begin{pmatrix} \tilde{\Sigma}_l & 0 \\ 0 & 0 \end{pmatrix} \in \mathbb{R}^{n \times m}, \quad \tilde{\Sigma}_l := \text{diag}(\sigma_1, \dots, \sigma_l, 0, \dots, 0) \in \mathbb{R}^{r \times r}$$

schreiben wir kurz

$$A_l = U \Sigma_l V^T.$$

Die Rang- l -Approximation ist in gewisser Hinsicht eine Bestapproximation an die Matrix A im Vektorraum aller Matrizen mit maximalen Rang l . Das *Eckart-Young-Mirsky Theorem* zeigt diesen Zusammenhang in allgemeinen Matrixnormen und auch im unendlich-dimensionalen Fall. Wir geben eine einfache Variante:

Satz 5.52 (Rang- l -Approximation) Es sei $A \in \mathbb{R}^{n \times m}$ mit Rang r . Sei $l < r$. Dann ist die Rang- l -Approximation $A_l \in \mathbb{R}^{n \times m}$ gemäß Definition 5.51 die Bestapproximation zu A im Vektorraum aller Matrizen bis zu Rang l , d. h.

$$\|A - A_l\|_2 \leq \min_{B_l \in \mathbb{R}^{n \times m}, \text{rang}(B_l) \leq l} \|A - B_l\|_2.$$

Es gilt

$$\|A - A_l\|_2 = \sigma_{l+1}.$$

Beweis (i) Es sei V, U, Σ die Singulärwertzerlegung der Matrix A und $r = \text{rang}(A)$. Es gilt

$$A - A_l = \sum_{i=l+1}^r \sigma_i u_i v_i^T \quad (5.38)$$

und die Fehlerdarstellung

$$\|A - A_l\|_2 = \sigma_{l+1},$$

folgt mit Korollar 5.50.

(ii) Nun sei $B_l \in \mathbb{R}^{n \times m}$ eine beliebige Matrix mit Rang $l < r$. Der Kern von B_l hat die Dimension $\text{Kern}(B_l) = m - l$ und wir bezeichnen mit $\{x_1, \dots, x_{m-l}\}$ eine Orthonormalbasis des Kerns. Das bedeutet, dass der Schnitt

$$\{x_1, \dots, x_{m-l}\} \cap \{v_1, \dots, v_{l+1}\} \neq \emptyset$$

nicht trivial ist. Wir wählen nun einen nichttrivialen Vektor $z \in \mathbb{R}^m$ mit $\|z\|_2 = 1$ aus dieser Schnittmenge. Für ihn gilt $B_l z = 0$ und also

$$(A - B_l)z = Az = \sum_{i=1}^{l+1} \sigma_i u_i v_i^T z$$

und folglich

$$\begin{aligned}\|A - B_l\|_2 &= \max_{w \in \mathbb{R}^m} \frac{\|(A - B_l)w\|_2}{\|w\|_2} \geq \|(A - B_l)z\|_2 = \|Az\|_2 \\ &= \sqrt{\sum_{i=1}^{l+1} \sigma_i^2 (u_i v_i^T z)^2} = \sqrt{\sum_{i=1}^{l+1} \sigma_i^2} \\ &\geq \sigma_{l+1} = \|A - A_l\|_2,\end{aligned}$$

da für die Koeffizienten $\alpha_i = u_i v_i^T z$ aufgrund der Orthonormalitäten von u_i und v_i^T sowie der Normierung von z somit $\sum_{i=1}^{l+1} \alpha_i^2 = 1$ gilt. $\square$

5.8 Exkurs: Singulärwertzerlegung zur Bildkompression

Wir wollen die Singulärwertzerlegung einer Matrix als eine einfache Methode zur Kompression von Bildern einsetzen. Die Grundlage hierfür ist die *Niedrigrang-Darstellung* einer Matrix auf Basis der Singulärwertzerlegung. Hierzu betrachten wir das Foto aus Abb. 5.8. Das Bild hat die Auflösung 2000×1000 und wir speichern die drei Farbkanäle *rot*, *grün* und *blau* in drei separaten Matrizen

$$A_{\text{rot}}, A_{\text{grün}}, A_{\text{blau}} \in [0, 1]^{2000 \times 1000}.$$

Zu den Farbkanälen rot, grün und blau erstellen wir jeweils eine Singulärwertzerlegung der Matrix. Rechts in Abb. 5.8 stellen wir den Verlauf der Singulärwerte (logarithmisch) dar. Man erkennt, dass die Werte sehr schnell abfallen und dies bietet Potential zu einer reduzierten Darstellung mit der Singulärwertzerlegung.

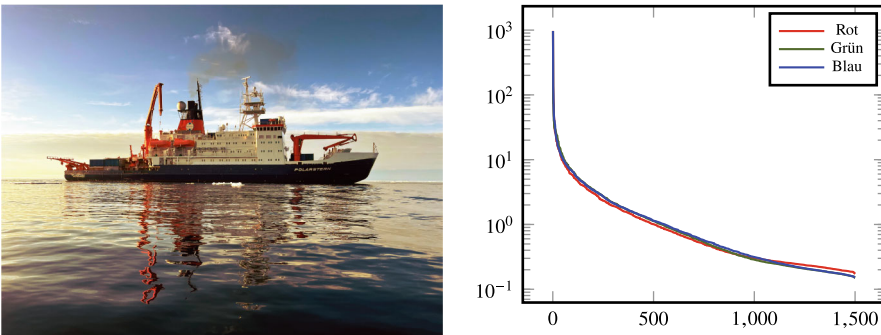


Abb. 5.8 Links: Foto des Forschungseisbrechers Polarstern. Rechts: Singulärwerte der Matrizen $A_{\text{rot}}, A_{\text{grün}}, A_{\text{blau}}$, welche die drei Farbkanäle des Bildes darstellen. Jeder Eintrag $A_{\text{rot},ij} \in [0, 1]$ entspricht dem Farbwert des Pixels (i, j)

Gerade die ersten Singulärwerte fallen sehr schnell ab. An der Abbildung rechts erkennt man, dass die ersten 50 Singulärwerte bereits 99% der Informationen beinhalten, da die Werte von 10^3 auf 10 abfallen. Zur Reduktion der Information verkürzen wir die Darstellung und führen die Schreibweise

$$A_q := U \Sigma_q V_q^T \quad (5.39)$$

ein, wobei

$$\Sigma_q := \text{diag}(\sigma_1, \dots, \sigma_q, 0, \dots, 0)$$

ist, d. h., die Diagonalmatrix der ersten q Singulärwerte. Die reduzierte Darstellung A_q gemäß (5.39) kann geschrieben werden als

$$A_q = \sum_{i=1}^q \sigma_i u_i v_i^T,$$

und benötigt anstelle von nm Pixeln nur die ersten q Zeilen von V und Spalten von U , also $q(m+n)$ Pixel. In Abb. 5.9 zeigen wir die reduzierte Darstellung für verschiedene Werte von q sowie einen kleinen Ausschnitt der den Schriftzug „Polarstern“ beinhaltet. Die Qualität der Approximation kann mithilfe von

$$\varepsilon(q) = \frac{\sum_{i=1}^q \sigma_i^2}{\sum_{i=1}^r \sigma_i^2}$$

gemessen werden.

Ein wesentliches Merkmal der Singulärwertzerlegung ist, dass gerade Strukturen mit scharfer Umrandung, z. B. der Schriftzug „Polarstern“ schon mit dem ersten Singulärwert dargestellt werden. Gespeichert werden gerade einmal $3(n+m) = 9000$ anstelle von $3nm = 6\,000\,000$ Pixelinformationen.

In der Praxis würde man die Singulärwertzerlegung so nicht zur Bildkompression einsetzen. Die kubische Skalierung würde bei großen Bildern zu sehr langen Laufzeiten führen. Abhilfe schafft hier eine Rasterung des Bildes. Dieses kann zunächst in Blöcke, z. B. der Größe 16×16 zerlegt werden und dann wird von jedem Block eine Singulärwertzerlegung erstellt. Auf diese Weise fällt die schlechte Skalierung nicht weiter ins Spiel. Weiter beziehen Bildkompressionsmethoden wie JPEG, siehe hierzu den entsprechenden Exkurs in Abschn. 9.8, physiologische Faktoren mit in die Analyse ein. Die Informationen werden nicht auf rein mathematischer Basis reduziert. Stattdessen wird die menschliche Wahrnehmung berücksichtigt und die Daten werden entsprechend gewichtet.

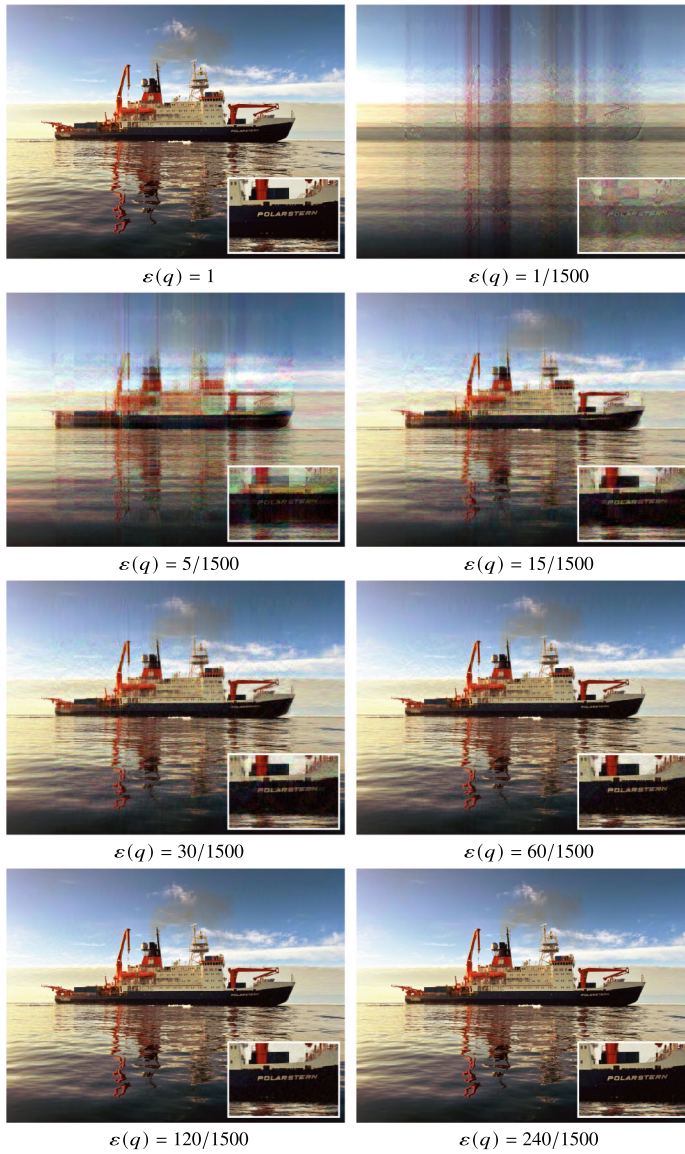


Abb. 5.9 Bildkompression mit der Singulärwertzerlegung. Das Originalbild (Ausschnitt oben rechts) wird als Matrix dargestellt und von den drei Kanälen rot, grün, blau, werden Singulärwertzerlegungen erstellt. Die Rekonstruktionen basieren jeweils nur auf einem Anteil der Singulärwerte und zugehörigen orthogonalen Vektoren

Krylowraumverfahren für lineare Gleichungssysteme und das Eigenwertproblem

6

In diesem Kapitel erweitern wir die bereits behandelten Verfahren zur Lösung von $Ax = b$. Zur Erinnerung sei bemerkt, dass wir in Kap. 3 direkte Verfahren (also LR, Cholesky sowie QR-Zerlegung) als auch iterative Verfahren (also Richardson, Jacobi, Gauß-Seidel) behandelt haben. Ein wesentlicher Grund für die iterativen Verfahren sind der geringere Speicheraufwand und die im Allgemeinen geringeren Kostenkomplexitäten. Die in Kap. 3 kennengelernten Zerlegungsverfahren (LR, Cholesky sowie QR-Zerlegung) haben alle eine kubische Laufzeit $O(n^3)$. Bei Bandmatrizen mit Bandbreite m reduziert sich der Aufwand zu $O(nm^2)$, dünn besetzte Matrizen können jedoch nicht einfach ausgenutzt werden. Gerade bei sehr großen Problemen wächst der Aufwand so schnell an, dass die Lösung in sinnvoller Zeit nicht zu erreichen ist, siehe Tab. 3.1. Neben der Laufzeit spielt der Speicheraufwand eine zentrale Rolle. Zur Speicherung einer voll besetzten Matrix $A \in \mathbb{R}^{n \times n}$ mit $n = 1\,000\,000$ sind bei doppelter Genauigkeit etwa 7 Terabyte Speicher notwendig. Dies ist nur auf größten Parallelrechnern möglich. Die linearen Gleichungssysteme, die aus den meisten Anwendungen resultieren (etwa bei der Diskretisierung von Differentialgleichungen), sind sehr dünn besetzt, d. h., in jeder Zeile stehen nur einige wenige Einträge. Eine dünn besetzte Matrix mit $n = 1\,000\,000$, aber nur 100 Einträgen pro Zeile benötigt zur Speicherung nur etwa 750 MB und passt auf jeden Laptop. In Abschn. 3.4 haben wir Sortierverfahren kennengelernt, um dieses dünne Besetzungsmuster auch für eine LR-Zerlegung nutzbar zu machen. Im Allgemeinen können die Matrizen L und R aber voll besetzt sein und somit den zur Verfügung stehenden Speicher wieder bei Weitem übersteigen.

Ein weiterer Nachteil der Zerlegungsverfahren sind numerische Stabilitätsprobleme. Durch Rundungsfehler beim Zerlegungsprozess gilt für die LR-Zerlegung üblicherweise nur $A \neq \tilde{L}\tilde{R}$. Das heißt, obwohl die LR-Zerlegung eine direkte Methode darstellt, kann das Gleichungssystem nicht exakt gelöst werden.

Die in Abschn. 3.7 diskutierten Fixpunktmethoden der Form

$$x^{k+1} = x^k + C(b - Ax^k)$$

mit einer Matrix $C \approx A^{-1}$ beheben dieses zweite Problem. Falls der Spektralradius der Matrix $I - CA$ klein ist, im Fall $\rho = \text{spr}(I - CA) < 1$, so konvergieren die Methoden robust. Die Konvergenz ist allerdings meist sehr langsam.

In diesem Kapitel werden wir immer davon ausgehen, dass die Matrix $A \in \mathbb{R}^{n \times n}$ dünn besetzt ist, nur $O(1)$ Einträge pro Zeile hat und dass eine Matrix-Vektor Multiplikation somit in $O(n)$ Operationen durchzuführen ist. Wir werden vollständig darauf verzichten, Matrixeinträge zu modifizieren, da dies zu *fill-ins* führen könnte, die den Aufwand wieder in die Höhe treiben; siehe Abschn. 3.4. Stattdessen werden die im Folgenden vorgestellten Algorithmen ausschließlich auf Matrix-Vektor-Multiplikationen basieren.

Die resultierenden Verfahren sind keine direkten Methoden und die Lösung eines linearen Gleichungssystems wird nicht exakt berechnet sondern nur approximiert.

6.1 Galerkin-Gleichung und Krylow-Teilräume

Wir betrachten ein lineares Gleichungssystem $Ax = b$ mit einer Matrix $A \in \mathbb{R}^{n \times n}$ und der rechten Seite $b \in \mathbb{R}^n$. Wenn $x \in \mathbb{R}^n$ das lineare Gleichungssystem $Ax = b$ löst, so verschwindet der Defekt

$$d(x) = b - Ax \stackrel{!}{=} 0.$$

Anders ausgedrückt gilt für die Lösung $x \in \mathbb{R}^n$ insbesondere

$$(b - Ax, y) = 0 \quad \forall y \in \mathbb{R}^n. \quad (6.1)$$

Diese Gleichung wird *variationelle Gleichung* genannt. Umgekehrt gilt

Satz 6.1 (Lineares Gleichungssystem als Variationsproblem) Sei $A \in \mathbb{R}^{n \times n}$ regulär, $b \in \mathbb{R}^n$. Die Lösung des linearen Gleichungssystems $Ax = b$ ist eindeutig als Lösung des Variationsproblems

$$\text{Finde } x \in \mathbb{R}^n : (b - Ax, y) = 0 \quad \forall y \in \mathbb{R}^n$$

bestimmt.

Beweis Ist $Ax = b$ die (eindeutige) Lösung, so gilt $b - Ax = 0$ und also auch die Variationsgleichung für alle $y \in \mathbb{R}^n$. Ist umgekehrt diese Gleichung für alle $y \in \mathbb{R}^n$ erfüllt, so auch für $y = b - Ax$, also gilt

$$0 = (b - Ax, b - Ax) = \|b - Ax\|^2 = 0.$$

Da $\|\cdot\|$ eine Norm ist, gilt $b - Ax = 0$ und somit $Ax = b$. □

An dieser Stelle setzen die *Krylow-Teilraumverfahren* an. Wir erstellen eine Approximation an die Lösung $x \in \mathbb{R}^n$, indem wir das Variationsproblem auf einen Teilraum beschränken. Wir suchen $x^k \in X_k \subset \mathbb{R}^n$, so dass

$$(b - Ax^k, \tilde{y}) = 0 \quad \forall \tilde{y} \in Y_k, \quad (6.2)$$

mit einem zweiten Teilraum $Y_k \subset \mathbb{R}^n$. Die Einschränkung einer variationellen Form auf einen Teilraum wird *Galerkin-Gleichung* genannt. Der Raum X_k in dem die Approximation gesucht wird heißt *Ansatzraum* und der Raum Y_k wird *Testraum* genannt. Die Beziehung (6.2) wird auch *Galerkin-Orthogonalität* genannt, da Gl. (6.2) besagt, dass der Defekt $b - Ax^k$ orthogonal auf dem Unterraum $Y_k \subset \mathbb{R}^n$ steht. Diese Verfahrensklasse wird auch *Projektionsverfahren* genannt.

Durch $\{x_1, \dots, x_k\} \subset X_k$ und $\{y_1, \dots, y_k\} \subset Y_k$ seien Basen von X_k und Y_k mit Dimension $\dim(X_k) = \dim(Y_k) = k$ gegeben. Wir schreiben die gesuchte Lösung zu (6.2) dann als $x^k = \sum_{i=1}^k \alpha_i x_i$ und die Galerkin-Gl. (6.2) ist äquivalent zu

$$\alpha_1, \dots, \alpha_k \in \mathbb{R} : \left(b - A \sum_{i=1}^k \alpha_i x_i, y_j \right) = 0, \quad j = 1, \dots, k,$$

was wir wie folgt umformulieren

$$\alpha_1, \dots, \alpha_k \in \mathbb{R} : \sum_{i=1}^k (Ax_i, y_j) \alpha_i = (b, y_j), \quad j = 1, \dots, k.$$

Dies ist äquivalent zum kleineren linearen Gleichungssystem $\tilde{A}\alpha = \tilde{b}$, wobei $\alpha = (\alpha_1, \dots, \alpha_k)$ und $\tilde{A} \in \mathbb{R}^{k \times k}$ mit $\tilde{A}_{ji} = (Ax_i, y_j)$ und $\tilde{b} \in \mathbb{R}^k$ mit $\tilde{b}_j = (b, y_j)$ gilt. Bedingungen für die Lösbarkeit des reduzierten Gleichungssystems und Approximationseigenschaften für allgemeine Projektionsverfahren werden z. B. in [78, Kap. 5] besprochen.

Wir beschränken uns hier auf die *Krylow-Teilraumverfahren*, die sich durch eine spezielle Wahl der Unterräume X_k und Y_k ergeben. Wir definieren:

► **Definition 6.2 (Krylow-Teilraum)** Es sei durch $Ax = b$ ein lineares Gleichungssystem mit Matrix $A \in \mathbb{R}^{n \times n}$ und rechter Seite $b \in \mathbb{R}^n$ gegeben. Weiter sei $x^0 \in \mathbb{R}^n$ beliebig gegeben und $d^0 := b - Ax^0$. Dann heißen die durch

$$K_k := K_k(d^0, A) := \underbrace{\text{span}\{d^0, Ad^0, \dots, A^{k-1}d^0\}}_{k \text{ Vektoren}}$$

aufgespannten Teilräume $K_k(d^0, A) \subset \mathbb{R}^n$ *Krylow-Unterräume*.

Wir können uns $x^0 \in \mathbb{R}^n$ als eine initiale Schätzung für die Lösung vorstellen. Die Approximation in dem Teilraum wird dann gesucht als $x^k \in x^0 + \hat{x}$ mit

$$\hat{x} \in K_k(d^0, A) : (b - A(x^0 + \hat{x}), y) = 0 \quad \forall y \in Y_k.$$

Ist keine Schätzung bekannt, so können wir einfach $x^0 = 0$ wählen.

Die Krylow-Unterräume sind durch den Startwert x^0 und die Matrix A eindeutig bestimmt. Es gilt $K_k(d^0, A) \subset K_{k+1}(d^0, A)$, da für größere k jeweils Vektoren hinzukommen.

6.2 Das CG-Verfahren für symmetrisch positiv definite Gleichungssysteme

Das *Verfahren der konjugierten Gradienten* (englisch *Conjugate Gradients*), kurz das CG-Verfahren, ist ein einfaches und sehr effizientes Krylow-Teilraumverfahren zum Lösen von linearen Gleichungssystemen $Ax = b$ mit symmetrisch positiv definiter Matrix $A \in \mathbb{R}^{n \times n}$. Als Ansatzraum und als Testraum wählen wir den Krylow-Unterraum $K_k(d^0, A)$. Die Galerkin-Gleichung in Schritt k lautet

$$x^k \in x^0 + K_k(d^0, A) : (b - Ax^k, y) = 0 \quad \forall y \in K_k(d^0, A). \quad (6.3)$$

Wir haben diskutiert, dass die Krylow-Räume für steigendes k eine Sequenz von Teilräumen $K_{k-1}(d^0, A) \subset K_k(d^0, A) \subset K_{k+1}(d^0, A)$ bilden. Es ist aber nicht gesagt, dass $K_{k+1}(d^0, A)$ wirklich echt größer ist als $K_k(d^0, A)$. Angenommen d^0 wäre ein Eigenvektor von A , dann ist $K_k(d^0, A) = \text{span}\{d^0\}$ für alle k . Hiermit stellt sich die Frage, ob diese Konstruktion überhaupt geeignet ist, um die Lösung des linearen Gleichungssystems zu bestimmen. Der folgende Satz gibt uns eine positive Antwort:

Hilfssatz 6.3 *Angenommen, es gilt $A^k d^0 \in K_k(d_0, A)$. Dann ist die Lösung $x \in \mathbb{R}^n$ von $Ax = b$ bereits im k -ten Krylow-Raum $K_k(d_0, A)$ enthalten.*

Beweis Es sei $K_k := K_k(d_0, A)$ gegeben und $x^k \in x_0 + K_k$ die beste Approximation, welche die Galerkin-Gleichung (6.3) erfüllt. Es sei $r^k := b - Ax^k$. Wegen

$$r^k = b - Ax^k = \underbrace{b - Ax^0}_{=d^0} + \underbrace{A(x^0 - x^k)}_{\in K_k} \in d^0 + AK_k$$

gilt $r^k \in K_{k+1}$. Angenommen, nun, $K_{k+1} \subset K_k$. Dann gilt $r^k \in K_k$. Die Galerkin-Gleichung besagt $r^k \perp K_k$, d. h., es gilt zwingend $r^k = 0$ und $Ax^k = b$. $\square$

Wir können also das Verfahren abbrechen, wenn der nächste Krylow-Raum nicht echt größer geworden ist als der letzte, und haben dabei die Lösung gefunden.

Wir gehen nun davon aus, dass der Krylow-Teilraum $K_k(d^0, A)$ durch k unabhängige Vektoren $d^0, \dots, d^{k-1}$ aufgespannt ist. Die Galerkin-Gleichung zur Suche nach den

Lösungskoeffizienten $\alpha_0, \dots, \alpha_{k-1}$ von

$$x^k = x^0 + \sum_{i=0}^{k-1} \alpha_i d^i$$

schreibt sich dann als

$$\sum_{i=0}^{k-1} (Ad^i, d^j) \alpha_i = (b - Ax^0, d^j) \quad \forall j = 0, \dots, k-1. \quad (6.4)$$

Um dieses reduzierte Gleichungssystem schnell lösen zu können werden wir die Richtungen d^i als Orthogonalbasis des $K_k(d^0, A)$ bestimmen. Wir haben die Matrix A als symmetrisch positiv definit vorausgesetzt. Damit ist durch (Ax, y) ein Skalarprodukt gegeben. Nun sei angenommen, die Vektoren d^j seien paarweise A -orthogonal, d. h.

$$(Ad^j, d^i) = 0 \quad i \neq j, \quad (6.5)$$

dann lässt sich die Lösung zu (6.4) einfach ablesen

$$\alpha_j = \frac{(b - Ax^0, d^j)}{(Ad^j, d^j)} \quad j = 0, \dots, k-1.$$

Die A -orthogonale Basis $\{d^0, \dots, d^{k-1}\}$ des Krylow-Raums $K_k(d^0, A)$ könnte z. B. mit dem Gram-Schmidt-Verfahren berechnet werden. Der Name *konjugierte Gradienten* stammt von der Orthogonalität der neuen Richtungen d^j , die Schritt für Schritt den Krylowraum aufspannen. Wir verweisen auch auf Abschn. 11.2, wo wir das CG-Verfahren aus einer ganz anderen Perspektive erneut kennenlernen werden.

Die Nachteile des Gram-Schmidt-Verfahrens sind der hohe Aufwand und die mangelnde numerische Robustheit. Zur Orthogonalisierung eines Vektors bzgl. einer bereits vorhandenen Basis $\{d^0, \dots, d^{k-1}\}$ sind k Skalarprodukte erforderlich. Seine Leistungsfähigkeit erlangt das CG-Verfahren erst durch Ausnutzen einer zweistufigen Rekursionsformel, welche die A -orthogonale Basis effizient und stabil berechnet:

Hilfssatz 6.4 (Zweistufige Rekursionsformel zur Orthogonalisierung) Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit sowie $x^0 \in \mathbb{R}^n$ und $d^0 := b - Ax^0$. Dann wird durch die Iteration für $k = 1, 2, \dots$

$$r^k := b - Ax^k, \quad \beta_{k-1} := -\frac{(r^k, Ad^{k-1})}{(d^{k-1}, Ad^{k-1})}, \quad d^k := r^k + \beta_{k-1}d^{k-1},$$

eine A -orthogonale Basis mit $(Ad^r, d^s) = 0$ für $r \neq s$ erzeugt. Dabei ist x^k in Schritt k definiert als die Galerkin-Lösung $(b - Ax^k, d^j) = 0$ für $j = 0, \dots, k-1$.

Beweis Es sei durch $\{d^0, \dots, d^{k-1}\}$ eine A -orthogonale Basis des $K_k(d^0, A)$ gegeben. Weiter sei $x^k \in x^0 + K_k(d^0, A)$ die Galerkin-Lösung zu (6.2). Es sei $r^k := b - Ax^k \in K_{k+1}$ und wir fordern, dass $r^k \notin K_k(d^0, A)$. Ansonsten bricht die Iteration nach Hilfssatz 6.3 ab. Wir bestimmen d^k mit dem Ansatz

$$d^k = r^k + \sum_{j=0}^{k-1} \beta_j^{k-1} d^j. \quad (6.6)$$

Die Orthogonalitätsbedingung besagt:

$$\begin{aligned} 0 &\stackrel{!}{=} (d^k, Ad^i) = (r^k, Ad^i) + \sum_{j=0}^{k-1} \beta_j^{k-1} (d^j, Ad^i) \\ &= (r^k, Ad^i) + \beta_i^{k-1} (d^i, Ad^i), \quad i = 0, \dots, k-1 \end{aligned}$$

Es gilt $(r^k, Ad^i) = (b - Ax^k, Ad^i) = 0$ für $i = 0, \dots, k-2$, da $r^k \perp AK_{k-1} \subset K_k$. Hieraus folgt $\beta_i^{k-1} = 0$ für $i = 0, 1, \dots, k-2$. Für $i = k-1$ gilt

$$\beta_{k-1} := \beta_{k-1}^{k-1} = -\frac{(r^k, Ad^{k-1})}{(d^{k-1}, Ad^{k-1})}.$$

Schließlich gilt mit (6.6) $d^k = r^k + \beta_{k-1} d^{k-1}$. □

Mit diesen Vorarbeiten können wir alle Bestandteile des CG-Verfahrens zusammensetzen. Es sei mit x^0 eine Startlösung und mit $d^0 := b - Ax^0$ der Startdefekt gegeben. Angenommen, $K_k := \text{span}\{d^0, \dots, d^{k-1}\}$ sowie $x^k \in x^0 + K_k$ und der Defekt $r^k = b - Ax^k$ liegen vor. Dann berechnet sich d^k gemäß Hilfssatz 6.4 als

$$\beta_{k-1} = -\frac{(r^k, Ad^{k-1})}{(d^{k-1}, Ad^{k-1})}, \quad d^k = r^k + \beta_{k-1} d^{k-1}. \quad (6.7)$$

Für den neuen Entwicklungskoeffizienten α_k in $x^{k+1} = x^0 + \sum_{i=0}^k \alpha_i d^i$ gilt durch Testen der Galerkin-Gleichung (6.2) mit d^k

$$\begin{aligned} \left(\underbrace{b - Ax^0}_{=d^0} - \sum_{i=0}^k \alpha_i Ad^i, d^k \right) &= (b - Ax^0, d^k) - \alpha_k (Ad^k, d^k) \\ &= (b - Ax^k - \underbrace{A(x^0 - x^k)}_{\in K_k \perp Ad^k}, d^k) - \alpha_k (Ad^k, d^k). \end{aligned}$$

Also

$$\alpha_k = \frac{(r^k, d^k)}{(Ad^k, d^k)}, \quad x^{k+1} = x^k + \alpha_k d^k. \quad (6.8)$$

Hieraus lässt sich auch unmittelbar der neue Defekt r^{k+1} bestimmen:

$$r^{k+1} = b - Ax^{k+1} = b - Ax^k - \alpha_k Ad^k = r^k - \alpha_k Ad^k \quad (6.9)$$

Wir fassen (6.7–6.9) zusammen und formulieren das klassische CG-Verfahren:

Algorithmus 6.1: CG-Verfahren

Eingabe: Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit, $x^0 \in \mathbb{R}^n$ und $r^0 = d^0 = b - Ax^0$ gegeben.

```

1 Für  $k = 0, 1, \dots$  tue
2    $\alpha_k = \frac{(r^k, d^k)}{(Ad^k, d^k)}$ 
3    $x^{k+1} = x^k + \alpha_k d^k$ 
4    $r^{k+1} = r^k - \alpha_k Ad^k$ 
5   Wenn  $\|r^{k+1}\| = 0$  dann
6     Stop
7    $\beta_k = -\frac{(r^{k+1}, Ad^k)}{(d^k, Ad^k)}$ 
8    $d^{k+1} = r^{k+1} + \beta_k d^k$ 

```

Ergebnis: Approximative Lösung $x = x^{k+1}$ von $Ax = b$.

Für unsere Python-Implementierung können wir das Verfahren noch etwas effizienter gestalten. Aufgrund der *Galerkin-Orthogonalität* und da die Krylow-Räume eine Sequenz von Teilräumen bilden, gilt

$$(r^k, d^j) = 0, \quad j = 0, \dots, k-1, \quad (r^k, r^j) = 0, \quad j = 0, \dots, k-1.$$

Damit haben wir mit (6.7), dass

$$(d^k, r^k) = (r^k - \beta_{k-1} d^{k-1}, r^k) = \|r^k\|^2,$$

und schließlich mit (6.8)

$$\alpha_k = \frac{\|r^k\|^2}{(Ad^k, d^k)}. \quad (6.8b)$$

Um β_k zu vereinfachen, beobachten wir zunächst, dass aus (6.8) folgt $Ax^{k+1} = Ax^k + \alpha_k Ad^k$. Mit der paarweisen Orthogonalität der Residuen und (6.8b), haben wir dann mit einer Nullergänzung

$$(r^{k+1}, Ad^k) = \frac{1}{\alpha_k} (r^{k+1}, Ax^{k+1} - Ax^k) = \frac{1}{\alpha_k} (r^{k+1}, r^k - r^{k+1}) = -\frac{\|r^{k+1}\|}{\|r^k\|} (Ad^k, d^k).$$

Also

$$\beta_k = \frac{\|r^{k+1}\|}{\|r^k\|}.$$

Damit sparen wir uns ein Skalarprodukt und das separate Auswerten der Norm $\|r^{k+1}\|$.

♣ Implementierung 6.1: CG-Verfahren

```

1 def cg_verfahren(A, b, x, it=100, tol=1e-5):
2     x = x.copy()
3     r = b - A.dot(x)
4     d = r.copy()
5     nrm_r2 = r.dot(r)
6     tol = tol**2
7     for i in range(1, it + 1):
8         if nrm_r2 < tol:
9             break
10        Ad = A.dot(d)
11        dAd = d.dot(Ad)
12        alpha = nrm_r2 / dAd
13        x[:] += alpha * d
14        r[:] -= alpha * Ad
15        nrm_r2_neu = r.dot(r)
16        beta = nrm_r2_neu / nrm_r2
17        d[:] = r + beta * d
18        nrm_r2 = nrm_r2_neu
19    else:
20        print(f'Das CG verfahren ist nach {i} Iterationen nicht
21              ↳ konvergiert.')
22    return x, i

```

Bei exakter Arithmetik liefert das CG-Verfahren für ein n -dimensionales Problem eine Lösung nach (höchstens) n Schritten und kann daher prinzipiell als direktes Verfahren betrachtet werden.

Satz 6.5 (CG als direkte Methode) Das CG-Verfahren bricht für jeden Startvektor $x^0 \in \mathbb{R}^n$ bei rundungsfreier Rechnung nach spätestens n Schritten mit $x^n = x$ ab.

Beweis Wir müssen zwei Fälle unterscheiden. Angenommen, die Iteration bricht im k -ten Schritt, mit $k \leq n$ ab. Dann haben wir nach Hilfssatz 6.3, dass

$$b - Ax^k \in K_k(d_0, A)$$

und der Galerkin-Orthogonalität, dass $r^k = 0$, also $x^k = x$.

Wird der n -Schritt des CG-Verfahrens erreicht, dann ist wieder nach Hilfssatz 6.3

$$K_0(d^0, A) \subsetneq K_1(d^0, A) \subsetneq \cdots \subsetneq K_{n-1}(d^0, A) = \mathbb{R}^n.$$

Somit sind $x^n, r^n \in \mathbb{R}^n$. Wir können also die Galerkin-Gleichung (6.3) mit $y = r^n = b - Ax^n$ testen. Damit folgt $\|r^n\| = 0$, was genau dann der Fall ist wenn $x^n = x$. $\square$

Obwohl das CG-Verfahren prinzipiell eine direkte Methode ist, wird es in der Praxis als approximative, iterative Methode eingesetzt. Durch Rundungsfehler werden die Suchrichtungen $\{d^0, \dots, d^{k-1}\}$ nie wirklich exakt A-orthogonal sein.

Die Analyse der Konvergenzgeschwindigkeit des CG-Verfahrens ist recht aufwändig und baut auf Aspekten der Approximationstheorie auf, die wir erst in Abschn. 9.5.2 dieses Buches einführen. Der Beweis für das folgende Konvergenzresultat ist daher als Abschn. 11.3 in Kap. 11 gegeben.

Satz 6.6 (Konvergenz des CG-Verfahrens) Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit, durch $b \in \mathbb{R}^n$ die rechte Seite und durch $x^0 \in \mathbb{R}^n$ ein beliebiger Startwert gegeben. Dann gilt für die Iterierten des CG-Verfahrens

$$\|x^k - x\|_A \leq 2 \left(\frac{1 - 1/\sqrt{\kappa}}{1 + 1/\sqrt{\kappa}} \right)^k \|x^0 - x\|_A, \quad k \geq 0,$$

mit der Spektralkondition $\kappa = \text{cond}_2(A)$ der Matrix A .

Beispiel 6.7

Wir betrachten wieder die Modellmatrix $A \in \mathbb{R}^{n \times n}$ mit $n = m^2$ und

$$A = \begin{pmatrix} B & -I & & \\ -I & B & \ddots & \\ & \ddots & \ddots & -I \\ -I & B & & \end{pmatrix}, \quad B = \begin{pmatrix} 4 & -1 & & \\ -1 & 4 & \ddots & \\ & \ddots & \ddots & -1 \\ -1 & 4 & & \end{pmatrix}, \quad I = \begin{pmatrix} 1 & & & \\ & \ddots & & \\ & & \ddots & \\ & & & 1 \end{pmatrix}$$

mit $B, I \in \mathbb{R}^{m \times m}$ aus Beispiel 3.51 und wenden unsere Pythonimplementierung hierauf an. Wir nehmen dabei $m = 10, 20, \dots, 100$ und setzen die Fehlertoleranz auf $\text{tol} = 10^{-6}$. Da wir wissen, dass $\kappa \approx n$, erwarten wir, dass das Verfahren mit steigendem n immer langsamer konvergiert. Dies beobachten wir auch in der Zusammenfassung der Ergebnisse in Tab. 6.1. $\blacktriangleleft$

Tab. 6.1 Ergebnisse für das CG-Verfahren, angewandt auf die Modellmatrix

n	# Schritte	Zeit	Residuum
100	16	0.0044	$3.58 \cdot 10^{-14}$
400	36	0.0090	$4.87 \cdot 10^{-7}$
900	55	0.0094	$6.42 \cdot 10^{-7}$
1600	74	0.0271	$6.92 \cdot 10^{-7}$
2500	93	0.1072	$6.86 \cdot 10^{-7}$
3600	112	0.2532	$7.18 \cdot 10^{-7}$
4900	131	1.0982	$7.47 \cdot 10^{-7}$
6400	150	2.5969	$7.76 \cdot 10^{-7}$
8100	169	4.6177	$8.36 \cdot 10^{-7}$
10000	188	7.9617	$8.60 \cdot 10^{-7}$

6.3 Vorkonditioniertes CG-Verfahren

Die Konvergenzrate des CG-Verfahrens hängt von der Konditionszahl der Systemmatrix ab. Es gilt

$$\rho_{CG} = \frac{1 - \frac{1}{\sqrt{\kappa}}}{1 + \frac{1}{\sqrt{\kappa}}} = 1 - \frac{2}{\sqrt{\kappa}} + O\left(\frac{1}{\kappa}\right).$$

Beispiel 6.8

Für Diskretisierungen von elliptischen partiellen Differentialgleichungen in zwei Dimensionen müssen wir z. B. $\kappa = O(N)$ erwarten, also

$$\rho_{CG} \sim 1 - \frac{1}{\sqrt{N}},$$

sodass die Konvergenzrate mit der Problemgröße N immer schlechter wird. ◀

Die Idee der Vorkonditionierung ist eine Reformulierung des linearen Gleichungssystems $Ax = b$ durch Multiplikation mit einer (regulären) Matrix $P^{-1} \approx A^{-1}$, d. h.

$$Ax = b \quad \Leftrightarrow \quad P^{-1}Ax = P^{-1}b \quad (6.10)$$

mit dem Ziel, dass die neue Matrix $P^{-1}A$ besser konditioniert ist als die Matrix A selbst. Dabei ist die Wahl von P wieder durch die folgenden Ziele bestimmt: P^{-1} muss einfach zu berechnen sein und sollte gleichzeitig eine gute Approximation an A^{-1} sein, auf jeden Fall die Konditionszahl verbessern, sprich $\text{cond}_2(P^{-1}A) \ll \text{cond}_2(A)$. Die Anwendung des

Vorkonditionierer von links in (6.10) heißt *Links-Vorkonditionierung*, entsprechend heißt die Anwendung von rechts die *Rechts-Vorkonditionierung*

$$Ax = b \Leftrightarrow AP^{-1}y = b, \quad Px = y.$$

Ein Vorkonditionierer muss dabei nicht als Matrix dargestellt und gespeichert werden. Es kann sich genauso gut um die Anwendung eines einfachen Verfahrens, z. B. ein Schritt der Jacobi-Iteration, handeln.

Beispiel 6.9 (Vorkonditionierung)

Die Matrix $A \in \mathbb{R}^{2 \times 2}$

$$A = \begin{pmatrix} 10000 & 0.9 \\ 0.9 & 1 \end{pmatrix}$$

hat die Spektralkondition $\text{cond}_2(A) \approx 10000$. Zum Vorkonditionieren wählen wir das Gauß-Seidel-Verfahren aus Abschn. 3.7 mit $P_{GS}^{-1} = (L + D)^{-1}$, wobei $A = L + D + R$. Die vorkonditionierte Matrix $P_{GS}^{-1}A$ hat dann die Konditionszahl $\text{cond}_2(P^{-1}A) \approx 2.4$. ◀

Das CG-Verfahren kann nur auf lineare Gleichungssysteme mit symmetrischer Matrix angewendet werden. D. h., auch die vorkonditionierte Matrix $P^{-1}A$ muss symmetrisch sein. Für das Folgende sei $P \in \mathbb{P}^{n \times n}$ eine Matrix, welche die Schreibweise

$$P = KK^T$$

erlaubt. Dann gilt

$$Ax = b \Leftrightarrow \underbrace{K^{-1}AK^{-T}}_{=: \tilde{A}} \underbrace{K^T x}_{=: \tilde{x}} = \underbrace{K^{-1}b}_{=: \tilde{b}},$$

also

$$\tilde{A}\tilde{x} = \tilde{b}$$

mit einer Matrix $\tilde{A}$, die wieder symmetrisch ist. Im Fall

$$\text{cond}_2(\tilde{A}) \ll \text{cond}_2(A)$$

und falls die Anwendung von K^{-1} „preiswert“ ist, so kann durch Betrachtung des *vorkonditionierten Systems* $\tilde{A}\tilde{x} = \tilde{b}$ eine wesentliche Beschleunigung erreicht werden.

Das CG-Verfahren mit Vorkonditionierung kann folgendermaßen formuliert werden:

Algorithmus 6.2: CG-Verfahren mit Vorkonditionierung

Eingabe: Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit, $P = K K^T$ ein symmetrischer Vorkonditionierer und $x^0 \in \mathbb{R}^n$ ein Startwert.

```

1  $r^0 = b - Ax^0$ 
2  $Pp^0 = r^0$ 
3 Für  $k = 0, 1, \dots$  tue
4    $\alpha_k = \frac{(r^k, d^k)}{(Ad^k, d^k)}$ 
5    $x^{k+1} = x^k + \alpha_k d^k$ 
6    $r^{k+1} = r^k - \alpha_k Ad^k$ 
7    $Pp^{k+1} = r^{k+1}$ 
8    $\beta_k = \frac{(r^{k+1}, p^{k+1})}{(r^k, p^k)}$ 
9    $d^{k+1} = p^{k+1} + \beta_k d^k$ 

```

Ergebnis: Approximative Lösung $x = x^{k+1}$ von $Ax = b$.

In jedem Schritt der Verfahrens kommt als zusätzlicher Aufwand die Anwendung des Vorkonditionierers P hinzu. Bei der Auswahl des Vorkonditionierers ist darauf zu achten, dass er die Schreibweise $P = K K^T$ erlaubt – auch wenn die Teile K und K^T im Verfahren nicht genutzt werden. Als Vorkonditionierer kommen die üblichen Approximationsverfahren aus Abschn. 3.7 zum Einsatz:

- *Jacobi-Vorkonditionierung*

Wir wählen $P \approx D^{-1}$, wobei D der Diagonalanteil der Matrix A ist. Hier gilt

$$D = D^{\frac{1}{2}} (D^{\frac{1}{2}})^T,$$

das heißt, bei $D_{ii} > 0$ ist dieser Vorkonditionierer zulässig. Für die vorkonditionierte Matrix gilt

$$\tilde{A} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}} \Rightarrow \tilde{a}_{ii} = 1$$

und die Vorkonditionierung bewirkt eine Skalierung der Matrixeinträge. Diese Art der Vorkonditionierung kann die Kondition verbessern.

- *SSOR-Vorkonditionierung*

Das SSOR-Verfahren ist eine symmetrische Variante des SOR-Verfahrens und basiert auf der Zerlegung

$$P = (D + \omega L) D^{-1} (D + \omega R) = \underbrace{(D^{\frac{1}{2}} + \omega L D^{-\frac{1}{2}})}_K \underbrace{(D^{\frac{1}{2}} + \omega D^{-\frac{1}{2}} R)}_{=K^T},$$

welche im Fall symmetrischer Matrizen $L = R^T$ die notwendige Bedingung an einen CG-Vorkonditionierer erfüllt.

Für die Modellmatrix der Diskretisierung der Laplace-Gleichung kann bei optimaler Wahl von ω (welches eine nicht triviale Aufgabe ist) die Beziehung

$$\text{cond}_2(\tilde{A}) = \sqrt{\text{cond}_2(A)}$$

gezeigt werden. Die Konvergenzrate verbessert sich somit erheblich. Die Anzahl der notwendigen Schritte zum Erreichen einer Fehlerreduktion um den festen Faktor ϵ verbessert sich zu

$$t_{CG}(\epsilon) = \frac{\log(\epsilon)}{\log(1 - \kappa^{-\frac{1}{2}})} \approx -\frac{\log(\epsilon)}{\sqrt{\kappa}}, \quad \tilde{t}_{CG}(\epsilon) = \frac{\log(\epsilon)}{\log(1 - \kappa^{-\frac{1}{4}})} \approx \frac{\log(\epsilon)}{\sqrt[4]{\kappa}}.$$

Anstelle von z. B. 100 werden nur zehn Schritte benötigt. Die Bestimmung eines optimalen Parameters ω ist im Allgemeinen jedoch sehr schwer.

- *Unvollständige Cholesky-Zerlegung*

Schließlich betrachten wir die Vorkonditionierung durch unvollständige Cholesky-Zerlegung. Zur symmetrisch positiv definiten Matrix A erstellen wir die approximative Zerlegung

$$A \approx C^T C,$$

wobei die Zerlegung in C^T und C die gleiche Besetzungsstruktur der Matrix A verwendet. Das heißt, $C_{ij} \neq 0$ nur dann, wenn auch $A_{ij} \neq 0$. Es gilt

$$A = C^T C + N,$$

wobei die Größe (also die Zahl der Nicht-Nullen) und Bedeutung von N ganz wesentlich von der Sortierung der Matrix A abhängt (siehe Abschn. 3.4). Eine Analyse dieser Vorkonditionierung ist schwierig. In der Praxis zeigt sich jedoch, dass dieses Verfahren den anderen Methoden weit überlegen ist. Insbesondere kommt es ohne die Bestimmung des Parameters ω aus. Die Anwendung ist natürlich weitaus „teurer“ als die Anwendung von Jacobi- oder SSOR-Vorkonditionierung.

Beispiel 6.10

Wir setzen Beispiel 6.7 fort und betrachten das Vorkonditionieren des CG-Verfahrens anhand der Modellmatrix. Insbesondere betrachten wir die Jacobi- und SOR-Vorkonditionierung, da wir den optimale Relaxationsparameter aus Beispiel 3.51 kennen. Die Ergebnisse sind in Tab. 6.2 zu sehen. Wir beobachten, dass die Jacobi-Vorkonditionierung die Anzahl der notwendigen Schritte kaum geringer ist. Allerdings ist die Anwendung dieses Vorkonditionierers sehr billig, sodass sich die Rechenzeiten sehr ähnlich zu dem CG-Verfahren in Beispiel 6.7 sind. Im Vergleich dazu wächst die Anzahl der notwendigen Schritte beim SOR-Vorkonditionierten CG-Verfahren sehr langsam. Allerdings ist die Anwendung dieses Vorkonditionierers in dieser Implementierung (mit `numpy.linalg.solve`) so teuer, dass die Rechenzeiten sogar schlechter geworden sind. ◀

Tab.6.2 Zu Beispiel 6.10: Ergebnisse für das CG-Verfahren, angewandt auf die Modellmatrix

n	Jacobi			SOR		
	# Schritte	Zeit	Residuum	# Schritte	Zeit	Residuum
100	16	0.0007	$3.58 \cdot 10^{-14}$	12	0.0040	$6.05 \cdot 10^{-7}$
400	35	0.0008	$1.25 \cdot 10^{-6}$	17	0.0246	$9.23 \cdot 10^{-7}$
900	54	0.0048	$1.40 \cdot 10^{-6}$	22	0.1803	$4.55 \cdot 10^{-7}$
1600	73	0.0344	$1.21 \cdot 10^{-6}$	26	0.8807	$4.67 \cdot 10^{-7}$
2500	91	0.1126	$1.73 \cdot 10^{-6}$	29	3.3557	$8.18 \cdot 10^{-7}$
3600	110	0.2500	$1.60 \cdot 10^{-6}$	33	11.6160	$6.75 \cdot 10^{-7}$
4900	128	1.1094	$1.98 \cdot 10^{-6}$	36	29.7653	$6.94 \cdot 10^{-7}$
6400	147	2.4710	$1.78 \cdot 10^{-6}$	39	71.3470	$5.25 \cdot 10^{-7}$
8100	166	4.6795	$1.71 \cdot 10^{-6}$	–	–	–
10000	185	7.7390	$1.63 \cdot 10^{-6}$	–	–	–

6.4 Das Arnoldi-Verfahren

Die Effizienz des CG-Verfahrens liegt an der schnellen zweistufigen Orthogonalisierung des Krylow-Raums $K_k(d^0, A)$. Wir betrachten nun den Fall allgemeiner, also nicht-symmetrischer Matrizen $A \in \mathbb{R}^{n \times n}$. Zu $v \in \mathbb{R}^n$ beliebig sei $K_k(v, A) = \text{span}\{v, Av, \dots, A^{k-1}v\}$ der Krylow-Teilraum. Der Vektor $v \in \mathbb{R}^n$ muss dabei zunächst nicht im Zusammenhang zu einem linearen Gleichungssystem stehen. Das *Arnoldi-Verfahren* ist eine Iteration zum Aufbau einer orthogonalen Basis des Krylow-Raums $K_k(v, A)$.

Die einfachste Variante des Arnoldi-Verfahrens kann mit dem Gram-Schmidt’schen Orthogonalisierungsverfahren geschrieben werden, siehe Abschn.4.2, sowie Satz 2.10.

Algorithmus 6.3: Arnoldi-Verfahren mit Gram-Schmidt

Eingabe: Matrix $A \in \mathbb{R}^{n \times n}$ sowie Vektor $v \in \mathbb{R}^n$

1

$v^1 = v / \|v\|$

2

Für $i = 1$ **bis** k **tue**

3

Für $j = 1$ **bis** i **tue**

4

$h_{ji} = (Av^i, v^j)$

5

$w^i = Av^i - \sum_{j=1}^i h_{ji} v^j$

6

$h_{i+1,i} = \|w^i\|$

7

Wenn $h_{i+1,i} = 0$ **dann**

8

Stop

9

$v^{i+1} = w^i / h_{i+1,i}$

Ergebnis: Matrix $H_k = (h_{ji})_{ji} \in \mathbb{R}^{k \times k}$ sowie Vektoren $v^1, \dots, v^k \in \mathbb{R}^n$. Gegebenenfalls $h_{k+1,k}$ sowie Vektor v^{k+1} .

Der Arnoldi-Algorithmus ist das klassische Gram-Schmidt-Verfahren zur Orthonormalisierung von $K_k(v, A)$. Ein Unterschied zum Basisalgorithmus aus Satz 2.10 ist, dass die zu orthogonalisierenden Vektoren erst im Laufe des Verfahrens, in Zeile 4, berechnet werden.

Satz 6.11 *Falls das Arnoldi-Verfahren nicht in Zeile 7 abbricht, so sind die Vektoren $v^1, \dots, v^k$ ein Orthonormalsystem des $K_k(v, A)$ bzgl. des euklidischen Skalarprodukts.*

Beweis Die Orthonormalität folgt aus der Konstruktion. Auch ist einfach zu sehen, dass

$$v^i \in K_i(v, A)$$

gilt. Dies kann mit Induktion gezeigt werden. Für $i = 1$ ist die Aussage klar. Es gelte also $v^i \in K_i(v, A)$. Dann gilt $Av^i \in K_{i+1}(v, A)$ und $\sum_{j=1}^i h_{ji} v^j \in K_i(v, A) \subset K_{i+1}(v, A)$, siehe Zeile 5. Hiermit folgt $w^i \in K_{i+1}(v, A)$ und mit Zeile 9 dann $v^{i+1} \in K_{i+1}(v, A)$.

Ein System aus k orthonormalen Vektoren muss die Dimension k besitzen. Somit ist klar, dass der ganze $K_k(v, A)$ aufgespannt wird. $\square$

Falls in Zeile 7 $h_{i+1,i} = 0$ gilt, so bedeutet dies, dass der Vektor Av^i bereits in dem von $\{v^1, \dots, v^i\}$ aufgespannten Raum liegt, vergleiche Hilfsatz 6.3.

Bemerkung 6.12 (Orthogonalität bei Arnoldi und CG) Das CG-Verfahren erzeugt eine Orthonormalbasis bzgl. des Skalarproduktes $(A \cdot, \cdot)$. Beim Arnoldi-Verfahren wird die Orthonormalität bzgl. des euklidischen Skalarproduktes $(\cdot, \cdot)$ erreicht. Die Wahl des gewichteten Skalarproduktes beim CG-Verfahren ist dadurch motiviert, dass die Galerkin-Gleichung (6.3) so formuliert ist, dass Testraum und Ansatzraum übereinstimmen

$$x^k \in x^0 + K_k(d^0, A) : (b - Ax^k, y) = 0 \quad \forall y \in K_k(d^0, A).$$

Setzt man die Entwicklung

$$x^k = x^0 + \sum_{i=1}^k \alpha_i d^i$$

ein, so ergibt sich

$$\sum_{i=1}^k \alpha_i (Ad^i, d^j) = (d^0, d^j) \quad \forall j = 1, \dots, k \Rightarrow \alpha_i = (d^0, d^i),$$

und die Lösung kann einfach bestimmt werden, wenn die Orthonormalbasis bekannt ist. Dieser Ansatz funktioniert beim CG-Verfahren aber natürlich nur, weil $(A \cdot, \cdot)$ wirklich ein Skalarprodukt ist, da die Matrix A symmetrisch positiv definit ist. Dies ist im allgemeinen Fall nicht mehr möglich. $\blacklozenge$

Satz 6.13 (Eigenschaften des Arnoldi-Verfahrens) Es sei $A \in \mathbb{R}^{n \times n}$, $v^1 \in \mathbb{R}^n$ mit $\|v^1\| = 1$ sowie h_{ij} für $i, j = 1, \dots, k$ und $h_{k,k+1}$ die vom Arnoldi-Verfahren erstellten Koeffizienten. Weiter seien $v^1, \dots, v^k$ die erstellten orthonormalen Vektoren und w^k der zuletzt orthogonalisierte Vektor. Mit den Matrizen $V_k = (v^1, \dots, v^k) \in \mathbb{R}^{n \times k}$ und

$$H_k = \begin{pmatrix} h_{1,1} & h_{1,2} & \cdots & \cdots & h_{1,k} \\ h_{2,1} & h_{2,2} & \cdots & \cdots & h_{2,k} \\ 0 & \ddots & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & h_{k,k-1} & h_{k,k} \end{pmatrix} \in \mathbb{R}^{k \times k}, \quad \tilde{H}_k = \begin{pmatrix} & & & & \\ & & & & \\ & & & & \\ & & & & \\ & & & & \\ \hline 0 & \cdots & 0 & h_{k+1,k} & \end{pmatrix} \in \mathbb{R}^{k+1 \times k},$$

gilt

$$AV_k = V_k H_k + w^k e_k^T = V_{k+1} \tilde{H}_k, \quad (6.11)$$

sowie

$$V_k^T AV_k = H_k. \quad (6.12)$$

Beweis Die Zeilen 5, 4 und 9 ergeben im Fall $h_{i+1,i} \neq 0$

$$Av^i = w^i + \sum_{j=1}^i h_{ji} v^j.$$

Für $i < k$ ist $w^i = h_{i+1,i} v^{i+1}$, d.h. es gilt

$$Av^i = \sum_{j=1}^{i+1} h_{ji} v^j \quad i = 1, \dots, k-1.$$

Zusammen ergibt dies gerade (6.11). Weiterhin folgt (6.12) bei Multiplikation mit V_k^T und Ausnutzen der Orthonormalität, d.h., das $V_k^T V_k$ die Einheitsmatrix im $\mathbb{R}^{n \times n}$ ist. Schließlich ist $V_k^T w^k = 0$, da entweder w^k Null ist, oder aber w^k orthogonal auf allen $v^1, \dots, v^k$ steht. $\square$

Bemerkung 6.14 (Arnoldi-Verfahren und Hessenberg-Normalform) Das Arnoldi-Verfahren ist eine iterative Methode zur Transformation einer Matrix A auf Hessenberg-Gestalt, die sich aber grundlegend von den in Abschn. 5.5 untersuchten Algorithmen unterscheidet.

Wir erinnern: die Erstellung der Hessenberg-Form auf Basis von Householder-Transformationen ist eine Ähnlichkeitstransformation, die die Matrix von links und rechts mit Householder-Matrizen multipliziert

$$Q^T A Q = H,$$

dabei ist $Q \in \mathbb{R}^{n \times n}$ eine orthogonale Matrix. Die Arnoldi-Iteration liefert ein ähnliches Ergebnis

$$V_k^T A V_k = H_k.$$

Dabei ist $V_k \in \mathbb{R}^{n \times k}$ bei $k < n$ jedoch nicht quadratisch und natürlich auch nicht regulär. Es liegt also keine Ähnlichkeitstransformation vor. Falls das Arnoldi-Verfahren bei n Iterationen nicht vorzeitig abbricht sondern bis $V_n \in \mathbb{R}^{n \times n}$ durchläuft, so erzeugt auch dieses die Hessenberg-Normalform und eine Ähnlichkeitstransformation. ♦

Der Aufwand zum Erstellen der Hessenberg-Form mit Householder-Transformationen ist in $\frac{2}{3}n^3 + O(n^2)$, vergleiche Bemerkung 5.27. Der wesentliche Unterschied des Arnoldi-Verfahrens ist jedoch einerseits, dass es approximativ für $k \ll n$ durchgeführt werden kann und dass es nicht auf Matrix-Transformationen sondern nur auf Matrix-Vektor Produkten sowie Skalarprodukten beruht.

Satz 6.15 (Aufwand des Arnoldi-Verfahrens) Es sei $A \in \mathbb{R}^{n \times n}$. Angenommen, das Arnoldi-Verfahren bricht nicht vorzeitig ab. Dann sind zur Durchführung des Verfahrens mittels Algorithmus 6.3 im Wesentlichen k Matrix-Vektor Produkte sowie $\frac{k^2}{2} + O(k)$ Skalarprodukte im $\mathbb{R}^n$ zu berechnen.

Beweis Das Produkt $\tilde{v}^i = A v^i$ muss pro Iteration nur einmal berechnet werden und kann dann gespeichert werden. In Schritt i müssen in Zeile 4 insgesamt i Skalarprodukte berechnet werden, insgesamt also $\frac{k(k+1)}{2}$. Hinzu kommen insgesamt k Vektoradditionen, die im Aufwand der Skalarprodukte inkludiert werden. □

Bei einer dünn besetzten Matrix mit nur $O(n)$ nicht-Null Einträgen ergibt sich der Aufwand $O(k^2n + kn)$ und bei einer voll besetzten ist der Aufwand $O(k^2n + kn^2)$ Operationen. Bei vollständiger Iteration ist der Aufwand kubisch.

Bemerkung 6.16 (Robustheit des Arnoldi-Verfahrens) In Abschn. 4.2 haben wir die Stabilitätsprobleme des Gram-Schmidt-Verfahrens diskutiert. Diese treffen ebenso auf das Arnoldi-Verfahren zu und der oben präsentierte Algorithmus ist zwar einfach, leidet aber unter Stabilitätsproblemen. In der praktischen Anwendung sollte das Verfahren daher auf Basis des modifizierten Gram-Schmidt-Verfahrens oder mit Hilfe von Householder-Transformationen erstellt werden. ♦

6.4.1 Das Eigenwertproblem

Die Ähnlichkeitstransformation auf Hessenberg-Gestalt erfolgt, da die Eigenwerte der Hessenberg-Matrix sehr effizient z.B. mit der QR-Iteration berechnet werden können, vergleiche Satz 5.28. Das gleiche gilt natürlich auch für das Arnoldi-Verfahren und die viel

kleinere Hessenberg-Matrix $H_k \in \mathbb{R}^{k \times k}$. Allerdings ist die Transformation $V_k^T A V_k = H_k$ keine Ähnlichkeitstransformation, da die im Allgemeinen nicht-quadratische Matrix V_k nicht intervierbar ist. Aus Satz 6.13 folgt der Zusammenhang:

Korollar 6.17 *Es sei $A \in \mathbb{R}^{n \times n}$, $V_k \in \mathbb{R}^{n \times k}$, $H_k \in \mathbb{R}^{k \times k}$ sowie $h_{k+1,k} \in \mathbb{R}$ das Ergebnis des Arnoldi-Verfahrens. Im Fall $h_{k+1,k} = 0$ sind die Eigenwerte von H_k auch Eigenwerte von A . Für $\lambda \in \mathbb{C}$ und $x \in \mathbb{C}^k$ mit*

$$H_k x = \lambda x$$

gilt

$$A(V_k x) = \lambda(V_k x).$$

Beweis Im Fall $h_{k+1,k}$ vereinfacht sich (6.11) zu

$$A V_k = V_k H_k.$$

Für ein Paar aus Eigenwert und Eigenvektor folgt dann sofort

$$A(V_k x) = V_k H_k x = \lambda(V_k x). \quad \square$$

Sollte die Arnoldi-Iteration vorzeitig mit $h_{i+1,i} = 0$ abbrechen, so erlaubt sie eine sehr effiziente Berechnung einiger der Eigenwerte der Matrix A . Interessanter ist natürlich der Fall $h_{k+1,k} \neq 0$, von dem üblicherweise auszugehen ist. Mit $v^k = h_{k+1,k} v^{k+1}$ und $\|v^{k+1}\| = 1$, vergleiche Algorithmus 6.3, gilt dann für einen Eigenwert $\lambda \in \mathbb{C}$ und Eigenvektor $x \in \mathbb{C}^k$ von H_k

$$A(V_k x) = \lambda(V_k x) + h_{k+1,k} v^{k+1} e_k^T x \Leftrightarrow (A - \lambda I)(V_k x) = h_{k+1,k} v^{k+1} e_k^T x.$$

Das Paar $\lambda \in \mathbb{C}$ und $V_k x \in \mathbb{C}^n$ erfüllt nicht die Eigenwertgleichung. Aber Multiplikation der Gleichung mit V_k^T zeigt

$$V_k^T (A - \lambda I)(V_k x) = 0,$$

da $V_k = (v_1, \dots, v_k)$ orthonormal auf v^{k+1} steht. Anders ausgedrückt ergibt sich die Galerkin-Gleichung

$$((A - \lambda I)(V_k x), v^j) = 0 \quad j = 1, \dots, m.$$

Das Residuum der Eigenwertgleichung steht orthogonal auf dem Krylow-Raum. Für $k \rightarrow n$ kann also von einer immer besseren Approximation ausgegangen werden. Die Eigenwerte und Eigenvektoren von H_k heißen die *Ritz-Werte* und *Ritz-Vektoren* von A bzgl. des Krylow-Raums $K_k(v, A)$.

Die Analyse des Arnoldi-Verfahrens ist noch nicht vollständig. Es zeigt sich, dass insbesondere separierte Eigenwerte sehr schnell konvergieren, d.h. Eigenwerte, die betragsmäßig isoliert sind. Dies ist eine Ähnlichkeit zur Potenzmethode und der inversen Ite-

ration, wo auch die Konvergenzgeschwindigkeit ebenso von der Separiertheit der Eigenwerte beeinflusst ist. Das Arnoldi-Verfahren konvergiert aber nicht nur gegen die größten Eigenwerte sondern im ganzen Spektrum. Für weitere Analysen verweisen wir auf die Literatur [79, 83].

Angewendet auf symmetrisch positiv definite Matrizen $A \in \mathbb{R}^{n \times n}$ kann die Arnoldi-Iteration stark vereinfacht werden. Wie beim CG-Verfahren aus Abschn. 6.2 kann das Gram-Schmidt-Verfahren durch eine zweistufige Iteration ersetzt werden. Das sogenannte *Lanczos-Verfahren* erzeugt statt einer Hessenberg-Matrix eine symmetrische Tridiagonalmatrix. Von dieser können die Eigenwerte wieder sehr schnell berechnet werden. Für Details verweisen wir wieder auf die beiden oben genannten Bücher.

6.5 Das GMRES-Verfahren

Die Idee des CG-Verfahrens lässt sich auf lineare Gleichungssysteme $Ax = b$ mit allgemeinen, nicht-symmetrischen, unter Umständen nicht positiv-definiten Matrizen $A \in \mathbb{R}^{n \times n}$ übertragen. Wir wählen einen Startvektor $x^0 \in \mathbb{R}^n$ und mit dem Defekt $d^0 = (b - Ax^0)$ bilden wir den Krylow-Raum $K_k(d^0, A)$.

Mit dem Arnoldi-Verfahren können wir eine Orthonormalbasis $\{v^1, \dots, v^k\}$ des $K_k(d^0, A)$ erstellen. Wir betrachten wieder die Galerkin-Gleichung

$$x^k \in x^0 + K_k(d^0, A) : (b - Ax^k, y) = 0 \quad \forall y \in Y_k, \quad (6.13)$$

geben aber noch nicht die genaue Form des Testraums Y_k an. Für die unbekannte Lösung x^k setzen wir die Entwicklung

$$x^k = x^0 + \sum_{i=1}^k \alpha_i d^i,$$

ein, so erhalten wir

$$x^k \in x^0 + K_k(d^0, A) : \sum_{i=1}^k \alpha_i (Ad^i, y) = (d^0, y) \quad \forall y \in Y_k. \quad (6.14)$$

Im Gegensatz zum CG-Verfahren, vergleiche Bemerkung 6.12, lässt sich dieses System nicht direkt nach den Koeffizienten α_i auflösen. Auch bei Wahl des Testraums $Y_k = K_k(d^0, A)$ gilt nicht $(Ad^i, d^j) = \delta_{ij}$, denn die Orthonormalität liegt bzgl. des euklidischen Produktes vor. Bei einer allgemeinen, also nicht-symmetrischen und nicht positiv definiten Matrix A ist durch $(A \cdot, \cdot)$ überhaupt kein Skalarprodukt gegeben.

Das GMRES-Verfahren basiert stattdessen auf der Darstellung der Lösung als Minimierungsproblems. Wir bestimmen die Approximation x^k als

$$x^k \in x^0 + K_k(d^0, A) \quad \|b - Ax^k\|_2 \leq \|b - Ay^k\|_2 \quad \forall y^k \in x^0 + K_k(d^0, A).$$

Von dieser Darstellung stammt auch der Name, GMRES steht für *Generalized minimal residual method*. Wir minimieren das Residuum und es ist eine Verallgemeinerung, da es auch auf nicht-symmetrische Matrizen angewendet werden kann. Ähnliche Minimierungsprobleme treten auch bei der Ausgleichsrechnung in Abschn. 4.5 auf.

Angenommen x^k sei ein Minimum, dann muss es zwingend ein stationärer Punkt sein. Es muss dann also für beliebige $y \in K_k(d^0, A)$ gelten, dass

$$0 \stackrel{!}{=} \frac{d}{ds} \frac{1}{2} \|b - A(x + sy)\|^2 \Big|_{s=0} = (b - A(x + sy), -Ay) \Big|_{s=0} = -(b - Ax, Ay).$$

Die Bedingung hierfür entspricht gerade der Galerkin-Gleichung

$$x^k \in x^0 + K_k(d^0, A) : (b - Ax^k, y) = 0 \quad \forall y \in AK_k(d^0, A). \quad (6.15)$$

Beim GMRES-Verfahren wird demnach als Ansatzraum $x^0 + K_k(d^0, A)$ und als Testraum $AK_k(d^0, A)$ gewählt.

Die Minimierung von $\|b - Ax^k\|$ im Teilraum $x^k \in x^0 + K_k(d^0, A)$ ist (bis auf den Startwert x^0) gerade die Lösung eines überbestimmten linearen Gleichungssystems, wie es in Abschn. 4.5 behandelt wurde. Anstelle die Lösung dieses überbestimmten Systems direkt mit der QR-Zerlegung wie in Kap. 4 zu berechnen, können wir die mit dem Arnoldi-Verfahren gewonnene Orthonormaldarstellung nutzen und das Problem extrem vereinfachen:

Satz 6.18 (GMRES – Lösen des Minimierungsproblems) Es sei $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$ und $x^0 \in \mathbb{R}^n$. Für $d^0 = b - Ax^0$ und $k \in \mathbb{N}$ seien $V_k \in \mathbb{R}^{n \times k}$, $\tilde{H}_k \in \mathbb{R}^{k+1, k}$ die vom Arnoldi-Verfahren zur Orthonormalisierung von $K_k(d^0, A)$ erstellten Matrizen. Dann gilt

$$\min_{x \in x^0 + K_k(d^0, A)} \|b - Ax\|_2 = \min_{y \in \mathbb{R}^k} \| \|d^0\| e_1 - \tilde{H}_k y \|_2,$$

wobei $e_1 \in \mathbb{R}^{k+1}$ der erste Einheitsvektor im $\mathbb{R}^{k+1}$ ist.

Beweis Mit dem Ansatz

$$x = x^0 + \sum_{i=1}^k \alpha_i v^i$$

und mit Satz 6.13 gilt

$$b - Ax = b - Ax^0 - \sum_{i=1}^k \alpha_i Av^i = d^0 - AV_k y = d^0 - V_{k+1} \tilde{H}_k y.$$

Es ist $v^1 = d^0 / \|d^0\|$ der erste Spaltenvektor in V_{k+1} , also

$$b - Ax = \|d^0\| v^1 - V_{k+1} \tilde{H}_k y = V_{k+1} (\|d^0\| e_1 - \tilde{H}_k y).$$

Nun gilt wegen der Orthonormalität der Vektoren v^i für beliebige Vektoren $z \in \mathbb{R}^{k+1}$ im euklidischen Skalarprodukt

$$\|V_{k+1} y\|_2^2 = (V_{k+1} y, V_{k+1} y)_2 = (V_{k+1}^T V_{k+1} y, y)_2 = (y, y)_2 = \|y\|_2^2.$$

Also folgt

$$\|b - Ax\|_2 = \|\|d^0\| e_1 - \tilde{H}_k y\|_2.$$

□

Zur Lösung des Minimierungsproblems im Krylow-Raum $K_k(d^0, A)$ reicht es also, ein überbestimmtes Gleichungssystem mit der Matrix $\tilde{H}_k \in \mathbb{R}^{k+1,k}$ zu lösen. Dabei hat dieses Gleichungssystem die spezielle Form

$$\begin{pmatrix} h_{11} & h_{12} & \cdots & \cdots & h_{1k} \\ h_{21} & h_{22} & \ddots & & \vdots \\ 0 & h_{32} & \ddots & & \vdots \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & h_{k,k-1} & h_{kk} \\ 0 & \cdots & \cdots & 0 & h_{k+1,k} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ \vdots \\ \vdots \\ y_k \end{pmatrix} = \begin{pmatrix} \|d^0\| \\ 0 \\ \vdots \\ \vdots \\ \vdots \\ 0 \end{pmatrix}. \quad (6.16)$$

Die Matrix $\tilde{H}_k$ hat schon fast obere rechte Dreiecksgestalt. Um eine QR-Zerlegung $\tilde{H}_k = Q_k R_k$ zu erstellen sind lediglich die Elemente unterhalb der Diagonalen, also $h_{21}, h_{32}, \dots, h_{k+1,k}$ zu eliminieren. Dies kann z.B. mit k Schritten der Givens-Rotationen, siehe Abschn. 4.4, erreicht werden. Der Aufwand hierdurch beträgt $O(k^2)$. Mit Hilfe der QR-Zerlegung kann das Normalgleichungssystem gelöst werden. Es ist

$$\tilde{H}_k^T \tilde{H}_k y = \|d^0\| \tilde{H}_k^T e_1 \Leftrightarrow R_k^T R_k y = \|d^0\| R_k^T Q_k^T e_1,$$

und wenn die Arnoldi-Iteration nicht vorzeitig abgebrochen ist, dann ist die Matrix R_k regulär und die Lösung kann mit Rückwärtseinsetzen in $O(k^2)$ Operationen ermittelt werden

$$R_k y = \|d^0\| Q_k^T e_1.$$

Es gilt

Satz 6.19 (GMRES-Verfahren) Sei $A \in \mathbb{R}^{n \times n}$ eine reguläre Matrix und $x^0 \in \mathbb{R}^n$ ein beliebiger Startvektor und es sei $k \leq n$. Angenommen, die Matrix A sei dünn besetzt und der Aufwand für ein Matrix-Vektor Produkt betrage $O(n)$ Operationen. Angenommen das Arnoldi-Verfahren zur Orthonormalisierung von $K_k(b - Ax^0, A)$ bricht nicht vorzeitig ab, dann ist das GMRES-Verfahren durchführbar, liefert eine eindeutige Lösung und benötigt

$$O(k^2n) + O(k^2)$$

Operationen.

Beweis Wenn das Verfahren nicht vorzeitig abbricht, dann ist V_k eine Matrix mit Rang k , bildet also eine Orthonormalbasis des $K_k(b - Ax^0, A)$. Da A selbst regulär ist folgt, dass $H_k = V_k^T A V_k$ auch regulär ist. Denn für $y \in \mathbb{R}^k$ folgt

$$y \neq 0 \Rightarrow V_k y \neq 0 \Rightarrow A V_k y \neq 0 \Rightarrow H_k = V_k^T A V_k y \neq 0.$$

Das überbestimmte lineare Gleichungssystem ist also eindeutig lösbar.

Die Anzahl der Operationen ergibt sich aus Satz 6.15 sowie aus der Erstellung der QR-Zerlegung von $\tilde{H}_k$ sowie dem anschließenden Rückwärtseliminieren. $\square$

Bemerkung 6.20 (GMRES und Orthonormalisierung) Beim GMRES-Verfahren wird die Orthonormalbasis nicht verwendet um die Lösungskoeffizienten direkt berechnen zu können. Die spezielle Zerlegung ist aber dennoch wichtig, da sie uns erlaubt, das Optimierungsproblem $\|b - Ax\| \rightarrow \min$ für $x \in x^0 + K_k(b - Ax^0, A)$ durch ein überbestimmtes Gleichungssystem mit der Matrix $\tilde{H}_k \in \mathbb{R}^{k+1,k}$ zu ersetzen. So reduziert sich der Aufwand erheblich, da nur mit „kurzen“ Vektoren der Länge $O(k)$ statt mit Vektoren der Länge $O(n)$ gearbeitet werden muss. $\blacklozenge$

Die Konvergenzanalyse des GMRES-Verfahrens ist, wie beim Arnoldi-Verfahren, nicht abgeschlossen. Bei exakter Arithmetik gilt jedoch, wie im Fall des CG-Verfahrens, dass das GMRES-Verfahren nach höchstens n Schritten mit der exakten Lösung abbricht. Darüber hinaus ist das GMRES-Verfahren monoton konvergent, was klar ist, da es auf Minimierung von $\|b - Ax\|$ in immer größeren Unterräumen basiert. Schließlich kann gezeigt werden, dass das GMRES-Verfahren, angewendet auf symmetrisch positiv definite Matrizen, genauso gut konvergiert wie das CG Verfahren.

6.5.1 Praktische Aspekte und Implementierung

Die bisher diskutierte einfache Variante des GMRES-Verfahrens hat noch einige Schwächen, die wir in diesem Abschnitt näher betrachten:

1. Wir wissen, dass das Gram-Schmidt-Verfahren zur Orthonormalisierung nicht stabil ist. Für einen robusten Algorithmus muss dieser Teil somit verbessert werden und z. B. durch Householder-Transformationen oder Givens-Rotationen ersetzt werden.
2. Der zugrundeliegende Arnoldi-Algorithmus 6.3 ist so formuliert, dass wir zu Beginn entscheiden müssen, wie viele Schritte k wir durchführen. Dann folgt die Orthonormalisierung und erst ganz zum Schluss das Lösen des linearen Gleichungssystems. Es wäre natürlicher, wenn wir, wie beim CG-Verfahren Schritt für Schritt vorgehen und in jeder Iteration einen neuen Vektor orthonormalisieren und dann gleich die Approximation des linearen Gleichungssystems $x^k \mapsto x^{k+1}$ verbessern. Dies würde einen Abbruch des Verfahrens erlauben, wenn die gewünschte Toleranz erreicht wird.
3. Der Aufwand des GMRES-Verfahrens steigt quadratisch mit der Anzahl der Schritte k und beträgt $O(k^2n)$. Falls eine sehr große Zahl von Schritten notwendig ist, so wird die Methode zu langsam.

6.5.1.1 Robuste Varianten

Eine besonders robuste Variante von GMRES kann auf Basis von Householder-Transformationen erreicht werden. Die effiziente Umsetzung ist aber recht aufwändig, wenn darauf geachtet werden soll, dass nur der wirklich notwendige Speicher verwendet wird. Wir verweisen daher auf die Literatur [78]. Eine einfachere, aber nicht so stabile Alternative ist der modifizierte Gram-Schmidt-Algorithmus, den wir auch für unsere Implementierung verwenden werden. Er unterscheidet sich vor allem in der Reihenfolge der Operationen vom ursprünglichen Gram-Schmidt-Verfahren und er ist mathematisch äquivalent, wenn keine Rundungsfehler gemacht werden, vergleiche Satz 4.9.

Algorithmus 6.4: Arnoldi-Verfahren mit modifiziertem Gram-Schmidt

Eingabe: Matrix $A \in \mathbb{R}^{n \times n}$ sowie Vektor $v \in \mathbb{R}^n$

```

1  $v^1 = v^1 / \|v^1\|$ 
2 Für  $i = 1$  bis  $k$  tue
3    $w^i = Av^i$ 
4   Für  $j = 1$  bis  $i$  tue
5      $h_{ji} = (w^i, v^j)$ 
6      $w^i = w^i - h_{ji}v^j$ 
7    $h_{i+1,i} = \|w^i\|$ 
8   Wenn  $h_{i+1,i} = 0$  dann
9     Stop
10   $v^{i+1} = w^i / h_{i+1,i}$ 

Ergebnis: Matrix  $H_k = (h_{ji})_{ji} \in \mathbb{R}^{k \times k}$  sowie Vektoren  $v^1, \dots, v^k \in \mathbb{R}^n$ . Gegebenenfalls  $h_{k+1,k}$  sowie Vektor  $v^{k+1}$ .
```


Diese Variante hat bessere Stabilitätseigenschaften, erreicht aber nicht die Robustheit der Householder-Spiegelungen.

6.5.1.2 Schrittweise Approximation

Wir betrachten nun das zweite angesprochene Problem und leiten eine Variante her, die es in jedem Schritt $i = 1, 2, 3, \dots$ erlaubt, eine aktuelle Approximation x^i zu ermitteln. Hierzu wiederholen wir zunächst das Vorgehen zur Transformation der Householder-Matrix $\tilde{H}_k$ mit Givens-Rotationen. Das Gleichungssystem, vergleiche (6.16), hat die Form

$$\begin{pmatrix} h_{11} & h_{12} & \cdots & \cdots & h_{1k} \\ h_{21} & h_{22} & \ddots & & \vdots \\ 0 & h_{32} & \ddots & & \vdots \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & h_{k,k-1} & h_{kk} \\ 0 & \cdots & \cdots & 0 & h_{k+1,k} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ \vdots \\ \vdots \\ y_k \end{pmatrix} = \begin{pmatrix} \|d^0\| \\ 0 \\ \vdots \\ \vdots \\ \vdots \\ 0 \end{pmatrix}.$$

Wir multiplizieren von links mit einer Givens-Rotation der Form

$$G_1 = \begin{pmatrix} c_1 & s_1 & & & \\ -s_1 & c_1 & & & \\ & & 1 & & \\ & & & \ddots & \\ & & & & 1 \end{pmatrix} \in \mathbb{R}^{k+1 \times k+1},$$

und wählen

$$s_1 = \frac{h_{21}}{\sqrt{h_{11}^2 + h_{21}^2}}, \quad c_1 = \frac{h_{11}}{\sqrt{h_{11}^2 + h_{21}^2}}.$$

Bei Multiplikation des ganzen Gleichungssystems von links ergibt sich

$$\begin{pmatrix} h_{11}^{(1)} & h_{12}^{(1)} & \cdots & \cdots & h_{1k}^{(1)} \\ 0 & h_{22}^{(1)} & \ddots & & \vdots \\ 0 & h_{32} & \ddots & & \vdots \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & & \ddots & h_{k,k-1} & h_{kk} \\ 0 & \cdots & \cdots & 0 & h_{k+1,k} \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ \vdots \\ \vdots \\ y_k \end{pmatrix} = \begin{pmatrix} c_1 \|d^0\| \\ -s_1 \|d^0\| \\ 0 \\ \vdots \\ \vdots \\ 0 \end{pmatrix}.$$

Wir gehen entsprechend weiter vor mit G_2 und

$$s_2 = \frac{h_{32}}{\sqrt{(h_{22}^{(1)})^2 + h_{32}^2}}, \quad c_2 = \frac{h_{22}^{(1)}}{\sqrt{(h_{22}^{(1)})^2 + h_{32}^2}}.$$

Nach k Schritten erhalten wir

$$\begin{pmatrix} h_{11}^{(1)} & h_{12}^{(1)} & \cdots & \cdots & h_{1k}^{(1)} \\ 0 & h_{22}^{(2)} & \ddots & & \vdots \\ 0 & \ddots & \ddots & & \vdots \\ 0 & \ddots & \ddots & \ddots & \vdots \\ \vdots & & & 0 & h_{kk}^{(k)} \\ 0 & \cdots & \cdots & 0 & 0 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ \vdots \\ y_k \end{pmatrix} = \begin{pmatrix} \gamma_1^{(k)} \\ \gamma_2^{(k)} \\ \vdots \\ \vdots \\ \gamma_{k+1}^{(k)} \end{pmatrix}.$$

Mit der orthogonalen Matrix

$$G^{(k)} = G_k G_{k-1} \cdots G_1$$

und mit

$$\tilde{R}_k = \tilde{H}_k^{(k)} = G^{(k)} \tilde{H}_k$$

sowie der rechten Seite

$$\tilde{b}_k = G^{(k)} \|d^0\| e_1 = \begin{pmatrix} \gamma_1^{(k)} \\ \gamma_2^{(k)} \\ \vdots \\ \vdots \\ \vdots \\ \gamma_{k+1}^{(k)} \end{pmatrix}$$

gilt

$$\min_{x \in x^0 + K_k(b - Ax^0, A)} \|b - Ax\|_2 = \min_{y \in \mathbb{R}^k} \|\|d^0\| e_1 - \tilde{H}_k y\|_2 = \min_{y \in \mathbb{R}^k} \|\tilde{b}_k - \tilde{R}_k y\|_2.$$

Dieses resultierende überbestimmte lineare Gleichungssystem ist einfach zu lösen, denn die Normalgleichung ist

$$R_k^T R_k y = R_k^T \tilde{b}_k \Leftrightarrow R_k y = \begin{pmatrix} \gamma_k^{(k)} \\ \vdots \\ \gamma_k^{(k)} \end{pmatrix},$$

da die letzte Zeile von $\tilde{R}_k$, also die letzte Spalte von $\tilde{R}_k^T$ null ist. Die Normalgleichung lässt sich durch Rückwärtseinsetzen von

$$R_k y = b_k$$

lösen, wobei sich $R_k \in \mathbb{R}^{k \times k}$ und $b_k \in \mathbb{R}^k$ jeweils aus $\tilde{R}_k$ bzw. aus $\tilde{b}_k$ durch Weglassen der letzten Zeile ergeben. Hiermit ist auch klar, dass für die Lösung y^k , entsprechend für $x^k = x^0 + V_k y^k$ gilt

$$\|b - Ax^k\|_2 = \|\tilde{b}_k - \tilde{R}_k y^k\|_2 = |\gamma^{(k+1)}|.$$

Das Residuum der aktuellen Approximation x^k ergibt sich somit im Laufe der Lösung des kleinen (wir gehen immer von $k \ll n$ aus) überbestimmten Gleichungssystems ohne dass y^k und damit x^k überhaupt bestimmt werden muss.

Wir gehen nun davon aus, dass für Iteration k die Zerlegung $\tilde{R}_k = G^{(k)} \tilde{H}_k$ sowie $\tilde{b}_k = G^{(k)} \|d^0\| e_1$ erstellt wurde. Springen wir dann für $i = k + 1$ in Algorithmus 6.4, so werden in Zeilen 4–6 die Einträge $h_{j,k+1}$ für $j = 1, \dots, k + 1$ und in Zeile 7 der Eintrag $h_{k+2,k+1}$ erstellt, also

$$\tilde{H}_{k+1} = \left(\begin{array}{c|c} \tilde{H}_k & \begin{matrix} h_{1,k+1} \\ \vdots \\ h_{k,k+1} \\ h_{k+1,k+1} \\ h_{k+2,k+1} \end{matrix} \\ \hline 0 & \end{array} \right).$$

Multiplikation von links mit der Matrix

$$\tilde{G}^{(k)} = \left(\begin{array}{c|c} G^{(k)} & \begin{matrix} 0 \\ \vdots \\ 0 \end{matrix} \\ \hline 0 & \dots & 0 & 1 \end{array} \right)$$

ergibt

$$\tilde{G}^{(k)} \tilde{H}_{k+1} = \left(\begin{array}{c|c} \tilde{R}_k & \begin{matrix} \tilde{h}_{1,k+1} \\ \vdots \\ \tilde{h}_{k,k+1} \\ \tilde{h}_{k+1,k+1} \\ \tilde{h}_{k+2,k+1} \end{matrix} \\ \hline 0 & \end{array} \right) = \left(\begin{array}{cccc} h_{11}^{(k)} & \dots & h_{1k}^{(k)} & \tilde{h}_{1,k+1} \\ 0 & \ddots & \vdots & \vdots \\ 0 & \ddots & h_{kk}^{(k)} & \tilde{h}_{k,k+1} \\ \vdots & \ddots & 0 & \tilde{h}_{k+1,k+1} \\ 0 & \dots & 0 & \tilde{h}_{k+2,k+1} \end{array} \right).$$

Die rechte Seite des überbestimmten linearen Gleichungssystems ist nur um einen Nulleintrag verlängert

$$\tilde{G}^{(k)} \|d^0\| \tilde{e}_1 = \begin{pmatrix} \gamma_1^{(k)} \\ \vdots \\ \gamma_{k+1}^{(k)} \\ 0 \end{pmatrix} \in \mathbb{R}^{k+2},$$

wobei wir mit $\tilde{G}^{(k)} \in \mathbb{R}^{k+2, k+1}$ die um eine Nullzeile verlängerte Matrix meinen und mit $\tilde{e}_1 \in \mathbb{R}^{k+2}$ den ersten Einheitsvektor.

Mit einer weiteren Givens-Rotation G_{k+1} mit den Koeffizienten

$$s_{k+1} = \frac{h_{k+2, k+1}}{\sqrt{h_{k+1, k+1}^2 + h_{k+2, k+1}^2}}, \quad c_{k+1} = \frac{h_{k+1, k+1}}{\sqrt{h_{k+1, k+1}^2 + h_{k+2, k+1}^2}},$$

ergibt sich das lineare Gleichungssystem

$$\tilde{R}_{k+1} = G_{k+1} \tilde{H}_{k+1} = \begin{pmatrix} h_{11}^{(k)} & \cdots & h_{1k}^{(k)} & h_{1, k+1}^{(k+1)} \\ 0 & \ddots & \vdots & \vdots \\ 0 & \ddots & h_{kk}^{(k)} & h_{k, k+1}^{(k+1)} \\ \vdots & \ddots & 0 & h_{k+1, k+1}^{(k+1)} \\ 0 & \cdots & 0 & 0 \end{pmatrix} \begin{pmatrix} y_1 \\ \vdots \\ \vdots \\ y_{k+2} \end{pmatrix} = \begin{pmatrix} \gamma_1^{(k)} \\ \vdots \\ \gamma_k^{(k)} \\ c_{k+1} \gamma^{(k)} \\ -s_{k+1} \gamma^{(k)} \end{pmatrix} =: \underbrace{\begin{pmatrix} \gamma_1^{(k+1)} \\ \vdots \\ \gamma_k^{(k+1)} \\ \gamma_{k+1}^{(k+1)} \\ \gamma_{k+2}^{(k+1)} \end{pmatrix}}_{=: \tilde{b}_{k+1}},$$

und das Residuum $\min_{x \in x^0 + K_{k+1}(b - Ax^0, A)} \|b - Ax\|_2 = |\gamma_{k+2}^{(k+1)}|$ lässt sich ablesen. Angenommen, die gewünschte Toleranz ist erreicht, so kann das Gleichungssystem gelöst werden und $y^{k+1} \in \mathbb{R}^{k+1}$ sowie $x^{k+1} = x^0 + V_{k+1} y^{k+1} \in \mathbb{R}^n$ wird bestimmt.

Wir fassen diese Herleitung in algorithmischer Form in Algorithmus 6.5 zusammen

Algorithmus 6.5: GMRES mit modifiziertem Gram-Schmidt

Eingabe: Matrix $A \in \mathbb{R}^{n \times n}$, die rechte Seite $b \in \mathbb{R}^n$ sowie ein Startwert $x^0 \in \mathbb{R}^n$.
Gewünschte Toleranz $\epsilon > 0$ sowie maximale Anzahl an Schritten $m \in \mathbb{N}$

```

1  $d^0 = b - Ax^0$  // berechne initiales Residuum
2  $\beta = \|d^0\|$ 
3  $v^1 = d^0/\beta$  // erster orthonormaler Vektor
4  $b^1 = \beta$  // erste rechte Seite
5 Für  $k = 1$  bis  $m$  tue
6    $w^k = Av^k$ 
7   Für  $j = 1$  bis  $k$  tue
8      $h_{jk} = (w^k, v^j)$ 
9      $w^k = w^k - h_{jk}v^j$ 
10     $h_{k+1,k} = \|w^k\|$ 
11    Wenn  $h_{k+1,k} = 0$  dann
12      Springe zu Zeile 25
13     $v^{k+1} = w^k/h_{k+1,k}$ 
14    Für  $j = 1$  bis  $k-1$  tue
15       $\begin{pmatrix} h_{jk} \\ h_{j+1,k} \end{pmatrix} = \begin{pmatrix} c_j & s_j \\ -s_j & c_j \end{pmatrix} \begin{pmatrix} h_{jk} \\ h_{j+1,k} \end{pmatrix}$  // alte Givens-Rotationen
16       $\alpha = \sqrt{h_{k,k}^2 + h_{k+1,k}^2}$  // Berechnung Givens-Rotation
17       $c_k = h_{k,k}/\alpha$ 
18       $s_k = h_{k+1,k}/\alpha$ 
19       $h_{k,k} = \alpha$  // Anwenden auf  $\tilde{H}_k$ 
20       $h_{k+1,k} = 0$ 
21       $z_{k+1} = -s_k z_k$  // Anwenden auf rechte Seite
22       $z_k = c_k z_k$ 
23      Wenn  $|z_{k+1}| < \epsilon$  dann
24        Springe zu Zeile 25
    // Abbruch der Orthonormalisierung, da Toleranz oder Anzahl der
    Schritte erreicht ist. Die Matrix  $h_{ij}$  für  $i, j = 1, \dots, k$  ist
    Dreiecksmatrix.
25  $y_k = z_k/h_{kk}$  // Berechnung von  $y \in \mathbb{R}^k$  mit Rückwärtseinsetzen
26 Für  $i = k-1$  bis  $1$  tue
27    $y_i = (z_i - \sum_{j=i+1}^k h_{ij}y_j)/h_{ii}$ 
28  $x^k = x^0 + \sum_{i=1}^k y_i v^i$  // Berechnung der Lösung
Ergebnis: Approximation  $x^k \in \mathbb{R}^n$  von  $x \in \mathbb{R}^n$  in  $Ax = b$ 

```

In Python können wir dies folgendermaßen Implementieren

♣ Implementierung 6.2: GMRES mit modifiziertem Gram-Schmidt

```

1 def gmres_verfahren(A, b, x, m, eps):
2     n = A.shape[0]
3     x = x.copy()
4     d = b.copy() - A.dot(x)

```

```

5     b = np.zeros(m + 1)
6     b[0] = np.linalg.norm(d)
7     v = np.zeros((n, m + 1))
8     h = np.zeros((m + 1, m))
9     R = np.zeros((m, 2, 2))
10    v[:, 0] = d / b[0]
11    for k in range(m):
12        w = A.dot(v[:, k])
13        for j in range(k + 1):
14            h[j, k] = np.dot(w, v[:, j])
15            w -= h[j, k] * v[:, j]
16        h[k + 1, k] = np.linalg.norm(w)
17        if np.abs(h[k + 1, k]) > 1e-12:
18            v[:, k + 1] = w / h[k + 1, k]
19            for j in range(k):
20                h[j:j + 2, k] = R[j].dot(h[j:j + 2, k])
21            alpha = np.linalg.norm(h[k:k + 2, k])
22            R[k, 0, 0] = h[k, k] / alpha
23            R[k, 0, 1] = h[k + 1, k] / alpha
24            R[k, 1, 1], R[k, 1, 0] = R[k, 0, 0], -R[k, 0, 1]
25            h[k:k + 2, k] = alpha, 0
26            b[k + 1] = R[k, 1, 0] * b[k]
27            b[k] = R[k, 0, 0] * b[k]
28            if np.abs(b[k + 1]) < eps:
29                break
30        else:
31            break
32    y = np.zeros(k + 1)
33    y[-1] = b[-1] / h[k, k]
34    for i in range(k - 2, -1, -1):
35        y[i] = (b[i] - h[i, i + 1:k + 1].dot(y[i + 1:])) / h[i, i]
36    for i in range(k):
37        x += y[i] * v[:, i]
38    return x

```

Diese Variante des GMRES-Verfahrens stoppt nach m Schritten, oder wenn das Residuum erreicht wird. Die eigentliche Approximation x^k wird erst dann berechnet, wenn klar ist, dass das Residuum klein genug ist. Der wesentliche Aufwand des Algorithmus liegt weiter bei $O(k)$ Matrix-Vektor-Produkten mit der Matrix A , sowie $O(k^2)$ Skalarprodukten im $\mathbb{R}^n$. Die Anwendung der Givens-Rotationen sowie die Berechnung der Minimierungslösung erfolgt in $O(k^2)$ Operationen und spielt bei $k \ll n$ keine Rolle. Bei dünn besetzten Matrizen verhält sich der Aufwand also wie $O(k^2n)$.

Tab. 6.3 Ergebnisse für das GMRES-Verfahren, angewandt auf die Modellmatrix

n	# Schritte	Zeit	Residuum	Fehler
400	34	0.0069	$2.95 \cdot 10^{-6}$	$1.54 \cdot 10^{-6}$
900	53	0.0248	$2.18 \cdot 10^{-6}$	$2.50 \cdot 10^{-6}$
1600	71	0.0913	$2.43 \cdot 10^{-6}$	$5.57 \cdot 10^{-6}$
2500	90	0.2866	$1.77 \cdot 10^{-6}$	$6.10 \cdot 10^{-6}$
3600	109	0.6605	$1.41 \cdot 10^{-6}$	$6.94 \cdot 10^{-6}$
4900	127	1.9946	$1.47 \cdot 10^{-6}$	$1.11 \cdot 10^{-5}$
6400	146	4.0009	$1.25 \cdot 10^{-6}$	$1.24 \cdot 10^{-5}$
8100	164	7.1323	$1.26 \cdot 10^{-6}$	$1.69 \cdot 10^{-5}$
10000	182	11.8184	$1.25 \cdot 10^{-6}$	$2.15 \cdot 10^{-5}$

Beispiel 6.21

Wir betrachten wieder die Modellmatrix $A \in \mathbb{R}^{n \times n}$ aus Beispielen 3.51, 6.7 und 6.10, und wenden unsere Implementierung des GMRES-Verfahrens hierauf an. Dabei nehmen wir $m = 20, 30, \dots, 100$ und die Toleranz $\epsilon = 10^{-6}$. Die Anzahl der notwendigen GMRES-Schritte, die Rechenzeit, das finale Residuum sowie der Fehler der Lösung finden wir in Tab. 6.3. Wir sehen dabei, dass die Anzahl der notwendigen Schritte in diesem Beispiel vergleichbar zum CG-Verfahren ist. Die Rechenzeit ist zwar etwa doppelt so groß, dafür lässt sich das GMRES-Verfahren auch auf allgemeine Matrizen anwenden. ◀

6.5.1.3 Restarted GMRES und truncated GMRES

Für große k kann der Aufwand zu schnell ansteigen und dies betrifft das dritte der eingangs angesprochenen Probleme des Verfahrens. Hinzu kommt, dass der Fehler bei der Orthonormalisierung für größere k ebenso steigen wird. Daher werden oft verkürzte Varianten von GMRES verwendet. Z.B. kann die Iteration nach einer festen Zahl von Schritten $m \in \mathbb{N}$ ganz neu gestartet werden. Dabei wird als Startwert $x^{(0)}$ die im letzten Durchgang berechnete Approximation gewählt. Alternativ wird die Orthonormalisierung in Zeilen 7–9 von Algorithmus 6.5 nur über die jeweils letzten m Vektoren durchgeführt, d.h., diese Zeilen werden durch

```
7 Für  $j = \max(1, k - m)$  bis  $k$  tue
8    $h_{jk} = (w^k, v^j)$ 
9    $w^k = w^k - h_{jk} v^j$ 
```

ersetzt. Die erste Variante wird *restarted GMRES* genannt und die zweite Variante *truncated GMRES*. Wir verweisen auf die Literatur [78].

6.5.2 Vorkonditioniertes GMRES-Verfahren

Ebenso wie beim CG-Verfahren hängt die Konvergenz des GMRES von der Konditionszahl der Matrix A ab und durch Vorkonditionierung kann die Konvergenz teilweise erheblich beschleunigt werden. Da GMRES nicht notwendigerweise symmetrisch positiv definite Matrizen erfordert, besteht mehr Freiheit in der Wahl von Vorkonditionieren.

Es sei also $P \in \mathbb{R}^{n \times n}$ der Vorkonditionierer mit $P^{-1} \approx A^{-1}$. Wir transformieren das zu lösende lineare Gleichungssystem in die Form

$$P^{-1}Ax = P^{-1}b.$$

Dies entspricht der Links-Vorkonditionierung, die wir auch beim CG-Verfahren betrachtet haben. Um auf dieses System die GMRES-Iteration anwenden zu können, müssen nur einige wenige Zeilen von Algorithmus 6.5 modifiziert werden. Der initiale Defekt in Zeile 1 wird bestimmt als

$$Pd^0 = b - Ax^0$$

und der jeweils neu zu orthogonalisierende Vektor in Zeile 6 muss als

$$Pw^k = Av^k$$

berechnet werden. Pro Iteration ist eine Anwendung des Vorkonditionierers notwendig.

Da die vorkonditionierte GMRES Iteration der Minimierung von $\|P^{-1}(b - Ax)\|$ entspricht, wird in Zeile 21 auch das vorkonditionierte Residuum $|z_{k+1}| = \|P^{-1}(b - Ax^k)\|$ bestimmt. Dieses kann sich erheblich vom eigentlichen Residuum $\|b - Ax^k\|$ unterscheiden. Um sicher zu stellen, dass das Residuum wirklich hinreichend minimiert wird sollte nach Lösen des Gleichungssystems in Zeilen 26–28 eine Berechnung von $b - Ax^k$ erfolgen.

Wie auch beim CG-Verfahren, kann das GMRES-Verfahren „von rechts“ vorkonditioniert werden, d. h., man betrachtet das lineare Gleichungssystem

$$AP^{-1}\hat{x} = b, \quad Px = \hat{x}.$$

Diese Variante hat den Vorteil, dass das im Laufe des Verfahrens berechnete Residuum $|z_{k+1}|$ wirklich dem Residuum des eigentlichen linearen Gleichungssystems entspricht. Zweitens erlaubt die Rechts-Vorkonditionierung den sogenannten FMGRES (engl. flexible GMRES), wo der Vorkonditionierer P in jedem Iterationsschritt neu angepasst werden kann. Beispielsweise ist dies der Fall, wenn keine feste Matrix P gewählt wird, sondern ein anderes numerisches Verfahren, wie z. B. ein weiteres Iterationsverfahren, sodass ein verschachteltes Gesamtverfahren entsteht. Wir verweisen auf die Literatur [78].

Teil II

Numerische Methoden der Analysis

Die Analysis befasst sich mit Folgen und Reihen sowie mit Funktionen im Raum der reellen Zahlen $\mathbb{R}$. Dabei geht es um Begriffe wie Konvergenz, Approximation, Stetigkeit, Differenzierbarkeit und Integration. Numerische Anwendungen der Analysis werden sich im Wesentlichen mit der konkreten Berechnung und Bestimmung dieser Größen befassen. Der Zwischenwertsatz sagt etwa, dass die Funktion

$$f(x) = x(1 + \exp(x)) + 10 \sin(3 + \log(x^2 + 1))$$

im Intervall $[-10, 2]$ mindestens eine Nullstelle hat. Denn es gilt

$$x > 2 \Rightarrow f(x) > 0, \quad x < -10 \Rightarrow f(x) < 0.$$

Die konkrete Berechnung dieser oder möglicherweise mehrerer dieser Nullstellen hingegen kann schon bei sehr einfachen Funktionen äußerst aufwendig sein. Hier kommt die numerische Mathematik ins Spiel. Ein weiteres Beispiel ist die Berechnung von Integralen. Aus der Stetigkeit der Funktion f können wir folgern, dass das Integral

$$I_{ab}(f) = \int_a^b f(x) \, dx$$

existiert, dessen explizite Berechnung „per Hand“ ist in vielen Fällen herausfordernd. Analytische Hilfsmittel schlagen oft fehl.

In den numerischen Methoden der Analysis werden wir uns mit Lösungsverfahren für Probleme wie *dem Finden von Nullstellen, numerischen Iterationsverfahren für lineare Gleichungssysteme* sowie *Interpolation und Approximation* befassen. Insbesondere das Nullstellenkapitel ist im weitesten Sinne zu verstehen. Die Lösungen von beliebigen Gleichungen in einer und höheren Dimensionen sowie Gleichungssysteme können immer auf Nullstellenprobleme zurückgeführt werden.

Wie bereits in Teil I (Numerische Methoden der Linearen Algebra) wird die Numerik im Allgemeinen nicht in der Lage sein, exakte Lösungen zu liefern, sondern meistens Näherungen an die Lösung angeben. Das heißt, es werden nicht nur die approximativen Lösungen explizit berechnet, sondern auch deren „Abstand“, in einem geeignetem Fehlermaß hinsichtlich der (unbekannten) exakten Lösung. Dies führt dann letztlich auf die Fragestellung, wie schnell die numerische Lösung zufriedenstellende Ergebnisse liefert, was auf die Analyse der Konvergenzgeschwindigkeit führt. Analog zu Teil I werden wir unsere Resultate anhand der *Konzepte* klassifizieren. Darüber hinaus werden wieder alle Themen mit zahlreichen Beispielen und Exkursen veranschaulicht.

Im zweiten Teil des Buches befassen wir uns mit numerischen Algorithmen zum Lösen von Problemen der Analysis. Hierzu setzen wir intensiv entsprechende analytische Methoden ein, um die numerischen Verfahren zu analysieren. Zentral ist der Begriff der Konvergenz einer Folge $(x_k)_{k \in \mathbb{N}}$, $x_k \in \mathbb{R}^n$, für $n \in \mathbb{N}$, gegen einen Grenzwert $z \in \mathbb{R}^n$. In der Analysis ist dies zumeist ein qualitativer Begriff der bedeutet, dass es zu jedem $\epsilon > 0$ ein $k_0 \in \mathbb{N}$ gibt, so dass

$$\|x_k - z\| \leq \epsilon \quad \forall k \geq k_0.$$

Folgen treten in der numerischen Mathematik z. B. als Zwischenlösungen in iterativen Verfahren auf. Um verschiedene numerische Verfahren vergleichen zu können werden wir in Kap. 8 die Begriffe der *Konvergenzgeschwindigkeit* und *Konvergenzordnung* einführen.

Eine der großen Aufgaben ist die Suche nach Nullstellen. D. h., zu einer gegebenen Funktion $f : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$ suchen wir einen Punkt $z \in D$ im Definitionsbereich D mit $f(z) = 0$. Für skalare Funktionen gibt der *Zwischenwertsatz* einen ersten Hinweis auf die Existenz einer Nullstelle:

Satz 7.1 (Zwischenwertsatz) Es sei $f : [a, b] \rightarrow \mathbb{R}$ eine stetige Funktion mit einem Vorzeichenwechsel im Intervall, d. h. $f(a)f(b) < 0$. Dann existiert mindestens eine Nullstelle $z \in (a, b)$ mit $f(z) = 0$.

Beweis Im Beweis konstruieren wir Folgen von Intervallen $[a_k, b_k]$, welche stets eine Nullstelle einklammern $a_k \leq z \leq b_k$, die im Fall von a_k monoton wachsend und im Fall von b_k monoton fallen und für deren Intervalllänge $b_k - a_k \rightarrow 0$ gilt.

Ohne Einschränkung sei $a_0 := a < b =: b_0$ und es gelte nach Voraussetzung $f(a_0)f(b_0) < 0$. Wir konstruieren für $k = 1, 2, \dots$ die Folge der Mittelpunkte

$$c_k := \frac{a_{k-1} + b_{k-1}}{2} \in (a_{k-1}, b_{k-1}).$$

Im Fall $f(c_k) = 0$ ist $z = c_k$ die gesuchte Nullstelle und das Verfahren bricht ab. Ansonsten wählen wir

$$a_k := \begin{cases} a_{k-1} & f(a_{k-1})f(c_k) < 0 \\ c_k & f(a_{k-1})f(c_k) > 0 \end{cases}, \quad b_k := \begin{cases} c_k & f(a_{k-1})f(c_k) < 0 \\ b_{k-1} & f(a_{k-1})f(c_k) > 0. \end{cases}$$

In beiden Fällen gilt $f(a_k)f(b_k) < 0$, d. h., im Intervall liegt wieder ein Vorzeichenwechsel vor.

Weiter gilt wegen $c_k \in (a_{k-1}, b_{k-1})$ nach Konstruktion, dass $a_k \geq a_{k-1}$ und $b_k \leq b_{k-1}$. Schließlich gilt

$$b_k - a_k = \frac{b_{k-1} - a_{k-1}}{2} = \frac{(b - a)}{2^k} \rightarrow 0 \quad (k \rightarrow \infty).$$

Die Folge c_k ist also nach oben und unten beschränkt und es gilt

$$\lim_{k \rightarrow \infty} a_k = \lim_{k \rightarrow \infty} b_k = \lim_{k \rightarrow \infty} c_k = z$$

sowie

$$f(a_k)f(b_k) \leq 0 \Rightarrow \lim_{k \rightarrow \infty} f(a_k)f(b_k) = f(z)^2 \leq 0 \Rightarrow f(z) = 0. \quad \square$$

Der Beweis lässt sich direkt als ein numerisches Verfahren zur Nullstellensuche umsetzen, dem Intervallhalbierungsverfahren, welches wir in Abschn. 8.2 untersuchen. Der Satz lässt sich nicht auf den höherdimensionalen Fall übertragen. Hier werden wir andere Hilfsmittel kennenlernen um Nullstellen zu charakterisieren und zu finden.

Weitere zentrale Aspekte der Analysis sind die Differentiation und Integration von Funktionen. Diese Themen werden wir in Abschn. 9.3 und Kap. 10 behandeln. Ein erstes Verfahren zur numerischen Berechnung von Ableitungen ist der Differenzenquotient

$$D_h f(x) := \frac{f(x+h) - f(x)}{h} \approx f'(x). \quad (7.1)$$

Die Aussage des Mittelwertsatzes der Differentialrechnung ist es, dass der Differenzenquotient mit der Ableitung in einer Zwischenstelle übereinstimmt:

Satz 7.2 (Mittelwertsatz der Differentialrechnung) Die Funktion $f : [a, b] \rightarrow \mathbb{R}$ sei eine stetig differenzierbare Funktion. Dann gibt es mindestens ein $\xi \in [a, b]$ so dass

$$f'(\xi) = \frac{f(b) - f(a)}{b - a}.$$

Ein Spezialfall des Mittelwertsatzes ist der Satz von Rolle

Satz 7.3 (Satz von Rolle) Gilt im Mittelwertsatz zusätzlich $f(a) = f(b)$, dann existiert mindestens ein $\xi \in (a, b)$, sodass $f'(\xi) = 0$.

Der Mittelwertsatz hat auch ein entsprechendes Gegenstück in der Integralrechnung.

Satz 7.4 (Mittelwertsatz der Integralrechnung) Die Funktion $f : [a, b] \rightarrow \mathbb{R}$ sei stetig. Dann gibt es ein $\xi \in [a, b]$ so dass

$$\int_a^b f(x) \, dx = f(\xi) \cdot (b - a).$$

Systematischer kann der Fehler bei der Approximation der Ableitung mit dem Differenzenquotienten $D_h f(x) \approx f'(x)$, siehe (7.1), mit der Taylor-Entwicklung einer Funktion untersucht werden.

Satz 7.5 (Satz von Taylor) Es sei $f \in C^{r+1}[a, b]$ mit $r \geq 0$. Dann gilt für alle $x, x_0 \in [a, b]$ die Taylor-Approximation mit differenziellem Restglied

$$f(x) = \sum_{l=0}^r \frac{f^{(l)}(x_0)}{l!} (x - x_0)^l + \frac{f^{(r+1)}(\xi_{x,x_0})}{(r+1)!} (x - x_0)^{r+1},$$

wobei $\xi_{x,x_0} \in [\min(x, x_0), \max(x, x_0)]$ ein Zwischenwert ist. Weiter gilt die Taylor-Approximation mit integralem Restglied

$$f(x) = \sum_{l=0}^r \frac{f^{(l)}(x_0)}{l!} (x - x_0)^l + \frac{1}{r!} \int_{x_0}^x (x - x_0)^r f^{(r+1)}(t) \, dt.$$

Für $r + 1$ mal stetig differenzierbare Funktionen $f : X \subset \mathbb{R}^n \rightarrow \mathbb{R}$ gilt für alle $x, x_0 \in X$

$$f(x) = \sum_{|\alpha|=0}^r \frac{1}{|\alpha|!} D^\alpha f(x_0) (x - x_0)^\alpha + \sum_{|\alpha|=r+1} \frac{1}{|\alpha|!} D^\alpha f(\xi) (x - x_0)^\alpha$$

mit einem Zwischenwert $\xi \in X$. Dabei verwenden wir die *Multiindex-Schreibweise*

$$\alpha \in \mathbb{N}^n, \quad |\alpha| = \alpha_1 + \cdots + \alpha_n, \quad x^\alpha = x_1^{\alpha_1} \cdots x_n^{\alpha_n}, \quad D^\alpha = \frac{\partial^{\alpha_1}}{\partial x_1} \cdots \frac{\partial^{\alpha_n}}{\partial x_n}.$$

Beweise finden sich z. B. in [60, Kap. 14] und [4, Kapitel IV]. Hiermit erreichen wir direkt eine Fehlerabschätzung für den Differenzenquotienten (7.1), falls die Funktion f zweimal stetig differenzierbar ist, also $D_h f(x) = f'(x) + \frac{h}{2} f'(\xi)$.

Der Satz von Taylor, Satz 7.5, beschreibt einen lokalen Weg zur Approximation von allgemeinen (differenzierbaren) Funktionen mit Taylor-Polynomen. Die Approximationstheorie ist ein weiterer Schwerpunkt der numerischen Analysis: gegeben sei eine Funktion f und gesucht ist eine einfache Funktion p , z. B. ein Polynom oder eine stückweise lineare Funktion, die f möglichst gut approximiert. Eine theoretische Grundlage bietet der *Weierstraßsche Approximationssatz*.

Satz 7.6 (Weierstraßscher Approximationssatz) Es sei $f \in C[a, b]$. Dann gibt es zu jedem $\varepsilon > 0$ ein auf $[a, b]$ definiertes Polynom p , sodass

$$|f(x) - p(x)| < \varepsilon \quad \forall x \in [a, b].$$

Für einen Beweis verweisen wir auf die Literatur [60]. Der Weierstraßsche Approximationssatz besagt, dass es möglich ist, jede stetige Funktion beliebig gut durch ein Polynom zu approximieren. Der Beweis wird konstruktiv geführt mit einer Folge von *Bernsteinpolynomen*, die gegen die Funktion f konvergieren. Diese Konvergenz ist jedoch sehr langsam und in der Realisierung aufwändig und führt nicht zu effizienten numerischen Approximationsmethoden. Der Satz von Stone-Weierstraß verallgemeinert das Resultat derart, dass der Raum der für die Approximation verwendeten Funktionen viel weiter gefasst wird. Die *Fourier-Analyse* wählt zur Approximation einer Funktion f die trigonometrischen Polynome. Wir geben eine einfache Version:

Satz 7.7 (Fourier-Analyse) Es sei $f \in C(\mathbb{R})$ eine 2π -periodische Funktion, d. h. $f(x) = f(x + 2\pi)$ für alle $t \in \mathbb{R}$. Dann konvergiert die Fourier-Summe

$$F_n^f(x) = \frac{1}{2}a_0 + \sum_{k=1}^n (a_k \cos(kx) + b_k \sin(kx))$$

mit den Koeffizienten

$$a_0 = \frac{1}{\pi} \int_0^{2\pi} f(x) dx, \quad a_k = \frac{1}{\pi} \int_0^{2\pi} f(x) \cos(kx) dx, \quad b_k = \frac{1}{\pi} \int_0^{2\pi} f(x) \sin(kx) dx$$

im quadratischen Mittel gegen die Funktion f , d. h.

$$\|f - F_n^f\|_{L^2[0, 2\pi]}^2 = \int_0^{2\pi} |f(x) - F_n^f(x)|^2 dx \rightarrow 0 \quad (n \rightarrow \infty).$$

Für einen Beweis verweisen wir wieder auf die Literatur [50]. Im Gegensatz zur Taylor-Approximation erfordert die Fourier-Analyse keine Differenzierbarkeit der Funktion f .

Algorithmisch von großer Bedeutung ist die *diskrete Fourier-Analyse*, bei der diskrete Zahlenwerte $a_0, \dots, a_n$, die als Messwerte eines periodischen Signals verstanden werden, und in Form von trigonometrischen Polynomen dargestellt werden. Dies behandeln wir in Abschn. 9.7.

Von großer Bedeutung für die algorithmische Herleitung und Analyse von numerischen Iterationsverfahren ist das Konzept von Fixpunkt-Iterationen.

► **Definition 7.8 (Fixpunkt)** Es sei $g : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Ein Punkt $x \in \mathbb{R}^n$ heißt Fixpunkt von g , falls $g(x) = x$.

Viele Algorithmen sind in Form von Fixpunktiterationen formuliert, d.h., ausgehend von einem Startwert $x^{(0)} \in \mathbb{R}^n$ wird eine Folge

$$x^{(k)} = g(x^{(k-1)}) \quad k = 1, 2, \dots$$

konstruiert und auf Konvergenz untersucht. Wir definieren:

► **Definition 7.9 (Lipschitz-Stetigkeit, Kontraktion)** Es sei $G \subset \mathbb{R}^n$ eine nichtleere Menge. Eine Abbildung $g : G \rightarrow \mathbb{R}^n$ wird Lipschitz-stetig genannt, falls

$$\|g(x) - g(y)\| \leq L\|x - y\|, \quad x, y \in G,$$

mit $L > 0$. Falls $0 < L < 1$, so nennt man g eine Kontraktion auf G .

Jede auf einer beschränkten Menge differenzierbare Funktion ist Lipschitz-stetig und jede Lipschitz-stetige Funktion ist (gleichmäßig) stetig. Ein Beispiel für eine gleichmäßig stetige aber nicht Lipschitz-stetig Funktion auf $[0, 1]$ ist die Wurzelfunktion $f(x) = \sqrt{x}$.

Der Banachsche Fixpunktsatz besagt nun, dass jede Selbstabbildung $g : G \rightarrow G$, welche eine Kontraktion ist, einen Fixpunkt besitzt:

Satz 7.10 (Banachscher Fixpunktsatz) Es sei $G \subset \mathbb{R}^n$ eine nichtleere und abgeschlossene Menge und $g : G \rightarrow G$ eine Lipschitz-stetige Selbstabbildung mit Lipschitz-Konstante $q < 1$ (also eine Kontraktion). Dann existiert genau ein Fixpunkt $z \in G$ von g und die Iterationsfolge $(x^{(k)})_k$ konvergiert für jeden Startpunkt $x^{(0)} \in G$ gegen diesen Fixpunkt.

Es gelten die a-priori-Abschätzung

$$\|x^{(k)} - z\| \leq \frac{q^{(k)}}{1 - q} \|x^{(1)} - x^{(0)}\|$$

sowie die a-posteriori-Abschätzung

$$\|x^{(k)} - z\| \leq \frac{q}{1-q} \|x^{(k)} - x^{(k-1)}\|.$$

Beweis Obwohl der Banachsche Fixpunktsatz in jeder Analysis-Vorlesung behandelt werden sollte [60], geben wir hier den Beweis, da dieser Fehlerabschätzungen enthält, die auch numerisch relevant sind.

(i) *Existenz eines Grenzwertes.* Da g eine Selbstabbildung in G ist, sind alle Iterierten $x^{(k)} = g(x^{(k-1)}) = \dots = g^{(k)}(x^{(0)})$ bei Wahl eines beliebigen Startpunkts $x^{(0)} \in G$ definiert. Aufgrund der Kontraktionseigenschaft gilt

$$\begin{aligned} \|x^{(k+1)} - x^{(k)}\| &= \|g(x^{(k)}) - g(x^{(k-1)})\| \\ &\leq q \|x^{(k)} - x^{(k-1)}\| \leq \dots \leq q^{(k)} \|x^{(1)} - x^{(0)}\|. \end{aligned}$$

Wir zeigen, dass $(x^{(k)})_k$ eine Cauchy-Folge ist. Für jedes $l \geq m$ erhalten wir

$$\begin{aligned} \|x^{(l)} - x^{(m)}\| &\leq \|x^{(l)} - x^{(l-1)}\| + \dots + \|x^{(m+1)} - x^{(m)}\| \\ &\leq (q^{l-1} + q^{l-2} + \dots + q^m) \|x^{(1)} - x^{(0)}\| \\ &= q^m \frac{1 - q^{l-m}}{1 - q} \|x^{(1)} - x^{(0)}\| \\ &\leq q^m \frac{1}{1 - q} \|x^{(1)} - x^{(0)}\| \rightarrow 0 \quad (l \geq m \rightarrow \infty). \end{aligned} \tag{7.2}$$

Das heißt, $(x^{(l)})_{l \in \mathbb{N}}$ ist eine Cauchy-Folge. Da alle Folgenglieder in G liegen und G als abgeschlossene Teilmenge des $\mathbb{R}^n$ vollständig ist, existiert der Grenzwert $x^{(l)} \rightarrow z \in G$.

(ii) *Fixpunkteigenschaft.* Als Zweites weisen wir nach, dass z tatsächlich ein Fixpunkt von g ist. Aus der Stetigkeit von g folgt mit $x^{(k)} \rightarrow z$ auch $g(x^{(k)}) \rightarrow g(z)$. Dann gilt für die Iteration $x^{(k+1)} := g(x^{(k)})$ bei Grenzübergang

$$z \leftarrow x^{(k+1)} = g(x^{(k)}) \rightarrow g(z) \quad (k \rightarrow \infty).$$

(iii) *Eindeutigkeit.* Die Eindeutigkeit folgt aus der Kontraktionseigenschaft. Es seien z und $\hat{z}$ zwei Fixpunkte von g . Dann ist

$$\|z - \hat{z}\| = \|g(z) - g(\hat{z})\| \leq q \|z - \hat{z}\|.$$

Dies kann wegen $q < 1$ nur dann gültig sein, wenn $\|z - \hat{z}\| = 0$, d. h. $z = \hat{z}$ ist. Damit ist der Fixpunkt eindeutig.

(iv) *A-priori-Fehlerabschätzung.* Es gilt mit (7.2)

$$\|z - x^{(m)}\| \leq \lim_{l \rightarrow \infty} \|x^{(l)} - x^{(m)}\| \leq q^m \frac{1}{1 - q} \|x^{(1)} - x^{(0)}\|.$$

(v) *A-posteriori-Fehlerabschätzung.* Es gilt wieder mit (7.2)

$$\|x^{(m)} - z\| \leq q \frac{1}{1-q} \|x^{(m)} - x^{(m-1)}\|. \quad \square$$

Jede stetig differenzierbare Funktion ist auch Lipschitz-stetig und die Ableitung kann als eine Schranke für die Lipschitz-Konstante herangezogen werden.

Satz 7.11 (Schränkensatz) Die Abbildung $g : G \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$ sei stetig differenzierbar auf der konvexen und offenen Menge G . Es sei

$$L := \sup_{\xi \in G} |Dg(\xi)| < \infty.$$

Dann gilt

$$\|g(x) - g(y)\| \leq L \|x - y\|, \quad x, y \in G,$$

mit der Jacobi-Matrix

$$Dg(x) = \left(\frac{\partial g_i}{\partial x_j} \right)_{i,j=1,\dots,n} \in \mathbb{R}^{n \times n}.$$

Falls $\sup_{\xi \in G} \|Dg(\xi)\| < 1$, dann ist g eine Kontraktion auf G . Insbesondere gilt im skalaren Fall auf dem Intervall $G := [a, b]$

$$q := \max_{\xi \in [a,b]} |Dg(\xi)|.$$

Eine weitere klassische Aufgabe der Analysis ist die Suche nach Minima und Maxima von stetigen Funktionen $f : G \subset \mathbb{R}^n \rightarrow \mathbb{R}$.

► **Definition 7.12 (Minimierung)** Es sei $f : X \rightarrow \mathbb{R}$ mit $X \subset \mathbb{R}^n$ eine stetige Funktion. Ein Punkt $x \in \mathbb{R}^n$ mit $x \in X$ heißt *zulässiger Punkt*. $x \in X$ heißt *lokales Minimum*, falls es ein $\epsilon > 0$ gibt, so dass

$$f(x) \leq f(y) \quad \forall y \in B_\epsilon(x) := \{z \in \mathbb{R}^n \cap X, \|z - x\| < \epsilon\},$$

er heißt *striktes oder isoliertes Minimum* falls es ein $\epsilon > 0$ gibt, so dass

$$f(x) < f(y) \quad \forall y \in B_\epsilon(x) := \{z \in \mathbb{R}^n \cap X, \|z - x\| < \epsilon\},$$

er heißt *globales Minimum* falls

$$f(x) \leq f(y) \quad \forall y \in X,$$

und *globales striktes Minimum* falls

$$f(x) < f(y) \quad \forall y \in X.$$

Im skalaren Fall $f : \mathbb{R} \rightarrow \mathbb{R}$ gibt der folgende Satz die Bedingung für die Existenz von Minima und Maxima.

Satz 7.13 (Satz vom Minimum und Maximum (in $\mathbb{R}$)) Es sei $[a, b]$ ein Intervall und $f : [a, b] \rightarrow \mathbb{R}$ eine stetige Funktion. Dann ist f beschränkt und nimmt auf $[a, b]$ das Minimum und Maximum an. D.h. es gibt zwei Werte $x_{\min}, x_{\max} \in [a, b]$ mit

$$f(x_{\min}) \leq f(x) \leq f(x_{\max}) \quad \forall x \in [a, b].$$

Beweis (i) Wir nennen Y den Wertebereich von f auf $[a, b]$. Dann existiert zu jeder Folge $y_k \in Y$ eine Folge $x_k \in [a, b]$ mit $y_k = f(x_k)$. Da $x_k \in [a, b]$ beschränkt ist existiert eine Teilfolge die konvergiert, $x_{k'} \rightarrow x^*$. Da $[a, b]$ abgeschlossen ist gilt $x^* \in [a, b]$ und wegen der Stetigkeit von f also $y^* := f(x^*) \in M$.

(ii) Wir zeigen nun, dass f (also Y) beschränkt sein muss. Angenommen, f ist nicht nach unten beschränkt. Dann existiert eine divergente Folge $y_k \in Y$. Es sei $x_k \in [a, b]$ die entsprechende Folge mit $y_k = f(x_k)$. Da y_k divergiert, muss auch jede Teilfolge $y_{k'}$ divergieren. Dies ist jedoch nach (i) ausgeschlossen, da $[a, b]$ beschränkt ist. Also ist f nach unten beschränkt durch

$$-\infty < f_{\min} := \inf_{x \in [a, b]} f(x).$$

Mit entsprechender Argumentation können wir die obere Schranke $f_{\max} < \infty$ herleiten.

(iii) Nun sei $y_k \in Y$ eine Folge, die gegen $f_{\min}$ konvergiert. Entsprechend ist $x_k \in [a, b]$ mit $y_k = f(x_k)$. Diese Folge hat eine konvergente Teilfolge $x_{k'} \rightarrow x_{\min}$ und für den Grenzwert gilt $f(x_{k'}) \rightarrow f_{\min} = f(x_{\min})$. Das Minimum wird also in $x_{\min}$ angenommen. $\square$

Im Gegensatz zum Beweis des Zwischenwertsatzes, Satz 7.1, ist dieser Beweis nicht konstruktiv, da wir mehrfache Teilfolgen auswählen. Der Beweis lässt sich nicht in einem Algorithmus umsetzen. Auch sagt der Satz nichts über die Eindeutigkeit des Minimums voraus. Existieren zwei Punkte x, x' , an denen jeweils $f(x) = f(x') = f_{\min}$ gilt, so sagt der Beweis nichts darüber, welcher dieser beiden Punkte mit der Folge erreicht wird.

Das Problem, das Minimum oder Maximum einer Funktion zu finden ist weit schwerer als die Suche nach Nullstellen und es gibt im allgemeinen Fall (d.h., wenn nicht zu starke Einschränkungen an die zu untersuchende Funktion gestellt werden) keine Algorithmen, die gleichzeitig effizient sind und verlässlich das Minimum identifizieren. Dies gilt insbesondere für globale Minima und Maxima. Einfacher wird es, wenn nur lokale Minima und Maxima gesucht werden und wenn zusätzliche Eigenschaften der Funktion bekannt sind. Im Folgenden stellen wir verschiedene *Kriterien für Minima und Maxima* vor. Das sind Bedingungen, die über die Stetigkeit hinausgehen, und zur Charakterisierung und Suche von Extrempunkten dienen. Das einfachste Kriterium kann angewendet werden, wenn die

Funktion f differenzierbar ist. Wir behandeln sofort den allgemeinen Fall höherdimensionaler Funktionen.

Satz 7.14 (Notwendiges Kriterium für Minima und Maxima) Es sei $f : X \subset \mathbb{R}^n \rightarrow \mathbb{R}$ auf der offenen Menge X differenzierbar und habe im Punkt $x_0 \in (a; b)$ ein (lokales) Minimum oder ein (lokales) Maximum. Dann gilt $\nabla f(x_0) = 0$.

► **Definition 7.15 (Stationäre Punkte)** Einen Punkt $x \in X \subset \mathbb{R}^n$ mit $\nabla f(x) = 0$ nennen wir *stationären Punkt*. Ein stationärer Punkt, der weder Minimum noch Maximum ist, heißt *Sattelpunkt*.

Es handelt sich also um ein notwendiges aber noch nicht um ein hinreichendes Kriterium. Ein stationärer Punkt muss nicht unbedingt Minimum oder Maximum sein.

Satz 7.16 (Hinreichendes Kriterium für Minima und Maxima) Es sei $f : X \subset \mathbb{R}^n \rightarrow \mathbb{R}$ zweimal stetig differenzierbar und im Punkt $x_0 \in X$ gelte das notwendige Kriterium $\nabla f(x_0) = 0$. Dann hat f in x_0 ein Minimum, genau dann, wenn die Hessematrix $H_f(x_0)$ positiv definit ist und ein Maximum, genau dann, wenn $H_f(x_0)$ negativ definit ist.

Weitere Details und Beweise finden sich z. B. in [87].

Das Lösen von nichtlinearen Gleichungen spielt eine grundlegende Rolle in der Mathematik. Oft werden solche Aufgabenstellungen als Nullstellenproblem formuliert. Die Nullstellenbestimmung von linearen und quadratischen Polynomen ist das einfachste Beispiel und bereits aus der Schule bekannt. Insbesondere können hier geschlossene Formeln, etwa die p/q -Formel, angegeben werden. Aber schon bei Gleichungen fünften Grades gibt es keine solche geschlossene Lösungsformel.

Wir betrachten im Folgenden eine allgemeine Funktion

$$f : D(f) \rightarrow \mathbb{R}$$

und suchen die (oder mehrere) Nullstellen $z \in D(f)$ im Definitionsbereich. Jede nichtlineare Gleichung $N(x) = b$ kann mittels $f(x) = N(x) - b$ in ein Nullstellenproblem überführt werden.

► **Definition 8.1 (Nullstelle)** Es sei $f : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^m$ eine stetige Funktion. Dann heißt ein $z \in D$ *reelle Nullstelle* von f , falls

$$f(z) = 0.$$

Angenommen, die Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$ ist mindestens k -mal stetig differenzierbar für ein $k \in \mathbb{N}$ und es gilt

$$f(z) = 0, \quad f'(z) = 0, \quad \dots, \quad f^{(k-1)}(z) = 0, \quad f^{(k)}(z) \neq 0,$$

so heißt $z \in D$ eine *k-fache Nullstelle* von f .

Das allgemeine Nullstellenproblem muss meist mit einem *Iterationsverfahren* approximiert werden. Ausgehend von einem Startwert x_0 erhalten wir eine Folge von Approximationen $x_k, k = 1, 2, 3, \dots$, deren Grenzwert gegen die Nullstelle $z \in \mathbb{R}$ konvergiert:

$$f(x_0) \rightarrow f(x_1) \rightarrow f(x_2) \cdots \rightarrow f(z) = 0$$

Die Iteration wird bei einem endlichen $N \in \mathbb{N}$ gestoppt und das Folgenglied x_N wird als Approximation der Nullstelle $x_N \approx z$ betrachtet.

Anhand des Nullstellenproblems lassen sich die meisten der in der Einleitung eingeführten *Konzepte der numerischen Mathematik* diskutieren:

- Wie ist die Konditionierung des Nullstellenproblems und welche Stabilität haben verschiedene Verfahren?
- Wie konstruiert man *effiziente* und *stabile* numerische Verfahren?
- Wie schnell konvergiert ein approximatives Verfahren $(x_k)_{k \in \mathbb{N}}$ gegen die Nullstelle z ? Wie können wir die Begriffe *Konvergenzordnung* und *Konvergenzrate* (Maße für die Konvergenzgeschwindigkeit) quantifizieren? In der Analysis wird untersucht, ob Folgen konvergieren: $x_k \rightarrow z$ für $k \rightarrow \infty$? In der Numerik reicht diese Aussage nicht aus. Es kommt das *Konzept der Effizienz* ins Spiel: Konvergenz muss hinreichend schnell vorliegen, sodass eine gute Näherung auch schnell berechnet werden kann.
- Konvergiert das ausgewählte numerische Verfahren auf beliebig großen Intervallen und mit beliebig gewählten Startwerten x_0 , oder nur dann, wenn der Startwert x_0 bereits hinreichend nahe an der gesuchten Nullstelle ist? In anderen Worten: Ist das Verfahren *lokal* oder *global* konvergent?
- Können wir für eine berechnete Approximation x_N eine Abschätzung für den Fehler $|x_N - z|$ angeben? Kann diese Abschätzung numerisch ausgewertet werden zur Beurteilung des Fehlers einer bereits berechneten Approximation x_k ? In diesem Fall sprechen wir von *a-posteriori Abschätzungen*.

Im Rest dieses Kapitels werden wir die oben genannten Fragen anhand verschiedener Verfahren diskutieren und die Schlüsselbegriffe spezifizieren.

8.1 Stabilität und Kondition

Wir betrachten zunächst die Kondition des allgemeinen Nullstellenproblems. Dazu sei $f : D \subset \mathbb{R} \rightarrow \mathbb{R}$ eine differenzierbare und auf D invertierbare Funktion mit Nullstelle $f(z) = 0$ also $z = f^{-1}(0)$. Wir betrachten das Nullstellenproblem

$$N : f \mapsto f^{-1}(0)$$

und wollen untersuchen, wie sich eine Störung in der Funktion f auf die Störung in der Nullstelle auswirken kann. Um die Konditionszahl mit Definition 1.11 berechnen zu können müssen wir zunächst klären, was hier eine Störung in der Eingabe bedeutet. Die Eingabe ist die Funktion f . Eine gestörte Eingabe ist demnach eine gestörte Funktion $\tilde{f} = f + \delta f$. Zur Berechnung der Konditionszahl müssen wir $N(\cdot)$ in Richtung der Eingabe f ableiten. Dieses Konzept ist komplizierter als die übliche Abbildung. Wir vereinfachen die Situation, indem wir eine beliebige differenzierbare feste Störfunktion δf wählen und die gestörte Funktion

$$\tilde{f}_s = f + s\delta f$$

mit einem Parameter $s \in \mathbb{R}$ betrachten. Die „Richtung“ der Störung δf ist fest gewählt, die Stärke der Störung s hingegen lassen wir variabel. Dann können wir die Ableitung in Bezug auf den skalaren Parameter s elementar ausrechnen. Es gilt bei $N(f) = f^{-1}(0) = z$ in Form der Richtungsableitung unter Ausnutzung der Ableitungsregeln der Umkehrfunktion

$$\frac{d}{ds}N(f + s\delta f) = \frac{1}{(f' + s\delta f')(z)} \frac{d}{ds}(f(z) + s\delta f(z)) = \frac{\delta f(z)}{f'(z) + s\delta f'(z)}.$$

Hiermit können wir die Verstärkung des relativen Fehlers und damit die Konditionierung der Aufgabe für kleine Störungen approximieren

$$\begin{aligned} \frac{N(f + s\delta f) - N(f)}{N(f)} &= \frac{N(f + s\delta f) - N(f)}{s} \cdot \frac{s}{N(f)} \\ &\approx \frac{d}{ds}N(f + s\delta f) \cdot \frac{s}{N(f)} = \frac{1}{f'(z) + s\delta f'(z)} \frac{s\delta f(z)}{N(f)}. \end{aligned} \quad (8.1)$$

Für kleine Störungen mit $s \approx 0$ ist $s\delta f'(z)$ im Vergleich zu $f'(z)$ vernachlässigbar, und wir nähern den Ausdruck an als

$$\frac{N(f + s\delta f) - N(f)}{N(f)} \approx \frac{1}{f'(z)} \frac{s\delta f(z)}{z}. \quad (8.2)$$

Die Kondition des Nullstellenproblems verhält sich demnach wie $\kappa = 1/f'(z)$. Ist die Ableitung an der Nullstelle klein, so kann es zu beliebig großen Fehlerverstärkungen kommen.

Im Folgenden untersuchen wir nun eine konkrete Situation für f und betrachten das quadratische Polynom

$$f(x) = x^2 - px + q, \quad p, q \in \mathbb{R},$$

und untersuchen die Konditionierung der Nullstellensuche sowie die Stabilität von einfachen Algorithmen. Die p/q -Formel ist ein direktes Lösungsverfahren:

$$x_{1/2} = \frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q}$$

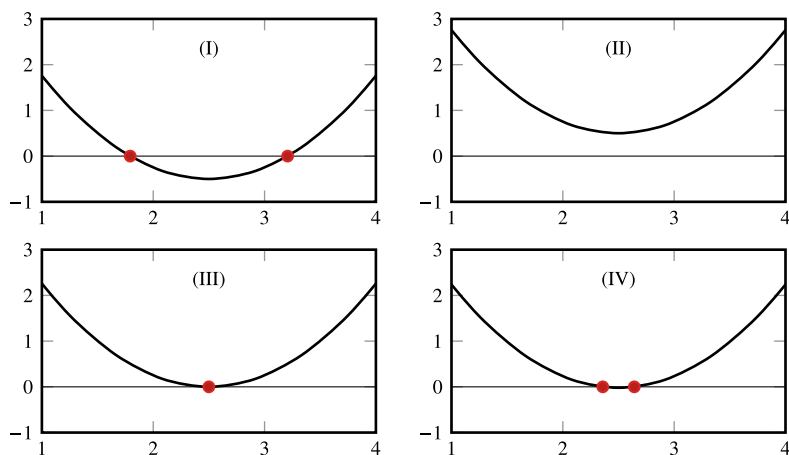


Abb. 8.1 Vier mögliche Situationen bei der Nullstellensuche quadratischer Funktionen. Von links oben nach rechts unten: (I) zwei klar getrennte Nullstellen, (II) keine reelle Nullstelle, (III) eine doppelte Nullstelle und (IV) zwei reelle Nullstellen nahe beieinander

In Abb. 8.1 zeigen wir vier verschiedene Situationen:

1. Es gibt zwei reelle Nullstellen, die weit separiert sind.
2. Es gibt nur eine reelle Nullstelle. Diese Nullstelle ist eine *doppelte Nullstelle* mit $f(z) = f'(z) = 0$.
3. Es gibt zwei reelle Nullstellen, die sehr nahe beieinanderliegen.
4. Es existiert keine reelle Nullstelle. Die Funktion hat dann zwei komplexe Nullstellen.

Beispiel 8.2 (Konditionierung der Nullstellensuche quadratischer Polynome)

Wir berechnen die Konditionierung der Nullstellenbestimmung in Abhängigkeit von den Koeffizienten. Dazu sei

$$f(x) = x^2 - px + q = 0, \quad x \in \mathbb{R}.$$

Für die Nullstellen $x_{1,2} := x_{1,2}(p, q)$ gilt

$$x_{1,2} = \frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q}$$

und wegen

$$f(x) = (x - x_1)(x - x_2) = x^2 - (x_1 + x_2)x + x_1x_2$$

auch

$$p = x_1 + x_2, \quad q = x_1 x_2.$$

Dies ist die Aussage vom *Satz von Vieta*. Wir berechnen nun Konditionszahlen der Nullstellensuche, wobei insbesondere $x_{1,2}(p, q)$ die Funktion darstellt, und p und q die Variablen sind:

$$\begin{aligned} k_{1p} &= \frac{\partial x_1}{\partial p} \frac{p}{x_1} = \frac{1 + x_2/x_1}{1 - x_2/x_1}, & k_{1q} &= \frac{\partial x_1}{\partial q} \frac{q}{x_1} = \frac{1}{1 - x_1/x_2}, \\ k_{2p} &= \frac{\partial x_2}{\partial p} \frac{p}{x_2} = -\frac{1 + x_2/x_1}{1 - x_2/x_1}, & k_{2q} &= \frac{\partial x_2}{\partial q} \frac{q}{x_2} = \frac{1}{1 - x_2/x_1}. \end{aligned}$$

Die Berechnung der Nullstellen x_1, x_2 ist demnach schlecht konditioniert, falls $x_1/x_2 \sim 1$, wenn also beide Nullstellen nahe beieinanderliegen. In diesem Fall können die Konditionszahlen beliebig groß werden. Dies ist die in Abb. 8.1 (III) skizzierte Situation. Das Beispiel ist in guter Übereinstimmung mit der Konditionierung des allgemeinen Nullstellenproblems (8.2), denn falls beide Nullstellen fast zusammenfallen, so hat das quadratische Polynom dort eine nahezu verschwindende Ableitung. ◀

Beispiel 8.3 (Stabilität der p/q -Formel)

Das vorherige Beispiel wird mit konkreten Zahlenwerten gefüllt. Es sei $p = 4$ und $q = 3.9999$:

$$f(x) = x^2 - 4x + 3.9999 = 0$$

mit den Nullstellen $x_{1,2} = 2 \pm 0.01$. Dann gilt für die Konditionszahl z. B.

$$|k_{1q}| = \left| \frac{1}{1 - x_1/x_2} \right| = 99.5.$$

Das entspricht einer möglichen Fehlerverstärkung um den Faktor 100. Bei einer Störung von p um 1 % zu $\tilde{p} = 4.04$ erhalten wir die gestörten Nullstellen

$$\tilde{x}_1 \approx 2.304, \quad \tilde{x}_2 \approx 1.736,$$

mit relativen Fehlern von etwa 15 %.

Bei einer Störung von p um minus 1 % zu $\tilde{p} = 3.96$ erhalten wir keine reellen Nullstellen mehr, sondern die komplexen Nullstellen

$$\tilde{x}_1 \approx 1.98 \pm 0.28196i.$$

◀

Die Nullstellenbestimmung quadratischer Polynome kann sehr schlecht konditioniert sein, falls die beiden Nullstellen nahe beieinanderliegen. Wir untersuchen nun die Stabilität des p/q -Verfahrens zur Berechnung der Nullstellen im gut konditionierten Fall, d. h. im Fall

von weit separierten Nullstellen. Es sei also ein Polynom

$$f(x) = x^2 - px + q$$

mit $|q| \ll p^2/4$ gegeben. Die beiden Lösungen lauten

$$x_{1,2} = \frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q},$$

also $x_1 \approx p$ und $x_2 \approx 0$. Aus den vorherigen Beispielen haben wir gelernt, dass die Aufgabe für $|x_1/x_2| \gg 1$ gut konditioniert ist. Wir berechnen beide Nullstellen mit der p/q -Formel:

Algorithmus 8.1: Nullstellenberechnung mit der p/q -Formel

Eingabe: Gegeben sei eine Funktion $f(x) = x^2 - px + q$.

$$^1 a_1 = p^2/4$$

$$^2 a_2 = a_1 - q$$

$$^3 a_3 = \sqrt{a_2} \geq 0$$

$$^4 x_1 = p/2 - a_3$$

$$^5 x_2 = p/2 + a_3$$

Ergebnis: Nullstellen $x_{1,2}$ des Polynoms $f(x)$

Dieses Verfahren kann nun mit den Mitteln der Fehleranalyse aus Abschn. 1.5 im Detail untersucht werden. Wir betrachten dabei nur die beiden letzten Schritte, also die Berechnung von x_1 und x_2 . Wir gehen dabei davon aus, dass a_3 mit einem relativen Fehler ϵ_a und $p/2$ mit einem relativen Fehler ϵ_p vorliegen. Jeweils sei $\epsilon_p = \epsilon_a = O(\epsilon)$. Dann gilt

$$\begin{aligned} \tilde{x}_{1/2} &= \left(\frac{p}{2}(1 + \epsilon_p) \pm a_3(1 + \epsilon_a) \right) (1 + \epsilon) \\ &= x_{1/2}(1 + \epsilon) + \frac{p}{2}\epsilon_p \pm a_3\epsilon_a + O(\epsilon^2). \end{aligned}$$

Für den relativen Fehler folgt

$$\frac{|\tilde{x}_{1/2} - x_{1/2}|}{|x_{1/2}|} \leq \epsilon + \frac{|p/2| + |a_3|}{|p/2 \pm a_3|} \epsilon.$$

Für $p/2 \pm a_3 \approx 0$ kann der Fehler beliebig verstärkt werden. Es hängt dann vom Vorzeichen ab. Angenommen, $p/2 \approx a_3$ mit gleichem Vorzeichen. Dann ist die Berechnung der Nullstelle

$$x_1 = \frac{p}{2} - a_3$$

möglicherweise sehr instabil, die Berechnung der Nullstelle

$$x_2 = \frac{p}{2} + a_3$$

hingegen sehr stabil. Um Auslöschung zu vermeiden, wird daher im Fall $p < 0$ die zweite Nullstelle mit der p/q -Formel berechnet $\tilde{x}_2 = p/2 - a_3$, ansonsten die erste $\tilde{x}_1 = p/2 + a_3$. Die jeweils andere Nullstelle kann alternativ mit dem Satz von Vieta bestimmt werden als

$$q = x_1 x_2 \Leftrightarrow x_1 = \frac{q}{x_2}, \quad x_2 = \frac{q}{x_1}.$$

Wegen der guten Konditionierung der Division ist dieses Vorgehen stets stabil. Wir fassen zusammen:

Algorithmus 8.2: Stabile Berechnung von Nullstellen quadratischer Funktionen

```

1  $a_1 = p^2/4$ 
2  $a_2 = a_1 - q$ 
3 Wenn  $a_2 < 0$  dann
4   Abbruch, keine reelle Nullstelle!
5  $a_3 = \sqrt{a_2}$ 
6 Wenn  $p < 0$  dann
7    $x_1 = p/2 - a_3$ 
8    $x_2 = q/x_1$ 
9 Sonst
10   $x_2 = p/2 + a_3$ 
11   $x_1 = q/x_2$ 
```

Bemerkung 8.4 (Konditionierung und Stabilität) Diese Analyse verdeutlicht noch einmal, dass Konditionierung eine Eigenschaft des Problems ist, Stabilität eine Eigenschaft des Algorithmus. Falls die reellen Nullstellen separiert sind, so ist die Aufgabe an sich gut konditioniert. Die Berechnung beider Nullstellen mit der p/q -Formel kann dann jedoch dennoch beliebig instabil sein. Bei richtiger Verwendung des Satzes von Vieta kann die Nullstelle immer stabil berechnet werden. ♦

Beispiel 8.5

Wir berechnen die Nullstelle von

$$x^2 - 4x + 0.01 = 0.$$

Es gilt

$$x_1 \approx 0.002502, \quad x_2 \approx 3.997498.$$

Viervestellige Rechnung ergibt beim stabilen Algorithmus:

1	$a_1 = p^2/4$	// $\tilde{a}_1 = 4.000$
2	$a_2 = a_1 - q$	// $\tilde{a}_2 = 3.990$
3	$a_3 = \sqrt{a_2}$	// $\tilde{a}_3 = 1.997$
4	Wenn $p < 0$ dann	
5	$x_1 = p/2 - a_3$	$x_2 = q/x_1$
6	Sonst	
7	$x_2 = p/2 + a_3$	// $\tilde{x}_2 = 3.997$
8	$x_1 = q/x_2$	// $\tilde{x}_1 = 0.0025$

Die ersten vier wesentlichen Stellen sind korrekt. Würden wir beide Nullstellen über die p/q -Formel berechnen, so wäre das Ergebnis bei vierstelliger Rechnung

1	$x_1 = p/2 - a_3$	// $\tilde{x}_1 = 0.003$
---	-------------------	--------------------------

mit einem relativen Fehler von 20 %.

Liegt von einem numerischen Verfahren die Näherung $\tilde{z}$ an die Nullstelle z vor, so kann der Fehler $|z - \tilde{z}|$ durch die folgende *a posteriori Fehlerabschätzung* einfach geschätzt werden, wenn die Funktion f zusätzlich differenzierbar ist:

Satz 8.6 (A-posteriori-Fehlerabschätzung) Es sei $f \in C^1[a, b]$ mit

$$0 < m \leq |f'(x)| \leq M < \infty.$$

Weiter sei $z \in [a, b]$ mit $f(z) = 0$. Dann gilt für jedes $\tilde{z} \in [a, b]$

$$\frac{|f(\tilde{z})|}{M_{z,\tilde{z}}} \leq |\tilde{z} - z| \leq \frac{|f(\tilde{z})|}{m_{z,\tilde{z}}},$$

wobei

$$I_{z,\tilde{z}} := [\min(z, \tilde{z}), \max(z, \tilde{z})], \quad M_{z,\tilde{z}} := \max_{x \in I_{z,\tilde{z}}} |f'(x)|, \quad m_{z,\tilde{z}} := \min_{x \in I_{z,\tilde{z}}} |f'(x)|.$$

Beweis Mit Taylor-Entwicklung gilt

$$f(\tilde{z}) = f(z) + f'(\xi)(\tilde{z} - z) = f'(\xi)(\tilde{z} - z) \quad (8.3)$$

mit einer Zwischenstelle $\xi \in M_{z,\tilde{z}}$. Die Fehlerabschätzung folgt nach Teilen durch $f'(\xi)$ und Übergang zu Minimum und Maximum. $\square$

Diese einfache Fehlerabschätzung kann z. B. als Abbruchskriterium in einem numerischen Verfahren verwendet werden.

8.2 Intervallschachtelung

Wir betrachten wieder die Funktion $f(x)$ aus der Einleitung

$$f(x) = x(1 + \exp(x)) + 10 \sin(3 + \log(x^2 + 1)). \quad (8.4)$$

Mit dem Zwischenwertsatz, Satz 7.1, können wir folgern, dass es im Intervall $[-10, 2]$ mindestens eine Nullstelle gibt, da $f(-10)f(2) < 0$ gilt. Der Beweis zum Zwischenwertsatz konstruiert eine Folge von Intervallen, deren Grenzen gegen die Nullstelle konvergiert. Dieses Verfahren lässt sich direkt als numerisches Verfahren, der sogenannten *Intervallschachtelung*, auch *Intervallhalbierung* genannt, umsetzen.

Wir beginnen mit $a_0 = a$ und $b_0 = b$, berechnen in jedem Schritt den Mittelwert $x_k = \frac{1}{2}(a_{k-1} + b_{k-1})$ und prüfen, ob es in den Teilintervallen $[a_{k-1}, x_n]$ oder $[x_n, b_{k-1}]$ zu einem Vorzeichenwechsel kommt. Mit diesem Intervall geht die Iteration dann weiter. Wir zeigen in Abb. 8.2 die ersten 8 Schritte angewendet auf die Funktion $f(x)$. Neben den Endpunkten $a = -10$ und $b = 2$ geben wir links die ersten 4 Intervallmittelpunkte x_k an. Zur besseren Darstellung starten wir ab x_5 neu in dem kleineren Intervall. In grün markieren wir jeweils das aktuelle Intervall.

Nach 8 Schritten erhalten wir das Intervall $(x_7, x_8) = (-0.34375, -0.296875)$ und die Nullstelle ist hier eingegrenzt. Wird die Nullstelle in einem Schritt exakt gefunden, gilt also $f((a+b)/2) = 0$, so kann die Iteration abgebrochen werden.

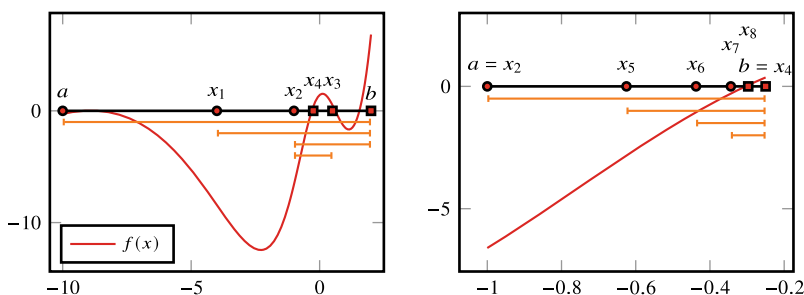


Abb. 8.2 Anwendung der Intervallschachtelung auf die Funktion $f(x)$ aus (8.4). Die orangenen Segmente zeigen jeweils das aktuelle Intervall, in dem eine Nullstelle liegen muss

Algorithmus 8.3: Intervallschachtelung

Eingabe: Gegeben sei ein Intervall $I = [a, b]$ sowie eine Funktion $f \in C[a, b]$ und eine Abbruchtoleranz $\epsilon > 0$.

```

1 Setze  $a_0 = a$  und  $b_0 = b$ 
2 Für  $n = 1, 2, \dots$  tue
3    $x_n = (a_{n-1} + b_{n-1})/2$ 
4   Wenn  $f(x_n) = 0$  dann
5     Abbruch mit Nullstelle  $x_n$ 
6   Sonst wenn  $f(a_{n-1})f(x_n) < 0$  dann
7      $a_n = a_{n-1}$ 
8      $b_n = x_n$ 
9   Sonst
10     $a_n = x_n$ 
11     $b_n = b_{n-1}$ 
12   Wenn  $|f(x_n)| < \epsilon$  dann
13     Abbruch mit Nullstelle  $x_n$ 
Ergebnis: (Approximative) Nullstelle  $x_n$ 

```

In Python können wir diesen Algorithmus folgendermaßen umsetzen:

🔗 Implementierung 8.1: Intervallschachtelung

```

1 def intervall_schachtelung(f, a, b, n, tol=1e-10):
2     for i in range(n):
3         x = (a + b) / 2
4         if abs(f(x)) < tol:
5             return x
6         elif f(a) * f(x) < 0:
7             b = x
8         else:
9             a = x
10
11     print(f'Die Intervallschachtelung ist nach {n} Iterationen nicht
12           ↳ konvergiert!\n x = {x}, abs(f(x)) = {abs(f(x))} > {tol}')
13     return x

```

Wir fassen zusammen

Satz 8.7 (Intervallschachtelung) Es sei $f \in C[a, b]$ mit $f(a)f(b) < 0$. Es sei a_n, b_n, x_n die durch Algorithmus 8.3 bestimmte Folge von Intervallen und Mittelpunkten. Es gilt

$$x_n \rightarrow z, \quad f(z) = 0,$$

sowie die Fehlerabschätzung

$$|x_n - z| \leq \frac{b - a}{2^{n+1}}.$$

Beweis (i) Bei $a < b$ gilt stets

$$a \leq a_1 \leq \cdots \leq a_n \leq a_{n+1} \leq x_n \leq b_{n+1} \leq b_n \leq \cdots \leq b_1 \leq b.$$

Das Verfahren ist damit durchführbar. Die Folge x_n ist durch zwei monotone (steigende und fallende) Folgen a_n und b_n beschränkt. Die Folge ist daher konvergent. Das Verfahren bricht nur dann ab, wenn $f(x_n) = 0$, wenn also $x_n = z$ die gesuchte Nullstelle ist.

(ii) Es gilt

$$b_{n+1} - a_{n+1} = \begin{cases} x_n - a_n = \frac{b_n - a_n}{2} & f(a_n)f(x_n) < 0 \\ b_n - x_n = \frac{b_n - a_n}{2} & f(a_n)f(x_n) > 0. \end{cases}$$

In beiden Fällen folgt

$$|b_n - a_n| = \frac{|b_0 - a_0|}{2^n}. \quad (8.5)$$

Wegen

$$a_n \leq x_n \leq b_n$$

folgt die Konvergenz von $x_n \rightarrow z$ gegen ein $z \in [a_n, b_n] \subset [a, b]$.

(iii) Für diesen Grenzwert z gilt wegen der Stetigkeit von $f(\cdot)$

$$0 \leq f(z)^2 = \lim_{n \rightarrow \infty} f(a_n)f(b_n) \leq 0 \Rightarrow f(z) = 0.$$

Der Grenzwert ist somit die gesuchte Nullstelle. Schließlich können wir aus (8.5) die Fehlerabschätzung herleiten. Denn es ist

$$|x_n - z| \leq \frac{|b_n - a_n|}{2} \leq \frac{|b - a|}{2^{n+1}}. \quad \square$$

Bemerkung 8.8 (A-priori-Fehlerabschätzung) Die in Satz 8.7 angegebene Fehlerabschätzung

$$|x_n - z| \leq \frac{b - a}{2^{n+1}} \quad (8.6)$$

ist eine *A-priori-Fehlerabschätzung*. Sie kann bereits vor der Rechnung ausgewertet werden, um z. B. die benötigte Anzahl von Schritten n zu bestimmen. Die Abschätzung aus Satz 8.6 ist eine *a posteriori* Abschätzung. Um sie auszuwerten muss die Näherung $\tilde{z} = x_n$ bereits bekannt sein.

Der Vorteil der a priori Abschätzung 8.6 ist, dass keine weiteren Informationen zur Auswertung notwendig sind. Bei der Abschätzung aus Satz 8.6 benötigen wir Schätzungen für das Minimum und Maximum der ersten Ableitung. Wir bringen ein Beispiel: Gesucht ist eine Nullstelle der Funktion $f(x) = \cos(x)$ auf $[0, 2]$. Wir starten die Intervallschachtelung mit $a_0 = 0$ und $b_0 = 2$. Nach 10 Schritten, also 10 Intervallhalbierungen gilt

$$b_{10} - a_{10} = 2^{-10}(b - a) \approx 0.002$$

und wir können folgern, dass der Fehler dann mindestens mit der Genauigkeit $|x_{10} - z| < 0.001$ vorliegt (x_{10} liegt im Mittelpunkt, d.h. wir gewinnen einen weiteren Faktor $\frac{1}{2}$). Wir führen nun 10 Schritte der Intervallschachtelung durch. Dann gilt

$$a_{10} = 1.5703125, \quad b_{10} = 1.572265625, \quad x_{11} = 1.5712890625$$

und $f(x_{11}) \approx -0.00049$. Es ist $z = \frac{\pi}{2}$ und somit gilt für den exakten Fehler

$$|z - x_{11}| \approx 0.000493.$$

Für die Kosinusfunktion gilt $|\cos'(x)| = |\sin(x)| \leq 1$ und in der Nähe der gesuchten Nullstelle ist $|\cos'(\tilde{z})| \approx 1$. Die a posteriori Abschätzung liefert somit

$$|x_{11} - z| \approx |f(x_{11})| \approx 0.00049,$$

also eine Schätzung sehr nahe am echten Fehler. ◆

Die Intervallschachtelung ist ein sehr stabiles Verfahren. Aus dem Konstruktionsprinzip folgt unmittelbar, dass die Nullstelle stets im Iterationsintervall eingeschlossen ist. Allerdings konvergiert das Verfahren sehr langsam gegen die gesuchte Nullstelle. Aufgrund seiner Stabilität wird das Intervallschachtelungsverfahren häufig als Startverfahren für andere Verfahren genutzt. Die Intervallschachtelung ist auf reelle skalare Funktionen beschränkt (im Gegensatz zu den weiteren Verfahren, die wir im Anschluss kennenlernen werden). Zum Auffinden von doppelten Nullstellen mit $f(z) = f'(z) = 0$ ist die Intervallschachtelung nicht geeignet.

8.3 Das Newton-Verfahren

Die Intervallschachtelung stellt an die Funktion f nur die geringe Anforderung der Stetigkeit. Die Idee des Newton-Verfahrens ist es, mehr Informationen über den Verlauf der Funktion zu nutzen, um die Iteration selbst zu verbessern. Es sei $f \in C^1[a, b]$ stetig differenzierbar. In der Nähe eines Punktes x_0 approximieren wir $f(x)$ durch die lineare Taylor-Approximation, also die Tangente

$$f(x) \approx t_{x_0}(x) = f(x_0) + f'(x_0)(x - x_0).$$

Im Anschluss approximieren wir die gesuchte Nullstelle z von $f(x)$ durch die Nullstelle der Tangente.

$$f(z) \approx t_{x_0}(z) = f(x_0) + f'(x_0)(z - x_0) \stackrel{!}{=} 0 \quad \Rightarrow \quad z = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

Die Nullstelle der Tangente wird als bessere Näherung an die Nullstelle der Funktion f gewählt und ausgehend vom Startwert x_0 wir definieren die Iteration

$$x_{n+1} := x_n - \frac{f(x_n)}{f'(x_n)},$$

welche gerade das klassische Newton-Verfahren ist.

Das Newton-Verfahren kann geometrisch interpretiert werden: Die Funktion f wird im Näherungswert x_n durch ihre Tangente linearisiert und der iterierte Wert x_{n+1} als Abszisse des Schnittpunktes der Tangente (an f) mit der x -Achse definiert, vergleiche Abb. 8.3.

Algorithmus 8.4: Newton-Verfahren

Eingabe: Gegeben sei $f \in C^1[a, b]$, ein Startwert $x_0 \in [a, b]$ und eine Toleranz ϵ .

1 $k = 0$

2 **Solange** $|f(x_k)| \geq \epsilon$ **tue**

3 $x_{k+1} = x_k - f(x_k)/f'(x_k)$

4 $k = k + 1$

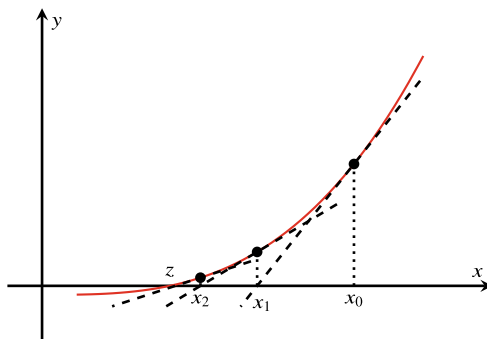
Ergebnis: Approximative Nullstelle $x \approx x_k$.

In Python können wir dies umsetzen durch

♣ Implementierung 8.2: Newton-Verfahren

```
1 def newton_skalar(f, f_abl, x, n=10, tol=1e-10):
2     for i in range(n):
3         x -= f(x) / f_abl(x)
4         if abs(f(x)) < tol:
5             return x
6     print(f'Die Newton Methode ist noch nicht konvergiert!')
7     return x
```

Abb. 8.3 Geometrische Interpretation des Newton-Verfahrens. Die neue Iterierte x_{k+1} ist jeweils die Nullstelle der Tangente an f im Punkt $(x_k, f(x_k))$



Bemerkung 8.9 *An dieser Stelle verweisen wir auch einmal auf Abschn. 1.4 mit Beispielen zur Interaktion der Maschinengenauigkeit und Toleranzen (Abbruchkriterien) von iterativen Verfahren. Letztere sollen hinreichend groß gewählt werden, um Endlosschleifen aufgrund von Rundungsfehlern zu vermeiden, aber gleichzeitig klein genug, um hinreichende Genauigkeit der numerischen Lösung zu garantieren. Dies trifft natürlich nicht nur auf das hier behandelte Newton-Verfahren zu, sondern auf die meisten iterativen Verfahren.* ♦

Die Iteration 8.4 gehört zur Klasse der Fixpunktiterationen mit der Iterationsfunktion

$$g(x) := x - \frac{f(x)}{f'(x)}, \quad x_{k+1} = g(x_k). \quad (8.7)$$

Für einen Fixpunkt $z = g(z)$ gilt offenbar $f(z) = 0$. Wir zeigen nun als Hauptresultat einen einfachen Konvergenzbeweis für das Newton-Verfahren:

Satz 8.10 (Newton-Verfahren) Die Funktion $f \in C^2[a, b]$ habe im Innern des Intervalls $[a, b]$ eine Nullstelle z und es seien

$$m := \min_{a \leq x \leq b} |f'(x)| > 0, \quad M := \max_{a \leq x \leq b} |f''(x)| \quad (8.8)$$

Schranken für erste (nach unten) und zweite Ableitung (nach oben). Es sei $\rho > 0$ so gewählt, dass

$$q := \frac{M}{2m}\rho < 1, \quad K_\rho(z) := \{x \in \mathbb{R} : |x - z| \leq \rho\} \subset [a, b]. \quad (8.9)$$

Dann sind für jeden Startpunkt $x_0 \in K_\rho(z)$ die Newton-Iterierten $x_k \in K_\rho(z)$ definiert und konvergieren gegen die Nullstelle z . Des Weiteren gelten die a-priori-Fehlerabschätzung

$$|x_k - z| \leq \frac{2m}{M} q^{2^k}, \quad k \in \mathbb{N},$$

und die a-posteriori-Fehlerabschätzung

$$|x_k - z| \leq \frac{1}{m} |f(x_k)|, \quad k \in \mathbb{N}.$$

Beweis (i) Der Beweis ist eine einfache Anwendung der Taylor-Approximation. Für $x, z \in [a, b]$ gilt (z ist die Nullstelle)

$$f(z) = f(x) + f'(x)(z - x) + \frac{f''(\xi)}{2}(z - x)^2$$

mit einem Zwischenwert $\xi \in [x, z]$ oder $\xi \in [z, x]$. Für $f'(x) \neq 0$, was wegen $|f'(x)| \geq m > 0$ in $[a, b]$ stets gegeben ist, erhalten wir den Zusammenhang

$$\frac{f(x)}{f'(x)} = (x - z) - \frac{f''(\xi)}{2f'(x)}(x - z)^2. \quad (8.10)$$

(ii) Wir zeigen zunächst, dass die Iteration unter den gegebenen Voraussetzungen durchführbar ist. Angenommen, $x_k \in K_\rho(z)$. Dann folgt mit (8.10) und der Vorschrift

$$x_{k+1} - z = x_k - \frac{f(x_k)}{f'(x_k)} - z = (x_k - z) - \left((x_k - z) - \frac{f''(\xi)}{2f'(x_k)}(x_k - z)^2 \right),$$

also

$$x_{k+1} - z = \frac{f''(\xi)}{2f'(x_k)}(x_k - z)^2$$

bzw. abgeschätzt

$$|x_{k+1} - z| = \frac{|f''(\xi)|}{2|f'(x_k)|}|x_k - z|^2 \leq \frac{M}{2m}|x_k - z|^2. \quad (8.11)$$

Im Fall $x_k \in K_\rho(z)$ ist $|x_k - z| \leq \rho$ folgt

$$|x_{k+1} - z| \leq \underbrace{\frac{M}{2m}}_{=q < 1} \rho \cdot \rho \leq \rho,$$

also bleibt $x_{k+1} \in K_\rho(z) \subset [a, b]$ in der Umgebung der Nullstelle und auch im Definitionsbereich von $f(\cdot)$, sodass das Verfahren durchführbar ist.

(iii) Wir können nun gleich die Fehlerabschätzung nachweisen, denn für

$$\rho_k := \frac{M}{2m}|x_k - z|$$

gilt mit (8.11)

$$\rho_k = \frac{M}{2m}|x_k - z| \leq \frac{M}{2m} \frac{M}{2m}|x_{k-1} - z|^2 = \rho_{k-1}^2$$

und iteriert

$$\rho_k \leq \rho_0^{2^k},$$

also

$$|x_k - z| \leq \frac{2m}{M} \left(\frac{M}{2m}|x_0 - z| \right)^{2^k} \leq \frac{2m}{M} q^{2^k}.$$

(iv) Es bleibt der Nachweis der *a-posteriori*-Abschätzung. Taylor-Entwicklung bis zur ersten Stufe gibt

$$f(x_k) = f(z) + f'(\xi)(x_k - z),$$

also folgt wegen $|f'(\xi)| \geq m$ die Fehlerabschätzung

$$|x_k - z| \leq \frac{|f(x_k)|}{m}. \quad \square$$

Dieser Satz zeigt die quadratische Konvergenz des Newton-Verfahrens, wenn die Funktion f zweimal stetig differenzierbar ist, im Intervall eine Nullstelle hat und wenn die erste Ableitung betragsmäßig von null weg beschränkt ist. Der Startwert muss bereits hinreichend nahe an der Nullstelle sein.

Bemerkung 8.11 (A-posteriori Fehlerabschätzung) Die a-posteriori Fehlerabschätzung lässt sich weiter durch den Abstand zwischen zwei aufeinander folgende Glieder der Lösungsfolge abschätzen. Mit der Newton-Vorschrift folgt mit der Taylor-Entwicklung

$$\begin{aligned} f(x_k) &= f\left(x_{k-1} - \frac{f(x_{k-1})}{f'(x_{k-1})}\right) \\ &= f(x_{k-1}) + f'(x_{k-1})\left(-\frac{f(x_{k-1})}{f'(x_{k-1})}\right) + \frac{f''(\xi)}{2}\left(\frac{f(x_{k-1})}{f'(x_{k-1})}\right)^2. \end{aligned}$$

Wir erhalten dann die Abschätzung

$$|x_k - z| \leq \frac{1}{m}|f(x_k)| \leq \frac{M}{2m}|x_k - x_{k-1}|^2, \quad k \in \mathbb{N}.$$

Im allgemeinen kennen wir die Konstanten in der zweiten Ungleichung nicht. Zudem ist es eine schwächere Abschätzung der Nullstelle. Daher eignet sich dies in der Regel nicht dafür zu prüfen, wie gut das Verfahren konvergiert ist. $\blacklozenge$

Bemerkung 8.12 (Abbruchkriterien des Newton-Verfahrens) Der vorherige Satz gibt uns mit der A-posteriori-Abschätzung auch ein Kriterium zum Abbruch der Iteration in Algorithmus 8.4. Falls zu einer gegebenen Toleranz $\epsilon > 0$

$$\frac{|f(x_k)|}{m} < \epsilon$$

oder

$$\frac{M}{2m}|x_k - x_{k-1}|^2 < \epsilon$$

gilt, so ist auch $|x_k - z| < \epsilon$. In diese Abbruchkriterien gehen jedoch die Konstanten m und M , also die Schranken der Ableitungen von $f(x)$ ein. Diese kennen wir im Allgemeinen nicht. Daher geben wir noch ein weiteres heuristisches, aber überprüfbares Kriterium an.

Die Iteration wird beendet (d.h., wir haben eine akzeptable Lösung x_{k+1} in der k -ten Iteration gefunden), falls

$$\frac{|x_{k+1} - x_k|}{|x_k|} < \epsilon \quad (8.12)$$

gilt, denn es ist

$$\frac{|x_k - z|}{|z|} \leq \frac{|x_{k+1} - x_k| + |x_{k+1} - z|}{|z|} = \frac{|x_{k+1} - x_k|}{|z|} + O(|x_k - z|^2),$$

falls wir im Bereich der quadratischen Konvergenz sind. Zur Approximation wird schließlich im Nenner $|z|$ durch $|x_k|$ ersetzt, um eine berechenbare Größe zu erhalten. ♦

Beispiel 8.13 (Newton-Verfahren und Intervallschachtelung)

Wir betrachten die Funktion f aus der Einleitung

$$f(x) = x(1 + \exp(x)) + 10 \sin(3 + \log(x^2 + 1)).$$

Wir haben argumentiert, dass $f(x) < 0$ für $x < -10$ und $f(x) > 0$ für $x > 2$ gilt. Im Intervall $[-10, 2]$ liegt somit mindestens eine Nullstelle. In der Tat hat die Funktion in diesem Intervall fünf Nullstellen. Es gilt (auf sechs Stellen Genauigkeit)

$$\begin{aligned} z_1 &= -9.16359, & z_2 &= -8.65847, & z_3 &= -0.304487, \\ z_4 &= 0.608758, & z_5 &= 1.54776. \end{aligned}$$

Wir führen zunächst die Intervallschachtelung mit Algorithmus 8.3 durch:

it	a	b	x	f(a)	f(b)	b - a
00	-1.000e+01	1.000e+01	0.000000e+00	-2.844e-01	2.203e+05	2.000e+01
01	-1.000e+01	0.000e+00	-5.000000e+00	-2.844e-01	1.411e+00	1.000e+01
02	-5.000e+00	0.000e+00	-2.500000e+00	-5.285e+00	1.411e+00	5.000e+00
03	-2.500e+00	0.000e+00	-1.250000e+00	-1.235e+01	1.411e+00	2.500e+00
...						
15	-3.046e-01	-3.040e-01	-3.042603e-01	-5.523e-04	3.766e-03	6.104e-04

Nach 16 Schritten erhalten wir eine Approximation $\tilde{x} \approx -0.3042603$, welche höchstens den Fehler $|\tilde{x} - z| \leq 0.0006104$ besitzt. Verglichen mit der exakten Nullstelle gilt $|\tilde{x} - z_3| \approx 0.00023$.

Wir führen nun das Newton-Verfahren aus. Hierbei gilt es einen Startwert zu wählen. Eigentlich müsste dieser Startwert im Sinne von Satz 8.10 gewählt werden. Dies ist bei der vorliegenden Funktion jedoch nur schwer möglich. Wir wählen daher zunächst $x^{(0)} = 0$. Es folgt $x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$

it	x	f(x)
0	0.00000000	1.4112e+00
1	-0.70560004	-3.6485e+00
2	-0.34944175	-3.3204e-01
3	-0.30623519	-1.2393e-02
4	-0.30449044	-2.1400e-05

```

5 -0.30448741 -6.4398e-11
6 -0.30448741 -2.8866e-15

```

Nach nur vier Schritten sind die ersten fünf wesentlichen Stellen der Nullstelle z_3 bestimmt. Wir betrachten als Startwert jetzt das linke Intervallende $x^{(0)} = a = -10$:

```

it x          f(x)
0 -10.00000000 -2.8437e-01
1 -9.46454160 -6.3943e-02
2 -9.24183447 -1.2229e-02
3 -9.17249436 -1.2325e-03
4 -9.16373585 -1.9882e-05
5 -9.16358985 -5.5322e-09
6 -9.16358981 0.0000e+00

```

Das Newton-Verfahren konvergiert jetzt gegen die Nullstelle z_1 . Es liegt wieder sehr schnelle Konvergenz vor, nicht jedoch gegen die eigentlich gesuchte Nullstelle z_3 . Falls das Newton-Verfahren im Bereich der quadratischen Konvergenz liegt, so ist es äußerst schnell. Wenn alle fünf Nullstellen gefunden werden sollen, so stellt sich jedoch die Frage nach guten Startwerten. Diese könnten z. B. mit einer Rasterung des Intervalls, also z. B. durch Berechnen aller $f(x)$ für $x \in \{-10, -9, \dots, 10\}$ gefunden werden. ◀

Bemerkung 8.14 Für eine zweimal stetig differenzierbare Funktion f existiert zu jeder einfachen Nullstelle z stets eine (evtl. sehr kleine) Umgebung $K_\rho(z)$, für die die Voraussetzungen von Satz 8.10 erfüllt sind. Das Problem beim Newton-Verfahren ist die Bestimmung eines im Einzugsbereich der Nullstelle z gelegenen Startpunktes x_0 (lokale Konvergenz). Dieser Startwert kann z. B. mithilfe der Intervallschachtelung (siehe entsprechende Bemerkung oben) berechnet werden. Dann konvergiert das Newton-Verfahren sehr schnell (quadratische Konvergenz) gegen die Nullstelle z . ♦

Falls der Startpunkt x_0 im Einzugsbereich der quadratischen Konvergenz liegt, dann konvergiert das Newton-Verfahren sehr schnell gegen die gesuchte Nullstelle. Es sei z. B. $q \leq \frac{1}{2}$, dann gilt nach nur zehn Iterationsschritten

$$|x_{10} - z| \leq \frac{2m}{M} q^{2^{10}} \sim \frac{2m}{M} 10^{-300}.$$

Beispiel 8.15 (Wurzelberechnung)

Die n -te Wurzel einer Zahl $a > 0$ ist die Nullstelle der Funktion $f(x) = x^n - a$. Das Newton-Verfahren zur Berechnung von $z = \sqrt[n]{a} > 0$ lautet (mit $f'(x) = nx^{n-1}$):

$$x_{k+1} = x_k - \frac{x_k^n - a}{nx_k^{n-1}} = \frac{1}{n} \left\{ (n-1)x_k + \frac{a}{x_k^{n-1}} \right\}$$

Folgende Spezialfälle leiten sich daraus ab:

$$x_{k+1} = \frac{1}{2} \left\{ x_k + \frac{a}{x_k} \right\} \quad (\text{Quadratwurzel})$$

$$x_{k+1} = \frac{1}{3} \left\{ 2x_k + \frac{a}{x_k^2} \right\} \quad (\text{Kubikwurzel})$$

$$x_{k+1} = (2 - ax_k)x_k \quad (\text{Kehrwert})$$

Aufgrund von Satz 8.10 konvergiert $x_k \rightarrow z$ für $k \rightarrow \infty$, falls x_0 nahe genug bei z gewählt wird.

Bei der Wurzelberechnung kann jedoch sogar globale Konvergenz für alle Startwerte $x_0 > 0$ gezeigt werden. Es kann – unabhängig von x_0 – gezeigt werden, dass

$$x_1 \geq \sqrt{a}$$

gilt. Im Anschluss kann monotone Konvergenz nachgewiesen werden:

$$x_{k+1} < x_k \quad k = 2, 3, \dots$$

Hieraus folgt globale Konvergenz. Quadratische Konvergenz liegt im Allgemeinen erst nach einigen Iterationsschritten vor. ◀

Bemerkung 8.16 (Praktische Berechnung der Wurzel) Das Newton-Verfahren kommt auf Computern bei der Berechnung von vielen Größen zum Einsatz, die nicht elementar bestimmt werden können. Da die Iteration nur Multiplikationen, Divisionen und Additionen erfordert, kann es sehr effizient umgesetzt werden. Im Exkurs 8.6 gehen wir auf einen besonders effizienten Algorithmus zur Approximation der Kehrwerts der Quadratwurzel ein. ♦

8.3.1 Varianten des Newton-Verfahrens

Hat die Funktion $f(x)$ im Intervall $[a, b]$ eine doppelte Nullstelle mit $f(z) = f'(z) = 0$, so ist $m = \min |f'| = 0$ und Satz 8.10 kann nicht angewendet werden, da die Voraussetzungen nicht erfüllt werden können. Es gibt keinen Konvergenzradius $\rho > 0$, sodass

$$\frac{M}{2m} \rho < 1.$$

Beispiel 8.17 (Newton-Verfahren für doppelte Nullstellen)

Wir betrachten die Funktion $f(x) = x^2$ mit der doppelten Nullstelle $f(0) = f'(0) = 0$. Wir führen einige Iterationen

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

durch und wählen als Startwert $x_0 = 1$. Es gilt

$$x_0 = 1, \quad x_1 = 0.5, \quad x_2 = 0.25, \quad x_3 = 0.125, \dots$$

Die Werte nähern sich der Nullstelle $z = 0$ an, der Fehler $x_k - z$ scheint sich aber in jedem Schritt nur zu halbieren. Wir betrachten das Beispiel genauer und erhalten

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} = x_k - \frac{x_k^2}{2x_k} = \frac{1}{2}x_k.$$

Der Iterationswert halbiert sich in jedem Schritt. Dies zeigt, dass das Newton-Verfahren für $f(x) = x^2$ in der Tat gegen die Nullstelle konvergiert. Es liegt sogar globale Konvergenz für beliebige Startwerte vor. Diese ist jedoch nur linear. ◀

Es stellt sich die Frage, ob das beobachtete Verhalten nur ein Spezialfall ist oder ob wir generell Konvergenz auch bei mehrfachen Nullstellen erwarten können. Das Problem bei der Durchführung ist, dass bei $x_k \rightarrow z$ sowohl $f(x_k) \rightarrow 0$ als auch $f'(x_k) \rightarrow 0$ gilt. Für den Quotienten gilt mit der *Regel von de l'Hospital* [60] bei einer doppelten Nullstelle mit $f''(z) \neq 0$

$$\lim_{x_k \rightarrow z} \frac{f(x_k)}{f'(x_k)} = \lim_{x_k \rightarrow z} \frac{f'(x_k)}{f''(x_k)} = \frac{f'(z)}{f''(z)} = 0.$$

Hieraus können wir folgern, dass bei Konvergenz von $x_k \rightarrow z$ die Iteration wohldefiniert bleibt. Es gilt:

Satz 8.18 (Newton-Verfahren bei mehrfachen Nullstellen) Es sei $z \in [a, b]$ eine p -fache Nullstelle der Funktion $f \in C^{p+1}([a, b])$, d. h., es gelte

$$f(z) = f'(z) = \dots = f^{(p-1)}(z) = 0, \quad f^{(p)}(z) \neq 0.$$

Weiter sei

$$m := \min_{x \in [a, b]} |f^{(p)}(x)| > 0, \quad M := \max_{x \in [a, b]} |f^{(p+1)}(x)| < \infty.$$

Dann konvergiert das Newton-Verfahren

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$

lokal gegen diese Nullstelle. Die Konvergenz ist nur linear. Das modifizierte Newton-Verfahren

$$x_{k+1} = x_k - p \frac{f(x_k)}{f'(x_k)} \quad (8.13)$$

konvergiert lokal quadratisch gegen die Nullstelle.

Beweis Wir verzichten hier darauf, eine quantitative Abschätzung für den Konvergenzradius anzugeben. Mit einem $\omega > 0$ sei

$$x_{k+1} = x_k - \omega \frac{f(x_k)}{f'(x_k)}.$$

Für den Fehler $x_{k+1} - z$ gilt wie im Beweis zu Satz 8.10

$$x_{k+1} - z = \left(1 - \omega \frac{f(x_k)}{(x_k - z)f'(x_k)} \right) (x_k - z). \quad (8.14)$$

Wir entwickeln $f(x_k)$ um die Nullstelle:

$$f(x_k) = \sum_{i=0}^{p-1} \underbrace{\frac{f^{(i)}(z)}{i!} (x_k - z)^i}_{=0} + \frac{f^{(p)}(\xi_{x_k,z})}{p!} (x_k - z)^p = \frac{f^{(p)}(\xi_{x_k,z})}{p!} (x_k - z)^p$$

mit einer Zwischenstelle $\xi_{x_k,z}$ aus $[x_k, z]$ bzw. $[z, x_k]$. Ableiten dieser Formel nach x_k liefert

$$f'(x_k) = \frac{f^{(p)}(\xi_{x_k,z})}{p!} p(x_k - z)^{p-1} + \frac{(x_k - z)^p}{p!} \underbrace{\frac{d}{dx_k} f^{(p)}(\xi_{x_k,z})}_{=: R(x_k, z, p)}.$$

Der Rest $R(x_k, z, p)$ beinhaltet die $p + 1$ -te Ableitung von f . Diese ist nach Voraussetzung beschränkt. Wir setzen diese Entwicklungen in (8.14) ein:

$$\begin{aligned} x_{k+1} - z &= \left(1 - \omega \frac{\frac{f^{(p)}(\xi_{x_k,z})}{p!} (x_k - z)^p}{\frac{f^{(p)}(\xi_{x_k,z})}{(p-1)!} (x_k - z)^p + \frac{R(x_k, z, p)}{p!} (x_k - z)^{p+1}} \right) (x_k - z) \\ &= \left(1 - \frac{\omega}{p} \frac{1}{1 + \frac{R(x_k, z, p)}{p f^{(p)}(\xi_{x_k,z})} (x_k - z)} \right) (x_k - z) \end{aligned}$$

Nach Voraussetzung gilt

$$\frac{|R(x_k, z, p)|}{p|f^{(p)}|} \leq \frac{M}{mp},$$

daher ist für $|x_k - z|$ klein genug (insbesondere kleiner als 1)

$$\frac{|R(x_k, z, p)|}{p|f^{(p)}|} |x_k - z| \leq 1.$$

Mit $(1 + h)^{-1} = 1 + O(h)$ folgt schließlich

$$x_{k+1} - z = \left(1 - \frac{\omega}{p} + O(|x_k - z|)\right) (x_k - z).$$

Für $\omega = 1$, also dem üblichen Newton-Verfahren, folgt lineare Konvergenz, für $\omega = p$ folgt lokal quadratische Konvergenz.

□

Bemerkung 8.19 Dieser Satz zeigt, dass sich das Newton-Verfahren auch für mehrfache Nullstellen eignet. Ist die Vielfachheit p der Nullstelle bekannt, so kann sogar quadratische Konvergenz gewonnen werden. Im Allgemeinen wird es jedoch schwer sein, p optimal zu bestimmen. ♦

Beispiel 8.20 (Double-Well-Potenzial)

Das Double-Well-Potenzial ist ein Polynom von Grad vier

$$f(x) = (1 - x^2)^2,$$

die zunächst eher unschuldig und einfach aussieht. Wir stellen in Abb. 8.4 den Funktionsverlauf dar.

Die Gleichung taucht in verschiedenen Zusammenhängen auf wie z.B. in Quantenmechanik, Phasensfeldgleichungen und Mehrphasenströmungen [27]. Wir suchen eine der beiden Nullstellen $x^* = \pm 1$ mit dem Newton-Verfahren. Auch hier stellen wir fest, dass die Vielfachheit $s = 2$ vorliegt und der Defektschritt gemäß (8.13) modifiziert wer-

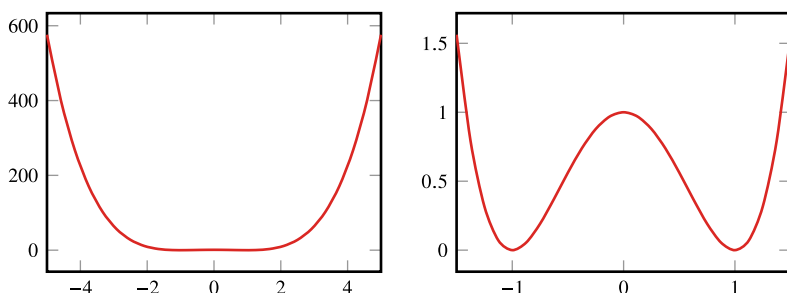


Abb. 8.4 Visualisierung der Double-Well-Funktion mit Fokussierung der beiden Nullstellen im rechten Bild

den sollte. Falls wir mit $x_0 = 10^{-8}$ (also nahe bei null) starten, dann finden wir (ohne Modifikation) in 77 Iterationen die Nullstelle $x = 1$.

```
newton_skalar(f, fabl, 1e-8, x_ex=1, n=100, tol=1e-10)
```

i	x_i	f(x_i)	x_i-x / x
0	0.00000001	1.000e+00	1.000e+00
1	25000000.0	3.906e+29	2.500e+07
2	18750000.0	1.236e+29	1.875e+07
...			
75	1.00001837	1.350e-09	1.837e-05
76	1.00000919	3.375e-10	9.185e-06
77	1.00000459	8.437e-11	4.593e-06

Bei $x_0 \approx 0$ ist die Tangentensteigung nahe bei Null. Das heißt der erste Schnittpunkt mit der x -Achse (siehe geometrische Idee des Newton-Verfahrens) liegt dann weit entfernt bei $x_1 = 2.5 \cdot 10^7$. Von diesem Wert nähert sich das Verfahren dann wieder der Stelle $x = 1$. Unter Ausnutzung der Modifikation 8.13 erhalten wir folgende Ergebnisse:

```
newton_skalar_mod(f, fabl, 1e-8, x_ex=1, s=2, n=100, tol=1e-10)
```

i	x_i	f(x_i)	x_i-x / x
0	1.000e-08	1.000e+00	1.000e+00
1	5.000e+07	6.250e+30	5.000e+07
...			
27	1.147e+00	9.894e-02	1.465e-01
28	1.009e+00	3.540e-04	9.364e-03
29	1.000e+00	7.547e-09	4.343e-05
30	1.000e+00	3.559e-18	9.433e-10

Immerhin erhalten wir Konvergenz in weniger als der Hälfte der Schritte und insbesondere am Ende in den Schritten 27 – 30 quadratische Konvergenz. ◀

Das Newton-Verfahren konvergiert im Einzugsbereich sehr schnell. Ein Nachteil ist natürlich, dass in jedem Schritt des Verfahrens die Ableitung $f'(x_k)$ berechnet werden muss. Das kann unter Umständen sehr aufwendig sein. In vielen Fällen ist man daher an Varianten des Verfahrens interessiert, welche die Berechnung der Ableitung vermeiden. Eine Möglichkeit ist, die Ableitung nicht in jedem Schritt neu zu berechnen:

Algorithmus 8.5: Vereinfachtes Newton-Verfahren

Eingabe: Gegeben sei $f \in C[a, b]$ und Toleranz ϵ .

- 1 Wähle Startwert $x_0 \in [a, b]$
- 2 Wähle $c \in [a, b]$
- 3 Berechne $f'(c)$
- 4 $k = 0$
- 5 **Solange** $|f(x_k)| \geq \epsilon$ **tue**
- 6 $x_{k+1} = x_k - f(x_k)/f'(c)$
- 7 $k = k + 1$

Ergebnis: Approximative Nullstelle $x_k \approx x$.

Unseren Python-Code können wir hierfür folgendermaßen anpassen:

🔗 Implementierung 8.3: Vereinfachtes Newton-Verfahren

```
1 def newton_vereinfacht_skalar(f, f_abl, x, c, n=10, tol=1e-10):
2     f_abl_x_inv = 1 / f_abl(c)
3     for i in range(n):
4         x -= f_abl_x_inv * f(x)
5         if abs(f(x)) < tol:
6             return x
7     print(f'Die Newton Methode ist nicht konvergiert')
8     return x
```

Eine sinnvolle Wahl ist etwa $c = x_0$. Es gilt:

Satz 8.21 (Konvergenz des vereinfachten Newton-Verfahrens) Es sei $f \in C^2([a, b])$ und $z \in (a, b)$ eine Nullstelle. Weiter sei

$$m = \min_{x \in [a, b]} |f'(x)| > 0, \quad M = \max_{x \in [a, b]} |f''(x)| < \infty,$$

sowie $\rho \in \mathbb{R}$ mit

$$q := \frac{2M}{m} \rho < 1, \quad K_\rho(z) \subset [a, b].$$

Dann konvergiert für alle $x_0, c \in K_\rho(z)$ das vereinfachte Newton-Verfahren linear gegen die Nullstelle. Es gilt

$$|x_k - z| \leq q^k |x_0 - z|.$$

Beweis Der Beweis ist eine einfache Modifikation von Satz 8.10. □

Das vereinfachte Newton-Verfahren konvergiert nur noch linear. Die Konvergenz kann jedoch, falls $f'(c)$ eine gute Approximation für $f'(x_k)$ ist, sehr gut sein. Das vereinfachte Newton-Verfahren spielt dann seine Vorteile aus, falls die erste Ableitung aufwendig zu

berechnen ist, was bei den hier betrachteten Beispielen nicht der Fall ist, aber bei der Nullstellensuche im $\mathbb{R}^n$ oder bei Problemen mit Differentialgleichungen vorkommen kann. Ein Kompromiss zwischen schneller Konvergenz und effizienter Durchführung kann erreicht werden, wenn die Ableitung nicht in jedem Schritt, aber z. B. in jedem n -ten Schritt neu aufgebaut wird. Dann kann superlineare Konvergenz gefolgert werden.

Beispiel 8.22 (Vereinfachtes Newton-Verfahren)

Wir betrachten wieder die Funktion aus Beispiel 8.13

$$f(x) = x(1 + \exp(x)) + 10 \sin(3 + \log(x^2 + 1)),$$

und wenden darauf das vereinfachte Newton-Verfahren an. Wir wissen bereits, dass die Funktion eine Nullstelle bei

$$z_1 \approx -9.1635898$$

hat. Daher wählen wir den Startwert $x_0 = -10$ und die Auswertungsstelle für die Ableitung ebenfalls als $c = -10$. Damit erhalten wir bei einer Fehlertoleranz von 10^{-7}

```

it x          f(x)
0 -10.00000000 -2.8437e-01
01 -9.46454160 -6.3943e-02
02 -9.34414034 -3.2920e-02
03 -9.28215371 -1.9753e-02
...
40 -9.16359138 -2.1427e-07
41 -9.16359098 -1.5936e-07
42 -9.16359068 -1.1852e-07
43 -9.16359045 -8.8146e-08

```

Wie benötigen also 43 Schritte um das Residuum auf unter 10^{-7} zu bekommen. Im Vergleich dazu hat das volle Newton-Verfahren in Beispiel 8.13 nur 6 Schritte benötigt um die Lösung bis auf ein Residuum von Maschinengenauigkeit zu bestimmen. Wir beobachten zudem die in Satz 8.21 bewiesene lineare Konvergenz. Nach den ersten Iterationsschritten wird das Residuum in jedem Schritt um den Faktor 0.743 kleiner.

Die Konvergenz hängt allerdings stark von der Wahl von c ab. Nehmen wir stattdessen $c = -9.344$, dann erhalten wir die Lösung nach 20 Schritten bei gleicher Genauigkeit.

```

it x          f(x)
0 -10.00000000 -2.8437e-01
01 -8.75202698  1.0689e-02
02 -8.79893618  1.4171e-02
03 -8.86112739  1.6883e-02
...
18 -9.16358732  3.3841e-07

```

```

19 -9.16358880  1.3629e-07
20 -9.16358940  5.4886e-08

```



Eine weitere Variante des Newton-Verfahrens basiert auf der Approximation der Ableitung mithilfe eines Differenzenquotienten. Wir stellen einen einfachen Algorithmus vor:

Algorithmus 8.6: Approximiertes Newton-Verfahren

Eingabe: Gegeben sei $f \in C[a, b]$ und Toleranz ϵ .

1 Wähle Startwert $x_0 \in [a, b]$

2 Wähle $\delta > 0$ $k = 0$

3 **Solange** $|f(x_k)| \geq \epsilon$ **tue**

4 $y_k = f(x_k)$

5 $z_k = f(x_k + \epsilon)$

6 $x_{k+1} = x_k - \delta y_k / (z_k - y_k)$

7 $k = k + 1$

Ergebnis: Approximative Nullstelle $x_k \approx x$.

In Python kann dies so aussehen:

🔗 Implementierung 8.4: Approximiertes Newton-Verfahren

```

1 def newton_approx_skalar(f, x, eps, n=10, tol=1e-10):
2     for i in range(n):
3         y = f(x)
4         z = f(x + eps)
5         x -= eps * y / (z - y)
6         if abs(f(x)) < tol:
7             return x
8     print(f'Die Approximierte Newton Methode ist nicht konvergiert')
9     return x

```

Jeder Iterationsschritt erfordert die zweifache Auswertung der Funktion f . Hier haben wir den einseitigen Differenzenquotienten gewählt. Ebenso kann der zentrale Differenzenquotient

$$f'(x) = \frac{f(x + \epsilon) - f(x - \epsilon)}{2\epsilon} + O(\epsilon^2)$$

gewählt werden. Dann müssen in jedem Schritt drei Auswertungen der Funktion f erfolgen. Für den einfachen Differenzenquotienten gilt

$$f'(x) = \frac{f(x + \epsilon) - f(x)}{\epsilon} + O(\epsilon)$$

mit $\epsilon \in [x, x + \epsilon]$. Hiermit folgt für die Iteration

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k) + \frac{f''(\xi)}{2}\epsilon} = x_k - \frac{f(x_k)}{f'(x_k)} \left(\frac{1}{1 + \frac{f''(\xi)}{f'(x_k)}\epsilon} \right).$$

Da ϵ klein ist, gilt mit $(1 + h)^{-1} = 1 + O(h)$ weiter

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} + \frac{f(x_k)f''(\xi)}{f'(x_k)^2} O(\epsilon).$$

Die approximierte Newton-Iteration ist also wirklich eine Approximation des üblichen Newton-Verfahrens. Für $\epsilon > 0$ klein genug ist in der praktischen Anwendung gute Konvergenz zu erreichen. Die korrekte Wahl von ϵ kann sich aber als sehr schwierig erweisen. Zu kleine Werte von ϵ führen bei der Approximation der Ableitung zu Abschneidefehlern. Zu große ϵ führen zu einer schlechten Approximation der Ableitungen. In Abschn. 9.3 werden wir das Zusammenspiel zwischen Approximation der Ableitung und Rundungsfehler an einem einfachen Beispiel analysieren.

Beispiel 8.23 (Approximiertes Newton-Verfahren)

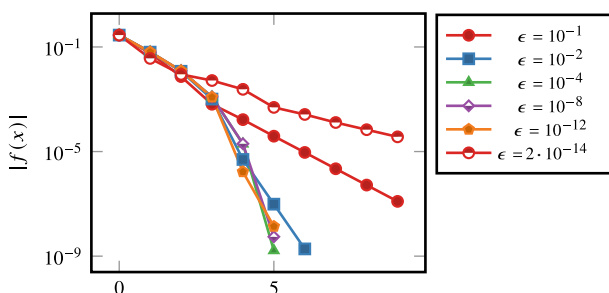
Wir betrachten wieder die Funktion aus Beispiel 8.13 und suchen auch wieder die Nullstelle

$$z_1 \approx -9.1635898.$$

Bei der Anwendung des approximierten Newton-Verfahrens wollen wir insbesondere den Effekt der Wahl der Toleranz ϵ untersuchen. Wir testen daher das Verfahren mit Werten von ϵ zwischen 10^{-1} und $2 \cdot 10^{-14}$. Wir zeigen die Ergebnisse in Abb. 8.5. Hier zeigt sich, dass die korrekte Wahl von ϵ entscheidend ist. Bei zu kleinem ϵ kann es sogar passieren, dass das Verfahren nicht mehr konvergiert. ◀

Eine etwas exotische Variante des Newton-Verfahrens ist die *Sekantenmethode*. Sie ist eng verwandt mit dem approximierten Newton-Verfahren. Die Ableitung in x_k wird approximiert als Differenzenquotient zwischen den beiden zurückliegenden Werten

Abb. 8.5 Approximation der Jacobi-Matrix durch finite Differenzen beim Newton-Verfahren. Optimale Konvergenz erhält man nur, wenn ϵ nicht zu klein und nicht zu groß gewählt wird



$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Dies führt zu der Iteration

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} f(x_k).$$

Gilt im Laufe der Approximation $x_k \rightarrow z$, so folgt $x_k - x_{k-1} \rightarrow 0$, sodass die Genauigkeit der Approximation immer besser wird. Dies lässt auf gute Konvergenz des Verfahrens hoffen. Geometrisch betrachtet wird bei dem Newton-Verfahren eine Tangente an die Funktion gelegt, während bei dem Sekantenverfahren eine Sekante als Approximation zur Ableitung $f'(x)$ genutzt wird.

Algorithmus 8.7: Sekantenverfahren

Eingabe: Gegeben sei $f \in C[a, b]$ und die Toleranz $\epsilon > 0$.

- ¹ Wähle Startwerte $x_0, x_1 \in [a, b]$ mit $x_0 \neq x_1$
- ² $k = 2$
- ³ **Solange** $|f(x_k)| \geq \epsilon$ **do**
- ⁴ $x_{k+1} = x_k - f(x_k)(x_k - x_{k-1}) / (f(x_k) - f(x_{k-1}))$
- ⁵ $k = k + 1$

Ergebnis: Approximative Nullstelle $x_k \approx x$.

Das Sekantenverfahren mag wenig verbreitet sein, ist jedoch interessant als kuriozes Beispiel für ein Verfahren mit einer nicht ganzzahligen Konvergenzordnung. Wir formulieren:

Satz 8.24 (Sekantenverfahren) Es sei $f \in C^2[a, b]$ mit Nullstelle $z \in (a, b)$ und

$$m := \min_{x \in [a, b]} |f'(x)| > 0, \quad M := \max_{x \in [a, b]} |f''(x)|.$$

Weiter sei $\rho > 0$ so gewählt, dass

$$q := \frac{M}{2m} \rho < 1.$$

Für beliebige Startwerte

$$x_0, x_1 \in K_\rho(z) := \{x \in [a, b] : |x - z| < \rho\}$$

mit $x_0 \neq x_1$ konvergiert das Sekantenverfahren gegen die eindeutige Nullstelle. Es gilt die Konvergenzabschätzung

$$|x_k - z| \leq \frac{2m}{M} q^{\mu_k} \gamma_k^k$$

mit einer Konvergenzordnung

$$\gamma_k \rightarrow \frac{1 + \sqrt{5}}{2} \approx 1.618 \quad \text{und} \quad \mu_k \rightarrow \frac{1 + \sqrt{5}}{2\sqrt{5}} \approx 0.723.$$

Beweis (i) Für den Fehler $e_k := x_k - z$ gilt wegen $f(z) = 0$

$$\begin{aligned} e_{k+1} &= x_{k+1} - z = e_k - \frac{f(x_k)}{f(x_k) - f(x_{k-1})}(x_k - x_{k-1}) \\ &= e_k \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} \left(\frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}} - \frac{f(x_k) - f(z)}{x_k - z} \right). \end{aligned} \quad (8.15)$$

Für drei beliebige Werte x, y, z gilt mit dem Hauptsatz der Differential- und Integralrechnung (zweifache Anwendung)

$$\begin{aligned} \frac{f(x) - f(y)}{x - y} - \frac{f(x) - f(z)}{x - z} &= \int_0^1 f'(x + s(y - x)) - f'(x + s(z - x)) ds \\ &= \int_0^1 \int_0^1 f''(x + s(y - x) + rs(z - y))s(z - y) dr ds. \end{aligned}$$

Es folgt

$$\left| \frac{f(x) - f(y)}{x - y} - \frac{f(x) - f(z)}{x - z} \right| \leq M \int_0^1 \int_0^1 s|z - y| dr ds = \frac{M}{2}|z - y|.$$

Aus (8.15) folgern wir so

$$|e_{k+1}| \leq \frac{M}{2m}|e_k||e_{k-1}|$$

bzw.

$$\frac{M}{2m}|e_{k+1}| \leq \left(\frac{M}{2m}|e_k| \right) \left(\frac{M}{2m}|e_{k-1}| \right). \quad (8.16)$$

(ii) Wir zeigen, dass das Verfahren durchführbar ist. Nun seien die Startwerte x_0, x_1 so, dass

$$|e_0| < \rho, \quad |e_1| < \rho \quad \Rightarrow \quad \frac{M}{2m}|e_0| < 1, \quad \frac{M}{2m}|e_1| < 1.$$

Dann folgt aus (8.16) für alle $k \in \mathbb{N}$, dass die Iterierten im Einzugsbereich bleiben.

Wir prüfen als nächstes, dass das Verfahren durchführbar ist, dass also nicht durch Null geteilt wird. Angenommen, $f(x_k) = f(x_{k-1})$. Da $|f'| \geq m > 0$, folgt $x_k = x_{k-1}$. In diesem Fall gilt

$$x_{k-1} = x_k = x_{k-1} - \frac{f(x_{k-1})(x_{k-1} - x_{k-2})}{f(x_{k-1}) - f(x_{k-2})},$$

also zwingend $f(x_{k-1}) = f(x_k) = 0$, und das Verfahren bricht mit der Nullstelle ab. Somit ist die Sekantenmethode unter den gegebenen Voraussetzungen immer durchführbar.

(iii) Die Konvergenz leiten wir aus (8.16) her. Es ist mit

$$q_k := \frac{M}{2m} |e_k| \Rightarrow q_{k+1} \leq q_k q_{k-1} \Rightarrow \log(q_{k+1}) \leq \log(q_k) + \log(q_{k-1}).$$

Logarithmieren ist möglich, da $0 < q < 1$ gilt. Wir definieren die majorisierende Folge

$$Q_0 = Q_1 = \log(q), \quad Q_{k+1} = Q_k + Q_{k-1}, \quad k = 1, 2, \dots$$

Dies ist gerade eine skalierte (und um eins verschobene) Version der Fibonacci-Folge

$$f_1 = f_2 = 1, \quad f_{k+1} = f_k + f_{k-1}, \quad Q_k = \log(q) f_{k+1}.$$

Für diese existiert eine geschlossene Darstellung [60]:

$$f_k = \frac{1}{\sqrt{5}} \left(\left(\frac{1 + \sqrt{5}}{2} \right)^k - \left(\frac{1 - \sqrt{5}}{2} \right)^k \right)$$

Da

$$\frac{1 + \sqrt{5}}{2} \approx 1.618, \quad \frac{1 - \sqrt{5}}{2} \approx -0.618,$$

gilt asymptotisch

$$f_k \sim \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^k.$$

Hiermit erhalten wir für Q_k die Abschätzung

$$Q_k = \log(q) f_{k+1} \approx \log(q) 0.723 \cdot 1.618^{k+1}.$$

Für den Fehler der Sekantenmethode gilt dann

$$q_k \leq \exp(\log(q) Q_k) \leq q^{Q_k} \leq q^{0.723 \cdot 1.618^{k+1}},$$

also

$$|x_k - z| \leq \frac{2m}{M} (q^{1.17})^{1.618^k}.$$

□

Die Sekantenmethode ist sehr effizient in der Durchführung. In jedem Schritt muss nur eine neue Auswertung der Funktion $f(\cdot)$ vorgenommen werden. Bei der Newton-Methode muss zusätzlich die Ableitung berechnet werden. Zwei Schritte der Sekantenmethode haben somit etwa den Aufwand von einem Schritt der Newton-Methode, liefern aber die Konvergenzordnung $1.618^2 \approx 2.618$. Es liegt somit schnellere Konvergenz vor. Ein

Problem der Sekantenmethode ist jedoch die mangelnde Stabilität. Bei der Berechnung des Differenzenquotienten droht Auslöschung, insbesondere falls beide Iterationswerte x_k und x_{k-1} auf derselben Seite der Nullstelle liegen.

In Kombination mit der bereits diskutierten Intervallschachtelung in Abschn. 8.2 erhalten wir ein stabilisiertes Verfahren:

Algorithmus 8.8: Regula falsi

Eingabe: Gegeben sei $f \in C[a, b]$ und Toleranz ϵ .

1 Wähle Startwerte $x_0, x_1 \in [a, b]$ mit $f(x_0) \cdot f(x_1) < 0$ $k = 1$

2 **Solange** $|f(x_k)| \geq \epsilon$ **tue**

3 $\tilde{x}_{k+1} := x_k - f(x_k) \cdot (x_k - x_{k-1}) / (f(x_k) - f(x_{k-1}))$

4 **Wenn** $f(x_k) \cdot f(\tilde{x}_{k+1}) < 0$ **dann**

5 $x_{k+1} = \tilde{x}_{k+1}$

6 **Sonst**

 /* Es gilt $f(x_{k-1}) \cdot f(\tilde{x}_{k+1}) < 0$ */

7 $x_{k+1} = \tilde{x}_{k+1}$

8 $x_k = x_{k-1}$

9 $k = k + 1$

Ergebnis: Approximative Nullstelle $x_k \approx x$.

Die *Regula falsi*-Methode stellt sicher, dass die beiden letzten Approximationen stets auf der anderen Seite der Nullstelle liegen. Auslöschung kann so vermieden werden. Es geht jedoch im Allgemeinen auch die schnelle Konvergenz der Sekantenmethode verloren. Man kann Beispiele konstruieren, bei denen Regula falsi noch weit schlechter konvergiert als die einfache Intervallschachtelung. Für Details verweisen wir auf die Literatur [35].

8.3.2 Konvergenzbegriffe

Im Folgenden untersuchen wir das Konvergenzverhalten der bisher analysierten Verfahren. Wir definieren

► **Definition 8.25 (Konvergenzordnung)** Wir nennen ein Iterationsverfahren mit der Folge $(x_k)_{k \in \mathbb{N}}$ zur Approximation einer Größe z (z. B. einer Nullstelle) konvergent mit der *Ordnung* p mit $p \geq 1$, wenn gilt

$$\|x_k - z\| \leq c \|x_{k-1} - z\|^p$$

mit einer festen Konstanten $c > 0$. Im Fall $p = 1$, d.h. *linearer* Konvergenz, heißt die *kleinste* Konstante $c \in (0, 1)$ die *lineare* Konvergenzrate. Gilt bei linearer Konvergenz zusätzlich $c_k \rightarrow 0$ für $k \rightarrow \infty$, sprich

$$\|x_k - z\| \leq c_k \|x_{k-1} - z\|, \quad c_k \rightarrow 0$$

so sprechen wir von *superlinearer* Konvergenz.

Für die Intervallschachtelung gilt

$$|x_k - z| \leq \frac{b-a}{2^{k+1}}$$

mit der Konvergenzrate $\frac{1}{2}$ und der Konvergenzordnung $p = 1$ (lineare Konvergenz).

Das Newton-Verfahren besitzt (lokal) in der Umgebung einer Nullstelle das Konvergenzverhalten

$$|x_k - z| \leq c |x_{k-1} - z|^2.$$

Wir sprechen von einem *quadratisch* konvergenten Verfahren oder auch von einem Verfahren *2ter Ordnung*. Schließlich konvergiert die Sekantenmethode mit der Ordnung 1.618, also

$$|x_k - z| \leq c |x_{k-1} - z|^{1.618}.$$

Grundsätzlich gilt (zumindest asymptotisch), dass superlinear konvergente Folgen schneller als (schlicht) lineare Folgen konvergieren. Außerdem konvergieren Verfahren mit Ordnung $p + 1$ schneller als Verfahren mit der Ordnung p . Außerdem gilt: Je kleiner die Konvergenzrate c ist, desto schneller ist die Konvergenz. Allerdings hat die Konvergenzordnung wesentlich größeren Einfluss auf die Konvergenzgeschwindigkeit, als die Konvergenzrate.

Zur Abstraktion des Konvergenzbegriffes betrachten wir nun Fixpunktprobleme der Form

$$x_{k+1} = g(x_k), \quad k = 0, 1, 2, \dots$$

Angenommen die Abbildung $g(\cdot)$ ist stetig differenzierbar, so gilt mit dem Mittelwertsatz der Analysis, Satz 7.2

$$\left| \frac{x_{k+1} - z}{x_k - z} \right| = \left| \frac{g(x_k) - g(z)}{x_k - z} \right| \rightarrow |g'(z)| \text{ für } k \rightarrow \infty \quad (8.17)$$

Hieraus folgern wir, dass die lineare Konvergenzrate asymptotisch für $k \rightarrow \infty$ gerade gleich $|g'(z)|$ ist. Dementsprechend liegt im Fall $g'(z) = 0$ (mindestens) superlineare Konvergenz vor. Im Fall $|g'(z)| > 1$ konvergiert die Fixpunktiteration im Allgemeinen nicht.

Aus (8.17) leiten wir die folgende Definition ab:

► **Definition 8.26 (Konvergenzordnung)** Ein Fixpunkt z einer stetig differenzierbaren Abbildung $g(\cdot)$ heißt *anziehend*, falls $|g'(z)| < 1$ ist. Im Fall $|g'(z)| > 1$ wird ein Fixpunkt *abstoßend* genannt.

Der folgende Satz liefert ein einfaches Kriterium zur Bestimmung der Konvergenzordnung einer differenzierbaren Fixpunktiteration. Er kann auch zur Konstruktion von Verfahren mit hoher Ordnung genutzt werden:

Satz 8.27 (Konvergenzordnung von Fixpunktiterationen) Die Funktion $g(\cdot)$ sei in einer Umgebung des Fixpunktes $z = g(z)$ p -mal stetig differenzierbar. Die Fixpunktiteration $x_{k+1} = g(x_k)$ hat genau dann die lokale Konvergenzordnung p , wenn

$$g'(z) = \dots = g^{(p-1)}(z) = 0 \quad \text{und} \quad g^{(p)}(z) \neq 0.$$

Beweis (i) „ $\Leftarrow$ “

Es sei $g'(z) = \dots = g^{(p-1)}(z) = 0$. Mithilfe der Taylor-Formel gilt

$$x_{k+1} - z = g(x_k) - g(z) = \sum_{j=1}^{p-1} \frac{(x_k - z)^j}{j!} g^{(j)}(z) + \frac{(x_k - z)^p}{p!} g^{(p)}(\xi_k)$$

für $\xi_k \in (x_{k+1}, z)$. Damit erhalten wir die Abschätzung

$$|x_{k+1} - z| \leq \frac{1}{p!} \max |g^{(p)}| |x_k - z|^p.$$

(ii) „ $\Rightarrow$ “

Es sei nun umgekehrt die Iteration von p -ter Ordnung, sprich

$$|x_{k+1} - z| \leq c |x_k - z|^p. \quad (8.18)$$

Falls es ein minimales $m \leq p-1$ mit $g^{(m)}(z) \neq 0$ gäbe, aber $g^{(j)}(z) = 0$, $j = 1, \dots, m-1$, so würde jede Iterationsfolge $(x_k)_{k \in \mathbb{N}}$ mit hinreichend kleinem $|x_0 - z| \neq 0$ notwendig gegen z konvergieren wie

$$|x_k - z| = \left| \frac{1}{m!} g^{(m)}(\xi_k) \right| |x_{k-1} - z|^m,$$

also mit der Ordnung $m < p$. Hier finden wir aber einen Widerspruch zu (8.18), also der Annahme, dass die Iteration von der Ordnung p ist. Also muss $g^{(j)}(z) = 0$ für $j = 1, \dots, p-1$ und $g^{(p)}(z) \neq 0$ gelten. $\square$

Wir überprüfen den Satz am Beispiel des Newton-Verfahrens, das wir mittels

$$g(x) = x - \frac{f(x)}{f'(x)}$$

mit Hilfe einer Fixpunktabbildung schreiben können. Es gilt

$$g'(x) = 1 - \frac{f'(x)^2 - f(x)f''(x)}{f'(x)^2} = f(x) \frac{f''(x)}{f'(x)^2} \Rightarrow g'(z) = 0,$$

sowie

$$g''(x) = \frac{f''(x)}{f'(x)} + \frac{f(x)f'''(x)}{f'(x)^2} - \frac{2f(x)f''(x)^2}{f'(x)^3} \Rightarrow g''(z) = \frac{f''(z)}{f'(z)},$$

was im Allgemeinen Fall nicht Null ist. Das Newton-Verfahren hat also – wie wir bereits wissen – zweite Ordnung. Die Analyse zeigt aber auch, dass im Fall von Funktionen mit $f''(z) = 0$ mindestens dritte Ordnung vorliegt. Die Funktion $f(x) = \sin(x)$ ist so ein Beispiel.

Wir gehen nun umgekehrt vor und suchen mit dem Ansatz

$$g(x) = x + h(x)f(x) \quad (8.19)$$

mit einer noch zu bestimmenden Funktion $h(x)$ eine Fixpunktiteration mit möglichst hoher Konvergenzordnung. Zunächst gilt

$$g(z) = z + h(z) \underbrace{f(z)}_{=0} = z,$$

d. h., es liegt wirklich eine Fixpunktiteration vor. Für die erste Ableitung gilt

$$g'(z) := 1 + h'(z) \underbrace{f(z)}_{=0} + h(z)f'(z) \stackrel{!}{=} 0.$$

Hieraus folgt die Bedingung an $h(x)$ in z

$$h(z) = -\frac{1}{f'(z)}.$$

Wir wählen $h(x)$ nun für alle x gemäß diese Vorschrift, sodass

$$h(x) := -\frac{1}{f'(x)}.$$

Einsetzen in (8.19) liefert die Iterationsfunktion

$$g(x) := x - \frac{f(x)}{f'(x)},$$

und damit die Iterationsvorschrift des Newton-Verfahrens

$$x_{k+1} := g(x_k) = x_k - \frac{f(x_k)}{f'(x_k)}, \quad k = 0, 1, 2, \dots$$

8.4 Nullstellensuche im $\mathbb{R}^n$

In diesem Kapitel betrachten wir Funktionen $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ und suchen Nullstellen $z \in \mathbb{R}^n$. Im Fall $n > 1$ handelt es sich hierbei um nichtlineare Gleichungssysteme. Lineare Gleichungssysteme lassen sich als Spezialfall dieser Aufgabe schreiben. Für eine Matrix $A \in \mathbb{R}^{n \times n}$ und einen Vektor $b \in \mathbb{R}^n$ lässt sich die Lösung des LGS $Ax = b$ als Nullstellensuche mit der Funktion

$$f(x) = b - Ax$$

schreiben.

Auch wenn diese Notation einen Bruch zum Bisherigen bedeutet, werden wir bei mehrdimensionalen Problemen den Iterationsindex k stets oben anbringen, also $x^{(k)}$ statt x_k . Als unteren Index wählen wir die Komponente der vektorwertigen Funktion.

Die Intervallschachtelung hat im $\mathbb{R}^n$ kein Gegenstück. Wir wollen dennoch mit einem entsprechend einfachen Verfahren beginnen und orientieren uns an der allgemeinen Konvergenzaussage für Fixpunktiterationen aus Satz 8.27. Es sei $f : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$. Dann wählen wir eine Matrix $C \in \mathbb{R}^{n \times n}$ und definieren die Fixpunktiteration

$$x^{(0)} \in \mathbb{R}^n, \quad x^{(k+1)} = g(x^{(k)}) := x^{(k)} + C^{-1}f(x^{(k)}), \quad k = 0, 1, 2, \dots$$

Dieses Verfahren konvergiert, falls f auf einer Kugel $K_\rho(z) \subset \mathbb{R}^n$ stetig differenzierbar ist und

$$\sup_{\zeta \in K_\rho(z)} \|g'(\zeta)\| = \sup_{\zeta \in K_\rho(z)} \|I + C^{-1}Df(\zeta)\| =: q < 1.$$

8.4.1 Newton-Verfahren im $\mathbb{R}^n$

Es sei $f : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$. Wir suchen eine Lösung vom Nullstellenproblem $f(x) = (f_1(x), \dots, f_n(x)) = 0$. Es sei $x^{(0)} \in D$ ein Startwert und wir approximieren die Nullstelle mit einer Iteration $x^{(k)}$. Für $x^{(k)} \in D$ beliebig linearisieren wir f mit der Taylor-Approximation.

$$f(x^{(k)} + w^{(k)}) = f(x^{(k)}) + \sum_{j=1}^n \frac{\partial f}{\partial x_j}(x^{(k)})w_j^{(k)} + O(|w^{(k)}|^2).$$

Die neue Iteration $x^{(k+1)} = x^{(k)} + w^{(k)}$ ist als Nullstelle der Linearisierung definiert. Das heißt, wir suchen die Lösung $w^{(k)} \in \mathbb{R}^n$ des linearen Gleichungssystems

$$\sum_{j=1}^n \frac{\partial f_i}{\partial x_j}(x^{(k)})w_j^{(k)} = -f_i(x^{(k)}), \quad i = 1, \dots, n.$$

Mit der Jacobi-Matrix $Df : \mathbb{R}^n \rightarrow \mathbb{R}^{n \times n}$

$$Df = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_n} \\ \vdots & & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_1} & \frac{\partial f_n}{\partial x_2} & \dots & \frac{\partial f_n}{\partial x_n} \end{pmatrix}$$

schreibt sich das zu lösende lineare Gleichungssystem kurz als

$$Df(x^{(k)})w^{(k)} = -f(x^{(k)}).$$

Hiermit lautet die Newton-Iteration mit Startwert $x^{(0)} \in D$ für $k = 0, 1, 2, \dots$

$$Df(x^{(k)})w^{(k)} = -f(x^{(k)}), \quad x^{(k+1)} = x^{(k)} + w^{(k)}. \quad (8.20)$$

Die Tatsache, dass die Ableitung von $f(x)$ im mehrdimensionalen Fall eine Matrix ist, stellt den wesentlichen Unterschied zum eindimensionalen Newton-Verfahren dar. Anstelle einer Ableitung sind nun n^2 Ableitungen zu berechnen. Und anstelle einer Division durch die Ableitung ist in jedem Schritt der Newton-Iteration ein lineares Gleichungssystem mit Koeffizientenmatrix $Df(x_k) \in \mathbb{R}^{n \times n}$ zu lösen, siehe Kap. 3.

Iteration (8.20) ist als *Defektkorrektur-Iteration* ausgeführt. Wir definieren analog zu Definition 3.32:

► **Definition 8.28 (Defekt)** Es sei $\tilde{x} \in \mathbb{R}^n$ eine Approximation der Lösung von $f(x) = y$. Mit

$$d(\tilde{x}) = y - f(\tilde{x})$$

wird der *Defekt* bezeichnet. Der Defekt wird auch *Residuum* genannt.

Beim Newton-Verfahren wählen wir meist $y = 0$ und eine mögliche rechte Seite ist in $f(\cdot)$ beinhaltet.

Ein Konvergenzbeweis für das mehrdimensionale Newton-Verfahren kann wie im eindimensionalen Fall mit der Taylor-Entwicklung geführt werden. Wir geben hier aber den Satz von Newton-Kantorovich an, der das Ergebnis einerseits erweitert und die Anforderungen an die Funktion f leicht abschwächt.

Satz 8.29 (Newton-Kantorovich) Es sei $D \subset \mathbb{R}^n$ eine offene und konvexe Menge. Weiterhin sei $f : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$ stetig-differenzierbar.

(1) Die Jacobi-Matrix Df sei gleichmäßig Lipschitz-stetig für alle $x, y \in D$:

$$\|Df(x) - Df(y)\| \leq L\|x - y\|, \quad x, y \in D, \quad (8.21)$$

mit einem $L < \infty$.

(2) Die Jacobi-Matrix habe auf D eine gleichmäßig beschränkte Inverse

$$\|Df(x)^{-1}\| \leq \beta, \quad x \in D, \quad (8.22)$$

mit einem $\beta < \infty$.

(3) Es gelte für den Startpunkt $x^{(0)} \in D$

$$q := \alpha\beta L < \frac{1}{2}, \quad \alpha := \|Df(x^{(0)})^{-1} f(x^{(0)})\|. \quad (8.23)$$

(4) Für $r := 2\alpha$ sei die abgeschlossene Kugel

$$K_r(x^{(0)}) := \{x \in \mathbb{R}^n : \|x - x^{(0)}\| \leq r\}$$

in der Menge D enthalten.

Dann besitzt die Funktion f genau eine Nullstelle $z \in K_r(x^{(0)})$ und die Newton-Iteration

$$Df(x^{(k)})w = -f(x^{(k)}), \quad x^{(k+1)} = x^{(k)} + w, \quad k = 0, 1, 2, \dots,$$

konvergiert quadratisch gegen diese Nullstelle z . Es gilt die A-priori-Fehlerabschätzung

$$\|x^{(k)} - z\| \leq 2\alpha q^{2^k - 1}, \quad k = 0, 1, \dots$$

Beweis Wir folgen hier dem Beweis aus [71], eng an der Arbeit von Kantorovich [2]. Alternative Darstellungen finden sich z. B. bei [24].

Der Beweis zum Satz ist weitaus aufwendiger als im eindimensionalen Fall, daher geben wir zunächst eine Skizze an:

- (i) Herleitung von zwei Hilfsabschätzungen.
- (ii) Wir zeigen: Alle Iterierten $x^{(k)}$ liegen in der Kugel $K_r(x^{(0)})$. Weiter gilt die A-priori-Fehlerabschätzung.
- (iii) Wir zeigen, dass die Iterierten $(x^{(k)})_{k \in \mathbb{N}}$ eine Cauchy-Folge sind und somit einen Grenzwert z haben.
- (iv) Wir zeigen, dass dieser Grenzwert eine Nullstelle von f sein muss.
- (v) Wir zeigen, dass es nur eine Nullstelle in $K_r(x^{(0)})$ geben kann.
- (vi) Wir zeigen die quadratische Konvergenz.

(i) Es seien $x, y, z \in D$. Da D konvex ist, gilt für alle $x, y \in D$

$$f(y) - f(x) = \int_0^1 Df(y + s(y - x))(y - x) ds.$$

Wir ziehen auf beiden Seiten $Df(z)(y - x)$ ab:

$$f(y) - f(x) - Df(z)(y - x) = \int_0^1 ((Df(x + s(y - x)) - Df(z))(y - x) ds$$

Hiermit erhalten wir

$$\begin{aligned}\|x^{(k+1)} - x^{(0)}\| &\leq \|x^{(k+1)} - x^{(k)}\| + \dots + \|x^{(1)} - x^{(0)}\| \\ &\leq \alpha(1 + q + q^3 + q^7 + \dots + q^{2^{(k)}-1}) \\ &\leq \frac{\alpha}{1-q} \leq 2\alpha = r,\end{aligned}$$

d. h., es gilt $x^{(k+1)} \in K_r(x^{(0)})$. Damit ist der Induktionsschritt von $k \rightarrow k+1$ gezeigt, d. h., die beiden Ungleichungen (8.26) sind gültig für $k+1$.

(iii) Nun zeigen wir, dass die $x^{(k)} \in K_r(x^{(0)})$ eine Cauchy-Folge bilden. Es sei $m > 0$. Da $q < \frac{1}{2}$, gilt

$$\begin{aligned}\|x^{(k)} - x^{(k+m)}\| &\leq \|x^{(k)} - x^{(k+1)}\| + \dots + \|x^{(k+m-1)} - x^{(k+m)}\| \\ &\leq \alpha(q^{2^{(k)}-1} + q^{2^{(k+1)}-1} + \dots + q^{2^{m+k-1}-1}) \\ &= \alpha q^{2^{(k)}-1} (1 + q^{2^{(k)}} + \dots + (q^{2^{(k)}})^{2^{m-1}-1}) \\ &\leq 2\alpha q^{2^{(k)}-1}.\end{aligned}\tag{8.27}$$

Damit ist gezeigt, dass $(x^{(k)}) \subset D$ eine Cauchy-Folge ist, da $q < \frac{1}{2}$. Im Banach-Raum $\mathbb{R}^n$ existiert der Limes

$$z = \lim_{k \rightarrow \infty} x^{(k)} \in \mathbb{R}^n.$$

Im Grenzübergang $k \rightarrow \infty$ erhalten wir dann mit (8.26)

$$\|z - x^{(0)}\| \leq r,$$

sodass $z \in K_r(x^{(0)})$. Im Grenzübergang $m \rightarrow \infty$ in (8.27) verifizieren wir die Fehlerabschätzung des Satzes:

$$\|x^{(k)} - z\| \leq 2\alpha q^{2^{(k)}-1}, \quad k = 0, 1, \dots$$

(iv) Es bleibt zu zeigen, dass $z \in K_r(x^{(0)})$ eine Nullstelle von f ist. Die Newton-Iterationsvorschrift sowie Bedingung (8.21) liefern

$$\begin{aligned}\|f(x^{(k)})\| &= \|Df(x^{(k)})(x^{(k+1)} - x^{(k)})\| \\ &\leq \|Df(x^{(k)}) - Df(x^{(0)}) + Df(x^{(0)})\| \|x^{(k+1)} - x^{(k)}\| \\ &\leq \left(L\|x^{(k)} - x^{(0)}\| + \|Df(x^{(0)})\| \right) \|x^{(k+1)} - x^{(k)}\| \rightarrow 0 \quad (k \rightarrow \infty).\end{aligned}$$

Daher gilt

$$f(x^{(k)}) \rightarrow 0, \quad k \rightarrow \infty.$$

Die Stetigkeit von f impliziert dann $f(z) = 0$.

(v) Schließlich ist zu beweisen, dass die gefundene Nullstelle $z \in K_r(x^{(0)})$ die einzige sein kann. Die Eindeutigkeit wird mithilfe der Kontraktionseigenschaft und der Formulierung

der Newton-Iteration als Fixpunktiteration gezeigt. Jede Nullstelle von $f(\cdot)$ ist Fixpunkt der vereinfachten Newton-Iteration:

$$g(x) := x - Df(x^{(0)})^{-1}f(x)$$

Die Fixpunktfunktion $g(\cdot)$ ist Lipschitz-stetig. Mit (8.25) gilt

$$\begin{aligned} \|g(x) - g(y)\| &= \|x - y - Df(x^{(0)})^{-1}f(x) + Df(x^{(0)})^{-1}f(y)\| \\ &= \|Df(x^{(0)})^{-1}(f(y) - f(x) - Df(x^{(0)})(y - x))\| \\ &\leq \underbrace{\|Df(x^{(0)})^{-1}\|}_{\leq \beta} rL \|y - x\| \\ &\leq \beta Lr \|y - x\|, \end{aligned}$$

für alle $x, y \in K_r(x^{(0)})$. Mit $r = 2\alpha$ folgt $\beta Lr = 2\alpha\beta L \leq 2q < 1$. Das heißt, g ist eine Kontraktion. Der Banachsche Fixpunktsatz sagt, dass es nur einen Fixpunkt, also eine Nullstelle gibt.

(vi) Durch die Definition der Newton-Iteration, (8.22) und der Hilfsabschätzung (8.24), haben wir

$$\begin{aligned} \|x^{(n+1)} - z\| &= \|x^{(n)} - Df(x^{(n)})^{-1}f(x^{(n)}) - z\| \\ &\leq \|Df(x^{(n)})^{-1}\| \underbrace{\|f(z) - f(x^{(n)}) - Df(x^{(n)})(z - x^{(n)})\|}_{=0} \\ &\leq \beta \frac{L}{2} \|x^{(n)} - z\|^2. \end{aligned}$$

□

Bemerkung 8.30 Der Satz von Newton-Kantorovich unterscheidet sich in einigen Punkten vom einfachen Satz 8.10 über das eindimensionale Newton-Verfahren. Die Existenz der Nullstelle folgt bei Newton-Kantorovich als Resultat der Aussage. Beim einfachen Beweis wurde die Existenz gefordert. Entscheidend ist auch die geforderte Regularität. Dem Satz von Newton-Kantorovich genügt eine Lipschitz-stetige erste Ableitung. Der Satz von Newton-Kantorovich gilt natürlich auch im eindimensionalen Fall. ♦

Aus Satz 8.29 können wir ein lokales Konvergenzresultat folgern:

Korollar 8.31 Es sei $D \subset \mathbb{R}^n$ offen und $f : D \subset \mathbb{R}^n \rightarrow \mathbb{R}^n$ zweimal stetig-differenzierbar. Wir nehmen an, dass $z \in D$ eine Nullstelle mit regulärer Jacobi-Matrix $Df(z)$ ist. Dann ist das Newton-Verfahren lokal konvergent, d. h., es existiert eine Umgebung B um z , sodass das Newton-Verfahren für alle Startwerte $x^{(0)} \in B$ konvergiert.

Beispiel 8.32 (Newton im $\mathbb{R}^n$)

Wir suchen die Nullstelle der Funktion

$$f(x_1, x_2) = \begin{pmatrix} 1 - x_1^2 - x_2^2 \\ (x_1 - 2x_2)/(1/2 + x_2) \end{pmatrix}$$

mit der Jacobi-Matrix

$$Df(x) = \begin{pmatrix} -2x_1 & -2x_2 \\ \frac{2}{1+2x_2} & -\frac{4+4x_1}{(1+2x_2)^2} \end{pmatrix}.$$

Die Nullstellen von f sind gegeben durch

$$x \approx \pm(0.894427191, 0.447213595).$$

Für die Implementierung in Python verwenden wir unsere LR-Zerlegung mit Pivotisierung um die entstehenden linearen Gleichungssysteme zu Lösen. Der Code dazu ist

♣ Implementierung 8.5: Newton-Verfahren in $\mathbb{R}^n$

```

1 def newton_vek(f, D, x, n=10, tol=1e-10):
2     for i in range(n):
3         b = - f(x)
4         if np.linalg.norm(b) < tol:
5             return i, x
6         jac = D(x)
7         pivot = LR_zerlegung_mit_pivot(jac)
8         for p in pivot:
9             b[p] = b[[p[1], p[0]]]
10        y = vorwaerts_einsetzen_ohne_diag(jac, b)
11        w = rueckwaerts_einsetzen(jac, y)
12        x[:] += w
13    else:
14        print('Die Newton Methode ist nicht konvergiert!')
15    return i, x

```

Um das obige Beispiel hiermit zu Lösen, müssen wir noch die Funktion und Jacobi-Matrix implementieren:

```

1 def f(x):
2     x, y = x[:]
3     return np.array([1 - x**2 - y**2, (x - 2 * y) / (1 / 2 + y)],
4                     ↪ dtype=np.double)

```

```

1 def Df(x):
2     x, y = x[:]
3     return np.array([[ -2 * x, -2 * y],
4                       [ 2 / (1 + 2 * y), - (4 + 4 * x) / (1 + 2 *
                        ↪ y)**2]], dtype=np.double)

```

Wir starten die Iteration mit $x^{(0)} = (1, 1)^T$

```

1 n, x = newton_vek(f, Df, x=np.array([1.0, 1.0]), n=15, tol=1e-10)
2 print(f'\nn = {n}, x = {x}')

```

und erhalten nach fünf Schritten die Lösung

```
n = 5, x = [0.89442719 0.4472136 ]
```

Vergleichen wir die Lösung in den einzelnen Schritten,

```

x_1 = (1.1428571429, 0.3571428571)
x_2 = (0.9265873016, 0.4420634921)
x_3 = (0.8949352419, 0.4473485186)
x_4 = (0.8944273077, 0.4472136708)
x_5 = (0.8944271910, 0.4472135955)

```

sehen wir, dass die Lösung auf mindesten sechs Nachkommastellen exakt bestimmt wurde. Vergleichen wir die Lösung mit dem exakten Wert, sehen wir, dass die Lösung sogar auf neun Nachkommastellen exakt bestimmt wurde. ◀

Im Gegensatz zum eindimensionalen Fall ist die Berechnung der Ableitung im $\mathbb{R}^n$ möglicherweise sehr aufwendig, da nicht eine, sondern n^2 partielle Ableitungen

$$(Df)_{ij} = \frac{\partial f_i}{\partial x_j}, \quad i, j = 1, \dots, n$$

berechnet werden. Der Aufwand hierzu kann wesentlich sein. Im zweiten Schritt des Newton-Verfahrens muss ein lineares Gleichungssystem gelöst werden. Dies kann z.B. mit einer Zerlegungsmethode in $O(n^3)$ Operationen geschehen. Das anschließende Lösen mit Vorwärts- und Rückwärtseinsetzen benötigt weitere $O(n^2)$ Operationen.

Wir haben beim eindimensionalen Newton-Verfahren bereits das vereinfachte Newton-Verfahren kennengelernt, Algorithmus 8.5. Wird die Ableitung $Df(x^{(k)})$ nicht zur stets aktuellen Näherung $x^{(k)}$ bestimmt, sondern zu einem festen Wert $Df(c)$, so liegt immer noch Konvergenz vor (wenn auch nur noch lineare Konvergenz). Beim mehrdimensionalen Newton-Verfahren kann diese Vereinfachung ihre Stärke ganz ausspielen, z.B., wenn das lineare Gleichungssystem mit der LR-Zerlegung gelöst wird:

Algorithmus 8.9: Vereinfachtes Newton-Verfahren

Eingabe: Gegeben sei $f \in C^1(D)$, ein Startwert $x^{(0)} \in D$, $c \in D$ sowie eine Toleranz $\epsilon > 0$.

- 1 Berechne die Jacobi-Matrix $Df(c)$
- 2 Erstelle die LR-Zerlegung $Df(c) = L(c)R(c)$
- 3 $k = 0$
- 4 **Solange** $\|f(x_k)\| \geq \epsilon$ **tue**
- 5 $L(c)R(c)w^{(k)} = -f(x^{(k-1)})$
- 6 $x^{(k)} = x^{(k-1)} + w^{(k)}$

Ergebnis: Approximative Nullstelle $x_k \approx x \in D$.

Beispiel 8.33 (Vereinfachtes Newton-Verfahren im $\mathbb{R}^n$)

Wir wenden das vereinfachte Newton-Verfahren auf die Funktion aus Beispiel 8.32 an. Dafür müssen wir zunächst unsere vektorwertige Implementierung anpassen, in dem wir die LR-Zerlegung nur einmal außerhalb der Schleife berechnen.

🔗 Implementierung 8.6: Vereinfachtes Newton-Verfahren in $\mathbb{R}^n$

```

1 def newton_vereinfacht_vek(f, D, x, n=10, tol=1e-10):
2     jac = D(x)
3     pivot = LR_zerlegung_mit_pivot(jac)
4     for i in range(n):
5         b = - f(x)
6         if np.linalg.norm(b) < tol:
7             return i, x
8         for p in pivot:
9             b[p] = b[[p[1], p[0]]]
10        y = vorwaerts_einsetzen_ohne_diag(jac, b)
11        w = rueckwaerts_einsetzen(jac, y)
12        x[:] += w
13    else:
14        print(f'Das vereinfachte Newton-Verfahren ist nicht
15              ↳ konvergiert!')
16    return i, x

```

Wenden wir diesen Code nun mit dem besseren Startwert $x^{(0)} = (1, 0.5)^T$ an, sehen wir, dass wir die Nullstelle nach 10 Schritten erreichen

```

1 n, x = newton_vereinfacht_vek(f, Df, x=np.array([1.0, 0.5]), n=15)
2 print(f'n = {n}, x = {x}')

```

$n = 10, x = [0.89442719 \ 0.4472136]$

Mit dem Startvektor $x^{(0)} = (1, 1)^T$ braucht das vereinfachte Newton Verfahren jedoch 670 Schritte um die Nullstelle auf einen Fehlertolerante $< 10^{-10}$ zu bestimmen. Wir müssen also immer zwischen den Kosten der Matrixzerlegung im Newton-Verfahren und der langsameren Konvergenz im vereinfachten Newton-Verfahren abwägen. ◀

Die beiden teuren Schritte des Newton-Verfahrens, Aufstellung und Zerlegen der Jacobi-Matrix, werden jeweils nur einmal durchgeführt. Die eigentliche Iteration kann mit Vorwärts- und Rückwärtseinsetzen effizient durchgeführt werden. Dieses Verfahren kann verfeinert werden, indem die Linearisierungsstelle $c \in \mathbb{R}^n$ nicht statisch gewählt, sondern regelmäßig aktualisiert wird. Eine Aktualisierung kann entweder nach einer festen Anzahl von Schritten $c = x^{(3)}$, $c = x^{(6)}$, ... oder immer dann durchgeführt werden, wenn das Verfahren schlecht konvergiert. Zur Kontrolle der Konvergenz eignet sich z. B. das Residuum zweier aufeinanderfolgender Iterationen

$$\rho_k = \frac{\|f(x^{(k)})\|}{\|f(x^{(k-1)})\|}.$$

Falls $\rho_k > \rho_0$ mit einer Grenze, z. B. $\rho_0 \approx 0.1$ gilt, falls die Konvergenz also schlecht ist, so wird mit $c = x^{(k)}$ ein Update der Jacobi-Matrix durchgeführt.

8.4.2 Globalisierung des Newton-Verfahrens

Die große Herausforderung bei der praktischen Durchführung des Newton-Verfahrens ist die Wahl eines guten Startwerts. Eine Globalisierung, also eine Vergrößerung des Konvergenzbereichs kann durch Dämpfung erreicht werden. Hierzu wird Schritt 6 in Algorithmus 8.9 durch die Vorschrift

$$x^{(k)} = x^{(k-1)} + \omega_k w^{(k)}$$

ersetzt, mit einem Dämpfungsparameter $\omega_k \in (0, 1]$.

Die Vergrößerung des Konvergenzbereichs (Globalisierung) kann mithilfe eines Dämpfungsparameters erreicht werden. Hierzu wird die Newton-Iteration abgewandelt:

$$x^{(k+1)} = x^{(k)} - \omega_k Df(x^{(k)})^{-1} f(x^{(k)}).$$

Satz 8.34 (Gedämpftes Newton-Verfahren) Es gelten die Voraussetzungen von Satz 8.29 mit Ausnahme der Bedingung für den Startwert, (8.23). Die gedämpfte Newton-Iteration

$$Df(x^{(k)})w = -f(x^{(k)}), \quad x^{(k+1)} = x^{(k)} + \omega_k w$$

mit

$$\omega_k := \min\{1, \frac{1}{\alpha_k \beta L}\}, \quad \alpha_k := \|Df(x^{(k)})^{-1} f(x^{(k)})\|$$

erzeugt eine Folge $(x^{(k)})_{k \in \mathbb{N}}$, für die nach k_* Schritten

$$q_* := \alpha_{k_*} \beta L < \frac{1}{2}$$

erfüllt ist. Für $k > k_*$ konvergiert $x^{(k)}$ quadratisch und es gilt die a-priori-Fehlerabschätzung

$$\|x^{(k)} - z\| \leq \frac{\alpha}{1 - q_*} q_*^{2^k - 1}, \quad k \geq k_*.$$

Beweis Wir wollen monotone Konvergenz im Sinne von $\|f(x^{(k+1)})\| \leq \|f(x^{(k)})\|$ der gedämpften Iteration zeigen. Zu $x^{(0)}$ ist die Menge

$$D_0 := \{x \in D : \|f(x)\| \leq \|f(x^{(0)})\|\}$$

abgeschlossen und nichtleer. Weiter definieren wir die gedämpfte Iteration

$$x_\omega = g_\omega(x) := x - \omega Df(x)^{-1} f(x).$$

Für ein $x \in D_0$ sei $\omega_{\max} \leq 1$ der maximale Dämpfungsparameter, sodass gilt:

$$f(x_\omega) \leq f(x^{(0)}) \quad 0 \leq \omega \leq \omega_{\max} \leq 1$$

Wir definieren

$$h(\omega) := f(x_\omega)$$

mit $h(0) = f(x)$. Für $h(\cdot)$ gilt weiter

$$h'(\omega) = \frac{d}{d\omega} f(x - \omega Df(x)^{-1} f(x)) = -Df(x_\omega) Df(x)^{-1} f(x).$$

Mit $h'(0) = -f(x) = -h(0)$ folgt

$$\begin{aligned} f(x_\omega) - f(x) &= h(\omega) - h(0) = \int_0^\omega h'(s) ds = - \int_0^\omega Df(x_s) Df(x)^{-1} f(x) ds \\ &= - \int_0^\omega \left\{ (Df(x_s) - Df(x)) Df(x)^{-1} f(x) + f(x) \right\} ds, \end{aligned}$$

also

$$f(x_\omega) = \left(- \int_0^\omega (Df(x_s) - Df(x)) Df(x)^{-1} ds \right) f(x) + (1 - \omega) f(x)$$

und abgeschätzt

$$\begin{aligned}
\|f(x_\omega)\| &\leq (1 - \omega)\|f(x)\| + \beta L \int_0^\omega \|x_s - x\| ds \|f(x)\| \\
&= (1 - \omega)\|f(x)\| + \beta L \int_0^\omega s \|Df(x)^{-1} f(x)\| ds \|f(x)\| \\
&\leq \left(1 - \omega + \alpha_x \beta L \frac{\omega^2}{2}\right) \|f(x)\|, \quad \alpha_x = \|Df(x)^{-1} f(x)\|.
\end{aligned}$$

Wir bestimmen nun $0 \leq \omega \leq \omega_{\max} \leq 1$ so, dass der Ausdruck

$$\rho(\omega) := \left| 1 - \omega + \alpha_x \beta L \frac{\omega^2}{2} \right|$$

minimal wird. Das Minimum wird bei

$$\omega_{\min} = \min \left\{ 1, \frac{1}{\alpha_x \beta L} \right\}$$

angenommen. Dann gilt

$$1 - \omega_{\min} + \frac{\omega_{\min}^2}{2} \alpha_x \beta L \leq 1 - \frac{1}{2\alpha_x \beta L} < 1.$$

Bei dieser Wahl des Dämpfungsparameters liegt somit monotone Konvergenz vor:

$$\|f(x^{(k+1)})\| \leq \left(1 - \frac{1}{2\alpha_k \beta L}\right) \|f(x^{(k)})\|$$

Nach einer endlichen Anzahl von Schritten gilt

$$\frac{\alpha_k \beta L}{2} \leq \frac{\beta^2 L}{2} \|f(x^{(k)})\| < 1$$

und der Einzugsbereich der quadratischen Konvergenz ist erreicht. $\square$

Beispiel 8.35 (Gedämpftes Newton-Verfahren)

Wir führen Beispiel 8.33 fort. Wenn wir den Startvektor schlecht wählen, dann konvergiert gegebenenfalls auch das vollständige Newton-Verfahren manchmal schlecht oder nicht. Nehmen wir zum Beispiel den Startvektor $x = (0, -0.49999)$ (bei dem die Jacobi-Matrix fast singulär ist), dann erhalten wir keine Konvergenz in 35 Schritten:

```
1 newton_vek(f, Df, x=np.array([0, -0.49999]), n=35, tol=1e-10)
```

```
it  ||f(x)||
-----
00  9.9998e+04
```

```

01  5.6255e+09
02  1.4064e+09
...
19  3.0457e+00
20  6.8497e+03
21  1.5718e+04
...
32  3.1562e+00
33  1.8035e+00
34  2.2157e+00

```

Wir versuchen das Einzugsgebiet zu erweitern, in dem wir eine Dämpfung mit einbauen. Um die Dämpfung in unser Newton-Verfahren zu implementieren, müssen wir also der Funktion eine Liste an Dämpfungsparametern `omega` geben und die Korrektur Zeile folgendermaßen anpassen

```

12      x[:] += omega [i] * w

```

Wenden wir dies nun an, erhalten wir

```

1 newton_gedämpft_vek(f, Df, x=np.array([0, -0.49999]), omega=[0.88] *
↪ 50, n=50, tol=1e-10)

```

```

it  ||f(x)||
-----
00  9.9998e+04
01  4.3564e+09
02  1.3662e+09
...
19  3.7163e+00
20  1.0136e+00
21  6.6323e-01
22  5.5081e-01
23  1.1150e-02
...
29  3.6487e-08
30  4.3784e-09
31  5.2541e-10
32  6.3050e-11

```

Selbst nachdem wir im Einzugsbereich sind konvergiert das Verfahren nur noch linear. Dies ist klar, da stets ein Dämpfungsparameter < 1 verwendet wird. Ziel muss es sein, die Dämpfung automatisch zu beenden, wenn der Einzugsbereich quadratischer Konvergenz gefunden ist. ◀

Beispiel 8.36 (Dämpfung im vereinfachten Newton-Verfahren)

Auch im vereinfachten Newton-Verfahren können wir die Dämpfung mit einbauen. Mit dem Startvektor $x^{(0)} = (1, 1)^T$ und $\omega = 0.74$ braucht das gedämpfte vereinfachte Newton-Verfahren nur 27 Schritte. Im Vergleich dazu hat das ungedämpfte vereinfachte Newton Verfahren 670 Schritte gebraucht, siehe Beispiel 8.33. ◀

Bemerkung 8.37 (Globalisierung und Übergang zur quadratischen Konvergenz) Das gedämpfte Newton-Verfahren ist eine Globalisierungsstrategie. Durch die Wahl eines Dämpfungsparameters kann der Konvergenzbereich des Newton-Verfahrens vergrößert werden. Die Startwertbedingung

$$x^{(0)} \in D : \underbrace{\|Df(x^{(0)})^{-1} f(x^{(0)})\|}_{=\alpha_0} \beta L < \frac{1}{2}$$

muss nicht erfüllt werden. Ist dieses Produkt zu groß, so wird mit der gedämpften Iteration so lange iteriert, bis für das Produkt $\alpha_k \beta L$ mit $\alpha_k = \|Df(x^{(k)})^{-1} f(x^{(k)})\|$ die Bedingung erfüllt ist. Globalisierungsstrategien vergrößern den Konvergenzbereich. Es liegt jedoch zunächst nur langsame Konvergenz vor. Wir haben oben die Konvergenzrate

$$\rho \leq 1 - \frac{1}{2\alpha_k \beta L}$$

nachgewiesen. Für α_k groß kann ρ beliebig nahe an 1 liegen.

Ein wichtiges Merkmal von *effizienten* Globalisierungsstrategien ist das automatische Umschalten auf schnelle Newton-Konvergenz, wenn die Iteration im Einzugsbereich quadratischer Konvergenz liegt. Dies ist hier gegeben, da

$$\alpha_k \beta L < \frac{1}{2} \Rightarrow \omega_{min} = \min \left\{ 1, \frac{2}{\alpha_k \beta L} \right\} = 1. \quad \blacklozenge$$

Ein Problem bei der Anwendung des gedämpften Newton-Verfahrens ist die praktische Bestimmung des Dämpfungsparameters ω_k . Die Größen

$$\alpha_k = \|Df(x^{(k)})^{-1} f(x^{(k)})\|, \quad \beta = \max_{x \in D} \|Df(x)^{-1}\|,$$

sowie die Lipschitz-Konstante von $Df(x)$ sind im Allgemeinen nicht berechenbar. Eine übliche Strategie zur Suche eines guten Dämpfungsparameters ist der *Line-Search-Algorithmus* (die beste Lösung wird entlang einer Linie gesucht):

Algorithmus 8.10: Line-Search

Eingabe: Gegeben sei $f \in C^1(D)$, ein Startwert $x^{(0)} \in D$, eine Toleranz $\epsilon > 0$ sowie ein Parameter $\sigma \in (0, 1)$ und eine maximale Zahl an Line-Search-Schritten $L_{\max} \in \mathbb{N}$.

```

1  $k = 0$ 
2 Solange  $\|f(x^{(k)})\| \geq \epsilon$  tue
3    $Df(x_k)w_{(k)} = -f(x^{(k)})$ 
4   Setze  $\omega_0^k = 1$ 
5   Für  $l = 0$  bis  $L_{\max}$  tue
6      $x^{(k+1)} = x^{(k)} + \omega_l w^{(k)}$ 
7     Wenn  $\|f(x^{(k+1)})\| < \|f(x^{(k)})\|$  dann
8       Abbruch
9      $\omega_{l+1}^{(k)} = \sigma \omega_l^{(k)}$ 
10   $k = k + 1$ 

```

Beim Line-Search-Verfahren wird in jedem Schritt monotone Konvergenz erzwungen. Zunächst wird ein voller Newton-Schritt mit $\omega = 1$ versucht. Solange $\|f(x^{(k+1)})\| > \|f(x^{(k)})\|$, wird ω reduziert. Übliche Werte für σ sind $\sigma = \frac{1}{2}$ oder $\sigma = \frac{1}{4}$. Es gibt Fälle, in denen monotone Konvergenz nicht erreichbar ist. Dann ist es sinnvoll, nur einige wenige Iterationen von Line-Search durchzuführen. Nach z.B. $L = 4$ Schritten wird die neue Approximation akzeptiert, auch wenn sich das Residuum in diesem Newton-Schritt vergrößert. [Oft wird dann in den folgenden Schritten Konvergenz erreicht. Dieses heuristische Verfahren muss aber nicht immer zur Konvergenz führen.]

Beispiel 8.38 (Globalisiertes Newton-Verfahren im $\mathbb{R}^n$)

In Beispiel 8.33 haben wir gesehen, dass das vereinfachte Newton-Verfahren 670 Schritte benötigt hat, mit einem Startwert bei dem das Newton-Verfahren nach fünf Schritten konvergiert ist. In Beispiel 8.36 haben wir dann gesehen, dass das gedämpfte, vereinfachte Newton-Verfahren bei der korrekten Wahl des Dämpfungsparameters nur 27 Schritte benötigt hat. Es ist aber nicht immer klar, was eine gute Wahl für den Dämpfungsparameter ist, sodass wir diese Wahl automatisieren wollen. Zudem haben wir in Beispiel 8.35 gesehen, dass das gedämpfte Newton-Verfahren nur linear konvergiert, sodass wir das vollständige Newton-Verfahren verwenden wollen, wenn wir im Bereich der quadratischen Konvergenz sind.

Um die Konvergenz und den Konvergenzradius zu verbessern, aktualisieren wir die Jacobi-Matrix nur dann, wenn das Residuum nicht um mindestens den Faktor 0.3 kleiner geworden ist, und implementieren den Line-Search Algorithmus um die Wahl der Dämpfung zu automatisieren.

🔗 Implementierung 8.7: Globalisiertes Newton-Verfahren

```

1 def newton_global_vek(f, D, x, sigma=0.5, Lmax=10, n=10, tol=1e-10):
2   b = - f(x)
3   res0, res1 = np.linalg.norm(b), float('nan')

```

```

4     if res0 < tol:
5         return 0, x
6
7     for i in range(n):
8         if i == 0 or res0 / res1 > 0.3:
9             print(f'i = {i}: Jacobi Matrix wird neu aufgestellt')
10            jac = D(x)
11            pivot = LR_zerlegung_mit_pivot(jac)
12            for p in pivot:
13                b[p] = b[[p[1], p[0]]]
14            y = vorwaerts_einsetzen_ohne_diag(jac, b)
15            w = rueckwaerts_einsetzen(jac, y)
16            for l in range(Lmax):
17                x_neu = x.copy() + sigma**l * w
18                b = -f(x_neu)
19                res2 = np.linalg.norm(b)
20                if res2 < res0:
21                    if l > 0:
22                        print(f'i = {i}: Es waren {l} Line-search Schritte
23                              ↳ notwendig')
24                    x = x_neu
25                    break
26                else:
27                    print(f'Schritt {i}: {l} Line-search Schritte haben das
28                          ↳ Residuum nicht verbessert')
29                    x = x_neu
30
31            res1 = res0
32            res0 = res2
33            if res0 < tol:
34                return i, x
35            else:
36                print(f'Die globalisierte Newton Methode ist nach nicht
37                      ↳ konvergiert')
38            return i, x

```

Angewandt auf die Funktion aus Beispiel 8.33 ergibt dann

```

1 n, x = newton_global_vek(f, Df, x=np.array([1.0, 1.0]), n=20)
2 print(f'\nn = {n}, x = {x}')

```

i = 0: Jacobi Matrix wird neu aufgestellt

i = 1: Jacobi Matrix wird neu aufgestellt

n = 15, x = [0.89442719 0.4472136]

Wir haben also nur eine Aktualisierung der Jacobi-Matrix benötigt, damit das Verfahren nach 15 statt 670 Schritten die Nullstelle berechnet hat. Nehmen wir den Startwert $x^{(0)} = (-1, 1)^T$, bei dem das vereinfachte Newton-Verfahren divergiert, haben wir jetzt Konvergenz nach 14 Schritten, wobei die Jacobi-Matrix sechs mal neu aufgestellt wurde und vier Line-Search Schritte notwendig waren um das Residuum im jeweiligen Schritt zu reduzieren.

```
1 n, x = newton_global_vek(f, Df, x=np.array([-1.0, 1.0]), n=50)
2 print(f'\nn = {n}, x = {x}')
```

```
i = 0: Jacobi Matrix wird neu aufgestellt
i = 0: Es waren 3 Line-search Schritte notwendig
i = 1: Jacobi Matrix wird neu aufgestellt
i = 1: Es waren 3 Line-search Schritte notwendig
i = 2: Jacobi Matrix wird neu aufgestellt
i = 2: Es waren 2 Line-search Schritte notwendig
i = 3: Jacobi Matrix wird neu aufgestellt
i = 3: Es waren 1 Line-search Schritte notwendig
i = 4: Jacobi Matrix wird neu aufgestellt
i = 6: Jacobi Matrix wird neu aufgestellt
```

```
n = 14, x = [0.89442719 0.4472136 ]
```



Das Line-Search Verfahren ändert die Länge des Schrittes, der durch $w^{(k)} = -Df(x^{(k)})^{-1} f(x^{(k)})$ gegeben ist, nicht aber die *Richtung* $w^{(k)}$ selbst. Wir betrachten jetzt die skalare Funktion

$$f(x) = x + \sin(2x).$$

mit der einzigen Nullstelle $f(0) = 0$. Das Newton-Verfahren ist nur lokal anwendbar, z. B. gilt für den Startwert $x_0 = 2$ das $f'(x_0) \approx -0.31$ und der erste Newton-Schritt lautet

$$x_0 + \omega w^{(1)} = x_0 - \frac{f(x_0)}{f'(x_0)} \approx 2 + 4.05\omega.$$

Für $\omega \in (0, 1]$ ist sicher keine Konvergenz zu erwarten. Die Suchrichtung $w^{(1)}$ zeigt also in die falsche Richtung. Eine weitere Klasse von Globalisierungsverfahren, die auch die Suchrichtung ändern, sind die *Trust-Region Methoden*. Deren Grundlage ist, dass das Newton-Verfahren die Funktion $f(x)$ in jedem Schritt durch eine lineare Funktion approximiert. Für sehr kleine Schritte kann davon ausgegangen werden, dass diese Approximation gut ist und schließlich führt das Vorgehen auch zur quadratischen Konvergenz. Ist die Iteration (oder der Startwert) aber noch weit von der gesuchten Lösung entfernt, so können wir im allgemeinen nicht davon ausgehen, dass sich die Funktion global gut als eine lineare Funktion darstellen lässt. Bei Line-Search war das Vorgehen, die Schrittweite zu reduzieren. Die Trust-Region-

Methoden modifizieren stattdessen die Jacobi-Matrix der Funktion, so dass sie ähnlicher zu einer linearen Funktion ist, z. B. durch die einfache Addition der gewichteten Einheitsmatrix

$$D_\gamma f(x^k) := \tilde{D}f(x^k) := Df(x^k) + \gamma I.$$

Die Trust-Region-Methoden kommen aus dem Gebiet der Optimierung, wo das Newton-Verfahren eingesetzt wird um stationäre Punkte, also Nullstellen der Ableitung zu finden, für Details verweisen wir auf Ulbrich & Ulbrich [87].

Das Newton-Verfahren ist eines der wichtigsten Verfahren in der numerischen Mathematik. Wenn alle Bedingungen erfüllt sind und insbesondere wenn ein Startwert im Einzugsbereich quadratischer Konvergenz bekannt ist, dann ist die Nullstelle schnell gefunden. Oft ist natürlich gerade die Bestimmung eines guten Startwerts nicht einfach. Daher ist das Newton-Verfahren in verschiedenen Anwendungsfeldern, z. B. bei der Betrachtung von Minimierungsproblemen (Suche von stationären Punkten, also Nullstellen der ersten Ableitung) oder bei der Approximation von nichtlinearen Differentialgleichungen, ein aktives und großes Forschungsfeld. Für eine sehr ausführliche Darstellung verweisen wir auf das Buch von Deuffhard [24].

8.5 Exkurs: Nullstellensuche im Komplexen

Der *Fundamentalsatz der Algebra* besagt, dass ein Polynom vom Grad n genau n Nullstellen (ihrer Vielfachheit nach) besitzt, siehe [60] oder [34]. Diese Nullstellen können jedoch komplex sein. Das Polynom

$$p(x) = x^3 - 1$$

hat in $\mathbb{R}$ genau eine einfache reelle Nullstelle $x = 1$. Betrachten wir das Polynom als Funktion der komplexen Zahlen

$$p : \mathbb{C} \rightarrow \mathbb{C}, \quad p(z) = z^3 - 1,$$

so sind die drei Nullstellen gerade die *komplexen Einheitswurzeln*

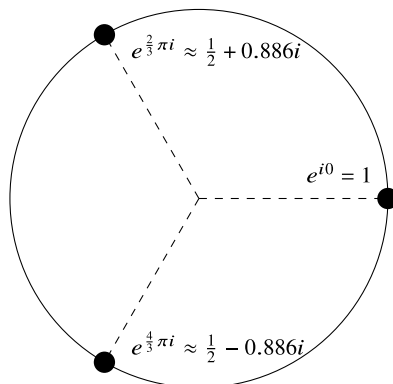
$$z_1 = e^{0i} = 1, \quad z_2 = e^{\frac{2}{3}\pi i} \approx -0.5 + 0.866i, \quad e^{\frac{4}{3}\pi i} \approx -0.5 - 0.866i.$$

In Abb. 8.6 zeigen wir diese drei Lösungen.

Wir wollen in diesem Exkurs der Frage nachgehen, inwieweit sich Methoden zur Nullstellensuche auf Funktionen in komplexen Zahlen übertragen lassen. Zu Beginn rekapitulieren wir die wichtigsten Fakten:

► **Definition 8.39 (Komplexe Zahlen)** Die Menge $\mathbb{C}$ der *komplexen Zahlen* ist ein kommutativer Körper $(\mathbb{C}, +, -)$, der den Körper der reellen Zahlen $\mathbb{R}$ als Teilkörper enthält. Es existiert ein Element $i \in \mathbb{C}$ mit der Eigenschaft

Abb. 8.6 Lösungen der Gleichung $z^3 - 1 = 0$ im Komplexen. Die drei Nullstellen $z_1, z_2, z_3 \in \mathbb{C}$ sind die *dritten Einheitswurzeln*. Die n -ten Einheitswurzeln können als $z_k^n = e^{2k\pi/ni}$ angegeben werden



$$i^2 = -1.$$

Dieses Element wird *imaginäre Einheit* genannt. Jede komplexe Zahl $z \in \mathbb{C}$ lässt sich mit zwei reellen Zahlen $a, b \in \mathbb{R}$ eindeutig schreiben als

$$z = a + ib,$$

dabei heißt $a = \operatorname{Re}(z)$ der *Realteil* und $b = \operatorname{Im}(z)$ der *Imaginärteil* von z . Das heißt wenn $b = 0$, arbeiten wir gerade auf der reellen Achse mit den bekannten reellen Zahlen.

Zu einer komplexen Zahl $z = a + ib$ heißt $\bar{z} = a - ib$ die *konjugiert komplexe Zahl*. Für zwei komplexe Zahlen $z_{1/2} = a_{1/2} + ib_{1/2}$ sind Addition und Multiplikation gegeben als

$$z_1 + z_2 = (a_1 + a_2) + i(b_1 + b_2),$$

$$z_1 \cdot z_2 = (a_1 \cdot a_2 - b_1 \cdot b_2) + i(a_1 b_2 + a_2 b_1).$$

Für die Division gilt

$$\frac{z_1}{z_2} = \frac{a_1 a_2 + b_1 b_2}{|z_2|^2} + i \frac{b_1 a_2 - a_1 b_2}{|z_2|^2}.$$

Der *Betrag einer komplexen Zahl* $z = a + ib$ ist definiert als

$$|z| = \sqrt{z\bar{z}} = \sqrt{a^2 + b^2}.$$

Bei Beachtung von $i^2 = -1$ übertragen sich die Rechenregeln von $\mathbb{R}$ auf $\mathbb{C}$. Der Körper der komplexen Zahlen $\mathbb{C}$ ist isomorph zum Vektorraum $\mathbb{R}^2$. Komplexe Zahlen werden daher als Punkte in der *komplexen Zahlenebene* dargestellt, siehe Abb. 8.6. Weiter gilt die *Euler-Formel* für $b \in \mathbb{R}$

$$e^{ib} = \cos(b) + i\sin(b) \Rightarrow |e^z| = |e^{a+ib}| = e^a.$$

► **Definition 8.40 (Komplexe Funktion)** Eine Abbildung $f : D \subset \mathbb{C} \rightarrow \mathbb{C}$ heißt *komplexe Funktion*. Mittels $z = x + iy$ und mit $f = u + iv$ mit zwei Funktionen $u, v : D \rightarrow \mathbb{R}$ lässt sich jede komplexe Funktion als Funktion $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ interpretieren:

$$f(z) = u(z) + iv(z), \quad f(x, y) = u(x, y) + iv(x, y).$$

Eine *Nullstelle* $z \in \mathbb{C}$ ist eine komplexe Zahl, in der die Funktion f den Wert null annimmt, d. h.

$$f(z) = u(z) + iv(z) = 0.$$

Es gilt also $\operatorname{Re}(f(z)) = \operatorname{Im}(f(z)) = 0$.

Die Analogie zu Funktionen in $\mathbb{R}^2$ erlaubt eine erste Einordnung des komplexen Nullstellenproblems. Es handelt sich um eine zweidimensionale Aufgabe. Konzepte wie die Intervallschachtelung können nicht funktionieren. Das Newton-Verfahren lässt sich jedoch direkt übertragen:

Satz 8.41 (Reelles Newton-Verfahren für komplexe Funktionen) Es sei $f : \mathbb{C} \rightarrow \mathbb{C}$ eine komplexe Funktion, die - aufgefasst als reellwertige Funktion $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ - die Voraussetzungen von Satz 8.29 (Newton-Kantorovich) erfüllt. Dann ist das Newton-Verfahren entsprechend anwendbar.

Beispiel 8.42

Wir betrachten die Funktion

$$p(z) = z^3 - 1.$$

Mit $z = a + ib$ gilt

$$p(x, y) = p(a + ib) = (a + ib)^3 - 1 = (a^3 - 3ab^2 - 1) + i(3a^2b - b^3)$$

und wir untersuchen im Folgenden in $\mathbb{R}^2$ die Funktion

$$f(x, y) = \begin{pmatrix} x^3 - 3xy^2 - 1 \\ 3x^2y - y^3 \end{pmatrix}.$$

Wir suchen x, y , sodass $f(x, y) = 0$ gilt. Diese Funktion ist in $C^\infty(\mathbb{R}^2)$, das Newton-Verfahren ist also anwendbar. Für die Jacobi-Matrix gilt

$$Df(x, y) = \begin{pmatrix} 3x^2 - 3y^2 & -6xy \\ 6xy & 3x^2 - 3y^2 \end{pmatrix}.$$

Wegen

$$\det(Df(x, y)) = 9(x^2 + y^2)^2$$

ist die Jacobi-Matrix – weg vom Nullpunkt $(0, 0)$ – überall regulär. Wir führen das zweidimensionale Newton-Verfahren für verschiedene Startwerte (x_0, y_0) durch, bis die ersten drei Stellen exakt bestimmt sind.

k			(x_k, y_k)	
1	(1.000,0.000)	(0.000,1.000)	(-1.000,0.000)	(0.000,-1.000)
2		(-0.333,0.667)	(-0.333,0.000)	(-0.333,-0.667)
3		(-0.582,0.924)	(2.778,0.000)	(-0.582,-0.924)
4		(-0.508,0.868)	(1.895,0.000)	(-0.508,-0.868)
4		(-0.500,0.866)	(1.356,0.000)	(-0.500,-0.866)
4			(1.085,0.000)	
4			(1.007,0.000)	
4			(1.000,0.000)	

Das Verfahren konvergiert wie erwartet und wir finden die drei komplexen Einheitswurzeln aus Abb. 8.6 wieder. Hier haben wir das Problem vollständig als Problem in reellen Zahlen aufgefasst. So können wir auf Bekanntes zurückgreifen, können jedoch nicht eine – vielleicht effizientere – komplexe Arithmetik nutzen. ◀

Im Folgenden werden wir uns ausschließlich mit den Nullstellen (reell und komplex) von reellen Polynomen beschäftigen. Wir fassen einige Aussagen zusammen:

Satz 8.43 (Nullstellen von Polynomen mit reellen Koeffizienten) Das Polynom

$$p(z) = z^n + \alpha_{n-1}z^{n-1} + \cdots + \alpha_0, \quad \alpha_i \in \mathbb{R}, \quad (8.28)$$

hat genau n (möglicherweise komplexe) Nullstellen $z_1, \dots, z_n \in \mathbb{C}$ (mehrfache Nullstellen sind möglich) und erlaubt die Linearfaktorzerlegung

$$p(z) = \prod_{k=1}^n (z - z_k).$$

Ist z_i eine Nullstelle mit imaginärem Teil $\operatorname{Im}(z_i) \neq 0$, so ist durch $\bar{z}_i$ eine weitere Nullstelle gegeben. Alle Nullstellen liegen im Kreis

$$z_i \in K_R(0) := \left\{ z \in \mathbb{C}, |z| < R := \max \left\{ 1, \sum_{k=0}^{n-1} |\alpha_k| \right\} \right\}.$$

Beweis Die Beweise für die einzelnen Aussagen finden sich in [34] oder ähnlichen Büchern zur Funktionalanalysis. ◻

8.5.1 Das Verfahren von Bairstow

Dieser Satz beinhaltet für die numerische Nullstellensuche einige wichtige Aussagen: Zunächst liegen alle Nullstellen in einem Kreis um den Nullpunkt. Dies liefert uns einen ersten Anhaltspunkt für das Gebiet, in dem wir Nullstellen suchen müssen. Weiter treten komplexe Nullstellen in Paaren auf:

$$p(a + ib) = 0 \Rightarrow p(a - ib) = 0$$

Angenommen, durch $z_1, \dots, z_{2k} \in \mathbb{C} \setminus \mathbb{R}$ mit $2k \leq n$ seien die komplexen Nullstellen gegeben. Für diese gilt $z_{2i} = \bar{z}_{2i-1}$ mit $z_{2i} = a_{2i} + ib_{2i}$ und $z_{2i-1} = a_{2i} - ib_{2i}$. Durch $z_{2k+1}, \dots, z_n \in \mathbb{R}$ seien die weiteren $n - 2k$ reellen Nullstellen von $p(\cdot)$ gegeben. Somit folgt die Faktorisierung

$$\begin{aligned} p(z) &= \left(\prod_{i=1}^k (z - z_{2i})(z - \bar{z}_{2i}) \right) \prod_{i=2k+1}^n (z - z_i) \\ &= \left(\prod_{i=1}^k ((z - a_{2i})^2 + b_{2i}^2) \right) \prod_{i=2k+1}^n (z - z_i). \end{aligned}$$

Das Polynom lässt sich also in Linearfaktoren mit reellen Nullstellen und in quadratische Faktoren mit rein komplexen Nullstellen teilen.

Auf dieser Schreibweise baut das *Verfahren von Bairstow* auf [35]. Anstelle einer direkten Suche nach allen Nullstellen wird zunächst nach quadratischen Faktoren der Art

$$a(x) := (x - z_i)(x - \bar{z}_i) = (x - a_i)^2 + b_i^2 = x^2 - 2xa_i + (a_i^2 + b_i^2)$$

gesucht. Ist ein solcher Faktor gefunden, so können die beiden Nullstellen als

$$z_{i,1/2} = a_i \pm ib_i$$

einfach bestimmt werden. Im Anschluss wird das Polynom $p \in P_n$ durch den gefundenen Faktor $a \in P_2$ geteilt und man fährt mit einem Polynom aus P_{n-2} fort.

Gesucht ist nun ein quadratisches Polynom

$$a(x) = x^2 - px + q,$$

mit $q > 0$, so dass die Polynomdivision

$$p(x) = a(x)b(x) + r(x) \tag{8.29}$$

ohne Rest, also mit $r \equiv 0$ aufgeht. Die Koeffizienten p, q von $a(x)$ werden dabei iterativ gesucht. Mit $p \in P_n$ und $a \in P_2$ folgt $b \in P_{n-2}$ und $r \in P_1$, d.h., der Rest ist null, ein konstantes oder ein lineares Polynom. Wir machen den allgemeinen Ansatz

$$b(x) = \sum_{k=-2}^n b_k x^k, \quad b_n = b_{n-1} = b_{-2} = b_{-1} = 0 \quad (8.30)$$

und berechnen das allgemeine Produkt

$$a(x)b(x) = \sum_{k=0}^n (b_{k-2} - pb_{k-1} + b_k q)x^k.$$

Ein Koeffizientenvergleich von (8.29) ergibt

$$a_k = b_{k-2} - pb_{k-1} + b_k q, \quad k = n, n-1, \dots, 0. \quad (8.31)$$

Dies sind $n+1$ Gleichungen für nur $n-1$ Koeffizienten $b_0, \dots, b_{n-2}$. Denn b_n und b_{n-1} wurden in (8.30) nur eingeführt, um die Schreibweise zu vereinfachen. Angenommen, wir bestimmen die Koeffizienten $b_0, \dots, b_{n-2}$ aus den Gl. (8.31), also

$$b_{k-2} = a_k + pb_{k-1} - qb_k, \quad n = 2, \dots, n.$$

Dann ergibt sich für den Rest die Darstellung

$$r(x) = p(x) - a(x)b(x) = (a_1 + pb_0 - qb_1)x + (a_0 - qb_0).$$

Das Ziel ist es, $p, q \in \mathbb{R}$ so zu bestimmen, dass der Rest verschwindet. Die Koeffizienten b_0 und b_1 hängen von p und q ab und können einfach berechnet werden:

Algorithmus 8.11: Berechnung der Bairstow-Koeffizienten

Eingabe: Gegeben sei $p(x) = \sum_{k=0}^n a_k x^k$.

- 1 Wähle $p, q \in \mathbb{R}$
- 2 Setze $b_0 = 1, b_1 = 0$
- 3 **Für** $k = n-1$ **bis** 2 **tue**
- 4 $b_2 = b_1$
- 5 $b_1 = b_0$
- 6 $b_0 = a_k + pb_1 - qb_2$

Die Koeffizienten b_0 und b_1 hängen implizit von den Werten $p, q \in \mathbb{R}$ ab. Um diese optimal zu bestimmen, wenden wir das Newton-Verfahren auf die Funktion

$$F(p, q) = \begin{pmatrix} a_1 + pb_0(p, q) - qb_1(p, q) \\ a_0 - qb_0(p, q) \end{pmatrix} = 0$$

an. Zu gegebener Näherung p_l, q_l kann das Residuum, also $F(p_l, q_l)$, mit Algorithmus 8.11 berechnet werden. Die Jacobi-Matrix ist formal gegeben als

$$DF(p, q) = \begin{pmatrix} b_0 + p\partial_p b_0 - q\partial_p b_1 & p\partial_q b_0 - b_1 - q\partial_q b_1 \\ -q\partial_p b_0 & -b_0 - q\partial_q b_0 \end{pmatrix},$$

wobei $b_0(\cdot)$ und $b_1(\cdot)$, sowie deren Ableitungen in Richtung p und q natürlich von p und q abhängen. Zur Berechnung der Ableitung könnte Algorithmus 8.11 formal nach p und q abgeleitet werden. Durch geschickte Transformation kann die Berechnung der Ableitungen jedoch weit effizienter gestaltet werden. Wir verweisen hier auf die Literatur [35] und betrachten ein einfaches Beispiel.

Beispiel 8.44 (Bairstow-Verfahren zur Bestimmung der dritten Einheitswurzeln)

Wir betrachten das Polynom

$$p(x) = x^3 - 1, \quad a_3 = 1, \quad a_2 = 0, \quad a_1 = 0, \quad a_0 = 1.$$

Mithilfe des Bairstow-Verfahrens suchen wir einen quadratischen Faktor dieses Polynoms. Division von $p(x) = x^3 - 1$ durch $a(x) = x^2 - px + q$ ergibt den Rest

$$p(x) = (x^2 - px + q)(x + p) - ((q - p^2)x + qp + 1), \quad (8.32)$$

also

$$r(x) = (q - p^2)x + qp + 1.$$

Wir suchen $p, q \in \mathbb{R}$, sodass

$$F(p, q) = \begin{pmatrix} q - p^2 \\ qp + 1 \end{pmatrix} \stackrel{!}{=} 0.$$

Die Jacobi-Matrix kann in diesem einfachen Fall sofort bestimmt werden als

$$DF(p, q) = \begin{pmatrix} -2p & 1 \\ q & p \end{pmatrix}, \quad DF(p, q)^{-1} = \frac{1}{2p^2 + q} \begin{pmatrix} -p & 1 \\ q & 2p \end{pmatrix}.$$

Schritt 1: Wir starten das Newton-Verfahren mit

$$p_0 = 0, \quad q_0 = 1 \quad \Rightarrow \quad a_0(x) = x^2 + 1, \quad r_0(x) = x + 1.$$

Dies ergibt das Residuum

$$F(p_0, q_0) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Und für das Update folgt

$$\begin{pmatrix} p_1 - p_0 \\ q_1 - q_0 \end{pmatrix} = -DF(p_0, q_0)^{-1} F(p_0, q_0) = -\frac{1}{1} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \end{pmatrix},$$

also $p_1 = -1$ und $q_1 = 0$, und

$$a_1(x) = x^2 + x, \quad r_1(x) = x + 1.$$

Schritt 2: Weiter ist

$$F(p_1, q_1) = \begin{pmatrix} -1 \\ 1 \end{pmatrix},$$

also

$$\begin{pmatrix} p_2 - p_1 \\ q_2 - q_1 \end{pmatrix} = -DF(p_1, q_1)^{-1} F(p_1, q_1) = -\frac{1}{2} \begin{pmatrix} 1 & 1 \\ 0 & -2 \end{pmatrix} \begin{pmatrix} -1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \end{pmatrix},$$

also $p_2 = -1$ und $q_2 = 1$, und somit

$$a_2(x) = x^2 + x + 1, \quad r_2(x) \equiv 0.$$

Der gesuchte Faktor ist gefunden und die beiden ersten Nullstellen ergeben sich sofort als

$$z_{1/2} = \frac{p}{2} \pm \sqrt{\frac{p^2}{4} - q} = -\frac{1}{2} \pm \frac{\sqrt{3}}{2}i.$$

Der verbleibende lineare Faktor ist laut (8.32)

$$b(x) := x + p = x - 1,$$

d. h., die dritte (reelle) Nullstelle ist

$$z_3 = 1.$$



Der große Vorteil des Bairstow-Verfahrens liegt darin, dass es vollständig in reeller Arithmetik durchführbar ist. Da es auf dem Newton-Verfahren zur Bestimmung der Koeffizienten $p, q \in \mathbb{R}$ aufbaut, ist es äußerst schnell und effizient.

8.5.2 Das komplexe Newton-Verfahren

Abschließend diskutieren wir die Durchführung des Newton-Verfahrens in komplexer Arithmetik. Zunächst führen wir den Begriff der *komplexen Ableitung* ein:

► **Definition 8.45 (Komplexe Differenzierbarkeit)** Eine Funktion $f : D \subset \mathbb{C} \rightarrow \mathbb{C}$ heißt *komplex differenzierbar* in $z_0 \in \mathbb{C}$, wenn der Grenzwert

$$f'(z_0) := \lim_{h \rightarrow 0} \frac{f(z_0 + h) - f(z_0)}{h}$$

mit beliebigen $h \in \mathbb{C}$ existiert. Die Funktion heißt *holomorph* in z_0 , falls eine Umgebung von z_0 existiert, in der f komplex differenzierbar ist. Falls f auf ganz D komplex differenzierbar

ist, so heißt f *holomorphe Funktion*. Ist f auf $\mathbb{C}$ komplex differenzierbar, so nennt man f eine *ganze Funktion*.

Eine komplexe Funktion $f = u + iv$ ist genau dann komplex differenzierbar, wenn ihre reelle Entsprechung

$$f = u + iv, \quad f(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}$$

im reellen Sinne total differenzierbar ist und wenn die *Cauchy-Riemann-Differentialgleichungen* gelten:

$$\frac{d}{dx}u(x, y) = \frac{d}{dy}v(x, y), \quad \frac{d}{dy}u(x, y) = -\frac{d}{dx}v(x, y). \quad (8.33)$$

Die Rechenregeln zur Bestimmung von Ableitungen entsprechen dann den bekannten Regeln von reellen Ableitungen. Das Polynom $p(z)$ ist beispielsweise überall komplex differenzierbar und es gilt

$$p'(z) = nz^{n-1} + a_{n-1}(n-1)z^{n-2} + \cdots + 2a_2z + a_1.$$

Die komplexe Konjugation $f(z) = \bar{z}$ ist nirgends komplex differenzierbar, denn es gilt mit $h \in \mathbb{R}$

$$\lim_{h \rightarrow 0} \frac{\overline{z+h} - \bar{z}}{h} = \lim_{h \rightarrow 0} \frac{h}{h} = 1, \quad \lim_{h \rightarrow 0} \frac{\overline{z+ih} - \bar{z}}{ih} = \lim_{h \rightarrow 0} \frac{-ih}{ih} = -1.$$

Dagegen sind die komplexe Exponentialfunktion $\exp(z)$ und die komplexen trigonometrischen Funktionen $\sin(z)$ und $\cos(z)$ überall komplex differenzierbar, also *ganze Funktionen*. In der Funktionentheorie gilt der erstaunliche Zusammenhang:

Satz 8.46 (Holomorphe Funktionen) Es sei $f : D \subset \mathbb{C} \rightarrow \mathbb{C}$ eine *holomorphe Funktion*. Dann ist f auf D beliebig oft komplex stetig differenzierbar und lässt sich in jedem Punkt in eine Potenzreihe entwickeln.

Für einen Beweis verweisen wir auf [34]. Holomorphe Funktionen lassen sich lokal wie bekannt als Taylor-Reihe darstellen.

Mit diesen Vorbereitungen können wir das Newton-Verfahren auf dem Körper der komplexen Zahlen definieren:

Satz 8.47 (Komplexes Newton-Verfahren) Es sei $f : D \subset \mathbb{C} \rightarrow \mathbb{C}$ eine holomorphe Funktion mit einer einfachen Nullstelle $\hat{z} \in D$. Dann existiert ein $\rho > 0$ und eine Umgebung $K_\rho(\hat{z})$, sodass das komplexe Newton-Verfahren für alle Startwerte $z_0 \in K_\rho(\hat{z})$ quadratisch gegen die Nullstelle konvergiert:

$$z_{k+1} = z_k - \frac{f(z_k)}{f'(z_k)}, \quad k = 0, 1, 2, \dots$$

Beweis Die Funktion f ist beliebig oft komplex stetig differenzierbar und es gilt $f'(\hat{z}) \neq 0$. Daher existiert eine Umgebung $K_{\rho'}(\hat{z})$, auf der $|f'(\hat{z})| \geq m > 0$ nach unten beschränkt ist. Wir zeigen im Folgenden, dass das Newton-Verfahren mit komplexer Arithmetik äquivalent zur Anwendung des vektorwertigen Newton-Verfahrens auf die Funktion $f(z) = f(x, y)$ ist.

Für die holomorphe Funktion $f = u + iv$ gilt mit den Cauchy-Riemann-Differentialgleichungen

$$f'(z) = u_x(z) + iv_x(z) = u_x(z) - iu_y(z).$$

Hiermit lässt sich das Newton-Verfahren schreiben als

$$\begin{aligned} z_{k+1} &= z_k - \frac{u(z_k) + iv(z_k)}{u_x(z_k) - iu_y(z_k)} \\ &= z_k - \frac{(u(z_k) + iv(z_k))(u_x(z_k) + iu_y(z_k))}{u_x(z_k)^2 + u_y(z_k)^2}, \\ &= z_k - \frac{u_x(z_k)u(z_k) - u_y(z_k)v(z_k) + i(u_x(z_k)v(z_k) + u_y(z_k)u(z_k))}{u_x(z_k)^2 + u_y(z_k)^2}, \end{aligned}$$

also gilt bei $z_k = a_k + ib_k$

$$\begin{aligned} a_{k+1} &= a_k - \frac{u_x(z_k)u(z_k) - u_y(z_k)v(z_k)}{u_x(z_k)^2 + u_y(z_k)^2}, \\ b_{k+1} &= b_k - \frac{u_x(z_k)v(z_k) + u_y(z_k)u(z_k)}{u_x(z_k)^2 + u_y(z_k)^2}. \end{aligned} \tag{8.34}$$

Nun betrachten wir entsprechend das reelle Newton-Verfahren, angewendet auf die Funktion

$$f(x, y) = \begin{pmatrix} u(x, y) \\ v(x, y) \end{pmatrix}.$$

Es ergibt sich

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \begin{pmatrix} u_x(x_k, y_k) & u_y(x_k, y_k) \\ v_x(x_k, y_k) & v_y(x_k, y_k) \end{pmatrix}^{-1} \begin{pmatrix} u(x_k, y_k) \\ v(x_k, y_k) \end{pmatrix}.$$

Da die Funktion f holomorph ist, gelten die Cauchy-Riemann-Differentialgleichungen, also

$$\begin{aligned} \begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} &= \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \begin{pmatrix} u_x(x_k, y_k) & u_y(x_k, y_k) \\ -u_y(x_k, y_k) & u_x(x_k, y_k) \end{pmatrix}^{-1} f(x_k, y_k) \\ &= \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \frac{1}{u_x(x_k, y_k)^2 + u_y(x_k, y_k)^2} \begin{pmatrix} u_x(x_k, y_k) & -u_y(x_k, y_k) \\ u_y(x_k, y_k) & u_x(x_k, y_k) \end{pmatrix} f(x_k, y_k). \end{aligned}$$

Ein Abgleich mit (8.34) zeigt, dass dies die identische Iteration ist.

Somit sind bei hinreichend kleinem Einzugsbereich $K_\rho(\hat{z})$ alle Voraussetzungen von Satz 8.29 erfüllt. $\square$

Beispiel 8.48 (Komplexes Newton-Verfahren zur Bestimmung der Einheitswurzeln)

Wir betrachten wieder

$$p(z) = z^3 - 1.$$

Die Newton-Vorschrift ergibt die Iteration

$$z_{k+1} = z_k - \frac{p(z_k)}{p'(z_k)} = \frac{2}{3}z_k + \frac{1}{3z_k^2}.$$

Mit der Schreibweise $x_k = a_k + ib_k$ führt dies auf

$$x_{k+1} + ib_{k+1} = \frac{2x_k^5 + 4x_k^3y_k^2 + x_k^2 + 2y_k^4x_k - y_k^2}{3(x_k^2 + y_k^2)^2} + i \frac{2x_k^4y_k + 4y_k^3x_k^2 - 2x_ky_k + y_k^5}{3(x_k^2 + y_k^2)^2}.$$

Es lässt sich einfach nachrechnen, dass dies exakt die Iterationsformel aus Beispiel 8.42 ist. Wir verzichten daher darauf, das Verfahren erneut durchzuführen. Stattdessen beschäftigen wir uns noch kurz mit der Frage nach dem Einzugsbereich der Konvergenz. Anstelle einer Analyse bestimmen wir den Einzugsbereich der drei Nullstellen numerisch und führen das Verfahren für verschiedene Werte, jeweils aus $x_0, y_0 \in [-8, 8]$ durch. In Abb. 8.7 zeigen wir das Konvergenzverhalten für alle Startwerte

$$z_0 = hk + ihl, \quad k, l = -250, \dots, 250, \quad h = \frac{8}{250}.$$

Dabei bedeutet ein roter Punkt, dass für den entsprechenden Startwert die Wurzel $z_1 = 1$ erreicht wird, ein blauer Startwert führt auf Konvergenz zu $z_2 = e^{\frac{2}{3}\pi i}$ und ein grüner Startwert auf Konvergenz zu $z_3 = e^{\frac{4}{3}\pi i}$. Graue Werte bedeuten, dass das Verfahren divergiert. Genauer gesagt wird die Iteration abgebrochen, sobald $|z_k| > 10$ gilt. Dann gibt die Intensität des Grauwerts den Index k an, bei dem die Iteration abgebrochen wird. Dunkle Werte (nahe bei 0 oder außen) stehen für sofortigen Abbruch. In weißen Bereichen konnte innerhalb der erlaubten 255 Iterationen noch nicht entschieden werden, ob das Newton-Verfahren konvergiert oder nicht. Die Menge der komplexen Zahlen an denen sich das Konvergenzverhalten schlagartig ändert, wird *Julia-Menge* oder auch *Newton-Fraktal* genannt. $\blacktriangleleft$

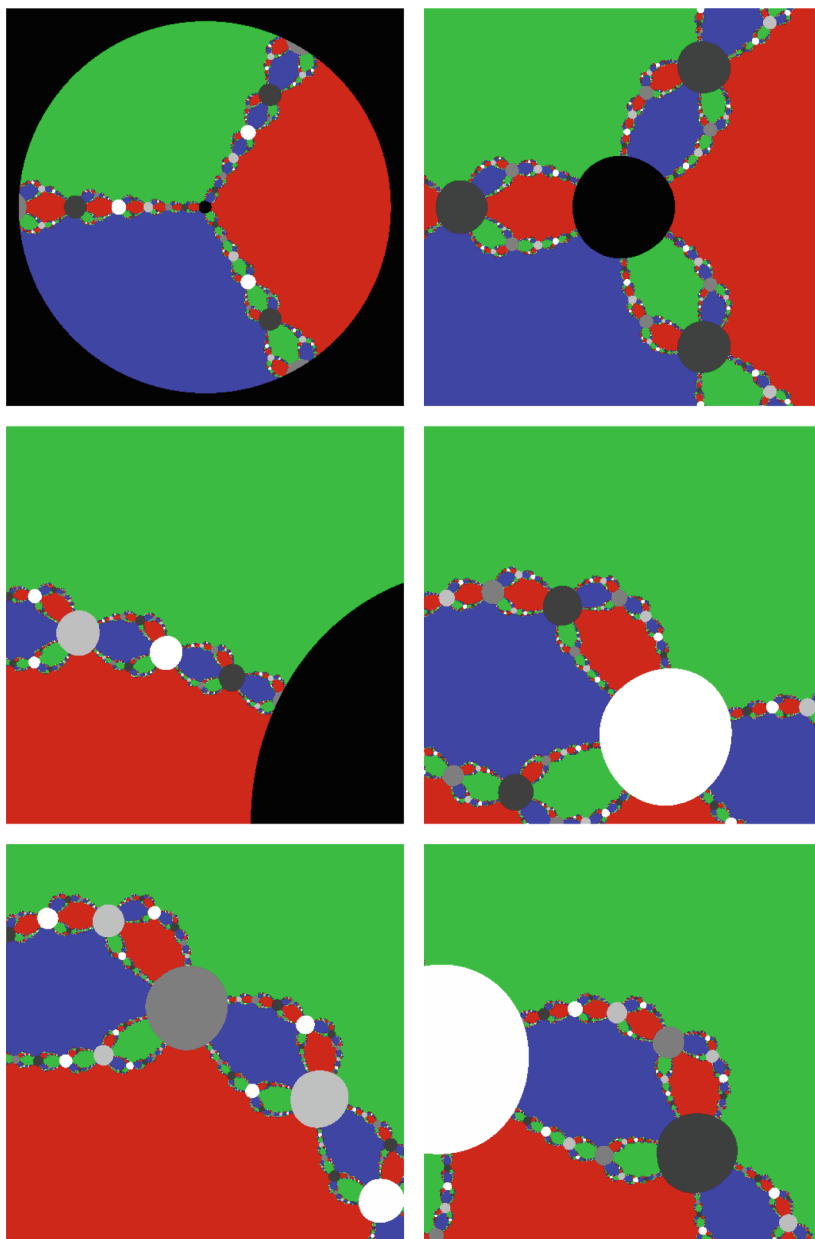


Abb. 8.7 Der Einzugsbereich des komplexen Newton-Verfahrens bildet eine *Julia-Menge*. Von links oben nach rechts unten werden die Startwerte aus jeweils kleineren Intervallen gewählt. Links oben gilt $z_0 = a_0 + ib_0$ mit $a_0, b_0 \in [-8, 8]$. Die Teilintervalle sind jeweils Ausschnitte der vorangehenden Darstellung

8.6 Exkurs: Fast Inverse Square Root

Im Jahr 1999 veröffentlichten Computer-Spiel QUAKE III ARENA¹ wurde Algorithmus 8.12 zur Approximation der Quadratwurzel, genauer gesagt zur Approximation der Inversen der Quadratwurzel

$$y = \frac{1}{\sqrt{x}}$$

verwendet. Die Urheberschaft des Verfahrens ist nicht ganz klar², Literatur findet sich zur Genüge [65].

Algorithmus 8.12: Fast Inverse Square Root

```

1 float Q_rsqrt( float number )
2 {
3     int i;
4     float x2, y;
5     const float threehalfs = 1.5F;
6
7     x2 = number * 0.5F;
8     y = number;
9     i = * ( int * ) &y;
10    i = 0x5f3759df - ( i >> 1 );
11    y = * ( float * ) &i;
12    y = y * ( threehalfs - ( x2 * y * y ) );
13    // y = y * ( threehalfs - ( x2 * y * y ) );
14
15    return y;
16 }
```

Ein erster numerischer Test, den wir in Abb. 8.8 vorstellen, zeigt die hohe relative Genauigkeit des Verfahrens mit einem relativen Fehler von maximal 0.18 % für beliebige Eingabewerte $x \in \mathbb{R}_+$. Das Verfahren muss im historischen Kontext bewertet werden. Im Jahr 1999 war der PENTIUM II ein gebräuchlicher Prozessor. Dieser Prozessor verfügte zwar bereits über eine Fließkommaeinheit, hatte jedoch einige Einschränkungen (siehe hierzu [33]):

- Berechnungen mit ganzen Zahlen waren schneller als die Fließkomma-Arithmetik.
- Additionen und Multiplikationen waren bei diesem Prozessor weit schneller als Divisionen. Anstelle einer Division konnten etwa 20 Multiplikationen oder 40 Additionen durchgeführt werden.
- Die Berechnung einer Wurzel wurde in der Fließkommaeinheit durchgeführt, war jedoch wenig effizient und hatte den Aufwand von etwa 35 Multiplikationen bzw. 70 Additionen.

¹ Ein 1999 von id Software entwickeltes Computerspiel.

² Siehe hierzu die Diskussion auf <https://www.beyond3d.com/content/articles/8/>.

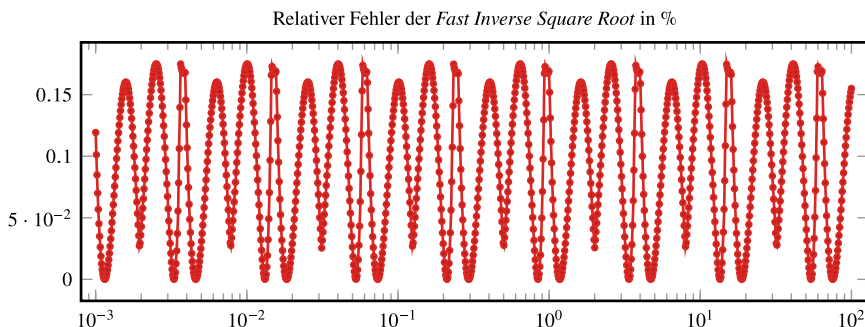


Abb. 8.8 Relativer Fehler der Funktion *Fast Inverse Square Root* aus Algorithmus 8.12 für $x \in [10^{-3}, 10^2]$. Es zeigt sich, dass die Inverse der Wurzel für alle Zahlen mit einem relativen Fehler kleiner als 0.2 % berechnet wird

Die Berechnung der Inversen einer Wurzel hatte somit einen ähnlichen Aufwand wie 100 Additionen bzw. wie 50 Multiplikationen.

Erst der im Jahr 1999 frisch eingeführte PENTIUM III Prozessor hatte mit der SSE-Erweiterung eine effiziente Funktion zur Approximation der Wurzel und deren Inversen und konnte diese (wie auch die Division) in der gleichen Zeit durchführen, die eine Multiplikation in Anspruch nimmt.

8.6.1 Shading – Schnelle Berechnung von Normalvektoren

QUAKE III war eines der ersten Spiele, die schnelle, realistisch wirkende 3D-Grafik sehr populär gemacht haben. Um bei einer zweidimensionalen Projektion auf den Computerbildschirm einen räumlichen Eindruck zu erzeugen, sind insbesondere Beleuchtungseffekte wichtig. In Abhängigkeit von der Position des Betrachters und der Position der Lichtquelle erscheinen Objekte unterschiedlich hell. Die Situation ist in Abb. 8.9 dargestellt. Wir wollen die Helligkeit in Punkt x bestimmen. Dabei ist der Betrachter (die Kamera) in Punkt C und die Lichtquelle in L . Die Helligkeit ergibt sich durch die einfache Regel *Einfallswinkel gleich Ausfallswinkel*, d. h., es kommt auf den Winkel zwischen der Verbindungsgerade vom Betrachter zum Punkt x , also $\mathbf{c} := \overline{Cx}$ und dem ausfallenden Lichtstrahl $\mathbf{l}_\perp$ an. Für diesen gilt die Projektion

$$\mathbf{l}_\perp = \mathbf{l} - 2\langle \mathbf{n}, \mathbf{l} \rangle \mathbf{n},$$

wobei $\mathbf{l} := \overline{Lx}$ die Verbindungsgerade zwischen Lichtquelle und Punkt x sowie $\mathbf{n}$ der Normalvektor in x an der Ebene ist. Der Winkel $\alpha = \angle(\mathbf{c}, \mathbf{l}_\perp)$ lässt sich nun mittels

$$\cos(\alpha) = \frac{\langle \mathbf{l}_\perp, \mathbf{c} \rangle}{|\mathbf{l}_\perp| |\mathbf{c}|}$$

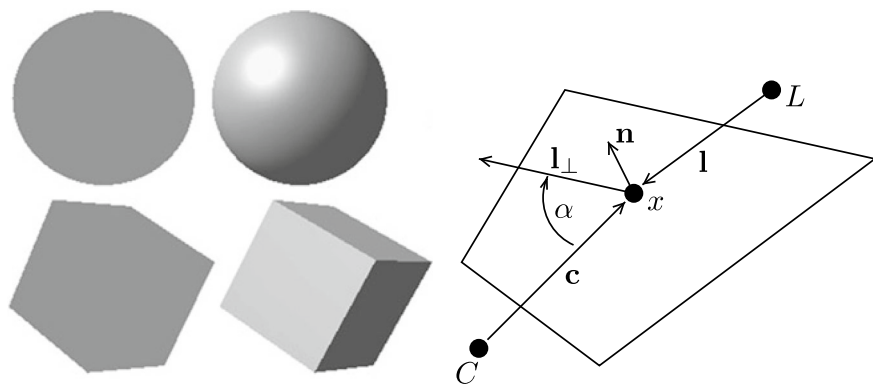


Abb. 8.9 Links: 3D-Wirkung durch Lichteffekte. Rechts: Berechnung der Helligkeit einer Fläche mit Normalvektor $\mathbf{v}$. Der Betrachter steht dabei im Punkt C , die Lichtquelle ist im Punkt L

bestimmen. Zur Berechnung der Skalarprodukte und Projektionen sind einige Multiplikationen und Additionen notwendig. Diese können alle schnell und effizient durchgeführt werden. Darüber hinaus muss das Ergebnis jedoch mit der Länge der Vektoren skaliert werden, also

$$\frac{1}{|\mathbf{c}|} = \frac{1}{\sqrt{c_1^2 + c_2^2 + c_3^2}}.$$

In diesem Schritt ist die Berechnung der Wurzel bzw. deren Inverser notwendig. Bei der Anwendung für Computerspiele ist hier natürlich eine Approximation ausreichend. In komplexen Spielen wie *QUAKE III* sind diese Berechnungen derart häufig durchzuführen, dass es sich lohnt, über einen schnellen Algorithmus nachzudenken.

8.6.2 Newton-Verfahren zur Berechnung der inversen Wurzel

Wir betrachten zunächst die Zeilen 12 und 13 in Algorithmus 8.12,

```

12  y = y * ( threehalfs - ( x2 * y * y ) );
13  // y = y * ( threehalfs - ( x2 * y * y ) );

```

also anscheinend zwei Iterationen eines Verfahrens, dessen zweite Instanz überhaupt nicht zum Einsatz kommt³. Wir formulieren diese Iteration als

$$y_{n+1} = y_n \left(\frac{3}{2} - \frac{y_n^2 x}{2} \right) =: g(y_n), \quad (8.35)$$

³ Die führenden schrägen Striche „//“ stehen für einen *Kommentar*, der Programmtext wird nicht ausgeführt.

wobei x die Zahl ist, deren inverse Wurzel zu bestimmen ist und die in Zeile 1 als `number` übergeben wird, für die `threehalfs` als $3/2$ definiert wurde (Zeile 5) und `x2` als $x/2$. Diese Iteration hat als Fixpunkt

$$y \stackrel{!}{=} g(y) \Leftrightarrow y^2 = \frac{1}{x},$$

gerade das gesuchte Ergebnis. Im Sinne von Satz 8.27 bestimmen wir die Konvergenzordnung dieser Fixpunktiteration durch Analyse der Ableitungen. Es gilt

$$g(y) = \frac{3y}{2} - \frac{y^3x}{2}, \quad g'(y) = \frac{3}{2} - \frac{3}{2}y^2x, \quad g''(y) = -3yx,$$

also für $y = x^{-\frac{1}{2}}$

$$g(x^{-\frac{1}{2}}) = x^{-\frac{1}{2}}, \quad g'(x^{-\frac{1}{2}}) = 0, \quad g''(x^{-\frac{1}{2}}) = -3x^{\frac{1}{2}}.$$

Es liegt somit eine Fixpunktiteration mit quadratischer Konvergenz vor. Die Iteration lässt sich als Anwendung des Newton-Verfahrens zur Suche der Nullstelle der Funktion

$$f(y) = x - \frac{1}{y^2}$$

schreiben, denn:

$$y_{n+1} = y_n - \frac{f(y_n)}{f'(y_n)} = y_n - \frac{x - y_n^{-2}}{2y_n^{-3}} = g(y_n).$$

Somit sind die letzten beiden Zeilen des Algorithmus verstanden. Im Rahmen der erforderlichen Genauigkeit bei `QUAKE III` scheint ein einziger Newton-Schritt zu genügen.

Es bleibt die Untersuchung der Zeilen 8 – 11 des Algorithmus. Diese vier Zeilen müssen eine gute Approximation des Startwerts darstellen, sodass im anschließenden Newton-Verfahren auch wirklich quadratische Konvergenz vorliegt. Eine besondere Bedeutung wird dabei dem Ausdruck `0x5f3759df` zukommen, der als Konstante in das Verfahren eingeht. Über diese Zahl wurde bereits viel (vielleicht zu viel) geschrieben. Konstanten, deren unmittelbarer Zweck nicht sofort ersichtlich ist, werden in der Informatik teils *magic numbers* genannt. Wir werden jedoch durch mathematische Analyse sehen, dass die hier gewählte Konstante die optimale Wahl ist.

Wir analysieren zunächst das Newton-Verfahren zur Bestimmung der inversen Wurzel im Detail. Es sei also $y \in \mathbb{R}_+$ eine erste Näherung, dann gilt mit (8.35) für $\bar{y} = g(y)$ (mit Polynomdivision)

$$\bar{y} - \frac{1}{\sqrt{x}} = -\frac{x}{2} \left(y + \frac{2}{\sqrt{x}} \right) \left(y - \frac{1}{\sqrt{x}} \right)^2. \quad (8.36)$$

Hier können wir einerseits quadratische Konvergenz, andererseits aber auch

$$\bar{y} = g(y) \leq \frac{1}{\sqrt{x}} \quad \forall y > 0$$

ablesen. Das Verfahren konvergiert dann monoton steigend von unten, da

$$y < \frac{1}{\sqrt{x}} \Rightarrow g(y) = \frac{y}{2}(3 - y^2x) > \frac{y}{2}(3 - 1) = y$$

gilt.

8.6.3 Bestimmung des Startwerts

Es gilt nun, einen möglichst guten Startwert für das Newton-Verfahren zu finden. In Algorithmus 8.12 geschieht dies durch die – zunächst schwer zu durchschauenden – Zeilen 9–11:

```

9 i = * ( int * ) &y;
10 i = 0x5f3759df - ( i >> 1 );
11 y = * ( float * ) &i;
```

Dabei werden wir etwas tiefer auf die Zahldarstellung von Computern im Binärsystem eingehen müssen. Wir verweisen auch auf Abschn. 1.4. In aktuellen C/C++ Versionen⁴ beschreibt der Datentyp `int` eine ganze Zahl, zu deren Speicherung 32 Bits zur Verfügung stellen – eines für das Vorzeichen und 31 für die weiteren Stellen:

$$x_{int} = (-1)^{b_{32}} \sum_{i=1}^{31} b_i \cdot 2^{i-1}, \quad b_1, \dots, b_{32} \in \{0, 1\}$$

Wir werden ganze Zahlen kürzer als

$$x_{int} = [\pm b_{32} b_{31} \dots b_1]$$

schreiben. Der Datentyp `float` beschreibt – damals wie heute – eine Fließkommazahl mit einfacher Genauigkeit, genauer gesagt mit 32 Bits, von denen eines, b_{32} , für das Vorzeichen, die acht Bits $b_{31}, \dots, b_{24}$ für den Exponenten und die verbleibenden 23 Bits $b_{23}, \dots, b_1$ für die Mantisse bereitstehen:

$$x_{float} = (-1)^{b_{32}} \cdot 2^{\left(\sum_{i=0}^7 b_{24+i} 2^i - 127\right)} \cdot \left(1 + 2^{-24} \sum_{i=1}^{23} b_i 2^i\right), \quad (8.37)$$

wobei durch $b = 127$ der *Bias* der Exponentendarstellung gegeben ist. Wir schreiben eine solche Zahl kürzer als

⁴ Dies trifft z. B. für gcc Version 5 zu. In aktuellen Versionen zur Zeit von Quake III hatte eine Variable vom Typ `int` nur 16 Bits zur Verfügung. Daher kam im Originalalgorithmus [89] der Datentyp `long` zum Einsatz, eine ganze Zahl mit 32 Bit, entsprechend den (schon damals verwendeten) 32 Bit im Fließkommatyp `float`.

$$x_{float} = [(-1)^{b_{32}} \cdot 2^{b_{31} \dots b_{24} - b} \mathbf{1}.b_{23} \dots b_1]_2 = (-1)^S \mathbf{1}.m \cdot 2^{E-b}.$$

Die in der Mantisse führende 1 dient der Normierung, sodass $m \in [0, 1)$, und muss nicht separat gespeichert werden. Im Folgenden betrachten wir nur positive Zahlen, sodass wir stets $S = 0$ voraussetzen und das Vorzeichen nicht extra aufführen. Den beiden Zeilen 9 und 11 im Algorithmus kommt eine besondere Bedeutung zu. Hier werden Zahldarstellungen „uminterpretiert“. Hinter

```
9 i = * (int *) & y;
```

etwa verbirgt sich kein Runden der Fließkommazahl y_{float} zu einer ganzen Zahl i_{int} , sondern eine Reinterpretation der einzelnen Bits. Aus der Gleitkommazahl

$$x_{float} = \mathbf{1}.m \cdot 2^{E-b}$$

wird so die ganze Zahl

$$i_{int} = 2^{23}(E + m) =: LE + M,$$

mit

$$L = 2^{23}, \quad M = Lm.$$

Wir bringen ein Beispiel. Die Zahl $x = 3.1415$ hat die Fließkommadarstellung

$$\begin{aligned} 3.1415 &= 1.57075 \cdot 2^{128-127} \\ &= [2^{10000000-127}_{10} \mathbf{1}.100100100001110201010110]_2. \end{aligned}$$

In entsprechender Binärdarstellung stellt die ganze Zahl mit gleicher Bitsequenz eine komplett verschiedene Dezimalzahl dar:

$$i_{int} = [10000000100100100001110201010110]_2 = 1\,078\,529\,622 \quad (8.38)$$

Diese steht in keinem Zusammenhang zu $x_{float} = 3.1415$. Wir müssen die Bedeutung der Zeilen 9 und 11 also anders erklären.

Eine weitere Besonderheit stellt der zweite Teil von Zeile 10 des Verfahrens

```
10 i = 0x5f3759df - ( i >> 1 );
```

dar, also die Operation $i \gg 1$. Die „ $\gg$ “-Operation verschiebt alle Bits in der binären Darstellung einer Zahl um eine Stelle nach rechts. Wir bleiben bei dem obigen Beispiel. Aus (8.38) wird hierdurch

$$\begin{aligned} i_{int} &= [10000000100100100001110201010110]_2 = [1\,078\,529\,622]_{[10]}, \\ i_{int} \gg 1 &= [01000000010010010000111020101011]_2 = [539\,264\,811]_{[10]}. \end{aligned}$$

Es ergibt sich also gerade die Hälfte der ursprünglichen Zahl. Das Verschieben von Bits kann von CPUs sehr schnell durchgeführt werden. Angewendet auf Fließkommazahlen macht die „ $\gg$ “-Operation wenig Sinn, da keine Rücksicht auf die Bedeutung der Bits genommen wird. Das Vorzeichen-Bit wird in den Exponenten, das letzte Exponenten-Bit in die Mantisse verschoben. Das Kürzel **0x** im ersten Teil von Zeile 10 in Algorithmus 8.12 steht für die Angabe einer Zahl im Hexadezimal-, also 16er-System, dabei ist **a** die 10, **b** die 11 und schließlich **f** die 15er-Stelle. Bei der Zahl **0x5f3759df** handelt es sich somit um die Zahl

$$5f3759df_{16} = 1\,597\,463\,007,$$

in Binärdarstellung gegeben als

$$5f3759df_{16} = [1011111001101110101100111011111]_2. \quad (8.39)$$

Zusammengefasst stehen die Zeilen 9–11 im Algorithmus für eine Operation

$$Y = Z - \frac{X}{2},$$

wobei Z die *magische Zahl* ist und Y sowie X die Reinterpretationen von der Eingabe $x \in \mathbb{R}_+$ sowie dem Startwert $y_0 \in \mathbb{R}_+$ als ganze Zahl darstellen.

Um diesen Trick zu verstehen, verwendet man eine Umformulierung der eigentlichen Aufgabe. Es gilt

$$y = x^{-\frac{1}{2}} \Leftrightarrow \log_2(y) = -\frac{1}{2} \log_2(x).$$

Die Berechnung des Logarithmus ist eine aufwendige Operation, eine Approximation des 2er-Logarithmus ist in üblicher Fließkomma-Schreibweise (8.37) jedoch einfach möglich. Für $x_{float} = \mathbf{1}.m \cdot 2^{E-b}$ gilt:

$$\begin{aligned} \log_2(y) &= -\frac{1}{2} \log_2(x_{float}) = -\frac{1}{2} \log_2(\mathbf{1}.m) - \frac{1}{2} \log_2(2^{E-b}) \\ &= -\frac{1}{2} \log_2(1 + m) - \frac{E-b}{2} \end{aligned}$$

Der erste Teil ist Logarithmus einer Zahl zwischen 1 und 2, der zweite Teil kann einfach als Division des Exponenten durch 2, also durch Bitverschiebung berechnet werden. Wir suchen nun eine Approximation zu y , geschrieben als

$$y_{float} = \mathbf{1}.m_y \cdot 2^{E_y-b},$$

sodass

$$\log_2(y_{float}) = \log_2(1 + m_y) + E_y - b \stackrel{!}{=} -\frac{1}{2} \log_2(1 + m) - \frac{E-b}{2}. \quad (8.40)$$

Aus der Reihenentwicklung von $\log(1+x)$ folgt, dass $\log_2(1+m) \approx z+m$ eine gute Approximation für $m \in [0, 1]$ ist, wobei wir z noch optimal bestimmen werden, siehe hierzu Abb. 8.10. Dies setzen wir in (8.40) ein und schreiben

$$\log_2(y_{float}) \approx z + m_y + E_y - b \stackrel{!}{\approx} -\frac{z+m}{2} - \frac{E-b}{2} \approx -\frac{1}{2} \log_2(x_{float}). \quad (8.41)$$

Gesucht ist eine einfache und gute Approximation für alle Argumente $m \in [0, 1]$ und für alle Exponenten. Eine natürliche Zuordnung scheint

$$E_y - b \approx -\frac{E-b}{2}, \quad z + m_y \approx -\frac{z+m}{2}$$

zu sein. Diese führt jedoch zu

$$m_y \approx -\frac{3z+m}{2} < 0,$$

entgegen der Konvention $m_y \in [0, 1)$. Wir führen daher eine Zahl $1 + \sigma \in \{1, 2, \dots\}$ ein, um stets einen gültigen Wertebereich von m_y zu erhalten. Die genaue Wahl von σ werden wir später klären, sie hängt auch von der – noch zu bestimmenden – Größe $z \in \mathbb{R}$ ab. Wir schreiben also (8.41) als

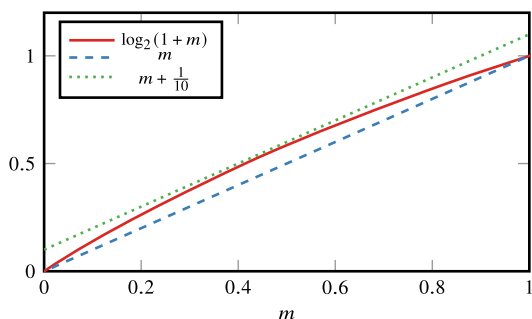
$$\log_2(y_{float}) = z + m_y + E_y - b \stackrel{!}{\approx} \left(-\frac{z+m}{2} + \frac{\sigma+1}{2} \right) - \frac{E-b+(\sigma+1)}{2}.$$

Nun bestimmen wir

$$\begin{aligned} z + m_y &= -\frac{z+m}{2} + \frac{\sigma+1}{2}, & m_y &= \frac{\sigma+1-3z-m}{2}, \\ E_y - b &= -\frac{E-b+\sigma+1}{2}, & E_y &= \frac{3b-1-\sigma-E}{2} = 190 - \frac{E+\sigma}{2}, \end{aligned} \quad (8.42)$$

wobei wir $b = 127$ verwendet haben. Zunächst können wir ablesen, dass die noch unbekannte Größe $z \in \mathbb{R}$ nicht in die Bestimmung des Exponenten E_y eingeht. Weiter können wir den Wert $\sigma \in \mathbb{N}$ näher bestimmen. Damit der neue Exponent E_y exakt durch Bitver-

Abb. 8.10 Approximation des Logarithmus durch eine lineare Funktion $\log_2(1+m) \approx \sigma + m$ mit Steigung 1 – hier, $\sigma = 0$ und $\sigma = \frac{1}{10}$ als einschließende Funktionen



schiebung berechnet werden kann, wählen wir σ gerade, falls E gerade ist, ansonsten wählen wir σ ungerade.

Exponent E_y und Mantisse m_y bestimmen die Approximation

$$y_0 = (1 + m_y) \cdot 2^{E_y - b},$$

welche wir auch als ganze Zahl interpretieren können. Mit $L = 2^{23}$ ist

$$Y_0 = Lm_y + LE_y = M_y + LE_y.$$

Setzen wir (8.42) ein, so ergibt sich

$$Y_0 = L \frac{\sigma + 1 - 3z}{2} + 190L - \frac{\sigma L}{2} - \frac{\overbrace{Lm + LE}^{=X}}{2}. \quad (8.43)$$

Wir definieren

$$Z = \lfloor L \frac{\sigma + 1 - 3z}{2} + 190L - \frac{\sigma L}{2} - \frac{\overbrace{Lm + LE}^{=X}}{2} \rfloor \Rightarrow Y_0 = Z - \frac{X}{2}$$

und die ganze Zahl Z ist gerade die *magische Zahl* des Verfahrens. Im Folgenden werden wir die Zahl Z optimal bestimmen.

8.6.3.1 Approximation des Exponenten

Das Bisherige erklärt bereits den zweiten Teil von Zeile 10. Unter Verwendung ganzzahliger Arithmetik ist

$$E_y = 190 - (E + \sigma) \gg 1$$

und in Binärdarstellung gilt

$$E_y = [10111110]_2 - (E + \sigma) \gg 1. \quad (8.44)$$

Die Ziffern der Binärdarstellung der Zahl 190 finden sich bereits in der Konstante `0x5f3759df` wieder, siehe (8.39):

$$5f3759df_{16} = [101111100110111010110011101111]_2$$

Wenn wird diese ganze Zahl wieder als Fließkommazahl interpretieren, wie es in Zeile 12 von Algorithmus 8.12 geschieht, so bildet $190 = 10111110_2$ gerade die Binärdarstellung des Exponenten, also die Bits $b_{31}, \dots, b_{24}$.

8.6.3.2 Approximation der Mantisse

Es bleibt, die Mantisse $1.m$ in (8.41) gut und effizient zu approximieren. Wir suchen in Abhängigkeit von $m \in [0, 1)$ und $\sigma \in \mathbb{N}$ ein $z \in \mathbb{R}$, sodass

$$m_y = \frac{\sigma + 1 - 3z - m}{2} = \frac{\sigma + 1 - 3z}{2} - \frac{m}{2}$$

möglichst gut approximiert. Wir müssen zwei Fälle unterscheiden, σ gerade und σ ungerade. Der erste Fall steht für gerade Exponenten E in der Darstellung der Zahl x_{float} . Aus der Analyse von Abb. 8.10 erwarten wir

$$0 \leq z \leq \frac{1}{10}$$

und betrachten auch nur diesen Bereich. Zunächst sei E ungerade, also z. B. $E = 127$ und somit $\sigma = 1$. Dann ist

$$0.3 = \frac{1 + 1 - 3 \cdot 0.1 - 0}{2} \leq \underbrace{\frac{1 + 1 - 3z}{2} - \frac{m}{2}}_{=m_y} \leq \frac{1 + 1 - 3 \cdot 0}{2} - \frac{0}{2} \leq 1,$$

d. h., m_y ist stets im gültigen Bereich. Mit (8.44) gilt für den Startwert

$$y_0 = \left(1 + \frac{2 - 3z}{2} - \frac{m}{2}\right) \cdot 2^{190 - \frac{E+1}{2}}.$$

Wir erinnern, dass für das exakte Ergebnis im Fall $1 + \sigma = 2$ gilt:

$$y = 2^{-\frac{1}{2} \log_2(1+m)+1} \cdot 2^{190 - \frac{E+1}{2}}$$

Somit gilt für den relativen Fehler des Startwerts (noch immer in Abhängigkeit von z):

$$y_0 = x^{-\frac{1}{2}} \frac{2 - \frac{3z+m}{2}}{2^{-\frac{1}{2} \log_2(1+m)+1}} =: x^{-\frac{1}{2}} \gamma(z, m)$$

Wir können nun z so bestimmen, dass $\gamma(z, m)$ für alle $m \in [0, 1]$ möglichst nahe bei 1 liegt. Wir wollen jedoch direkt das Ergebnis der ersten Newton-Iteration (8.35) berücksichtigen und berechnen

$$y_1 = g(y_0) = x^{-\frac{1}{2}} \frac{3\gamma(z, m) - \gamma(z, m)^3}{2} =: x^{-\frac{1}{2}} \delta(z, m).$$

Nun gilt es $z \in \mathbb{R}$ so zu bestimmen, dass der relative Fehler nach einem Newton-Schritt minimiert wird:

$$\max_{x \in [0, 1]} |1 - \delta(z, m)| \rightarrow \min$$

Da für Newton-Iterierte $y_n \leq x^{-\frac{1}{2}}$ gilt, ist dies gleichbedeutend mit

$$\min_{x \in [0,1]} \delta(z, m) \rightarrow \max. \quad (8.45)$$

Mögliche Extremstellen finden wir am Rand, also für $m = 0$ und $m = 1$

$$\begin{aligned} \delta_1(z) &:= \delta(z, 0) = \frac{27}{128}z^3 - \frac{27}{32}z^2 + 1, \\ \delta_2(z) &:= \delta(z, 1) = \frac{9\sqrt{2}}{64}(3z^3 - 9z^2 + z + 5), \end{aligned} \quad (8.46)$$

sowie im Innern an den Nullstellen der ersten Ableitung, d. h.

$$\frac{\partial}{\partial m} \delta(z, m) \stackrel{!}{=} 0.$$

Eine reelle Nullstelle ergibt sich bei

$$m' = \frac{2}{3} - z,$$

und hier gilt

$$\delta_3(z) := \delta(z, m') = -\frac{\sqrt{3(5-3z)}}{3888} (81z^4 - 540z^3 + 1350z^2 - 528z - 995). \quad (8.47)$$

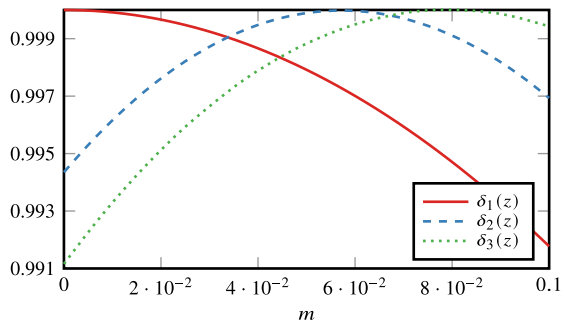
Es gilt nun, den Wert z so zu bestimmen, dass

$$\delta_{\min}(z) := \min\{\delta_1(z), \delta_2(z), \delta_3(z)\} \rightarrow \max \quad \forall z \in [0, 0.1]$$

maximiert wird. Eine grafische Analyse in Abb. 8.11 zeigt, dass dies gerade im Schnittpunkt von $\delta_1(z)$ und $\delta_3(z)$ geschieht. Im Intervall $z \in [0, 0.1]$ existiert nur ein Schnittpunkt

$$\delta_1(z) = \delta_3(z) \Rightarrow z \approx 0.044513572756643.$$

Abb. 8.11 Maximieren des Minimums von $\delta_1(z)$, $\delta_2(z)$ und $\delta_3(z)$. Das optimale Ergebnis ergibt sich im Schnittpunkt von $\delta_1(z)$ und $\delta_3(z)$



Wir approximieren den Startwert somit als

$$y_0 = \left(2 - \frac{3z}{2} - \frac{m}{2}\right) 2^{190 - \frac{E+1}{2}}.$$

Aus den gefundenen Werten für z können wir mittels (8.43) mit $\sigma = 1$ die optimale Konstante bestimmen als

$$Z = \left\lfloor L \frac{1 + 1 - 3z}{2} + 190L - \frac{1L}{2} \right\rfloor = 1\,597\,469\,713 = 5f377411_{16}.$$

Diese Zahl weicht leicht von der im Algorithmus verwendeten Konstante `0x5f3759df` ab, wir haben aber bisher auch nur den Fall ungerader Exponenten E betrachtet. Wir bestimmen aber zunächst den Wert z' , der zu der Zahl `0x5f3759df` gehört. Es gilt

$$\begin{aligned} Z' = 5f3759df_{16} = 1\,597\,463\,007 &= \left\lfloor L \frac{1 + 1 - 3z'}{2} + 190L - \frac{L}{2} \right\rfloor \\ &\Leftrightarrow z' = 0.04504656792. \end{aligned}$$

In Abb. 8.12 stellen wir das Ergebnis des inversen Wurzelalgorithmus für beide *magischen Zahlen* `0x5f3759df` und `0x5f377411` gegenüber. Es zeigt sich, dass die hier gefundene Konstante kleinere Fehler liefert, falls die Eingabe aus einem Intervall mit ungeradem Exponenten E kommt – hier also für

$$x_{float} = (1 + m) \cdot 2^{E-127},$$

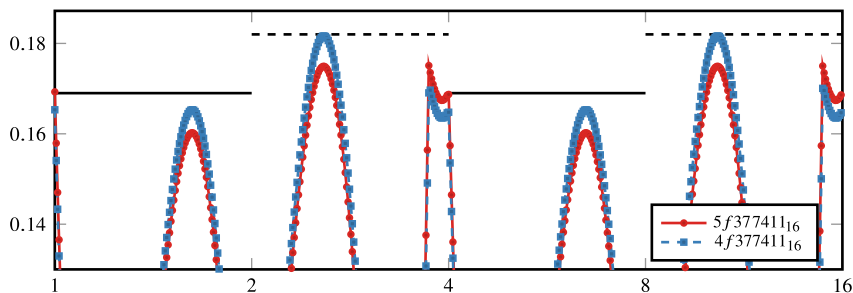


Abb. 8.12 Vergleich der *Fast Inverse Square Root* für unterschiedliche Wahl der *magischen Zahl*. Die durchgezogene (rote) Linie steht für die Konstante aus dem Originalalgorithmus. Die gestrichelte Linie stammt aus der Optimierung im Fall E ungerade. In diesen Intervallen $[1, 2]$ ($1.M \cdot 2^0 = 2^{127-127}$) sowie $[4, 8]$ ($1.M \cdot 2^2 = 2^{129-127}$) ist der Fehler bei Verwendung von `0x5f377411` kleiner. Werden alle möglichen Eingaben $x \in \mathbb{R}_+$ betrachtet, ist die Konstante aus dem Originalverfahren besser

sprich

$$x_{float} \in [1, 2], \quad x_{float} \in [4, 8],$$

mit $E = 127$ bzw. mit $E = 129$. Auf Intervallen zu geradem E ist die Konstante des ursprünglichen Verfahrens überlegen. Die Abweichung ist jedoch klein. Eine entsprechende Analyse für diesen Fall kann wie oben beschrieben durchgeführt werden.

In diesem Kapitel befassen wir uns einerseits mit der Frage, wie eine Reihe von Daten (z. B. aus physikalischen Messungen, experimentellen Beobachtungen, Börsendaten etc.) durch eine möglichst *einfache* Funktion $p(x)$ (z. B. Polynome) angenähert werden kann. Auf der anderen Seite soll geklärt werden, wie *komplizierte* Funktionen durch einfachere Funktionen approximiert werden können.

Bei der **Interpolation** wird die approximierende Funktion p derart konstruiert, dass diese an den vorliegenden (diskreten) Daten (x_k, y_k) exakt ist:

$$p(x_k) = y_k, \quad k = 0, 1, 2, 3, \dots, n$$

Die Stützstellen und Stützwerte (x_k, y_k) sind entweder diskrete Datenwerte (z. B. von Experimenten) oder durch Funktionen bestimmt $y_k = f(x_k)$. Mithilfe der Interpolation $p(x)$ können bis dato unbekannte Werte an Zwischenstellen $\xi \in (x_k, x_{k+1})$ oder das Integral bzw. die Ableitung von $p(x)$ approximiert werden. Darüber hinaus hat die Interpolation eine wesentliche Bedeutung für die Entwicklung weiterer numerischer Verfahren wie beispielsweise numerische Quadratur, Differentiation oder in weiterführenden Vorlesungen für die Entwicklung finiter Elemente [73].

Die **Approximation**¹ ist allgemeiner gefasst. Wieder wird eine einfache Funktion $p(x)$ (also z. B. ein Polynom) gesucht, welche diskrete oder durch Funktionen bestimmte Datenwerte (x_k, y_k) möglichst gut approximiert. Im Gegensatz zur Interpolation wird jedoch nicht zwingend $p(x_k) = y_k$ in ausgezeichneten Punkten x_k gefordert. Was die Approximation

¹ Approximation [26] (Annäherung): „Darstellung einer Zahl, einer Funktion, einer Kurve oder eines anderen mathematischen Objekts durch eine einfachere Zahl, Funktion, Kurve, Objekt, wobei der Fehler möglichst klein gehalten werden soll.“

auszeichnet, wird von Fall zu Fall entschieden. Möglich ist z. B. bei der Approximation einer Funktion f die beste Approximation bzgl. einer Norm

$$p \in P : \|p - f\| = \min_{q \in P} \|q - f\|,$$

wobei P die Klasse der betrachteten einfachen Funktionen ist (also z. B. alle quadratischen Polynome). Eine Anwendung der Approximation ist das „Fitten“ von diskreten Datenwerten, die durch Experimente gegeben sind. Oft werden viele tausend Messwerte berücksichtigt, die von der zugrundeliegenden (etwa physikalischen) Formel jedoch aufgrund von statistischen Messfehlern nicht exakt im Sinne der Interpolation, sondern nur approximativ erfüllt werden sollen. Eine zweite Anwendung ist wieder die möglichst einfache Darstellung von gegebenen Funktionen.

Grundsätzlich hat die Darstellung von diskreten Daten als einfache Funktion oder die Approximation einer allgemeinen Funktion durch z. B. ein Polynom den großen Vorteil, dass Elementaroperationen wie Ableitungsbildung und Integration viel einfacher ausgeführt werden können. Als Funktionenräume zur Approximation werden möglichst *einfache* Funktionen verwendet. Beispiele sind die Polynomräume mit Funktionen

$$p(x) = \sum_{k=0}^n \alpha_k x^k$$

oder z. B. die Räume der trigonometrischen Funktionen

$$q(x) = \sum_{k=0}^n \alpha_k \cos(k\pi x) + \beta_k \sin(k\pi x).$$

Der in der Einleitung vorgestellte *Weierstraßsche Approximationssatz*, Satz 7.6 besagt, dass es zu jeder stetigen Funktion $f : [a, b] \rightarrow \mathbb{R}$ und jeder vorgegebenen Genauigkeit ϵ ein Polynom p gibt, so dass $|f(x) - p(x)| < \epsilon$ für alle $x \in [a, b]$ gilt. Ein Beweis zum Weierstraßschen Approximationssatz ist konstruktiv und erzeugt eine Folge von sogenannten *Bernsteinpolynomen*, die gegen f konvergieren. Diese Konvergenz ist jedoch langsam und eignet sich nicht als Grundlage für ein effizientes numerisches Verfahren [47].

Ein weiterer Zugang zur Approximation von Funktionen mit Polynomen ist die Taylor-Entwicklung, die wir in Satz 7.5 beschrieben haben. Diese benötigt hohe Regularität der Funktion f und liefert dann (lokal) sehr gute Konvergenz. Jedoch eignet sich auch die Taylor-Entwicklung nicht als Grundlage für ein effizientes und stabiles Verfahren. Taylor-Approximationen werden jedoch ein wichtiges Hilfsmittel in der kommenden Analyse sein.

Ganz aktuell spielt die Approximationstheorie in Zusammenhang mit dem *maschinellen Lernen*, speziell auf dem Gebiet der *künstlichen neuronalen Netze* eine große Rolle. Hierbei handelt es sich um spezielle Funktionen, die aus sehr einfachen Bausteinen aufgebaut aber über eine Vielzahl von freien Parametern verfügen und die in der Lage sind, sehr komplexe Zusammenhänge gut darzustellen. Wir gehen in Abschn. 11.5 hierauf ein.

9.1 Polynominterpolation

Wir bezeichnen mit P_n den Vektorraum der Polynome vom Grad $\leq n$:

$$P_n := \left\{ p(x) = \sum_{k=0}^n a_k x^k, \ a_k \in \mathbb{R}, \ k = 0, \dots, n \right\}$$

► **Definition 9.1 (Lagrangesche Interpolationsaufgabe)** Die Lagrangesche Interpolationsaufgabe besteht darin, zu $n + 1$ paarweise verschiedenen Stützstellen (Knoten) $x_0, \dots, x_n \in \mathbb{R}$ und zugehörigen gegebenen Stützwerten $y_0, \dots, y_n \in \mathbb{R}$ ein Polynom $p \in P_n$ zu bestimmen, sodass die Interpolationsbedingung

$$p(x_k) = y_k, \quad k = 0, \dots, n,$$

erfüllt ist.

Satz 9.2 Die Lagrangesche Interpolationsaufgabe ist eindeutig lösbar.

Beweis (i) Wir weisen zunächst die Eindeutigkeit nach. Angenommen, $p_1, p_2 \in P_n$ seien zwei Lösungen, gegeben als

$$p_i(x) = \sum_{k=0}^n a_k^{(i)} x^k.$$

Für die Differenz $q := p_1 - p_2 \in P_n$ gilt dann $q(x_i) = 0$ in den $n + 1$ paarweise verschiedenen Punkten. Nach dem Satz von Rolle, Satz 7.3 in Kap. 7, hat die Ableitung q' dann n paarweise verschiedene Nullstellen $x_i^{(1)}$, jeweils in den Intervallen

$$q'(x_i^{(1)}) = 0, \quad x_i^{(1)} \in (x_i, x_{i+1}), \quad i = 0, \dots, n-1.$$

Wir können den Satz von Rolle wiederholt anwenden, bis wir schließlich noch eine Nullstelle $x_0^{(n)}$ der n -ten Ableitung von $q(x)$, also $q^{(n)}(x)$ erhalten. Das Polynom $q^{(n)}(x)$ hat aber die Darstellung

$$q^{(n)}(x) = n!(a_n^{(1)} - a_n^{(2)}),$$

also folgt $a_n^{(1)} = a_n^{(2)}$ und $q^{(n)} \equiv 0$. Für die $n - 1$ -te Ableitung ist

$$q^{(n-1)}(x) = (n-1)!(a_{n-1}^{(1)} - a_{n-1}^{(2)}),$$

und da diese zwei Nullstellen hat, folgt wieder $a_{n-1}^{(1)} = a_{n-1}^{(2)}$, also $q^{(n-1)} \equiv 0$. Schließlich gilt $q \equiv 0$, also $p_1 = p_2$.

(ii) Zum Nachweis der Existenz betrachten wir die bestimmenden Gleichungen

$$p(x_k) = y_k, \quad k = 0, \dots, n.$$

Dies kann als ein lineares Gleichungssystem aufgefasst werden mit $n+1$ Gleichungen für die $n+1$ unbekannten Koeffizienten $a_0, \dots, a_n$ des Polynoms $p \in P_n$. Aus der Eindeutigkeit der Lösung (also der Injektivität) folgt auch die Lösbarkeit (also die Surjektivität). $\square$

Die gegebenen Stützwerte y_k können Werte einer gegebenen Funktion f sein, d. h.

$$f(x_k) = y_k, \quad k = 0, 1, \dots, n,$$

oder auch beliebige diskrete Datenwerte

$$(x_k, y_k), \quad k = 0, 1, \dots, n.$$

Sind die Stützstellen paarweise verschieden, so lässt sich die Interpolationsaufgabe als ein System von linearen Gleichungen formulieren

$$\begin{aligned} a_0 + a_1 x_0 + a_2 x_0^2 + \dots + a_n x_0^n &= y_0 \\ a_0 + a_1 x_1 + a_2 x_1^2 + \dots + a_n x_1^n &= y_1 \\ &\vdots \\ a_0 + a_1 x_n + a_2 x_n^2 + \dots + a_n x_n^n &= y_n, \end{aligned}$$

bzw. in Matrixschreibweise

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{pmatrix}.$$

Das Erstellen des Interpolationspolynoms erfordert demnach das Lösen eines quadratischen linearen Gleichungssystems mit $n+1$ Gleichungen. Die Matrix ist die sogenannte *Vandermonde-Matrix*, siehe [32]. Im allgemeinen Fall ist diese Matrix sehr schlecht konditioniert mit einer exponentiell steigenden Konditionszahl. Dieser direkte Weg eignet sich daher nicht zum praktischen Lösen der Interpolationsaufgabe, vergleiche Kap. 3.

9.1.1 Lagrangesche Basispolynome

Zur Bewältigung der Interpolationsaufgabe werden wir eine Basis des Raums P_n definieren, welche ein unmittelbares Aufstellen des Interpolationspolynoms erlaubt. Wir definieren:

Satz 9.3 (Lagrange-Basispolynome) Es seien $x_k \in \mathbb{R}$ für $k = 0, \dots, n$ paarweise verschiedene Stützstellen. Die Lagrange-Polynome

$$L_k^{(n)}(x) := \prod_{j=0, j \neq k}^n \frac{x - x_j}{x_k - x_j} \in P_n, \quad k = 0, 1, \dots, n,$$

definieren eine Basis des P_n . Weiter gilt

$$L_k^{(n)}(x_l) = \delta_{kl}. \quad (9.1)$$

Beweis (i) Wir zeigen zunächst die Eigenschaft (9.1). Es gilt

$$L_k^{(n)}(x_k) = \prod_{j=0, j \neq k}^n \frac{x_k - x_j}{x_k - x_j} = 1.$$

Weiter ist für $l \neq k$

$$L_k^{(n)}(x_l) = \prod_{j=0, j \neq k}^n \frac{x_l - x_j}{x_k - x_j} = \frac{x_l - x_l}{x_k - x_l} \prod_{j=0, j \neq k, j \neq l}^n \frac{x_l - x_j}{x_k - x_j} = 0.$$

(ii) Nach Konstruktion gilt $L_k^{(n)} \in P_n$ für $k = 0, \dots, n$. Darüber hinaus sind alle Polynome $L_k^{(n)}$ linear unabhängig. Ansonsten gäbe es Koeffizienten $\alpha_1, \dots, \alpha_n$ mit

$$L_0^{(n)}(x) = \sum_{i=1}^n \alpha_i L_i^{(n)}(x).$$

Für $x = x_0$ ergibt sich ein Widerspruch. □

Satz 9.4 (Lagrangesche Darstellung) Es seien $x_0, \dots, x_n \in \mathbb{R}$ paarweise verschiedene Stützstellen und $y_0, \dots, y_n \in \mathbb{R}$. Die (eindeutige) Lösung der Lagrangeschen Interpolationsaufgabe $p(x_k) = y_k$ für $k = 0, \dots, n$ ist gegeben durch

$$p(x) := \sum_{k=0}^n y_k L_k^{(n)}(x).$$

Beweis Diese Aussage folgt unmittelbar aus soeben nachgewiesenen Eigenschaften der Lagrangeschen Basispolynome. □

Die Lagrangesche Darstellung des Interpolationspolynoms besteht durch ihre Einfachheit. Sie hat allerdings den großen Nachteil, dass jedes Basispolynom $L_k^{(n)}(x)$ von sämtlichen Stützstellen $x_0, x_1, \dots, x_n$ abhängt. Angenommen, zur Steigerung der Genauigkeit soll eine Stützstelle x_{n+1} hinzugenommen werden, so müssen sämtliche Basispolynome ausgetauscht werden.

Beispiel 9.5

Gegeben seien die Punkte $(x_0, y_0) = (1, 3)$ und $(x_1, y_1) = (4, 8)$. Wir konstruieren nun die Lagrange-Basispolynome und das entsprechende lineare Interpolationspolynom. Wir erhalten

$$L_0^{(1)}(x) = \frac{x - x_1}{x_0 - x_1} = \frac{x - 4}{-3} \in P_1,$$

$$L_1^{(1)}(x) = \frac{x - x_0}{x_1 - x_0} = \frac{x - 1}{-3} \in P_1.$$

Insbesondere gelten die Interpolationsbedingungen, sprich die Kronecker-Delta-Eigenschaft, in den entsprechenden Interpolationspunkten:

$$L_0^{(1)}(x_0) = \frac{1 - 4}{-3} = 1, \quad L_0^{(1)}(x_1) = \frac{4 - 4}{-3} = 0$$

Das Interpolationspolynom lautet

$$p(x) = y_0 L_0^{(1)}(x) + y_1 L_1^{(1)}(x) = 3 \frac{x - 4}{-3} + 8 \frac{x - 1}{-3}.$$



9.1.2 Newtonsche Basispolynome

Die Newton-Darstellung beruht auf einer Basis, die eine sukzessive Erweiterung des Interpolationspolynoms um neue Stützstellen erlaubt.

Satz 9.6 (Newton-Basispolynome) Es seien $x_0, \dots, x_n \in \mathbb{R}$ paarweise verschiedene Stützstellen. Durch

$$N_0(x) := 1,$$

$$N_k(x) := \prod_{j=0}^{k-1} (x - x_j), \quad k = 1, \dots, n,$$

ist eine Basis des P_n gegeben.

Beweis Die Basiseigenschaft kann wieder einfach nachgewiesen werden. Zunächst gilt $N_k \in P_n$ für $k = 0, \dots, n$. Weiter sind die Basispolynome linear unabhängig. Dies folgt aus der Eigenschaft

$$\begin{cases} N_k(x_l) = 0 & k > l, \\ N_k(x_k) \neq 0 & k \leq l. \end{cases}$$

Ein Polynom N_k kann nicht durch eine lineare Kombination von Polynomen P_l mit $l < k$ dargestellt werden. $\square$

Das Basispolynom $N_k(x)$ hängt nur von den Stützstellen $x_0, \dots, x_k$ ab. Bei Hinzunahme einer Stützstelle x_{k+1} müssen die ersten Basispolynome nicht geändert werden. Das Interpolationspolynom wird nach dem Ansatz

$$p(x) = \sum_{k=0}^n a_k N_k(x)$$

bestimmt. Für die Stützstelle x_k gilt $N_l(x_k) = 0$ für alle $l > k$. Wir gehen rekursiv vor:

$$\begin{aligned} y_0 &\stackrel{!}{=} p(x_0) = a_0 \\ y_1 &\stackrel{!}{=} p(x_1) = a_0 + a_1(x_1 - x_0) \\ &\vdots \\ y_n &\stackrel{!}{=} p(x_n) = a_0 + a_1(x_n - x_0) + \dots + a_n(x_n - x_0) \cdots (x_n - x_{n-1}) \end{aligned}$$

Im Gegensatz zur vorher kennengelernten Lagrangeschen Darstellung kann ein weiteres Datenpaar (x_{n+1}, y_{n+1}) leicht hinzugefügt werden. Ist das Interpolationspolynom $p_n \in P_n$ gegeben, so kann eine Stützstelle x_{n+1} einfach hinzugenommen werden:

$$\begin{aligned} y_{n+1} &\stackrel{!}{=} p_{n+1}(x_{n+1}) = p_n(x_{n+1}) + a_{n+1}N_{n+1}(x_{n+1}) \\ &\Rightarrow a_{n+1} = \frac{y_{n+1} - p_n(x_{n+1})}{N_{n+1}(x_{n+1})} \quad (9.2) \end{aligned}$$

In der Praxis werden die Koeffizienten a_k durch den folgenden Algorithmus numerisch stabil berechnet:

Satz 9.7 Es seien $x_k \in \mathbb{R}$ für $k = 0, \dots, n$ paarweise verschiedene Stützstellen und $y_k \in \mathbb{R}$. Das Lagrangesche Interpolationspolynom lautet bezüglich der Newtonschen Polynombasis

$$p(x) = \sum_{k=0}^n y[x_0, \dots, x_k] N_k(x).$$

Die Notation $y[x_0, \dots, x_k]$ bezeichnet die dividierten Differenzen, welche über die folgende rekursive Vorschrift definiert sind:

Eingabe: Stützstellenpaare (x_k, y_k) für $k = 1, \dots, n$.

1 **Für** $k = 0$ **bis** n **tue**

2 $y[x_k] := y_k$

3 **Für** $l = 1$ **bis** n **tue**

4 **Für** $k = 0$ **bis** $n - l$ **tue**

5 $y[x_k, \dots, x_{k+l}] := \frac{y[x_{k+1}, \dots, x_{k+l}] - y[x_k, \dots, x_{k+l-1}]}{x_{k+l} - x_k}$

Ergebnis: Die Koeffizienten des Lagrangesche Interpolationspolynom bezüglich der Newtonschen Polynombasis.

Beweis Wir bezeichnen mit $p_{k,k+l} \in P_l$ das Polynom, welches die Interpolation zu den $l + 1$ Punkten $(x_k, y_k), \dots, (x_{k+l}, y_{k+l})$ darstellt. Insbesondere ist durch $p_{0,n}$ das gesuchte Polynom $p \in P_n$ gegeben. Wir zeigen, dass die folgende Aussage für alle $l = 0, \dots, n$ und $k = 0, \dots, n - l$ gilt:

$$p_{k,k+l}(x) = y[x_k] + y[x_k, x_{k+1}](x - x_k) + \dots + y[x_k, \dots, x_{k+l}](x - x_k) \cdots (x - x_{k+l-1}) \quad (9.3)$$

Wir führen den Beweis durch Induktion nach dem Polynomgrad l . Für $l = 0$ gilt $p_{k,k}(x) = y[y_k] = y_k$. Angenommen, die Behauptung sei richtig für $l - 1 \geq 0$. Das heißt insbesondere, das Polynom $p_{k,k+l-1}$ ist in Darstellung (9.3) gegeben und interpoliert die Punkte $(x_k, y_k), \dots, (x_{k+l-1}, y_{k+l-1})$, und das Polynom $p_{k+1,k+l}$ interpoliert die Punkte $(x_{k+1}, y_{k+1}), \dots, (x_{k+l}, y_{k+l})$. Dann ist durch

$$q(x) = \frac{(x - x_k)p_{k+1,k+l}(x) - (x - x_{k+l})p_{k,k+l-1}(x)}{x_{k+l} - x_k} \quad (9.4)$$

ein Interpolationspolynom durch die Punkte $(x_k, y_k), \dots, (x_{k+l}, y_{k+l})$ gegeben. Dies wird durch Einsetzen von x_i in $q(x)$ deutlich. Für innere Punkte $i = k + 1, \dots, k + l - 1$ gilt $p_{k,k+l-1}(x_i) = p_{k+1,k+l}(x_i) = y_i$, für x_k und x_{k+l} ist jeweils einer der beiden Faktoren gleich null. Es gilt somit für das gesuchte Interpolationspolynom $p_{k,k+l} = q$. Nach Konstruktion (9.2) gilt jedoch auch die folgende Darstellung:

$$p_{k,k+l}(x) = p_{k,k+l-1}(x) + a(x - x_k) \cdots (x - x_{k+l-1})$$

Koeffizientenvergleich des führenden Monoms x^n zwischen dieser Darstellung und (9.4) liefert mithilfe von (9.3) für $p_{k,k+l-1}$ sowie $p_{k+1,k+l}$

$$a = \frac{y[x_{k+1}, \dots, x_{k+l}] - y[x_k, \dots, x_{k+l-1}]}{x_{k+l} - x_k} = y[x_k, \dots, x_{k+l}].$$

□

Beispiel 9.8

Wir führen Beispiel 9.5 fort. Gegeben seien die Punkte $(x_0, y_0) = (1, 3)$ und $(x_1, y_1) = (4, 8)$. In der Newtonschen Darstellung haben wir die Basispolynome

$$N_0(x) = 1, \quad N_1(x) = (x - x_0) = (x - 1).$$

Die Koeffizienten bestimmen sich durch

$$3 = y[x_0], \quad 8 = y[x_1]$$

und die erste dividierte Differenz

$$y[x_0, x_1] = \frac{y[x_1] - y[x_0]}{x_1 - x_0} = \frac{8 - 3}{4 - 1} = \frac{5}{3}.$$

Damit erhalten wir als Interpolationspolynom

$$p(x) = y[x_0]N_0(x) + y[x_0, x_1]N_1(x) = 3 \cdot 1 + \frac{5}{3}(x - 1).$$

Wir machen eine kleine Probe, ob wir die Interpolationsbedingungen zurückgewinnen können:

$$\begin{aligned} p(x_0) &= 3 + \frac{5}{3}(1 - 1) = 3 \\ p(x_1) &= 3 + \frac{5}{3}(4 - 1) = 3 + 5 = 8 \end{aligned}$$

Die Probe ist erfolgreich. ◀

Dieses rekursive Konstruktionsprinzip legt sofort einen Algorithmus zum Auswerten der Interpolation an einer Stelle $\xi \in \mathbb{R}$ nahe, ohne dass das gesuchte Interpolationspolynom $p(x)$ zuvor explizit bestimmt werden muss. Wir gehen hierzu von der Darstellung (9.4) aus:

Algorithmus 9.1: Neville-Schema

Eingabe: Es seien die Stützstellenpaare (x_k, y_k) für $k = 0, \dots, n$ gegeben sowie ein Auswertungspunkt ξ .

1 **Für** $k = 0$ **bis** n **tue**

2 $p_{k,k} := y_k$

3 **Für** $j = 1$ **bis** n **tue**

4 **Für** $k = 0$ **bis** $n - j$ **tue**

5 $p_{k,k+j} := p_{k,k+j-1} + (\xi - x_k) \frac{p_{k+1,k+j} - p_{k,k+j-1}}{x_{k+j} - x_k}$

Ergebnis: $p_{0,n}$ ist der Wert des Interpolationspolynoms ausgewertet an der Stelle ξ .

♣ Implementierung 9.1: Neville-Schema

```

1 def neville(data, xi):
2     n = data.shape[0]
3     x = data[:, 0]
4     p = np.diag(data[:, 1])
5
6     for j in range(1, n):
7         for k in range(n - j):
8             p[k, k + j] = p[k, k + j - 1] + (xi - x[k]) * (p[k + 1, k +
9                 ↪ j] - p[k, k + j - 1]) / (x[k + j] - x[k])
10
11     return p

```

Bei der Durchführung des Neville-Schemas erhalten wir mit $p_{k,l}$ automatisch die Auswertung an der Stelle ξ der Interpolationspolynome durch (x_k, y_k) bis (x_{k+l}, y_{k+l}) als Zwischenergebnisse.

Bemerkung 9.9 (Dividierte Differenzen) Die dividierten Differenzen $y[x_0, \dots, x_n]$ erscheinen in ihrem Sinn und Zweck zunächst unklar, sind aber für die algorithmische Nutzung bestens geeignet, da sie sich sehr einfach berechnen lassen. Wir gehen nun davon aus, dass die Stützwerte von einer analytischen Funktion $f \in C^\infty$ abgegriffen sind, sodass

$$y_k = f(x_k)$$

gilt. Weiter seien die Stützstellen gleichmäßig verteilt mit $x_k = x_0 + hk$ und einem $h \in \mathbb{R}$. Dann gilt mit Taylor-Entwicklung

$$y[x_k, x_{k+1}] = \frac{f(x_k + h) - f(x_k)}{h} = f'(x_{k+\frac{1}{2}}) + \sum_{j=1}^{\infty} \frac{f^{(2j+1)}(x_{k+\frac{1}{2}})}{(2j+1)!} h^{2j}.$$

Die erste dividierte Differenz ist also eine Approximation an die erste Ableitung der Funktion f . Dann gilt weiter

$$\begin{aligned}
 y[x_k, x_{k+2}] &= \frac{y[x_{k+1}, x_{k+2}] - y[x_{k+1}, x_{k+2}]}{2h} \\
 &= \frac{f'(x_{k+\frac{3}{2}}) - f'(x_{k+\frac{1}{2}}) + \mathcal{R}_{k,k+2}}{2h} \\
 &= \frac{1}{2} f''(x_{k+1}) + \sum_{j=1}^{\infty} \frac{f^{(2j+1)}}{2(2j+1)!} h^{2j}
 \end{aligned}$$

mit einem Rest

$$\frac{\mathcal{R}_{k,k+2}}{2h} = \sum_{j=1}^{\infty} \frac{f^{(2j+1)}(x_{k+\frac{3}{2}}) - f^{(2j+1)}(x_{k+\frac{1}{2}})}{2(2j+1)!} h^{2j-1}.$$

Wir betrachten nur den ersten kritischen Term für $j = 1$, da das Restglied hier nur von erster Ordnung ist. Es gilt

$$\frac{f'''(x_{k+\frac{3}{2}}) - f'''(x_{k+\frac{1}{2}})}{12} h = \sum_{j=1}^{\infty} \frac{f^{(2j+2)}(x_{k+1})}{12(2j)!} h^{2j}.$$

Damit gilt wieder

$$y[x_k, x_{k+2}] = \frac{1}{2} f''(x_{k+\frac{1}{2}}) + O(h^2).$$

Allgemein kann für analytische Funktionen gezeigt werden, dass

$$y[x_k, x_{k+1}, \dots, x_{k+l}] = \frac{f^{(l)}\left(\frac{x_k + x_{k+l}}{2}\right)}{l!} + O(h^2)$$

gilt. Die dividierten Differenzen sind Approximationen der Taylor-Koeffizienten. ◆

Beispiel 9.10 (Neville-Schema)

Wir betrachten die Stützstellenpaare

$$\{(x_k, y_k)_k, k = 0, 1, 2, 3\} = \{(0, 0), (1, 1), (2, 8), (3, 27)\},$$

welche von der Funktion $f(x) = x^3$ abgegriffen sind. Wir führen das Neville-Schema zur Berechnung der Interpolation im Punkt $\xi = 0.5$ rekursiv aus und erhalten folgende Ergebnisse mit unserem Code:

```
[[ 0.      0.5     -0.25    0.125]
 [ 0.      1.      -2.5     2.    ]
 [ 0.      0.       8.      -20.5  ]
 [ 0.      0.       0.       27.   ]]
```

Die finale Approximation $p_{03} = 0.125 = 0.5^3$ ist exakt. Dies ist zu erwarten, da $f \in P_3$. Als Stichprobe betrachten wir die sehr schlechte Approximation p_{23} , welche sich durch das lineare Interpolationspolynom $p_{2,3}(x)$ durch die Stützstellen $(2, 8)$ und $(3, 27)$, also durch

$$p_{2,3}(x) = 8 + \frac{27-8}{3-2}(x-2) = 19x - 30$$

ergibt. Es gilt $p_{2,3}(0.5) = -20.5$. ◀

9.1.3 Interpolation von Funktionen und Fehlerabschätzungen

In diesem Abschnitt diskutieren wir die Interpolation von Funktionen. Die Punkte sind nun nicht mehr durch einen Datensatz gegeben, sondern durch Auswertung einer gegebenen Funktion f auf $[a, b]$:

$$y_k = f(x_k), \quad x_k \in [a, b], \quad k = 0, \dots, n$$

Die Durchführbarkeit, also Existenz und Eindeutigkeit eines Interpolationspolynoms wurde bereits in den vorangehenden Abschnitten beantwortet. Bei der Interpolation von Funktionen stellt sich hier die Frage, *wie gut* das Interpolationspolynom $p \in P_n$ die Funktion f auf $[a, b]$ approximiert.

Satz 9.11 (Interpolationsfehler mit differenziellem Restglied) Es sei $f \in C^{n+1}[a, b]$ und $p \in P_n$ das Interpolationspolynom zu f in den $n + 1$ paarweise verschiedenen Stützstellen $x_0, \dots, x_n$. Dann gibt es zu jedem $x \in [a, b]$ ein $\xi \in (a, b)$, sodass

$$f(x) - p(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^n (x - x_j). \quad (9.5)$$

Insbesondere gilt

$$|f(x) - p(x)| \leq \frac{\max_{\xi \in (a,b)} |f^{(n+1)}(\xi)|}{(n+1)!} \prod_{j=0}^n |x - x_j|. \quad (9.6)$$

Beweis Falls x mit einer Stützstelle zusammenfällt, d. h. $x = x_k$ für ein $k \in \{0, \dots, n\}$, dann verschwindet der Fehler und wir sind fertig. Es sei daher $x \neq x_k$ für alle $k = 0, 1, \dots, n$ und $F \in C^{n+1}[a, b]$ und $p_n \in P_n$ mit

$$F(t) := f(t) - p_n(t) - K(x) \prod_{j=0}^n (t - x_j).$$

$K(x)$ sei so bestimmt, dass $F(x) = 0$. Dies ist möglich, da

$$\prod_{j=0}^n (x - x_j) \neq 0 \quad \Rightarrow \quad K(x) = \frac{f(x) - p_n(x)}{\prod_{j=0}^n (x - x_j)}.$$

Dann besitzt $F(t)$ in $[a, b]$ mindestens $n + 2$ verschiedene Nullstellen $x_0, x_1, \dots, x_n, x$. Durch wiederholte Anwendung des Satzes von Rolle hat die Ableitung $F^{(n+1)}$ mindestens eine Nullstelle $\xi \in (a, b)$. Mit

$$\begin{aligned} 0 &= F^{(n+1)}(\xi) = f^{(n+1)}(\xi) - p^{(n+1)}(\xi) - K(x)(n+1)! \\ &= f^{(n+1)}(\xi) - 0 - K(x)(n+1)! \end{aligned}$$

folgt die Behauptung mittels

$$K(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \Rightarrow f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{j=0}^n (x - x_j).$$

□

Etwas aufwändiger ist der Nachweis eines integralen Restgliedes der Interpolation. Hierfür benötigen wir zunächst einen Hilfssatz, die *Relation von Hermite*.

Satz 9.12 (Relation von Hermite) Für paarweise verschiedene Stützstellen $x_0, \dots, x_{m+1} \in [a, b]$ gilt:

$$f[x_0, \dots, x_{l-1}, x] = f[x_0, \dots, x_{l-1}, x_l] + f[x_0, \dots, x_{l-1}, x_l, x](x - x_l)$$

mit $1 \leq l \leq m + 1$.

Beweis Einerseits gilt $x = x_l$ für

$$\begin{aligned} &f[x_0, \dots, x_{l-1}, x_l] + f[x_0, \dots, x_{l-1}, x_l, x](x - x_l) \\ &= f[x_0, \dots, x_{l-1}, x_l] + f[x_0, \dots, x_{l-1}, x_l, x_l](x_l - x_l) \\ &= f[x_0, \dots, x_{l-1}, x]. \end{aligned}$$

Andererseits sei nun $x \neq x_l$. Wir definieren

$$F(t_{l+1}) := f^{(l)}(x_0 + \dots + t_l(x_l - x_{l-1}) + t_{l+1}(x - x_l)).$$

Hieraus folgt für die Ableitung

$$\frac{d}{dt_{l+1}} F(t_{l+1}) := f^{(l+1)}(x_0 + \dots + t_l(x_l - x_{l-1}) + t_{l+1}(x - x_l)) \cdot (x - x_l).$$

Damit schließen wir

$$\begin{aligned}
 & (x - x_l) f[x_0, \dots, x_l, x] \\
 &= (x - x_l) \int_0^1 dt_1 \dots \int_0^{t_{l-1}} dt_l \dots \\
 & \quad \dots \int_0^{t_l} dt_{l+1} f^{(l+1)}(x_0 + t_1(x_1 - x_0) + \dots + t_l(x_l - x_{l-1}) + t_{l+1}(x - x_l)) \\
 &= (x - x_l) \frac{1}{x - x_l} \int_0^1 dt_1 \dots \int_0^{t_{l-1}} dt_l \dots \int_0^{t_l} dt_{l+1} \frac{d}{dt_{l+1}} F(t_{l+1}) \\
 &= \int_0^1 dt_1 \dots \int_0^{t_{l-1}} dt_l F(t_{l+1}) \Big|_0^{t_l} \\
 &= \int_0^1 dt_1 \dots \int_0^{t_{l-1}} dt_l f^{(l)}(x_0 + \dots + t_l(x - x_{l-1})) \\
 & \quad - \int_0^1 dt_1 \dots \int_0^{t_{l-1}} dt_l f^{(l)}(x_0 + \dots + t_l(x_l - x_{l-1})) \\
 &= f[x_0, \dots, x_{l-1}, x] - f[x_0, \dots, x_{l-1}, x_l].
 \end{aligned}$$

Die Aussage ist damit bewiesen. $\square$

Satz 9.13 (Interpolationsfehler mit Integral-Restglied) Es sei $f \in C^{n+1}[a, b]$. Dann gilt für $x \in [a, b] \setminus \{x_0, \dots, x_n\}$ die Darstellung

$$f(x) - p(x) = f[x_0, \dots, x_n, x] \prod_{j=0}^n (x - x_j)$$

mit den Interpolationsbedingungen $f[x_i, \dots, x_{i+k}] := y[x_i, \dots, x_{i+k}]$ und

$$f[x_0, \dots, x_n, x] = \int_0^1 \int_0^{t_1} \dots \int_0^{t_n} f^{(n+1)}(x_0 + t_1(x_1 - x_0) + \dots + t_n(x - x_n)) dt \dots dt_2 dt_1.$$

Beweis Das Ziel ist zu zeigen, dass die Integraldarstellung mit den dividierten Differenzen aus Satz 9.7 übereinstimmt. Sind also $x_0, \dots, x_n$ paarweise verschieden, dann wollen wir zeigen, dass gilt:

$$y[x_0, \dots, x_n] = f[x_0, \dots, x_n]$$

Dies begründen wir im Folgenden. Das Newton-Interpolationspolynom p zu f und paarweise verschiedenen $x_0, \dots, x_n$ kann dargestellt werden als

$$p(x) = y[x_0] + y[x_0, x_1](x - x_0) + \dots + y[x_0, \dots, x_n](x - x_0) \cdots (x - x_{n-1}).$$

Somit gilt $f(x_k) = p(x_k)$ für $k = 0, \dots, n$. Auf der anderen Seite folgt aus der Relation von Hermite, Satz 9.12:

$$\begin{aligned}
 f(x) &= f[x_0] + (x - x_0)f[x_0, x] \\
 &= f[x_0] + (x - x_0)(f[x_0, x_1] + (x - x_1)f[x_0, x_1, x]) \\
 &= f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)f[x_0, x_1, x] \\
 &= f[x_0] + (x - x_0)f[x_0, x_1] + (x - x_0)(x - x_1)(f[x_0, x_1, x_2] \\
 &\quad + (x - x_2)f[x_0, x_1, x_2, x]) \\
 &= \dots \\
 &= f[x_0] + (x - x_0)f[x_0, x_1] + \dots \\
 &\quad + (x - x_0) \cdots (x - x_{n-1})f[x_0, \dots, x_{n-1}, x].
 \end{aligned}$$

Oben hatten wir aber bereits $f(x_k) = p(x_k)$ für $k = 0, \dots, n$ und daher

$$y[x_0] = p(x_0) = f(x_0) = f[x_0].$$

Da die Stützstellen paarweise verschieden sind, gilt insbesondere $x_1 - x_0 \neq 0$ und wir erhalten für x_1

$$\begin{aligned}
 p(x_1) &= y[x_0] + y[x_0, x_1](x_1 - x_0) \\
 \text{und } f(x_1) &= f[x_0] + f[x_0, x_1](x_1 - x_0).
 \end{aligned}$$

Durch Vergleich folgt

$$f[x_0, x_1] = y[x_0, x_1].$$

Mit vollständiger Induktion kann diese Relation nun für $k = 1, \dots, n$ gezeigt werden:

$$\begin{aligned}
 p(x_k) &= y[x_0] + y[x_0, x_1](x_1 - x_0) + \dots \\
 &\quad + y[x_0, \dots, x_k](x_k - x_0) \cdots (x_k - x_{k-1}) \\
 \text{und } f(x_k) &= f[x_0] + f[x_0, x_1](x_1 - x_0) + \dots \\
 &\quad + f[x_0, \dots, x_k](x_k - x_0) \cdots (x_k - x_{k-1}).
 \end{aligned}$$

Durch Vergleich folgt:

$$f[x_0, \dots, x_k] = y[x_0, \dots, x_k]$$

für $k = 1, \dots, n$. □

Für den Fehler der Lagrange-Interpolation können die folgenden Betrachtungen angestellt werden. In (9.5) wird für großes n der Term $\frac{1}{(n+1)!}$ sehr klein. Das Produkt $\prod_{j=0}^n (x - x_j)$ wird klein, wenn die Stützstellen sehr dicht beieinander liegen. Sind alle Ableitungen von f gleichmäßig (bzgl. der Ableitungsstufe) beschränkt auf $[a, b]$, so gilt mit (9.6), dass

$$\max_{a \leq x \leq b} |f(x) - p(x)| \rightarrow 0, \quad n \rightarrow \infty.$$

Haben die Ableitungen der zu interpolierenden Funktion jedoch ein zu starkes Wachstumsverhalten für $n \rightarrow \infty$, z. B.

$$f(x) = (1 + x^2)^{-1}, \quad |f^{(n)}(x)| \approx 2^n n! O(|x|^{-2-n}),$$

so konvergiert die Interpolation nicht gleichmäßig auf $[a, b]$.

Beispiel 9.14

Die Funktion $f(x) = |x|$, $x \in [-1, 1]$ werde mithilfe der Lagrange-Interpolation in den Stützstellen

$$x_k = -1 + kh, \quad k = 0, \dots, 2m, \quad h = 1/m, \quad x \neq x_k$$

interpoliert. Dies ergibt das globale Verhalten

$$p_m(x) \not\rightarrow f(x), \quad m \rightarrow \infty.$$

Zwar ist f in diesem Beispiel nicht differenzierbar, dennoch ist dieses Verhalten der Lagrange-Interpolation auch bei anderen Beispielen zu beobachten. Man betrachte z. B. die Funktion

$$f(x) = (1 + x^2)^{-1}, \quad x \in [-5, 5].$$



Wir fassen die bisherigen Ergebnisse zusammen:

Bemerkung 9.15 Der Approximationssatz von Weierstraß besagt, dass jede Funktion $f \in C([a, b])$ durch ein Polynom beliebig gut approximiert werden kann. Die Analysis gibt jedoch keine Hilfestellung bei der konkreten Durchführung der Approximation. Die Lagrangesche Interpolation ist eine Möglichkeit zur Approximation. Die Qualität dieser Approximation wird jedoch wesentlich durch die Regularität der Daten, also durch f bestimmt. Eine gleichmäßige Approximation von Funktionen mit Lagrangeschen Interpolationspolynomen ist im Allgemeinen nicht möglich. ♦

Die Lagrangesche Interpolation „krankt“ demnach an den gleichen Einschränkungen wie die Taylor-Entwicklung, Satz 7.5. Von der Möglichkeit, eine nur stetige Funktion $f \in C([a, b])$ beliebig gut zu approximieren, sind wir noch weit entfernt.

Ein zweiter Nachteil der Lagrange-Interpolation ist die fehlende Lokalität. Eine Störung in einer Stützstelle $(\tilde{x}_k, \tilde{y}_k)$ hat Auswirkung auf alle Lagrange-Polynome und insbesondere auf das gesamte Interpolationsintervall. Wir betrachten hierzu ein Beispiel:

Beispiel 9.16 (Globaler Fehlereinfluss)

Wir suchen das Interpolationspolynom zu der Funktion $f(x) = 0$ in den $2m + 1$ Stützstellen

$$x_k = -1 + kh, \quad k = -m, \dots, m, \quad h = \frac{1}{m}.$$

Das exakte Interpolationspolynom $p \in P_{2m}$ ist natürlich durch $p = 0$ gegeben. Wir nehmen an, dass die Funktionsauswertung im Nullpunkt gestört ist:

$$y_k = \begin{cases} 0 & k \neq 0 \\ \epsilon & k = 0 \end{cases}$$

mit einem kleinen ϵ . Das gestörte Interpolationspolynom ist in Lagrangescher Darstellung gegeben durch

$$\tilde{p}(x) = \epsilon \prod_{i=-m, i \neq 0}^m \frac{x - x_i}{x_i}.$$

In Abb. 9.1 zeigen wir $\tilde{p}_m \in P_{2m}$ für die Fälle $m = 1, 2, 4, 8$ mit einer Störung $\epsilon = 0.01$. Trotz dieser kleinen Störung an der Stelle $x = 0$ weichen die Interpolationspolynome am Intervallrand sehr stark von $y_k = 0$ ab. In der rechten Abbildung sieht man, dass für kleine Polynomgrade, also $m = 1$ und $m = 2$, der maximale Fehler auf dem Intervall nicht größer als die anfängliche Störung $\epsilon = 0.01$ ist. Die Lagrangesche Interpolation ist instabil für große Polynomgrade. ◀

Das ungünstige Verhalten der Interpolation an den Intervallrändern wird *Runges Phänomen* genannt. Es erklärt sich aus der Tatsache, dass jedes Polynom $p(x)$ für $x \rightarrow \pm\infty$ gegen $p \rightarrow \pm\infty$ strebt. Eine zu interpolierende Funktion, welche diese Eigenschaft nicht hat, etwa

$$r(x) = \frac{1}{1 + x^2},$$

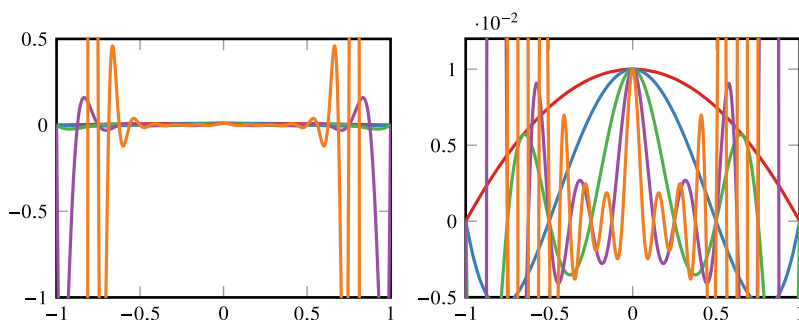


Abb. 9.1 Interpolationspolynome $\tilde{p}_m(x) \in P_{2m}$ für $m = 1, 2, 4, 8$. Links: das komplette Interpolationsintervall. Rechts: Ausschnitt nahe $y = 0$

für die $r \rightarrow 0$ für $x \rightarrow \pm\infty$, kann im Allgemeinen mit Polynomen global nur schlecht dargestellt werden.

9.1.4 Hermiteinterpolation

Zum Abschluss der Funktionsinterpolation erwähnen wir noch eine Verallgemeinerung, die sogenannte *Hermite'sche Interpolationsaufgabe*. Diese unterscheidet sich von der Lagrange-Interpolation durch die Möglichkeit, neben Funktionswerten $p(x_k) = f(x_k)$ auch Gleichheit von Ableitungswerten $p^{(i)}(x_k) = f^{(i)}(x_k)$ zu fordern. Es gilt:

Satz 9.17 (Hermiteinterpolation) Es sei $f \in C^{(n+1)}([a, b])$. Es seien $x_0, \dots, x_m$ paarweise verschiedene Stützstellen und $\mu_k \in \mathbb{N}$ für $k = 0, \dots, m$ ein Ableitungsindex. Ferner gelte $n = m + \sum_{k=0}^m \mu_k$. Das Hermite'sche Interpolationspolynom zu

$$k = 0, \dots, m : \quad p^{(i)}(x_k) = f^{(i)}(x_k), \quad i = 0, \dots, \mu_k,$$

ist eindeutig bestimmt und zu jedem $x \in [a, b]$ existiert eine Zwischenstelle $\xi \in [a, b]$, sodass gilt:

$$\begin{aligned} f(x) - p(x) &= f[\underbrace{x_0, \dots, x_0}_{\mu_0+1\text{-fach}}, \dots, \underbrace{x_m, \dots, x_m}_{\mu_m+1\text{ mal}}, x] \prod_{k=0}^m (x - x_k)^{\mu_k+1} \\ &= \frac{1}{(n+1)!} f^{(n+1)}(\xi) \prod_{k=0}^m (x - x_k)^{\mu_k+1} \end{aligned}$$

Für den Beweis sowie für weitere Details zur praktischen Berechnung verweisen wir auf die Literatur [35].

Im Fall $\mu_k = 1$ für alle k fallen Hermite- und Lagrange-Interpolation zusammen. Im Fall einer einzigen Stützstelle $m = 0$ mit $\mu_0 = n$ entspricht die Hermite'sche Interpolation genau der Taylor-Summe, d. h.,

$$f(x) = \sum_{i=0}^n \frac{(x - x_0)^i}{i!} f^{(i)}(x_0).$$

9.2 Stückweise Interpolation

Ein wesentlicher Defekt der *globalen* Interpolation aus dem vorherigen Abschnitt ist, dass die interpolierenden Polynome starke Oszillationen zwischen den Stützstellen mit immer größeren Werten annehmen. Der Grund ist die generische Steifheit, die durch die *implizite*

Forderung von C^∞ -Übergängen in den Stützstellen gegeben ist. Die Steifheit kann dadurch reduziert werden, dass die globale Funktion als *stückweise* polynomiale (Teil-) Funktionen bzgl. der Zerlegung $a = x_0 < x_1 < \dots < x_n < b$ zusammengesetzt wird. In den Stützstellen x_i werden dann geeignete Differenzierbarkeitseigenschaften gefordert.

Der Begriff *Spline* wird eine zentrale Rolle spielen und in der Literatur meist für den *kubischen Spline* verwendet, auf den wir noch eingehen werden. Allgemeiner bezeichnet ein Spline aber stückweise Interpolierende, die an den Übergangsstellen eine vorgegebene Glattheitseigenschaft besitzen. Einfache stückweise Interpolierende, etwa stückweise lineare Funktionen, haben darüber hinaus eine herausragende Bedeutung bei der Finite-Elemente-Methode zur Approximation der Lösung von Differentialgleichungen.

Das globale Intervall (wie vorher $I := [a, b]$) wird in Teilintervalle $I_i = [x_{i-1}, x_i]$ mit der Länge $h_i = x_i - x_{i-1}$ unterteilt. Die Feinheit der gesamten Intervallunterteilung wird durch $h := \max_{i=1, \dots, n} h_i$ charakterisiert, siehe Abb. 9.2. Zur Definition der Spline-Funktion (kurz Spline) seien die Vektorräume von stückweisen polynomialen Funktionen wie folgt gegeben:

$$S_h^{k,r}[a, b] := \{s \in C^r[a, b], s|_{I_i} \in P_k(I_i)\}.$$

Dabei ist $k \in \mathbb{N}$ der lokale Polynomgrad und $r < k$ die globale Regularität der Funktionen. Zu einem Satz gegebener Stützpunkte (die wie in Abschn. 9.1 aus gegebenen Daten- oder Funktionswerten stammen können) in dem Gesamtintervall I wird eine Interpolierende $p \in S_h^{k,r}[a, b]$ mithilfe von geeigneten Interpolationsbedingungen bestimmt.

Wir diskutieren nun einige Beispiele, wobei der Fokus auf der Idee des ganzen Prozesses liegt und weniger auf der Beweisführung bei Fragen zu Existenz, Eindeutigkeit und Fehlerabschätzung.

Beispiel 9.18 (Stückweise lineare Interpolation)

In Abb. 9.3 zeigen wir die stückweise lineare Lagrange-Interpolierende (d.h. $k = 1, r = 0$) zur Approximation einer gegebenen Funktion f auf $[a, b]$ durch einen Polygonzug in den Stützstellen $x_i, i = 0, \dots, n$:

$$s \in S_h^{1,0}[a, b] = \{s \in C[a, b], s|_{I_i} \in P_1(I_i)\}$$

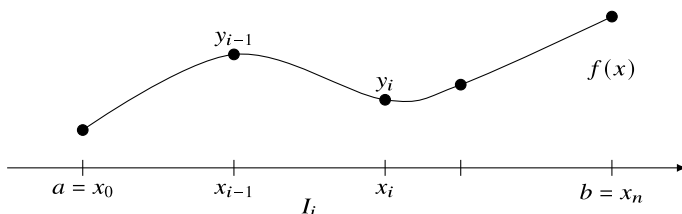


Abb. 9.2 Stückweise Interpolation

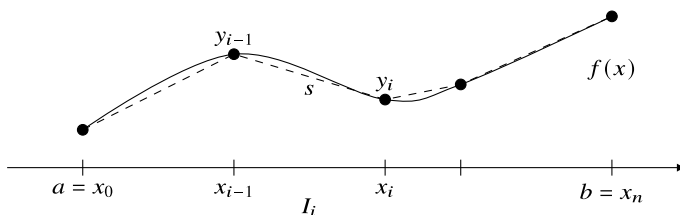


Abb. 9.3 Stückweise lineare Interpolation s einer Funktion f (gestrichelt)

mit den Interpolationsbedingungen

$$s(x_i) = f(x_i), \quad i = 0, \dots, n.$$

Die Anwendung der Fehlerabschätzung für die Lagrange-Interpolation separat auf jedem I_i liefert die globale Fehlerabschätzung

$$\max_{x \in [a, b]} |f(x) - p(x)| \leq \frac{1}{2} h^2 \max_{x \in [a, b]} |f''(x)|. \quad (9.7)$$

Für kleine Schrittweiten gilt

$$\max_{x \in [a, b]} |f(x) - s(x)| \rightarrow 0 \quad (h \rightarrow 0)$$

gleichmäßig auf dem gesamten Intervall. ◀

Die Interpolierende des vorangegangenen Beispiels wird mithilfe der sogenannten Knotenbasis von $S_h^{1,0}([a, b])$ konstruiert. Diese Knotenbasis besteht aus den *Hutfunktionen* $\phi_i \in S_h^{1,0}[a, b]$, $i = 0, \dots, n$, die durch die Bedingung

$$\phi_i(x_j) = \delta_{ij}$$

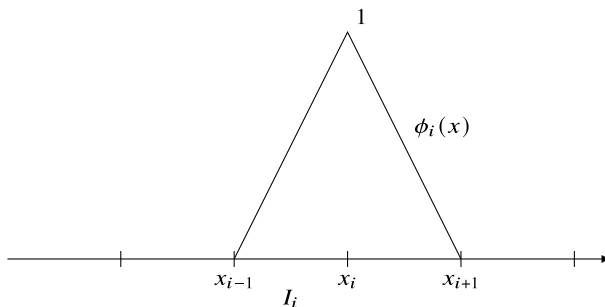
eindeutig bestimmt sind. Die Interpolierende s von f erlaubt dann die Darstellung in Form von

$$s(x) = \sum_{i=0}^n f(x_i) \phi_i(x).$$

Diese Konstruktion stellt die Analogie zur Lagrangeschen Darstellung des Lagrange-Interpolationspolynoms her. Im Gegensatz zu dieser *globalen* Sichtweise arbeiten wir aber bei den Splines *lokal*, da die Hutfunktionen ϕ_i nur in den direkt an der jeweiligen Stützstelle x_i angrenzenden Teilintervallen von null verschieden ist (siehe Abb. 9.4).

Erhöht man jedoch den lokalen Polynomgrad auf jedem Teilintervall, so kann mithilfe der Interpolationsbedingungen höhere Glattheit auch global erzielt werden. Dies führt auf die kubischen Splines, d. h. $k = 3$.

Abb. 9.4 Lineare Knoten-
basisfunktion, auch
Hutfunktion genannt



Beispiel 9.19 (Stückweise kubische Interpolation)

Es sei auf jedem Teilintervall I_i ein kubisches Polynom vorgeschrieben, d. h. $k = 3$ mit $r = 0$, sodass $S_h^{3,0}[a, b]$. Die Interpolationsbedingungen für den Fall $r = 0$ (d. h. globale Stetigkeit) lauten

$$s(x_i) = f(x_i).$$

Zur eindeutigen Bestimmung eines kubischen Polynoms sind in jedem I_i vier Bedingungen notwendig, sodass zwei zusätzliche Interpolationspunkte vorgegeben werden

$$s(x_{ij}) = f(x_{ij}),$$

wobei $x_{ij} \in I_i$, $i = 1, \dots, n$, $j = 1, 2$. Durch diese stückweise kubische Lagrange-Interpolation ist eindeutig eine global stetige Funktion $s \in S_h^{3,0}[a, b]$ festgelegt. ◀

Anstatt die Interpolationsbedingung durch Zwischenwerte $x_{ij} \in (x_{i-1}, x_i)$ anzureichern, ist es auch möglich, Ableitungswerte vorzugeben:

Beispiel 9.20 (Stückweise kubische Hermiteinterpolation)

Es sei wiederum auf jedem Teilintervall I_i ein kubisches Polynom vorgeschrieben, d. h. $k = 3$, und dieses Mal mit $r = 1$, sodass $S_h^{3,1}[a, b]$. Die Interpolationsbedingungen für den Fall $r = 1$ (d. h. globale Stetigkeit und einmalige globale Differenzierbarkeit) lauten

$$s(x_i) = f(x_i), \quad s'(x_i) = f'(x_i).$$

Durch diese vier Bedingungen ist $s \in S_h^{3,1}[a, b]$ eindeutig festgelegt. ◀

Satz 9.21 (Stückweise kubische Interpolation) Für die stückweise kubische Interpolation s (d. h. $k = 3$) mit $r = 0$ oder $r = 1$ zur Approximation der Funktion f gilt die globale Fehlerabschätzung

$$\max_{x \in [a, b]} |f(x) - s(x)| \leq \frac{1}{4!} h^4 \max_{x \in [a, b]} |f^{(4)}(x)|.$$

Beweis Auf jedem Intervall $I_i = [x_{i-1}, x_i]$ mit $h_i = x_i - x_{i-1}$ kann die Abschätzung für die Lagrange- oder Hermiteinterpolation angewendet werden:

$$f(x) - s(x) \Big|_{I_i} \leq \frac{\max_{I_i} f^{(4)}}{4!} h_i^4 \quad \square$$

Zur eindeutigen Bestimmung einer stückweise kubischen Funktion sind auf jedem Intervall I_i vier Bedingungen erforderlich. Dennoch unterscheiden sich die Sätze an Bedingungen in den beiden betrachteten Beispielen. Bei der stückweise kubischen Lagrange-Interpolation in Beispiel 9.19 werden in den n Intervallen jeweils vier Bedingungen $s(x_{ij}) = f(x_{ij})$ gestellt, wobei an den Intervallenden x_i die gleiche Interpolationsbedingung doppelt auftaucht. Insgesamt ist die stückweise kubische Lagrange-Interpolation durch $3n + 1$ Bedingungen gegeben. Die stückweise Hermiteinterpolation in Beispiel 9.20 hat auch vier Bedingungen pro Intervall. An den Intervallenden treten nun jedoch zwei Bedingungen doppelt auf, sodass sich global $2n + 2$ Bedingungen ergeben.

Soll die globale Regularität der Interpolation weiter gesteigert werden, so schlägt der triviale Ansatz, zusätzlich $s''(x_i) = f''(x_i)$ zu fordern, fehl, da dies zu sechs Bedingungen pro Teilintervall führen würde (bei nur vier Unbekannten eines kubischen Polynoms). Anstatt Funktionswerte für Ableitung und zweite Ableitung in jeder Stützstelle x_i vorzuschreiben, fordern wir lediglich die Stetigkeit der Ableitungen:

$$\lim_{h \downarrow 0} s'(x_i + h) = \lim_{h \downarrow 0} s'(x_i - h), \quad \lim_{h \downarrow 0} s''(x_i + h) = \lim_{h \downarrow 0} s''(x_i - h).$$

Wir zählen die Bedingungen und Unbekannten. Der Raum $S_h^{(3,2)}$ hat auf jedem der n Intervalle I_i für $i = 1, \dots, n$ vier unbekannte Parameter

$$s(x) \Big|_{I_i} = \alpha_0^{(i)} + \alpha_1^{(i)} x + \alpha_2^{(i)} x^2 + \alpha_3^{(i)} x^3.$$

Die Interpolationsbedingung $s(x_i) = y_i$ stellt insgesamt $n + 1$ Bedingungen. Durch die geforderte globale Stetigkeit von Funktionswert, Ableitung und zweiter Ableitung kommen auf jedem der $n - 1$ Intervallübergänge weitere drei Bedingungen hinzu. So stehen den $4n$ unbekannten Parametern $4n - 2$ Bedingungen gegenüber. Das Problem wird geschlossen durch die Vorgabe von weiteren Randbedingungen in den Punkten a und b . Durch Vorgabe von

$$s''(a) = s''(b) = 0$$

wird der *natürliche kubische Spline* definiert.

Die Übersetzung von *Spline* ins Deutsche bedeutet *Straklatte* oder *Biegestab*, was eine elastische Latte aus Holz oder Kunststoff bezeichnet, die traditionell im Schiffsbau zum Entwurf und Bau verwendet wurde. Man stelle sich einen elastischen Stab vor, der an

gewissen Punkten festgehalten wird, dazwischen jedoch eine freie Form annehmen darf. Nach physikalischen Grundprinzipien wird diese Form die Energie des Stabes minimieren. Die Energie ist gerade durch die Krümmung des Stabes gegeben. Der natürliche Spline kann als die energieminimierende Funktion beschrieben werden, welche global zweimal stetig differenzierbar ist und in den Stützstellen die Interpolationseigenschaft erfüllt:

$$\min_{s \in S[a,b]} E(s)$$

mit

$$E(s) := \int_a^b |s''(x)|^2 dx, \quad S := \{\phi \in S_h^{3,2}, \phi(x_i) = y_i \text{ für } i = 0, \dots, n\}.$$

Diese Form des Splines ist die am häufigsten genutzte Spline-Interpolationsaufgabe und hat z. B. Anwendungen in der Computergrafik.

► **Definition 9.22 (Kubischer Spline)** Eine Funktion $s : [a, b] \rightarrow \mathbb{R}$ wird *kubischer Spline* bzgl. der Zerlegung $a = x_0 < x_1 < \dots < x_n = b$ genannt, wenn gilt:

- i) $s \in C^2[a, b]$
- ii) $s|_{[x_{i-1}, x_i]} \in P_3, \quad i = 1, \dots, n.$

Falls zusätzlich

- iii) $s''(a) = s''(b) = 0$

gilt, so heißt der Spline *natürlich*.

Satz 9.23 (Natürliche kubische Splines) Es seien $x_0, \dots, x_n \in \mathbb{R}$ paarweise verschiedene Stützstellen und $y_i \in \mathbb{R}$ gegeben. Der durch die Interpolationsvorschrift

$$s(x_i) = y_i \in \mathbb{R}, \quad i = 0, \dots, n \tag{9.8}$$

beschriebene natürliche kubische Spline $s \in S_h^{3,2}$ existiert eindeutig.

Für jede Funktion $g \in S_h^{3,2}$ mit $g(x_k) = y_k$ für $k = 0, \dots, n$ gilt

$$\int_a^b |s''(x)|^2 dx \leq \int_a^b |g''(x)|^2 dx.$$

Im Fall $y_k = f(x_k)$ für eine Funktion $f \in C^4[a, b]$ gilt die Fehlerabschätzung

$$\max_{a \leq x \leq b} |f(x) - s(x)| \leq \frac{1}{2} h^4 \max_{a \leq x \leq b} |f^{(4)}(x)|.$$

Beweis (i) *Bestimmung des Splines*: Wir zeigen, dass der Spline als Lösung eines linearen Gleichungssystems gegeben ist. Auf jedem Intervall $I_i := [x_{i-1}, x_i]$ schreiben wir

$$s(x) \Big|_{I_i} = a_0^{(i)} + a_1^{(i)}(x - x_i) + a_2^{(i)}(x - x_i)^2 + a_3^{(i)}(x - x_i)^3, \quad i = 1, \dots, n.$$

Es gilt die $4n$ Koeffizienten $a_j^{(i)}$ zu bestimmen. Der erste Koeffizient kann unmittelbar als $a_0^{(i)} = y_i$ abgelesen werden:

$$y_i = s(x_i) = a_0^{(i)}, \quad i = 1, \dots, n$$

Es bleiben für die $3n$ unbekannten Koeffizienten die Bedingungen

$$\begin{array}{lll} s(x_{i-1}) = y_{i-1} & y_{i-1} = a_0^{(i)} - a_1^{(i)}h_i + a_2^{(i)}h_i^2 - a_3^{(i)}h_i^3 & i = 1, \dots, n \\ s'(x_i) \Big|_{I_i} = s'(x_i) \Big|_{I_{i+1}} & a_1^{(i)} = a_1^{(i+1)} - 2a_2^{(i+1)}h_i + 3a_3^{(i+1)}h_i^2 & i = 1, \dots, n-1 \\ s''(x_i) \Big|_{I_i} = s''(x_i) \Big|_{I_{i+1}} & 2a_2^{(i)} = 2a_2^{(i+1)} - 6a_3^{(i+1)}h_i & i = 1, \dots, n-1 \\ s''(x_0) = 0 & 0 = a_2^{(1)} - 2a_3^{(1)}h_1 & i = 0 \\ s''(x_n) = 0 & 0 = a_2^{(n)} & i = n, \end{array}$$

d.h. $n + 2(n-1) + 2 = 3n$ Gleichungen. Bei entsprechender Nummerierung kann das LGS als Bandmatrix mit Bandbreite 3 geschrieben werden. Zur effizienten Bestimmung des Splines kann das Gleichungssystem unter Reformulierung der Bedingungen weiter reduziert werden, sodass sich ein Tridiagonalsystem ergibt.

(ii) *Existenz und Eindeutigkeit*: Da der Spline als Lösung eines LGS bestimmt ist, genügt es, die Eindeutigkeit zu zeigen. Wir betrachten hierzu den Raum

$$N := \{\phi \in S_h^{3,2}, \phi(x_i) = 0, i = 0, \dots, n\}.$$

Auf diesem Raum ist durch

$$(u, v)_N := \int_a^b u''(x)v''(x) dx$$

ein Skalarprodukt gegeben. Linearität, Symmetrie und Positivität sind klar. Die Homogenität folgt, da aus $(v, v)_N = 0$ sofort punktweise $v'' = 0$ folgt. Das heißt, v ist eine lineare Funktion. Aus $v(a) = v(b) = 0$ folgt dann $v = 0$.

Es seien jetzt s_1, s_2 zwei Splines zu den Stützstellen und Werten $\{(x_i, y_i), i = 0, \dots, n\}$. Dann ist $s = s_1 - s_2 \in N$ und es gilt für $w \in N$ mit partieller Integration (beachte, dass $s \in C^\infty(I_i)$, da Polynom)

$$\begin{aligned}
(s, w)_N &= \int_a^b s'' w'' \, dx = \sum_{i=1}^N \left\{ s'' w' \Big|_{x_{i-1}}^{x_i} - \int_{I_i} s'''(x) w'(x) \, dx \right\} \\
&= \sum_{i=1}^N \left\{ s'' w' \Big|_{x_{i-1}}^{x_i} - \underbrace{s'''}_{=0} \underbrace{w}_{=0} \Big|_{x_{i-1}}^{x_i} + \int_{I_i} \underbrace{s^{(4)}(x)}_{=0} w(x) \, dx \right\} \\
&= \sum_{i=1}^n \left\{ s''(x_i) w'(x_i) - s''(x_{i-1}) w'(x_{i-1}) \right\} \\
&= s''(b) w'(b) - s''(a) w'(a) = 0,
\end{aligned} \tag{9.9}$$

da $s''(a) = s''_1(a) - s''_2(a) = 0$. Diese Argumentation gilt analog für b . Hieraus folgt

$$(s, s)_N = 0 \quad \Rightarrow \quad s'' = 0 \quad \Rightarrow \quad s = 0$$

und also die Eindeutigkeit $s_1 = s_2$ und somit auch die Existenz einer Lösung.

(iii) *Energieminimierung*: Für $w \in N$ gilt der Zusammenhang

$$(s, w)_N = 0$$

nicht nur für $s \in N$, sondern für alle natürlichen Splines ($s''(a) = s''(b) = 0$), welche die beschreibende Bedingung (9.8) erfüllen. Man vergleiche hierzu die Argumente in (9.9). Das heißt, der Raum N steht orthogonal auf dem Spline s . Es sei nun $f \in S_h^{3,2}$ mit $y_i = f(x_i)$ für $i = 0, \dots, n$. Dann gilt $w := s - f \in N$ und

$$0 = (s, s - f)_N = (s, s)_N - (s, f)_N,$$

also

$$\int_a^b s''(x)^2 \, dx = \int_a^b s''(x) f''(x) \, dx \leq \left(\int_a^b s''(x)^2 \, dx \right)^{\frac{1}{2}} \left(\int_a^b f''(x)^2 \, dx \right)^{\frac{1}{2}},$$

woraus die Energieminimierung folgt.

(iv) *Fehlerabschätzung*: Der Nachweis der Fehlerabschätzung ist aufwendig und wir verweisen für den Beweis auf die Literatur [80]. $\square$

Die Minimierung der zweiten Ableitung lässt sich einerseits als physikalisches Prinzip verwenden. Für die Numerik ist jedoch relevant, dass durch die Minimierung der zweiten Ableitungen auch Oszillationen unterbunden werden. Der natürliche Spline hat sehr gute Stabilitätseigenschaften. Zur Berechnung des Splines ist ein lineares Gleichungssystem zu lösen. Die Gleichungen sind im Beweis zu Satz 9.23 angegeben. Dabei können die Werte $a_0^{(i)} = y_i$ sofort angegeben werden. Weiter können in einem Vorbereitungsschritt die Unbekannten $a_1^{(i)}$ sowie $a_3^{(i)}$ durch die $a_2^{(i)}$ ausgedrückt werden. Somit bleibt ein lineares

Gleichungssystem in den n Unbekannten $a_2^{(1)}, \dots, a_2^{(n)}$. Die Matrix hat dabei eine Tridiagonalgestalt und kann sehr effizient gelöst werden.

9.3 Numerische Differentiation

In diesem Abschnitt befassen wir uns mit der folgenden numerischen Aufgabe: Zu einer gegebenen Funktion $f : [a, b] \rightarrow \mathbb{R}$ soll in einem Punkt $x_0 \in (a, b)$ die Ableitung $f'(x_0)$ oder die n -te Ableitung $f^{(n)}(x_0)$ berechnet werden. Wir gehen davon aus, dass es mit vertretbarem Aufwand nicht möglich ist, die Funktion $f(x)$ symbolisch zu differenzieren und die Ableitung an der Stelle x_0 auszuwerten. Wir benötigen also Approximationsverfahren zur Bestimmung der Ableitung. Die grundlegende Idee wird durch das folgende Verfahren beschrieben:

Algorithmus 9.2: Numerische Differentiation

Eingabe: Gegeben sei eine Funktion $f \in C(a, b)$.

- 1 Interpoliere $f(x)$ durch Polynom $p \in P_m$ mit $m \geq n$
- 2 Berechne $p^{(n)}(x_0)$

Ergebnis: Approximation $f^{(n)}(x_0) \approx p^{(n)}(x_0)$

Im Folgenden entwickeln wir einige einfache Verfahren zur Approximation von Ableitungen, welche auf der Interpolation beruhen.

9.3.1 Approximation der ersten Ableitung

Wir interpolieren eine Funktion $f(x)$ linear in den beiden Stützstellen x_0 und x_1

$$p_1(x) = \frac{x - x_1}{x_0 - x_1} f(x_0) + \frac{x - x_0}{x_1 - x_0} f(x_1)$$

und approximieren die Ableitung $f'(x)$ durch die entsprechende Ableitung des Polynoms

$$p_1'(x) = \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

Wir stellen uns die Frage, wie gut $p_1'(x)$ die wahre Ableitung $f'(x)$ approximiert. Für das lineare Interpolationspolynom ist die Ableitung eine konstante Funktion. Wir erhalten somit für alle Werte x die gleiche Approximation an die Ableitung. Es gilt:

Satz 9.24 (Differenzenquotient der ersten Ableitung) Es sei $f \in C^2[a, b]$. Für beliebige Punkte $x_0, x_1 \in [a, b]$ mit $h := x_1 - x_0 > 0$ gelten die *einseitigen Differenzenquotienten* (rechtsseitig und linksseitig)

$$\begin{aligned}\frac{f(x_1) - f(x_0)}{h} &= f'(x_0) + \frac{1}{2}hf''(x_0) + O(h^2), \\ \frac{f(x_1) - f(x_0)}{h} &= f'(x_1) - \frac{1}{2}hf''(x_0) + O(h^2).\end{aligned}$$

Im Fall $f \in C^4[a, b]$ gilt für $x_0, x_0 - h, x_0 + h \in [a, b]$ für den *zentralen Differenzenquotient*

$$\frac{f(x_0 + h) - f(x_0 - h)}{2h} = f'(x_0) + \frac{1}{6}h^2f'''(x_0) + O(h^4).$$

Beweis (i) Mit Taylor-Entwicklung gilt

$$f(x_0 \pm h) = f(x_0) \pm hf'(x_0) + \frac{1}{2}h^2f''(x_0) \pm \frac{1}{6}h^3f'''(x_0) + \frac{1}{24}h^4f^{(4)}(\xi)$$

mit einem $\xi \in [a, b]$. Hieraus folgt z. B. für den rechtsseitigen Differenzenquotienten

$$\begin{aligned}\frac{f(x_0 + h) - f(x_0)}{h} &= \frac{hf'(x_0) + \frac{1}{2}h^2f''(x_0) + O(h^3)}{h} \\ &= f'(x_0) + \frac{1}{2}hf''(x_0) + O(h^2).\end{aligned}$$

Für den linksseitigen Differenzenquotienten folgt das Ergebnis entsprechend.

(ii) Für den zentralen Differenzenquotienten heben sich in der Taylor-Entwicklung alle geraden Potenzen von h aus Symmetriegründen auf:

$$f(x_0 + h) - f(x_0 - h) = 2hf'(x_0) + \frac{1}{3}h^3f'''(x_0) + O(h^5)$$

Hieraus folgt das gewünschte Ergebnis. □

Bei der Berechnung der Approximation der Ableitung mithilfe der Polynominterpolation kommt es somit auf die Auswertungsstelle an. Numerische Verfahren, die in einzelnen Punkten – oder in speziellen Situationen – besser konvergieren als im allgemeinen Fall, werden *superkonvergent* genannt. Bei der Berechnung des zentralen Differenzenquotienten liegt ein superkonvergentes Verhalten vor. Es kommt hier entscheidend auf die Symmetrie an. Angenommen, die Ableitung in x_0 wird durch die lineare Interpolation zwischen $x_0 - h$ und $x_0 + 2h$ approximiert, so gilt mit Taylor-Entwicklung um x_0

$$\begin{aligned}\frac{f(x_0 + 2h) - f(x_0 - h)}{3h} \\ &= f'(x_0) + \left(2hf''(x_0) + \frac{8}{6}h^2f'''(x_0)\right) - \left(\frac{1}{2}hf''(x_0) - \frac{1}{6}h^2f'''(x_0)\right) + O(h^3) \\ &= f'(x_0) + \frac{3}{2}hf''(x_0) + O(h^2).\end{aligned}$$

Terme höherer Ordnung in h heben sich nicht gegenseitig auf und der zentrale, aber unsymmetrische Differenzenquotient (welcher ja eigentlich nicht mehr zentral ist) konvergiert nur von erster Ordnung.

9.3.2 Approximation der zweiten Ableitung

Wir interpolieren $f(x)$ nun mithilfe der drei Stützstellen $x_0 - h$, x_0 , $x_0 + h$ durch ein quadratisches Polynom:

$$p_2(x) = f(x_0 - h) \frac{(x_0 - x)(x_0 + h - x)}{2h^2} + f(x_0) \frac{(x - x_0 + h)(x_0 + h - x)}{h^2} + f(x_0 + h) \frac{(x - x_0 + h)(x - x_0)}{2h^2}$$

In $x = x_0$ gilt für erste und zweite Ableitungen:

$$p_2'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h}, \quad p_2''(x_0) = \frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h)}{h^2}$$

Für die erste Ableitung ergibt sich erstaunlicherweise wieder der zentrale Differenzenquotient, der schon durch lineare Interpolation hergeleitet wurde. Die Ordnung muss also wirklich von Fall zu Fall nachgewiesen werden. Mithilfe der linearen Interpolierenden wird ein Ergebnis erreicht, das eigentlich erst bei quadratischer Interpolierender zu erwarten wäre. Dieses bessere Ergebnis wird jedoch nur bei Abgreifen der Ableitung im Mittelpunkt erreicht.

Für die zweite Ableitung erhalten wir mit Taylor-Entwicklung

$$p_2''(x_0) = \frac{-2f(x_0) + f(x_0 - h) + f(x_0 + h)}{h^2} = f''(x_0) + \frac{1}{12}h^2 f^{(4)}(x_0) + O(h^4)$$

den zentralen Differenzenquotient für die zweite Ableitung.

Wir können auf der Basis von p_2 auch einen einseitigen Differenzenquotienten für die zweite Ableitung herleiten. Dies erfolgt durch Approximation von $f''(x_0 - h) \approx p_2''(x_0 - h)$. Wieder mit Taylor-Entwicklung erhalten wir mit

$$p_2''(x_0 - h) = f''(x_0 - h) + hf'''(x_0 - h) + O(h^2)$$

lediglich eine Approximation erster Ordnung. Neben der Ordnung des Interpolationspolynoms $p(x)$ kommt es entscheidend auf die entsprechende Wahl der Stützstellen an.

Wir betrachten ein Beispiel:

Beispiel 9.25

Es sei

$$f(x) = \tanh(x).$$

Tab. 9.1 Differenzenapproximation von $f'(1/2)$ (zwei Tabellen links) und $f''(1/2)$ (rechts) der Funktion $f(x) = \tanh(x)$. Dabei ist jeweils der einseitige bzw. der zentrale Differenzenquotient genutzt worden

h	$\frac{f(\frac{1}{2}+h)-f(\frac{1}{2})}{h}$	$\frac{f(\frac{1}{2}+h)-f(\frac{1}{2}-h)}{2h}$	$\frac{f(\frac{1}{2}+2h)-2f(\frac{1}{2}+h)+f(\frac{1}{2})}{h^2}$	$\frac{f(\frac{1}{2}+h)-2f(\frac{1}{2})+f(\frac{1}{2}-h)}{h^2}$
$\frac{1}{2}$	<u>0.598954</u>	<u>0.761594</u>	-0.623692	-0.650561
$\frac{1}{4}$	<u>0.692127</u>	<u>0.780461</u>	-0.745385	-0.706667
$\frac{1}{8}$	<u>0.739861</u>	<u>0.784969</u>	-0.763733	-0.721740
$\frac{1}{16}$	<u>0.763405</u>	<u>0.786079</u>	-0.753425	-0.725577
$\frac{1}{32}$	<u>0.775004</u>	<u>0.786356</u>	-0.742301	-0.726540
$\frac{1}{64}$	<u>0.780747</u>	<u>0.786425</u>	-0.735134	-0.726782
Exakt	0.786448	0.786448	-0.726862	-0.726862

Wir suchen eine Approximation von

$$f'\left(\frac{1}{2}\right) \approx 0.7864477329659274, \quad f''\left(\frac{1}{2}\right) \approx -0.7268619813835874.$$

Zur Approximation verwenden wir für die vier bisher diskutierten Differenzenquotienten zu verschiedenen Schrittweiten $h > 0$, siehe Tab. 9.1.

In Abb. 9.5 tragen wir die Fehler der Approximationen gegenüber der Schrittweite auf. Hier ist deutlich der Unterschied zwischen linearer und quadratischer Ordnung in h zu erkennen. ◀

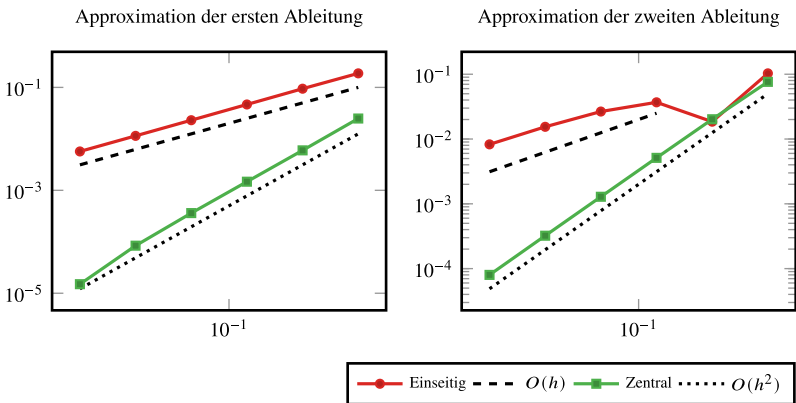


Abb. 9.5 Fehler bei der Differenzenapproximation der ersten (links) und zweiten (rechts) Ableitung von $f(x) = \tanh(x)$ im Punkt $x_0 = \frac{1}{2}$

9.3.3 Stabilität der numerischen Differentiation

Abschließend untersuchen wir die Stabilität der Differenzenapproximation. Die Koeffizienten der verschiedenen Formeln wechseln das Vorzeichen, somit besteht die Gefahr der Auslöschung. Exemplarisch führen wir die Stabilitätsanalyse für die zentrale Differenzenapproximation zur Bestimmung der ersten Ableitung durch. Wir gehen davon aus, dass die Funktionswerte an den beiden Stützstellen nur fehlerhaft (mit relativem Fehler $|\epsilon_i| \leq \epsilon$) ausgewertet werden können, und erhalten

$$\begin{aligned}\tilde{p}'(x_0) &= \frac{f(x_0 + h)(1 + \epsilon_1) - f(x_0 - h)(1 + \epsilon_2)}{2h} (1 + \epsilon_3) \\ &= \frac{f(x_0 + h) - f(x_0 - h)}{2h} (1 + \epsilon_3) + \frac{\epsilon_1 f(x_0 + h) - \epsilon_2 f(x_0 - h)}{2h} + O(\epsilon^2).\end{aligned}$$

Für den relativen Fehler gilt

$$\left| \frac{\tilde{p}'(x_0) - p'(x_0)}{p'(x_0)} \right| \leq \epsilon + \frac{|f(x_0 + h)| + |f(x_0 - h)|}{|f(x_0 + h) - f(x_0 - h)|} \epsilon + O(\epsilon^2).$$

Im Fall $f(x_0 + h) \approx f(x_0 - h)$, also $f'(x_0) \approx 0$, kann der Fehler beliebig stark verstärkt werden. Je kleiner h gewählt wird, umso größer wird dieser Effekt, denn zunächst gilt

$$f(x_0 + h) - f(x_0 - h) = 2hf'(x_0) + O(h^3).$$

Dieses Ergebnis ist gegenläufig zur Fehlerabschätzung für den Differenzenquotienten in Satz 9.24. Bei der numerischen Approximation müssen beide Fehleranteile addiert werden:

$$\begin{aligned}\frac{|f'(x_0) - \tilde{p}'(x_0)|}{|f'(x_0)|} &\leq \frac{|f'(x_0) - p'(x_0)|}{|f'(x_0)|} + \frac{|p'(x_0) - \tilde{p}'(x_0)|}{|f'(x_0)|} \\ &= \frac{|f'(x_0) - p'(x_0)|}{|f'(x_0)|} + \frac{|p'(x_0) - \tilde{p}'(x_0)|}{|p'(x_0)|} \cdot \frac{|p'(x_0)|}{|f'(x_0)|} \\ &\leq h^2 \frac{|f'''(x_0)|}{6|f'(x_0)|} + O(h^4) + \frac{\max_{\xi} |f(\xi)|}{|f'(x_0)|h} \epsilon + O(\epsilon^2)\end{aligned}$$

Für kleines h steigt der Rundungsfehleranteil. Der Gesamtfehler wird minimal, falls beide Fehleranteile balanciert sind, also im Fall

$$h^2 \approx \frac{\epsilon}{h} \quad \Rightarrow \quad h \approx \sqrt[3]{\epsilon}.$$

Für $\epsilon \approx 10^{-16}$ sind Schrittweiten $h < 10^{-5}$ demnach nicht sinnvoll. Im Allgemeinen gilt, dass die numerische Differentiation weniger stabil ist, als die numerische Integration.

9.4 Richardson-Extrapolation zum Limes

Eine wichtige Anwendung der Interpolation ist die *Extrapolation zum Limes*. Die Idee lässt sich am einfachsten anhand eines Beispiels erklären. Wir wollen die Ableitung $f'(x_0)$ einer Funktion f im Punkt x_0 mithilfe des einseitigen Differenzenquotienten berechnen:

$$a(h) := \frac{f(x_0 + h) - f(x_0)}{h}$$

Der Schrittweitenparameter h bestimmt die Qualität der Approximation $a(h) \approx f'(x_0)$. Für $h \rightarrow 0$ gilt (bei Vernachlässigung von Rundungsfehlern) $a(h) \rightarrow f'(x_0)$. An der Stelle $h = 0$ lässt sich $a(h)$ jedoch nicht auswerten. Die Idee der Extrapolation zum Limes ist es nun, ein Interpolationspolynom $p(x)$ durch die Stützstellenpaare $(h_i, a(h_i))$ für eine Folge von Schrittweiten $h_0, h_1, \dots, h_n$ zu legen und den Wert $p(0)$ als Approximation für $a(0)$ zu verwenden.

Es stellt sich die grundlegende Frage, ob die Interpolierende in den Stützstellen $h_0, \dots, h_n$ auch Aussagekraft außerhalb des Intervalls $I := [\min_i \{h_i\}, \max_i \{h_i\}]$ hat. Beispiel 9.16 lässt dies zunächst nicht vermuten. Hier war die Approximationseigenschaft durch Oszillationen in Punkten zwischen den Stützstellen schon am Rande des Intervalls I stark gestört. Wir betrachten dennoch ein einfaches Beispiel:

Beispiel 9.26 (Extrapolation des einseitigen Differenzenquotienten)

Es sei $f(x) = \tanh(x)$ und wir wollen die Ableitung an der Stelle $x_0 = 1/2$ auswerten. Der exakte Wert ist $f'(1/2) \approx 0.786448$. Hierzu definieren wir den Differenzenquotienten

$$a(h) = \frac{\tanh(x + h) - \tanh(x)}{h}$$

und werten zu den Schrittweiten $h = 2^{-1}, 2^{-2}, 2^{-3}, 2^{-4}$ aus:

h	$a(h)$
2^{-1}	0.5989
2^{-2}	0.6921
2^{-3}	0.7399
2^{-4}	0.7634

Wir können ein Interpolationspolynom $p(h)$ durch die Punkte $h_i = 2^{-i}$ und $a_i = a(h_i)$ legen und dieses an der Stelle $h = 0$ auswerten. Dies geschieht effizient mit dem Neville-Schema zum Punkt $h = 0$, welches wir noch einmal kurz angeben:

$$a_{k,k} = a_k, \quad k = 1, \dots, n,$$

$$a_{k,k+l} = a_{k,k+l-1} - h_k \frac{a_{k+1,k+l} - a_{k,k+l-1}}{h_{k+l} - h_k}, \quad l = 1, \dots, n-1, \quad k = 1, \dots, n-l$$

h_i	$a_i = p_{ii}$	$p_{i,i+1}$	$p_{i,i+2}$	$p_{i,i+3}$
2^{-1}	0.5989	<u>0.7853</u>	<u>0.78836</u>	<u>0.7865</u>
2^{-2}	0.6921	<u>0.7976</u>	<u>0.78673</u>	
2^{-3}	<u>0.7399</u>	<u>0.7869</u>		
2^{-4}	<u>0.7634</u>			

Die jeweils exakten Stellen sind unterstrichen. Durch Extrapolation kann die Genauigkeit also wirklich verbessert werden. ◀

Wir wollen dieses Beispiel nun für den einfachsten Fall einer linearen Interpolation untersuchen. Zu $f \in C^3([a, b])$ sei

$$a(h) = \frac{f(x_0 + h) - f(x_0)}{h} = f'(x_0) + \frac{h}{2} f''(x_0) + \frac{h^2}{6} f'''(\xi_{x_0, h}) \quad (9.10)$$

die einseitige Approximation der ersten Ableitung. Wir legen durch die Stützstellen $(h, a(h))$ sowie $(h/2, a(h/2))$ das lineare Interpolationspolynom

$$p(t) = \left(\frac{t - \frac{h}{2}}{h - \frac{h}{2}} \right) a(h) + \left(\frac{t - h}{\frac{h}{2} - h} \right) a\left(\frac{h}{2}\right)$$

und werten dieses an der Stelle $t = 0$ als Approximation von $a(0)$ aus:

$$a(0) \approx p(0) = 2a\left(\frac{h}{2}\right) - a(h)$$

Für $a(h/2)$ sowie $a(h)$ setzen wir die Taylor-Entwicklung (9.10) ein und erhalten mit

$$\begin{aligned} p(0) &= 2 \left(f'(x_0) + \frac{h}{4} f''(x_0) + \frac{h^2}{24} f'''(\xi_{x_0, h/2}) \right) \\ &\quad - \left(f'(x_0) + \frac{h}{2} f''(x_0) + \frac{h^2}{6} f'''(\xi_{x_0, h}) \right) \\ &= f'(x_0) + O(h^2) \end{aligned}$$

eine Approximation der ersten Ableitung von zweiter Ordnung in der Schrittweite h .

Durch Extrapolation können Verfahren höherer Ordnung generiert werden. Eine solche Vorgehensweise, bei der Ergebnisse eines numerischen Verfahrens weiter verarbeitet werden, wird *Postprocessing* genannt.

Satz 9.27 (Einfache Extrapolation) Es sei $a(h) : \mathbb{R}_+ \rightarrow \mathbb{R}$ eine $(n + 1)$ -mal stetig differenzierbare Funktion mit der Summenentwicklung

$$a(h) = a_0 + \sum_{j=1}^n a_j h^j + a_{n+1}(h) h^{n+1}$$

mit Koeffizienten $a_j \in \mathbb{R}$ sowie $a_{n+1}(h) = a_{n+1} + o(1)$. Weiter sei $(h_k)_{k=0,1,\dots}$ mit $h_k \in \mathbb{R}_+$ eine monoton fallende Schrittweitenfolge mit der Eigenschaft

$$0 < \frac{h_{k+1}}{h_k} \leq \rho < 1. \quad (9.11)$$

Es sei $p_n^{(k)} \in P_n$ das Interpolationspolynom zu

$$(h_k, a(h_k)), \dots, (h_{k+n}, a(h_{k+n})).$$

Dann gilt

$$a(0) - p_n^{(k)}(0) = O(h_k^{n+1}) \quad (k \rightarrow \infty).$$

Beweis In Lagrangescher Darstellung gilt

$$p_n^{(k)}(t) = \sum_{i=0}^n a(h_{k+i}) L_{k+i}^{(n)}(t), \quad L_{k+i}^{(n)} = \prod_{l=0, l \neq i}^n \frac{t - h_{k+l}}{h_{k+i} - h_{k+l}}. \quad (9.12)$$

Wir setzen die Entwicklung von $a(h)$ in die Polynomdarstellung (9.12) ein und erhalten für $t = 0$

$$p_n^{(k)}(0) = \sum_{i=0}^n \left\{ a_0 + \sum_{j=1}^n a_j h_{k+i}^j + a_{n+1}(h_{k+i}) h_{k+i}^{n+1} \right\} L_{k+i}^{(n)}(0).$$

Für die Lagrangeschen Basispolynome gilt die Beziehung

$$\sum_{i=0}^n h_{k+i}^r L_{k+i}^{(n)}(0) = \begin{cases} 1 & r = 0, \\ 0 & r = 1, \dots, n, \\ (-1)^n \prod_{i=0}^n h_{k+i} & r = n + 1. \end{cases} \quad (9.13)$$

Der Nachweis erfolgt durch Analyse der Lagrangeschen Interpolation des Polynoms $p(x) = x^r$ für verschiedene Exponenten r . Mit (9.13) und $a_{n+1}(h_{k+i}) = a_{n+1} + o(h_{k+i})$ folgt

$$p_n^{(k)}(0) = a_0 + a_{n+1}(-1)^n \prod_{i=0}^n h_{i+k} + \sum_{i=0}^n o(1) h_{k+i}^{n+1} L_{k+i}^{(n)}(0).$$

Mit der Schrittweitenbedingung (9.11) $h_{k+1} \leq \rho h_k$ und $\rho < 1$ gilt für die Lagrangeschen Basispolynome in $t = 0$

$$|L_{k+i}^{(n)}(0)| = \prod_{l=0, l \neq i}^n \left| \frac{1}{\frac{h_{k+i}}{h_{k+l}} - 1} \right| \leq c(\rho, n), \quad (9.14)$$

da $|h_{k+1}/h_{k+l}|$ von 1 weg beschränkt ist. Insgesamt folgt mit $h_{k+i} \leq h_k$

$$|p_n^{(k)} - a(0)| = O(h_k^{n+1}).$$

□

Die Schrittweitenbedingung ist notwendig zum Abschätzen von $|L_{k+i}^{(n)}| = O(1)$ und verhindert das starke Oszillieren der Basisfunktionen bei $t = 0$ wie in Beispiel 9.16 beobachtet. Eine zulässige Schrittweitenfolge zur Extrapolation ist $h_i = \frac{h_0}{2^i}$. Die einfache Wahl $h_i = h_0/i$ hingegen ist nicht zulässig, da hier $h_{k+1}/h_k \rightarrow 1$ gilt und eine Abschätzung in Schritt (9.14) nicht mehr gleichmäßig in k möglich ist.

Um die Extrapolationsvorschrift auf ein gegebenes Verfahren anzuwenden, werden die Approximationen $p_n^{(k)}(0)$ mithilfe des Neville-Schemas aus Algorithmus 9.1 berechnet. Wir führen hierzu Beispiel 9.26 fort:

Beispiel 9.28 (Extrapolation des zentralen Differenzenquotienten)

Wir approximieren wieder die Ableitung von $f(x) = \tanh(x)$ an der Stelle $x_0 = 0.5$ und berechnen die Approximationen nun mit dem zentralen Differenzenquotienten. Der exakte Wert ist $\tanh'(0.5) \approx 0.78645$. Wir erhalten

h	$a(h) = a_{kk}$	$a_{k,k+1}$	$a_{k,k+2}$	$a_{k,k+3}$
2^{-1}	0.7616	0.7993	0.78620	0.7864593
2^{-2}	0.7805	0.7895	0.78643	
2^{-3}	0.7850	0.7872		
2^{-4}	0.7861			

Die Approximation wird wieder verbessert. Es zeigt sich jedoch, dass im ersten Schritt, d. h. für alle Werte $a_{k,k+1}$, welche aus linearer Interpolation zweier Werte folgen, keine Verbesserung erreicht werden kann. ◀

Im vorangegangenen Beispiel wird der zentrale Differenzenquotient extrapoliert. Für diesen gilt bei $f \in C^\infty$ die Reihenentwicklung

$$\begin{aligned}
 a(h) &= \frac{f(x+h) - f(x-h)}{2h} \\
 &= \frac{1}{2h} \left(\sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} h^k - \sum_{k=0}^{\infty} \frac{f^{(k)}(x)}{k!} (-h)^k \right) = \sum_{k=0}^{\infty} \frac{f^{(2k+1)}(x)}{(2k+1)!} h^{2k}.
 \end{aligned}$$

Das heißt, bei $a(h)$ liegt eine Entwicklung in geraden Potenzen von h vor. Dies erklärt, warum die Interpolation mit linearen Polynomen keinen Gewinn bringt. Stattdessen werden wir nun die bekannte Potenzentwicklung von $a(h)$ ausnutzen und suchen nach Interpolationspolynomen in h^2

$$p(h) = a_0 + a_1 h^2 + a_2 h^4 + \dots$$

Beispiel 9.29 (Extrapolation des zentralen Differenzenquotienten mit quadratischen Polynomen)

Wir approximieren wieder die Ableitung von $f(x) = \tanh(x)$ und interpolieren die bereits berechneten Werte $(a(h_3), h_3)$ sowie $a(h_4), h_4$ mithilfe eines linearen Polynoms in h^2 . Es gilt

$$p(h) = a(h_3) \frac{h^2 - h_4^2}{h_3^2 - h_4^2} + a(h_4) \frac{h^2 - h_3^2}{h_4^2 - h_3^2} \Rightarrow p(0) = \frac{h_3^2 a(h_4) - h_4^2 a(h_3)}{h_3^2 - h_4^2}.$$

Mit den Werten aus Beispiel 9.28 folgt

$$p(0) = \underline{\underline{0.7864493}},$$

d.h., nach nur einem Schritt mit richtiger Ordnung sind bereits die ersten vier Stellen exakt. ◀

Die Richardson-Extrapolation spielt ihre Stärke erst dann voll aus, wenn die zugrundeliegende Funktion $a(h)$ eine spezielle Reihenentwicklung besitzt, welche z.B. nur gerade Potenzen von h beinhaltet. Für den zentralen Differenzenquotienten gilt bei hinreichender Regularität von f

$$a(h) := \frac{f(x_0+h) - f(x_0-h)}{2h} = f'(x_0) + \sum_{i=1}^n \frac{f^{(2i+1)}(x_0)}{(2i+1)!} h^{2i} + O(h^{2n+1}).$$

Die Idee ist es nun, diese Entwicklung bei der Interpolation auszunutzen und nur Polynome in $1, h^2, h^4, \dots$ zu berücksichtigen. Der zentrale Differenzenquotient ist kein Einzelfall. Bei der numerischen Quadratur werden wir das Romberg-Verfahren kennenlernen, welches ebenso auf der Extrapolation einer Vorschrift mit Entwicklung in h^2 beruht. Auch bei der Approximation von gewöhnlichen Differentialgleichungen lernen wir mit dem Gragg'schen Extrapolationsverfahren eine Methode kennen, die auf diesem Prinzip beruht. Daher formulieren wir ohne Beweis den allgemeinen Satz zur Richardson-Extrapolation:

Satz 9.30 (Richardson-Extrapolation zum Limes) Es sei $a(h) : \mathbb{R}_+ \rightarrow \mathbb{R}$ für ein $q > 0$ eine $q(n+1)$ -mal stetig differenzierbare Funktion mit der Summenentwicklung

$$a(h) = a_0 + \sum_{j=1}^n a_j h^{qj} + a_{n+1}(h) h^{q(n+1)}$$

mit Koeffizienten $a_j \in \mathbb{R}$ sowie $a_{n+1}(h) = a_{n+1} + o(1)$. Weiter sei $(h_k)_{k=0,1,\dots}$ eine monoton fallende Schrittweitenfolge $h_k > 0$ mit der Eigenschaft

$$0 < \frac{h_{k+1}}{h_k} \leq \rho < 1. \quad (9.15)$$

Es sei $p_n^{(k)} \in P_n$ (in h^q) das Interpolationspolynom zu

$$(h_k^q, a(h_k)), \dots, (h_{k+n}^q, a(h_{k+n})).$$

Dann gilt

$$a(0) - p_n^{(k)}(0) = O(h_k^{q(n+1)}) \quad (k \rightarrow \infty).$$

Beweis Der Beweis ist eine einfache Modifikation von Satz 9.27 und kann im Wesentlichen mithilfe der Substitution $\tilde{h}_k = h_k^q$ übertragen werden. Für Details verweisen wir auf [71]. $\square$

Prinzipiell erlaubt die Richardson-Extrapolation bei hinreichender Regularität eine beliebige Steigerung der Verfahrensordnung. Üblicherweise werden jedoch nur einige wenige Extrapolationsschritte aus den letzten Verfahrenswerten $h_k, h_{k+1}, \dots, h_{k+n}$ verwendet. Die Extrapolation wird am einfachsten mit dem Neville-Schema an der Stelle $\xi = 0$ in Algorithmus 9.1 durchgeführt.

Wir schließen die Richardson-Extrapolation mit einem weiteren Beispiel ab:

Beispiel 9.31 (Extrapolation des zentralen Differenzenquotienten)

Wir approximieren die Ableitung von $f(x) = \tanh(x)$ an der Stelle $x = 0.5$ mit dem zentralen Differenzenquotienten und Extrapolation. Exakter Wert ist $\tanh'(0.5) \approx 0.7864477329$:

h	$a(h) = p_{kk}$	$p_{k,k+1}$	$p_{k,k+2}$	$p_{k,k+3}$
2^{-1}	0.761594156	0.7867493883	0.7864537207	0.7864477443
2^{-2}	0.780460580	0.7864721999	0.7864478377	
2^{-3}	0.784969295	0.7864493603		
2^{-4}	0.786079344			

Bereits die jeweils erste Extrapolation $p_{k,k+1}$ liefert weit bessere Ergebnisse als die Extrapolation des einseitigen Differenzenquotienten. Die beste Approximation $p_{k,k+3}$ verfügt über acht richtige Stellen. ◀

Beispiel 9.32 (Extrapolation mit verschiedenen Fehlerordnungen)

Wir betrachten

$$f(x) = -e^{1-\cos(\pi x)}$$

und approximieren $f''(1) \approx 72.9270606$ mit dem zentralen Differenzenquotienten für die zweite Ableitung

$$a(h) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}.$$

Wir führen jeweils zwei Schritte der Extrapolation durch. Dabei wählen wir zunächst suboptimal $q = 1$.

h	$a(h_k) = p_{kk}$	$a(h_k) - f'(1)$	$p_{k,k+1}$	$p_{k,k+1} - f'(1)$	$p_{k,k+2}$	$p_{k,k+2} - f'(1)$
2^{-2}	60.03	$1.29 \cdot 10^1$	78.61	$5.68 \cdot 10^0$	73.36353	$4.36 \cdot 10^{-1}$
2^{-3}	69.32	$3.60 \cdot 10^0$	74.68	$1.75 \cdot 10^0$	<u>72.95825</u>	$3.12 \cdot 10^{-2}$
2^{-4}	<u>72.00</u>	$9.28 \cdot 10^{-1}$	73.39	$4.61 \cdot 10^{-1}$	<u>72.92908</u>	$2.02 \cdot 10^{-3}$
2^{-5}	<u>72.69</u>	$2.34 \cdot 10^{-1}$	73.04	$1.17 \cdot 10^{-1}$	<u>72.92719</u>	$1.27 \cdot 10^{-4}$
2^{-6}	<u>72.87</u>	$5.85 \cdot 10^{-2}$	<u>72.96</u>	$2.93 \cdot 10^{-2}$		
2^{-7}	<u>72.91</u>	$1.46 \cdot 10^{-2}$				
Ordnung	$O(h^2)$		$O(h^2)$		$O(h^4)$	

Ein Schritt der Extrapolation bringt keinen Vorteil. Dies erklärt sich dadurch, dass sich die rein quadratische Fehlerentwicklung des Differenzenquotienten durch ein lineares Polynom nicht approximieren lässt. Wir wiederholen die Rechnung und wählen nun – dem Differenzenquotienten angepasst – die Extrapolationsordnung $q = 2$:

h	$a(h_k) = p_{kk}$	$a(h_k) - f'(1)$	$p_{k,k+1}$	$p_{k,k+1} - f'(1)$	$p_{k,k+2}$	$p_{k,k+2} - f'(1)$
2^{-2}	60.03	$1.29 \cdot 10^1$	<u>72.41896</u>	$5.08 \cdot 10^{-1}$	<u>72.9227341</u>	$4.33 \cdot 10^{-3}$
2^{-3}	69.32	$3.60 \cdot 10^0$	<u>72.89125</u>	$3.58 \cdot 10^{-2}$	<u>72.9269850</u>	$7.56 \cdot 10^{-5}$
2^{-4}	<u>72.00</u>	$9.28 \cdot 10^{-1}$	<u>72.92475</u>	$2.31 \cdot 10^{-3}$	<u>72.9270594</u>	$1.22 \cdot 10^{-6}$
2^{-5}	<u>72.69</u>	$2.34 \cdot 10^{-1}$	<u>72.92692</u>	$1.45 \cdot 10^{-4}$	<u>72.9270606</u>	$1.91 \cdot 10^{-8}$
2^{-6}	<u>72.87</u>	$5.85 \cdot 10^{-2}$	<u>72.92705</u>	$9.11 \cdot 10^{-6}$		
2^{-7}	<u>72.91</u>	$1.46 \cdot 10^{-2}$				
Ordnung	$O(h^2)$		$O(h^4)$		$O(h^6)$	

Hier erhalten wir durch Extrapolation den entsprechenden Ordnungsgewinn. ◀

9.5 Bestapproximation

Bisher haben wir uns im Wesentlichen mit der Interpolation beschäftigt. Die Approximation ist weiter gefasst: Wir suchen eine einfache Funktion $p \in P$ (dabei ist der Funktionenraum P meist wieder der Raum der Polynome), welche die *beste Approximation* zu gegebener Funktion f oder zu gegebenen diskreten Daten $y_i \in \mathbb{R}$ darstellt. Wir verstehen unter der besten Approximation den minimalen Abstand von p zu f (oder zu den diskreten Daten) in einer Norm. Die allgemeine Approximationsaufgabe besteht nun im Auffinden von $p \in P$, sodass

$$\|f - p\| = \min_{q \in P} \|f - q\|.$$

Die Wahl der Norm $\|\cdot\|$ ist dabei zunächst beliebig. Es stellt sich jedoch heraus, dass die mathematische Approximationsaufgabe je nach betrachteter Norm einer sehr unterschiedlichen Vorgehensweise bedarf. Auch die Lagrange-Interpolation kann als eine Bestapproximation formuliert werden:

Bemerkung 9.33 (Lagrange-Interpolation als Approximation) Angenommen, in den $n + 1$ paarweise verschiedenen Stützstellen $x_0, x_1, \dots, x_n$ soll für das Polynom $p_n \in P_n$ die Bedingung $p(x_i) = f(x_i)$ gelten, dann definieren wir die Norm

$$|p|_n := \max_{i=0, \dots, n} |p(x_i)|.$$

Man kann zeigen, dass dies wirklich eine Norm auf dem Polynomraum P_n erzeugt. Die Approximationsaufgabe: Suche $p \in P_n$, sodass

$$|p - f|_n = \min_{q \in P_n} |q - f|_n,$$

ist gerade die Lagrange-Interpolationsaufgabe. ♦

In diesem Abschnitt werden wir uns hingegen hauptsächlich mit der L^2 -Norm

$$\|f\|_{L^2([a,b])} := \sqrt{(f, f)} = \left(\int_a^b f(x)^2 dx \right)^{\frac{1}{2}}$$

sowie mit der Maximumsnorm

$$\|f\|_{\infty} := \max_{x \in [a,b]} |f(x)|$$

befassen. Die beste Approximation in der L^2 -Norm taucht in der Analysis in Form der *Fourier-Entwicklung* in trigonometrischen Polynomen auf. Konvergenz wird hier bezüglich der L^2 -Norm

$$\|f - f_n\|_{L^2([a,b])} \rightarrow 0 \quad (n \rightarrow \infty)$$

gezeigt. Die Approximation bezüglich der Maximumsnorm ist für die praktische Anwendung von großer Bedeutung: Denn eine Approximation, die den Fehler gleichmäßig auf dem gesamten Intervall $[a, b]$ unter Kontrolle hält, schließt die typischen Oszillationen der Lagrange-Interpolation an den Intervallenden (siehe Beispiel 9.16) systematisch aus. Es zeigt sich allerdings, dass gerade dieser für die Anwendung wichtige Approximationsbegriff mathematisch schwer zu greifen ist.

9.5.1 Gauß-Approximation – beste Approximation in der L^2 -Norm

Zunächst betrachten wir die beste Approximation einer Funktion $f : [a, b] \rightarrow \mathbb{R}$ mit Polynomen $p \in P$ bezüglich der L^2 -Norm:

$$\|f - p\|_{L^2([a,b])} = \min_{\phi \in P} \|f - \phi\|_{L^2([a,b])}$$

Vektorräume mit Skalarprodukt heißen *Prä-Hilbert-Raum*. Speziell in reellen Vektorräumen spricht man von *euklidischen Räumen*, im Komplexen auch von *unitären Räumen*. Das Skalarprodukt dient zur Beschreibung von Orthogonalitätsbeziehungen. Für die Approximation bezüglich der L^2 -Norm gilt die folgende Charakterisierung:

Satz 9.34 (Approximation und Orthogonalität) Es sei $f \in C[a, b]$ und $S \subset C[a, b]$ ein endlich-dimensionaler Unterraum. Es sei durch

$$(f, g) := \int_a^b f(x)g(x) \, dx, \quad \|f\| = (f, f)^{\frac{1}{2}},$$

das L^2 -Skalarprodukt mit zugehöriger Norm gegeben. Dann ist $p \in S$ beste Approximation zu $f \in C[a, b]$

$$\|f - p\| = \min_{\phi \in S} \|f - \phi\|$$

genau dann, wenn der Fehler $f - p$ orthogonal auf dem Raum S steht:

$$(f - p, \phi) = 0 \quad \forall \phi \in S$$

Beweis (i) Wir zeigen zunächst, dass jede beste Approximation $p \in S$ die Orthogonalitätsbedingung erfüllt. Die quadratische Funktion

$$F_\phi(t) := \|f - p - t\phi\|^2 = (f - p - t\phi, f - p - t\phi), \quad t \in \mathbb{R}$$

hat für jedes fest gewählte $\phi \in S$ bei $t = 0$ ein Minimum. Das heißt, $t = 0$ muss stationärer Punkt sein:

$$\left. \frac{d}{dt} F_\phi(t) \right|_{t=0} = 0 \quad \Rightarrow \quad \frac{d}{dt} F_\phi(t) = -2(f - p - t\phi, \phi)$$

Aus $t = 0$ folgt die Orthogonalitätsbeziehung (für alle $\phi \in S$).

(ii) Umgekehrt gelte nun die Beziehung

$$(f - p, \phi) = 0 \quad \forall \phi \in S$$

für ein $p \in S$. Dann gilt für den Fehler für beliebige $\phi \in S$

$$\begin{aligned} \|f - p\|^2 &= (f - p, f - p) = (f - p, f - \phi) + \underbrace{(f - p, \phi - p)}_{=0} \\ &\leq \|f - p\| \|f - \phi\|, \end{aligned}$$

was die Minimalbedingung zeigt. $\square$

Wir können die beste Approximation $p \in S$ durch eine Orthogonalitätsbeziehung beschreiben. Dieser Zusammenhang ist der Schlüssel zur Analyse der Gauß-Approximation und auch zur praktischen numerischen Realisierung. Wir können sofort den allgemeinen Satz beweisen:

Satz 9.35 (Allgemeine Gauß-Approximation) Es sei H ein Vektorraum mit Skalarprodukt $(\cdot, \cdot)$ und $S \subset H$ ein endlich-dimensionaler Teilraum. Dann existiert zu jedem $f \in H$ eine eindeutig bestimmte beste Approximation $p \in S$ in der induzierten Norm $\|\cdot\|$:

$$\|f - p\| = \min_{\phi \in S} \|f - \phi\|$$

Beweis Für den Beweis nutzen wir die Charakterisierung der Bestapproximation mithilfe des Skalarprodukts aus Satz 9.34.

(i) *Eindeutigkeit*: Seien $p_1, p_2 \in S$ zwei Bestapproximationen. Dann gilt notwendigerweise

$$(f - p_i, \phi) = 0 \quad \Rightarrow \quad (p_1 - p_2, \phi) = 0 \quad \forall \phi \in S,$$

für $\phi := p_1 - p_2$ folgt, $\|p_1 - p_2\| = 0$, somit $p_1 = p_2$.

(ii) *Existenz*: Der endlich-dimensionale Teilraum $S \subset H$ besitzt eine Basis $\{\phi_1, \dots, \phi_n\}$ mit $n := \dim(S)$. Die beste Approximation $p \in S$ stellen wir als Linearkombination in dieser Basis dar, als

$$p = \sum_{i=1}^n \alpha_i \phi_i$$

mit eindeutig bestimmten Koeffizienten $\alpha_k \in \mathbb{R}$. Dieser Ansatz für p wird in die Orthogonalitätsbedingung eingesetzt:

$$(f - p, \phi_j) = (f - \sum_{i=1}^n \alpha_i \phi_i, \phi_j) = (f, \phi_j) - \sum_{i=1}^n \alpha_i (\phi_i, \phi_j) = 0$$

für $j = 1, \dots, n$. Dies entspricht dem linearen Gleichungssystem

$$Ax = b$$

mit Koeffizientenmatrix $A = (a_{ij})_{i,j=1}^n$, rechter Seite $b = (b_j)_{j=1}^n$ und Lösung $x = (\alpha_i)_{i=1}^n$:

$$a_{ji} = (\phi_i, \phi_j), \quad b_j = (f, \phi_j)$$

Die Matrix A ist die *Gramsche Matrix* zur Basis $\phi_1, \dots, \phi_n$ und stets regulär. Damit ist das Gleichungssystem stets eindeutig lösbar. $\square$

Der Beweis liefert sofort ein Konstruktionsprinzip zur Bestimmung der Bestapproximation $p \in S$. Wir stellen zu gegebener Basis $\{\phi_1, \dots, \phi_n\}$ das lineare $(n \times n)$ -Gleichungssystem auf:

$$\sum_{j=1}^n a_{ij} \alpha_j = b_i, \quad i = 1, \dots, n$$

mit $a_{ij} = (\phi_i, \phi_j)$, und berechnen den Ergebnisvektor $x = (\alpha_j)_j$. Die gesuchte Bestapproximation $p \in S$ ist dann in der Basisdarstellung gegeben als

$$p = \sum_{i=1}^n \alpha_i \phi_i.$$

Zur numerischen Realisierung muss zunächst das lineare Gleichungssystem aufgestellt werden, d. h., insbesondere müssen die Einträge der Systemmatrix (numerisch) integriert werden. Schließlich ist das lineare Gleichungssystem $Ax = b$ zu lösen. Angenommen, wir starten die Konstruktion mit der Monombasis $\{1, x, \dots, x^{n-1}\}$ auf dem Intervall $[0, 1]$. Dann ist die durch

$$a_{ij} = \int_0^1 x^i x^j dx$$

gegebene Matrix die sogenannte *Hilbert-Matrix*

$$H_n = \begin{pmatrix} 1 & \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \cdots & \frac{1}{n} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \cdots & \frac{1}{n+1} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} & \ddots & & \frac{1}{n+2} \\ \frac{1}{4} & \frac{1}{5} & \ddots & \ddots & & \vdots \\ \vdots & & & \ddots & \ddots & \vdots \\ \frac{1}{n} & \frac{1}{n+1} & \frac{1}{n+2} & \frac{1}{n+3} & \cdots & \frac{1}{2n-1} \end{pmatrix}.$$

Das Lösen eines Gleichungssystems mit der Hilbert-Matrix ist äußerst schlecht konditioniert. Schon für $n = 4$ liegt die Fehlerverstärkung der Hilbert-Matrix bei 15 000. Diese

Matrix muss also unbedingt vermieden werden. Der direkte Weg zum Erstellen der Bestapproximation ist somit nicht numerisch stabil.

Auch wenn die Wahl der Basisvektoren $\{\phi_1, \dots, \phi_n\}$ keinen Einfluss auf das Ergebnis hat, so bestimmt sie doch wesentlich den Aufwand bei der Realisierung. Angenommen, die Basis sei ein Orthonormalsystem, d. h., es gelte $(\phi_i, \phi_j) = \delta_{ij}$ für alle $i, j = 1, \dots, n$. Dann gilt für die Systemmatrix

$$a_{ji} = (\phi_i, \phi_j) = \delta_{ij} \Rightarrow A = I.$$

Die Systemmatrix ist die Einheitsmatrix und die gesuchten Koeffizienten lassen sich sofort ablesen,

$$\alpha_i = (f, \phi_i),$$

womit die Bestapproximation durch die Relation

$$p = \sum_{i=1}^n (f, \phi_i) \phi_i$$

trivial gegeben ist. Für dieses vereinfachte Vorgehen benötigen wir zunächst eine Orthonormalbasis von $S \subset H$. Die Orthonormalisierung kann mithilfe des bereits diskutierten Gram-Schmidt-Algorithmus durchgeführt werden. Bei Verwendung der Monombasis des $S = P_n$ führt dieser Algorithmus auf die Legendre-Polynome.

Für die bezüglich des $L^2(-1, 1)$ orthogonalen Polynome lässt sich eine geschlossene Formel angeben:

Satz 9.36 (Legendre-Polynome) Die aus der Monombasis $\{1, x, x^2, \dots\}$ mit dem Gram-Schmidt-Verfahren (ohne Normierung) bzgl. des $L^2(-1, 1)$ -Skalarprodukt erstellte Orthogonalbasis $\{p_0, \dots, p_k\}$ hat die explizite Darstellung

$$p_k(x) := \frac{k!}{(2k)!} \frac{d^k}{dx^k} (x^2 - 1)^k. \quad (9.16)$$

Für diese Polynome gilt

$$\|p_k\| = \frac{k!^2}{(2k)!} \sqrt{\frac{2^{2k+1}}{2k+1}}, \quad p_k(1) = \frac{2^k k!^2}{(2k)!}. \quad (9.17)$$

Sie lassen sich durch eine zweistufige Rekursionsformel berechnen:

$$\begin{aligned} p_0(x) &= 1 \\ p_1(x) &= x \\ k = 1, \dots, n-1: \quad p_{k+1}(x) &= x p_k(x) - \frac{k^2}{4k^2 - 1} p_{k-1}(x) \end{aligned} \quad (9.18)$$

Durch Normierung bei $x = 1$ sind die *Legendre-Polynome* $L_n \in P_n$ definiert als

$$L_k(x) := \frac{(2k)!}{2^k k!^2} p_k(x), \quad L_k(1) = 1.$$

Beweis (i) Wir zeigen zunächst per partieller Differentiation die Orthogonalität. Es gilt für $k \geq l$

$$\begin{aligned} \frac{(2k!)(2l)!}{k!l!} (p_k, p_l) &= \left(d^k(x^2 - 1)^k, d^l(x^2 - 1)^l \right) \\ &= \underbrace{d^{k-1}(x^2 - 1)^k \cdot d^l(x^2 - 1)^l \Big|_{-1}^1}_{=0} - \left(d^{k-1}(x^2 - 1)^k, d^{l+1}(x^2 - 1)^l \right), \end{aligned}$$

da $d^s(x^2 - 1)^k$ für $s < k$ stets den Faktor $x^2 - 1$ enthält. Nach $l \leq k$ -facher partieller Differentiation ergibt sich daher

$$\frac{(2k!)(2l)!}{k!l!} (p_k, p_l) = (-1)^l \left(d^{k-l}(x^2 - 1)^k, d^{2l}(x^2 - 1)^l \right). \quad (9.19)$$

Für $k = l$ folgern wir hieraus

$$\frac{(2k)!^2}{k!^2} \|p_k\|^2 = (-1)^k \left((x^2 - 1)^k, 1 \right) (2k)! = (-1)^k (2k)! \int_{-1}^1 (x^2 - 1)^k dx, \quad (9.20)$$

also

$$\|p_k\|^2 = (-1)^k \frac{k!^2}{(2k)!} \int_{-1}^1 (x^2 - 1)^k dx. \quad (9.21)$$

Das Integral berechnet sich mit der Gammafunktion (siehe [50]) als

$$\int_{-1}^1 (x^2 - 1)^k dx = (-1)^k \frac{\Gamma(k+1)\sqrt{\pi}}{\Gamma(\frac{3}{2} + k)} = (-1)^k \frac{k!(k+1)!2^{2k+2}}{(2(k+1))!}.$$

Zusammen mit (9.21) folgt

$$\|p_k\|^2 = \frac{k!^4 2^{2k+1}}{(2k)!^2 (2k+1)}$$

und somit die Darstellung (9.17).

Falls $k > l$, folgt aus (9.19) bei weiterer partieller Differentiation

$$\frac{(2k!)(2l)!}{k!!} (p_k, p_l) = \underbrace{(-1)^l d^{k-l-1} (x^2 - 1)^k \cdot d^{2l} (x^2 - 1)^l}_{=0} \Big|_{-1}^1 - (-1)^l \left(d^{k-l-1} (x^2 - 1)^k, \underbrace{d^{2l+1} (x^2 - 1)^l}_{=0} \right)$$

die Orthogonalität.

(ii) Die Normierung an $x = 1$ ist klar für $k = 0$ sowie $k = 1$. Rekursiv folgt:

$$\begin{aligned} p_{k+1}(1) &= 1 \cdot \frac{2^k k!^2}{(2k)!} - \frac{k^2}{4k^2 - 1} \frac{2^{k-1} (k-1)!^2}{(2(k-1))!} \\ &= \frac{2^k k!^2}{(2k)!} - \frac{2^{k-1} k!^2}{(2k-1)(2k+1)(2(k-1))!} \\ &= \frac{2^k k!^2 (2k+1)(2k+2)}{(2(k+1))!} - \frac{2^{k-1} k!^2 2k(2k+2)}{(2(k+1))!} \\ &= \frac{2^{k+1} (k+1)!^2}{(2(k+1))!} \left(\frac{(2k+1)}{k+1} - \frac{k}{k+1} \right) = \frac{2^{k+1} (k+1)!^2}{(2(k+1))!} \end{aligned}$$

(iii) Der führende Koeffizient ist eins, also

$$p_k(x) = x^k \pm \dots$$

Mit der Normierung des führenden Koeffizienten sind die aus der Monombasis orthogonalisierten Polynome eindeutig bestimmt.

(iv) Es bleibt der Nachweis der zweistufigen Rekursionsformel. Diese werden mithilfe eines allgemeinen Prinzips herleiten und den Beweis später abschließen, siehe Satz 9.37. $\square$

Einschub: Zweistufige Orthogonalisierung

In Abschn. 6.2 haben wir bereits eine zweistufige Orthogonalisierung im Rahmen des CG-Verfahrens verwendet. Wir werden dieses Prinzip nun verallgemeinern. Es sei für das Folgende V ein allgemeiner, endlich- oder abzählbar unendlich-dimensionaler Vektorraum, also z. B. der Raum der Polynome (bis zu einem Grad n). Weiter sei durch $A : V \rightarrow V$ eine Abbildung auf V gegeben. Der Krylow-Unterraum, lässt sich dann ganz analog definieren mit $d \in V$ als

$$K_l(d, A) := \text{span}\{d, Ad, \dots, A^{l-1}d\} \subset V$$

Am Beispiel der Polynome wählen wir z. B. $d = 1$, also das konstante Monom, und die Abbildung $A : P_k \rightarrow P_{k+1}$ als

$$p(z) \mapsto zp(z).$$

Dann ist der Krylow-Raum $K_k(1, z)$ gerade der Polynomraum P_{k-1} .

Es gilt:

Satz 9.37 (Zweistufige Orthogonalisierung) Es sei V ein Vektorraum mit Skalarprodukt $(\cdot, \cdot)$ und induzierter Norm $\|\cdot\|$. Es sei $A : V \rightarrow V$ so gegeben, mit

$$(Ax, y) = (x, Ay) \quad \forall x, y \in V \quad (9.22)$$

Für ein $d \in V$ lässt sich eine Orthonormalisierung $\{q_1, \dots, q_l\}$ des Krylow-Raums $K_l(d, A)$ in folgendem zweistufigen Verfahren berechnen:

$$\begin{aligned} q_1 &:= \frac{d}{\|d\|}, \\ \tilde{q}_2 &:= Aq_1 - (Aq_1, q_1)q_1, \quad q_2 := \frac{\tilde{q}_2}{\|\tilde{q}_2\|}, \\ i = 3, \dots, l : \quad \tilde{q}_i &:= Aq_{i-1} - (Aq_{i-1}, q_{i-1})q_{i-1} - (Aq_{i-1}, q_{i-2})q_{i-2}, \\ q_l &:= \frac{\tilde{q}_l}{\|\tilde{q}_l\|} \end{aligned}$$

Beweis Wir führen den Beweis durch Induktion. Hierzu sei

$$q_1 := \frac{d}{\|d\|}.$$

Angenommen, nun sei durch $q_1, \dots, q_l$ eine Orthonormalbasis des $K_l(d, A)$ gegeben. Wir betrachten zwei Fälle:

Fall 1: Es gilt $K_{l+1}(d, A) \subset K_l(d, A)$, d.h., der Krylow-Raum wird nicht größer und das Verfahren bricht ab, da die Orthonormalbasis $\{q_1, \dots, q_l\}$ bereits gegeben ist.

Fall 2: Der Krylow-Raum $K_{l+1}(d, A)$ ist größer als $K_l(d, A)$.

(i) Wir zeigen zunächst durch ein Widerspruchsargument, dass $Aq_l \notin K_l(d, A)$ gilt. Denn angenommen es sei $Aq_l \in K_l(d, A)$. Dann existieren Koeffizienten $\alpha_1, \dots, \alpha_l \in \mathbb{R}$ mit

$$Aq_l = \sum_{i=1}^l \alpha_i q_i \quad \Rightarrow \quad (Aq_l, q_j) = \alpha_j \quad j = 1, \dots, l,$$

da die $q_1, \dots, q_l$ eine Orthonormalbasis bilden. Mit (9.22) folgt

$$\alpha_j = (Aq_l, q_j) = (q_l, \underbrace{Aq_j}_{\in K_{j+1}(d, A)}) = 0 \quad \text{für } j = 1, \dots, l-1$$

und somit im Gegensatz zur Annahme

$$Aq_l = \alpha_l q_l \in K_l(d, A).$$

(ii) Wir orthogonalisieren $Aq_l \in K_{l+1}(d, A)$ bzgl. $q_1, \dots, q_l$ gemäß

$$\tilde{q}_{l+1} := Aq_l - \sum_{i=1}^l \beta_i q_i.$$

Dies ergibt wieder

$$0 \stackrel{!}{=} (\tilde{q}_{l+1}, q_j) = (Aq_l, q_j) - \beta_j \quad j = 1, \dots, l,$$

da die $q_1, \dots, q_l$ eine Orthonormalbasis bilden. Weiter gilt

$$\beta_j = (Aq_l, q_j) = (q_l, \underbrace{Aq_j}_{\in K_{j+1}(d, A)}) = 0 \quad \text{für } j = 1, \dots, l-2.$$

Hieraus lesen wir die Iterationsvorschrift ab:

$$\tilde{q}_2 = Aq_1 - (Aq_1, q_1)q_1$$

sowie

$$\tilde{q}_{l+1} = Aq_l - (Aq_l, q_l)q_l - (Aq_l, q_{l-1})q_{l-1}$$

für $l > 1$. □

Damit wir eine Sequenz in diesem verkürzten Verfahren orthogonalisieren können, benötigen wir zwei Zutaten: Die anfängliche Sequenz muss im Sinne eines Krylow-Raums mit einer Abbildung $A : V \rightarrow V$ erzeugt werden. Das Skalarprodukt muss bzgl. A die Symmetrie $(Ax, y) = (x, Ay)$ erhalten.

Im Folgenden werden wir dieses Resultat auf die Legendre-Polynome anwenden. Zunächst gilt

$$p_{k+1}(x) = xp_k(x) - (tp_k, p_k)p_k(x) - (tp_k, p_{k-1})p_{k-1}(x).$$

Anhand der Darstellung (9.16) folgt, dass p_k entweder eine gerade oder ungerade Funktion ist. Denn $(x^2 - 1)^k$ ist gerade und die k -te Ableitung ist gerade, wenn k gerade ist, ansonsten ungerade. Damit ist p_k^2 eine gerade und $x p_k^2$ eine ungerade Funktion mit Mittelwert null. Also gilt die zweistufige Rekursion

$$p_{k+1}(x) = xp_k(x) - (tp_k, p_{k-1})p_{k-1}(x).$$

Wir berechnen den Faktor $\gamma_k = (tp_k, p_{k-1})$ indirekt. Mit (9.17) gilt

$$\frac{2^{k+1}(k+1)!^2}{(2(k+1))!} = \frac{2^k k!^2}{(2k)!} - \gamma_k \frac{2^{k-1}(k-1)!^2}{(2(k-1))!},$$

also

$$\frac{4k^2(k+1)^2}{(2k-1)2k(2k+1)(2k+2)} = \frac{2k^2}{(2k-1)2k} - \gamma_k$$

und schließlich

$$\gamma_k = \frac{k}{2k-1} - \frac{k(k+1)}{(2k-1)(2k+1)} = \frac{k^2}{4k^2-1}.$$

Hiermit ist der Beweis zu Satz 9.36 nun abgeschlossen.

Mit den Legendre-Polynomen existiert eine explizite Darstellung für die orthogonalen Polynome bezüglich des $L^2([-1, 1])$ -Skalarprodukts. Die ersten Legendre-Polynome lauten:

$$\begin{aligned} L_0(x) &= 1, \\ L_1(x) &= x, & x_0 &= 0, \\ L_2(x) &= \frac{1}{2}(3x^2 - 1), & x_{0/1} &= \pm\sqrt{\frac{1}{3}}, \\ L_3(x) &= \frac{1}{2}(5x^3 - 3x), & x_0 &= 0, \quad x_{1/2} = \pm\sqrt{\frac{3}{5}}, \\ L_4(x) &= \frac{1}{8}(35x^4 - 30x^2 + 3), & x_{0/1} &\approx \pm 0.861136, \quad x_{2/3} \approx \pm 0.339981. \end{aligned} \tag{9.23}$$

Die Legendre-Polynome erlauben eine einfache Anwendung der Gauß-Approximation:

Korollar 9.38 (Gauß-Approximation mit Polynomen) Es sei $f \in C[-1, 1]$. Die Bestapproximation $p \in P_n$ bezüglich der L^2 -Norm im Raum der Polynome von Grad n eindeutig bestimmt durch

$$p(x) = \sum_{i=0}^n \frac{1}{\|L_i\|_{L^2[-1,1]}^2} (L_i, f) L_i(x)$$

mit den Legendre-Polynomen

$$L_i(x) := \frac{1}{2^i i!} \frac{d^i}{dx^i} (x^2 - 1)^i.$$

Die Normierung mit $\|L_n\|^{-2}$ ist notwendig, da die Legendre-Polynome bezüglich der L^2 -Norm nicht normiert sind, vergleiche Satz 9.36.

Die Eigenschaft von $p \in P_n$, die beste Approximation zu f im Raum der Polynome zu sein, bedeutet auch, dass p bezüglich der L^2 -Norm eine bessere Approximation ist als jede Lagrange-Interpolation zu beliebiger Wahl von Stützstellen $x_0, \dots, x_n$ in $[-1, 1]$. Hieraus können wir auf triviale Weise eine einfache Fehlerabschätzung herleiten. Dazu sei $f \in C^{n+1}([a, b])$ und $p \in P_n$ die Bestapproximation. Es gilt

$$\|f - p\|_{L^2[a,b]} \leq \|f - I_h f\|_{L^2[a,b]} \quad \forall I_h f,$$

wobei $I_h f$ eine beliebige Interpolation zu f in $n + 1$ Stützstellen ist. Mit der (punktweisen) Fehlerabschätzung der Interpolation, Satz 9.11, folgt:

$$\|f - p\|_{L^2([a,b])} \leq \frac{\|f^{(n+1)}\|_\infty}{(n+1)!} \min_{x_0, \dots, x_n \in [a,b]} \left(\int_{-1}^1 \prod_{j=0}^n (x - x_j)^2 dx \right)^{\frac{1}{2}} \quad (9.24)$$

Das Minimum des zweiten Ausdrucks ist nicht einfach zu bestimmen, es können alle Stützstellen im Intervall frei variiert werden. Wir werden aber später auf diesen Punkt zurückkommen und diese Lücke schließen.

Beispiel 9.39 (Gauß-Approximation vs. Lagrange-Interpolation)

Wir approximieren die Funktion $f(x) = \sin(\pi x)$ auf dem Intervall $I = [-1, 1]$ mit Polynomen vom Grad drei. Zunächst erstellen wir in den äquidistant verteilten vier Stützstellen

$$x_i = -1 + \frac{2i}{3}, \quad i = 0, \dots, 3$$

das zugehörige Lagrangesche Interpolationspolynom. Aufgrund von $f(x_0) = f(x_3) = 0$ gilt

$$p_L(x) = \sin(x_1)L_1^{(3)}(x) + \sin(x_2)L_2^{(3)}(x) = \frac{27\sqrt{3}}{16}(x - x^3).$$

Als zweite Approximation bestimmen wir die Gauß-Approximation in der L^2 -Norm. Hierzu verwenden wir die Darstellung über die normierten Legendre-Polynome $\tilde{L}_n(x)$ auf $[-1, 1]$. Es gilt

$$\int_{-1}^1 \tilde{L}_0(x) f(x) dx = 0, \quad \int_{-1}^1 \tilde{L}_2(x) f(x) dx = 0,$$

sowie

$$\int_{-1}^1 \tilde{L}_1(x) f(x) dx = \frac{\sqrt{6}}{\pi}, \quad \int_{-1}^1 \tilde{L}_3(x) f(x) dx = \frac{\sqrt{14}(\pi^2 - 15)}{\pi^3}.$$

Hieraus erhalten wir die Gauß-Approximation

$$\begin{aligned} p_G(x) &= \frac{\sqrt{6}}{\pi} \tilde{L}_1(x) + \frac{\sqrt{14}(\pi^2 - 15)}{\pi^3} \tilde{L}_3(x) \\ &= \frac{5x(7\pi^2 x^2 - 105x^2 - 3\pi^2 + 63)}{2\pi^3}. \end{aligned}$$

Für die beiden Approximationen gilt

$$\|f - p_L\|_{L^2([-1,1])} \approx 0.2, \quad \|f - p_G\|_{L^2([-1,1])} \approx 0.1,$$

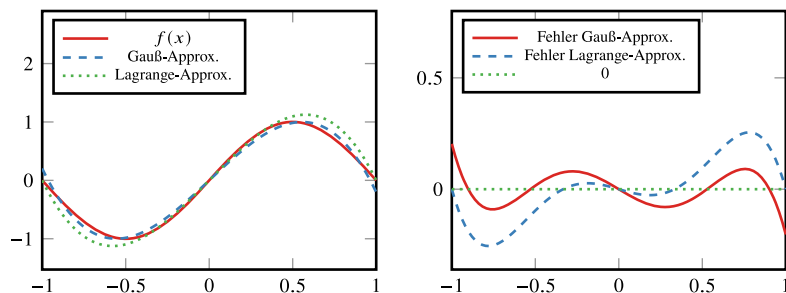


Abb.9.6 Gauß-Approximation sowie Lagrange-Interpolation (jeweils kubisch) der Funktion $f(x) = \sin(\pi x)$. Rechts: Fehler der Approximationen

d. h., die Gauß-Approximation liefert in der L^2 -Norm ein doppelt so gutes Ergebnis. In Abb. 9.6 zeigen wir beide Approximationen, die Funktion $f(x)$ sowie die Fehler $f(x) - p_l(x)$ und $f(x) - p_g(x)$. Hier sehen wir zunächst, dass die Lagrange-Interpolation an den Stützstellen die Interpolationsbedingung, also $f(x_i) = p_L(x_i) = 0$, erfüllt, wohingegen die Gauß-Approximation gerade am Rand einen großen Fehler aufweist. Der maximale Fehler im Intervall ist hingegen bei der Gauß-Approximation etwas geringer. ◀

9.5.1.1 Diskrete Gauß-Approximation

Abschließend betrachten wir nun die Bestapproximation diskreter Messwerte (x_i, y_i) , $i = 1, 2, \dots, m$. Wir definieren auf der Punktmenge die Norm

$$|y|_2 = \left(\sum_{i=1}^m y_i^2 \right)^{\frac{1}{2}}.$$

Es sei S ein endlich-dimensionaler Funktionenraum, z. B. der Raum der Polynome P_n . Wir suchen die beste Approximation $p \in S$ bezüglich der Punkt-Norm:

$$|p - y|_2 = \left(\sum_{i=1}^m |p(x_i) - y_i|^2 \right)^{\frac{1}{2}} = \min_{\phi \in S} |\phi - y|_2$$

Im Fall $m > n$ können wir keine Gleichheit in den Punkten x_i fordern. Das heißt, $\phi(x)$ ist keine Interpolation. Ein Beispiel ist die Bestimmung der linearen Ausgleichsgeraden $p_1(x) = \alpha_0 + \alpha_1 x$ zu vielen (evtl. tausenden) Messwerten. Die euklidische Vektornorm ist aus dem euklidischen Skalarprodukt abgeleitet, sodass

$$|x|_2 = (x, x)_2^{\frac{1}{2}},$$

daher kann die bisher entwickelte Theorie unmittelbar angewendet werden. Schlüssel zur Bestimmung der Bestapproximation ist wieder gemäß Satz 9.34 die Charakterisierung über die Orthogonalität:

$$|p - y|_2 = \min_{\phi \in S} |\phi - y|_2 \Leftrightarrow (p - y, \phi)_2 = 0 \quad \forall \phi \in \mathbb{R}^n$$

Alle Eigenschaften der kontinuierlichen Gauß-Approximation übertragen sich und wir können gleich den Existenzsatz formulieren:

Satz 9.40 (Diskrete Gauß-Approximation) Es seien durch (x_i, y_i) für $i = 1, \dots, m$ diskrete Datenwerte gegeben. Weiter sei S ein endlich-dimensionaler Funktionenraum mit Basis $\{\phi_1, \dots, \phi_n\}$. Die diskrete Gauß-Approximation $p \in S$

$$|p - y|_2 := \left(\sum_{i=1}^m |p(x_i) - y_i|^2 \right)^{\frac{1}{2}} = \min_{\phi \in S} |\phi - y|_2$$

ist eindeutig bestimmt.

Beweis Über die Basisdarstellung ist S äquivalent zum euklidischen Raum $\mathbb{R}^n$. Eindeutigkeit und Existenz folgen nun im $\mathbb{R}^n$ analog zum Beweis zu Satz 9.35. $\square$

Die Konstruktion der diskreten Gauß-Approximation folgt wieder über die Basisdarstellung

$$p(x) = \sum_{i=1}^n \alpha_i \phi_i(x)$$

aus der beschreibenden Orthogonalitätsbeziehung

$$(p - y, \phi_k)_2 = \sum_{i=1}^m (p(x_i) - y_i) \phi_k(x_i) = 0, \quad k = 1, \dots, n.$$

Setzen wir für p die Basisdarstellung ein, so erhalten wir das Gleichungssystem

$$\sum_{j=1}^n \alpha_j \underbrace{\sum_{i=1}^m \phi_j(x_i) \phi_k(x_i)}_{=(\phi_j, \phi_k)_2} = \underbrace{\sum_{i=1}^m y_i \phi_k(x_i)}_{=(y, \phi_k)_2}, \quad k = 1, \dots, n.$$

Der gesuchte Koeffizientenvektor $\alpha = (\alpha_k)_{k=1}^n$ ist gegeben als Lösung des Gleichungssystems

$$\begin{pmatrix} (\phi_0, \phi_0)_2 & (\phi_0, \phi_1)_2 & \cdots & (\phi_0, \phi_n)_2 \\ (\phi_1, \phi_0)_2 & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ (\phi_n, \phi_0)_2 & \cdots & \cdots & (\phi_n, \phi_n)_2 \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_n \end{pmatrix} = \begin{pmatrix} (y, \phi_0)_2 \\ (y, \phi_1)_2 \\ \vdots \\ (y, \phi_n)_2 \end{pmatrix}.$$

Aus der allgemeinen Darstellung kann eine spezielle und häufig genutzte Approximation abgeleitet werden, nämlich die Gaußsche Ausgleichsrechnung:

Beispiel 9.41 (Lineare Ausgleichsrechnung)

Wir suchen die lineare Approximation $p(x) \in P_1$

$$p(x) = \alpha_0 + \alpha_1 x$$

zu gegebenen Messwerten. Es seien $\phi_0(x) \equiv 1$, $\phi_1(x) = x$. Dann ergibt sich das folgende lineare Gleichungssystem:

$$\begin{pmatrix} (\phi_0, \phi_0)_2 & (\phi_0, \phi_1)_2 \\ (\phi_1, \phi_0)_2 & (\phi_1, \phi_1)_2 \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \end{pmatrix} = \begin{pmatrix} (y, \phi_0)_2 \\ (y, \phi_1)_2 \end{pmatrix}$$

$$\Leftrightarrow \begin{pmatrix} m & \sum x_i \\ \sum x_i & \sum x_i^2 \end{pmatrix} \begin{pmatrix} \alpha_0 \\ \alpha_1 \end{pmatrix} = \begin{pmatrix} \sum y_i \\ \sum x_i y_i \end{pmatrix}$$

Dieses System ist regulär, falls $m \geq 2$ und es mindestens zwei Stützwerte $x_i \neq x_j$ gibt. ◀

Verallgemeinerung der Gauß-Approximation

Die diskrete Gauß-Approximation sowie die Approximation von Funktionen lassen sich durch die Verwendung von gewichteten Skalarprodukten und entsprechenden gewichteten induzierten Normen vereinfachen.

Die Verwendung eines Gewichts dient dazu, die Approximationsgenauigkeit der Gauß-Approximierenden an den Intervallenden zu verbessern. Für jedes integrierbare und positive $\omega(x) > 0$ ist durch

$$(f, g)_\omega := \int_a^b f(x)g(x)\omega(x) \, dx$$

wieder ein Skalarprodukt mit entsprechender Norm

$$\|f\|_\omega = (f, f)_\omega^{\frac{1}{2}}$$

gegeben. Wird die Gewichtsfunktion $w(x) = \frac{1}{\sqrt{1-x^2}}$ auf dem Intervall $(-1, 1)$ verwendet, so legt die Bestapproximation ein größeres Gewicht auf die Intervallenden. Die hieraus abgeleiteten orthonormalen Polynome sind die Tschebyscheff-Polynome.

Satz 9.42 (Tschebyscheff-Polynome) Die Tschebyscheff-Polynome $T_n \in P_n$ mit

$$T_n(x) := \cos(n \arccos x), \quad -1 \leq x \leq 1,$$

sind die bezüglich des $(\cdot, \cdot)_\omega$ Skalarprodukts orthogonalisierten Monome $\{1, x, \dots\}$ mit der Gewichtsfunktion

$$\omega(x) = \frac{1}{\sqrt{1-x^2}}.$$

Die n paarweise verschiedenen Nullstellen des n -ten Tschebyscheff-Polynoms $T_n(x)$ sind gegeben durch

$$x_k = \cos\left(\frac{2k+1}{2n}\pi\right), \quad k = 0, \dots, n-1.$$

Auf dem Intervall $\mathbb{R} \setminus [-1, 1]$ haben die Tschebyscheff-Polynome die Darstellung

$$T_n(x) = \frac{1}{2} \left[(x + \sqrt{x^2 - 1})^n + (x - \sqrt{x^2 - 1})^n \right].$$

Beweis (i) Wir müssen zunächst nachweisen, dass durch $T_n(x)$ überhaupt Polynome gegeben sind. Wir führen den Beweis induktiv durch Herleiten einer Iterationsformel für die T_n . Es gilt:

$$T_0(x) = 1, \quad T_1(x) = x,$$

also insbesondere $T_0 \in P_0$ und $T_1 \in P_1$. Aus dem Additionstheorem für die Kosinus-Funktion

$$\cos((n+1)y) + \cos((n-1)y) = 2 \cos(ny) \cos(y)$$

erhalten wir mit $y = \arccos x$ eine zweistufige Rekursionsformel

$$T_{n+1}(x) + T_{n-1}(x) = 2xT_n(x) \quad \Rightarrow \quad T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x).$$

Hieraus schließen wir induktiv $T_n \in P_n$ mit

$$T_n(x) = 2^{n-1}x^n + \dots$$

(ii) Wir weisen nun die Orthogonalität der Tschebyscheff-Polynome bzgl. des gewichteten Skalarprodukts auf $[-1, 1]$ nach. Mit $x = \cos(t)$ gilt unter Ausnutzung der Orthogonalität trigonometrischer Polynome

$$\int_{-1}^1 \frac{T_n(x)T_m(x)}{\sqrt{1-x^2}} dx = \int_0^\pi \cos(nt) \cos(mt) dt = \begin{cases} \pi, & n = m = 0, \\ \frac{\pi}{2}, & n = m > 0, \\ 0, & n \neq m. \end{cases}$$

(iii) Die Nullstellen der Tschebyscheff-Polynome können unmittelbar aus der expliziten Darstellung berechnet werden.

(iv) Die zweite Darstellung der Tschebyscheff-Polynome kann ebenso auf die Iterationsformel zurückgeführt werden. $\square$

Auch die diskrete Gauß-Approximation lässt sich diesbezüglich verallgemeinern. Hierzu sei durch $\omega \in \mathbb{R}^n$ ein Vektor mit $\omega_i > 0$ gegeben. Dann ist für $x, y \in \mathbb{R}^n$ durch

$$(x, y)_\omega = \sum_{i=1}^n x_i y_i \omega_i, \quad |x|_\omega = (x, x)_\omega^{\frac{1}{2}},$$

ein gewichtetes Skalarprodukt mit entsprechender Norm gegeben. Gerade für die diskrete Approximation spielen die Gewichte eine große Rolle in der Anwendung: Angenommen, für die Stützstellen und Stützwerte sind Abschätzungen für den Messfehler bekannt. Dann können Stützwerte mit kleinerem Messfehler stärker gewichtet werden.

Sämtliche Sätze zur Gauß-Approximation wurden für beliebige, von Skalarprodukten induzierte Normen bewiesen. Daher übertragen sich alle Eigenschaften auch auf die gewichteten Normen.

9.5.2 Tschebyscheff-Approximation

Wählen wir anstelle der $L^2[a, b]$ -Norm die Maximumsnorm

$$\|f\|_\infty := \max_{[a,b]} |f(x)|,$$

so kann die Bestapproximationsaufgabe nicht entsprechend der Gauß-Approximation angegangen werden. Es fehlt der Zusammenhang der Maximumsnorm zu einem Skalarprodukt und somit der Bezug zur Orthogonalität und eine Charakterisierung der Bestapproximation mithilfe einer Variationsgleichung.

► **Definition 9.43 (Tschebyscheff-Approximation)** Es sei $f \in C[a, b]$ und $S \subset C[a, b]$ ein endlich-dimensionaler Teilraum. Die Funktion $p \in S$ mit

$$\|f - p\|_\infty \leq \|f - g\|_\infty \quad \forall g \in S$$

heißt *Tschebyscheff-Approximation* zu f .

Beispiel 9.44 (Bestapproximation in der Maximumsnorm)

Wir betrachten $f(x) = \cos(\pi x)$ auf $I = [0, 1]$. Wir suchen zunächst die Bestapproximation mit der konstanten Funktion $p \in P_0$, also $p(x) = c$:

$$\min_{c \in \mathbb{R}} \|f - c\|_\infty$$

Wegen

$$-1 = \min_I f(x) < \max_I f(x) = 1,$$

muss auch $-1 \leq c \leq 1$ gelten. Für diese Wahl erhalten wir

$$\|f - c\|_\infty = \max\{1 - c, 1 + c\}$$

und die optimale Lösung für $c = 0$.

Im Fall der linearen Bestapproximation $p(x) = c + \alpha x$ ist diese Aufgabe bereits schwerer zu lösen. ◀

Die Existenz der Tschebyscheff-Approximation kann nicht mithilfe einer Orthogonalitätsbeziehung gezeigt werden. Wir können jedoch ein ganz allgemeines Approximationsresultat verwenden:

Satz 9.45 (Existenz der Tschebyscheff-Approximation) Es sei E ein Vektorraum mit Norm $\|\cdot\|$ und $S \subset E$ ein endlich-dimensionaler Teilraum. Dann existiert zu jedem $f \in E$ eine Bestapproximation $p \in S$ mit

$$\|f - p\| = \min_{q \in S} \|f - q\|.$$

Beweis Wir zeigen zunächst, dass eine Bestapproximation in der Menge

$$S_f := \{\phi \in S, \|\phi\| \leq 2\|f\|\}$$

liegen muss. Diese Menge ist nicht leer, natürlich liegt etwa die Nullfunktion $g \equiv 0 \in S_f$ in dieser Menge.

Angenommen, $g \in S$ mit $\|g\| > 2\|f\|$. Dann folgt mit der umgekehrten Dreiecksungleichung

$$\|g - f\| \geq \|g\| - \|f\| > \|f\| = \|f - 0\| \geq \inf_{q \in S} \|f - q\|,$$

d.h., $g \in S$ ist keine Bestapproximation. Die Teilmenge $S_f \subset S$ ist abgeschlossen, beschränkt, endlich-dimensional und daher kompakt. Auf S_f ist die Funktion

$$F(\phi) := \|f - \phi\|$$

stetig und nimmt als stetige Funktion auf der kompakten Menge S_f ein Minimum $p \in S_f$ an. Dieses Minimum ist die Bestapproximation. ◻

Zur Charakterisierung der Bestapproximation benötigen wir nun ein überprüfbares Kriterium. Viele der folgenden Sätze lassen sich auf allgemeine Vektorräume E und endlich-dimensionale Teilräume S übertragen. Wir gehen jedoch davon aus, dass $E = C[a, b]$ der Raum der stetigen Funktionen ist und $S = P_n$ der Raum der Polynome vom Grad bis n .

Angenommen, $p \in P_n$ sei eine Bestapproximation zu $f \in C[a, b]$. Dann definieren wir

$$e(x) = f(x) - p(x), \quad \mathbf{e} := \max_{[a,b]} |e(x)|,$$

und die Menge aller Punkte $x \in [a, b]$, in denen die Fehlerfunktion maximal wird:

$$E(f, p) := \{x \in [a, b] : |e(x)| = \mathbf{e}\}.$$

Es gilt:

Satz 9.46 (Satz von Kolmogoroff) Die Funktion $p \in P_n$ ist genau dann Bestapproximation zu $f \in C[a, b]$, falls das *Kolmogoroff Kriterium*

$$\forall q \in P_n : \min \{(f(x) - p(x))q(x) : \forall x \in E(f, p)\} \leq 0$$

gilt.

Beweis (i) Angenommen, p sei Bestapproximation und das Kolmogoroff-Kriterium gilt nicht. Dann existiert ein $q \in P_n$ und ein $\epsilon > 0$, sodass für $e(x) := f(x) - p(x)$ gilt:

$$\forall x \in E(f, p) : e(x)q(x) > 2\epsilon$$

Die Menge $E(f, p)$ ist kompakt. Daher existiert eine offene Menge U mit $E(f, p) \subset U$ und

$$\forall x \in U : (f(x) - p(x))q(x) > \epsilon.$$

Wir definieren $M := \|q\|_\infty$ und $p_1 := p + \lambda q$. Für $\lambda > 0$ folgt dann für $x \in U$

$$\begin{aligned} (f(x) - p_1(x))^2 &= (e(x) - \lambda q(x))^2 = e(x)^2 - 2\lambda e(x)q(x) + \lambda^2 q(x)^2 \\ &< \mathbf{e}^2 - 2\lambda\epsilon + \lambda^2 M^2. \end{aligned}$$

Für alle

$$0 < \lambda < \frac{\epsilon}{M^2}$$

gilt

$$(f(x) - p_1(x))^2 < \mathbf{e}^2 - \lambda\epsilon.$$

Es existiert also eine Funktion p_1 , die in der Umgebung U eine bessere Approximation zu f ist als p . Wir betrachten jetzt $x \in [a, b] \setminus U$. Diese Menge ist kompakt (U ist offen). Also existiert ein $\delta > 0$ mit

$$|e(x)| < \mathbf{e} - \delta \quad \forall x \in [a, b] \setminus U,$$

denn die Maxima werden gerade innerhalb der offenen Umgebung U angenommen. Für

$$\lambda < \frac{\delta}{2M}$$

gilt dann

$$|f(x) - p_1(x)| \leq |f(x) - p(x)| + \lambda |q(x)| \leq \mathbf{e} - \delta + \frac{\delta}{2M} M = \mathbf{e} - \frac{\delta}{2}.$$

Die Funktion p_1 ist also auf ganz $[a, b]$ eine bessere Approximation zu f . Dies ist ein Widerspruch und das Kolmogoroff-Kriterium muss demnach gelten.

(ii) Jetzt nehmen wir an, dass das Kolmogoroff-Kriterium für p gilt. Es sei $p_1 \in P_n$ beliebig und $q := p_1 - p$. Es existiert also mindestens ein $t \in E(f, p)$ mit

$$e(t)q(t) \leq 0.$$

Hieraus folgt

$$(f(t) - p_1(t))^2 = (e(t) - q(t))^2 = e(t)^2 - 2e(t)q(t) + q(t)^2 \geq e(t)^2 = \mathbf{e}^2,$$

d. h.

$$\|f - p_1\|_\infty \geq \|f - p\|_\infty \quad \forall p_1 \in P_n$$

und p ist die Bestapproximation. □

Die Eindeutigkeit der Bestapproximation in Polynomräumen kann mit dem Folgenden Kriterium nachgewiesen werden:

Satz 9.47 (Haarscher Eindeutigkeitssatz) Es sei $f \in C[a, b] \setminus P_n$ und P_n der Raum der Polynome bis zum Grad n und $p \in P_n$ die Bestapproximation zu f . Dann besitzt die Menge $E(f, p)$ mindestens $n + 2$ Punkte und die Bestapproximation ist eindeutig bestimmt.

Beweis (i) Angenommen, die Menge $E(f, p)$ habe nur $n + 1$ Punkte $x_0, x_1, \dots, x_n \in [a, b]$. Zu diesen $n + 1$ paarweise verschiedenen Punkten existiert die eindeutige Lagrange-Interpolation $q \in P_n$ mit

$$q(x_j) = e(x_j) = f(x_j) - p(x_j), \quad j = 0, \dots, n.$$

Wir überprüfen das Kolmogoroff-Kriterium (welches für die Bestapproximation prinzipiell gelten müsste)

$$e(x_j)q(x_j) = e(x_j)^2 = \mathbf{e}^2 > 0.$$

Das Kriterium ist nicht erfüllt, d. h., p kann keine Bestapproximation, also existieren mindestens $n + 2$ Punkte in $E(f, p)$.

(ii) Angenommen, p_1, p_2 seien zwei Bestapproximationen zu $f \in C[a, b]$. Dann gilt

$$\|f - \frac{1}{2}(p_1 + p_2)\|_\infty \leq \frac{1}{2}\|f - p_1\|_\infty + \frac{1}{2}\|f - p_2\|_\infty,$$

d. h., $p := (p_1 + p_2)/2$ ist auch Bestapproximation. Nach Teil (i) existieren mindestens $n + 2$ Punkte $x_0, \dots, x_{n+1} \in E(f, p)$ mit

$$|f(x_j) - p(x_j)| = \mathbf{e}, \quad j = 0, \dots, n + 1,$$

also

$$\left| \frac{1}{2}(f(x_j) - p_1(x_j)) + \frac{1}{2}(f(x_j) - p_2(x_j)) \right| = \mathbf{e}, \quad j = 0, \dots, n + 1.$$

Gleichzeitig gilt für die beiden Polynome p_1, p_2

$$|f(x_j) - p_i(x_j)| \leq \mathbf{e}, \quad i = 1, 2, \quad j = 0, \dots, n + 1.$$

Hieraus folgt

$$f(x_j) - p_1(x_j) = f(x_j) - p_2(x_j) \Rightarrow p_1(x_j) = p_2(x_j),$$

d. h. die beiden Polynome $p_i \in P_n$ stimmen in $n + 2$ Stellen überein, sind also identisch. Die Bestapproximation ist somit eindeutig. $\square$

Zur konkreten Bestimmung der Bestapproximation und zu ihrer weiteren Charakterisierung zitieren wir nun noch die folgende Verschärfung des Kolmogoroff-Kriteriums. Der Tschebyscheffsche Alternantensatz besagt, dass die maximalen Fehler mit alternierendem Vorzeichen angenommen werden:

Satz 9.48 (Tschebyscheffscher Alternantensatz) Es sei $f \in C[a, b]$ und $p \in P_n$ die Bestapproximation zu f . Genau dann existiert eine Sequenz aus $n + 2$ paarweise verschiedenen und aufsteigend sortierten Stützstellen $E(f, p) := (x_0, \dots, x_{n+1}) \subset [a, b]^{n+2}$ der sogenannten *Alternante*, so dass für den Fehler $e(x_i) = f(x_i) - p(x_i)$ gilt:

$$e(x_i) = -e(x_{i+1}), \quad i = 0, \dots, n,$$

insbesondere $|e(x_i)| = \mathbf{e} \in \mathbb{R}_+$ für alle $i = 0, \dots, n + 1$.

Beweis Der Haarsche Eindeutigkeitssatz besagt bereits, dass die Menge $E(f, p)$ mindestens $n + 2$ paarweise verschiedene Punkte besitzt, an denen gilt:

$$|e(x_j)| = \mathbf{e}, \quad j = 0, \dots, n + 1$$

Weiter gilt das Kriterium von Kolmogoroff in diesen $n + 2$ Punkten. Wir müssen nun noch zeigen, dass das Vorzeichen von $e(x_j)$ von Punkt zu Punkt wechselt. Angenommen, es gäbe nur n Vorzeichenwechsel, d. h. z. B.

$$\begin{aligned}
e(x_j) &= (-1)^j \mathbf{e}, & j = 0, \dots, i, \\
e(x_i) &= e(x_{i+1}) = (-1)^i \mathbf{e}, \\
e(x_j) &= (-1)^{j+1} \mathbf{e}, & j = i+2, \dots, n+1.
\end{aligned} \tag{9.25}$$

Wir versuchen dies zum Widerspruch zu bringen, indem wir eine Funktion $q_\alpha \in P_n$ konstruieren, die das Kriterium von Kolmogoroff nicht erfüllt. Dazu wählen wir das eindeutig bestimmte Polynom $q_\alpha \in P_n$ mit

$$\begin{aligned}
q_\alpha(x_j) &= e(x_j), & j = 0, \dots, i-1 \\
q_\alpha(x_i) &= \alpha e(x_i), & j = i \\
q_\alpha(x_j) &= e(x_j), & j = i+2, \dots, n+1,
\end{aligned}$$

mit einem beliebigen $\alpha > 0$. Dies sind $n+1$ Bedingungen für die $n+1$ Unbekannten des Polynomraums P_n , damit ist q_α eindeutig bestimmt. Der Punkt x_{i+1} wird ausgelassen. Nun seien $L_j^{(n)}$ für $j = 0, \dots, n$ die Lagrange-Polynome zu den $n+1$ Punkten

$$x_0, x_1, \dots, x_i, x_{i+2}, \dots, x_{n+1}.$$

Dann schreibt sich q als

$$q_\alpha(x) = \sum_{j=0}^{i-1} e(x_j) L_j^{(n)}(x) + \alpha e(x_i) L_i^{(n)}(x) + \sum_{j=i+2}^{n+1} e(x_j) L_{j-1}^{(n)}(x). \tag{9.26}$$

Hiermit gilt

$$e(x_j) q_\alpha(x_j) = \begin{cases} \mathbf{e}^2 & j = 0, \dots, i-1 \\ \alpha \mathbf{e}^2 & j = i, \\ \mathbf{X}_{i+1} & j = i+1, \\ \mathbf{e}^2 & j = i+2, \dots, n+1. \end{cases}$$

An der Stelle $j = i+1$ haben wir als Platzhalter $\mathbf{X}_{i+1}$ eingesetzt. Diese Stelle müssen wir noch berechnen. Hier gilt mit Darstellung (9.26)

$$q_\alpha(x_{i+1}) = \tilde{q}_\alpha(x_{i+1}) + \alpha e(x_i) L_i^{(n)}(x_{i+1}),$$

wobei $\tilde{q}(x) = q_\alpha(x) - \alpha e(x_i) L_i^{(n)}(x)$ ist, siehe (9.26). Das Lagrange-Basispolynom $L_i^{(n)}(x)$ kann zwischen x_i und x_{i+2} keinen Vorzeichenwechsel haben. Ansonsten hätte dieses $n+1$ Nullstellen und wäre selbst das Nullpolynom. Wegen der Annahme $e(x_i)e(x_{i+1}) > 0$, siehe (9.25), gilt

$$\mathbf{X}_{i+1} = e(x_{i+1}) q_\alpha(x_{i+1}) = e(x_{i+1}) \tilde{q}_\alpha(x_{i+1}) + \underbrace{\alpha e(x_{i+1}) e(x_i)}_{=\mathbf{e}^2 > 0} \underbrace{L_i^{(n)}(x_{i+1})}_{> 0}.$$

Für α groß genug folgt

$$\mathbf{X}_{i+1} = e(x_{i+1})q_\alpha(x_{i+1}) > 0$$

im Widerspruch zum Satz von Kolmogoroff. Es muss somit eine Alternante existieren. $\square$

Ist eine Alternante $\{x_0, \dots, x_{n+1}\}$ bekannt, so kann hieraus die Bestapproximation

$$p_n(x) = \sum_{i=0}^n \alpha_i x^i$$

bestimmt werden. Für diese gilt

$$\sum_{i=0}^n \alpha_i x_k^i + (-1)^k \alpha_{n+1} = f(x_k), \quad k = 0, \dots, n+1, \quad (9.27)$$

wobei der Koeffizient α_{n+1} gerade den maximalen Fehler angibt:

$$|\alpha_{n+1}| = \|f - p_n\| = \mathbf{e}$$

Durch (9.27) ist ein System von $n+2$ linearen Gleichungen in den $n+2$ Unbekannten $\alpha_0, \dots, \alpha_{n+1}$ gegeben.

Der Remez-Algorithmus

Der *Remez-Algorithmus* konstruiert eine Alternante. Denn falls eine Alternante $x_0, \dots, x_{n+1}$ bekannt ist, so können mithilfe von (9.27) die Koeffizienten $\alpha_0, \dots, \alpha_n$ der Bestapproximation

$$p(x) = \sum_{i=0}^n \alpha_i x^i$$

bestimmt werden. Der Remez-Algorithmus versucht, die Alternante iterativ zu ermitteln. Im Folgenden skizzieren wir das Vorgehen: Zunächst muss eine erste Schätzung für die Alternante $x_0^{(0)}, \dots, x_{n+1}^{(0)}$ vorgenommen werden. Als Möglichkeiten bieten sich gleich verteilte Stützstellen im Intervall oder die Nullstellen des entsprechenden Tschebyscheff-Polynoms an.

Wir gehen nun davon aus, dass eine Schätzung $A^{(l)} := \{x_0^{(l)}, \dots, x_{n+1}^{(l)}\}$ für die Alternante vorliegt. Der Übergang $l \mapsto l+1$ besteht aus vier Schritten:

1. Mithilfe von $A^{(l)}$ wird das lineare Gleichungssystem (9.27) gelöst. Die Lösung $\alpha_0^{(l)}, \dots, \alpha_{n+1}^{(l)}$ bestimmt das Polynom

$$p^{(l)}(x) = \sum_{j=0}^n \alpha_j^{(l)} x^j.$$

2. Man bestimme die Fehlerfunktion

$$e^{(l)}(x) = f(x) - p^{(l)}(x)$$

und die Menge der Extremstellen $y_0^{(l)}, \dots, y_m^{(l)}$ von $e^{(l)}$.

3. Falls $n + 2$ dieser Extremstellen die Alternanteneigenschaft approximativ erfüllen, so bricht die Iteration ab. Die Alternanteneigenschaft ist erfüllt, falls

$$\frac{|e^{(l)}(y_i)| - \|e^{(l)}\|_\infty}{\|e^{(l)}\|_\infty} < tol, \quad e^{(l)}(y_i) \cdot e^{(l)}(y_{i+1}) < 0.$$

4. Falls keine Alternante vorliegt, so wird aus der Menge der Extremstellen $y_0^{(l)}, \dots, y_m^{(l)}$ sowie der Menge der bisherigen Punkte $x_0^{(l)}, \dots, x_{n+1}^{(l)}$ eine neue Schätzung gewonnen. Dabei wird der neue Punkt $x_j^{(l+1)}$ als eine der Extremstellen $y_k^{(l)}$ gewählt, die möglichst nahe an $x_j^{(l)}$ liegt und die Vorzeichenbedingung

$$e^{(l)}(x_j^{(l+1)}) \cdot e^{(l)}(x_{j-1}^{(l+1)}) < 0$$

erfüllt. Falls dies nicht möglich ist, werden die alten Punkte $x_j^{(l)}$ weiterverwendet.

Beispiel 9.49 (Tschebyscheff-Approximation)

Wir betrachten die Funktion $f(x) = \sqrt{x}$ auf dem Intervall $[0, 1]$ und suchen mit dem Remez-Algorithmus die quadratische Tschebyscheff-Approximationen $p \in P_2$. Für $n = 2$ hat die gesuchte Alternante 4 Punkte und wir starten mit

$$A^{(0)} = \{0, 0.3, 0.7, 1\}.$$

Wir stellen das lineare Gleichungssystem (9.27) auf (Zahlen sind auf drei Stellen gerundet angegeben)

$$\begin{pmatrix} 1 & 0 & 0 & 1 \\ 1 & 0.3 & 0.09 & -1 \\ 1 & 0.7 & 0.49 & 1 \\ 1 & 1 & 1 & -1 \end{pmatrix} \begin{pmatrix} \alpha_0^{(0)} \\ \alpha_1^{(0)} \\ \alpha_2^{(0)} \\ \alpha_3^{(0)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0.548 \\ 0.837 \\ 1 \end{pmatrix}$$

mit der Lösung

$$\alpha^{(0)} = \begin{pmatrix} 0.0396 \\ 1.84 \\ -0.917 \\ -0.0397 \end{pmatrix} \Rightarrow p^{(0)}(x) = 0.0397 + 1.84x - 0.917x^2.$$

In Abb. 9.7 zeigen wir links die Approximation und rechts die Fehlerfunktion.

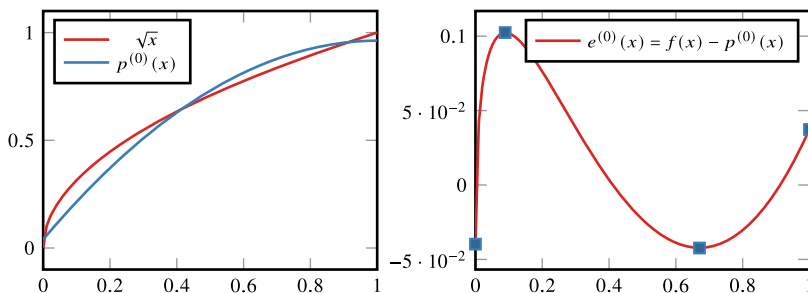


Abb. 9.7 Erste Approximation und Fehlerfunktion aus dem Remez-Algorithmus zur Tschebyscheff-Approximation von $\sqrt{x}$

Neben den beiden Randpunkten hat die Fehlerfunktion 2 innere Extremstellen

$$\xi_0^{(0)} = 0, \quad \xi_1^{(0)} \approx 0.0889, \quad \xi_2^{(0)} \approx 0.671, \quad \xi_3^{(0)} = 1.$$

In den Extremstellen gilt

$$\begin{aligned} f(\xi_0^{(0)}) - p^{(0)}(\xi_0^{(0)}) &\approx -0.0397, & f(\xi_1^{(0)}) - p^{(0)}(\xi_1^{(0)}) &\approx 0.102, \\ f(\xi_2^{(0)}) - p^{(0)}(\xi_2^{(0)}) &\approx -0.0423, & f(\xi_3^{(0)}) - p^{(0)}(\xi_3^{(0)}) &\approx 0.0373. \end{aligned}$$

Die Bedingung $e^{(0)}(\chi_i)e^{(0)}(\chi_{i+1}) < 0$ ist erfüllt, allerdings gilt nicht $|e^{(0)}(\chi_i)| \approx |e^{(0)}(\chi_{i+1})| \approx |\alpha_3^{(0)}|$, so dass wir die Alternante noch nicht akzeptieren. In der nächsten Iteration wählen wir daher

$$A^{(1)} = \{\xi_0^{(0)}, \xi_1^{(0)}, \xi_2^{(0)}, \xi_3^{(0)} = 1\} = \{0, 0.0894, 0.669, 1\}$$

Wir stellen wieder das lineare Gleichungssystem auf

$$\begin{pmatrix} 1 & 0 & 0 & 1 \\ 1 & 0.0889 & 0.00790 & -1 \\ 1 & 0.671 & 0.450 & 1 \\ 1 & 1 & 1 & -1 \end{pmatrix} \begin{pmatrix} \alpha_0^{(1)} \\ \alpha_1^{(1)} \\ \alpha_2^{(1)} \\ \alpha_3^{(1)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0.298 \\ 0.819 \\ 1 \end{pmatrix}$$

mit der Lösung

$$\alpha^{(0)} = \begin{pmatrix} 0.0669 \\ 1.94 \\ -1.08 \\ -0.0669 \end{pmatrix} \Rightarrow p^{(1)}(x) = 0.0669 + 1.94x - 1.08x^2,$$

die wir links in Abb. 9.8 zeigen, wohingegen rechts die Fehlerfunktion angegeben ist. Die Fehlerfunktion hat die Extremstellen

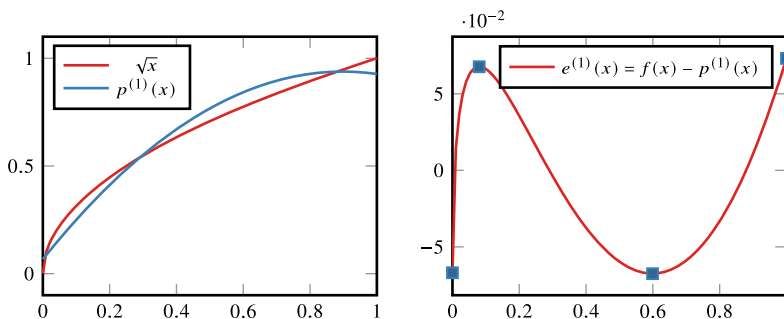


Abb. 9.8 Zweite Approximation und Fehlerfunktion aus dem Remez-Algorithmus zur Tschebyscheff-Approximation von $\sqrt{x}$

$$\xi_0^{(1)} = 0, \quad \xi_1^{(1)} \approx 0.0801, \quad \xi_2^{(1)} \approx 0.599, \quad \xi_3^{(1)} = 1.$$

und hier gilt

$$\begin{aligned} f(\xi_0^{(1)}) - p^{(1)}(\xi_0^{(1)}) &\approx -0.0669, & f(\xi_1^{(1)}) - p^{(1)}(\xi_1^{(1)}) &\approx 0.0677, \\ f(\xi_2^{(1)}) - p^{(1)}(\xi_2^{(1)}) &\approx -0.0675, & f(\xi_3^{(1)}) - p^{(1)}(\xi_3^{(1)}) &\approx 0.0731. \end{aligned}$$

Die Alternanteneigenschaft ist nun bereits sehr gut erfüllt und wir akzeptieren $p^{(1)}(x) = 0.0669 + 1.94x - 1.08x^2 \in P_2$ als die quadratische Tschebyscheff-Approximation. ◀

9.5.3 Optimale Lagrange-Interpolation

Abschließend wollen wir die Tschebyscheff-Approximation verwenden, um für eine Funktion $f \in C^{n+1}[a, b]$ die optimale Lagrange-Interpolation, also eine optimale Wahl der Stützstellen zu erstellen. Es gilt laut Satz 9.11 für die allgemeine Lagrange-Interpolation zu den paarweise verschiedenen Stützstellen $x_0, \dots, x_n$ die Abschätzung

$$f(x) - p_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

mit einem Zwischenwert ξ . Ziel ist nun die Bestimmung der Stützstellen $x_0, \dots, x_n$, sodass

$$\|L\|_{\infty, [a, b]} \rightarrow \min, \quad L(x) = \prod_{i=0}^n (x - x_i).$$

Diese Stützstellenwahl ist dann die optimale für allgemeine Funktionen $f \in C^{n+1}[a, b]$ und geht nicht auf die spezielle Funktion ein. Diese Aufgabe kann als Bestapproximationsaufgabe beschrieben werden. Es gilt

$$L(x) = x^{n+1} - g(x)$$

mit einem Polynom $g \in P_n$. Gesucht ist also die Bestapproximation $g \in P_n$ zur Funktion $f(x) = x^{n+1}$ im Intervall $I = [a, b]$. Aus Satz 9.48 wissen wir, dass die Fehlerfunktion $e(x) = f(x) - g(x)$ mindestens $n + 1$ Nullstellen hat (zwischen den $n + 2$ Extremstellen).

Satz 9.50 (Optimale Stützstellen der Lagrange-Interpolation) Auf dem Intervall $[-1, 1]$ ist die Tschebyscheff-Approximation $g \in P_n$ zu $f(x) = x^{n+1}$ gegeben durch

$$g(x) = x^{n+1} - 2^{-n} T_{n+1}(x)$$

mit dem Tschebyscheff-Polynom (siehe Satz 9.42)

$$T_{n+1}(x) = \cos((n+1) \arccos(x)).$$

Die Nullstellen von $T_{n+1}(x)$

$$x_k = \cos\left(\frac{\pi(2k+1)}{2(n+1)}\right), \quad k = 0, \dots, n,$$

sind die gesuchten *optimalen Stützstellen* der Lagrange-Interpolation auf $[-1, 1]$. Auf dem allgemeinen Intervall $[a, b]$ sind die optimalen Stützstellen gegeben durch

$$y_k = a + \frac{b-a}{2}(1+x_k).$$

Beweis Die Nullstellen der Tschebyscheff-Polynome sind einfach zu erkennen. Siehe auch Satz 9.42. Wie ebenso in dem Satz gezeigt wird, gilt die rekursive Beziehung

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x).$$

Damit folgt

$$T_{n+1}(x) = 2^n x^{n+1} + q(x)$$

mit einem $q \in P_n$. Andererseits ist T_{n+1} gegeben als Vielfaches der Linearfaktoren

$$2^{-n} T_{n+1}(x) = \prod_{k=0}^n (x - x_k) = L(x).$$

Hieraus folgt weiter

$$\max_{[-1,1]} |L(x)| = \max_{[-1,1]} \prod_{k=0}^n |x - x_k| = 2^{-n} \max_{[-1,1]} |T_{n+1}(x)| = 2^{-n}.$$

Die Funktion $T_{n+1}(x)$ nimmt im Intervall $[-1, 1]$ genau $n + 2$ -mal die Extremwerte ± 1 an. Diese $n + 2$ Extremstellen bilden eine Alternante für die eindeutige Bestapproximation zu $f(x) = x^{n+1}$

$$g(x) = x^{n+1} - 2^{-n} T_{n+1}(x) \in P_n.$$

Es bleibt zu zeigen, dass diese Bestapproximation wirklich das Produkt $|L(x)|$ in $[-1, 1]$ minimiert:

$$\begin{aligned} \max_{[-1,1]} |L(x)| &= 2^{-n} \max_{[-1,1]} |T_{n+1}(x)| \\ &= \max_{[-1,1]} \left| x^{n+1} - (x^{n+1} - 2^{-n} T_{n+1}(x)) \right| = \max_{[-1,1]} \left| x^{n+1} - g(x) \right| \\ &\leq \min_{p \in P_n} \max_{[-1,1]} \left| x^{n+1} - p(x) \right| \\ &\leq \min_{-1 \leq y_0 < \dots < y_n \leq 1} \max_{[-1,1]} \left| \prod_{k=0}^n (x - y_k) \right|, \end{aligned}$$

d. h., die Stützstellenwahl ist wirklich optimal. □

9.6 Trigonometrische Interpolation

Bisher haben wir uns ausschließlich mit der Interpolation in Polynomräumen befasst. In diesem Abschnitt betrachten wir die Interpolation mit trigonometrischen Polynomen, also Funktionen der Art

$$t_n(x) = \frac{1}{2} a_0 + \sum_{k=1}^m \left(a_k \cos\left(\frac{2\pi kx}{\omega}\right) + b_k \sin\left(\frac{2\pi kx}{\omega}\right) \right),$$

wobei $n = 2m$ und $a_k, b_k \in \mathbb{R}$ die reellen Koeffizienten sind. Für diese *trigonometrischen Polynome* gilt

$$t_n(x) = t_n(x + \omega),$$

d. h., $\omega > 0$ ist die Periode der Funktion $t_n(x)$. Die trigonometrische Interpolation eignet sich somit zur Interpolation von periodischen Funktionen oder Daten. Die aus der Analysis bekannte Fourier-Analyse ist keine Interpolation, sondern kann als Bestapproximation (siehe Abschn. 9.5) im Raum der trigonometrischen Funktionen betrachtet werden.

Im Folgenden betrachten wir ohne Einschränkung die Periode $\omega = 2\pi$, sodass sich die trigonometrischen Basispolynome vereinfacht schreiben lassen als

$$t_n(x) = \frac{1}{2} a_0 + \sum_{j=1}^m \left\{ a_j \cos(jx) + b_j \sin(jx) \right\}. \quad (9.28)$$

Wir untersuchen die Interpolationsaufgabe

$$t_n(x_k) = y_k, \quad k = 0, \dots, n.$$

Dabei betrachten wir äquidistant verteilte Stützstellen im Intervall $[0, 2\pi]$. Im Rahmen der trigonometrischen Interpolation wird sich dies als das typische Szenario erweisen:

$$x_k = \frac{2\pi k}{n+1}, \quad k = 0, \dots, n \quad (9.29)$$

Bei trigonometrischen Funktionen ist die Betrachtung komplexer Zahlen oft einfacher. Wir definieren für Koeffizienten $c_0, \dots, c_n \in \mathbb{C}$ die 2π -periodische Funktion

$$t_n^*(x) = \sum_{k=0}^n c_k e^{ikx}$$

mit der komplexen Einheit $i^2 = -1$. Es gilt:

Satz 9.51 (Komplexe trigonometrische Interpolation) Es seien $x_0, \dots, x_n \in [0, \pi)$ die $n+1$ paarweise verschiedenen äquidistanten Stützstellen und $y_0, \dots, y_n \in \mathbb{C}$ Stützwerte. Dann ist das trigonometrische Interpolationspolynom

$$t_n^*(x_i) = y_i, \quad i = 0, \dots, n$$

eindeutig bestimmt. Die Koeffizienten bestimmen sich als

$$c_k = \frac{1}{n+1} \sum_{j=0}^n y_j e^{-ijx_k}.$$

Beweis (i) Wir definieren

$$w := e^{ix}, \quad w_k := e^{ix_k}.$$

Dann schreibt sich das Polynom t_n^* als

$$t_n^*(x) = \sum_{k=0}^n c_k w^k$$

und die Interpolationsaufgabe als

$$t_n^*(x_k) = y_k, \quad k = 0, \dots, n.$$

Das Interpolationspolynom ist dann im Sinne der Lagrange-Interpolation eindeutig bestimmt.

(ii) Zur Berechnung der Koeffizienten beobachten wir, dass die w_k gerade die $n+1$ -ten Einheitswurzeln sind, denn

$$w_k^{n+1} = \exp(2\pi i k) = 1 \quad \forall k \in \mathbb{N}.$$

Weiter gilt

$$w_k^j = \exp\left(\frac{2\pi i j k}{n+1}\right) = w_j^k.$$

Dann ist

$$\begin{aligned} \sum_{j=0}^n y_j w_k^{-j} &= \sum_{j=0}^n t_n^*(x_j) w_k^{-j} = \sum_{j,l=0}^n c_l w_j^l w_k^{-j} \\ &= \sum_{l=0}^n c_l \sum_{j=0}^n w_j^{l-k} = \sum_{l=0}^n c_l \sum_{j=0}^n w_{l-k}^j. \end{aligned} \quad (9.30)$$

Für eine $(n+1)$ -te Einheitswurzel gilt

$$0 = w^{n+1} - 1 = (w-1)(w^n + w^{n-1} + \dots + 1),$$

also

$$\sum_{j=0}^n w_j^{l-k} = \begin{cases} 0 & l \neq k \\ n+1 & l = k \end{cases},$$

da $w_0 = 1$. Dann folgt mit (9.30) die Darstellung

$$\sum_{j=0}^n y_j w_k^{-j} = (n+1)c_k \Leftrightarrow c_k = \frac{1}{n+1} \sum_{j=0}^n y_j w_k^{-j}.$$

□

Aufgrund der periodischen Struktur der Basispolynome und der äquidistanten Verteilung der Stützstellen gilt:

$$\begin{aligned} c_{n+1-k} &= \frac{1}{n+1} \sum_{j=0}^n y_j e^{-i j x_{n+1-k}} = \frac{1}{n+1} \sum_{j=0}^n y_j e^{-\frac{2i\pi j(n+1-k)}{n+1}} \\ &= \frac{1}{n+1} \sum_{j=0}^n y_j e^{\frac{2i\pi j k}{n+1}} =: c_{-k} \end{aligned} \quad (9.31)$$

Der Bezug zwischen den komplexen Polynomen $t_n^*(x)$ und einer gesuchten reellen trigonometrischen Interpolation $t_n(x)$ gelingt auf Basis der Eulerschen Formel

$$e^{ix} = \cos(x) + i \sin(x).$$

Satz 9.52 (Reelle trigonometrische Interpolation) Für $n \in \mathbb{N}$ gibt es zu gegebenen reellen Zahlen $y_0, \dots, y_n$ genau ein trigonometrisches Polynom der Form

$$t_n(x) = \frac{1}{2}a_0 + \sum_{k=1}^m \left(a_k \cos(kx) + b_k \sin(kx) \right) + \frac{\theta}{2} a_{m+1} \cos((m+1)x),$$

mit $t_n(x_j) = y_j$ für $j = 0, \dots, n$ und

$$m = \begin{cases} \frac{n}{2} & n \text{ gerade} \\ \frac{n-1}{2} & n \text{ ungerade} \end{cases}, \quad \theta = \begin{cases} 0 & n \text{ gerade} \\ 1 & n \text{ ungerade} \end{cases}.$$

Die reellen Koeffizienten bestimmen sich als

$$a_k = \frac{2}{n+1} \sum_{j=0}^n y_j \cos(jx_k), \quad b_k = \frac{2}{n+1} \sum_{j=0}^n y_j \sin(jx_k).$$

Beweis (i) Es sei t_n^* das nach Satz 9.51 bestimmte komplexe Interpolationspolynom mit

$$t_n^*(x_j) = y_j, \quad j = 0, \dots, n.$$

Wir definieren mithilfe von (9.31) die Koeffizienten

$$a_k := c_k + c_{-k}, \quad b_k = i(c_k - c_{-k}), \quad k = 1, \dots, m, \\ a_{m+1} := 2c_{m+1}.$$

Die Koeffizienten sind reell, denn

$$c_k \pm c_{-k} = \frac{1}{n+1} \sum_{j=0}^n y_j \left((\cos(jx_k) - i \sin(jx_k)) \pm (\cos(jx_k) + i \sin(jx_k)) \right) \\ \Rightarrow a_k = \frac{2}{n+1} \sum_{j=0}^n y_j \cos(jx_k), \\ b_k = \frac{2}{n+1} \sum_{j=0}^n y_j \sin(jx_k).$$

Für $n = 2m + 1$ ungerade erhalten wir

$$m+1 = \frac{n+1}{2} = (n+1) - \frac{n+1}{2} = (n+1) - (m+1),$$

also $a_{m+1} = c_{m+1} + c_{-(m+1)} \in \mathbb{R}$. Wir definieren das Polynom

$$t_n(x) = \frac{a_0}{2} + \sum_{k=1}^m (a_k \cos(kx) + b_k \sin(kx)) + \frac{\theta}{2} a_{m+1} \cos((m+1)x).$$

(ii) Wir zeigen nun, dass

$$t_n(x_j) = t_n^*(x_j), \quad j = 0, \dots, n.$$

Es gilt

$$\begin{aligned} t_n(x_j) &= c_0 + \sum_{k=1}^m \left((c_k + c_{-k}) \cos(kx_j) + i(c_k - c_{-k}) \sin(kx_j) \right) \\ &\quad + \theta c_{m+1} \cos((m+1)x_j) \\ &= c_0 + \sum_{k=1}^m \left(c_k (\cos(kx_j) + i \sin(kx_j)) + c_{-k} (\cos(kx_j) - i \sin(kx_j)) \right) \\ &\quad + \theta c_{m+1} \left(\cos((m+1)x_j) + \underbrace{i \sin((m+1)x_j)}_{=0 \text{ für } n=2m+1 \text{ ungerade}} \right) \\ &= c_0 + \sum_{k=1}^m \left(c_k e^{ikx_j} + c_{-k} e^{-ikx_j} \right) + \theta c_{m+1} e^{i(m+1)x_j}, \end{aligned}$$

da $\cos(x) = \cos(-x)$ sowie $-\sin(x) = \sin(-x)$. Wir konnten im Fall $n = 2m + 1$ ungerade den Term

$$\sin((m+1)x_j) = \sin\left(\frac{n+1}{2} \frac{2\pi j}{n+1}\right) = \sin(\pi j) = 0, \quad j = 0, \dots, n,$$

einschieben. Mithilfe von (9.31), also $c_{-k} = c_{n+1-k}$ und

$$e^{-ikx_j} = e^{i\frac{2\pi kj}{n+1}} = e^{i\frac{2\pi(n+1-k)j}{n+1}} = e^{i(n+1-k)x_j}$$

folgt

$$t_n(x_j) = \sum_{k=0}^n c_k e^{ikx_j} y_j = 0,$$

d. h., die Interpolationsbedingung ist in allen Punkten erfüllt.

(iii) Es bleibt die Eindeutigkeit dieser Interpolation zu zeigen. Hier können wir wie üblich vorgehen: Zur Bestimmung der $n + 1$ Koeffizienten $a_0, \dots, a_n$ und $b_1, \dots, b_n$ sind $n + 1$ Bedingungen $t_n(x_j) = y_j$ zu erfüllen. Wir haben gezeigt, dass dieses Gleichungssystem für beliebige Werte $y_0, \dots, y_n$ lösbar ist. Die zugehörige $(n + 1) \times (n + 1)$ -Matrix hat somit vollen Rang, ist regulär. Die Interpolation ist somit eindeutig bestimmt. $\square$

Die trigonometrische Interpolation eignet sich zur Approximation periodischer Funktionen. Oft sind Funktionen nicht periodisch oder liegen überhaupt nur auf einem Intervall $[a, b]$ vor. In diesem Fall kann die Funktion zunächst auf das Intervall $[0, \pi]$ transformiert und dann periodisch fortgesetzt werden:

$$\bar{f}(x) = \begin{cases} f(x) & x \in [0, 2\pi] \\ f(x - 2j\pi) & x \in (2\pi j, 2\pi(j+1)) \end{cases}$$

Im Fall $f(x) \neq f(2\pi)$ hat die periodisch fortgesetzte Funktion an den Punkten $2\pi j$ Unstetigkeitsstellen. Hier definieren wir

$$f(2\pi j) = \frac{1}{2}(f(0) + f(2\pi)).$$

Diese Mittelwertbildung ist durch die Fourier-Entwicklung von Funktionen motiviert. Diese konvergiert an Unstetigkeitsstellen gerade gegen den Mittelwert der beidseitigen Grenzwerte.

Wir gehen nun davon aus, dass eine stetige Funktion $f \in C[0, \pi]$ mit $f(0) = f(\pi) = 0$ vorliegt. Diese kann einfach auf zwei Arten fortgesetzt werden. Die *gerade Fortsetzung* ist definiert als

$$\bar{f}(x) = \begin{cases} f(x) & x \in [0, \pi] \\ f(2\pi - x) & x \in [\pi, 2\pi] \\ 2\pi - \text{periodisch} & \text{sonst,} \end{cases} \quad (9.32)$$

die *ungerade Fortsetzung* als

$$\bar{f}(x) = \begin{cases} f(x) & x \in [0, \pi] \\ -f(2\pi - x) & x \in [\pi, 2\pi] \\ 2\pi - \text{periodisch} & \text{sonst.} \end{cases} \quad (9.33)$$

Für diese beiden ergeben sich Spezialfälle bei der Berechnung der Koeffizienten a_k und b_k :

Satz 9.53 (Kosinus- und Sinus-Transformation) Es sei $f \in C[0, \pi]$ eine Funktion mit $f(0) = f(\pi) = 0$. Für die trigonometrische Interpolation der geraden Fortsetzung

$$x_k = \frac{2\pi k}{n+1}, \quad y_k = \begin{cases} f(x_k) & k = 0, \dots, m, \\ f(2\pi - x_k) & k = m+1, \dots, n, \end{cases}$$

gilt

$$c(x) = \frac{1}{2}a_0 + \sum_{k=1}^{m+1} a_k \cos(kx), \quad a_k = \frac{2}{m+1} \sum_{j=0}^m f(x_j) \cos(jx_k).$$

Für die trigonometrische Interpolation der ungeraden Fortsetzung

$$x_k = \frac{2\pi k}{n+1}, \quad y_k = \begin{cases} f(x_k) & k = 0, \dots, m, \\ -f(2\pi - x_k) & k = m+1, \dots, n, \end{cases}$$

gilt

$$s(x) = \sum_{k=1}^m b_k \sin(kx), \quad a_k = \frac{2}{m+1} \sum_{j=0}^m f(x_j) \sin(jx_k).$$

Beweis Wir betrachten zunächst die komplexen Koeffizienten. Im Fall der geraden Fortsetzung gilt mit $y_k = y_{n+1-k}$

$$\begin{aligned} c_k &= \frac{1}{n+1} \sum_{j=0}^n y_j e^{-ijx_k} = \frac{1}{n+1} \sum_{j=0}^n y_{n+1-j} e^{-ijx_k} \\ &= \frac{1}{n+1} \sum_{j=0}^n y_j e^{-i(n+1-j)x_k} = c_{n+1-k} = c_{-k}. \end{aligned}$$

Also folgt

$$b_k = i(c_k - c_{-k}) = 0$$

sowie

$$\begin{aligned} a_k &= c_k + c_{-k} = 2c_k = \frac{2}{n+1} \sum_{j=0}^m y_j e^{-ijx_k} + \frac{2}{n+1} \sum_{j=m+1}^n y_j e^{-ijx_k} \\ &= \frac{2}{n+1} \sum_{j=0}^m y_j e^{-ijx_k} + \frac{2}{n+1} \sum_{j=1}^m \underbrace{y_{n+1-j}}_{=y_j} e^{-i(n+1-j)x_k} \\ &= \frac{2}{n+1} \sum_{j=0}^m y_j e^{-ijx_k} + \frac{2}{n+1} \sum_{j=1}^m y_j e^{ijx_k}, \end{aligned}$$

da

$$e^{-i(n+1-j)x_k} = e^{-i2\pi\left(1-\frac{j}{n+1}\right)k} = e^{-i2\pi k} e^{ijx_k} = e^{ijx_k},$$

wobei wir $e^{-i2\pi k} = 1$ im letzten Schritt ausgenutzt haben. Hiermit folgt (bei Berücksichtigung von $y_0 = y_{m+1} = 0$)

$$a_k = \frac{4}{n+1} \sum_{j=1}^m y_j \frac{e^{ijx_k} + e^{-ijx_k}}{2} = \frac{2}{m+1} \sum_{j=1}^m y_j \cos(jx_k).$$

Im Fall der Sinus-Transformation kann entsprechend vorgegangen werden. □

Wir beschließen diesen Abschnitt mit einem Beispiel, in dem wir eine 2π -periodische Funktion mithilfe der trigonometrischen Interpolation approximieren.

Beispiel 9.54

Es sei f eine 2π -periodische Funktion mit

$$f(x) = \begin{cases} -1 & x \in [0, \pi), \\ +1 & x \in [\pi, 2\pi). \end{cases}$$

Die Funktion ist in Abb. 9.9 dargestellt.

Wir wenden nun Satz 9.52 mit äquidistanter Wahl der Stützstellen x_j gemäß (9.29) an. Im Folgenden berechnen wir bis $n = 3$ die trigonometrischen Polynome $t_n(x)$. Unsere Ergebnisse listen wir zunächst in Tab. 9.2 auf und die zugehörige Visualisierung finden wir in Abb. 9.9. ◀

Tab. 9.2 Trigonometrische Polynominterpolation zu Beispiel 9.54

n	m	θ	x_j	y_j	$t_n(x)$	a_k	b_k
0	0	0	0	-1	$\frac{1}{2}a_0$	$a_0 = -2$	
1	0	1	0	-1		$a_0 = 0$	
			π	+1	$\frac{1}{2}a_0 + \frac{1}{2}a_1 \cos(x)$	$a_1 = -2$	
2	1	0	0	-1		$a_0 = -\frac{2}{3}$	
			$\frac{2}{3}\pi$	-1		$a_1 = -\frac{2}{3}$	
			$\frac{4}{3}\pi$	+1	$\frac{1}{2}a_0 + a_1 \cos(x) + b_1 \sin(x)$		$b_1 = -1.1547$
3	1	1	0	-1		$a_0 = 0$	
			$\frac{1}{2}\pi$	-1		$a_1 = -1$	
			π	+1			$b_1 = -1$
			$\frac{3}{2}\pi$	+1	$\frac{1}{2}a_0 + a_1 \cos(x) + b_1 \sin(x) + \frac{1}{2}a_2 \cos(2x)$	$a_2 = 0$	

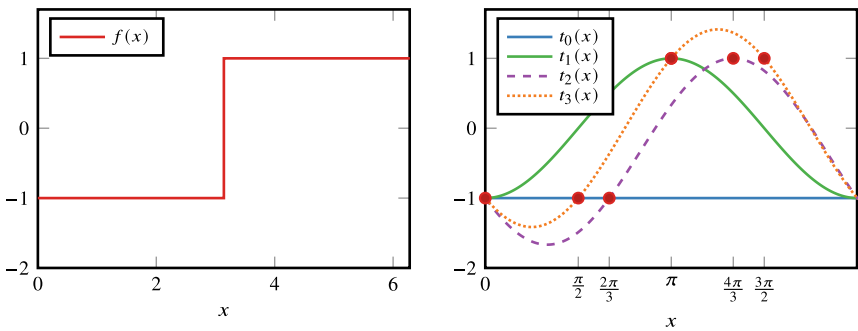


Abb. 9.9 Beispiel 9.54. Links: Die Funktion f soll mithilfe der trigonometrischen Interpolation dargestellt werden. Rechts: Approximation der Funktion f mithilfe der trigonometrischen Polynome $t_0(x), \dots, t_3(x)$, welche in Tab. 9.2 berechnet wurden. Wir sehen sofort, dass die Interpolationsbedingungen $t_n(x_j) = y_j$ jeweils erfüllt sind

9.7 Diskrete Fourier-Transformation

Die Transformation von Stützwerten $y_0, \dots, y_n$ zu Koeffizienten a_k, b_k heißt *diskrete Fourier-Transformation* bzw. im Fall einer geraden oder ungeraden Fortsetzung *diskrete Sinus-Transformation* und *diskrete Kosinus-Transformation*. Im diesem Abschnitt werden wir mit der *schnellen Fourier-Transformation*, im Englischen *Fast Fourier Transformation (FFT)*, einen sehr schnellen Algorithmus zur Berechnung der Koeffizienten a_k und b_k der Transformation herleiten.

Wir haben im vorherigen Abschnitt gesehen, dass dieser Prozess umkehrbar ist, es existieren bijektive Abbildungen

$$\{y_0, \dots, y_n\} \leftrightarrow \{a_0, \dots, a_{m+1}, b_1, \dots, b_m\}$$

sowie

$$\{y_0, \dots, y_m\} \leftrightarrow \begin{cases} \{a_0, \dots, a_{m+1}\} & \text{gerade Fortsetzung} \\ \{b_1, \dots, b_m\} & \text{ungerade Fortsetzung.} \end{cases}$$

Die Basispolynome $\cos(kx)$ sowie $\sin(kx)$ können als die wesentlichen Schwingungen der periodischen Funktion $f(x)$ betrachtet werden. Die Koeffizienten a_k, b_k geben die jeweilige Dominanz der Frequenzen an. Die diskrete Fourier-Transformation dient einerseits zur Analyse von Daten. Aus Messdaten können z. B. die dominanten Schwingungen von mechanischen Strukturen ermittelt werden. Auf der anderen Seite ist die Fourier-Transformation die Grundlage für viele Kompressionsverfahren.

Das *JPEG*-Format für Bilder baut z. B. auf der Kosinus-Transformation auf, siehe Exkurs 9.8.

Zur Herleitung der *schnellen Fourier-Transformation* wählen wir die komplexe Schreibweise, bei der zunächst die komplexen Werte

$$c_k = \sum_{j=0}^n \bar{y}_j w^{jk}, \quad k = 0, \dots, n, \quad (9.34)$$

mit den Abkürzungen

$$\bar{y}_j := \frac{1}{n+1} y_j, \quad w := e^{-i \frac{2\pi}{n+1}},$$

zu berechnen sind. Aus diesen bestimmen sich a_k und b_k als

$$a_k = c_k + c_{-k}, \quad b_k = i(c_k - c_{-k}). \quad (9.35)$$

Die Berechnung der $n+1$ Werte c_k mit dem Horner-Schema würde $n^2 + O(n)$ Operationen benötigen. Eine effizientere Methode kann durch eine hierarchische Aufteilung der Summen erreicht werden. Für das Folgende sei $n+1 = 2^p$ für $p \in \mathbb{N}$. Dann ist

$$\begin{aligned}
c_k &= \sum_{j=0}^{2^p-1} \bar{y}_j w^{jk} \\
&= \sum_{j=0}^{2^{p-1}-1} \left(\bar{y}_{2j} (w^2)^{jk} + \bar{y}_{2j+1} (w^2)^{jk} w^k \right) \\
&= \sum_{j=0}^{2^{p-1}-1} \bar{y}_{2j} (w^2)^{jk} + \sum_{j=0}^{2^{p-1}-1} \bar{y}_{2j+1} (w^2)^{jk} w^k.
\end{aligned} \tag{9.36}$$

Nun sei $k_1 \in \{0, 1, \dots, 2^{p-1} - 1\}$ der ganzzahlige Rest bei Division von k durch 2^{p-1} , also

$$k \equiv k_1 \pmod{2^{p-1}}.$$

Wegen $w^{n+1} = w^{2^p} = 1$ folgt für ein $\lambda \in \mathbb{N}$

$$(w^2)^{jk} = (w^2)^{\lambda 2^{p-1} j} (w^2)^{jk_1} = (w^{2^p})^j (w^2)^{jk_1} = (w^2)^{jk_1}.$$

Hiermit lässt sich (9.36) schreiben als

$$c_k = \sum_{j=0}^{2^{p-1}-1} \bar{y}_{2j} (w^2)^{jk_1} + w^k \sum_{j=0}^{2^{p-1}-1} \bar{y}_{2j+1} (w^2)^{jk_1}$$

oder aber

$$c_k = \tilde{c}_{k_1} + w^k \bar{c}_{k_1}, \quad k = 0, \dots, 2^p - 1, \quad k \equiv k_1 \pmod{2^{p-1}},$$

wobei

$$\tilde{c}_{k_1} = \sum_{j=0}^{2^{p-1}-1} \bar{y}_{2j} (w^2)^{jk_1}, \quad \bar{c}_{k_1} = \sum_{j=0}^{2^{p-1}-1} \bar{y}_{2j+1} (w^2)^{jk_1}, \quad k_1 = 0, \dots, 2^{p-1} - 1.$$

Zur Berechnung der $n+1 = 2^p$ Größen $c_0, \dots, c_n$ genügt es also, die zweimal 2^{p-1} Größen $\tilde{c}_0, \dots, \tilde{c}_{2^{p-1}-1}$ und $\bar{c}_0, \dots, \bar{c}_{2^{p-1}-1}$ zu berechnen. Die $n+1 = 2^p$ -dimensionale Fourier-Transformation wurde durch zwei Fourier-Transformationen der Dimension 2^{p-1} ersetzt. Würden wir diese Koeffizienten jeweils mit dem Horner-Schema bestimmen, so hätten wir bereits den Aufwand auf die Hälfte reduziert, da die quadratische Laufzeit nun weniger zum Tragen kommt:

$$2 \cdot \left(\frac{n}{2}\right)^2 = \frac{n^2}{2}$$

Der FFT-Algorithmus basiert nun auf rekursiver Anwendung dieser Technik. Die zwei Fourier-Transformationen der Dimension 2^{p-1} werden durch vier Transformationen der Dimension 2^{p-2} ersetzt. Schließlich ergeben sich 2^p Fourier-Transformationen der trivialen

Dimension 1. Diese können ohne weiteren Rechenaufwand durch Zuweisung der Funktionswerte $\bar{y}_j$ „gelöst“ werden. Es gilt:

Satz 9.55 (Schnelle Fourier-Transformation) Im Fall $n + 1 = 2^p$ berechnet die *schnelle Fourier-Transformation* die $n + 1$ Koeffizienten $c_0, \dots, c_n$ mit

$$2(n + 1) \log_2(n + 1)$$

komplexen Operationen.

Beweis Angenommen, durch r_p sei der Aufwand gegeben, die $n + 1 = 2^p$ Komponenten zu berechnen. Sind $\tilde{c}_{k_1}, \bar{c}_{k_1}$ bekannt, so sind hierfür die Potenzen w^k und die Summen zu berechnen:

$$c_k = \tilde{c}_{k_1} + w^k \bar{c}_{k_1}, \quad k = 0, \dots, n$$

Hierzu sind $(n + 1) + (n - 1) = 2n \leq 2 \cdot 2^p$ komplexe Operationen notwendig. Iterativ ergibt sich also

$$r_p \leq 2 \cdot r_{p-1} + 2 \cdot 2^p, \quad p = 1, 2, \dots$$

Wir zeigen per Induktion $r_p \leq 2p \cdot 2^p$. Wir starten mit $r_0 = 0$. Dann ist

$$\begin{aligned} r_p &\leq 2 \cdot r_{p-1} + 2 \cdot 2^p = 2 \cdot (2(p-1) \cdot 2^{p-1}) + 2 \cdot 2^p \\ &= 2p \cdot 2^p = 2(n + 1) \log_2(n + 1). \end{aligned}$$

□

Bei der praktischen Durchführung der schnellen Fourier-Transformation startet man mit den 2^p atomaren Koeffizienten, welche gerade durch die Stützwerte y_j gegeben sind. Im Anschluss werden die rekursiven Formeln

$$c_k = \tilde{c}_{k_1} + \bar{c}_{k_1}, \quad k = 0, \dots, 2^p - 1, \quad k \equiv k_1 \bmod 2^{p-1},$$

durchgeführt. Bei $n + 1 = 1024$ benötigt die schnelle Fourier-Transformation 20 480 komplexe Operationen. Auf der Basis des Horner-Schemas würden 1 000 000 komplexe Operationen benötigt. Die schnelle Fourier-Transformation ist eines der wenigen Verfahren, welche zur Lösung der Aufgabe – falls sie anwendbar sind – ausschließlich Vorteile mit sich bringen. Dies ist selten. Die meisten „fortgeschrittenen“ numerischen Methoden bringen gewisse Nachteile mit sich, z.B. höhere Anforderungen an die Regularität (bei der Gauß-Quadratur) oder geringere Robustheit (bei der Newton-Iteration bei schlechten Startwerten). Als ein weiteres Verfahren dieser Art stellt sich das Mehrgitterverfahren zur Lösung dünn besetzter Gleichungssysteme heraus. Kann es angewendet werden, so wird eine feste Fehlerreduktion in linearem Aufwand $O(n)$ erreicht.

9.8 Exkurs: Das JPEG-Format zur Komprimierung von Bildern

Eines der gebräuchlichsten Formate zur effizienten Speicherung von digitalen Bildern ist das *JPEG*²-Dateiformat [84]. Der mathematische Hintergrund des Dateiformats ist die *diskrete Kosinus-Transformation*, siehe Satz 9.53 und Abschn. 9.7. Die Bilddaten werden mithilfe der schnellen Kosinus-Transformation in ihre Frequenzanteile transformiert. Zur Kompression werden dann nicht alle Frequenzen, sondern – je nach gewünschter Qualität – stets nur die dominanten Anteile gespeichert. Das JPEG-Bild ergibt sich dann als Rücktransformation dieser reduzierten Daten. Das JPEG-Format ist ein *verlustbehaftetes Kompressionsverfahren*. In Abb. 9.10 zeigen wir ein Beispielbild und einen kleinen Ausschnitt der Größe 256×256 Punkte. Der Ausschnitt ist in vier verschiedenen Qualitätsstufen dargestellt: 100 % (dies entspricht einer verlustfreien Darstellung), 50 %, 12 % und 6 %. Man sieht zunehmend Artefakte in der Darstellung. Insbesondere ist eine Blockstruktur des Bildes zu erkennen. Wir werden später die Ursachen erkennen.

Auch wenn der mathematische Kern des JPEG-Formats die Kosinus-Transformation ist, so sind viele Zwischenschritte erforderlich, um eine effiziente, also sparsame Darstellung, die gleichzeitig eine hohe Qualität hat, zu erzeugen. Wir stellen die einzelnen Schritte im Folgenden detailliert dar:

1. Anpassung des Farbraums und *Subsampling*. Das Bild wird in drei Kanäle zur Darstellung von Helligkeit und Farbe zerlegt. Die einzelnen Kanäle werden im Anschluss wie Schwarz-Weiß-Bilder behandelt.



Abb. 9.10 Originalbild (links) und Ausschnitt (rechts) mit verschiedenen Kompressionsstärken. Dateigrößen 100 kB, 26 kB, 18 kB und 14 kB (von links oben nach rechts unten)

² Joint Photographic Expert Group wurde 1992 in der Norm ISO/IEC 10918-1 vorgestellt.

2. Blockbildung und Kosinus-Transformation. Die einzelnen Kanäle werden in Blöcke der Größe 8×8 zerlegt. In jedem dieser Blöcke wird die diskrete Kosinus-Transformation durchgeführt.
3. Quantisierung. Dies ist die verlustbehaftete Kompression, es werden nicht alle Frequenzen gleichermaßen gespeichert.
4. Weitere Komprimierung durch Kodierung. Das Ergebnis wird durch weitere Kodierungsverfahren komprimiert (aber verlustfrei) gespeichert.

Farbräume

Die Darstellung von s/w-Bildern (Schwarz-Weiß) ist auf einem Computer sehr einfach. Ein Bild der Größe $X \times Y$ besteht aus $X \cdot Y$ Werten für die Helligkeit der einzelnen Punkte. Die *Farbtiefe* beschreibt, wie viele Stufen zur Darstellung der Helligkeit eines jeden Punktes zur Verfügung stehen. Übliche Werte sind 8 Bit oder 16 Bit. Wir beschränken uns hier auf Farbtiefen von 8 Bit. Es lassen sich also für jeden Punkt 256 verschiedene Helligkeiten angeben, siehe Abschn. 1.4. Dabei kann zum Beispiel der Wert 0 für Schwarz, der Wert 255 für Weiß stehen. Die Qualität eines Bildes hängt somit einerseits von der Auflösung, also der Anzahl der Bildpunkte $X \cdot Y$, auf der anderen Seite von der Farbtiefe ab.

Um farbige Bilder darzustellen, existieren verschiedene Farbräume. Das gebräuchlichste Modell ist der *RGB-Farbraum*. Hier werden für die drei Farben Rot, Grün und Blau jeweils Intensitätsanteile (entsprechend der Helligkeit) gespeichert. Das eigentliche Bild ergibt sich dann durch additive Mischung dieser drei Anteile. Dieses Farbmodell ist üblich für die Darstellung auf Monitoren. Ein RGB-Bild mit Auflösung 800×800 und Farbtiefe 8 Bit lässt sich also in Form von drei Feldern beschreiben:

```
int8 R[640000], G[640000], B[640000];
```

Hierbei bezeichnet `int8` einen Datentyp für eine ganze Zahl mit 8 Bit. Ohne Kompression werden 1 920 000 Byte, also etwa 2 Megabyte zur Speicherung benötigt. Die JPEG-Komprimierung beruht auf der Analyse von *monochromen* Bildern. Eine Variante zur Kodierung wäre also eine getrennte Anwendung des Verfahrens auf die drei Farbkanäle Rot, Grün und Blau. Stattdessen wird das Bild jedoch zunächst in einen anderen Farbraum, in der *Y'CbCr*-Modell transformiert. Auch dieses Modell beruht auf drei Kanälen, dem *Y'*-Kanal, der die Helligkeit (Luminanz) angibt, und den *Cb*- und *Cr*-Kanälen, welche die Chrominanz, also die Farbigkeit bzgl. der Blau-Gelb- und Rot-Grün-Wertigkeit angeben. In Abb. 9.11 zeigen wir die Zerlegung eines RGB-Bildes in diese drei Kanäle. Die einzelnen Kanäle sind wieder monochrom, sodass das Bild (verlustfrei) in der folgenden Form gespeichert werden kann.

```
int8 Y[640000], Cb[640000], Cr[640000];
```

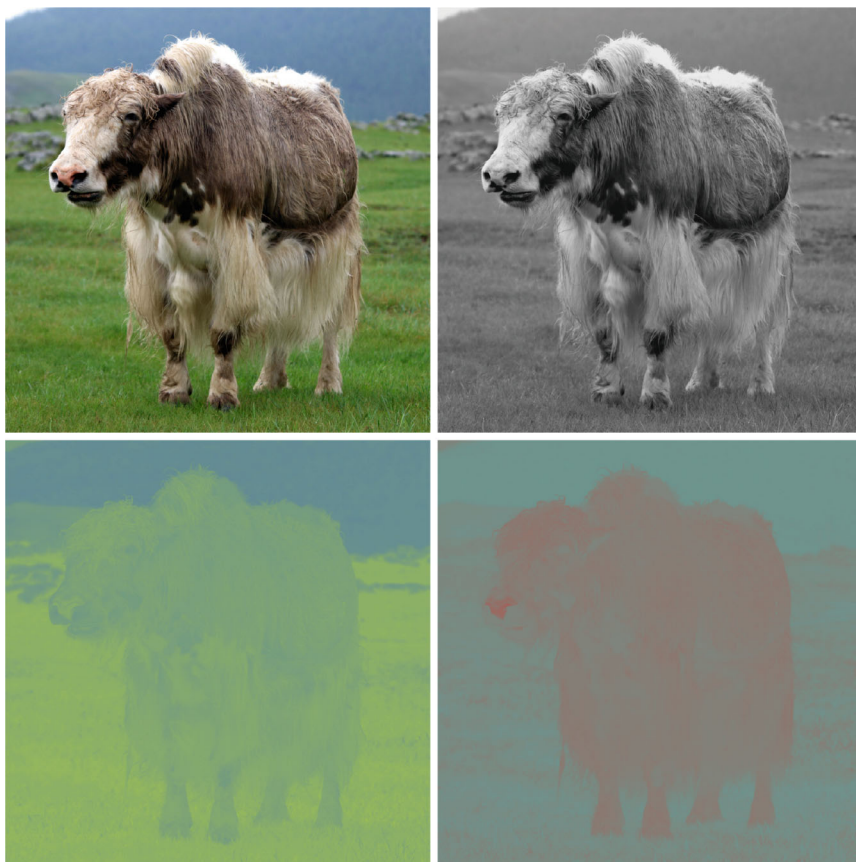


Abb. 9.11 Originalbild (links oben) und Darstellung im $Y'CbCr$ -Farbraum. Rechts oben ist die Helligkeit (Luminanz) Y' angegeben, links unten die Blau-Gelb-Chrominanz (Farbigkeit) Cb und rechts unten die Rot-Türkis-Chrominanz Cr

Falls die Rot/Grün/Blau-Werte im Bereich $\{0, 255\}$ vorliegen, so lässt sich das Bild durch eine einfache lineare Transformation in den $Y'CbCr$ -Farbraum transformieren:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} \approx \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix} + \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.168736 & -0.331264 & 0.5 \\ 0.5 & -0.418688 & -0.081312 \end{bmatrix} \cdot \begin{bmatrix} R \\ G \\ B \end{bmatrix} \quad (9.37)$$

Bis auf die Rundung ist diese Transformation verlustfrei. Der Grund für diese Transformation findet sich in der Wahrnehmung des menschlichen Auges: Die Auflösung für Helligkeitsunterschiede ist im Auge weit ausgeprägter als die Auflösung für die Wahrnehmung von Farbunterschieden. Für eine gute Darstellung eines Bildes ist der Y' -Kanal somit wichtiger als die Cb - und Cr -Kanäle.

An dieser Stelle greift der erste Schritt der JPEG-Komprimierung, das sogenannte *Subsampling*: In den Feldern für *Cb* und *Cr* wird horizontal und vertikal nur jede zweite Information gespeichert.³ Dies wird *4 : 2 : 0-Subsampling* genannt und es sind bereits weit weniger Informationen zu speichern:

```
int8 YY[640000], Cb[160000], Cr[160000];
```

Insgesamt wird der Speicherbedarf um den Faktor 2 auf etwa 1 Megabyte reduziert. Das menschliche Auge kann diese Unterschiede normalerweise nicht wahrnehmen.

Diese drei *monochromen* Felder dienen nun separat als Grundlage für die folgende eigentliche JPEG-Komprimierung. Wir werden diese daher auch nur anhand von Grautönen beschreiben, unabhängig davon, ob die Intensität eine Helligkeit oder einen Farbwert angibt.

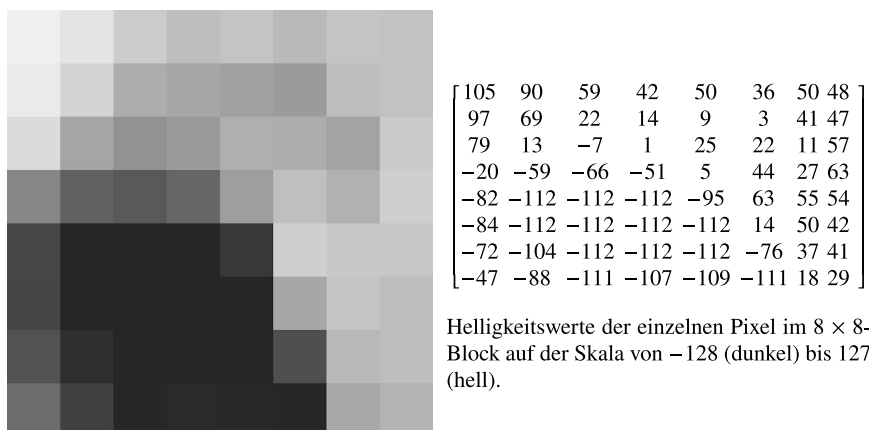
Blockbildung und Kosinus-Transformation

Wir gehen nun davon aus, dass ein monochromes Intensitätsfeld mit Farbtiefe 8 Bit und Auflösung $x \times y$ vorliegt. In einem ersten Schritt wird dieses Feld in Blöcke der Größe 8×8 zerlegt. Wir gehen davon aus, dass sowohl Bildbreite x als auch Bildhöhe y ein Vielfaches von 8 sind. Im Fall der Farbigkeiten *Cb* und *Cr* bedeutet dies aufgrund des *Subsamplings*, dass Bildbreite und Bildhöhe des Originalbildes Vielfache von 16 sein müssen. An dieser Stelle verweisen wir noch einmal auf Abb. 9.10. Die Artefakte bei der Speicherung mit niedriger Qualität zeigen gerade diese Blockung. Falls sich das Originalbild nicht in Blöcke der Größe 8×8 zerlegen lässt, so sind am Rand weitere Anpassungen erforderlich, siehe [84]. In Abb. 9.12 zeigen wir einen solchen Ausschnitt der Größe 8×8 aus dem *Y*-Kanal, also der Helligkeit des Bildes. Wir geben auch die Intensitäten im Bereich $\{-128, 127\}$ als Matrix **B** an. Die Verschiebung der diskreten Werte um den Mittelpunkt 0 erleichtert die spätere diskrete Kosinus-Transformation.

Im folgenden Schritt wird eine diskrete Kosinus-Transformation der Intensitätswerte durchgeführt. Wir erinnern hier an Satz 9.53 und die diskrete Variante in Abschn. 9.7. Wir wollen die Matrix $\mathbf{B} \in \mathbb{R}^{8 \times 8}$ mithilfe einer zweidimensionalen Kosinus-Transformation darstellen. Hierzu wählen wir die Form

$$\mathbf{B}_{ij} = \frac{1}{4} \sum_{u,v=1}^8 \alpha(u)\alpha(v)\mathbf{G}_{uv} \cos\left(\frac{2i-1}{16}u\pi\right) \cos\left(\frac{2j-1}{16}v\pi\right) \quad (9.38)$$

³ Der JPEG-Standard sieht hier verschiedene Raten für das Subsampling vor. Die Rate $4 : 4 : 4$ bedeutet kein Subsampling, also volle Informationsmitnahme, die übliche Rate $4 : 2 : 0$ bedeutet horizontale und vertikale Reduktion der *Cb*- und *Cr*-Informationen um den Faktor 2 und die Rate $4 : 2 : 2$ bedeutet eine horizontale Reduktion der Farbraten um den Faktor 2, jedoch volle Informationsmitnahme in der vertikalen Richtung. Diese Kompression ist bei verschiedenen Videosystemen üblich, siehe [84].



Helligkeitswerte der einzelnen Pixel im 8×8 -Block auf der Skala von -128 (dunkel) bis 127 (hell).

Abb. 9.12 Block der Größe 8×8 für die Helligkeit, also den Y -Wert. Ausschnitt aus Bild 9.10. Rechts sind die Wertigkeiten für die einzelnen Punkte des 8×8 -Blocks im Bereich $\{-128, 127\}$ angegeben

mit den Faktoren

$$\alpha(u) = \begin{cases} \frac{1}{\sqrt{2}} & u = 1 \\ 1 & 1 < u \leq 8. \end{cases}$$

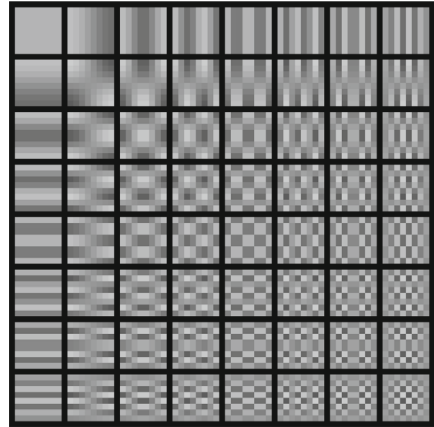
Die spezielle Form der Basis (9.38) ergibt sich aus der gewählten Symmetrie der geraden Fortsetzung an den Punkten $\frac{1}{2}$ sowie $8 + \frac{1}{2}$, die beiden Endpunkte werden also bei der Fortsetzung verdoppelt. In Abb. 9.13 zeigen wir die 64 verschiedenen Basisfunktionen der diskreten Kosinus-Transformation. Es gilt:

Satz 9.56 (Diskrete zweidimensionale Kosinus-Transformation) Es sei $\mathbf{B} \in \mathbb{R}^{8 \times 8}$. Für die Koeffizientenmatrix $\mathbf{G} \in \mathbb{R}^{8 \times 8}$ der diskreten zweidimensionalen Kosinus-Transformation gilt

$$\mathbf{G}_{uv} = \frac{1}{4} \alpha(u) \alpha(v) \sum_{i=1}^8 \sum_{j=1}^8 \mathbf{B}_{ij} \cos\left(\frac{2i-1}{16} u \pi\right) \cos\left(\frac{2j-1}{16} v \pi\right).$$

Beweis Der Beweis wird aufgrund der Tensorproduktform zunächst auf den eindimensionalen Fall zurückgeführt. Einsetzen der Koeffizienten ergibt

Abb. 9.13 Die 8×8 Basisfunktionen der zweidimensionalen Kosinus-Transformation. Jedes Feld aus 8×8 Werten lässt sich aus diesen Basiselementen linear kombinieren. Das JPEG-Format baut auf dem Prinzip auf, dass das menschliche Auge verschiedene Basisfunktionen verschieden gut unterscheiden kann



$$\mathbf{B}_{ij} \stackrel{!}{=} \sum_{k,l=1}^8 \mathbf{B}_{kl} \left(\sum_{u=1}^8 \frac{\alpha(u)^2}{4} \cos\left(\frac{2k-1}{16}u\pi\right) \cos\left(\frac{2i-1}{16}u\pi\right) \right) \cdot \left(\sum_{v=1}^8 \frac{\alpha(v)^2}{4} \cos\left(\frac{2l-1}{16}v\pi\right) \cos\left(\frac{2j-1}{16}v\pi\right) \right).$$

Das Ergebnis folgt dann mit der Orthogonalität der Kosinusfunktion. $\square$

Für das obige Beispiel gilt für die Transformation von $\mathbf{B}$

$$\mathbf{G} \approx \begin{bmatrix} -110 & -203 & 214 & 45.8 & 4.87 & 33.5 & 6.42 & -2.63 \\ 365 & 187 & -72.5 & 26.1 & 25.1 & -44.9 & 10.6 & -2.22 \\ 58.6 & 139 & 24.9 & -60.5 & 24.9 & 12.7 & -60.1 & 35.5 \\ -16.7 & -63.2 & -0.407 & 10.9 & -65.5 & 17 & 14.5 & -22.7 \\ -1.38 & -30 & -17.8 & 12.6 & -7.38 & -10.9 & 6.41 & -0.757 \\ 21.4 & 1.96 & -24.8 & -4.62 & 24.1 & -13.8 & -1.85 & 15.6 \\ 21 & 13.9 & -11.8 & 9.18 & 7.43 & -2.24 & 13.1 & -3.74 \\ -2.13 & -0.791 & 7.58 & 2.12 & -2.35 & 5.35 & 8.92 & -3.74 \end{bmatrix}, \quad (9.39)$$

und eine Rücktransformation gemäß Satz 9.56 würde wieder die ursprüngliche Matrix $\mathbf{B}$, also die Intensitäten des Bildausschnitts der Größe 8×8 ergeben. Bis zu diesem Punkt wird keine Datenkompression erreicht. Die Speicherung der Koeffizientenmatrix $\mathbf{G}$ ist genauso aufwendig wie die Speicherung der Intensitäten $\mathbf{B}$. Da die Koeffizienten nicht ganzzahlig sind, ist sogar von größerem Speicheraufwand auszugehen.

Quantisierung

Eine weitere wesentliche Idee der JPEG-Komprimierung ist die Beobachtung, dass das menschliche Auge hochfrequente Unterschiede nur schwer wahrnehmen kann. Daher werden die ermittelten Koeffizienten, bei uns die Matrix (9.39), je nach Frequenz mit einem unterschiedlichen Faktor gewichtet, dies ist die sogenannte *Quantisierung*. Eine übliche Quantisierungsmatrix ist:

$$\mathbf{Q} = \begin{bmatrix} 10 & 15 & 25 & 37 & 51 & 66 & 82 & 100 \\ 15 & 19 & 28 & 39 & 52 & 67 & 83 & 101 \\ 25 & 28 & 35 & 45 & 58 & 72 & 88 & 105 \\ 37 & 39 & 45 & 54 & 66 & 79 & 94 & 111 \\ 51 & 52 & 58 & 66 & 76 & 89 & 103 & 119 \\ 66 & 67 & 72 & 79 & 89 & 101 & 114 & 130 \\ 82 & 83 & 88 & 94 & 103 & 114 & 127 & 142 \\ 100 & 101 & 105 & 111 & 119 & 130 & 142 & 156 \end{bmatrix}$$

Je nach gewünschter Qualitätsstufe des JPEG-Formats werden unterschiedliche Matrizen $\mathbf{Q}$ gewählt. Die eigentliche Quantisierung erfolgt dann durch elementweises Teilen der Einträge von $\mathbf{G}$ durch $\mathbf{Q}$ und Runden auf die nächste ganze Zahl, d. h.

$$\mathbf{G}_{ij}^q = \left\lfloor \frac{\mathbf{G}_{ij}}{\mathbf{Q}_{ij}} \right\rfloor.$$

In unserem Beispiel ergibt sich:

$$\mathbf{G}^q = \begin{bmatrix} -11 & -17 & 9 & 1 & 0 & 1 & 0 & 0 \\ 24 & 10 & -3 & 1 & 0 & -1 & 0 & 0 \\ 2 & 5 & 1 & -1 & 0 & 0 & -1 & 0 \\ 0 & -2 & 0 & 0 & -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad (9.40)$$

Das Runden in der Quantisierung ist die wichtigste Approximation und somit Komprimierung im JPEG-Format.

Kodierung

Schließlich müssen die Matrizen $\mathbf{G}^q$ effizient gespeichert werden. Hier macht man sich den Umstand zunutze, dass die Einträge kleine (durch die Quantisierung) ganze Zahlen sind und dass viele Einträge verschwinden. Dabei stehen üblicherweise die meisten von null verschiedenen Einträge links oben in der Matrix, gehören somit zu niedrigen Frequenzen. Die weitere Kodierung erfolgt verlustfrei. Grundidee ist eine Umnummerierung der Einträge

von $\mathbf{G}^q$, schlängelförmig von links-oben nach rechts-unten, etwa

$$\mathbf{G}^q = \{-11, -17, 24, 2, 10, 9, 1, -3, 5, 0, 0, -2, 1, 1, 0, 1, 0, -1, \dots\}.$$

Durch diese Art der Sortierung werden die Einträge kleiner und üblicherweise stehen ab einer bestimmten Stelle nur noch Nullen. Gespeichert wird pro 8×8 -Block jeweils die Anzahl der Einträge bis zu der Stelle, wo nur noch Nullen auftreten. Diese nicht verschwindenden Einträge werden in Form einer *Huffman-Kodierung*⁴ dargestellt. Für Details verweisen wir auf [84].

Dekodierung

Bei der Anzeige von JPEG-Bildern müssen diese zunächst dekodiert werden. Hierzu müssen die oben beschriebenen Schritte rückgängig gemacht werden. Wir setzen bei der Matrix $\mathbf{G}^q$ in (9.40) ein und machen zunächst den Quantisierungsschritt rückgängig, berechnen also die Matrix $\tilde{\mathbf{G}}$ als

$$\tilde{\mathbf{G}}_{ij} = \mathbf{Q}_{ij} \mathbf{G}_{ij}^q.$$

Es ergibt sich:

$$\tilde{\mathbf{G}} = \begin{bmatrix} -110 & -204 & 225 & 37 & 0 & 66 & 0 & 0 \\ 360 & 190 & -84 & 39 & 0 & -67 & 0 & 0 \\ 50 & 140 & 35 & -45 & 0 & 0 & -88 & 0 \\ 0 & -78 & 0 & 0 & -66 & 0 & 0 & 0 \\ 0 & -52 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Der Fehler der Komprimierung lässt sich durch den Ausdruck

$$\frac{\|\mathbf{G} - \tilde{\mathbf{G}}\|}{\|\mathbf{G}\|}$$

angeben. In der Frobenius-Norm (2.1) gilt

$$\frac{\|\mathbf{G} - \tilde{\mathbf{G}}\|_F}{\|\mathbf{G}\|_F} \approx 0.19.$$

Dies entspricht einem Darstellungsfehler von knapp 20%. Die eigentliche Rekodierung erfolgt mittels der Basisdarstellung aus (9.38), d. h. in der Form

⁴ Kodiervorgang, das im Jahr 1952 von David Huffman entwickelt wurde. Dabei werden im Sinne eines Wörterbuches Zahlen oder Zahlenketten zu Wörtern zusammengefasst. Wörter, die sehr häufig vorkommen, erhalten einen kurzen „Namen“, seltene Wörter einen längeren. In unserem Beispiel kommen z. B. die Wörter „0“, „1“ oder „-1“ häufig vor. Siehe [54].

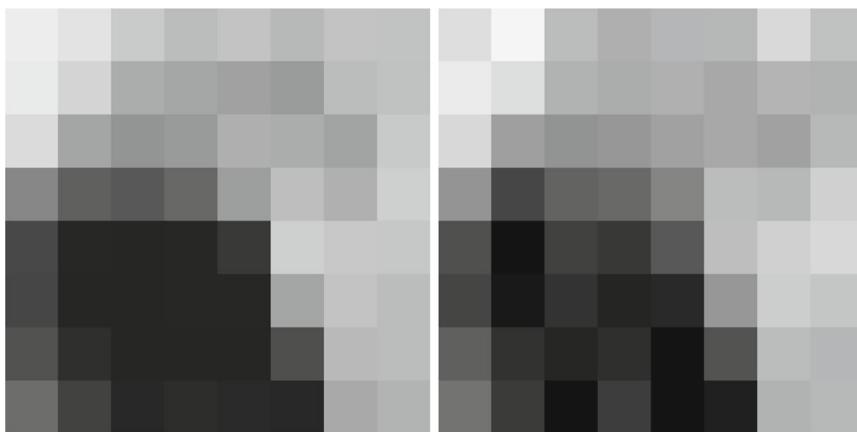


Abb. 9.14 Links: Block der Größe 8×8 aus dem Originalbild. Rechts: Rekodierung der JPEG-Kompression mit dem hier angegebenen Verfahren. Anstelle von 64 Intensitätsstufen (je ein Byte) müssen nur 38 von Null verschiedene Frequenzen gespeichert werden. In Huffman-Kodierung kann dies effizient in etwa 10 Byte erfolgen

$$\bar{\mathbf{B}}_{ij} = \frac{1}{4} \sum_{u,v=1}^8 \alpha(u)\alpha(v)\bar{\mathbf{G}}_{uv} \cos\left(\frac{2i-1}{16}u\pi\right) \cos\left(\frac{2j-1}{16}v\pi\right).$$

In Abb. 9.14 zeigen wir die Rekodierung sowie den ursprünglichen 8×8 -Block. Bei Farbbildern werden die drei Kanäle Y' , Cb und Cr getrennt behandelt. Im Anschluss muss der *Subsampling*-Schritt rückgängig gemacht werden, indem die beiden Chrominanz-Kanäle Cb und Cr vergrößert werden. Schließlich lässt sich die lineare Transformation aus (9.37) einfach umkehren.

9.9 Exkurs: Numerische Bausteine zur Lösung von Anfangswertproblemen

Anfangswertprobleme (siehe z. B. [25, 72, 76, 82, 92]) spielen eine große Rolle in der numerischen Mathematik, da sie bei der mathematischen Modellierung in vielen Anwendungen auftauchen. Ein zeitabhängiger Prozess, z. B. Wachstum einer Spezies, kann oft durch *Differentialgleichungen* beschrieben werden. Weitere Beispiele von Differentialgleichungen in diesem Buch sind Schwingungen eines Federpendels 5.6 oder auch das Minimalflächenproblem 3.8.

► **Definition 9.57 (Gewöhnliche Differentialgleichung)** Es sei $y(t)$ eine Funktion und $t \in \mathbb{R}$ die unabhängige Variable. Eine Differentialgleichung ist eine mathematische

Gleichung die die Funktion $y(t)$ mit mindestens einer ihrer Ableitungen $y'(t)$, $y''(t)$, $\dots$ in Relation setzt, sodass die Funktion die Differentialgleichung erfüllen.

9.9.1 Herleitung des Anfangswertproblems

Anfangswertprobleme bilden eine solche Klasse von Differentialgleichungen. Hier ist man ausgehend von einem gegebenen Anfangswert an der zukünftigen Entwicklung des Prozesses interessiert. Es sei $y := y(t)$ die Anzahl einer Spezies zum Zeitpunkt t . Zu einem Startpunkt t_0 sei die Anzahl y_0 . Wir sind nun an der Entwicklung interessiert. Sehr oft sind lokale Änderungen von Interesse. In einer infinitesimalen Zeiteinheit dt sei die lokale relative Änderung dy/y beschrieben durch die proportionale Relation

$$\frac{dy}{y} = (g - m)dt,$$

wobei g die Geburtsrate und m die Sterblichkeitsrate beschreiben. Wenn $g - m > 0$, dann wird die Spezies wachsen. Für $g - m = 0$ wird sich die Anzahl an Mitgliedern der Spezies nicht verändern und wir sollten auf dem Anfangsstatus y_0 verbleiben. Für $g - m < 0$ sollte die Spezies aussterben. Im Limes $dt \rightarrow 0$ gilt nach Umstellung $dy/dt = y'(t)$, was gerade der Differenzenquotient (Exkurs 3.8 und Kap. 7) ist. Damit haben wir die Differentialgleichung:

$$y'(t) = (g - m)y(t).$$

Es ist klar, dass die Lösung $y(t)$ noch nicht eindeutig bestimmt ist und mit Hilfe einer zusätzlichen Bedingung festgezurr werden muss. Dieses ist gerade die Anfangsbedingung $y(t_0) = y_0$.

Im Folgenden verallgemeinern wir die konkrete Herleitung. Es sei $I = [t_0, t_0 + T]$ ein Zeitintervall mit Anfangszeitpunkt $t_0 > 0$ und Endzeitpunkt $T > t_0$. Auf dem Intervall $I = [t_0, t_0 + T]$ ist eine Funktion $y \in C^1(I)$ gesucht, sodass die Differentialgleichung als auch der Anfangswert erfüllt werden

$$y'(t) = f(t, y(t)) \quad t \in I, \quad y(t_0) = y_0.$$

Dabei ist $f(t, y(t))$ eine gegebene rechte Seite. Beispielsweise $f(t, y) := (g - m)y(t)$.

9.9.2 Numerische Behandlung

Die Approximation von Anfangswertaufgaben basiert auf einer *Diskretisierung* des Intervalls $I = [0, T]$ in diskrete Zeitschritte mit den Zeitpunkten

$$t_0 < t_1 < \dots < t_N = t_0 + T \tag{9.41}$$

und in einer *Approximation* der gesuchten Lösung $y(t)$ in diesen diskreten Punkten

$$y_n \approx y(t_n).$$

Anstelle einer Funktion $y \in C^1(I)$ sind nun vielmehr die diskreten Funktionswerte $y_0, \dots, y_N$ gesucht

$$(t_0, y_0), \dots, (t_N, y_N).$$

Diese Punkte können dann z.B. durch lineare Interpolation (Abschn. 9.1) miteinander verbunden werden, um so eine Lösungskurve zu erhalten. Die einfachsten Verfahren zur Approximation sind das explizite und das implizite Euler-Verfahren. Beide beruhen auf der Approximation der Ableitung $y'(t_n)$ mit einem Differenzenquotienten erster Ordnung und die Iterationsvorschriften lauten

$$y_{n+1} = y_n + (t_{n+1} - t_n) f(t_n, y_n), \quad n = 0, \dots, N-1 \quad (9.42)$$

für das explizite Euler-Verfahren und

$$y_{n+1} = y_n + (t_{n+1} - t_n) f(t_{n+1}, y_{n+1}), \quad n = 0, \dots, N-1 \quad (9.43)$$

im Fall des impliziten Euler-Verfahrens. Details zu Diskretisierungsmethoden finden sich z. B. in den Werken [25, 72, 76, 82, 92]). Wir werden hier die Unterschiede zwischen beiden Herangehensweisen, also explizit und implizit beleuchten:

- *Explizite Methoden* können sehr schnell implementiert und ausgeführt werden. Sie lassen sich als iterativer Prozess der Art

$$y_{n+1} = F(y_n)$$

schreiben und in jedem Schritt muss die Funktion $F(\cdot)$ ausgewertet werden (welche von der rechten Seite $f(t, y(t))$ abhängt). Ein Nachteil von *expliziten Methoden* ist oft die mangelnde Stabilität. Explizite Methoden benötigen oft eine sehr große Zahl an Iterationen, so dass dieser Vorteil wieder verloren gehen kann.

- *Implizite Methoden* lassen sich als iterative Prozesse der Art

$$y_{n+1} = G(y_{n+1}; y_n)$$

schreiben. In jedem Schritt muss basierend auf y_n zur Bestimmung von y_{n+1} ein Gleichungssystem gelöst werden. Ist der Zusammenhang in $G(\cdot)$ nichtlinear, so ist sogar ein nichtlineares Gleichungssystem zu lösen. Der Vorteil von *impliziten Methoden* ist, dass sie oft sehr stabil sind. Eine Näherung der gesuchten Lösung kann mit weit weniger Iterationen bestimmt werden, als dies bei *expliziten Methoden* der Fall ist. Dafür ist jede Iteration deutlich aufwändiger.

In diesem Exkurs sind wir speziell an der numerischen Lösung und rechnergestützten Analyse eines *impliziten Verfahrens* interessiert. Aufgrund des (im allgemeinen nichtlinearen) Zusammenhangs in der Funktion $G(\cdot)$, müssen zur Lösung der Aufgabe oft andere numerische Verfahren wie zum Beispiel das Newton-Verfahren herangezogen werden. Das Ziel dieses Exkurses sind die Herleitungen solcher Konstruktionen. Insbesondere sind die Verschachtelung verschiedener numerischer Verfahren ein zentraler Aspekt in den meisten praxis-orientierten Anwendungen. Konkrete numerische Mittel in diesem Exkurs sind der Differenzenquotient (Abschn. 9.3), die Polynominterpolation (zur finalen graphischen Darstellung) aus Abschn. 9.1 bzw. stückweise Polynominterpolation 9.2, als auch im Fall von nichtlinearen Differentialgleichungen Fixpunkt- oder Newtonverfahren.

9.9.3 Ein lineares Anfangswertproblem

Wir betrachten im Folgenden das bereits oben angesprochene Wachstum einer Spezies. Ein einfaches lineares Wachstumsmodell ist durch

$$y'(t) = ay(t) \text{ für } t \geq t_0 \text{ und } y(t_0) = y_0 \quad (9.44)$$

gegeben, wobei $y := y(t)$ die Anzahl der Spezies kennzeichnet und t die Zeitvariable. Des Weiteren ist $a \in \mathbb{R}$ der Wachstumsparameter⁵. Die Anfangsbedingung zur Zeit t_0 ist y_0 . Für $a < 0$ wird die Spezies irgendwann aussterben und für $a > 0$ wird sich die Spezies exponentiell vermehren. In der Tat kann zu diesem Modell die Lösung sogar explizit mithilfe der *Trennung der Variablen* angegeben werden:

$$y(t) = y_0 \exp(a(t - t_0)) \quad (9.45)$$

Beispiel 9.58

Mittels eines konkreten Fallbeispiels berechnen wir die Entwicklung einer kleinen Einheit einer solchen Spezies. Im Jahr $t_0 = 2011$ existieren zwei Individuen, d. h. $y(2011) = 2$. Wir nehmen eine Wachstumsrate von $a = 0.25$ an. Mithilfe der obigen Gleichungen können wir abschätzen (bzw. hier exakt ausrechnen), dass z. B. im Jahr $t = 2014$

$$y(2014) = 2 \exp(0.25 \cdot (2014 - 2011)) = 4.117 \approx 4,$$

also vier Mitglieder existieren, und im Jahr $t = 2022$ bereits

$$y(2022) = 2 \exp(0.25 \cdot (2022 - 2011)) = 31.285 \approx 31,$$

⁵ Der Parameter a ist ein problemspezifischer Koeffizient, der im Allgemeinen durch gegebene Daten oder experimentelle Ergebnisse bestimmt werden muss. Oft ist eine solche Messung aber ebenfalls fehlerbehaftet und letztlich eine Approximation.

also 31 Individuen leben. Hieran sehen wir, dass das Modell prinzipiell korrekt ist, auf uns Menschen übertragen jedoch etwas unrealistisch erscheint. Wir beschäftigen uns hier aber nicht weiter mit der besseren mathematischen Modellierung dieses Problems. ◀

Da in den meisten Fällen die exakte Lösung nicht bekannt ist, entwickeln wir wie gehabt einen entsprechenden Algorithmus, der es uns erlaubt, das Problem mit dem Computer zu lösen. Hierzu zerlegen wir zunächst das Lösungsintervall $I = [t_0, t_0 + T]$ gemäß (9.41) und bezeichnen mit $k_n = t_n - t_{n-1}$ die *Schrittweite*. Zur Approximation der Differentialgleichung müssen wir insbesondere die Ableitung $y'(t)$ durch eine diskrete Approximation ersetzen, sprich einem Differenzenquotienten (Abschn. 9.3). Mit Taylor-Entwicklung gilt in jedem diskreten Zeitpunkt t_n

$$y'(t_n) = \frac{y(t_n) - y(t_{n-1})}{k_n} + \frac{y''(\xi_n)}{2} k_n$$

mit einer Zwischenstelle $\xi_n \in [t_{n-1}, t_n]$. Für eine ausführliche Analyse von Differenzenquotienten verweisen wir auf Abschn. 9.3.1. In den diskreten Punkten t_n bezeichnen wir nun mit $y_n \approx y(t_n)$ die gesuchte numerische Näherung an die Lösung. Das Ziel ist die schrittweise Konstruktion einer Sequenz

$$y_0, y_1, \dots, y_N \in \mathbb{R}, \quad N \in \mathbb{N}.$$

Der folgende Algorithmus beschreibt das *implizite Euler-Verfahren* (siehe z. B. [25, 72, 76]).

Algorithmus 9.6: Implizites Euler-Verfahren

Eingabe: Gegeben sei Anfangswert y_0

1 **Für** $n = 1$ **bis** N **tue**

2 $y_n = y_{n-1} + k_n f(t_n, y_n)$

Ergebnis: Approximation $y_n \approx y(t^n)$.

In unserem obigen Beispiel ist die rechte Seite gegeben als

$$f(t_n, y_n) = ay_n$$

und ein Schritt des impliziten Euler-Verfahrens lässt sich schreiben als

$$y_n = y_{n-1} + k_n a y_n \quad \Leftrightarrow \quad y_n = (1 - k_n a)^{-1} y_{n-1}.$$

Wann immer $1 - k_n a \neq 0$ gilt, können wir das Verfahren durchführen.

Ein Grundproblem des impliziten Euler-Verfahrens ist die Tatsache, dass die gesuchte Lösung y_n sowohl in der linken als auch in der rechten Seite f vorkommt. Für Allgemeine Funktion $f(t, y(t))$ lässt sich die Gleichung in jedem Schritt des Verfahrens nicht mehr explizit auflösen. An dieser Stelle kommt nun das Newton-Verfahren ins Spiel. Wir schreiben für $n = 1, 2, \dots, N$ jeden Teilschritt

$$\frac{y_n - y_{n-1}}{k_n} = f(t_n, y_n) \quad (9.46)$$

als Nullstellenproblem. Zu diesem Zweck bringen wir alle Terme auf eine Seite und definieren die Funktion $g(\cdot)$ durch

$$g(y) := y - y_{n-1} - k_n f(t_n, y).$$

In jedem Schritt ist nun $y_n \in \mathbb{R}$ gesucht, sodass

$$g(y_n) = 0. \quad (9.47)$$

Wie wir leicht sehen, ist Aufgabe (9.47) äquivalent zu (9.46). Ausgehend von einem Startwert $y_n^{(0)} \in \mathbb{R}$ können wir das Newton-Verfahren aus Algorithmus 8.4 anwenden und erhalten die Iteration

$$y_n^{(l+1)} = y_n^{(l)} - \frac{g(y_n^{(l)})}{g'(y_n^{(l)})}, \quad l = 0, 1, 2, \dots,$$

wobei l die Newton-Iteration anzeigt. Die Ableitung ist hier gegeben als

$$g'(y_n^{(l)}) := 1 - k_n f_y'(t_n, y_n^{(l)}),$$

wobei f_y' die Ableitung bezüglich der Variablen y ist. In unserem konkreten Beispiel erhalten wir

$$g'(y_n^{(l)}) = 1 - k_n a,$$

da $f(t_n, y_n^{(l)}) = a y_n^{(l)}$.

Die Iteration wird gestoppt, wenn das Newton-Abbruchkriterium

$$|g(y_n^{(l+1)})| < \epsilon$$

erfüllt ist.

Bemerkung 9.59 Als guter Startwert für ein Newton-Verfahren innerhalb eines Zeitschrittsverfahrens kommt die vorherige Zeitschrittlösung y_{n-1} infrage, also $y_n^{(0)} := y_{n-1}$. Für diesen Startwert können wir sogar eine Fehlerabschätzung geben. Angenommen, y_n sei die exakte Lösung. Dann gilt mithilfe der Verfahrensvorschrift des impliziten Euler-Verfahrens

$$y_n - y_n^{(0)} = y_n - y_{n-1} = k_n f(t_n, y_n).$$

Für den Fehler gilt die Abschätzung $|y_n - y_n^{(0)}| = O(k_n)$. Je kleiner wir die Startschrittweite wählen, umso kleiner ist der anfängliche Fehler des Newton-Verfahrens. ♦

Mit dieser Iteration erhalten wir eine Approximation $y_n^{(l)}$ für den gesuchten Wert y_n (welcher selbst eine Approximation des tatsächlichen Werts $y(t_n)$ darstellt). Das heißt, wir haben es hier mit einer Verschachtelung zweier numerischer Verfahren zu tun.

Für unser obiges Anfangswertproblem ist jeder Schritt des Newton-Verfahrens gegeben als

$$y_n^{(l+1)} = y_n^{(l)} - \frac{y_n^{(l)} - y_{n-1} - k_n a y_n^{(l)}}{1 - k_n a}.$$

Wir haben bereits diskutiert, dass dieses Problem linear ist. Eine Anwendung des Newton-Verfahrens ist somit nicht notwendig. Bei komplexeren Problemstellungen wird in der Berechnung der Ableitung $g'(y_n^{(l)})$ der wesentliche Aufwand liegen. In höherdimensionalen Problemen, welche wir in Abschn. 8.4.1 behandeln, ist die Ableitung eine Matrix, die *Jacobi-Matrix*. Dann kommt als erheblicher numerischer Aufwand noch das Lösen eines linearen Gleichungssystems mit der Jacobi-Matrix hinzu.

Wir berechnen für verschiedene N und $a = 0.25$ die numerische Lösung des Wachstumsproblems und stellen die Ergebnisse in Tab. 9.3 zusammen.

Wir sehen, dass wir für größere N erwartungsgemäß eine immer bessere Approximation zur exakten Lösung erhalten. Dies veranschaulichen wir auch in Abb. 9.15.

Des Weiteren beobachten wir, dass sich der Fehler am Endzeitpunkt T halbiert, wenn doppelt so viele Intervalle zur Berechnung der numerischen Lösung herangezogen werden. Zuletzt kommen wir auf unsere ursprüngliche Motivation zurück, das Newton-Verfahren innerhalb des Zeitschrittverfahrens zu nutzen. Wir beobachten, dass wir stets einen einzigen Newton-Schritt benötigen. Das ist auch klar, da die Funktion $g(y_n)$ linear in y_n ist und hier Newton in einem Schritt konvergiert.

Tab. 9.3 Numerischer Fehler des impliziten Euler-Verfahrens zur Approximation von $y'(t) = ay(t)$ mit $a = 0.25$. Wir zeigen für verschiedene N die numerische Approximation $y_N \approx y(T)$ sowie die relativen und absoluten Fehler zu der exakten Lösung $y(T) \approx 31.28526$

N	y_N	$ y(T) - y_N / y(T) $	$ y(T) - y_N $
10	49.848	0.52082	12.377
20	38.534	0.21894	5.9697
40	34.544	0.10144	2.9629
80	32.835	0.04893	1.4791
160	32.042	0.02404	0.7394
320	31.659	0.01192	0.3697
640	31.471	0.00593	0.1848
1280	31.378	0.00296	0.0924

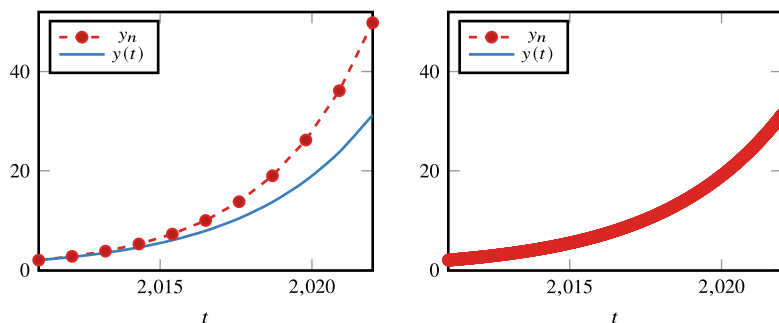


Abb. 9.15 Lineares Problem $y'(t) = ay(t)$ mit $a = 0.25$. Vergleich von numerischer Lösung mittels implizitem Euler-Verfahren und exakter Lösung für $N = 10$ (links) und $N = 1280$ (rechts). Im Fall $N = 1280$ liegen numerische und exakte Lösung übereinander

9.9.4 Ein nichtlineares Anfangswertproblem

Die Spezies aus dem vorherigen Abschnitt hat sich in elf Jahren von zwei auf 31 Individuen vermehrt. Wir betrachten nun das Aussterben (aufgrund von Nahrungsmangel), welches folgendem (nichtlinearen) Gesetz genüge:

$$y'(t) = ay^2(t) \text{ für } t \in [t_0, T] \text{ und } y(t_0) = y_0.$$

Insbesondere ist ein Rückgang durch einen negativen Parameter $a < 0$ charakterisiert. Wir wählen $a = -0.1$, $t_0 = 2022$, $T = 2050$ und $y_0 = 31$. Nach unseren vorherigen Betrachtungen sind wir wieder an einer Formulierung als Nullstellenproblem

$$g(y_n) = 0$$

mit Lösung via Newton-Verfahren interessiert. Die Funktion $g(y_n)$ und deren Ableitung sind gegeben durch

$$\begin{aligned} g(y_n) &= y_n - y_{n-1} - k_n a y_n^2, \\ g'(y_n) &= 1 - 2k_n a y_n. \end{aligned}$$

Dann ist die Vorschrift für die $l + 1$ -te Iteration bestimmt durch

$$y_n^{(l+1)} = y_n^{(l)} - \frac{g(y_n^{(l)})}{g'(y_n^{(l)})} = y_n^{(l)} - \frac{y_n^{(l)} - y_{n-1} - k_n a (y_n^{(l)})^2}{1 - 2k_n a y_n^{(l)}}.$$

Aufgrund der Nichtlinearität der rechten Seite $f(y_n, t) = ay_n^2$ erwarten wir nun mehr als eine Newton-Iteration pro Zeitschrittlösung n .

Für die exakte Lösung zum Zeitpunkt $Z = 2050$ ist $y(T) \approx 0.35$ (siehe Tab. 9.4 mit der feineren Diskretisierung $N = 1280$). Das heißt, im Jahr 2050 sind – bezüglich des hier zugrundeliegenden Modells – wahrscheinlich alle Individuen ausgestorben. In der Tat sehen wir einen sehr schnellen Abfall der Lösungskurve. Bereits zum Zeitpunkt $t = 2026.75$ leben nur noch $y_{2026.75} = 1.995961$, also weniger als zwei, Individuen. Damit ist die Fortpflanzung hinfällig, unter der Annahme des Ausschlusses ungeschlechtlicher Reproduktion. Zuletzt bemerken wir noch, dass bereits bei $t = 2031.73$ dann $y_{2031.73} = 0.9995613$ gilt.

Wir betrachten nun das Newton-Verfahren genauer. Bei einer Toleranz von $\text{tol}_{\text{neu}} = 10^{-10}$ erhalten wir bei $N = 10$ Zeitschritten im ersten Zeitschritt ($n = 1$) sechs Iterationen und drei Iterationen bei $n = 10$. In der Gesamtsumme sind das 39 N-Iterationen. Für $N = 1280$ beobachten wir drei Iterationen im ersten Zeitschritt und zwei Iterationen bei $n = 1280$. Die Gesamtsumme der Newton-Schritte haben wir in Tab. 9.4 zusammengefasst.

Wir beobachten zuerst, dass wir für dieses Problem in der Tat mehr als eine Newton-Iteration benötigen. Danach stellen wir eine Abhängigkeit bezüglich der Intervalllänge (sprich N) des Zeitschrittverfahrens fest. Dies ist auch klar, da wir in jedem Zeitschritt als Startwert für das Newton-Verfahren den vorherigen Zeitschritt wählen. Wie wir in Abb. 9.16 leicht erkennen können, ändern sich die Funktionswerte von $y(t)$ aber gerade am Anfang sehr stark. Das heißt, die vorherige Zeitschrittlösung y_{n-1} ist definitiv keine gute Newton-Initiallösung. Dies ist insbesondere der Fall, wenn nur wenige Punkte (also kleines N) zur Zeitdiskretisierung herangezogen werden. Da die Lösung exponentiell abfällt und die Kurve zum Endzeitpunkt $T = 2050$ immer flacher wird, eignen sich dann auch die vorherigen Lösungen y_{n-1} besser als Startwerte für die Newton-Iteration. Für größere N sehen wir dann auch eine erhebliche Verbesserung der Newton-Iteration mit durchschnittlich zwei Iterationen pro Zeitschritt. Gleichzeitig haben wir im vorherigen Abschnitt gesehen, dass sich bei großem N auch die Genauigkeit der Zeitschrittlösung selbst erheblich verbessert. Mit größerem N erreichen wir also zwei Ziele: höhere Effizienz des inneren Löser, hier

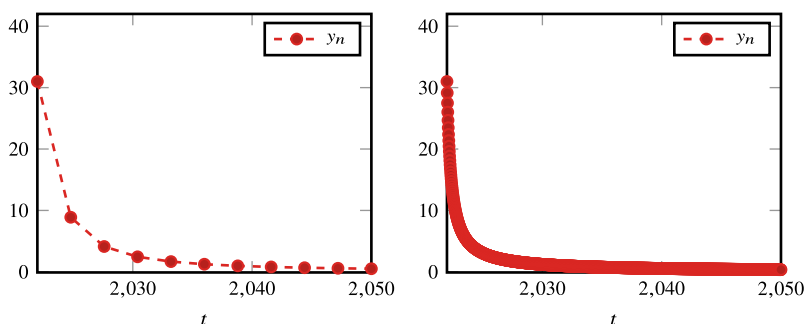


Abb. 9.16 Numerische Lösungen des nichtlinearen Problems $y'(t) = ay(t)^2$ mit negativem Wachstum $a = -0.1$ für $N = 10$ (links) und $N = 1280$ (rechts). Wie bereits vorher erkennen wir erhebliche Unterschiede der beiden Approximationen. Dies ist insbesondere zu Beginn signifikant, wenn die Funktion steil abfällt, sprich die erste Ableitung sehr groß ist

Tab. 9.4 Genauigkeit des Endzeitwerts $y_n(T)$ und summierte Anzahl der Newton-Iterationen sowie durchschnittliche Anzahl der Newton-Iterationen pro Zeitschritt n für verschiedene N und zwei verschiedene Newton-Toleranzen

tol_{neu}	N	y_N	#Newton	#Newton/ N
10^{-4}	10	0.5054	28	2.80
10^{-4}	1280	0.3543	1312	1.02
10^{-10}	10	0.5054	39	3.90
10^{-10}	1280	0.3543	2583	2.01

Newton, und auch höhere Genauigkeiten des äußeren Verfahrens, hier implizites Euler-Verfahren. Demgegenüber steht ein insgesamt höherer Rechenaufwand, da natürlich viel mehr Zeitschritte gelöst werden müssen.

Die Rechenkosten können weiter gesenkt werden, wenn wir die Newton-Toleranz von $\text{tol}_{\text{neu}} = 10^{-10}$ auf $\text{tol}_{\text{neu}} = 10^{-4}$ hinaufsetzen. Die entsprechenden Werte sind in Tab. 9.4 eingetragen. Bei solchen Genauigkeitsveränderungen muss allerdings darauf geachtet werden, dass sich die Genauigkeit der Lösungskurve bzw. des Zielfunktional, also hier der Endzeitwert y_N , nicht erheblich verschlechtert. Dies ist hier nicht der Fall bzw. bei der groben Diskretisierung mit $N = 10$ ändert sich der Wert in der 10ten Nachkommastelle und bei $N = 1280$ in der 12ten Nachkommastelle. Diese Unterschiede sind offensichtlich vernachlässigbar. Daher ist es in diesem Beispiel gerechtfertigt, die Newton-Toleranz relativ grob zu wählen, wodurch sich der Rechenaufwand insbesondere auf feinen Diskretisierungen hier nahezu halbiert.

Die numerische Approximation von Integralen, *Quadratur* oder *numerische Integration* genannt, kann aus verschiedenen Gründen notwendig sein:

- Die Stammfunktion eines Integrals lässt sich nicht durch eine elementare Funktion ausdrücken, etwa bei

$$\int_0^\infty \exp(-x^2) dx, \quad \int_0^\infty \frac{\sin(x)}{x} dx.$$

- Eine Stammfunktion existiert in geschlossener Form, aber die Berechnung ist aufwendig, sodass numerische Methoden vorzuziehen sind.
- Der Integrand ist nur an diskreten Stellen bekannt, beispielsweise bei Messreihendaten.

Bei der numerischen Integration basieren die Methoden auf den bereits kennengelernten Interpolationsmethoden. Sie sind eine klassische Anwendung der Polynominterpolation sowie der Spline-Interpolation. Erstere führt auf die sogenannten *interpolatorischen Quadraturformeln*, die Spline-Interpolation dagegen auf die *stückweise interpolatorischen Quadraturformeln* (die in der Literatur auch häufig unter dem Namen der zusammengesetzten oder summierten Quadratur zu finden sind).

Bei der interpolatorischen Quadratur einer Funktion $f(x)$ auf dem Intervall $[a, b]$ wird zunächst ein Interpolationspolynom zu gegebenen Stützstellen $x_0, x_1, \dots, x_n$ kreiert, welches dann integriert wird (basiert dementsprechend auf Abschn. 9.1). Bei der Gauß-Quadratur wird die Position der Stützstellen $x_i \in [a, b]$ im Intervall so bestimmt, dass die resultierende Integrationsformel eine *optimale* Ordnung erzielt. Zuletzt betrachten wir als Anwendung der Extrapolation zum Limes (Abschn. 9.4), das sogenannte Rombergsche Integrationsverfahren in Abschn. 10.3.

► **Definition 10.1 (Quadraturformel)** Es sei $f \in C[a, b]$. Unter einer *numerischen Quadraturformel* zur Approximation von $I(f) := \int_a^b f(x) dx$ verstehen wir die Vorschrift

$$I^n(f) := \sum_{i=0}^n \alpha_i f(x_i)$$

mit $n + 1$ paarweise verschiedenen *Stützstellen* $x_0, \dots, x_n$ sowie $n + 1$ *Quadraturgewichten* $\alpha_0, \dots, \alpha_n$.

Zunächst bringen wir zur Veranschaulichung einige einfache Beispiele:

► **Definition 10.2 (Boxregel)** Die Boxregel zur Integration auf $[a, b]$ ist die einfachste Quadraturformel und basiert auf Interpolation von $f(x)$ mit einem konstanten Polynom $p(x) = f(a)$ und Integration von $p(x)$ (siehe Abb. 10.1):

$$I^0(f) = (b - a)f(a)$$

Neben dieser *linksseitigen Boxregel* existiert mit $I^0(f) = (b - a)f(b)$ auch die *rechtsseitige Boxregel*.

Die Boxregel hat ihre Bedeutung in der Herleitung des Riemann-Integrals. Die Boxregel ist vergleichbar mit dem einseitigen Differenzenquotienten. Eine bessere Quadraturformel erhalten wir durch Ausnutzen von Symmetrieeigenschaften.

► **Definition 10.3 (Mittelpunktsregel)** Zur Herleitung der Mittelpunktsregel wird die Funktion $f(x)$ in der Mitte des Intervalls mit einem konstanten Polynom interpoliert:

$$I^0(f) = (b - a)f\left(\frac{a + b}{2}\right)$$

Siehe Abb. 10.1.

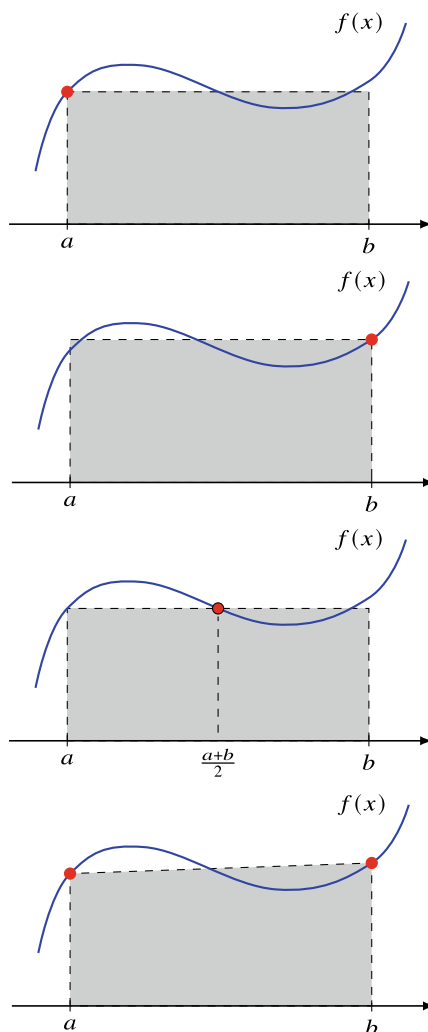
Diese beiden Regeln basieren auf einer Interpolation der Funktion f mit einer konstanten Funktion. Gemäß der Trapezregel wird $f(x)$ in den beiden Intervallenden linear interpoliert:

► **Definition 10.4 (Trapezregel)** Die Trapezregel ist durch Integration der linearen Interpolation in $(a, f(a))$ sowie $(b, f(b))$ gebildet:

$$I^1(f) = \frac{b - a}{2}(f(a) + f(b))$$

Siehe auch Abb. 10.1.

Abb. 10.1 Grunzlegende
Quadraturformeln zur
Approximation des Integrals
 $\int_a^b f(x) dx$. Oben: Links- und
rechtsseitige Boxregel. Unten
links: Mittelpunktsregel mit
dem Quadraturpunkt
 $x_m = (a + b)/2$ und rechts:
Trapezregel



10.1 Interpolatorische Quadratur

Die interpolatorischen Quadraturformeln werden über die Konstruktion eines geeigneten Interpolationspolynoms hergeleitet. Zu den gegebenen Stützstellen $a \leq x_0 < \dots < x_n \leq b$ wird gemäß Abschn. 9.1 das Lagrangesche Interpolationspolynom als Approximation der Funktion f gebildet:

$$p_n(x) = \sum_{i=0}^n f(x_i) L_i^{(n)}(x)$$

Dieses wird dann integriert:

$$I^{(n)}(f) := \int_a^b p_n(x) \, dx = \sum_{i=0}^n f(x_i) \underbrace{\int_a^b L_i^{(n)}(x) \, dx}_{=: \alpha_i} = \sum_{i=0}^n \alpha_i f(x_i)$$

Die Quadraturgewichte

$$\alpha_i = \int_a^b L_i^{(n)}(x) \, dx \quad (10.1)$$

hängen offensichtlich nicht von der zu integrierenden Funktion $f(x)$ ab, dafür aber vom Intervall $[a, b]$ sowie von den Stützstellen $x_0, \dots, x_n$. Dies impliziert die Frage, ob durch geschickte Verteilung der Stützstellen die Qualität der Gewichte verbessert werden kann.

Bevor wir einzelne Quadraturformeln analysieren und nach möglichst leistungsfähigen Formeln suchen, können wir zunächst ein einfaches, aber doch allgemeines Resultat herleiten.

Satz 10.5 (Lagrange-Quadratur) Für die interpolatorischen Quadraturformeln $I^n(f)$ mit $n + 1$ paarweise verschiedenen Stützstellen $x_0, x_1, \dots, x_n$ gilt zur Approximation des Integrals $I(f) = \int_a^b f(x) \, dx$ die Fehlerdarstellung

$$I(f) - I^n(f) = \int_a^b f[x_0, \dots, x_n, x] \prod_{j=0}^n (x - x_j) \, dx$$

mit Newtonscher Restglieddarstellung.

Beweis Der Beweis folgt unmittelbar durch Integration der entsprechenden Fehlerabschätzung für die Lagrange-Interpolation in Satz 9.13. $\square$

Hieraus folgt eine wichtige Eigenschaft der interpolatorischen Quadraturformeln:

Korollar 10.6 Die interpolatorische Quadraturformel $I^{(n)}(\cdot)$ ist exakt für alle Polynome von Grad n . Dieses Ergebnis ist unabhängig von der konkreten Wahl der $n + 1$ paarweise verschiedenen Stützstellen.

Beweis Folgt direkt aus der Konstruktion der interpolatorischen Quadraturformeln, da für jedes $f \in P_n$ sofort $p = f$ gilt. $\square$

► **Definition 10.7 (Ordnung von Quadraturregeln)** Eine Quadraturformel $I^{(n)}(\cdot)$ wird (mindestens) von der Ordnung m genannt, wenn durch sie mindestens alle Polynome aus P_{m-1} exakt integriert werden. Das heißt, die interpolatorischen Quadraturformeln $I^{(n)}(\cdot)$ zu $n + 1$ Stützstellen sind mindestens von Ordnung $n + 1$.

Im Folgenden werden wir die bereits eingeführten einfachen Quadraturformeln näher analysieren und ihre Fehlerabschätzung sowie Ordnung bestimmen. Hierzu werden wir die Newtonsche Integraldarstellung des Interpolationsfehlers aus Satz 10.5 nutzen.

Satz 10.8 (Boxregel) Es sei $f \in C^1[a, b]$. Die Boxregel (Definition 10.2) ist von erster Ordnung und es gilt die Fehlerabschätzung

$$I^0(f) := (b-a)f(a), \quad I(f) - I^0(f) = \frac{(b-a)^2}{2} f'(\xi)$$

mit einer Zwischenstelle $\xi \in (a, b)$.

Beweis Die Boxregel basiert auf der Interpolation mit einem konstanten Polynom, hat daher erste Ordnung. Aus Satz 10.5 folgt mit $x_0 = a$

$$I(f) - I^0(f) = \int_a^b f[a, x](x-a) dx,$$

wobei mit Satz 9.13 weiter gilt:

$$I(f) - I^0(f) = \int_a^b \int_0^1 f'(a + t(x-a))(x-a) dt dx.$$

Wir wenden den Mittelwertsatz der Integralrechnung zweimal an und erhalten

$$I(f) - I^0(f) = f'(\xi) \int_a^b \int_0^1 (x-a) dt dx = \frac{1}{2} f'(\xi)(b-a)^2$$

mit einem Zwischenwert $\xi \in (a, b)$. □

Die Boxregel ist die einfachste Quadraturformel. In Analogie zur Diskussion bei den Differenzenquotienten können wir bei der Mittelpunktsregel durch geschickte Ausnutzung der Symmetrie eine bessere Approximationsordnung erwarten. Um diese bessere Ordnung zu erreichen, müssen wieder Superapproximationseigenschaften genutzt werden, welche im Allgemeinen etwas Mehraufwand erfordern:

Satz 10.9 (Mittelpunktsregel) Es sei $f \in C^2[a, b]$. Die Mittelpunktsregel (Definition 10.3) ist von Ordnung 2 und es gilt die Fehlerabschätzung

$$I_m^0(f) := (b-a)f\left(\frac{a+b}{2}\right), \quad I(f) - I_m^0(f) = \frac{(b-a)^3}{24} f''(\xi),$$

mit einer Zwischenstelle $\xi \in (a, b)$.

Beweis Aufgrund der Interpolation mit konstantem Polynom ist zunächst nur erste Ordnung zu erwarten. Es gilt mit Satz 9.13

$$I(f) - I_m^0(f) = \int_a^b f\left[\frac{a+b}{2}, x\right] \left(x - \frac{a+b}{2}\right) dx.$$

Zur Vereinfachung setzen wir $x_m := (a+b)/2$ und es sei $\bar{f} \in \mathbb{R}$ ein beliebiger, fester Wert. Dann gilt

$$I(f) - I_m^0(f) = \int_a^b (f[x_m, x] - \bar{f})(x - x_m) dx + \underbrace{\bar{f} \int_a^b (x - x_m) dx}_{=0}. \quad (10.2)$$

Mit der Integraldarstellung der dividierten Differenzen aus Satz 9.13 gilt bei Taylor-Entwicklung um x_m

$$f[x_m, x] = \int_0^1 f'(x_m + t(x - x_m)) dt = \int_0^1 f'(x_m) + t(x - x_m)f''(\xi) dt$$

für ein $\xi \in [x_m, x]$. Bei Wahl von $\bar{f} = f'(x_m)$ folgt mit (10.2) für den Fehler

$$I(f) - I_m^0(f) = f''(\xi) \int_a^b \int_0^1 t(x - x_m)^2 dt dx = \frac{|b-a|^3}{24} f''(\xi).$$

□

Mit dem gleichen Aufwand wie die Boxregel, also mit einer Auswertung der Funktion $f(x)$, erreicht die Mittelpunktsregel die doppelte Ordnung.

Satz 10.10 (Trapezregel) Es sei $f \in C^2([a, b])$. Die Trapezregel (Definition 10.4) ist von Ordnung 2 und es gilt die Fehlerabschätzung

$$I^1(f) := \frac{b-a}{2}(f(a) + f(b)), \quad I(f) - I^1(f) = -\frac{(b-a)^3}{12} f''(\xi),$$

mit einer Zwischenstelle $\xi \in (a, b)$.

Beweis Eine optimale Fehlerabschätzung für die Trapezregel kann mit einem Trick hergeleitet werden. Wir schieben eine Funktion $g'' \equiv 1$ ein und erhalten mit partieller Integration

$$\begin{aligned} \int_a^b f(x) dx &= \int_a^b f(x) g''(x) dx = f g' \Big|_a^b - \int_a^b f'(x) g'(x) dx \\ &= f g' \Big|_a^b - f' g \Big|_a^b + \int_a^b f''(x) g(x) dx. \end{aligned} \quad (10.3)$$

Nun gilt es, eine Funktion $g \in C^2[a, b]$ so zu bestimmen, dass der Randterm $f'g$ wegfällt und der Term fg' gerade die Trapezregel erfüllt. Aus $g'' \equiv 1$ folgt

$$g(x) = \frac{1}{2}(x - c_1)(x - c_2)$$

und aus $g(a) = g(b) = 0$, damit $f'g = 0$, folgt

$$g(x) = \frac{1}{2}(x - a)(x - b),$$

also

$$g'(x) = x - \frac{a+b}{2} \Rightarrow g'(a) = -\frac{b-a}{2}, \quad g'(b) = \frac{b-a}{2}.$$

Wir setzen diese Funktion in (10.3) ein und erhalten

$$\int_a^b f(x)dx = \frac{b-a}{2} (f(a) + f(b)) + \frac{1}{2} \int_a^b f''(x)(x-a)(x-b)dx.$$

Da die Funktion $g(x)$ im Intervall $[a, b]$ keinen Vorzeichenwechsel hat, folgern wir mit dem Mittelwertsatz der Integralrechnung

$$\begin{aligned} \int_a^b f(x)dx - \frac{b-a}{2} (f(a) + f(b)) &= f''(\xi) \frac{1}{2} \int_a^b (x-a)(x-b)dx \\ &= -f''(\xi) \frac{(b-a)^3}{12}, \end{aligned}$$

mit einem Zwischenwert $\xi \in [a, b]$. □

Die Verwendung eines quadratischen Interpolationspolynoms führt auf die Simpson-Regel.

Satz 10.11 (Simpson-Regel) Es sei $f \in C^4[a, b]$. Die *Simpson-Regel*, basierend auf Interpolation mit quadratischem Polynom

$$I^2(f) = \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right),$$

ist von Ordnung vier und es gilt

$$I(f) - I^2(f) = -\frac{f^{(4)}(\xi)}{2880}(b-a)^5,$$

wobei $\xi \in (a, b)$.

Beweis Der Beweis folgt analog zu den Beweisen zu Mittelpunkts- und Trapezregel und sei dem Leser als Übung gestellt. Dabei muss wieder das Superapproximationsprinzip ausgenutzt werden. $\square$

Bemerkung 10.12 *Wie bei der Mittelpunktsregel ist die Ordnung der Simpson-Regel eine Potenz höher, als zu erwarten wäre. Treibt man dieses Spiel weiter und konstruiert noch höhere Quadraturformeln, dann erkennen wir ein allgemeines Prinzip: Bei Quadratur mit geraden Interpolationspolynomen wird die Fehlerordnung um eine Potenz erhöht.* $\blacklozenge$

Bei allen bisherigen Quadraturformeln sind die Stützstellen gleichmäßig im Intervall $[a, b]$ verteilt. Diese Formeln werden *Newton-Cotes-Formeln* genannt:

► **Definition 10.13 (Newton-Cotes-Formeln)** Quadraturregeln mit äquidistant im Intervall $[a, b]$ verteilten Stützstellen heißen *Newton-Cotes-Formeln*. Gehören die Intervallenden a sowie b zu den Stützstellen, so heißen die Formeln *abgeschlossen*, ansonsten *offen*.

In Tab. 10.1 fassen wir einige gebräuchliche Newton-Cotes-Formeln zusammen. Ab $n \geq 8$ bei den abgeschlossenen Formeln und ab $n = 2$ bei den offenen Formeln treten negative Quadraturgewichte α_i auf. Dies führt zu dem Effekt, dass auch bei rein positiven Integranden Auslöschung auftreten kann. Daher sind diese Formeln aus numerischer Sicht nicht mehr anwendbar. Für die Gewichte einer Quadraturregel gilt stets

$$\sum_{i=0}^n \alpha_i = b - a.$$

Im Fall negativer Gewichte gilt

$$\sum_{i=0}^n |\alpha_i| > (b - a).$$

Tab. 10.1 Eigenschaften und Gewichte einiger Newton-Cotes-Formeln

n	Gewichte α_i		Name	Ordnung
0	1	offen	Boxregel	1
1	$\frac{1}{2}, \frac{1}{2}$	geschlossen	Trapezregel	2
2	$\frac{1}{6}, \frac{4}{6}, \frac{1}{6}$	geschlossen	Simpson-Regel	4
3	$\frac{1}{8}, \frac{3}{8}, \frac{3}{8}, \frac{1}{8}$	geschlossen	Newton-3/8-Regel	4
4	$\frac{7}{90}, \frac{32}{90}, \frac{12}{90}, \frac{32}{90}, \frac{7}{90}$	geschlossen	Milne-Regel	6
6	$\frac{41}{840}, \frac{216}{840}, \frac{27}{840}, \frac{272}{840}, \frac{27}{840}, \frac{216}{840}, \frac{41}{840}$	geschlossen	Weddle-Regel	8

Ein allgemeiner Satz (Theorem von Kusmin) besagt, dass für $n \rightarrow \infty$ sogar gilt:

$$\sum_{i=0}^n |\alpha_i| \rightarrow \infty \quad (n \rightarrow \infty)$$

Regeln von sehr hoher Ordnung sind somit numerisch nicht stabil einsetzbar.

10.2 Stückweise interpolatorische Quadraturformeln

Die interpolatorische Quadratur aus dem vorangegangenen Abschnitt beruht auf der Integration eines Interpolationspolynoms p . Für dieses gilt die Fehlerabschätzung (Satz 9.11)

$$f(x) - p(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i).$$

Die Genauigkeit der Quadratur kann prinzipiell über zwei Wege verbessert werden: Der Vorfaktor kann durch Wahl von mehr Stützstellen klein gehalten werden, denn es gilt $1/(n+1)! \rightarrow 0$. Dies führt jedoch nur dann zur Konvergenz des Fehlers, falls die Ableitungen $f^{(n)}$ nicht zu schnell wachsen, siehe Beispiel 9.16. Als zweite Option bleibt das Produkt der Stützstellen. Eine einfache Abschätzung besagt wegen $x, x_0, \dots, x_n \in [a, b]$

$$\left| \prod_{i=0}^n (x - x_i) \right| \leq (b - a)^{n+1}.$$

Interpolationsfehler (und somit auch Quadraturfehler) lassen sich durch Potenzen der Intervallgröße beschränken. Dies wird zur Entwicklung von stabilen und konvergenten Quadraturformeln entsprechend dem Vorgehen der stückweisen Interpolation (Abschn. 9.2) genutzt. Hierzu sei

$$a = y_0 < y_1 < \dots < y_m = b, \quad h_i := y_i - y_{i-1},$$

eine Zerlegung des Intervalls $[a, b]$ in m Teilintervalle mit Schrittweite h_i . Auf jedem dieser m Teilintervalle wird die Funktion f mithilfe einer Quadraturformel approximiert:

$$I_h^n(f) = \sum_{i=1}^m I_{[y_{i-1}, y_i]}^n(f)$$

Aus Satz 10.5 folgt sofort eine allgemeine Fehlerabschätzung für die summierten Quadraturformeln:

Satz 10.14 (Summierte Quadratur) Es sei $f \in C^{n+1}([a, b])$ sowie $a = y_0 < \dots < y_m = b$ eine Zerlegung des Intervalls mit Schrittweiten $h_i := y_i - y_{i-1}$ sowie $h := \max h_i$. Für die summierte Quadraturformel $I_h^n(f)$ gilt die Fehlerabschätzung

$$I(f) - I_h^n(f) \leq c \max_{[a,b]} |f^{(n+1)}| h^{n+1}$$

mit einer Konstante $c(n) > 0$.

Beweis Der Beweis folgt einfach aus Kombination von Satz 10.5 sowie der differentiellen Restglieddarstellung der Lagrange-Interpolation. $\square$

Aus diesem Resultat kann ein wichtiges Ergebnis abgelesen werden: Für $h \rightarrow 0$, also für steigende Feinheit der Intervallzerlegung, konvergiert die summierte Quadratur für beliebiges n . Mithilfe von summierten Quadraturregeln lassen sich somit auch Funktionen mit schnell wachsenden Ableitungen integrieren. Darüber hinaus eignen sich summierte Regeln auch zur Integration von Funktionen, die nur eine geringe Regularität, etwa $f \in C^1([a, b])$, aufweisen. Da es möglich ist, die Ordnung n klein zu halten, sind summierte Quadraturregeln numerisch stabiler. Zur Veranschaulichung betrachten wir in Abb. 10.2 die summierte Trapezregel.

Im Folgenden konkretisieren wir einige einfache summierte Quadraturregeln. Hierzu betrachten wir ausschließlich äquidistante Zerlegungen des Intervalls $[a, b]$:

$$a = y_0 < \dots < y_m = b, \quad h := \frac{b-a}{m}, \quad y_i := a + ih, \quad i = 0, \dots, m \quad (10.4)$$

Satz 10.15 (Summierte Boxregel) Es sei $f \in C^1[a, b]$ sowie durch (10.4) eine äquidistante Zerlegung des Intervalls gegeben. Für die summierte Boxregel

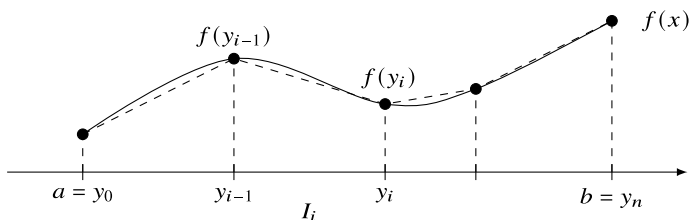


Abb. 10.2 Summierte Trapezregel zur Integralapproximation

$$I_h^0(f) = h \sum_{i=1}^m f(y_i)$$

gilt die Fehlerabschätzung

$$|I(f) - I_h^0(f)| \leq \frac{b-a}{2} h \max_{[a,b]} |f'|.$$

Beweis Aus Satz 10.8 folgern wir durch Summation über die Teilintervalle

$$I(f) - I_h^0(f) = \sum_{i=1}^m \frac{h^2}{2} f'(\xi_i)$$

mit Zwischenstellen $\xi_i \in [y_{i-1}, y_i]$. Das Ergebnis folgt nun mit $(b-a) = \sum_i h_i$ sowie durch Übergang zum Maximum. $\square$

Mit der summierten Boxregel haben wir also sogar für nur stückweise stetig differenzierbare Funktionen eine konvergente Quadraturformel für $h \rightarrow 0$. Entsprechend gilt für die Trapezregel:

Satz 10.16 (Summierte Trapezregel) Es sei $f \in C^2[a, b]$ sowie durch (10.4) eine äquidistante Zerlegung des Intervalls gegeben. Für die summierte Trapezregel

$$I_h^1(f) = \frac{h}{2} \sum_{i=1}^m (f(y_{i-1}) + f(y_i)) = \frac{h}{2} f(a) + h \sum_{i=1}^{m-1} f(y_i) + \frac{h}{2} f(b)$$

gilt die Fehlerabschätzung

$$|I(f) - I_h^1(f)| \leq \frac{b-a}{12} h^2 \max_{[a,b]} |f''|.$$

Beweis Übung! $\square$

Die summierte Trapezregel ist besonders attraktiv, da sich die Stützstellen überschneiden: Der Punkt y_i ist sowohl Stützstelle im Teilintervall $[y_{i-1}, y_i]$ als auch $[y_i, y_{i+1}]$ und $f(y_i)$ müssen nur einmal bestimmt werden. Darüber hinaus eignet sich die summierte Trapezregel als Grundlage von *adaptiven Quadraturverfahren*, bei denen die Genauigkeit Stück für Stück dem Problem angepasst wird: Wurde auf einer Zerlegung zur Schrittweite h die Approximation $I_h^1(f)$ bestimmt, so kann die Genauigkeit durch Intervallhalbierung $I_{h/2}^1(f)$ einfach gesteigert werden. Alle Stützstellen $f(y_i)$ können weiterverwendet werden, die Funktion $f(x)$ muss lediglich in den neuen Intervallmitten berechnet werden. Ein entsprechendes Resultat gilt für die summierte Simpson-Regel

$$\begin{aligned}
I_h^2(f) &= \sum_{i=1}^m \frac{h}{6} \left(f(y_{i-1}) + 4f\left(\frac{y_{i-1} + y_i}{2}\right) + f(y_i) \right) \\
&= \frac{h}{6} f(a) + \sum_{i=1}^m \frac{h}{3} f(y_i) + \sum_{i=1}^{m-1} \frac{2h}{3} f\left(\frac{y_{i-1} + y_i}{2}\right) + \frac{h}{6} f(b).
\end{aligned}$$

Bei geschickter Anordnung sind nur $2m + 1$ statt $3m$ Funktionsauswertungen notwendig.

10.3 Romberg-Quadratur

Die Idee der Richardson-Extrapolation in Abschn. 9.4, Approximationsprozesse hoher Ordnung zu erzielen, die auf Basismethoden niedriger Ordnung basieren, wird hier auf die numerische Quadratur angewendet. Konkret werden wir die summierte Trapezregel zur Extrapolation nutzen. Die Trapezregel gehört zu den sehr einfachen Verfahren mit niedriger Ordnung. Wir fassen die wesentlichen Ergebnisse aus Abschn. 10.2 zusammen. Auf einer gleichmäßigen Zerlegung des Intervalls $[a, b]$ in N Teilintervalle mit Schrittweite $h = 1/N$ ist die summierte Trapezregel zur Approximation von $\int_a^b f \, dx$ gegeben durch

$$I_h(f) = h \left(\frac{1}{2} f(a) + \sum_{j=1}^{N-1} f(x_j) + \frac{1}{2} f(b) \right), \quad x_j := a + jh,$$

und liefert im Fall $f \in C^2([a, b])$ die Fehlerabschätzung

$$I(f) - I_h(f) = \frac{b-a}{12} h^2 f^{(2)}(\xi)$$

mit einer Zwischenstelle $\xi \in [a, b]$. Um die Extrapolationsmethode erfolgreich anwenden zu können brauchen wir Kenntnis über die weitere Fehlerentwicklung der Trapezregel. Die Basis hierzu ist die Euler-Maclaurinsche Summenformel, die eine Entwicklung des Fehlers in geraden Potenzen von h zeigt:

Satz 10.17 (Euler-Maclaurinsche Summenformel) Es sei $f \in C^{2m+2}[0, 1]$. Dann besitzt die Trapezregel die Fehlerentwicklung

$$\begin{aligned}
\int_0^1 f(x) dx - \frac{1}{2} (f(0) + f(1)) &= \sum_{k=1}^m \frac{B_{2k}}{(2k)!} (f^{(2k-1)}(1) - f^{(2k-1)}(0)) \\
&\quad + h^{2m+2} \frac{B_{2m+2}}{(2m+2)!} f^{(2m+2)}(\xi)
\end{aligned}$$

mit $\xi \in [0, 1]$ und den Bernoulli-Zahlen B_{2k} .

Beweis Der Beweis zu dieser Fehlerabschätzung orientiert sich eng am Vorgehen zur einfachen Fehlerabschätzung der Trapezregel in Satz 10.10. Wir zeigen hier nur die Idee und verweisen auf die Literatur [35].

(i) Wir definieren eine Folge von Polynomen $b_k \in P$ mit den Eigenschaften

$$b_0 \equiv 1, \quad b'_{k+1} = (k+1)b_k. \quad (10.5)$$

Die Polynome sind hierdurch noch nicht eindeutig festgelegt, da in jedem Schritt beim Bilden der Stammfunktion eine frei wählbare Konstante hinzukommt. Es gilt aber $b_k \in P^k$ und der Koeffizient vor dem höchsten Monom ist stets eins. Nun gilt

$$\begin{aligned} \int_0^1 f(x) dx &= \int_0^1 f(x)b_0(x) dx = f b_0 \Big|_0^1 - \int_0^1 f'(x)b_1(x) dx \\ &= f(1)(1+c_1) - f(0)c_1 - \int_0^1 f'(x)b_1(x) dx. \end{aligned}$$

Wir wählen $c_1 = -\frac{1}{2}$ und erhalten

$$\int_0^1 f(x) dx - \frac{1}{2}(f(0) + f(1)) = - \int_0^1 f'(x)b_1(x) dx. \quad (10.6)$$

Nun gilt sukzessive für $k = 1, 2, \dots$

$$\int_0^1 f^{(k)}(x)b_k(x) dx = \frac{1}{k} f^{(k)}(x)b_{k+1} \Big|_0^1 - \frac{1}{k} \int_0^1 f^{(k+1)}(x)b_{k+1}(x) dx. \quad (10.7)$$

Die Polynome $b_k = b'_{k+1}$ sind bis auf jeweils eine Konstante vorgegeben. Wir können diese Konstanten so wählen, dass für ungerade b_{2k+1} gilt:

$$b_{2k+1}(0) = b_{2k+1}(1) = 0, \quad k = 1, 2, \dots$$

Die so definierten Polynome sind die *Bernoulli-Polynome* [59]. Durch $B_k := b_k(0)$ sind die zugehörigen *Bernoulli-Zahlen* gegeben. Für die ungeraden Bernoulli-Zahlen gilt $B_{2k+1} = 0$. Für die Bernoulli-Polynome gilt eine Symmetrie

$$b_k(x) = (-1)^k b_k(1-x), \quad k = 0, 1, \dots,$$

also gilt $B_k = b_k(0) = b_k(1)$ (da $B_k = 0$ für k ungerade).

(ii) Ausgehend von (10.6) und (10.7) gilt

$$\begin{aligned} \int_0^1 f(x) dx - \frac{1}{2}(f(0) + f(1)) &= \sum_{k=2}^{2m} \frac{(-1)^k}{k!} f^{(k-1)}(x)b_k \Big|_0^1 \\ &\quad + \frac{1}{(2m+1)!} \int_0^1 f^{(2m+1)}(x)b_{2m+1}(x) dx. \end{aligned}$$

Für alle ungeraden Summanden gilt mit $B_{2k+1} = b_{2k+1}(0) = b_{2k+1}(1)$

$$f^{(2k)} b_{2k+1} \Big|_0^1 = f^{(2k)}(1) b_{2k+1}(1) - f^{(2k)}(0) b_{2k+1}(0) = 0.$$

Für die geraden Summanden gilt mit $B_{2k} = b_{2k}(0) = b_{2k}(1)$

$$f^{(2k-1)} b_{2k} \Big|_0^1 = B_{2k} \left(f^{(2k-1)}(1) - f^{(2k-1)}(0) \right).$$

Es folgt

$$\begin{aligned} \int_0^1 f(x) dx - \frac{1}{2} (f(0) + f(1)) &= \sum_{k=1}^m \frac{B_{2k}}{(2k)!} \left(f^{(2k-1)}(1) - f^{(2k-1)}(0) \right) \\ &\quad + \frac{1}{(2m+1)!} \int_0^1 f^{(2m+1)}(x) b_{2m+1}(x) dx. \end{aligned}$$

Um das Restglied abzuschätzen, führen wir eine weitere partielle Integration durch:

$$\begin{aligned} \int_0^1 f^{(2m+1)}(x) b_{2m+1}(x) dx &= \frac{1}{2m+2} f^{(2m+1)} (b_{2m+2} - B_{2m+2}) \Big|_0^1 \\ &\quad - \frac{1}{2m+2} \int_0^1 f^{(2m+2)}(x) (b_{2m+2}(x) - B_{2m+2}) dx \end{aligned}$$

Wir nutzen hier, dass sowohl $b_{2m+2}(x)$ als auch $b_{2m+2}(x) - B_{2m+2}$ Stammfunktionen zu $(2m+2)b_{2m+1}$ sind. Der Randterm verschwindet, da $B_{2m+2} = b_{2m+2}(0) = b_{2m+2}(1)$. Es kann gezeigt werden, dass die Funktion

$$b_{2m+2}(x) - B_{2m+2}$$

auf dem Intervall $[0, 1]$ keinen Vorzeichenwechsel hat, siehe [35]. Dann folgt mit dem Mittelwertsatz der Integralrechnung

$$\int_0^1 f^{(2m+1)}(x) b_{2m+1}(x) dx = \frac{1}{2m+2} f^{(2m+2)}(\xi) \int_0^1 B_{2m+2} dx,$$

da $\int_0^1 b_{2m+2}(x) dx = 0$. Dies vervollständigt den Beweis. □

Wir können nun eine Variante der Summenformel für die summierte Trapezregel angeben:

Satz 10.18 (Euler-Maclaurinsche Summenformel für die summierte Trapezregel) Es sei $I = [a, b]$ und $h = (b - a)/N$ für ein $N \in \mathbb{N}$. Für die Funktion $f^{2m+2}[a, b]$ gilt die Euler-Maclaurinsche Summenformel in der Form

$$\begin{aligned}
\int_a^b f(x) dx - h \left(\frac{f(a)}{2} + f(x_1) + \cdots + f(x_{n-1}) + \frac{f(b)}{2} \right) \\
= \sum_{k=1}^m h^{2k} \frac{B_{2k}}{(2k)!} \left(f^{(2k-1)}(b) - f^{(2k-1)}(a) \right) \\
+ h^{2m+2} \frac{b-a}{(2m+2)!} B_{2m+2} f^{(2m+2)}(\xi)
\end{aligned}$$

mit einem Zwischenwert $\xi \in [a, b]$, den Bernoulli-Zahlen B_{2k} sowie den Stützstellen

$$x_n = a + hn.$$

Beweis Diese allgemeine Form folgt durch Transformation der Teilintervalle $[x_{n-1}, x_n]$ auf ein Referenzintervall $[0, 1]$. Auf jedem der Teilintervalle wird Satz 10.17 angewendet. Aufgrund von

$$\lim_{s \downarrow 0} f^{(k)}(x_n + s) = \lim_{s \downarrow 0} f^{(k)}(x_n - s), \quad n = 1, \dots, N-1,$$

fallen alle Ableitungsterme in den inneren Punkten weg. □

Die Bernoulli-Zahlen sind definiert als die Koeffizienten der Taylor-Reihendarstellung von

$$\frac{x}{e^x - 1} = \sum_{k=0}^{\infty} \frac{B_k}{k!} x^k = 1 - \frac{1}{2}x + \frac{1}{6} \frac{x^2}{2!} - \frac{1}{30} \frac{x^4}{4!} + \frac{1}{42} \frac{x^6}{6!} + \dots$$

und genügen der Rekursionsformel

$$B_0 = 0, \quad B_k = - \sum_{j=0}^{k-1} \frac{k!}{j!(k-j+1)!} B_j, \quad k = 1, 2, \dots$$

Die ersten Bernoulli-Zahlen sind gegeben als

$$1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, 0, -\frac{1}{30}, \dots$$

Ausgenommen $B_1 = -\frac{1}{2}$ gilt für jede zweite (ungerader Index) Bernoulli-Zahl $B_{2k+1} = 0$. Ansonsten folgen sie keinem erkennbaren Gesetz. Für große k wachsen die Bernoulli-Zahlen sehr schnell an und verhalten sich asymptotisch wie (siehe z. B. [63])

$$|B_{2k}| \sim 2(2k)!(2\pi)^{-2k}, \quad k \rightarrow \infty.$$

Die Euler-Maclaurinsche Summenformel besagt, dass die Trapezregel eine Entwicklung in geraden Potenzen von h besitzt. Daher eignet sie sich gleich in zweifacher Hinsicht optimal

zur Extrapolation: Aufgrund der quadratischen Fehlerentwicklung gewinnen wir in jedem Extrapolationsschritt zwei Ordnungen Genauigkeit und aufgrund der Stützstellenwahl an den Enden der Teilintervalle x_{j-1} und x_j können bereits berechnete Werte $f(x_j)$ zu einer Schrittweite h bei der nächstfeineren Approximation zu $h/2$ weiterverwendet werden. Zur Approximation von

$$a(0) \approx \int_a^b f(x) dx, \quad a(h) := I_h(f) = h \left(\frac{1}{2} f(a) + \sum_{j=1}^{N-1} f(x_j) + \frac{1}{2} f(b) \right)$$

verwenden wir das Extrapolationsprinzip aus Abschn. 9.4:

Algorithmus 10.1: Romberg-Quadratur

Eingabe : Gegeben sei eine Folge von Schrittweiten h_k für $k = 1, 2, \dots$ mit $\frac{h_{k+1}}{h_k} \leq \rho < 1$.

1 **Für** $k = 0$ **bis** m **tue**

2 Werte summierte Quadraturformel aus $a_k := a(h_k)$

3 **Für** $k = 0$ **bis** m **tue**

4 Extrapoliere die Paare $(h_k^2, a(h_k))$ mit Polynomen in h^2

Ergebnis : Approximation von $a(0) = I_0(f)$

Als Schrittweitenfolge kann mit einem $h > 0$ die einfache Vorschrift

$$h_k = 2^{-k} h$$

verwendet werden. Diese Folge wird *Romberg-Folge* genannt und hat den Vorteil, dass bereits berechnete Stützstellen weiter verwendet werden können. Der Nachteil dieser Folge ist das schnelle Wachstum der Anzahl der Stützstellen. Die Extrapolation selbst wird mit dem Neville-Schema aus Algorithmus 9.1 durchgeführt.

Die Diagonalelemente $a_{k,k}$ sind gerade die Näherungen zu $a(0)$. Basierend auf Satz 9.30 zur allgemeinen Richardson-Extrapolation erhalten wir die Fehlerabschätzung:

Satz 10.19 (Romberg-Quadratur) Es sei $f^{2m+2}[a, b]$ sowie $h > 0$ gegeben. Das Romberg-Verfahren zur Schrittweitenfolge $h_k = 2^{-k}h$, $k = 0, \dots, m$ liefert nach m Extrapolationsschritten die Approximation

$$I(f) - a_{m,m} = O(h^{2m+2}).$$

Beweis Der Beweis folgt durch Kombination von Satz 9.30 über die Richardson-Extrapolation mit der Euler-Maclaurinschen Summenformel aus Satz 10.17. $\square$

Bemerkung 10.20 Anstatt der Romberg-Folge $h_k = 2^{-k}h$ kann auch mit der Burlisch-Folge

$$h_k = \begin{cases} h \cdot 2^{-\frac{k}{2}} & k \text{ gerade,} \\ \frac{h}{3} 2^{-\frac{k-1}{2}} & k \text{ ungerade} \end{cases}$$

gearbeitet werden, da diese wesentlich weniger Funktionsauswertungen benötigt. Bei $h = 1$ sind die ersten Folgenglieder gegeben durch

$$\frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{1}{6}, \frac{1}{8}, \frac{1}{12}, \dots$$



Beispiel 10.21 (Rombergsches Quadraturverfahren)

Wir untersuchen das Integral

$$I(f) = \int_0^1 \exp(1 + \sin^2(x)) \, dx \approx 4.7662017991376737307.$$

Für die zu integrierende Funktion gilt $f \in C^\infty(\mathbb{R})$. Das Romberg-Verfahren sollte sich also gut anwenden lassen. In der folgenden Tabelle führen wir das Romberg-Verfahren mit der summierten Trapezregel durch:

h	$I_h(f) = a(h)$	$I(f) - I_h(f)$	$a^{(1)}(h)$	$I(f) - a^{(1)}(h)$	$a^{(2)}(h)$	$I(f) - a^{(2)}(h)$
2^{-0}	4.11830	$-4.56 \cdot 10^{-1}$	3.65324	$9.17 \cdot 10^{-3}$	3.6623410	$7.47 \cdot 10^{-5}$
2^{-1}	3.76951	$-1.07 \cdot 10^{-1}$	3.66177	$6.43 \cdot 10^{-4}$	3.6624169	$1.24 \cdot 10^{-6}$
2^{-2}	3.68871	$-2.63 \cdot 10^{-2}$	3.66238	$3.91 \cdot 10^{-5}$	3.6624157	$2.30 \cdot 10^{-8}$
2^{-3}	3.66896	$-6.54 \cdot 10^{-3}$	3.66241	$2.42 \cdot 10^{-6}$	3.6624157	$3.65 \cdot 10^{-10}$
2^{-4}	3.66405	$-1.63 \cdot 10^{-3}$	3.66242	$1.51 \cdot 10^{-7}$	3.6624157	$5.72 \cdot 10^{-12}$
2^{-5}	3.66282	$-4.08 \cdot 10^{-4}$	3.66242	$9.42 \cdot 10^{-9}$	3.6624157	$8.93 \cdot 10^{-14}$
2^{-6}	3.66252	$-1.02 \cdot 10^{-4}$	3.66242	$5.89 \cdot 10^{-10}$		
2^{-7}	3.66244	$-2.55 \cdot 10^{-5}$				
Ordnung		$O(h^2)$		$O(h^4)$		$O(h^6)$

Bei diesem Beispiel wird beim Rombergschen Quadratur-Verfahren bei nur zwei Extrapolationsschritten sehr schnell die Maschinengenauigkeit erreicht.

Wir betrachten als zweites Beispiel die Funktion

$$f(x) = |\sin(x) - 0.75|, \quad I(f) = \int_0^1 f(x) \, dx \approx 0.304666468.$$

Hier ergibt sich bei Extrapolation der Trapezregel keine wesentliche Verbesserung:

h	$I_h(f) = a(h)$	$I(f) - I_h(f)$	$a^{(1)}(h)$	$I(f) - a^{(1)}(h)$	$a^{(2)}(h)$	$I(f) - a^{(2)}(h)$
2^{-0}	0.42074	$-1.16 \cdot 10^{-1}$	0.32063	$1.60 \cdot 10^{-2}$	0.3045314	$1.35 \cdot 10^{-4}$
2^{-1}	0.34565	$-4.10 \cdot 10^{-2}$	0.30554	$8.71 \cdot 10^{-4}$	0.3036544	$1.01 \cdot 10^{-3}$
2^{-2}	0.31557	$-1.09 \cdot 10^{-2}$	0.30377	$8.94 \cdot 10^{-4}$	0.3049991	$3.33 \cdot 10^{-4}$
2^{-3}	0.30672	$-2.05 \cdot 10^{-3}$	0.30492	$2.56 \cdot 10^{-4}$	0.3045325	$1.34 \cdot 10^{-4}$
2^{-4}	0.30537	$-7.06 \cdot 10^{-4}$	0.30456	$1.10 \cdot 10^{-4}$	0.3046924	$2.59 \cdot 10^{-5}$
2^{-5}	0.30476	$-9.42 \cdot 10^{-5}$	0.30468	$1.75 \cdot 10^{-5}$	0.3046680	$1.58 \cdot 10^{-6}$
2^{-6}	0.30470	$-3.66 \cdot 10^{-5}$	0.30467	$2.57 \cdot 10^{-6}$		
2^{-7}	0.30468	$-1.11 \cdot 10^{-5}$				

Konvergenzordnungen lassen sich bei diesem Beispiel nicht klar bestimmen. Der Grund hierfür ist die fehlende Regularität der Funktion f . Die Funktion ist zwar integrierbar, ihre Ableitungen haben jedoch an der Stelle $\arcsin(0.75) \approx 0.86$ Singularitäten. ◀

Integration periodischer Funktionen

Im Fall periodischer Funktionen, d.h. es sei $f^{2m+2}(-\infty, \infty)$ mit dem Periodenintervall $[a, b]$ und

$$f^{(2k-1)}(a) = f^{(2k-1)}(b), \quad k = 1, 2, \dots,$$

kann die Euler-Maclaurinsche Summenformel *entscheidend* vereinfacht werden. Aus

$$\begin{aligned} I(f) - I_h(f) &= \sum_{k=1}^m h^{2k} \frac{B_{2k}}{(2k)!} \left(f^{(2k-1)}(b) - f^{(2k-1)}(a) \right) \\ &\quad + h^{2m+2} \frac{b-a}{(2m+2)!} B_{2m+2} f^{(2m+2)}(\xi) \end{aligned}$$

mit

$$I_h(f) = h \left(\frac{1}{2} f(a) + \sum_{j=1}^{N-1} f(x_j) + \frac{1}{2} f(b) \right), \quad x_j := a + jh,$$

folgt dann

$$I(f) - I_h(f) = h^{2m+2} \frac{b-a}{(2m+2)!} B_{2m+2} f^{(2m+2)}(\xi) = O(h^{2m+2})$$

mit

$$\begin{aligned} I_h(f) &= h \left(\frac{1}{2} f(a) + \sum_{j=1}^{N-1} f(x_j) + \frac{1}{2} f(b) \right) \\ &= h \left(f(a) + \sum_{j=1}^{N-1} f(x_j) \right) = h \left(\sum_{j=0}^{N-1} f(x_j) \right) \end{aligned}$$

wobei $x_j := a + jh$. Die summierte Trapezregel *vereinfacht* sich dadurch zur summierten linksseitigen Boxregel.

Falls $f \in C^\infty(-\infty, \infty)$, dann konvergiert die summierte Trapezregel (d.h. bei $[a, b]$ -periodischen Funktionen die summierte Boxregel) schneller gegen den Integralwert für $h \rightarrow 0$ als jede andere Quadraturregel.

Beispiel 10.22 (Quadratur periodischer Funktionen)

Wir betrachten wie oben die Funktion

$$f(x) = \exp(1 + \sin^2(x)),$$

nun aber das Integral

$$I(f) = \int_0^\pi f(x) \, dx \approx 14.97346455769737.$$

Die summierte Trapezregel liefert:

h	$I_h(f) = a(h)$	$I(f) - I_h(f)$
$2^{-0}\pi$	8.53973	$6.43 \cdot 10^0$
$2^{-1}\pi$	15.87657	$-9.03 \cdot 10^{-1}$
$2^{-2}\pi$	14.97811	$-4.64 \cdot 10^{-3}$
$2^{-3}\pi$	14.97346	$-1.07 \cdot 10^{-8}$
$2^{-4}\pi$	14.97346	$< \text{eps}$



10.4 Gauß-Quadratur

Die interpolatorischen Quadraturformeln

$$I^{(n)}(f) = \sum_{i=0}^n \alpha_i f(x_i)$$

zu den Stützstellen $x_0, \dots, x_n \in [a, b]$ sind nach Konstruktion mindestens von Ordnung $n + 1$, das heißt

$$I(p) = I^{(n)}(p) \quad \forall p \in P_n.$$

Wir haben jedoch mit der Mittelpunktsregel $n = 0$ und der Simpson-Regel $n = 2$ bereits Quadraturformeln kennengelernt, die exakt sind für alle Polynome P_{n+1} , die also von der Ordnung $n + 2$ sind.

Wir untersuchen in diesem Abschnitt die Frage, ob die Ordnung weiter erhöht werden kann. Bisher lag die Freiheit lediglich in der Wahl des Polynomgrads und somit in der Anzahl der Stützstellen. Die Frage nach der optimalen Verteilung der Stützstellen im Intervall $[a, b]$ wurde bisher nicht untersucht. In diesem Abschnitt werden wir mit der *Gauß-Quadratur* interpolatorische Quadraturformeln kennenlernen, welche durch optimale Positionierung der Stützstellen die maximale Ordnung $2n + 2$ erreichen.

Satz 10.23 (Ordnungsbarriere der Quadratur) Eine interpolatorische Quadraturformel zu $n + 1$ Stützstellen kann höchstens die Ordnung $2n + 2$ haben.

Beweis Angenommen, $I^{(n)}(\cdot)$ mit den Stützstellen $x_0, \dots, x_n$ wäre von höherer Ordnung, d. h., sie wäre exakt auch für Polynome von Grad $2n + 2$, insbesondere exakt für das folgende Polynome aus P_{2n+2}

$$p(x) = \prod_{j=0}^n (x - x_j)^2 \in P_{2n+2}.$$

Die eindeutige Interpolierende $p_n \in P_n$ mit $p_n(x_i) = p(x_i) = 0$ hat $n + 1$ Nullstellen und stellt somit die Nullfunktion dar. Dies ergibt einen Widerspruch:

$$0 < \int_a^b p(x) dx = I^{(n)}(p) = I^{(n)}(p_n) = 0 \quad \square$$

Im Folgenden untersuchen wir interpolatorische Quadraturregeln genauer und versuchen Bedingungen herzuleiten, unter denen die höchstmögliche Ordnung $2n + 2$ wirklich erreicht werden kann. Wir müssen demnach eine Quadraturformel in $n + 1$ Stützstellen finden, die exakt ist für alle Polynome aus P_{2n+1} . Hierzu wählen wir zunächst eine Quadraturformel mit den $2n + 2$ Stützstellen $x_0, x_1, \dots, x_n, x_{n+1}, \dots, x_{2n+1}$. Von dieser wissen wir, dass sie auf jeden Fall der Ordnung $2n + 2$ ist. Es gilt in Newtonscher Darstellung des Interpolationspolynoms

$$\begin{aligned} I(f) - I^{2n+1}(f) &= I(f) - \sum_{i=0}^{2n+1} f[x_0, \dots, x_i] \int_a^b \prod_{j=0}^{i-1} (x - x_j) dx \\ &= I(f) - I^n(f) - \sum_{i=n+1}^{2n+1} f[x_0, \dots, x_i] \int_a^b \prod_{j=0}^{i-1} (x - x_j) dx, \end{aligned}$$

wobei $I^n(\cdot)$ diejenige Quadraturregel ist, welche durch die ersten $n + 1$ Stützstellen $x_0, \dots, x_n$ gegeben ist. Wir versuchen nun die Stützstellen $x_0, \dots, x_{2n+1}$ so zu bestimmen, dass das Restglied

$$\sum_{i=n+1}^{2n+1} f[x_0, \dots, x_i] \int_a^b \prod_{j=0}^{i-1} (x - x_j) dx = 0$$

für alle Funktionen $f \in C^{2n+2}([a, b])$ verschwindet. Das bedeutet, dass die Hinzunahme der Stützstellen $x_{n+1}, \dots, x_{2n+1}$ nicht notwendig ist:

$$I(f) - I^{2n+1}(f) = I(f) - I^n(f) \Rightarrow I^{2n+1}(f) = I^n(f)$$

Die Formel $I^n(f)$ mit $n+1$ Stützstellen hat dann bereits die Ordnung $2n+1$. Die Newton-Basispolynome für $i = n+1, \dots, 2n+1$ müssen die folgende Bedingung erfüllen:

$$\int_a^b \underbrace{\prod_{j=0}^{i-1} (x - x_j)}_{\in P_{2n+1}} dx = \int_a^b \underbrace{\prod_{j=0}^n (x - x_j)}_{\in P_{n+1}} \underbrace{\prod_{j=n+1}^{i-1} (x - x_j)}_{\in P_n} dx \stackrel{!}{=} 0$$

Ziel ist also eine Bestimmung der ersten $n+1$ Stützstellen $x_0, \dots, x_n$, sodass das Integral

$$\int_a^b \underbrace{\prod_{j=0}^n (x - x_j)}_{=: p_{n+1}(x)} q(x) dx = 0, \quad \forall q \in P_n, \quad (10.8)$$

für alle $q \in P_n$ verschwindet. Wählen wir eine Basis des P_n (mit Dimension $n+1$), so ergibt sich ein System von $n+1$ Gleichungen für die $n+1$ Unbekannten $x_0, \dots, x_n$. Geometrisch betrachtet handelt es sich hier um eine Orthogonalisierungsaufgabe bzgl. des L^2 -Skalarprodukts

$$(f, g)_{L^2[a,b]} := \int_a^b f(x)g(x) dx.$$

Kurz geschrieben: Suche $x_0, \dots, x_n \in \mathbb{R}$, sodass für $p_{n+1}(x) := \prod_{i=0}^n (x - x_i)$

$$(p_{n+1}, q) = 0 \quad \forall q \in P_n \quad (10.9)$$

gilt. Der Raum der Polynome P_n ist ein linearer Vektorraum. Durch $(\cdot, \cdot)$ ist ein Skalarprodukt gegeben. Mithilfe des Gram-Schmidt-Verfahrens, Satz 9.10, lässt sich aus der Monombasis $\{1, x, \dots, x^n\}$ ein Polynom $p_{n+1} \in P_{n+1}$ erzeugen, welches auf allen $q \in P_n$ orthogonal ist. Dies löst jedoch noch nicht das Problem, eine entsprechende Quadraturformel zu finden, da nicht gesagt ist, dass dieses Polynom p_{n+1} auch $n+1$ paarweise verschiedene Nullstellen im Intervall $[a, b]$ besitzt.

Bevor wir die allgemeine Lösbarkeit dieser Aufgabe betrachten, untersuchen wir zunächst als einfaches Beispiel die Fälle $n = 0$ sowie $n = 1$:

Beispiel 10.24 (Optimale Stützstellenwahl durch Orthogonalisierung)

Wir wollen die Polynome $p_1(x) = (x - x_0)$ sowie $p_2(x) = (x - x_0)(x - x_1)$ bestimmen, sodass sie im L^2 -Skalarprodukt orthogonal auf allen Polynomen von Grad 0 bzw. 1 stehen. Ohne Einschränkung betrachten wir das Intervall $[-1, 1]$:

Für $n = 0$ gilt mit $q \equiv \alpha_0 \in P_0$

$$0 \stackrel{!}{=} (x - x_0, \alpha_0)_{L^2([-1,1])} = -2x_0\alpha_0 \quad \forall \alpha_0 \in \mathbb{R},$$

also muss $x_0 = 0$ gelten.

Im Fall $n = 1$ gilt mit $q = \alpha_0 + \beta_0 x$

$$\begin{aligned} 0 &\stackrel{!}{=} ((x - x_0)(x - x_1), \alpha_0 + \beta_0 x)_{L^2([-1,1])} \\ &= \frac{-2(x_0 + x_1)}{3} \beta_0 + \frac{2(1 + 3x_0 x_1)}{3} \alpha_0 \quad \forall \alpha_0, \beta_0 \in \mathbb{R}. \end{aligned}$$

Aus $\alpha_0 = 0$ sowie $\beta_0 = 1$ folgern wir $x_0 = -x_1$ und aus $\alpha_0 = 1$ und $\beta_0 = 0$ folgt hiermit $x_{1/2} = \pm \frac{1}{\sqrt{3}}$. Die zugehörigen Gewichte bestimmen wir gemäß

$$\alpha_i = \int_a^b L_i^{(n)}(x) dx.$$

Vergleiche hierzu auch Formel (10.1) zu

$$\alpha_0^0 = \int_{-1}^1 1 dx = 2, \quad \alpha_0^1 = \int_{-1}^1 \frac{x - \frac{-1}{\sqrt{3}}}{\frac{1}{\sqrt{3}} - \frac{-1}{\sqrt{3}}} dx = 1, \quad \alpha_1^1 = \int_{-1}^1 \frac{x - \frac{1}{\sqrt{3}}}{\frac{-1}{\sqrt{3}} - \frac{1}{\sqrt{3}}} dx = 1,$$

und wir erhalten die beiden ersten Gauß-Quadraturformeln

$$I_G^0(f) = 2f(0), \quad I_G^1(f) = f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right).$$

Die erste Formel ist gerade die Mittelpunktsregel, von der wir bereits die Ordnung zwei (also $2n + 2 = 2 \cdot 0 + 2$) kennen. Die zweite Formel hat die Ordnung 4, dies kann einfach durch Integration der Monome $1, x, x^2, x^3$ überprüft werden. ◀

10.4.1 Gauß-Legendre-Quadratur

In diesem Abschnitt werden allgemeine Resultate zur Existenz und Eindeutigkeit von Gaußschen Quadraturregeln $I_G^n(f)$ untersucht. Wir betrachten zunächst immer das Intervall $[-1, 1]$. Der Übergang zu allgemeinen Integrationsgebieten erfolgt im Anschluss durch eine Integraltransformation.

Bemerkung 10.25 (Transformation von Quadraturregeln) Es sei durch

$$\int_{-1}^1 f(x) dx \approx I_n(f) = \sum_{i=0}^n \alpha_i f(x_i)$$

eine numerische Quadraturregel auf dem Intervall $[-1, 1]$ mit Stützstellen $x_i \in [-1, 1]$ gegeben. Der Übergang zu einem allgemeinen Intervall $[a, b]$ erfolgt durch lineare Transformation

$$z = a + \frac{b-a}{2}(x+1)$$

und es ergibt sich die Quadraturregel

$$\int_a^b f(z) dz \approx \frac{b-a}{2} \sum_{i=1}^n \alpha_i f\left(a + \frac{b-a}{2}(x_i+1)\right).$$



► **Definition 10.26 (Gauß-Quadratur)** Eine Quadraturformel zur Integration einer Funktion $f : [a, b] \rightarrow \mathbb{R}$ mit

$$I_G^n(f) = \sum_{k=0}^n \alpha_k f(x_k)$$

mit $n+1$ Quadratur-Stützstellen wird Gauß-Quadraturformel genannt, falls alle Polynome $p \in P_{2n+1}$ exakt integriert werden, d. h. falls gilt:

$$\int_{-1}^1 p(x) dx = \sum_{k=0}^n \alpha_k p(x_k) \quad \forall p \in P_{2n+1}$$

Die beiden im Beispiel hergeleiteten Quadraturregeln sind somit Gaußsche Quadraturformeln. Wir wollen im Folgenden die Existenz von Gaußschen Quadraturregeln mit allgemeiner Ordnung, also für beliebige Stützstellenzahl $n \in \mathbb{N}$ untersuchen. Zunächst weisen wir nach, dass die Stützstellen der Gauß-Quadratur stets Nullstellen von orthogonalen Polynomen sein müssen:

Satz 10.27 Es seien $x_0, \dots, x_n$ paarweise verschiedene Quadratur-Stützstellen einer Gauß-Quadraturformel $I^n(\cdot)$. Dann gilt die Orthogonalitätsbeziehung

$$\int_{-1}^1 p_{n+1}(x)q(x) dx = 0 \quad \forall q \in P_n, \quad p_{n+1}(x) := (x-x_0) \cdots (x-x_n).$$

Beweis Diese Aussage wurde bereits in der Herleitung der Gauß-Regeln bewiesen, vergleiche (10.8) und (10.9). Falls die Quadraturregel mit $n+1$ Punkten die Ordnung $2n+2$ hat, also eine Gaußsche Regel ist, so muss das Restglied der möglichen Punkte $x_{n+1}, \dots, x_{2n+1}$ wegfallen. Dies entspricht gerade der Orthogonalitätsbeziehung in (10.8). $\square$

Falls eine interpolatorische Quadraturformel also die Gauß-Ordnung $2n+2$ besitzt, so bilden die $n+1$ Stützstellen ein Polynom $p_{n+1}(x) = \prod (x-x_i)$, welches orthogonal auf

P_n steht. Es bleibt zu zeigen, dass die Nullstellen eines orthogonalen Polynoms auch immer die Stützstellen einer Gauß-Formel bilden, d. h. die Rückrichtung.

Satz 10.28 *Es sei $p_{n+1} \in P_{n+1}$ ein Polynom*

$$p_{n+1}(x) = \prod_{j=0}^n (x - x_j), \quad (10.10)$$

mit paarweise verschiedenen und reellen Nullstellen $x_j \in (-1, 1)$. Dieses Polynom stehe L^2 -orthogonal auf allen Polynomen $q \in P_n$, d. h.

$$\int_{-1}^1 p_{n+1}(x)q(x) dx = 0 \quad \forall q \in P_n.$$

Dann ist durch die Stützstellen $x_0, \dots, x_n$ und Gewichte

$$\alpha_i = \int_{-1}^1 L_i^{(n)}(x) dx = \int_{-1}^1 \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j} dx$$

eine Gaußsche Quadraturformel von Ordnung $2n + 2$ gegeben.

Beweis Es sei p_{n+1} wie in (10.10) gegeben und es sei $f \in C^{2n+2}([-1, 1])$ beliebig. Des Weiteren sei $p_n \in P_n$ das Interpolationspolynom durch die Stützstellen $x_0, \dots, x_n$. Darüber hinaus sei durch p_{2n+1} ein zweites Interpolationspolynom durch die Stützstellen $x_0, \dots, x_n$ sowie durch weitere (paarweise verschiedene) Stützstellen $x_{n+1}, \dots, x_{2n+1}$ gegeben. Damit wird durch p_{2n+1} , mit $2n + 2$ paarweise verschiedenen Stützstellen, mindestens eine Quadraturregel der Ordnung $2n + 2$ erzeugt. In Newtonscher Darstellung gilt für p_{2n+1} und p_n

$$\begin{aligned} p_{2n+1}(x) &= \sum_{i=0}^{2n+1} f[x_0, \dots, x_i] \prod_{j=0}^{i-1} (x - x_j) \\ &= p_n(x) + \sum_{i=n+1}^{2n+1} f[x_0, \dots, x_i] \prod_{j=0}^{i-1} (x - x_j). \end{aligned}$$

Für die Differenz von p_{2n+1} und p_n gilt

$$p_{2n+1}(x) - p_n(x) = \sum_{i=n+1}^{2n+1} f[x_0, \dots, x_i] \underbrace{\prod_{j=0}^n (x - x_j)}_{=: p_{n+1}(x)} \underbrace{\prod_{j=n+1}^{i-1} (x - x_j)}_{=: q(x)}$$

mit einem Polynom $q \in P_n$. Da $p_{n+1} \perp q$, folgt

$$\int_{-1}^1 p_n(x) \, dx = \int_{-1}^1 p_{2n+1}(x) \, dx.$$

Das heißt, die Stützstellen $x_0, \dots, x_n$ erzeugen bereits eine Gaußsche Quadraturregel mit der Ordnung $2n + 2$. $\square$

Diese beiden Sätze besagen, dass die Charakterisierung von Gaußschen Quadraturregeln durch Nullstellen orthogonaler Polynome eindeutig ist. Diese Frage haben wir bereits in Abschn. 9.5.1 bei der Gauß-Approximation beantwortet und in Satz 9.36 die orthogonalen Legendre-Polynome kennengelernt. In dem Satz wurde auch eine schnelle, zweistufige Rekursionsformel zur Berechnung der orthogonalen Basis hergeleitet. In (9.23) geben wir die ersten dieser Polynome an.

Zum Aufstellen der Gauß-Formeln benötigen wir die Nullstellen dieser Polynome. Für diese existiert keine geschlossene Formel und bei höherem Polynomgrad bleibt nur die numerische Bestimmung, die einfach mit dem Newton-Verfahren erfolgen kann. Die entsprechenden Quadraturgewichte können einfach über die Formel (10.1) berechnet werden:

$$\alpha_k = \int_{-1}^1 \prod_{j=0, j \neq k}^n \frac{x - x_j}{x_k - x_j} \, dx$$

In Tab. 10.2 fassen wir die Stützstellen und Gewichte der ersten Gauß-Legendre-Regeln zusammen.

Satz 10.29 (Nullstellen orthogonaler Polynome) Die bezüglich des $L^2([-1, 1])$ -Skalarprodukts orthogonalen Polynome p_n besitzen reelle, einfache Nullstellen im Intervall $(-1, 1)$.

Tab. 10.2 Einige Gauß-Legendre-Quadraturregeln

# Punkte	$n - 1$	x_i	α_i	Ordnung	Exakt
1	0	0	2	2	1
2	1	$\pm\sqrt{\frac{1}{3}}$	1	4	3
3	2	0 $\pm\sqrt{\frac{3}{5}}$	$\frac{8}{9}$ $\frac{5}{9}$	6	5
4	3	± 0.8611363116 ± 0.3399810436	0.347854845 0.652145154	8	7
5	4	0 ± 0.9061798459 ± 0.5384693101	0.568888889 0.236926885 0.478628670	10	9

Beweis Der Beweis wird über Widerspruchsargumente geführt. Die Polynome $p_n(x)$ sind alle reellwertig.

(i) Wir nehmen zunächst an, dass p_n eine reelle Nullstelle λ hat, die aber nicht im Intervall $(-1, 1)$ liegt. Wir definieren

$$q(x) := \frac{p_n(x)}{x - \lambda}.$$

Die Funktion q ist ein Polynom $q \in P_{n-1}$. Daher gilt $q \perp p_n$. Dies impliziert

$$0 = (q, p_n) = \int_{-1}^1 \frac{p_n^2(x)}{x - \lambda} dx.$$

Dies ist aber ein Widerspruch, da $p_n(x)^2$ auf $[-1, 1]$ stets positiv ist (da p_n reellwertig ist), noch die Nullfunktion ist und darüber hinaus der Faktor $(x - \lambda)^{-1}$ keinen Vorzeichenwechsel haben kann, da λ außerhalb von $[-1, 1]$ liegt. Deshalb kann p_n keine Nullstelle außerhalb von $(-1, 1)$ haben.

(ii) Im zweiten Teil zeigen wir, dass im Intervall $(-1, 1)$ nur einfache und reelle Nullstellen liegen. Wir arbeiten wieder mit einem Widerspruchsargument. Wir nehmen an, dass p_n eine Nullstelle λ besitzt, die entweder mehrfach oder nicht reell ist. Falls λ nicht reell ist, so ist auch durch $\bar{\lambda}$ eine Nullstelle gegeben. In beiden Fällen definieren wir

$$q(x) := \frac{p_n(x)}{(x - \lambda)(x - \bar{\lambda})} = \frac{p_n(x)}{|x - \lambda|^2}.$$

Das Polynom $q(x)$ liegt in P_{n-2} . Es gilt

$$0 = (p_n, q) = \int_{-1}^1 \frac{p_n^2(x)}{|x - \lambda|^2} dx.$$

Wieder folgt ein Widerspruch, da p_n^2 und $|x - \lambda|^2$ beide größer gleich null, nicht aber die Nullfunktion sind. $\square$

Mit diesen Vorarbeiten sind wir in der Lage, den Hauptsatz dieses Abschnitts aufzustellen:

Satz 10.30 (Existenz und Eindeutigkeit der Gauß-Quadratur) Zu $n \in \mathbb{N}$ existiert eine eindeutige Gauß-Quadraturformel der Ordnung $2n + 2$ mit $n + 1$ paarweise verschiedenen Stützstellen, die als Nullstellen des orthogonalen Polynoms $p_{n+1} \in P_{n+1}$ gegeben sind.

Beweis Der Beweis besteht aus der Zusammenfassung der bisherigen Resultate:

1. Satz 9.36 liefert die eindeutige Existenz des orthogonalen Polynoms $p_{n+1} \in P_{n+1}$.
2. Satz 10.29 garantiert, dass dieses Polynom $n + 1$ paarweise verschiedene, reelle Nullstellen besitzt.

3. Satz 10.28 besagt, dass durch Wahl der Nullstellen als Stützstellen bei entsprechenden Gewichten eine Quadraturformel der Ordnung $2n+2$, also eine Gauß-Quadratur gegeben ist, siehe Definition 10.26.
4. Schließlich besagt Satz 10.27, dass nur durch diese Wahl der Stützstellen eine Gauß-Quadraturformel erzeugt wird, liefert also die Eindeutigkeit des Konstruktionsprinzips.

Damit ist der Beweis geführt. $\square$

Ein Nachteil der Newton-Cotes-Formeln sind negative Gewichte ab $n \geq 8$ für geschlossene Formeln. Die Gauß-Quadraturgewichte hingegen sind stets positiv.

Satz 10.31 (Gewichte der Gauß-Quadratur) Die Gewichte der Gauß-Quadratur zu den Stützstellen $x_0, \dots, x_n$ sind stets positiv und es gilt

$$\alpha_k = \int_{-1}^1 L_k^{(n)}(x) dx = \int_{-1}^1 L_k^{(n)}(x)^2 dx > 0.$$

Beweis Es sei durch $I_G^n(f) = \sum_k \alpha_k f(x_k)$ die Gauß-Quadratur zu den $n+1$ Stützstellen $x_0, \dots, x_n$ gegeben. Für die Lagrangeschen Basispolynome gilt

$$L_i^{(n)}(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}, \quad L_i^{(n)}(x_k) = \delta_{ik}.$$

Es gilt $L_i^{(n)} \in P_n$, also wird auch $(L_i^{(n)})^2$ von der Gauß-Formel exakt integriert. Somit ist

$$0 < \int_{-1}^1 L_k^{(n)}(x)^2 dx = \sum_{i=0}^n \alpha_i L_k^{(n)}(x_i)^2 = \alpha_k.$$

$\square$

Die Gaußschen Quadraturformeln haben neben der hohen Ordnung den weiteren Vorteil, dass bei beliebiger Ordnung alle Gewichte positiv sind, also keine Auslöschung auftritt.

Weiter haben wir gesehen, dass die Newton-Cotes-Formeln unter dem gleichen Mangel wie die Lagrangesche Interpolation leiden: Falls die Ableitungen des Integranden zu schnell steigen, muss keine Konvergenz bei steigendem Polynomgrad vorliegen. Im Fall der Gauß-Quadratur erhalten wir hingegen Konvergenz des Integrals:

Satz 10.32 (Konvergenz der Gauß-Quadratur) Es sei $f \in C^\infty([-1, 1])$. Die Folge $(I^{(n)}(f))_n$, $n = 1, 2, \dots$ der Gauß-Formeln zur Berechnung von

$$I(f) = \int_{-1}^1 f(x) dx$$

ist konvergent:

$$I^{(n)}(f) \rightarrow I(f) \quad (n \rightarrow \infty)$$

Beweis Es gilt

$$I^{(n)}(f) = \sum_{i=0}^n \alpha_i^{(n)} f(x_i^{(n)}), \quad \alpha_i^{(n)} > 0, \quad \sum_{i=0}^n \alpha_i^{(n)} = 2.$$

Es sei $\epsilon > 0$. Nach dem Approximationssatz von Weierstraß gibt es ein $p \in P_m$ (wobei m hinreichend groß), sodass

$$\max_{-1 \leq x \leq 1} |f(x) - p(x)| \leq \frac{\epsilon}{4}.$$

Für hinreichend großes $2n + 2 > m$ gilt $I(p) - I_G^n(p) = 0$. Damit folgern wir

$$|I(f) - I^{(n)}(f)| \leq \underbrace{|I(f - p)|}_{\leq \frac{\epsilon}{4} \cdot 2} + \underbrace{|I(p) - I^{(n)}(p)|}_{=0} + \underbrace{|I^{(n)}(p - f)|}_{\leq \frac{\epsilon}{4} \cdot 2} \leq \epsilon.$$

Da $\epsilon > 0$ beliebig gewählt worden ist, muss $I^{(n)}(f) \rightarrow I(f)$ für $n \rightarrow \infty$ konvergieren. $\square$

Schließlich beweisen wir noch ein optimales Resultat für den Fehler der Gauß-Quadratur:

Satz 10.33 (Fehlerabschätzung der Gauß-Quadratur) Es sei $f \in C^{2n+2}[a, b]$. Dann lautet die Restglieddarstellung zu einer Gauß-Quadraturformel der Ordnung $2n + 2$

$$R^{(n)}(f) = \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \int_a^b p_{n+1}^2(x) dx$$

für $\xi \in [a, b]$ und mit $p_{n+1} = \prod_{j=0}^n (x - \lambda_j)$.

Beweis Der Beweis erfolgt unter Zuhilfenahme der Hermite-Interpolation (siehe Satz 9.17). Hiernach existiert ein Polynom $h \in P_{2n+1}$ zu einer zu interpolierenden Funktion $f \in C^{2n+2}[-1, 1]$, welches die Hermite'sche Interpolationsaufgabe löst, mit den Interpolationsbedingungen

$$h(x_i) = f(x_i), \quad h'(x_i) = f'(x_i), \quad i = 0, \dots, n.$$

Hierzu lautet die (bereits bekannte) Restglieddarstellung

$$f(x) - h(x) = f[x_0, x_0, \dots, x_n, x_n, x] \prod_{j=0}^n (x - x_j)^2.$$

Wendet man nun die Gaußsche Quadraturformel auf $h(x)$ an, dann gilt zunächst $I_G^{(n)}(h) = I(h)$ und weiter unter Anwendung des Mittelwertsatzes der Integralrechnung

$$\begin{aligned}
I(f) - I_G^{(n)}(f) &= I(f - h) - I_G^{(n)}(f - h) \\
&= \int_{-1}^1 f[x_0, x_0, \dots, x_n, x_n, x] \prod_{j=0}^n (x - x_j)^2 dx - \sum_{i=0}^n \alpha_i [f(x_i) - h(x_i)] \\
&= \frac{f^{(2n+2)}(\xi)}{(2n+2)!} \int_{-1}^1 \prod_{j=0}^n (x - x_j)^2 dx.
\end{aligned}$$

Der Mittelwertsatz darf hier angewendet werden, da stets $\prod_{j=0}^n (x - x_j)^2 \geq 0$. Der Term $[f(x_i) - h(x_i)] = 0$, da hier die Interpolationsbedingungen ausgenutzt worden sind. Damit ist alles gezeigt. $\square$

Bemerkung 10.34 (Zur Regularität in der Fehlerformel) Für die Gauß-Quadratur gelten die gleichen Anforderungen wie für alle anderen Integrationsmethoden: Um die volle Ordnung $2n+2$ zu erreichen, muss die zu integrierende Funktion $f(x)$ entsprechend regulär sein, also $f \in C^{2n+2}[a, b]$. Liegt diese Regularität nicht vor, so kann die Gauß-Quadratur nicht die volle Ordnung entfalten. In diesem Fall ist eine summierte Formel von niedriger Ordnung (durchaus auch eine Gauß-Formel) angebracht. $\blacklozenge$

Beispiel 10.35 (Gauß-Formeln mit Legendre-Polynomen)

Abschließend diskutieren wir ein Beispiel zur Gauß-Quadratur. Integriere

$$\int_0^5 e^{-x^2} dx \approx 0.8862269255. \quad (10.11)$$

Wir nutzen Bemerkung 10.25 zur Transformation des Integrationsgebietes sowie die Quadraturpunkte und Gewichte aus Tab. 10.2. Für $n = 0$ gilt

$$I^0(f) = \frac{5}{2} \alpha_0 f\left(\frac{5}{2}x_0 + \frac{5}{2}\right) = 5f\left(\frac{5}{2}\right) \approx 0.00966.$$

Für $n = 1$ erhalten wir

$$I^1(f) = \frac{5}{2} \sum_{k=0}^1 \alpha_k f\left(\frac{5}{2}x_k + \frac{5}{2}\right) = \frac{5}{2} \left(f\left(-\frac{1}{\sqrt{3}}\frac{5}{2} + \frac{5}{2}\right) + f\left(\frac{1}{\sqrt{3}}\frac{5}{2} + \frac{5}{2}\right) \right) \approx 0.81860.$$

Für $n = 2$ erhalten wir

$$\begin{aligned}
I^2(f) &= \frac{5}{2} \sum_{k=0}^2 \alpha_k f\left(\frac{5}{2}x_k + \frac{5}{2}\right) \\
&= \frac{5}{2} \left(\frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\frac{5}{2} + \frac{5}{2}\right) + \frac{8}{9} f\left(0 \cdot \frac{5}{2} + \frac{5}{2}\right) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\frac{5}{2} + \frac{5}{2}\right) \right) \\
&= 1.01531.
\end{aligned}$$

Ohne Details ergibt sich für $n = 3, 4, 5$

$$I^3(f) \approx 0.87803,$$

$$I^4(f) \approx 0.87941,$$

$$I^5(f) \approx 0.88774.$$



Beispiel 10.36 (Integration des Double-Well-Potenzials)

Wir integrieren das Double-Well-Potenzial, siehe Abb. 8.4, und untersuchen, bis zu welchem Polynomgrad die Gauß-Formeln exakt integrieren.

$$I(f) = \int_{-3}^3 f(x) dx, \quad f(x) = (1 - x^2)^2.$$

Eine Gauß-Formel mit $n = 1$ (Zweipunktformel) sollte Polynome bis zum Grad $2n + 1 = 3$ exakt integrieren und daher nicht das exakte Ergebnis liefern:

$$I^1(f) = \frac{6}{2} \sum_{k=0}^1 \alpha_k f\left(\frac{6}{2}x_k\right) = 3\left(f\left(-\frac{3}{\sqrt{3}}\right) + f\left(\frac{3}{\sqrt{3}}\right)\right) = 24$$

Der exakte Wert ist $336/5 = 67.2$. Das heißt, wir haben einen relativen Fehler von gut 64 %. Die Formel ist also sehr ungenau. Die Hinzunahme eines einzigen zusätzlichen Stützpunktes führt auf eine Dreipunkt-Gauß-Formel mit $n = 2$ mit der Ordnung 6 und sollte Polynome bis zum Grad 5 exakt integrieren und daher das richtige Ergebnis liefern. Aus Tab. 10.2 lesen wir die Stützstellen und Gewichte ab. Nachrechnen zeigt

$$\begin{aligned} I^2(f) &= \frac{6}{2} \sum_{k=0}^2 \alpha_k f(3x_k) \\ &= 3\left(\alpha_0 f(3x_0) + \alpha_1 f(3x_1) + \alpha_2 f(3x_2)\right) \\ &= 3\left(\frac{5}{9} f\left(-3\sqrt{\frac{3}{5}}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(3\sqrt{\frac{3}{5}}\right)\right) = 67.2, \end{aligned}$$

also das exakte Ergebnis.



10.4.2 Gauß-Tschebyscheff-Quadratur

Die Gaußschen Quadraturformeln sind so konstruiert, dass Polynome bis zu einem bestimmten Grad exakt integriert werden können. Oft sind die zu integrierenden Funktionen aber überhaupt keine Polynome, z. B.

$$f(x) = \sqrt{1 - x^2}$$

auf dem Intervall $[-1, 1]$. Die Ableitungen dieser Funktion haben an den Intervallenden Singularitäten. Die einfache Gauß-Quadratur ist zur Approximation von $\int_{-1}^1 f(x) dx$ nicht geeignet, da hohe Regularität vorausgesetzt wird.

Die Idee der Gauß-Quadratur kann verallgemeinert werden. Ziel ist die möglichst effiziente Quadratur von Funktionen, welche nach der Art

$$f(x) = \omega(x)p(x)$$

zusammengesetzt sind, wobei $p(x)$ ein Polynom ist und $\omega(x)$ eine *Gewichtsfunktion*. Mithilfe dieser Gewichtsfunktion definieren wir ein modifiziertes Skalarprodukt:

Satz 10.37 (Gewichtetes Skalarprodukt) Durch $\omega \in L^\infty([-1, 1])$ sei eine positive

$$\omega(x) > 0 \quad \text{fast überall}$$

Gewichtsfunktion gegeben. Dann ist durch

$$(f, g)_\omega := \int_{-1}^1 \omega(x) f(x) g(x) dx$$

ein Skalarprodukt gegeben.

Beweis Übung. □

Wir können nun die Konstruktion der Gauß-Legendre-Formeln mithilfe des gewichteten Skalarprodukts durchführen. Angenommen, durch $p_{n+1}^\omega(x)$ sei ein entsprechendes orthogonales Polynom gegeben, d. h.

$$(p_{n+1}^\omega, q)_\omega = (\omega p_{n+1}^\omega, q) = 0 \quad \forall q \in P_n.$$

Satz 10.29 kann unmittelbar übertragen werden, d. h., p_{n+1}^ω hat $n+1$ paarweise verschiedene Nullstellen in $(-1, 1)$. Laut Satz 10.28, der ebenfalls sofort übertragen werden kann, bilden diese Nullstellen eine Gaußsche Quadraturformel für Integrale der Art

$$\int_{-1}^1 \omega(x) f(x) dx.$$

Wir können die gleiche Argumentation wie bei den Gauß-Legendre-Formeln anwenden.

Die Gauß-Tschebyscheff-Quadratur wählt als spezielles Gewicht die Funktion

$$\omega(x) = \frac{1}{\sqrt{1-x^2}}.$$

Diese Funktion legt ein starkes Gewicht auf die beiden Intervallenden. Die entsprechenden orthogonalen Polynome sind die Tschebyscheff-Polynome, siehe Satz 9.42, die eine besondere Rolle bei der optimalen Wahl der Stützstellen für die Lagrange-Interpolation spielen. Zur Herleitung der Gauß-Tschebyscheff-Quadraturformeln müssen schließlich noch die Nullstellen der orthogonalen Polynome, also die Stützstellen, sowie die Quadraturgewichte bestimmt werden. Die Nullstellen lassen sich sofort an der Darstellung (vergleiche Satz 9.42)

$$T_n(x) := \cos(n \arccos x), \quad -1 \leq x \leq 1,$$

ablesen. Auch für die Gewichte gilt ein sehr einfacher Zusammenhang.

Satz 10.38 (Gewichte der Gauß-Tschebyscheff-Quadraturformeln) Für die Gewichte α_k der Gauß-Tschebyscheff-Quadratur mit n Stützstellen gilt

$$\alpha_k = \frac{\pi}{n}, \quad k = 0, \dots, n-1.$$

Beweis Die Funktionen

$$\frac{T_m(x)}{\sqrt{1-x^2}}, \quad m = 0, \dots, n-1,$$

werden von der Tschebyscheff-Formel mit n Stützstellen exakt integriert. Das heißt, es gilt

$$\sum_{k=0}^{n-1} \alpha_k T_m(x_k) = \int_{-1}^1 \frac{T_m(x)}{\sqrt{1-x^2}} dx, \quad m = 0, \dots, n-1.$$

Mit Satz 9.42 folgt

$$\sum_{k=0}^{n-1} \alpha_k \cos\left(\frac{(2k+1)m}{2n}\pi\right) = \int_{-1}^1 \frac{T_m(x)}{\sqrt{1-x^2}} dx, \quad m = 0, \dots, n-1,$$

mit (Nachrechnen)

$$\int_{-1}^1 \frac{T_m(x)}{\sqrt{1-x^2}} dx = \begin{cases} \pi & m = 0, \\ 0 & m = 1, \dots, n-1. \end{cases}$$

Aus diesen Gleichungen kann ein lineares Gleichungssystem in den $\alpha_0, \dots, \alpha_{n-1}$ mit den Lösungen $\alpha_k = \pi/n$ hergeleitet werden. $\square$

Damit haben wir alle Komponenten gesammelt, um die Gauß-Tschebyscheff-Formel inklusive Restglied anzugeben:

Satz 10.39 (Gauß-Tschebyscheff-Formel) Die Gauß-Tschebyscheff-Formel vom Grad $2n$ mit Restglied lautet

$$\int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx = \frac{\pi}{n} \sum_{k=0}^{n-1} f\left(\cos\left(\frac{2k+1}{2n}\pi\right)\right) + \frac{\pi}{2^{2n-1}(2n)!} f^{(2n)}(\xi)$$

mit $\xi \in [-1, 1]$.

Beweis Die Quadraturformel folgt sofort aus Einsetzen der Stützstellen und Gewichte in die allgemeine Definition einer Quadraturformel. Die Restglieddarstellung folgt aus Satz 10.33. $\square$

Die Gauß-Tschebyscheff-Quadratur hat den Vorteil, dass Stützstellen und Gewichte unmittelbar explizit angegeben werden können. Sie sind natürlich auf den Spezialfall von Funktionen zugeschnitten, die am Rande des Integrationsbereichs Singularitäten haben.

Beispiel 10.40

Wir verwenden die Gauß-Tschebyscheff-Quadratur zur Berechnung des Integrals (halber Kreisinhalt):

$$I(f) = \int_{-1}^1 \sqrt{1-x^2} dx = \int_{-1}^1 \omega(x) \underbrace{(1-x^2)}_{=:f(x)} dx$$

mit dem Gauß-Tschebyscheff-Gewicht $\omega(x)$ und $(1-x^2) \in P_2$. Da $f \in P_2$, wählen wir $n = 2$ im Verfahren 10.39 zur exakten Berechnung des Integrals, d. h.

$$\int_{-1}^1 \omega(x) f(x) dx \approx \frac{\pi}{2} \sum_{k=0}^1 f\left(\cos\left(\frac{2k+1}{2 \cdot 2}\pi\right)\right)$$

mit $\xi \in (-1, 1)$ und den Quadraturgewichten $\alpha_0 = \alpha_1 = \frac{\pi}{2}$. Wir erhalten

$$\begin{aligned} I(f) &= \int_{-1}^1 \sqrt{1-x^2} dx = \int_{-1}^1 \omega(x)(1-x^2) dx \\ &= \frac{\pi}{2} \sum_{k=0}^1 \left(1 - \left(\cos\left(\frac{2k+1}{4}\pi\right)\right)^2\right) \\ &= \frac{\pi}{2} \left(\left(1 - \cos\left(\frac{1}{4}\pi\right)\right)^2 + \left(1 - \cos\left(\frac{3}{4}\pi\right)\right)^2 \right) \\ &= \frac{\pi}{2} ((1 - 0.5) + (1 - 0.5)) = \frac{\pi}{2}, \end{aligned}$$

d. h., das Integral wird exakt berechnet.



10.5 Exkurs: Keplersche Fassregel

Die Keplersche Fassregel wurde von Johannes Kepler im 17. Jahrhundert entwickelt und basiert auf einem Spezialfall der Simpson-Regel auf Rotationskörper. Hintergrund war der, dass zur damaligen Zeit der Preis eines Weinfasses nach dessen Volumen berechnet wurde. Für die Stadt Ulm sollte Kepler im 17. Jahrhundert neue Maßeinheiten festlegen, u. a. für die Volumina von Weinfässern. Kepler stellte allerdings fest, dass es hier zu großen Abweichungen aufgrund ungenauer Berechnungen kam. Daher entwickelte er selbst eine Formel, welche auf einfachen geometrischen Überlegungen basiert. Diese Formel wurde 1615 veröffentlicht [57] und basiert auf der Simpson-Regel in Satz 10.11. Dabei wird der Verlauf der äußeren Wand durch ein quadratisches Polynom approximiert:

$$K_2 = \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right) \quad (10.12)$$

Diese Formel ist von 4. Ordnung und daher für Polynome 3. Grades exakt.

Diese approximative Berechnung gilt aber zunächst für eine Fläche. Wir wenden die Formel nun auf eine Volumenberechnung an.

1. Versuch

Die folgende Herleitung führen wir nun für ein Fass durch. Zunächst stellen wir fest, dass ein Fass ein Rotationskörper bezüglich der y -Achse ist. Die Querschnittsfläche senkrecht zur Längsachse (also der y -Achse) eines Fasses ist ein Kreis mit Radius $r = f(x)$. Der Flächeninhalt ist bekanntermaßen gegeben durch $A := A(r) = \pi r^2$. Die Höhe des Fasses sei h bzw. allgemeiner $b - a = h$, wobei $a = 0$ der Boden ist und b der Deckel. Die Keplersche Fassregel für Volumina lautet dann:

$$\begin{aligned} V_K &= \frac{b-a}{6} \left(A(a) + 4A\left(\frac{a+b}{2}\right) + A(b) \right) \\ &= \frac{h}{6} \left(A(0) + 4A\left(\frac{h}{2}\right) + A(h) \right) \end{aligned} \quad (10.13)$$

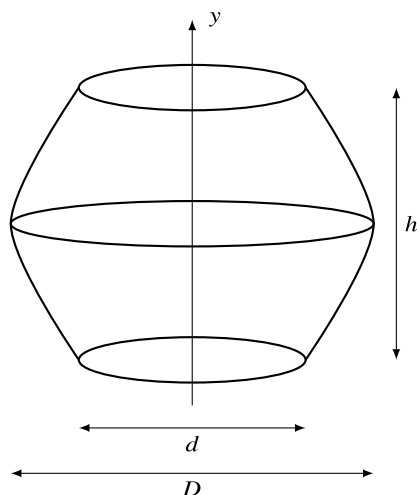
Wir stellen die Situation in Abb. 10.3 grafisch dar.

Wir müssen nun lediglich die Umfänge u von Boden und Deckel kennen sowie den Umfang U in der Mitte des Fasses, um damit das Volumen näherungsweise bestimmen zu können. Dann erhalten wir

$$V_K = \frac{h}{6} \left(\pi \left(\frac{u}{2\pi} \right)^2 + 4 \left(\frac{U}{2\pi} \right)^2 \pi + \pi \left(\frac{u}{2\pi} \right)^2 \right) = \frac{h}{12\pi} \cdot (u^2 + 2U^2).$$

Damit haben wir alle Daten, um schnell und relativ präzise das Volumen eines Fasses bestimmen zu können.

Abb. 10.3 Illustration der Keplerschen Fassregel



Ein Weinfass aus Eichenholz (frz. Barrique) für Rotwein aus Frankreich hat beispielsweise folgende Maße:

- Höhe: $h = 98$ cm
- Bauchdurchmesser: $D = 70$ cm
- Bodendurchmesser: $d = 58$ cm

Leider konnte der Hersteller des Weinfasses auf die Schnelle keine Angaben zum Volumen machen. Mithilfe der Keplerschen Fassregel rechnen wir das Volumen selbst aus. Zunächst berechnen wir die Umfänge u und U :

$$u = 2 \cdot \pi \cdot \frac{d}{2} = 182.21$$

$$U = 2 \cdot \pi \cdot \frac{D}{2} = 219.91$$

Damit erhalten wir

$$V_K = \frac{h}{12\pi} \cdot (u^2 + 2U^2) = \frac{98}{12\pi} (182.21^2 + 2 \cdot 219.91^2) = 3.3773 \cdot 10^5 \text{ cm}^3.$$

Wir wissen, dass $1 \text{ dm}^3 = 1000 \text{ cm}^3 = 1 \text{ l}$ entspricht. Damit bekommen wir als Volumen $V_K = 3381$.

2. Versuch

Das oben berechnete Volumen ist eine grobe Überschätzung (also eine obere Schranke) und entspricht nicht dem tatsächlichen Füllvolumen. Der Grund ist, dass die Weinfässer

vergleichsweise starke Wände haben (ca. 3 cm), Deckel und Boden um etwa 5 cm nach innen versetzt und diese jeweils 4 cm stark sind. Die modifizierten Daten sind somit:

- Höhe: $h = 80$ cm
- Bauchdurchmesser: $D = 64$ cm
- Bodendurchmesser: $d = 52$ cm

Für die Umfänge erhalten wir:

$$u = 2 \cdot \pi \cdot \frac{d}{2} = 163.36$$

$$U = 2 \cdot \pi \cdot \frac{D}{2} = 201.06$$

Damit erhalten wir

$$V_K = \frac{h}{12\pi} \cdot (u^2 + 2U^2) = \frac{80}{12\pi} (163.36^2 + 2 \cdot 201.06^2) = 2.28315 \cdot 10^5 \text{ cm}^3,$$

was 228 l entspricht. Diese Volumenberechnung ist bei Weitem realistischer als das Ergebnis in Versuch 1.



Gradientenverfahren und künstliche neuronale Netze

11

Eine Grundaufgabe der Analysis ist die Suche nach dem Minimum oder Maximum einer Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Für einfache Funktionen, z. B.

$$f : \mathbb{R}^3 \rightarrow \mathbb{R}, \quad f(x) = \|x - x_0\|^2 + 5$$

ist diese Aufgabe einfach und das eindeutige Minimum liegt bei $x = x_0$ und hier nimmt die Funktion den Wert $f(x_0) = 5$ an. Numerisch ist die Suche nach Minima und Maxima eine Realisierung des *Satzes vom Minimum und Maximum*, Satz 7.13. Vergleicht man den Beweis zu diesem Satz z. B. mit dem des Zwischenwertsatzes, so fällt auf, dass der Zwischenwertsatz auf einem konstruktiven Argument beruht, welches sich als Intervallschachtelung (Abschn. 8.2) direkt als numerisches Verfahren umsetzen lässt. Der Beweis zum Satz vom Minimum und Maximum baut auf der Auswahl einer Teilfolge. Es ist ein typisches Resultat der Art *es existiert ein Punkt x , so dass ...*, der Beweis liefert uns jedoch keinen klaren Weg, diesen Punkt auch zu finden. Und in der Tat ist die Aufgabe, Minima und Maxima zu finden, ungleich schwerer als die Suche nach Nullstellen. Dies ist insbesondere dann wahr, wenn wir kaum etwas über die Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ wissen. Satz 7.13 fordert die Stetigkeit von f . Falls wir nichts weiter über f wissen, so werden wir auch keine effizienten Verfahren herleiten können.

Schlüssel zum Erfolg sind Charakterisierungen von Minima und Maxima als Extremstellen, siehe die Sätze 7.14 und 7.16. Ist eine Funktion differenzierbar, so ist ein Minimum eine Extremstelle, wo der Gradient verschwindet. Auf diese Weise können wir die Suche nach Minima und Maxima mit der Nullstellensuche in Verbindung bringen.

Im folgenden werden wir nur einen sehr kleinen Einblick in grundlegende Verfahren der numerischen Optimierung geben. Wie oben beschrieben haben diese einen direkten Bezug zu verschiedenen Themen der numerischen Mathematik. Für eine breite Einführung in das Gebiet der Optimierung verweisen wir auf die Literatur [38, 39, 67, 87]. Wir beginnen in

Abschn. 11.1 mit nichtlinearer Optimierung mit Funktionen und nutzen zur numerischen Lösung das Gradientenverfahren und diskutieren danach – das bereits bekannte – Newton-Verfahren. Anschließend in Abschn. 11.2. betrachten wir die Lösung linearer Gleichungssysteme mittels Gradientenverfahren. Dieses stellt dann auch Techniken bereit, um das bereits behandelte CG-Verfahren in Abschn. 11.3 vollständig analysieren zu können. Eine erste Anwendung des Gradientenverfahrens ist die Bestimmung der unbekannten Modellparameter in der linearen Regression mit Hilfe eines Exkurses in Abschn. 11.4. Eine zweite – derzeit in vielen Bereichen hinreichende – Anwendung ist die numerische Lösung des Approximationsproblems künstlicher neuronaler Netze. Hier werden in Abschn. 11.5 Optimierungsmethoden verwendet, um die optimalen Parameter zu bestimmen.

11.1 Nichtlineare Optimierung

Gegeben sei eine stetige Funktion

$$f : X \subset \mathbb{R}^n \rightarrow \mathbb{R}$$

und gesucht ist die Stelle $x \in X$, welche die Funktion $f(\cdot)$ minimiert:

$$f(x) = \min_{y \in X} f(y),$$

sodass

$$f(x) \leq f(y) \quad \forall y \in X$$

gilt. In Definition 7.12 haben wir die Begriffe von lokalen und globalen Minima eingeführt.

In Abb. 11.1 zeigen wir eine Funktion $f \in C(\mathbb{R})$, welche verschiedene Typen von Minima besitzt:

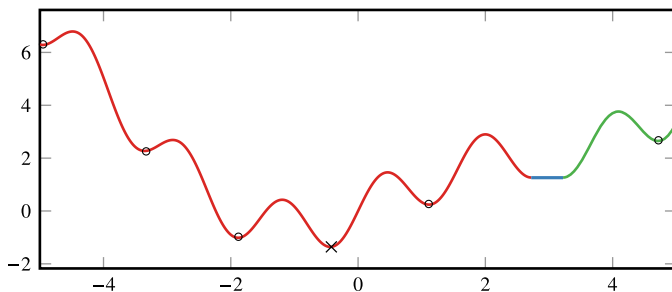


Abb. 11.1 Funktion mit verschiedenen Minima. Durch $\circ$ werden lokale isolierte Minima gekennzeichnet, durch $\times$ das globale (strikte) Minimum und der dick gekennzeichnete Bereich kennzeichnet ein nicht isoliertes lokales Minimum

$$f(x) = \begin{cases} \sin(4x) + \sin(x) + \frac{1}{4}x^2 & x < x_* \\ \sin(4x_*) + \sin(x_*) + \frac{1}{4}x_*^2 & x_* \leq x \leq x_* + \frac{1}{2} \\ \sin(4(x - \frac{1}{4})) + \sin(x - \frac{1}{4}) + \frac{1}{4}(x - \frac{1}{4})^2 & x > x_* \end{cases}$$

mit $x_* \approx 2.72086$. Das Auffinden von *globalen Minima* ist eine sehr schwierige Aufgabe, vergleichbar mit dem Finden von *allen Nullstellen* einer Funktion. Für Beispiele verweisen wir auf ein Standardwerk der Optimierungsliteratur [67].

Wir werden uns daher im Wesentlichen mit der Suche nach lokalen Minima befassen.

11.1.1 Das Gradientenverfahren

Das allgemeine Abstiegsverfahren zur Minimierung einer stetigen Funktion $f : X \subset \mathbb{R}^n \rightarrow \mathbb{R}$ beginnt mit einem Startwert $x^0 \in X$ und sucht sukzessive Iterationen $x^1, x^2, \dots$ für die jeweils $f(x^k) < f(x^{k-1})$ gilt. Dabei wählen wir in jedem Schritt k zunächst die Abstiegsrichtung $d^k \in \mathbb{R}^n$ und suchen die nächste Iteration dann nur noch in dieser Richtung, d. h., wir suchen $x^k = x^{k-1} + s_k d^k \in X$ so dass

$$f(x^{k-1} + s_k d^k) < f(x^k)$$

für ein $s_k \in \mathbb{R}$ gilt. Auf diese Art und Weise haben wir das n -dimensionale Minimierungsproblem in ein eindimensionales Problem überführt. Auch dieses ist nicht einfach zu lösen, hier werden wir aber effiziente Methoden zur Approximation kennenlernen. Wir nennen d^k die *Abstiegsrichtung* und s_k die *Suchweite*. In Richtung d^k muss eine Reduktion der Funktion f möglich sein.

► **Definition 11.1 (Abstiegsrichtung)** Es sei $f \in C(X)$ für $X \subset \mathbb{R}^d$ und $x \in X$. Ein Vektor $d \in \mathbb{R}^n$ heißt *Abstiegsrichtung*, falls es eine Zahl $\alpha > 0$ gibt, so dass

$$f(x + sd) < f(x) \quad \forall s \in (0, \alpha)$$

gilt.

Definition 11.1 bedeutet, dass die Funktion in Richtung d lokal streng monoton fallend sein muss. Ist die Funktion f zusätzlich differenzierbar, so muss gelten, dass die Ableitung in Richtung d negativ ist, d. h.

$$\nabla f(x) \cdot d < 0.$$

Dieser Zusammenhang hilft uns bei der Suche nach der optimalen Abstiegsrichtung:

Satz 11.2 (Optimale Abstiegsrichtung) Es sei $f : X \subset \mathbb{R}^n \rightarrow \mathbb{R}$ eine stetig differenzierbare Funktion. Sei $x \in X$ mit $\nabla f(x) \neq 0$. Die Richtung des steilsten Abstiegs in $x \in \mathbb{R}^n$

ist eindeutig gegeben als der negative Gradient

$$d = -\frac{\nabla f(x)}{\|\nabla f(x)\|}.$$

Beweis Es gilt für beliebige $d \in \mathbb{R}^n$ mit $\|d\| = 1$

$$(\nabla f(x), d) \geq -\|\nabla f(x)\| \|d\| = -\|\nabla f(x)\|.$$

Einzig im Fall $d = -\nabla f(x)/\|\nabla f(x)\|$ gilt Gleichheit

$$(\nabla f(x), d) = -\|\nabla f(x)\|.$$

Somit ist durch $-\nabla f(x)/\|\nabla f(x)\|$ die eindeutige normierte Richtung des steilsten Abstiegs gegeben. $\square$

Im Zuge dessen erhalten wir auf natürliche Weise das *Gradientenverfahren* als Abstiegsverfahren mit optimaler Abstiegsrichtung:

Algorithmus 11.1: Minimierung mittels Gradientenverfahren

Eingabe : Es sei $f : X \subset \mathbb{R}^n \rightarrow \mathbb{R}$ stetig differenzierbar und $x^0 \in X$ ein Startwert.

- 1 **Für** $k = 1, 2, \dots$ **tue**
- 2 **Wenn** $\nabla f(x^{k-1}) = 0$ **dann**
- 3 Abbruch
- 4 Bestimme Abstiegsrichtung $d^k = -\nabla f(x^{k-1})/\|\nabla f(x^{k-1})\|$
- 5 Wähle eine Schrittweite $s_k \in \mathbb{R}$
- 6 Berechne neue Approximation $x^k = x^{k-1} + s_k d^k$ mit $f(x^k) < f(x^{k-1})$

Ergebnis : Minimierer x^k von $f(x)$.

Das Gradientenverfahren stellt eine Konkretisierung dar, lässt die Wahl der Schrittweite jedoch nach wie vor offen.

Definition 11.1 gibt aber eine Idee für das grundsätzliche Vorgehen: Wenn d^k Abstiegsrichtung ist, was durch Satz 11.2 garantiert ist, so muss es eine Schrittweite $s_k > 0$ geben, so dass die Zielfunktion reduziert wird. Diese Schrittweite kann aber gegebenenfalls Nahe bei Null liegen. Die Verfahren funktionieren im Allgemeinen iterativ. In Iteration $k \in \mathbb{N}$ wählen wir zunächst eine initiale Schrittweite, z. B. $s_k^{(0)} = 1$, testen, ob durch $x^{k-1} + s_k^{(0)} d^k$ eine Reduktion erreicht wird. Ist dies aber nicht der Fall, so wird die Schrittweite um einen Faktor $\beta \in (0, 1)$ reduziert $s_k^{(1)} = \beta \cdot s_k^{(0)}$ und der Test wird wiederholt.

Die einfachste Methode wird *line search*, also *Liniensuche* genannt. Diese Methode haben wir bereits in Abschn. 8.4.2 als Globalisierungsmethode des Newton-Verfahrens kennengelernt.

Algorithmus 11.2: Line-Search

Eingabe : Sei $\beta \in (0, 1)$ gegeben. Mit $x^k \in \mathbb{R}^n$ bezeichnen wir den aktuellen Schritt, mit $d^k \in \mathbb{R}^n$ die gewählte Suchrichtung.

```

1  $s_k^{(0)} = 1$ 
2 Für  $l = 0, 1, 2, \dots$  tue
3   Wenn  $f(x^k + s_k^{(l)} d^k) < f(x^k)$  dann
4     Abbruch
5    $s_k^{(l+1)} = \beta \cdot s_k^{(l)}$ 
Ergebnis : Schrittweite  $s_k = s_k^{(l+1)}$ 

```

Der einfache Line-Search-Algorithmus stellt sicher, dass ein Abstieg erreicht wird, kann darüber hinaus aber keinerlei Eigenschaften der Abstiegsrichtung garantieren. Eine Modifikation ist die *Armijo-Schrittweitenregel*. Diese nutzt aus, dass sich die Funktion f in Suchrichtung lokal als lineare Funktion mit der Steigung $-\nabla f(x^k)$ darstellen lässt. Die Line-Search-Suche wird erst dann abgebrochen, wenn die gefundene Richtung diesen linearen Abstieg erreicht.

Algorithmus 11.3: Armijo-Schrittweitenregel

Eingabe : Sei $\beta, \gamma \in (0, 1)$ gegeben. Mit $x^k \in \mathbb{R}^n$ bezeichnen wir den aktuellen Schritt, mit $d^k \in \mathbb{R}^n$ die gewählte Suchrichtung.

```

1  $s_k^{(0)} = 1$ 
2 Für  $l = 0, 1, 2, \dots$  tue
3   Wenn  $f(x^k + s_k^{(l)} d^k) < f(x^k) + \gamma s_k^{(l)} \nabla f(x^k) \cdot d^k$  dann
4     Abbruch
5    $s_k^{(l+1)} = \beta \cdot s_k^{(l)}$ 
Ergebnis : Schrittweite  $s_k = s_k^{(l+1)}$ 

```

Für beide Algorithmen kann gezeigt werden, dass sie in endlich vielen Schritten terminieren und so wirklich eine geeignete Schrittweite gefunden wird. Wir zeigen dies für die Armijo-Regel. Aber jede Schrittweite, welche die Armijo-Bedingung erfüllt, genügt natürlich auch der einfachen Line-Search Bedingung.

Satz 11.3 (Armijo-Schrittweitenregel) Es sei $f : X \rightarrow \mathbb{R}$ auf der offenen Menge $X \subset \mathbb{R}^n$ stetig differenzierbar. Es sei $\gamma \in (0, 1)$ und $d \in \mathbb{R}^n$ eine beliebige Abstiegsrichtung zu einem Punkt $x \in X$ mit

$$\nabla f(x) \cdot d < 0.$$

Dann existiert ein $k \in \mathbb{N}$, sodass

$$s_k = \beta^k, \quad f(x + s_k d) - f(x) \leq \gamma s_k (\nabla f(x), d).$$

Beweis Es gilt

$$\frac{f(x + sd) - f(x)}{s} - \gamma \nabla f(x) \cdot d \rightarrow (1 - \gamma) \nabla f(x) \cdot d < 0 \quad (s \rightarrow 0).$$

Für hinreichend kleines s folgt wegen der Stetigkeit von ∇f , dass es ein ϵ gibt, sodass für alle $s < \epsilon$ gilt:

$$\frac{f(x + sd) - f(x)}{s} - \gamma \nabla f(x) \cdot d \leq 0 \quad \forall |s| < \epsilon$$

Für $\beta \in (0, 1)$ findet sich ein minimales $k \in \mathbb{N}$, sodass $s_k := \beta^k < \epsilon$ gilt. $\square$

Das Gradientenverfahren konvergiert oft nur sehr langsam. In Abb. 11.2 zeigen wir den Konvergenzverlauf am Beispiel der sogenannten *Rosenbrock-Funktion*, ein typischer Testfall für Optimierungsverfahren:

$$\min_{x, y \in \mathbb{R}} \{f(x, y) = 10(y - x^2)^2 + (1 - x)^2\}$$

Wir wählen die Startwerte $x^0 = (-1, 1.5)$ und $x^0 = (2, -2)$. Für den ersten Fall fassen wir den Konvergenzverlauf in Tab. 11.1 zusammen.

Es gibt viele Methoden zur Effizienzsteigerung des Gradientenverfahrens, wie verweisen auf die Literatur [67, 87].

11.1.2 Das Newton-Verfahren zur Minimierung

Wenn die zugrundeliegende Funktion $f(x)$ zweimal stetig differenzierbar ist, dann kann das Newton-Verfahren (Abschn. 8.3) zur Lösung des Minimierungsproblems herangezogen

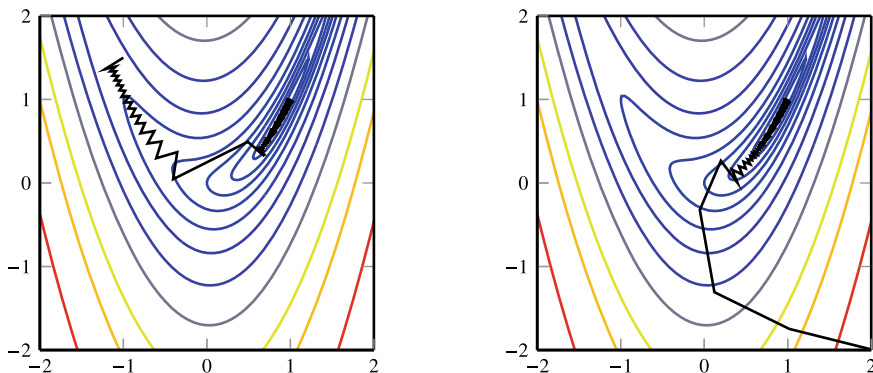


Abb. 11.2 Konvergenzverlauf des Gradientenverfahrens zur Approximation des Minimums der sogenannten *Rosenbrock-Funktion* $f(x, y) = 10(y - x^2)^2 + (1 - x)^2$ bei verschiedenen Startwerten

Tab. 11.1 Resultate zur Approximation der Rosenbrock-Funktion

k	x^k	y^k	$f(x^k)$	$\ \nabla f(x^k)\ $
0	-1	1.5	6.5	18.8680
1	-1.212	1.3675	9.5597	4.9958
2	-1.0898	1.3940	6.3382	4.7929
3	-1.1373	1.3533	1.9585	4.6039
4	-1.0384	1.2768	5.7600	4.5494
5	-1.0836	1.2337	1.9861	4.3770
...				
100	1.0050	1.0100	0.012493	$2.55 \cdot 10^{-05}$
101	1.0048	1.0101	0.012492	$2.52 \cdot 10^{-05}$
102	1.0049	1.0099	0.012491	$2.48 \cdot 10^{-05}$
103	1.0047	1.0099	0.012491	$2.44 \cdot 10^{-05}$
104	1.0049	1.0097	0.012490	$2.41 \cdot 10^{-05}$
105	1.0046	1.0098	0.012489	$2.37 \cdot 10^{-05}$

werden. Wie vorher diskutiert, ist die notwendige Bedingung gegeben durch

$$F(x) := \nabla f(x) \stackrel{!}{=} 0.$$

Die allgemeine Iteration, ausgehend von einem Startwert $x^0 \in X$, lautet für $k = 1, 2, 3, \dots$

$$DF(x^k)(x^{k+1} - x^k) = -F(x^k).$$

Dabei gilt

$$DF(x^k) = Hf(x^k),$$

wobei wir mit $Hf(x^k)$ die Hesse-Matrix der Funktion f bezeichnen:

$$Hf(x) := \begin{pmatrix} \frac{\partial}{\partial x_1} F_1 & \cdots & \frac{\partial}{\partial x_n} F_1 \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1} F_n & \cdots & \frac{\partial}{\partial x_n} F_n \end{pmatrix}$$

Damit das Newton-Verfahren zur Optimierung durchführbar ist, muss die Zielfunktion $f(x)$ mindestens zweimal stetig differenzierbar sein. Damit der Satz von Newton-Kantorovich, Satz 8.29, anwendbar ist, muss die zweite Ableitung von f sogar Lipschitz-stetig sein. Wir zeigen in Abb. 11.3 den Konvergenzverlauf des Newton-Verfahrens für die Rosenbrock-Funktion und fassen den Verlauf in Tab. 11.2 zusammen.

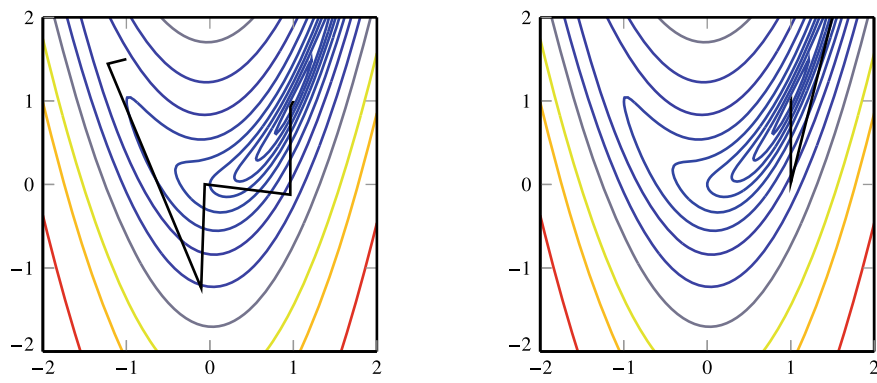


Abb. 11.3 Konvergenzverlauf des Newton-Verfahrens zur Approximation des Minimums der Rosenbrock-Funktion $f(x, y) = 10(y - x^2)^2 + (1 - x)^2$

Tab. 11.2 Konvergenzverlauf des Newton-Verfahrens für die Rosenbrock-Funktion

k	x^k	y^k	$\ \nabla f(x^k)\ $	$f(x^k)$
0	-1	1.5	18.86	6.5
1	-1.222	1.444	6.92	4.963
2	-0.104	-1.239	26.08	16.84
3	-0.063	0.002	2.128	1.127
4	0.963	-0.123	45.51	11.03
5	0.965	0.931	7.05e-02	0.00124
6	0.999	0.999	5.56e-02	1.55e-05
7	1.000	1.000	9.67e-08	2.34e-15
0	2	-2	496.71	361
1	1.992	3.966	1.988	0.983
2	1.001	0.021	43.91	9.620
3	1.001	1.002	2.57e-03	1.65e-06
4	1.000	1.000	7.41e-05	2.74e-11

In beiden Fällen können die Minima in nur sehr wenigen Iterationen sehr gut approximiert werden. An den Werten in der Tabelle (letzte Spalte) zeigt sich, dass das Newton-Verfahren kein *Abstiegsverfahren* ist. Die Iterierten $f(x^k)$ konvergieren nicht monoton gegen das Minimum.

Insbesondere in der Optimierung ist die große Schwäche des Newton-Verfahrens der oft sehr kleine Einzugsbereich der Konvergenz. Hier kommt den *Globalisierungsmethoden* wieder eine besondere Rolle zu. Bei der Anwendung des Newton-Verfahrens zur Minimierung existieren hier sehr effiziente Methoden, die auf einer Kombination des Gradientenverfahrens mit dem Newton-Verfahren basieren: Da wir wissen, dass das Gradientenverfahren

global konvergiert, kann dieses eingesetzt werden, wenn wir noch weit vom Einzugsbereich quadratischer Konvergenz entfernt sind. In der Nähe des Minimums erfolgt dann ein Wechsel zum Newton-Verfahren.

11.2 Abstiegs- und Gradientenverfahren für lineare Gleichungssysteme

Abstiegs- und Gradientenverfahren können auch zum Lösen linearer Gleichungssysteme verwendet werden. So kommen wir auf einem ganz anderem Weg schließlich zum CG-Verfahren, welches wir in Kap. 6 als Krylow-Teilraumverfahren eingeführt haben. Ausgangspunkt ist zunächst der folgende Satz, der die Lösung eines linearen Gleichungssystems als Minimierungsproblem darstellt.

Satz 11.4 (Lineares Gleichungssystem und Minimierung) Es sei $A \in \mathbb{R}^{n \times n}$ eine symmetrisch positiv definite Matrix, $x, b \in \mathbb{R}^n$. Die folgenden Bedingungen sind äquivalent:

- (i) $Ax = b$,
- (ii) $Q(x) \leq Q(y) \quad \forall y \in \mathbb{R}^n, \quad Q(y) = \frac{1}{2}(Ay, y)_2 - (b, y)_2$.

Beweis (i) $\Rightarrow$ (ii) x sei Lösung des linearen Gleichungssystems $Ax = b$. Dann gilt mit beliebigem $y \in \mathbb{R}^n$:

$$\begin{aligned} 2Q(y) - 2Q(x) &= (Ay, y)_2 - 2(b, y)_2 - (Ax, x)_2 + 2(b, x) \\ &= (Ay, y)_2 - 2(Ax, y)_2 + (Ax, x)_2 \\ &= (A(y - x), y - x)_2 \geq 0 \end{aligned}$$

wobei wir von der vorletzten zur letzten Zeile die Symmetrie von A nutzen mit $(Ax, y)_2 = (Ay, x)_2$ und damit den Term $-2(Ax, y)_2$ aufspalten können. Letztlich haben wir somit $Q(y) \geq Q(x)$ gezeigt.

(ii) $\Rightarrow$ (i) Umgekehrt sei $Q(x)$ nun Minimum. Das heißt, $x \in \mathbb{R}^n$ ist stationärer Punkt der quadratischen Form $Q(x)$, also

$$0 \stackrel{!}{=} \frac{\partial}{\partial x_i} Q(x) = \frac{\partial}{\partial x_i} \left\{ \frac{1}{2}(Ax, x)_2 - (b, x)_2 \right\} = (Ax)_i - b_i, \quad i = 1, \dots, n. \quad (11.1)$$

Das heißt, x ist Lösung des linearen Gleichungssystems. □

Anstelle der Bestimmung einer Lösung eines linearen Gleichungssystems $Ax = b$, suchen wir das Minimum des sogenannten *Energiefunctionals* $Q(x)$. Mit dieser äquivalenten Formulierung sind wir in der Welt der Minimierungsprobleme angelangt. Anstelle das System $Ax = b$ zu lösen, minimieren wir die Funktion $Q : \mathbb{R}^n \rightarrow \mathbb{R}$. Zunächst erscheint dies nicht

effizient, da die Lösung eines Minimierungsproblem im Allgemeinen weit aufwändiger ist als die Lösung oder Approximation eines linearen Gleichungssystems. Das Zielfunktional $Q(x)$ ist allerdings keine ganz allgemeine Funktion sondern hat besondere Struktureigenschaften.

Satz 11.5 *Sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit. Das Funktional*

$$Q(y) = \frac{1}{2}(Ay, y) - (b, y)$$

ist quadratisch und streng konvex.

Beweis Wir haben bereits den Gradienten von $Q(\cdot)$ bestimmt, siehe 11.1. Es gilt

$$\nabla Q(x) = \begin{pmatrix} (Ax)_1 - b_1 \\ (Ax)_2 - b_2 \\ \vdots \\ (Ax)_n - b_n \end{pmatrix} = Ax - b \in \mathbb{R}^n. \quad (11.2)$$

Für die zweiten Ableitungen, also die Hessematrix von $Q(\cdot)$ gilt entsprechend

$$H_Q(x) = A.$$

Da A symmetrisch positiv definit ist folgt sofort, dass $Q(\cdot)$ streng konvex ist. $\square$

Da A symmetrisch positiv definit ist, ist durch $\|x\|_A := \sqrt{(Ax, x)_2}$ eine Norm, die sogenannte *Energienorm*, gegeben. Die Minimierung des Energiefunktional $Q(\cdot)$ ist auch äquivalent zur Minimierung des Fehlers $x^k - x$ in der zugehörigen Energienorm. Denn angenommen $x \in \mathbb{R}^n$ sei die Lösung des Gleichungssystems und $y \in \mathbb{R}^n$ eine beliebige Approximation an diese Lösung. Dann gilt

$$\|y - x\|_A^2 = (A(y - x), y - x)_2 = (Ay, y) - \underbrace{2(Ay, x)}_{=2(b, y)} + (Ax, x) = 2Q(y) + (Ax, x).$$

Die Minimierung von $Q(y)$ minimiert auch den Fehler in der Energienorm.

Wir können nun die Idee Abstiegsverfahren und des Gradientenverfahrens auf die Minimierung des Zielfunktional $Q(x)$ anwenden und eine Folge von Approximationen $x^k \in \mathbb{R}^n$ bestimmen. Dabei wird in jedem Schritt des Verfahrens zunächst die Abstiegsrichtung d^k gewählt, im Anschluss wird die Approximation in dieser Richtung verbessert: $x^{k+1} = x^k + \omega^k d^k$. Die Schrittweite $\omega^k \in \mathbb{R}$ wird dabei so gewählt, dass der resultierende Wert des Energiefunktional minimal wird. Wir fassen zusammen:

Algorithmus 11.4: Abstiegsverfahren für lineare Gleichungssysteme

Eingabe : Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit, $x^0, b \in \mathbb{R}^n$.

Für $k = 0, 1, 2, \dots$ **tue**

1 Wähle Abstiegsrichtung $d^k \in \mathbb{R}^n$

2 Bestimme ω^k als Minimum von $\omega^k = \arg \min_{\omega \in \mathbb{R}} Q(x^k + \omega d^k)$

3 Update $x^{k+1} = x^k + \omega^k d^k$

Ergebnis : Approximative Lösung x^{k+1} von $Ax = b$.

Wir gehen davon aus, dass die Suchrichtungen gegeben seien und suchen jeweils die Lösung des skalaren Minimierungsproblem. Bei der allgemeinen nichtlinearen Minimierungsaufgabe, die wir in Abschn. 11.1 besprochen haben, konnten wir dieses Problem

$$\omega^k \in \mathbb{R} : Q(x^k + \omega^k d^k) \leq Q(x^k + s d^k) \quad \forall s \in \mathbb{R} \quad (11.3)$$

nicht exakt lösen. Stattdessen haben wir Methoden zur Schrittweitenbestimmung, z. B. die Armijo-Methode diskutiert. Im Fall des linearen Gleichungssystems ist die Situation einfacher und wir können die Lösung der quadratischen skalaren Minimierungsaufgabe als stationären Punkt bestimmen

$$0 \stackrel{!}{=} \frac{\partial}{\partial \omega^k} Q(x^k + \omega^k d^k) = \omega^k (A d^k, d^k) + (A x^k, d^k) - (b, d^k),$$

also ergibt sich

$$\omega^k = \frac{(b - A x^k, d^k)}{(A d^k, d^k)}. \quad (11.4)$$

Bei der Wahl der Suchrichtung könnten wir uns an den einfachen Fixpunktiterationen aus Abschn. 3.7 orientieren und z. B. d^k als auf Basis von Jacobi- oder Gauß-Seidel-Verfahren bestimmen, d. h.

$$d_J^k = -D^{-1}(b - A x^k), \quad d_{GS}^k = -(D + L)^{-1}(b - A x^k).$$

Im Anschluss wird dann die optimale Schrittweite mit Hilfe von (11.4) bestimmt. Es zeigt sich jedoch, dass diese Methoden kaum Vorteile gegenüber Jacobi- und Gauß-Seidel-Verfahren selbst besitzen und wir werden gleich eine effizientere Methoden kennenlernen.

Das *Gradientenverfahren zum Lösen linearer Gleichungssysteme* entspricht dem allgemeinen Gradientenverfahren aus Abschn. 11.1 und wir wählen als Suchrichtung in Schritt k den negativen Gradienten, d. h.

$$d^k = -\nabla Q(x^k) = b - A x^k,$$

siehe (11.2). Denn Satz 11.2 gilt entsprechend und $b - A x^k$ ist gerade die Richtung des *stärksten Abfalls*, in der sich der Wert von $Q(x)$ also am schnellsten reduziert. Zu einem Punkt $x \in \mathbb{R}^n$ ist diese Richtung gerade die Richtung $d \in \mathbb{R}^n$, die normal (sprich senkrecht)

auf der Niveaumenge $N(x)$ steht:

$$N(x) := \{y \in \mathbb{R}^n : Q(y) = Q(x)\}$$

In einem Punkt x ist die Niveaumenge aufgespannt durch alle Richtungen $\delta x \in \mathbb{R}^n$ mit

$$0 \stackrel{!}{=} Q'(x) \cdot \delta x = (\nabla Q(x), \delta x) = (b - Ax, \delta x).$$

Die Vektoren δx , welche die Niveaumenge aufspannen, stehen orthogonal auf dem Defekt $b - Ax$, dieser zeigt daher in Richtung der stärksten Änderung von $Q(\cdot)$. Wir wählen $d^k := b - Ax^k$. Die so gefundene Suchrichtung wird dann mit dem Abstiegsverfahren kombiniert, d. h., wir iterieren

$$d^k := b - Ax^k, \quad \omega^k := \frac{\|d^k\|_2^2}{(Ad^k, d^k)_2}, \quad x^{k+1} := x^k + \omega^k d^k.$$

Wir definieren das *Gradientenverfahren*:

Algorithmus 11.5: Gradientenverfahren

Eingabe : Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit, $b \in \mathbb{R}^n$. Es sei $x^0 \in \mathbb{R}^n$ beliebig und $d^0 := b - Ax^0$.

1 **Für** $k = 0, 1, 2, \dots$ **tue**

2 $r^k := Ad^k$

3 $\omega^k = \frac{\|d^k\|_2^2}{(r^k, d^k)_2}$

4 $x^{k+1} = x^k + \omega^k d^k$

5 $d^{k+1} = d^k - \omega^k r^k$

Ergebnis : Approximative Lösung x^{k+1} von $Ax = b$.

Durch Einführen eines zweiten Hilfsvektors $r^k \in \mathbb{R}^n$ kann in jeder Iteration ein Matrix-Vektor-Produkt gespart werden. Für Matrizen mit Diagonalanteil $D = \alpha I$ ist das Gradientenverfahren gerade das Jacobi-Verfahren in Verbindung mit dem Abstiegsverfahren. Daher kann für dieses Verfahren im Allgemeinen auch keine verbesserte Konvergenzaussage erreicht werden. Es gilt:

Satz 11.6 (Gradientenverfahren) Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit. Dann konvergiert das Gradientenverfahren für jeden Startvektor $x^0 \in \mathbb{R}^n$ gegen die Lösung des Gleichungssystems $Ax = b$.

Beweis Es sei $x^k \in \mathbb{R}^n$ eine gegebene Approximation. Weiter sei $d := b - Ax^k$. Dann berechnet sich ein Schritt des Gradientenverfahrens als

$$x^{k+1} = x^k + \frac{(d, d)}{(Ad, d)} d.$$

Für das Energiefunktional gilt:

$$\begin{aligned}
 Q(x^{k+1}) &= \frac{1}{2}(Ax^{k+1}, x^{k+1}) - (b, x^{k+1}) \\
 &= \frac{1}{2}(Ax^k, x^k) + \frac{1}{2} \frac{(d, d)^2}{(Ad, d)^2} (Ad, d) + \frac{(d, d)}{(Ad, d)} (Ax^k, d) \\
 &\quad - (b, x^k) - \frac{(d, d)}{(Ad, d)} (b, d) \\
 &= Q(x^k) + \frac{(d, d)}{(Ad, d)} \left\{ \frac{1}{2}(d, d) + (Ax^k, d) - (b, d) \right\} \\
 &= Q(x^k) + \frac{(d, d)}{(Ad, d)} \left\{ \frac{1}{2}(d, d) + \underbrace{(Ax^k - b, d)}_{=-d} \right\}
 \end{aligned}$$

Also folgt

$$Q(x^{k+1}) = Q(x^k) - \frac{(d, d)^2}{2(Ad, d)}.$$

Wegen der positiven Definitheit von A gilt

$$\lambda_{\min}(A)(d, d) \leq (Ad, d) \leq \lambda_{\max}(A)(d, d)$$

und schließlich ist mit

$$Q(x^{k+1}) \leq Q(x^k) - \underbrace{\frac{(d, d)}{2\lambda_{\max}}}_{>0}$$

die Folge $Q(x^k)$ monoton fallend. Solange $d = b - Ax \neq 0$, fällt die Folge streng monoton. Weiter ist $Q(x^k)$ nach unten durch $Q(x)$ beschränkt. Also konvergiert die Folge $Q(x^k) \rightarrow c \in \mathbb{R}$. Im Grenzwert muss $0 = (d, d) = \|b - Ax\|^2$ gelten, sprich $Ax = b$. $\square$

Schließlich zeigen wir noch eine Abschätzung der Konvergenzgeschwindigkeit des Gradientenverfahrens:

Satz 11.7 (Konvergenz des Gradientenverfahrens (vereinfacht)) Es sei $A \in \mathbb{R}^{n \times n}$ eine symmetrisch positiv definite Matrix. Dann gilt für das Gradientenverfahren zur Lösung von $Ax = b$ die Fehlerabschätzung

$$\|x^k - x\|_A \leq \left(1 - \frac{1}{\kappa}\right)^k, \quad \kappa := \text{cond}_2(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}.$$

Beweis Die Matrix $A \in \mathbb{R}^{n \times n}$ ist symmetrisch positiv definit. Es gibt also ein System aus n Eigenwerten

$$0 < \lambda_{\min} =: \lambda_1 \leq \dots \leq \lambda_n =: \lambda_{\max}$$

und orthonormalen Eigenvektoren $w_1, \dots, w_n \in \mathbb{R}^n$. Es sei

$$e^k = x^k - x = \sum_{i=1}^n e_i^k w_i \quad (11.5)$$

eine Entwicklung des Fehlers in den Eigenvektoren. Für einen Iterationsschritt des Gradientenverfahrens gilt mit

$$d^k = b - Ax^k = -Ae^k$$

die Fehlerfortpflanzung

$$x^{k+1} = x^k + \frac{(d^k, d^k)}{(Ad^k, d^k)} d^k \Rightarrow e^{k+1} = e^k - \frac{(Ae^k, Ae^k)}{(A^2 e^k, Ae^k)} Ae^k. \quad (11.6)$$

Mit der Darstellung (11.5) und mit $Aw_i = \lambda_i w_i$ folgt

$$e^{k+1} = \sum_{i=1}^N \left(1 - \frac{(Ae^k, Ae^k)}{(A^2 e^k, Ae^k)} \lambda_i \right) e_i^k w_i = \sum_{i=1}^N (1 - \mu_i) e_i^k w_i \quad (11.7)$$

mit

$$\mu_i = \lambda_i \frac{(Ae^k, Ae^k)}{(A^2 e^k, Ae^k)}.$$

Wegen der Orthonormalität der Eigenvektoren und wegen $\lambda_i > 0$ folgt

$$\begin{aligned} \mu_i &= \lambda_i \frac{\left(\sum_{j=1}^n \lambda_j e_j^k w_j, \sum_{j=1}^n \lambda_j e_j^k w_j \right)}{\left(\sum_{j=1}^n \lambda_j^2 e_j^k w_j, \sum_{j=1}^n \lambda_j e_j^k w_j \right)} \\ &= \lambda_i \frac{\sum_{j=1}^n \lambda_j^2 (e_j^k)^2}{\sum_{j=1}^n \lambda_j^3 (e_j^k)^2} \\ &\geq \lambda_{\min} \frac{\sum_{j=1}^n \lambda_j^2 (e_j^k)^2}{\lambda_{\max} \sum_{j=1}^n \lambda_j^2 (e_j^k)^2} = \frac{\lambda_{\max}}{\lambda_{\min}}. \end{aligned} \quad (11.8)$$

Wir machen nun mit (11.7) weiter und erhalten in der A -Norm

$$\begin{aligned} \|e^{k+1}\|_A^2 &= (Ae^{k+1}, e^{k+1}) = \left(A \sum_{i=1}^N (1 - \mu_i) e_i^k w_i, \sum_{i=1}^N (1 - \mu_i) e_i^k w_i \right) \\ &= \sum_{i=1}^N (1 - \mu_i)^2 \lambda_i (e_i^k)^2. \end{aligned}$$

Mit (11.8) und dem Zusammenhang

$$\|e^k\|_A^2 = (Ae^k, e^k) = \sum_{i=1}^N \lambda_i (e_i^k)^2$$

folgt weiter

$$\|e^{k+1}\|_A^2 \leq \left(1 - \frac{1}{\kappa}\right)^2 \|e^k\|_A^2, \quad \kappa := \text{cond}_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

Wiederholte Abschätzung ergibt schließlich

$$\|e^k\|_A \leq \left(1 - \frac{1}{\kappa}\right)^k \|x^0 - x\|_A. \quad \square$$

Diese Fehlerabschätzung ist nicht optimal. Mit größerem Aufwand lässt sich für das Gradientenverfahren unter denselben Voraussetzungen die bessere Fehlerabschätzung

$$\|x^k - x\|_A \leq \left(\frac{1 - 1/\kappa}{1 + 1/\kappa}\right)^k, \quad \kappa := \text{cond}_2(A),$$

herleiten. Es gilt asymptotisch

$$\frac{1 - \frac{1}{\kappa}}{1 + \frac{1}{\kappa}} = 1 - \frac{2}{\kappa} + O\left(\frac{1}{\kappa^2}\right),$$

d. h., es liegt doppelt so schnelle Konvergenz vor. Schlüssel für diesen optimalen Beweis ist der Satz von Kantorovich, eine Abschätzung für die Eigenwerte einer positiv definiten Matrix. Für den Beweis verweisen wir auf die Literatur [38].

Die asymptotische Konvergenzrate des Gradientenverfahrens wird durch die Kondition der Matrix bestimmt. Für die Modellmatrix gilt $\kappa = O(n^2)$, siehe Beispiel 3.49. Also gilt

$$\rho = \frac{1 - \frac{1}{n^2}}{1 + \frac{1}{n^2}} = 1 - \frac{2}{n^2} + O\left(\frac{1}{n^4}\right).$$

Die Konvergenz ist demnach ebenso langsam wie die des Jacobi-Verfahrens. Für die Suchrichtungen des Gradientenverfahren gilt der folgende Zusammenhang:

Satz 11.8 (Abstiegsrichtungen im Gradientenverfahren) Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit. Dann stehen je zwei aufeinanderfolgende Abstiegsrichtungen d^k und d^{k+1} des Gradientenverfahrens orthogonal aufeinander, also $(d^k, d^{k+1}) = 0$.

Beweis Zum Beweis siehe Algorithmus 11.5. Es gilt

$$d^{k+1} = d^k - \omega^k r^k = d^k - \frac{(d^k, d^k)}{(Ad^k, d^k)} Ad^k.$$

Also gilt

$$(d^{k+1}, d^k) = (d^k, d^k) - \frac{(d^k, d^k)}{(Ad^k, d^k)}(Ad^k, d^k) = (d^k, d^k) - (d^k, d^k) = 0. \quad \square$$

In Abb. 11.4 zeigen wir die ersten 10 Schritte des Gradientenverfahrens zur Approximation der Lösung des linearen Gleichungssystems

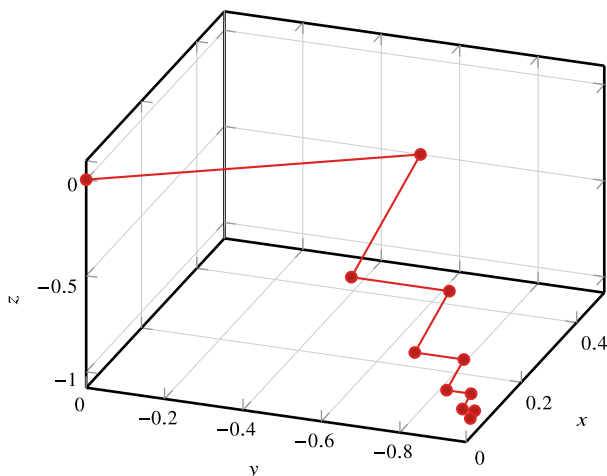
$$\begin{pmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}$$

mit der exakten Lösung $x = (0, -1, -1)$. Hier zeigt sich, dass zwei aufeinanderfolgende Richtungen stets parallel liegen. Über viele Iterationen hinweg verläuft Näherung allerdings in einer zick-zack Linie und die Konvergenz ist sehr langsam.

Der Zusammenhang aus Satz 11.8 gilt nur für jeweils aufeinanderfolgende Abstiegsrichtungen, im Allgemeinen gilt jedoch $d^k \not\perp d^{k+2}$, dies zeigt auch der Konvergenzverlauf in Abb. 11.4. Zwei direkt aufeinanderfolgende Richtungen sind zwar je orthogonal, die dritte Richtung steht jedoch wieder nahezu parallel auf der ersten. Dies führt dazu, dass das Gradientenverfahren im Allgemeinen sehr langsam konvergiert.

Das in Abschn. 6.2 beschriebene CG-Verfahren kann auch als eine Weiterentwicklung des Gradientenverfahrens hergeleitet werden. Anstelle der skalaren Minimierungsprobleme (11.3), die in jedem Schritt des Gradientenverfahrens gelöst werden, betrachten wir beim CG-Verfahren ein Minimierungsproblem von steigender Komplexität. Im k -ten Schritt wird die Approximation $x^k = x^0 + \sum_{i=0}^{k-1} \alpha_i d^i$ als das Minimum über alle $\alpha = (\alpha_0, \dots, \alpha_{k-1})$ bezüglich $Q(x^k)$ gesucht:

Abb. 11.4 Konvergenzverlauf des Gradientenverfahrens zur Approximation eines linearen Gleichungssystems $Ax = b$ mit $A \in \mathbb{R}^{n \times n}$. Aufeinanderfolgende Suchrichtungen sind jeweils orthogonal, d. h. $d^k \perp d^{k+1}$, für d^k und d^{k+2} gilt jedoch, dass diese Richtungen fast parallel laufen können



$$\begin{aligned} \min_{\alpha \in \mathbb{R}^k} \mathcal{Q} \left(x^0 + \sum_{i=0}^{k-1} \alpha_i d^i \right) \\ = \min_{\alpha \in \mathbb{R}^k} \left\{ \frac{1}{2} \left(Ax^0 + \sum_{i=0}^{k-1} \alpha_i Ad^i, x^0 + \sum_{i=0}^{k-1} \alpha_i d^i \right) - \left(b, x^0 + \sum_{i=0}^{k-1} \alpha_i d^i \right) \right\} \end{aligned}$$

Der stationäre Punkt ist bestimmt durch

$$\begin{aligned} 0 &\stackrel{!}{=} \frac{\partial}{\partial \alpha_j} \mathcal{Q}(x^k) = \left(Ax^0 + \sum_{i=0}^{k-1} \alpha_i Ad^i, d^j \right) - (b, d^j) \\ &= - \left(b - Ax^k, d^j \right), \quad j = 0, \dots, k-1. \end{aligned}$$

Das heißt, das neue Residuum $b - Ax^k$ steht orthogonal auf allen Suchrichtungen d^j für $j = 0, \dots, k-1$. Das resultierende System aus k Gleichungen für die k Unbekannten $\alpha_0, \dots, \alpha_{k-1}$

$$(b - Ax^k, d^j) = 0 \quad \forall j = 0, \dots, k-1, \quad (11.9)$$

wobei $x^k = x^0 + \sum_{i=0}^{k-1} \alpha_i d^i$ gilt, ist genau die *Galerkin-Gleichung*, die wir als (6.3) in Abschn. 6.2 kennengelernt haben. Dort war die Gleichung der Startpunkt unserer Herleitung des CG-Verfahrens als ein Krylow-Teilraumverfahren.

Betrachten wir das CG-Verfahren jedoch als eine Weiterentwicklung des Gradientenverfahrens, so werden hier Suchrichtungen zur Minimierung verwendet, die paarweise orthogonal sind. Dies erlaubt eine sehr effiziente Lösung der Galerkingleichung (11.9).

11.3 Konvergenzbeweis des CG-Verfahrens

Die Konvergenzanalyse des CG-Verfahrens aus Abschn. 6.2 erweist sich als sehr aufwendig und benötigt Techniken des Gradientenverfahrens. Daher kann diese Analyse nun erst durchgeführt werden. Schlüssel ist die folgende Charakterisierung einer Iteration $x^k = x^0 + K_k$ als

$$x^k = x^0 + p_{k-1}(A)d^0,$$

wobei $p_{k-1} \in P_{k-1}$ ein Polynom in A ist:

$$p_{k-1}(A) = \sum_{i=0}^{k-1} \alpha_i A^i$$

Hiermit kann die Minimierungseigenschaft aus Satz 6.5 geschrieben werden als

$$\|b - Ax^k\|_{A^{-1}} = \min_{y \in x^0 + K_k} \|b - Ay\|_{A^{-1}} = \min_{q \in P_{k-1}} \|b - Ax^0 - Aq(A)d^0\|_{A^{-1}}.$$

Wenn wir zur $\|\cdot\|_A$ -Norm übergehen, folgt mit $d^0 = b - Ax^0 = A(x - x^0)$

$$\|b - Ax^k\|_{A^{-1}} = \|x - x^k\|_A = \min_{q \in P_{k-1}} \|(x - x^0) - q(A)A(x - x^0)\|_A,$$

also

$$\|x - x^k\|_A = \min_{q \in P_{k-1}} \|[I - q(A)A](x - x^0)\|_A.$$

Im Sinne einer Bestapproximation können wir diese Aufgabe schreiben als

$$p \in P_{k-1} : \|[I - p(A)A](x - x^0)\|_A = \min_{q \in P_{k-1}} \|[I + q(A)A](x - x^0)\|_A. \quad (11.10)$$

Der Konvergenzbeweis zum CG-Verfahren baut daher auf dem entsprechenden Abschn. 9.5 und insbesondere Abschn. 9.5.2 auf. Es ist $q(A)A \in P_k(A)$, also suchen wir ein Polynom $q \in P_k$ mit der Eigenschaft $q(0) = 1$, sodass

$$\|x^k - x\|_A \leq \min_{q \in P_k, q(0)=1} \|q(A)\|_A \|x - x^0\|_A. \quad (11.11)$$

Die Konvergenz des CG-Verfahrens hängt davon ab, ob es uns gelingt, ein Polynom $q \in P_k$ mit der Eigenschaft $p(0) = 1$ zu finden, mit möglichst kleiner A -Norm. Wir zeigen:

Hilfssatz 11.9 (Schranke für Matrixpolynome) Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit mit Eigenwerten $0 < \lambda_1 \leq \dots \leq \lambda_n$, $p \in P_k$ ein Polynom mit $p(0) = 1$:

$$\|p(A)\|_A \leq M, \quad M := \min_{p \in P_k, p(0)=1} \sup_{\lambda \in [\lambda_1, \lambda_n]} |p(\lambda)|.$$

Beweis Es sei $\{q_1, \dots, q_n\}$ eine Orthogonalbasis aus Eigenvektoren. Ein beliebiges $y \in \mathbb{R}^n$ hat die Darstellung

$$y = \sum_{i=1}^n \gamma_i q_i.$$

Mit $Q = [q_1, \dots, q_n]$ gilt

$$A = Q^T D Q, \quad D = \text{diag}(\lambda_1, \dots, \lambda_n),$$

sodass für Polynome $p \in P_{k-1}$ folgt:

$$p(A) = \sum_{i=0}^{k-1} \alpha_i A^i = \sum_{i=0}^{k-1} \alpha_i (Q^T D Q)^i = Q^T \left(\sum_{i=0}^{k-1} \alpha_i D^i \right) Q = Q^T p(D) Q$$

Dann ist

$$\|p(A)y\|_A^2 = \sum_{i=1}^n \lambda_i p(\lambda_i)^2 \gamma_i^2 \leq \underbrace{\sup_{\lambda \in [\lambda_1, \lambda_n]} |p(\lambda)|^2}_{=: M^2} \sum_{i=1}^n \lambda_i \gamma_i^2 =: M^2 \|y\|_A^2.$$

Und schließlich folgt

$$\|p(A)\|_A = \sup_{y \in \mathbb{R}^n, y \neq 0} \frac{\|p(A)y\|_A}{\|y\|_A} = M. \quad \square$$

Mit diesem Resultat und der Fehlerabschätzung (11.11) können wir nun eine Konvergenzabschätzung für das CG-Verfahren herleiten.

Satz 11.10 (Konvergenz des CG-Verfahrens) Es sei $A \in \mathbb{R}^{n \times n}$ symmetrisch positiv definit, durch $b \in \mathbb{R}^n$ die rechte Seite und durch $x^0 \in \mathbb{R}^n$ ein beliebiger Startwert gegeben. Dann gilt

$$\|x^k - x\|_A \leq 2 \left(\frac{1 - 1/\sqrt{\kappa}}{1 + 1/\sqrt{\kappa}} \right)^k \|x^0 - x\|_A, \quad k \geq 0,$$

mit der Spektralkondition $\kappa = \text{cond}_2(A)$ der Matrix A .

Beweis Aus dem Hilfssatz und der Abschätzung (11.11) folgt

$$\|x^k - x\|_A \leq M \|x^0 - x\|_A$$

mit

$$M = \min_{q \in P_k, q(0)=1} \max_{\lambda \in [\lambda_1, \lambda_n]} |q(\lambda)|.$$

Es gilt, eine möglichst scharfe Abschätzung für die Größe M zu finden. Gesucht ist ein Polynom $q \in P_k$, welches am Nullpunkt den Wert eins annimmt, $q(0) = 1$, und auf dem Bereich $[\lambda_1, \lambda_n]$ möglichst nahe (in der Maximumsnorm) an 0 liegt.

Hierzu verwenden wir die Tschebyscheff-Approximation. Wir suchen die beste Approximation $p \in P_k$ zur Nullfunktion auf $[\lambda_1, \lambda_n]$. Dieses Polynom soll zusätzlich die Normierungseigenschaft $p(0) = 1$ besitzen. Daher scheidet die triviale Lösung $p = 0$ aus. Das Tschebyscheff-Polynom (siehe Abschn. 9.5.2 und Satz 9.42)

$$T_k = \cos(k \arccos(x))$$

hat die Eigenschaft

$$2^{-k-1} \max_{[-1,1]} |T_k(x)| = \min_{\alpha_0, \dots, \alpha_{k-1}} \max_{[-1,1]} |x^k + \sum_{i=0}^{k-1} \alpha_i x^i|,$$

ist also bei Normierung des größten Monoms das Polynom, dessen Maximum auf $[-1, 1]$ minimal ist. Wir wählen nun die Transformation

$$x \mapsto \frac{\lambda_n + \lambda_1 - 2t}{\lambda_n - \lambda_1}$$

und erhalten mit

$$p(t) = T_k \left(\frac{\lambda_n + \lambda_1 - 2t}{\lambda_n - \lambda_1} \right) T_k \left(\frac{\lambda_n + \lambda_1}{\lambda_n - \lambda_1} \right)^{-1}$$

das Polynom von Grad k , welches auf $[\lambda_1, \lambda_n]$ minimal ist und der Normierung

$$p(0) = 1$$

genügt. Es gilt

$$\sup_{t \in [\lambda_1, \lambda_n]} |p(t)| = T_k \left(\frac{\lambda_n + \lambda_1}{\lambda_n - \lambda_1} \right)^{-1} = T_k \left(\frac{\kappa + 1}{\kappa - 1} \right)^{-1} \quad (11.12)$$

mit der Spektralkondition

$$\kappa := \frac{\lambda_n}{\lambda_1}.$$

Wir nutzen die Darstellung der Tschebyscheff-Polynome außerhalb von $[-1, 1]$ aus Satz 9.42:

$$T_n(x) = \frac{1}{2} \left[(x + \sqrt{x^2 - 1})^n + (x - \sqrt{x^2 - 1})^n \right]$$

Für $x = \frac{\kappa+1}{\kappa-1}$ gilt

$$\frac{\kappa+1}{\kappa-1} + \sqrt{\left(\frac{\kappa+1}{\kappa-1}\right)^2 - 1} = \frac{\kappa+2\sqrt{\kappa}+1}{\kappa-1} = \frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1}$$

und entsprechend

$$\frac{\kappa+1}{\kappa-1} - \sqrt{\left(\frac{\kappa+1}{\kappa-1}\right)^2 - 1} = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}.$$

Hiermit kann (11.12) abgeschätzt werden:

$$T_k \left(\frac{\kappa+1}{\kappa-1} \right) = \frac{1}{2} \left[\left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1} \right)^k + \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \right)^k \right] \geq \frac{1}{2} \left(\frac{\sqrt{\kappa}+1}{\sqrt{\kappa}-1} \right)^k$$

Also folgt

$$\sup_{t \in [\lambda_1, \lambda_n]} T_k \left(\frac{\kappa+1}{\kappa-1} \right)^{-1} \leq 2 \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \right)^k = 2 \left(\frac{1 - \frac{1}{\sqrt{\kappa}}}{1 + \frac{1}{\sqrt{\kappa}}} \right)^k.$$

□

In Abb. 11.5 zeigen wir für die Situation $\lambda_1 = 1$ und $\lambda_n = 4$ die entsprechenden Polynome für $n = 2, 3, 4$.

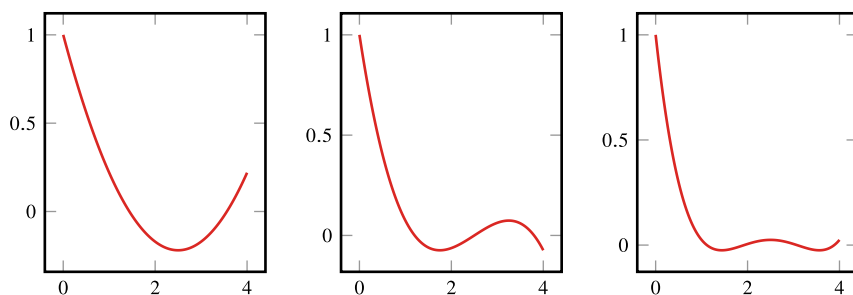


Abb. 11.5 Optimale Polynome zur Beschränkung der CG-Konvergenz. Es gilt $p(0) = 1$ sowie eine Minimierung auf dem Intervall $[1, 4]$. Links $n = 2$, Mitte $n = 3$ und rechts $n = 4$

Die Konvergenz des CG-Verfahrens ist gerade doppelt so schnell wie die des Gradientenverfahrens. Es gilt

$$\rho := \frac{1 - 1/\sqrt{\kappa}}{1 + 1/\sqrt{\kappa}} = 1 - 2\sqrt{\kappa} + O(\kappa).$$

Für die Modellmatrix folgt $\rho = 1 - \frac{1}{n}$. Das CG-Verfahren ist für dünn besetzte symmetrische Gleichungssysteme eines der effizientesten Iterationsverfahren. Die Konvergenz hängt wesentlich von der Kondition $\text{cond}_2(A)$ der Matrix A ab. Für $\text{cond}_2(A) \approx 1$ ist das Verfahren optimal: Zur Reduktion des Fehlers um einen gegebenen Faktor ϵ ist eine feste Anzahl von Schritten notwendig.

11.4 Exkurs: Preisbildung für Honigverkauf

In diesem Exkurs veranschaulichen wir die Lösung der kleinsten Quadrate (siehe auch Abschn. 4.5) mithilfe des Gradientenverfahrens (Abschn. 11.2). Dominik, der Bruder von Thomas, möchte seinen Honigverkauf auf Weihnachtsmärkten optimieren, siehe Abb. 11.6. Zunächst möchte er besser verstehen, wie sich die Erhöhung oder Reduzierung des Preises für ein Glas (0.5 kg) Honig auf seine Verkaufszahlen und den Umsatz auswirkt. Danach möchte er sein persönliches Gewinnmaximum bestimmen. Das heißt, bei welchem Preis erzielt er Verkaufszahlen, sodass der Umsatz bzw. der Gewinn maximal wird. Weiterhin stellen wir uns die Frage, ob diese Verkaufszahlen identisch sind? Zu diesem Zweck arbeitet er mit einer Stichprobe von N Messungen zu denen $(x_1, p_1), \dots, (x_N, p_N)$ er testet, wie viele Gläser x_k zum Preis p_k verkauft werden.

In der Volkswirtschaftslehre arbeitet man bei solchen Problemstellungen mit der sogenannten Preis-Absatz-Kurve (PAK), siehe z. B. [70] (bzw. Preis-Absatzfunktion). Das Gewinnmaximum ist derjenige Punkt der PAK, für den der maximale Gewinn erzielt wird. Um die beiden oben genannten Fragestellungen zu beantworten, führt Dominik eine Stichprobe auf 14 verschiedenen Weihnachtsmärkten in Deutschland und Österreich zu jeweils



Abb. 11.6 Bienenstöcke mit Bienen auf der Honigsuche und das fertige Produkt. Blütenhonig, auskristallisiert, cremig gerührt, und flüssiger Waldhonig

verschiedenen Preisen pro Honigglas durch. Hieraus konstruieren wir dann eine Regressionsgerade, die die PAK darstellt. Darauf aufbauend können wir dann die Umsatzfunktion und die Gewinnfunktion bestimmen, um die zweite Frage zu beantworten.

Die Stichprobe auf $N = 14$ verschiedenen Weihnachtsmärkten führt auf die in Tab. 11.3 angegebene Messreihe.

Modell der linearen Regression

Für die mathematische Analyse gehen wir von einem statistischen Modell und linearem Zusammenhang aus, der letztendlich auf die Ermittlung einer linearen Regressionsgeraden zurückgeführt wird. Die Basisgleichung lautet also

$$p(x) = b + mx,$$

wobei $p(x)$ für den Preis (pro Glas), b für den Höchstpreis, m für die Steigung und x für die Absatzmenge ($x = 1$ Glas) steht. Der Höchstpreis (auch Prohibitivpreis genannt) wird bei $x = 0$ erreicht und ist zu hoch, als dass ein Käufer diesen akzeptieren würde. Die Steigung m ist negativ, da die abgesetzte Menge x mit zunehmendem Preis abnimmt. Die Nullstelle von p charakterisiert die maximal abzusetzende Menge, auch Sättigungsmenge genannt. Letztere ist leicht zu verstehen, denn wenn Dominik seinen Honig verschenkt, wird er den größtmöglichen Absatz erzielen. Letztendlich müssen wir zwei unbekannte Parameter b und m der obigen Gleichung bestimmen.

Tab. 11.3 Stichprobe von $N = 14$ Messungen zu verschiedenen Preisen mit den jeweils erzielten Verkaufszahlen pro Glas Honig

Anz. x verkaufter Gläser	50	30	21	22	27	30	26	32	28	26	21	16	8	4
Preis $p(x)$ pro Glas	3.5	4.0	4.5	5.0	5.5	6.0	6.5	7.0	7.5	8.0	8.5	9.0	9.5	10.0

Ermittlung der unbekannten Parameter mit kleinsten Fehlerquadraten

Für die Ermittlung der beiden Unbekannten greifen wir auf Tab. 11.3 zurück und nutzen das Modell der kleinsten Fehlerquadrate (siehe auch Abschn. 4.5) der Ausgleichsrechnung:

$$S := S(m, b) = \frac{1}{N} \sum_{k=1}^N (p_k - (b + mx_k))^2,$$

wobei (x_k, p_k) die Verkaufs-Preis-Paare aus Tab. 11.3 darstellen. Die Anzahl der Messungen ist $N = 14$. Unser Ziel besteht darin, den Fehler S zu minimieren, sprich $\min S$.

Numerische Lösung mittels Gradientenverfahren

Wir sind an dem Minimum von S interessiert unter gleichzeitiger Berechnung der beiden unbekannten Parameter m und b . Diese Aufgabe kann als nichtlineares Optimierungsproblem klassifiziert werden, für das wir ein Iterationsverfahren nutzen, um dessen Lösung zu approximieren. Hierzu verwenden wir das bereits kennengelernte Gradientenverfahren aus Abschn. 11.2. Das heißt, wir benötigen

- Anfangswerte m_0 und p_0 ,
- eine Schrittweite ρ ,
- den Gradienten der Funktion $S := S(m, b)$.

Wir beginnen mit letzterem Schritt, den partiellen Ableitungen:

$$\begin{aligned} \frac{\partial S}{\partial m} &= \frac{2}{N} \sum_{k=1}^N -x_k (p_k - (mx_k + b)) \\ \frac{\partial S}{\partial b} &= \frac{2}{N} \sum_{k=1}^N -(p_k - (mx_k + b)) \end{aligned}$$

Der finale Algorithmus ist wie folgt gegeben: Für $l = 1, 2, 3, \dots, N_{\max}$ iteriere

$$\begin{pmatrix} m_{l+1} \\ b_{l+1} \end{pmatrix} = \begin{pmatrix} m_l \\ b_l \end{pmatrix} - \rho \begin{pmatrix} \frac{\partial S(m_l)}{\partial m} \\ \frac{\partial S(b_l)}{\partial b} \end{pmatrix}.$$

Wenn wir als Initialwerte $b_0 = 10.5$ und $m_0 = -2$ nehmen und die Schrittweite ist $\rho = 10^{-4}$, dann erhalten wir nach $N_{\max} = 200$ Iterationen die Werte

$$b = 10.5613025417, \quad m = -0.154072634203.$$

Ergebnis der Schätzung und Interpretation

Nachdem wir nun m und b bestimmt haben, erhalten wir folgende Regressionsgerade für den Preis

$$p(x) = \underbrace{10.561}_{=b} - \underbrace{0.1541}_{=m} x. \quad (11.13)$$

Der Prohibitivpreis liegt also bei rund 10.50 EUR pro Glas. Wenn wir die Nachfragefunktion bilden (also die Inverse zu (11.13)), sprich

$$x(p) = 68.548 - 6.4904p,$$

dann können wir auch relativ einfach ausrechnen, um wie viel der durchschnittliche Verkauf sinken wird, wenn wir den Preis um 1 EUR pro Glas erhöhen: Das sind hier durchschnittlich 6.5 Gläser. Die Sättigungsmenge liegt demnach bei 68.548 Gläsern (bei diesem Preis würde Dominik jedes Glas verschenken). Die beiden Regressionsgeraden und die Stichprobendaten sind in Abb. 11.7 veranschaulicht.

Wir kehren nun zu der zweiten unserer ursprünglichen Fragen zurück, nämlich an welchem Punkt ein Gewinnmaximum erzielt würde.

Der Gewinn G setzt sich bekanntermaßen aus Umsatz U minus Kosten K zusammen. Zunächst untersuchen wir die Umsatzfunktion, welche aus dem Preis multipliziert mit der Menge x berechnet wird:

$$U := U(x) = p(x) \cdot x = 10.561x - 0.1541x^2$$

Der Umsatz U ist also eine quadratische Funktion, siehe Abb. 11.8. Das Maximum von U stellt dann das Umsatzmaximum dar:

$$U'(x) = 0 \Leftrightarrow 10.5613 - 0.30815x = 0 \Rightarrow x = 34.273$$

Das heißt, bei (abgerundet) 34 verkauften Gläsern zu einem Preis von 5.28 EUR pro Glas (Einsetzen von $x = 34$ in (11.13)) wird Dominik den maximalen Umsatz erzielen (mathematisch ist dies auch gerechtfertigt, da die zweite Ableitung von U negativ ist ($U''(x) = -0.30815 < 0$) und daher tatsächlich ein Maximum vorliegt). Angesichts die-

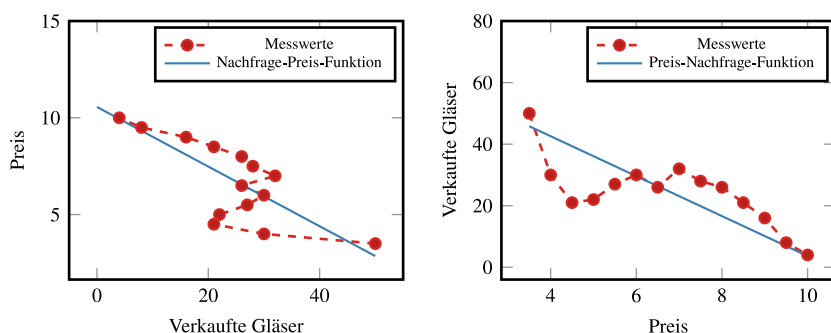
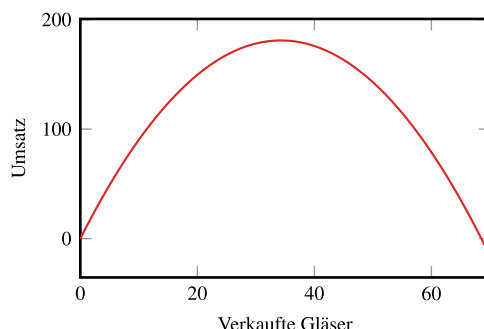


Abb. 11.7 Die Nachfragefunktion (oben) und die PAK (unten)

Abb. 11.8 Die Umsatzfunktion U . Der größte Umsatz wird bei 34.273 Gläsern erzielt. Kein Umsatz wird gemacht bei keinem Verkauf (linke Nullstelle) sowie bei der Sättigungsmenge $x = 68.548$ (rechte Nullstelle), wenn jedes Glas verschenkt wird



ser Auswertungen sollte also der Preis pro Glas bei 5,30 EUR liegen, um den maximalen Umsatz zu erzielen.

Abschließend betrachten wir die Gewinnkurve: $G = U - K$. Für die Kosten nehmen wir einen Mittelwert (bei 20 Bienenvölkern) von 2.59 EUR pro Glas an [16]. Dieser Mittelwert wird als variable Kosten pro Glas klassifiziert (d. h., etwaige globale Fixkosten sind bereits enthalten):

$$\begin{aligned} G(x) &= U(x) - K(x) = p(x) \cdot x - K(x) \\ &= 10.561x - 0.1541x^2 - 2.59x \\ &= 7.971x - 0.1541x^2 \end{aligned}$$

Hieraus können wir sofort den Grenzgewinn ermitteln, $G'(p)$, der angibt, um wie viel sich der Gewinn ändert, wenn ein Glas Honig mehr verkauft wird¹:

$$G'(x) = 7.971 - 0.30815x$$

Aus $G'(x)$ berechnen wir auch das Maximum von $G(x)$, welches hier bei $25.87 \approx 26$ Gläsern liegt. Basierend auf unserem mathematischen Modell, der zugrundeliegenden Stichprobe und der numerischen Lösung wird das Gewinnmaximum bei einem Verkauf von 26 Gläsern zu einem Preis von 5.28 EUR pro Glas erzielt.

In Abb. 11.9 erkennen wir auch das typische Verhalten des Gewinnmaximums: Dieses liegt links vom Umsatzmaximum. Um maximalen Gewinn zu erzielen, verkauft Dominik eine geringere Menge an Honig im Vergleich zum maximalen Umsatz.

¹ Diese Aussage ist identisch zur Definition des Grenzsteuersatzes in Beispiel 1.6 in der Einleitung.

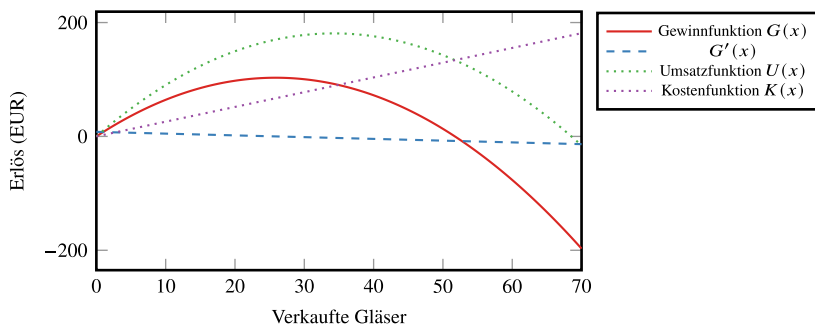


Abb. 11.9 Die Gewinnfunktion $G(x)$ und deren Ableitung $G'(x)$ im Vergleich zur Umsatzfunktion $U(x)$ und der Kostenfunktion $K(x)$

11.5 Künstliche neuronale Netze

Mathematisch betrachtet sind *künstliche neuronale Netzwerke* (KNN) Funktionen, die aus sehr einfach aufgebauten Bausteinen zusammengesetzt sind und die über eine sehr große Anzahl von freien Parametern verfügen. Diese Parameter können bestimmt werden, damit das künstliche neuronale Netzwerk eine bestimmte mathematische Aufgabe erledigt, was nichts andere bedeutet, als dass es eine Funktion approximiert, welche das Ergebnis der Aufgabe bestimmt. Das *maschinelle Lernen* und insbesondere die Arbeit mit künstlichen neuronalen Netzen kann daher als eine Art der Approximation angesehen werden. Aber anders als bei der Polynominterpolation oder der trigonometrischen Interpolation kommt beim *maschinellen Lernen* mit den *künstlichen neuronalen Netzen* eine gänzlich andere Art von Funktionen zum Einsatz. Hier wählen wir zur Approximation einer Funktion keinen strukturierten Vektorraum, wie den Raum der Polynome oder der trigonometrischen Polynome, sondern setzen Funktionen aus elementaren Bauteilen, den künstlichen Neuronen, zusammen.

Zunächst war die mathematische Forschung an künstlichen neuronalen Netzen wirklich von den natürlichen Vorbildern, d.h. von der Funktionsweise von Neuronen im Gehirn, inspiriert. Heute sind sie insbesondere ein enorm leistungsfähiges Werkzeug, dass sich auf modernen Computern sehr effizient einsetzen lässt. Viele Probleme, die lange offen waren, konnten mit Hilfe künstlicher neuronaler Netze endlich erfolgreich bewältigt werden. Hier stechen insbesondere die Fortschritte in der Sprachbearbeitung wie automatische Übersetzung, Spracherkennung und generative Sprachmodelle wie ChatGPT heraus, und ebenso die großen Erfolge in der Bild- und Videoverarbeitung.

Mathematisch können wir künstliche neuronale Netzwerke als eine Methode zur Approximation von Funktionen betrachten. Deren Analyse ist jedoch schwieriger und viele Hilfsmittel fehlen: die Lagrange-Interpolation beruhte wesentlich auf der Taylor-Entwicklung sowie auf der Darstellung des Interpolationspolynoms mit Basispolynomen. Neuronale Netze sind jedoch oft nicht differenzierbar und sie stellen auch keinen Vektorraum mit einer Basis dar.

Die Bestapproximation nach Gauss ist ein Minimierungsproblem mit einem quadratischen Funktional, welches sich mit Hilfe des Skalarproduktes exakt lösen lässt. Neuronale Netze werden auch mit Hilfe von Minimierungsproblemen bestimmt, diese Minimierungsaufgaben sind jedoch von komplizierter nichtlinearer Struktur, nicht quadratisch und im Normalfall nicht eindeutig lösbar. Es fehlen also wichtige Struktureigenschaften für die Analyse.

11.5.1 Aufbau künstlicher neuronaler Netze

Das Grundelement ist das *künstliche Neuron* und der Name ist der Funktionsweise von Neuronen im Gehirn nachempfunden:

► **Definition 11.11 (Künstliches Neuron)** Es sei $\mathbf{w} \in \mathbb{R}^n$ und $b \in \mathbb{R}$ sowie $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Dann heißt die Funktion

$$f : \mathbb{R}^n \rightarrow \mathbb{R}, \quad f(x) = \sigma\left(\sum_{i=1}^n w_i x_i + b\right),$$

ein *künstliches Neuron*. Der Vektor $w \in \mathbb{R}^n$ heißt Gewicht und $b \in \mathbb{R}$ ist der Bias. Die Funktion $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ heißt *Aktivierungsfunktion*.

Es gibt keine klare Definition des Begriffs Aktivierungsfunktion. Stattdessen meint man damit eine (nichtlineare) Funktion, die entscheidet, wann die Ausgabe eines künstlichen Neurons aktiv ist. Die einfachste Variante ist die Heaviside-Funktion:

$$\sigma_{\text{HS}}(s) = H(s) := \begin{cases} 0 & s < 0, \\ 1 & s \geq 0 \end{cases} \quad (11.14)$$

Eine stetige Aktivierungsfunktion ist die *Rectified Linear Unit (ReLU)*

$$\sigma_{\text{ReLU}}(s) = \text{ReLU}(s) := \begin{cases} 0 & s \leq 0, \\ s & s > 0. \end{cases} \quad (11.15)$$

Stetig differenzierbar (beliebig oft) sind die *Sigmoid-Aktivierung*

$$\sigma(s) = \frac{1}{1 + \exp(-s)}, \quad (11.16)$$

oder der hyperbolische Tangens

$$\sigma_{\tanh}(s) = \tanh(s). \quad (11.17)$$

Diese beiden sind eng verwandt, es gilt

$$\sigma(s) = \frac{1}{2} \tanh\left(\frac{1}{2}s\right) + \frac{1}{2}.$$

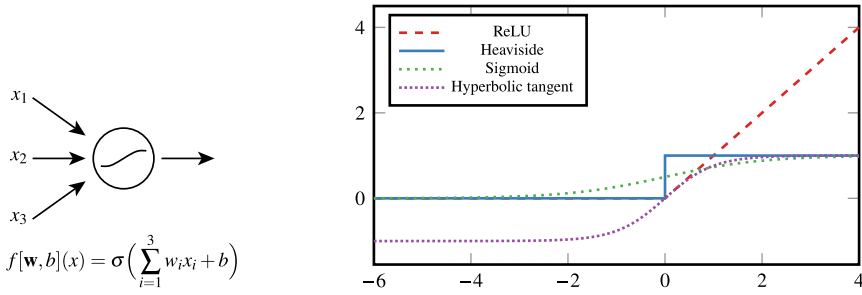


Abb. 11.10 Links: Skizze eines künstlichen Neurons mit 3 Eingangssignalen, der linearen Schicht und der nichtlinearen Aktivierung. Rechts: Verlauf einiger typischer Aktivierungsfunktionen

In Abb. 11.10 zeigen wir den Verlauf dieser Aktivierungsfunktionen sowie den schematischen Aufbau eines künstlichen Neurons.

Beispiel 11.12 (Klassifikation mit künstlichen Neuronen)

Ein künstliches Neuron mit Heaviside-Aktivierung

$$f(x) = H((\mathbf{w}, x)_2 + b),$$

trennt den Raum an einer Hyperebene, denn jede Ebene mit Normalenvektor n und ausgewähltem Punkt x_0 ist gegeben durch

$$0 = n^T (x - x_0) = n^T x - n^T x_0.$$

Das Gewicht entspricht demnach dem Normalvektor und der Bias ist das Skalarprodukt mit dem ausgewählten Punkt. Ein Neuron trennt somit den Raum in die beiden Hälften jenseits der Ebene und dies ist eine der grundlegenden Aufgaben, für die künstliche neuronale Netze eingesetzt werden können. Gegeben seien $x_1, \dots, x_N$, denen jeweils eine Eigenschaft y_i , hier einfach $y_i \in \{0, 1\}$ zugeordnet ist. Wir suchen die Gewichte $\mathbf{w}$ und den Bias b des künstlichen Neurons, so dass $f[\mathbf{w}, b](x_i) = y_i$ gilt. D.h., wir suchen ein Modell, welches die Klassifizierung der Daten in Bezug auf die Eigenschaften vornimmt. Die Gewichte verstärken oder schwächen den Einfluß des Inputs und somit den Übergang von Nicht-Aktivierung eines Neurons zur Aktivierung. Der Bias sorgt für eine Verschiebung der Aktivierung.

Ein Modell bestehend aus einem einzelnen Neuron ist sehr eingeschränkt und es muss davon ausgegangen werden, dass die Aufgabe nicht exakt zu lösen ist. Stattdessen versuchen wir das beste Modell zu finden, d.h. wir betrachten analog zur Regression in den Abschn. 4.5 und dem Exkurs 11.4 die Minimierungsaufgabe

$$\min_{(\mathbf{w}, b)} \sum_{i=1}^N \|y_i - f[\mathbf{w}, b](x_i)\|^2.$$

Künstliche neuronale Netzwerke (KNN) werden aus einzelnen künstlichen Neuronen zusammengesetzt. Die einfachsten KNN sind sogenannte *Ein-Layer-Netzwerke*:

► **Definition 11.13 (Ein-Schicht Netzwerk)** Es sei $W \in \mathbb{R}^{n \times m}$ und $b \in \mathbb{R}^n$ sowie $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ eine Aktivierungsfunktion. Dann heißt die Funktion

$$f(x) = \sigma(Wx + b),$$

ein *künstliches neuronales Netzwerk mit einer Schicht* und n künstlichen Neuronen. Dabei ist die Anwendung der Aktivierungsfunktion komponentenweise zu verstehen, d. h. für $y \in \mathbb{R}^n$ ist

$$\sigma(y) = (\sigma(y_i))_{i=1, \dots, n}.$$

Von dieser Definition kommen wir schnell zu allgemeineren künstlichen neuronalen Netzen, den sogenannten *tiefen neuronalen Netzen* oder eben *deep neural networks*. Hierzu schalten wir mehrere Ein-Schicht-Netzwerke in Reihe. Die Ausgabe von Schicht l ist Eingabe von Schicht $l + 1$.

► **Definition 11.14 (Tiefes neuronales Netz)** Es sei $L \in \mathbb{N}$ die Anzahl der Schichten. Für $l = 1, \dots, L$ seien $W^l \in \mathbb{N}^{n_l \times n_{l-1}}$ sowie $b^l \in \mathbb{R}^{n_l}$, und durch $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ sei eine Aktivierungsfunktion gegeben. Dann ist durch $f : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}$ gemäß

$$\begin{aligned} x^0 &= x \\ z^l &= W^l x^{l-1} + b^l, \quad x^l = f^l(x^{l-1}) = \sigma(z^l), \quad l = 1, \dots, L-1, \\ f(x) &= x^L = W^L x^{L-1} + b^L, \end{aligned} \tag{11.18}$$

ein *deep neural network* gegeben. Die erste Schicht $f^1(\cdot)$ heißt *Eingabeschicht*, die letzte Schicht $f^L(\cdot)$ die *Ausgabeschicht* und hier wird im Allgemeinen keine Aktivierungsfunktion hinzugefügt. Die inneren Schichten heißen *verborgene Schichten (hidden Layer)*. Wir benennen die Architektur eines tiefen neuronalen Netzwerkes mit $\mathcal{N}(\sigma; n_0, n_1, \dots, n_L)$, bzw. kurz mit $\mathcal{N}(L, n)$ mit $n = \max_l n_l$. Den zugehörigen Parameterraum bezeichnen wir mit

$$\mathcal{P}_{\mathcal{N}} = \{\mathbf{W} = (W_1, \dots, W_L), \mathbf{b} = (b_1, \dots, b_L), W_l \in \mathbb{R}^{n_l \times n_{l-1}}, b_l \in \mathbb{R}^{n_l}\}.$$

Für eine Parameterwahl $(\mathbf{W}, \mathbf{b}) \in \mathcal{P}$ bezeichnen wir mit $f[\mathbf{W}, \mathbf{b}] : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}$ die resultierende Funktion gemäß (11.18). Die Menge der *Realisierungen*, d. h., die Menge der in der Architektur darstellbaren Funktionen benennen wir mit

$$\mathcal{F}_{\mathcal{N}} = \{f[\mathbf{W}, \mathbf{b}] : (\mathbf{W}, \mathbf{b}) \in \mathcal{P}_{\mathcal{N}}\}.$$

Das Abzählen der freien Parameter ergibt:

Korollar 11.15 (Anzahl freier Parameter) Es sei $\mathcal{N}(\sigma; n_0, \dots, n_L)$ ein künstliches neuronales Netz mit L Schichten und jeweils n_l künstlichen Neuronen. Dann hat der Parameterraum $\mathcal{P}_{\mathcal{N}}$ insgesamt

$$\#\mathcal{P}_{\mathcal{N}} = \sum_{l=1}^L n_l(n_{l-1} + 1)$$

freie Parameter, davon $\sum_{l=1}^L n_l n_{l-1}$ Gewichte und $\sum_{l=1}^L n_l$ Biaswerten.

Eine Grundoperation im maschinellen Lernen ist das Auswerten eines gegebenen Netzes $\mathcal{N}(\sigma; n_0, \dots, n_L)$ bei gegebenen Parametern $(\mathbf{W}, \mathbf{b}) \in \mathcal{P}_{\mathcal{N}}$ für einen Datenpunkt $x \in \mathbb{R}^{n_0}$ oder, meistens für eine ganze Liste von (eventuell sehr vielen) Datenpunkten $x_i \in \mathbb{R}^{n_0}$ für $i = 1, \dots, N_{tr}$.

Algorithmus 11.6: Auswertung eines künstlicher neuronaler Netze

Eingabe : Künstliches neuronales Netzwerk $\mathcal{N}(\sigma; n_0, \dots, n_L)$, Parameter $(\mathbf{W}, \mathbf{b}) \in \mathcal{P}$ und Eingabe $x \in \mathbb{R}^{n_0}$

```

1  $x^0 = x$ 
2 Für  $l = 1, 2, \dots, L$  tue
3    $z^l = W^l x^{l-1} + b^l$ 
4   Wenn  $l < L$  dann
5      $x^l = \sigma(z^l)$ 
6   Sonst
7      $x^L = z^L$ 

```

Ergebnis : Netzwerkausgabe $x^L = f[\mathbf{W}, \mathbf{b}](x)$

Bemerkung 11.16 (Netzwerk-Architekturen) Die obige Definition führt nur die einfachste Art von künstlichen neuronalen Netzwerken ein: alle Ausgaben von Schicht $l-1$ sind Eingabe für alle künstlichen Neuronen der Schicht l . Diese Netze werden auch *Multilayer perceptron (mehrlagiges Perzeptron)* genannt. Sie gehören zur Kategorie der *Feedforward neural networks*, d. h., die Information geht nur in eine Richtung, von Schicht 1 zu Schicht 2, und so weiter. Neuronale Netze, in denen die Ausgabe der Schicht l auch als Eingabe von früheren Schichten $l' < l$ dienen kann, heißen *recurrent neural networks* (rekurrente neuronale Netze).

Prinzipiell kann in jeder Schicht eine andere Aktivierungsfunktion verwendet werden. Wird in der letzten Schicht eine Aktivierungsfunktion angewendet, so schränkt dies den Bildraum ein, bei der Sigmoid-Aktivierung auf $(0, 1)$, bei der ReLU-Aktivierung auf die nicht-negativen Zahlen.

Sogenannte *skip connections* lassen einzelne Schichten aus. D. h., die Ausgabe von Schicht l kann direkt als Eingabe von z. B. Schicht $l+2$ verwendet werden. Eine andere einfache Modifikation sind *residual networks*, hier bildet sich eine Schicht in der Form

$$f(x) = x + \sigma(Wx + b),$$

d. h., das Ergebnis des künstlichen Neurons wird zur Eingabe addiert.

In der Bildverarbeitung spielen die *convolutional neural networks* eine große Rolle. Hier ist jede Schicht eine *Faltung* (engl. convolution). Das besondere an convolution networks ist, dass die Größe der Eingabe x nicht unbedingt an die Anzahl von Gewichten gekoppelt sein muss. Mit den Faltungs-Gewichten $W \in \mathbb{R}^f$ definieren wir

$$f_{conv}(x) = \sigma(W * x), \quad (W * x)_i = \sum_{j=1}^f W_j x_{i-j},$$

wobei $x_0 = x_{-1} = \dots = x_{1-f} = 0$ gesetzt wird. Die gleichen Gewichte werden immer nur auf einen kleinen Teil der Eingaben angewendet. Auf diese Weise können mit nur wenigen freien Parametern (den Gewichten) Funktionen in beliebig hochdimensionalen Räumen dargestellt werden.

Moderne Architekturen, wie sie zum Beispiel in Sprachmodellen wie ChatGPT² verwendet werden sind weitaus komplexer, nutzen und kombinieren aber auch die hier eingeführten Basismodelle. In der Praxis eingesetzte künstliche neuronale Netze haben eine enorme Größe, z. B. hat ChatGPT-3 etwa 200 Milliarden, also $2 \cdot 10^{11}$ freie Parameter.

In Abb. 11.11 stellen wir einige künstliche neuronale Netze graphisch dar. Einen ausführlichen Überblick gibt die Literatur [11, 17, 43, 51, 66]. ◆

11.5.2 Approximation mit neuronalen Netzen

Wir betrachten ausschließlich tiefe künstliche neuronale Netze vom Typ der Definition 11.14 und fassen ein Netz mit L Schichten als Funktion $f : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}$ mit den Gewichtsmatrizen W_l und Bias-Vektoren b_l als freie Parameter. Zunächst stellen wir einen einfachen aber wichtigen Zusammenhang fest:

Satz 11.17 *Es sei $\mathcal{N} = \mathcal{N}(\sigma, n_0, \dots, n_L)$ eine Netzwerkarchitektur mit L Schichten und Parametern $W_l \in \mathbb{R}^{n_l \times n_{l-1}}$ sowie $b_l \in \mathbb{R}^{n_l}$ für $l = 1, \dots, L$. Die Menge der Realisierungen*

$$\mathcal{F}_{\mathcal{N}} := \{f[\mathbf{W}, \mathbf{b}] : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}, \quad (\mathbf{W}, \mathbf{b}) \in \mathcal{P}_{\mathcal{N}}\},$$

ist im allgemeinen kein Vektorraum. Insbesondere ist $\mathcal{F}_{\mathcal{N}}$ nicht abgeschlossen bzgl. der Addition oder skalaren Multiplikation.

Beweis Wir konstruieren ein einfaches Gegenbeispiel:

$$\mathcal{F}_{\mathcal{N}} := \{f : \mathbb{R} \rightarrow \mathbb{R}, \text{ReLU}(wx + b), \quad w, b \in \mathbb{R}\}$$

² Chat Generative Pre-trained Transformer, ein Sprachmodell basierend auf neuronalen Netzen, siehe <https://en.wikipedia.org/wiki/ChatGPT>.

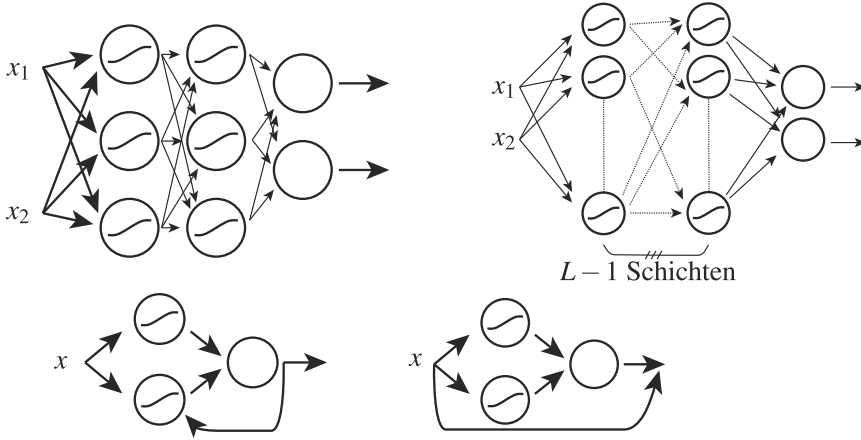


Abb. 11.11 Beispiele für künstliche neuronale Netze. Oben links: einfaches mehrschichtiges Netzwerk (multilayer perceptron) $\mathcal{N}(\sigma; 2, 3, 3, 2)$ mit Sigmoid-Aktivierung (bis auf die Ausgangsschicht). Oben rechts: dieses Netzwerk stellt Funktionen $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ dar. Tiefes neuronales Netzwerk 2 Eingaben und Ausgaben und $L - 1$ verborgenen Schichten mit jeweils n Neuronen. Unten links: *rekurrentes Neuronales Netzwerk*, bei dem Ausgaben auch Eingaben für frühere Schichten sein können. Unten rechts: Künstliches neuronales *residual network*. Die Eingabe wird zur Ausgabe addiert. Dieses Netz stellt Funktionen $f : \mathbb{R} \rightarrow \mathbb{R}$ dar mit dem Aufbau $f(x) = x + f[\mathbf{W}, \mathbf{b}](x)$

und wählen die zwei Realisierungen $f_1, f_2 \in \mathcal{F}_{\mathcal{N}}$

$$f_1(x) = \text{ReLU}(x) = \max\{x, 0\}, \quad f_2(x) = \text{ReLU}(-x) = \max\{-x, 0\}.$$

Es gilt

$$f_1(x) + f_2(x) = |x| \notin \mathcal{F}_{\mathcal{N}},$$

da die Betragsfunktion nicht in der Form $\text{ReLU}(wx + b)$ zu schreiben ist. $\square$

Der Zusammenhang mag klar sein, hat aber wesentliche Konsequenzen: Es gibt keine Basis von $\mathcal{F}_{\mathcal{N}}$ und wir können auch keinerlei vertraute Struktureigenschaften wie Skalarprodukte oder die Linearität nutzen. Diese Hilfsmittel waren entscheidend für die effiziente Umsetzung von Bestapproximationen oder Lagrange-Interpolationen. Zur Suche der optimalen Parameter und Koeffizienten müssen wir also neue Wege einschlagen.

Bereits in den 1980er Jahren wurde nachgewiesen, dass neuronale Netze sehr gute Approximationseigenschaften haben, genauer gesagt, dass die Menge $\mathcal{F}_{\mathcal{N}(L,n)}$ für $L \rightarrow \infty$ oder $n \rightarrow \infty$ dicht in der Menge der stetigen Funktionen liegt.

Theorem 11.1 (Universelles Approximationstheorem) Es sei $I = [a, b] \subset \mathbb{R}$, und σ sei eine stetige Aktivierungsfunktion mit der Eigenschaft

$$\sigma(s) \xrightarrow{s \rightarrow -\infty} 0, \quad \sigma(s) \xrightarrow{s \rightarrow \infty} 1.$$

Dann ist die Menge der neuronalen Netze mit einer verborgenen Schicht $\mathcal{F}_{\mathcal{N}(\sigma; N, 1)}$, also

$$\mathcal{F}(N) = \left\{ \sum_{i=1}^N w_i^2 \sigma(w_i^1 x + b_i^1) \right\},$$

dicht in $C(I)$.

Beweis Beweise sind z. B. bei Cybenko [20] oder Hornik [52] gegeben. Es werden zum Teil tief liegende Resultate der Funktionalanalysis benötigt. $\square$

Der Satz bedeutet, dass es zu jeder stetigen Funktion $f \in C(I)$ und jedem positiven $\epsilon > 0$ ein neuronales Netzwerk mit Architektur $\mathcal{N}(\sigma; 1, N, 1)$ gibt und eine Realisierung $f[\mathbf{W}, \mathbf{b}] \in \mathcal{F}_{\mathcal{N}}$, so dass $\max_{x \in I} \|f(x) - f[\mathbf{W}, \mathbf{b}](x)\| \leq \epsilon$ gilt. Das Resultat gibt jedoch noch keine quantitativen Abschätzungen, d. h., es sagt nichts darüber aus, wie schnell N wachsen muss, wenn $\epsilon \rightarrow 0$ kleiner wird.

Das ursprüngliche Resultat wurde für Aktivierungsfunktionen vom Sigmoid-Typ gezeigt, kann jedoch auf fast beliebige Funktionen erweitert werden und die Eigenschaft bleibt erhalten, solange die Aktivierungsfunktion kein Polynom ist. Die Dichtheit überträgt sich auch auf Funktionen $C(X)$ für $X \subset \mathbb{R}^n$ und ebenso auf vektorwertige Funktionen.

Einfacher zu verstehen sind ReLU-Netze. Es gilt:

Satz 11.18 (ReLU-Netze) Es sei $\mathcal{N}$ eine neuronale Netzwerkarchitektur mit ReLU-Aktivierung. Dann ist jedes $f[\mathbf{W}, \mathbf{b}] \in \mathcal{F}_{\mathcal{N}}$ eine stückweise lineare Funktion.

Beweis Das Netz ist aufgebaut als Verkettung von einschichtigen Netzen

$$f = f_L \circ f_{L-1} \circ \cdots \circ f_1.$$

Jedes $f_l(x) = \text{ReLU}(W_l x_l + b_l)$ ist als Komposition der affin linearen Abbildung $W_l x_l + b_l$ und der stückweise linearen Abbildung $\text{ReLU}(\cdot)$ stückweise linear. Also ist auch ganz f stückweise linear. $\square$

Im einfachen skalaren Fall können wir zu jedem Polygonzug ein entsprechendes neuronales Netz konstruieren:

Satz 11.19 Es sei $I = [a, b]$. Jede stückweise lineare Funktion mit $N \in \mathbb{N}$ paarweise verschiedenen Stützstellen lässt sich als künstliches neuronales ReLU-Netzwerk mit einer Schicht und N künstlichen Neuronen darstellen.

Beweis Es seien $(x_i, y_i) \in [a, b] \times \mathbb{R}$ für $i = 1, \dots, N$ Stützstellenpaare der stückweise linearen Funktion p_h , mit $p_h(x_i) = y_i$. Ohne Einschränkung nehmen wir an, dass $x_{i-1} < x_i$ gilt. Dann konstruieren wir das Netzwerk als

$$f(x) = y_1 + \sum_{i=2}^N \frac{y_i - y_{i-1}}{x_i - x_{i-1}} \text{ReLU}(x - x_{i-1}) - \sum_{i=3}^N \frac{y_{i-1} - y_{i-2}}{x_{i-1} - x_{i-2}} \text{ReLU}(x - x_{i-1}). \quad (11.19)$$

Auf jedem Teilintervall (x_{k-1}, x_k) ist $f(x)$ eine lineare Funktion, denn es gilt

$$\text{ReLU}(x - x_{i-1}) = \begin{cases} 0 & x < x_{i-1}, \\ x & x \geq x_{i-1}, \end{cases}$$

d. h., $\text{ReLU}(x - x_{i-1})$ ist stetig und stückweise linear. Damit ist auch die Summe stückweise linear.

Es sei $x \in [x_{k-1}, x_k]$. Dann gilt $\text{ReLU}(x - x_{i-1}) = x - x_{i-1}$ für $x \geq x_{i-1}$, also $i \leq k$ und $\text{ReLU}(x - x_{i-1}) = 0$ für $x \leq x_{i-1}$ also für $k < i$. Hiermit folgt

$$\begin{aligned} f(x) \Big|_{[x_{k-1}, x_k]} &= y_1 + \sum_{i=2}^k \frac{y_i - y_{i-1}}{x_i - x_{i-1}} (x - x_{i-1}) - \sum_{i=3}^k \frac{y_{i-1} - y_{i-2}}{x_{i-1} - x_{i-2}} (x - x_{i-1}) \\ &= y_1 + \sum_{i=2}^k \frac{y_i - y_{i-1}}{x_i - x_{i-1}} x - \sum_{i=2}^{k-1} \frac{y_i - y_{i-1}}{x_i - x_{i-1}} x \\ &\quad - \sum_{i=2}^k \frac{y_i - y_{i-1}}{x_i - x_{i-1}} (x_{i-1}) + \sum_{i=2}^{k-1} \frac{y_i - y_{i-1}}{x_i - x_{i-1}} (x_i) \\ &= y_1 + \underbrace{\sum_{i=2}^{k-1} y_i - y_{i-1}}_{=y_{k-1}} + \frac{y_k - y_{k-1}}{x_k - x_{k-1}} (x - x_{k-1}). \end{aligned}$$

Dies entspricht gerade der stückweise linearen Funktion auf $[x_{k-1}, x_k]$. Der Ausdruck (11.19) lässt sich schreiben als

$$f(x) = b_1 + \sum_{i=2}^N w_i \text{ReLU}(x - b_i),$$

mit den Gewichten $N - 1$ Gewichten

$$w_2 = \frac{y_2 - y_1}{x_2 - x_1}, \quad w_i = \frac{y_i - y_{i-1}}{x_i - x_{i-1}} - \frac{y_{i-1} - y_{i-2}}{x_{i-1} - x_{i-2}}, \quad i = 3, \dots, N,$$

und den N Biaswerten

$$b_1 = y_1, \quad b_i = x_{i-1}, \quad i = 2, \dots, N.$$

□

Hiermit können wir ein einfaches quantitatives Konvergenzresultat für neuronale Netze herleiten:

Korollar 11.20 *Die einschichtigen ReLU Netze sind gerade die stückweise linearen Funktionen. Zu einer Funktion $f \in C^2[a, b]$ existiert eine Folge von neuronalen Netzwerkfunktionen $f_N \in \mathcal{F}_{\mathcal{N}(\text{ReLU}, N)}$ mit*

$$\max_{x \in [a, b]} \|f(x) - f_N(x)\| \leq C \frac{1}{N^2} \max_{x \in [a, b]} \|f''(x)\|$$

mit einer Konstanten $C > 0$.

Beweis Satz 11.19 zeigt, dass f_N eine stückweise lineare Funktion mit N Intervallen ist. Bei gleichmäßiger Zerlegung von $[a, b]$ ist $h = (b - a)/N$ die Intervallgröße. Mit der Fehlerabschätzung für stückweise lineare Funktionen (9.7) folgt dann sofort die Fehlerabschätzung. $\square$

Bemerkung 11.21 (Approximation hochdimensionaler Funktionen) Künstliche neuronale Netze spielen ihre Stärke gerade bei der Approximation hochdimensionaler Funktionen $f : \mathbb{R}^n \rightarrow \mathbb{R}$ mit $n \gg 1$ aus. Wollen wir so eine Funktion mit klassischen Mitteln, z. B. mit stückweise linearen Funktionen approximieren, so ist der erste Schritt stets eine Zerlegung des Definitionsbereiches $X \subset \mathbb{R}^n$ in ein Gitter. Als einfaches Beispiel wählen wir den n -dimensionalen Einheitswürfel

$$W_n = \{x \in \mathbb{R}^n, 0 \leq x_i \leq 1, i = 1, \dots, n\}.$$

Ein regelmäßiges Punktgitter von W_n mit Gitterweite $h = 1/M$ besteht aus den Punkten

$$(x_i^1, x_i^2, \dots, x_i^n), \quad x_i^j = \frac{i}{M}, \quad i = 0, \dots, M.$$

Insgesamt hat das Gitter somit mit $(M + 1)^n$ Punkten eine exponentiell steigende Komplexität. Dies bedeutet, dass der Aufwand zum Erreichen einer gewissen Genauigkeit exponentiell mit der Dimension steigt. Dieser Zusammenhang ist üblich bei der Approximation aller hochdimensionalen Probleme und wird der *Fluch der Dimension* genannt.

In bestimmten Situationen können künstliche neuronale Netze solche Funktionen viel effizienter approximieren. Das ist zumeist dann der Fall, wenn die zu approximierende Funktion eine inhärente niederdimensionale Struktur hat. Hierzu diskutieren wir ein einfaches Beispiel. Die zehndimensionale Funktion

$$f : \mathbb{R}^{10} \rightarrow \mathbb{R}, \quad f(x) = \sum_{i=1}^{10} x_i^2,$$

lässt sich über eine Koordinatentransformation

$$f(r) = r^2, \quad r = \left(\sum_{i=1}^{10} x_i^2 \right)^{\frac{1}{2}},$$

als eindimensionale Funktion darstellen. Hier haben wir die symmetrische Struktur der Funktion ausgenutzt und sie einfacher dargestellt. Der Aufwand würde von $O(M^{10})$ auf $O(M)$ sinken. Solche niederdimensionalen Strukturen finden sich in vielen Aufgaben, meist sind diese aber nicht so offensichtlich.

Künstliche neuronale Netze sind unter Umständen in der Lage, diese Zusammenhänge automatisch zu *erlernen* und können dann sehr effiziente Approximationen liefern. Wir verweisen auf die Literatur [12]. $\blacklozenge$

11.5.3 Trainieren von künstlichen neuronalen Netzen

In diesem Abschnitt beschreiben wir die Suche nach Parametern $(\mathbf{W}, \mathbf{b}) \in \mathcal{P}_{\mathcal{N}}$, die in einer gegebenen Netzwerkarchitektur $\mathcal{N}(\sigma; n_0, \dots, n_L)$ die optimale Realisierung $f[\mathbf{W}, \mathbf{b}] \in \mathcal{F}_{\mathcal{N}}$ darstellen, um eine gegebene Funktion $f: \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_L}$ auf einer Menge $X \subset \mathbb{R}^{n_0}$ zu approximieren, d.h., $f[\mathbf{W}, \mathbf{b}] \approx f$. Hierzu wählen wir eine Menge von Punkten $x_i \in X$ sowie Werten $y_i = f(x_i)$ für $i = 1, \dots, N_{tr}$. Diese Menge nennen wir die *Trainingsdaten*

$$\mathcal{T} := \{(x_1, y_1), \dots, (x_{N_{tr}}, y_{N_{tr}})\}.$$

Die Interpolationsaufgabe

$$f[\mathbf{W}, \mathbf{b}](x_i) = y_i, \quad i = 1, \dots, N_{tr}, \quad (11.20)$$

werden wir nicht lösen können. Zunächst ist N_{tr} in der Regel sehr groß. Der große Unterschied zur Polynominterpolation ist, dass die unbekannten Koeffizienten, die Gewichte und der Bias in nichtlinearer Form vorkommen und dass es keine Basis der Menge der zur Interpolation zur Verfügung stehenden Funktionen gibt. Im Sinne der Bestapproximation (siehe Abschn. 4.5) werden wir daher ein Funktional (also eine Abbildung mit Bildraum $\mathbb{R}$) definieren, welches das Erfüllen der Interpolationsaufgabe (11.20) misst

$$J(\mathbf{W}, \mathbf{b}) := \frac{1}{N_{tr}} \sum_{i=1}^{N_{tr}} \|f[\mathbf{W}, \mathbf{b}](x_i) - y_i\|^2, \quad (11.21)$$

und versuchen, das Minimum von diesem Funktional zu bestimmen, also

$$(\mathbf{W}_{\text{opt}}, \mathbf{b}_{\text{opt}}) = \arg \min_{\mathbf{W}, \mathbf{b} \in \mathcal{P}} J(\mathbf{W}, \mathbf{b}). \quad (11.22)$$

Das Funktional $J(\cdot)$ wird oft die *loss-Funktion* also *Verlust-Funktion* genannt.

Dieses Optimierungsproblem können wir im Allgemeinen nicht exakt lösen. Stattdessen müssen wir uns einem Minimum mit einem approximativen Verfahren, wie dem Gradientenverfahren aus Abschn. 11.1 annähern. Wir geben hier Algorithmus 11.1 im Kontext der künstlichen neuronalen Netze noch einmal an

Algorithmus 11.7: Gradientenverfahren zum Trainieren künstlicher neuronaler Netze

Eingabe : Es sei $J : \mathcal{P}_{\mathcal{N}} \rightarrow \mathbb{R}$ eine stetig differenzierbare Verlustfunktion und $p^0 := (\mathbf{W}^0, \mathbf{b}^0) \in \mathcal{P}$ ein Startwert.

```

1 Für  $k = 1, 2, \dots$  tue
2   Wenn  $\nabla J(p^{k-1}) = 0$  dann
3     Abbruch
4   Bestimme Abstiegsrichtung  $d^k = -\nabla J(p^{k-1}) / \|\nabla J(p^{k-1})\|$ 
5   Wähle eine Schrittweite  $s_k \in \mathbb{R}$ 
6   Berechne neue Approximation  $p^k = p^{k-1} + s_k d^k$  mit  $J(p^k) < J(p^{k-1})$ 
Ergebnis : Minimierer  $p^k = (\mathbf{W}^k, \mathbf{b}^k) \in \mathcal{P}$  von  $J(x)$ .
```

Die Schrittweite s_k wird im Bereich der KNN üblicherweise mit learning rate bezeichnet. Die Anwendung des Verfahrens auf künstliche neuronale Netze bringt einige Probleme mit sich. Je nach gewählter Aktivierungsfunktion ist die Zielfunktion $J(\cdot)$ gar nicht überall differenzierbar, dies ist z. B. bei der ReLU Funktion der Fall. Künstliche neuronale Netze sind oft stark überparametrisiert, d. h., die Anzahl der freien Parameter kann sehr groß sein. Die Zielfunktion ist darüber hinaus nicht konvex und hat üblicherweise sehr viele lokale Minima. Wir müssen also davon ausgehen, dass es kaum möglich sein wird, ein globales Minimum zu finden. Für eine leistungsfähige Optimierung müssen zahlreiche Heuristiken angewendet werden, um robust ein gutes Minimum zu finden und nicht im erstbesten lokalen Minimum zu verharren. Schließlich kommt hinzu, dass die Auswertung von Zielfunktion und deren Gradient sehr aufwändig ist, wenn die Anzahl der Datenpunkte groß ist. Daher wird das sogenannte stochastische Gradientenverfahren angewandt, wo aus nicht der komplette Gradientenvektor ausgewertet wird, sondern nur einzelne – zufällig gewählte – Komponenten. Dies reduziert den Rechenaufwand erheblich und führt trotzdem oft zu zufriedenstellenden Ergebnissen. Die Arbeit [37] gibt einen guten Überblick über die Forschung auf diesem Gebiet.

11.5.3.1 Backpropagation - Die Ableitung eines künstlichen neuronalen Netzes

Wie wir bereits wissen, ist der wichtigste Bestandteil im Gradientenverfahren die Bestimmung der Abstiegsrichtung, also des Gradienten der Verlustfunktion $J(p) = J(\mathbf{W}, \mathbf{b})$. Hierzu müssen alle Ableitungen von $J(\cdot)$ in Bezug auf alle Parameter des künstlichen neuronalen Netzes bestimmt werden, d. h. in Richtung der Gewichtsmatrizen W_l und der Bias-Vektoren b_l , jeweils für $l = 1, \dots, L$. Korollar 11.15 hat gezeigt, dass die Parame-

terzahl sehr schnell ansteigen kann. Hinzu kommt, dass die Verlustfunktion $J(\cdot)$ natürlich vom Ergebnis des neuronalen Netzes $f[\mathbf{W}, \mathbf{b}](\cdot)$ abhängt und dieses nur in algorithmischer Schreibweise gegeben ist, vergleiche Definition 11.14. Die Auswertung der Ableitung der Verlustfunktion ist der aufwändigste Teil beim Trainieren des neuronalen Netzes.

Im Folgenden benennen wir mit „ X “ eine beliebige Ableitungsrichtung, also z. B. den Eintrag einer Gewichtsmatrix $X = W_{rs}^l$ in Schicht l . Dann gilt mit (11.21) zunächst

$$\frac{dJ(\mathbf{W}, \mathbf{b})}{dX} = \frac{2}{N_{tr}} \sum_{i=1}^{N_{tr}} \left(y_i - f[\mathbf{W}, \mathbf{b}](x_i), \frac{f[\mathbf{W}, \mathbf{b}](x_i)}{dX} \right), \quad (11.23)$$

mit dem euklidischen Skalarprodukt $(\cdot, \cdot)$. Diese äußere Ableitung ist einfach aufzustellen, allerdings ist die Berechnung der Ableitungen von der Netzwerkfunktion $f[\mathbf{W}, \mathbf{b}](x)$ involvierter. Wir werden diese Ableitung in den folgenden Schritten entwickeln.

Schematisch lässt sich ein künstliches neuronales Netzwerk als eine verschachtelte Anwendung von Aktivierungsfunktionen und affin linearen Funktionen verstehen, d. h.,

$$f[\mathbf{W}, \mathbf{b}](x) = W_L(\cdots \sigma(W_2 \sigma(W_1 x + b_1) + b_2) \cdots) + b_L. \quad (11.24)$$

Ableitungen werden nach allen Gewichtsmatrizen W_l und Bias-Vektoren b_l und eventuell nach der Eingabe x benötigt. Das Grundwerkzeug zur Berechnung ist ganz einfach die Kettenregel, die wir aber effizient für genau diese Anwendung formulieren werden. In Definition 11.14 haben wir die Größen

$$z^l = A_l(x^{l-1}) := W^l x^{l-1} + b^l, \quad l = 1, \dots, L, \quad x^l = \sigma(z^l), \quad l = 1, \dots, L-1,$$

eingeführt, wobei $x^0 = x$ ist und $x^L = z^L = f[\mathbf{W}, \mathbf{b}](x)$ die gesuchte Ausgabe ist. Dann können wir das gesamte Netzwerk kompakt in der Form

$$f[\mathbf{W}, \mathbf{b}](x) = A_L \circ \underbrace{\sigma \circ A_{L-1} \circ \sigma \circ \cdots \circ \sigma \circ A_2 \circ \sigma \circ A_1(x)}_{\substack{z^2 \\ \underbrace{\quad \quad \quad}_{z^1} \\ \underbrace{\quad \quad \quad}_{=x^1} \\ x^2}}, \quad (11.25)$$

schreiben. Fassen wir weiter x^l und z^l zusammen, d. h. wir nutzen die Notation

$$\begin{aligned} x^l : \mathbb{R}^{n_{l-1}} &\rightarrow \mathbb{R}^{n_l} : & x^l(x^{l-1}) &= \sigma(W^l x^{l-1} + b^l), \quad l = 1, \dots, L-1, \\ & & x^L(x^{L-1}) &= W^L x^{L-1} + b^L, \end{aligned}$$

so ist das Netzwerk einfach als

$$f[\mathbf{W}, \mathbf{b}](x) = x^L \circ x^{L-1} \circ \cdots \circ x^1(x), \quad (11.26)$$

gegeben. Mit der wiederholten Anwendung der Kettenregel schreiben wir die Ableitung nach einer abstrakten Variable „ X “, die in Schicht l oder tiefer vorkommt, als

$$\begin{aligned} \frac{df[\mathbf{W}, \mathbf{b}]_{i_L}(x)}{dX} &= \frac{dX_{i_L}^L}{dX} \\ &= \sum_{i_{L-1}=1}^{n_{L-1}} \sum_{i_{L-2}=1}^{n_{L-2}} \cdots \sum_{i_l=1}^{n_l} \frac{dx_{i_L}^L}{dx_{i_{L-1}}^{L-1}} \frac{dx_{i_{L-1}}^{L-1}}{dx_{i_{L-2}}^{L-2}} \cdots \frac{dx_{i_{l+1}}^{l+1}}{dx_{i_l}^l} \frac{dx_{i_l}^l}{dX}. \end{aligned} \quad (11.27)$$

Bevor wir konkreter werden, vereinfachen wir die Notation weiter. Die Ableitung dx^{l+1}/dx^l entspricht gerade dem Gradienten der Funktion $x^l(x)$. D. h., wir können (11.27) schreiben als

$$\frac{df[\mathbf{W}, \mathbf{b}]_{i_L}(x)}{dX} = \sum_{i_l=1}^{n_l} (\nabla x^L \cdot \nabla x^{L-1} \cdots \nabla x^{l+1})_{i_L i_l} \frac{dx_{i_l}^l}{dX}, \quad (11.28)$$

wobei „ $\cdot$ “ das Matrix-Matrix Produkt ist. Noch kürzer schreiben wir

$$\frac{df[\mathbf{W}, \mathbf{b}]_{i_L}(x)}{dX} = \sum_{i_l=1}^{n_l} (\nabla_{x^l} x^L)_{i_L i_l} \frac{dx_{i_l}^l}{dX}, \quad (11.29)$$

wobei $\nabla_{x^l} x^L$ der Gradient der zusammengesetzten Funktion $x^L \circ \cdots \circ x^{l+1}(x^l)$ ist.

Wir beginnen nun mit der Berechnung des Gradienten einer Schicht in Bezug auf ihre Eingabe.

Satz 11.22 (Ableitung künstlicher neuronaler Netze nach der Eingabe) Es sei $\mathcal{N}(\sigma; n_0, \dots, n_L)$ ein künstliches neuronales Netz mit differenzierbarer Aktivierungsfunktion $\sigma(\cdot)$. Für $x^0 := x \in \mathbb{R}^{n_0}$ seien x^l und z^l für $l = 1, \dots, L$ die Zwischenergebnisse aus Definition 11.14. Dann gilt für $l = 1, \dots, L-1$

$$\frac{dx^l}{dx^{l-1}} = \sigma'(z^l) \star W^l, \text{ sowie } \frac{dx^L}{dx^{L-1}} = W^L.$$

Dabei ist für einen Vektor $a \in \mathbb{R}^n$ und eine Matrix $A \in \mathbb{R}^{n \times m}$ das komponentenweise Produkt $a \star A$ definiert als

$$(a \star A)_{ij} = a_i A_{ij}, \quad i = 1, \dots, n, \quad j = 1, \dots, m. \quad (11.30)$$

Es gilt

$$\nabla x^l = \frac{dx^l}{dx^{l-1}} \in \mathbb{R}^{n_l \times n_{l-1}}.$$

Beweis Das Ergebnis kann komponentenweise nachgerechnet werden. Es sei $l < L$. Dann ist für $i = 1, \dots, n_l$ und $j = 1, \dots, n_{l-1}$ mit der Kettenregel angewendet auf $x_i^l = \sigma(z_i^l)$

$$\frac{d}{dx_j^{l-1}} \sigma(z_i^l) = \sigma'(z_i^l) \frac{d}{dx_j^{l-1}} z_i^l = \sigma'(z_i^l) \sum_{i_{l-1}=1}^{n_{l-1}} \frac{d}{dx_j^{l-1}} W_{i, i_{l-1}}^l x_{i_{l-1}}^{l-1} = \sigma'(z_i^l) W_{i, j}^l.$$

Wir betrachten die Aktivierungsfunktion und ihre Ableitung weiter als eine skalare Funktion, die auf jeden Eintrag des Vektors separat angewendet wird, d. h.,

$$(\sigma'(z^l))_i = \sigma'(z_i^l), \quad i = 1, \dots, n_l.$$

Das Ergebnis kann bei Verwendung des komponentenweisen Produktes (11.30) abgelesen werden. Auch folgt sofort, dass $\nabla x^l \in \mathbb{R}^{n_l \times n_{l-1}}$. Für das letzte Element x^L gilt $x^L = z^L$, d. h., der Gradient ist die innere Ableitung, also $\nabla x^L = W^L$. $\square$

Bemerkung 11.23 (Sonderbehandlung der Ausgabeschicht) Bei der Berechnung des Gradienten haben wir die letzte Schicht x^L separat behandelt. Dies liegt daran, dass wir im Allgemeinen auf die Ausgabe nicht mehr die Aktivierungsfunktion anwenden, siehe auch Bemerkung 11.16. Es gibt durchaus auch Anwendungen, wo es wünschenswert ist, dass das Ergebnis z. B. im Intervall $[0, 1]$ liegt und dann würde die Aktivierungsfunktion auch in der Ausgabeschicht zum Einsatz kommen, d. h.

$$f[\mathbf{W}, \mathbf{b}](x) = \sigma(W^L \sigma(\dots) + \dots + b^L).$$

Der erste Faktor der Ableitung wäre dann entsprechend $\sigma'(z^L) \star W^L$ anstelle von W^L .

Mit dem mehrlagigen Perzeptron, Definition 11.14, betrachten wir hier nur einen sehr einfachen Fall von künstlichen neuronalen Netzen. Bei komplexer aufgebauten Architekturen können aber die Bausteine zusammengefügt werden, die wir hier entwickeln. $\blacklozenge$

Die Ableitungen der Stufen hängen von den Ableitungen der Aktivierungsfunktion ab. Es gilt:

Satz 11.24 (Ableitung der Aktivierungsfunktionen) Für die Ableitungen der Sigmoid Funktion sowie des hyperbolischen Tangens gilt

$$\sigma'(s) = \sigma(s)(1 - \sigma(s)), \quad \tanh'(s) = 1 - \tanh^2(s).$$

Die Ableitung der ReLU Aktivierung ist nur stückweise definiert:

$$\text{ReLU}'(s) = \begin{cases} 0 & s < 0 \\ 1 & s \geq 0 \end{cases}$$

Beweis Nachrechnen. $\square$

Mit dem bisher gezeigten können wir bereits die Ableitung des neuronalen Netzes nach der Eingabe berechnen.

Korollar 11.25 (Ableitung des neuronalen Netzes nach seiner Eingabe) Mit den Voraussetzungen von Satz 11.22 gilt

$$\nabla f[\mathbf{W}, \mathbf{b}](x) = W^L \cdot (\sigma'(z^{L-1}) \star W^{L-1}) \cdots (\sigma'(z^1) \star W^1).$$

sowie für den Gradienten in Bezug auf die Ausgabe von Schicht l :

$$\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x) = W^L \cdot (\sigma'(z^{L-1}) \star W^{L-1}) \cdots (\sigma'(z^{l+1}) \star W^{l+1}).$$

Es ist $\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x) \in \mathbb{R}^{n_L \times n_l}$.

Der nächste Schritt ist die Berechnung der Ableitungen in Richtung der Gewichte W^l und b^l in Schicht l , siehe die abstrakte Form der Ableitung (11.27). Hierzu genügt es, wenn wir nur l -te Schicht des Netzes betrachten:

Satz 11.26 (Ableitung einer Schicht nach den Gewichten) Für $l = 1, \dots, L$, sei

$$x^l : \mathbb{R}^{n_{l-1}} \rightarrow \mathbb{R}^{n_l}, \quad x^l(x) = \begin{cases} W^l x^{l-1} + b^l & l = L, \\ \sigma(W^l x^{l-1} + b^l) & l < L, \end{cases}$$

mit $W^l \in \mathbb{R}^{n_l \times n_{l-1}}$, $b^l \in \mathbb{R}^{n_l}$ und einer differenzierbaren Aktivierungsfunktion $\sigma(\cdot)$. Dann gilt für $l = 1, \dots, L-1$

$$\begin{aligned} \frac{dx_i^l}{dW_{rs}^l} &= \delta_{ir} \sigma'(z_i^l) x_s^{l-1}, & i, r &= 1, \dots, n_l, \quad s = 1, \dots, n_{l-1}, \\ \frac{dx_i^l}{db_r} &= \delta_{ir} \sigma'(z_i^l), & i, r &= 1, \dots, n_l \end{aligned} \quad (11.31)$$

sowie

$$\begin{aligned} \frac{dx_i^L}{dW_{rs}^L} &= \delta_{ir} x_s^{L-1}, & i, r &= 1, \dots, n_L, \quad s = 1, \dots, n_{L-1}, \\ \frac{dx_i^L}{db_r} &= \delta_{ir}, & i, r &= 1, \dots, n_L. \end{aligned} \quad (11.32)$$

Beweis Wir rechnen die Ableitung komponentenweise nach. Für $i, r = 1, \dots, n_l$ und $s = 1, \dots, n_{l-1}$ gilt

$$\frac{dx_i^l}{dW_{rs}^l} = \frac{d}{dW_{rs}^l} \sigma(W^l x^{l-1} + b^l)_i = \sigma'(z_i^l) \sum_{j=1}^{n_{l-1}} \frac{d}{dW_{rs}^l} W_{ij}^l x_j^{l-1} = \delta_{ir} \sigma'(z_i^l) x_s^{l-1}.$$

Die meisten Ableitungen, genauer gesagt alle Ableitungen für $i \neq r$, verschwinden. Anstelle von $n_l^2 n_{l-1}$ müssen nur $n_l n_{l-1}$ Ableitungen berechnet werden. Entsprechend gilt

$$\frac{dx_i^l}{db_r} = \frac{d}{db_r} \sigma(Wx^{l-1} + b^l)_i = \sigma'(z_i^l) \frac{d}{db_r} b_i^l = \delta_{ir} \sigma'(z_i^l).$$

Auch hier verbleiben nur n_l der eigentlich n_l^2 Ableitungen, die nicht verschwinden. $\square$

Hiermit können wir nun die Ableitungen des künstlichen neuronalen Netzes in Richtung aller Parameter angeben.

Korollar 11.27 (Ableitung des neuronalen Netzes nach den Parametern) Mit den Voraussetzungen von Satz 11.22 gilt für $l = 1, \dots, L - 1$, dass

$$\begin{aligned} \frac{df_i[\mathbf{W}, \mathbf{b}](x)}{dW_{rs}^l} &= (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x))_{ir} \sigma'(z_r^l) x_s^{l-1}, \\ \frac{df_i[\mathbf{W}, \mathbf{b}](x)}{db_r^l} &= (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x))_{ir} \sigma'(z_r^l), \end{aligned}$$

sowie

$$\frac{df_i[\mathbf{W}, \mathbf{b}](x)}{dW_{rs}^L} = \delta_{ir} x_s^{L-1}, \quad \frac{df_i[\mathbf{W}, \mathbf{b}](x)}{db_r^L} = \delta_{ir}.$$

Beweis Einsetzen der Beziehungen aus Satz 11.26 in (11.29) gibt

$$\begin{aligned} \frac{df_i[\mathbf{W}, \mathbf{b}](x)}{dW_{rs}^l} &= \sum_{j=1}^{n_l} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x))_{ij} \delta_{jr} \sigma'(z_j^l) x_s^{l-1} \\ \frac{df_i[\mathbf{W}, \mathbf{b}](x)}{db_r^l} &= \sum_{j=1}^{n_l} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x))_{ij} \delta_{jr} \sigma'(z_j^l), \end{aligned}$$

d.h., es bleibt in der Summe jeweils nur der Term für $j = r$ übrig. Für $l = L$ vereinfacht sich die Rechnung entsprechend. $\square$

Die Ableitung nach dem Bias kann kompakt als Gradient geschrieben werden, d.h. $\nabla_{b^l} f[\mathbf{W}, \mathbf{b}](x) = \nabla_{x^l} x^L \star \sigma'(z^l)$, wobei wir entsprechend zu (11.30) für $A \in \mathbb{R}^{n \times m}$ und $a \in \mathbb{R}^m$ die Notation

$$(A \star a)_{ij} = A_{ij} a_j$$

verwenden. Die Ableitungen nach den Gewichten W_{rs}^l bilden hingegen einen Tensor dritter Stufe. Wir benötigen die Ableitungen jedoch ausschließlich zur Berechnung des Gradienten der Verlustfunktion und brauchen hierfür keine weitere kompakte Schreibweise.

Satz 11.28 (Gradient der Verlustfunktion) Mit den Voraussetzungen von Satz 11.26 gilt für $l = 1, \dots, L - 1$

$$\begin{aligned}\nabla_{W^l} J(\mathbf{W}, \mathbf{b}) &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} \left(\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}})^T d_{i_{tr}} \circ \sigma'(z_{i_{tr}}^l) \right) (x_{i_{tr}}^{l-1})^T, \\ \nabla_{b^l} J(\mathbf{W}, \mathbf{b}) &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} \nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}})^T d_{i_{tr}} \circ \sigma'(z_{i_{tr}}^l),\end{aligned}$$

sowie

$$\begin{aligned}\nabla_{W^L} J(\mathbf{W}, \mathbf{b}) &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} d_{i_{tr}} (x_{i_{tr}}^{L-1})^T, \\ \nabla_{b^L} J(\mathbf{W}, \mathbf{b}) &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} d_{i_{tr}}.\end{aligned}$$

Dabei ist

$$d_{i_{tr}} = y_{i_{tr}} - f[\mathbf{W}, \mathbf{b}](x_{i_{tr}}), \quad i_{tr} = 1, \dots, N_{tr},$$

der Defekt der Trainingsdaten und $A \circ B \in \mathbb{R}^{n \times m}$ das Hadamard-Produkt zweier Matrizen $A, B \in \mathbb{R}^{n \times m}$ (bzw. Vektoren), gegeben durch

$$(A \circ B)_{ij} = A_{ij} B_{ij}, \quad i = 1, \dots, n, \quad j = 1, \dots, m.$$

Beweis Wir kombinieren (11.23) mit den Ergebnissen aus Korollar 11.27 und fassen die auftretenden Summen in kompakter Schreibweise zusammen

$$\begin{aligned}\frac{dJ(\mathbf{W}, \mathbf{b})}{dW_{rs}^l} &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} \sum_{i=1}^{n_L} d_{i_{tr},i} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}}))_{i_r} \sigma'(z_{i_{tr},r}^l) x_{i_{tr},s}^{l-1} \\ &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}})^T d_{i_{tr}})_r \sigma'(z_{i_{tr},r}^l) x_{i_{tr},s}^{l-1} \\ &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}})^T d_{i_{tr}} \circ \sigma'(z_{i_{tr}}^l))_r x_{i_{tr},s}^{l-1},\end{aligned}$$

und entsprechend

$$\begin{aligned}\frac{dJ(\mathbf{W}, \mathbf{b})}{db_r^l} &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} \sum_{i=1}^{n_L} d_{i_{tr},i} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}}))_{i_r} \sigma'(z_{i_{tr},r}^l) \\ &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}})^T d_{i_{tr}})_r \sigma'(z_{i_{tr},r}^l) \\ &= \frac{2}{N_{tr}} \sum_{i_{tr}=1}^{N_{tr}} (\nabla_{x^l} f[\mathbf{W}, \mathbf{b}](x_{i_{tr}})^T d_{i_{tr}} \circ \sigma'(z_{i_{tr}}^l))_r.\end{aligned}$$

□

Mit Satz 11.28 haben wir die Herleitung des Gradienten der Verlustfunktion nach allen Parametern abgeschlossen. Es bleibt noch die effiziente algorithmische Umsetzung. Der maßgebliche Teil des Aufwandes liegt in der Berechnung des Gradienten nach der l -ten Schicht, d.h. $\nabla_{x^l} x^L$, der für jedes Element der Trainingsdaten berechnet werden muss. In Korollar 11.25 haben wir hierfür Beziehung

$$\nabla_{x^l} x^L = W^L \cdot (\sigma'(z^{L-1}) \star W^{L-1}) \dots (\sigma'(z^{l+1}) \star W^{l+1}),$$

hergeleitet. Die Ableitung nach Schicht l benötigt das Produkt der Gewichtsmatrizen $W^L, W^{L-1}, \dots, W^{l+1}$ sowie die linearen Zwischenergebnisse $z^{L-1}, z^{L-2}, \dots, z^{l+1}$. Um diese zu berechnen, muss das Netzwerk allerdings von der Eingabe $x = x^0$ an durchlaufen werden. Damit diese Werte nicht für jede Ableitung neu berechnet werden müssen, erfolgt das Aufstellen des Gradienten in zwei Schritten: zunächst wird das Netzwerk für eine Eingabe $x = x_i$ ausgewertet und alle Zwischenergebnisse $z_i^1, \dots, z_i^{L-1}$ werden gespeichert. Dieser Schritt wird der *Vorwärts-Modus* genannt. Im Anschluss werden die Gradienten, beginnend bei Schicht L berechnet. Dieser zweite Schritt heißt entsprechend der *Rückwärts-Modus*. Der gesamte Algorithmus wird *Backpropagation* genannt.

Algorithmus 11.8: Backpropagation: Auswertung der Ableitungen des künstlicher neuronalen Netzes und der Verlustfunktion

```

Eingabe : Künstliches neuronales Netzwerk  $\mathcal{N}(\sigma; n_0, \dots, n_L)$ , Parameter  $(\mathbf{W}, \mathbf{b}) \in \mathcal{P}$  und
  Trainingsdaten  $(x_{i_{tr}}, y_{i_{tr}}) \in \mathbb{R}^{n_0} \times \mathbb{R}^{n_L}$ 
/* Initialisierung                                     */
1  Für  $l = 1, \dots, L$  tue
2     $\nabla_{W^l} J = 0$ 
3     $\nabla_{b^l} J = 0$ 
4  Für  $i_{tr} = 1, 2, \dots, N_{tr}$  tue
  /* Vorwärts-Modus                                     */
5     $x^0 = x_{i_{tr}}$ 
6    Für  $l = 1, \dots, L-1$  tue
7      Speichern von  $z_{i_{tr}}^l = W^l x_{i_{tr}}^{l-1} + b^l$ 
8      Speichern von  $x_{i_{tr}}^l = \sigma(z_{i_{tr}}^l)$ 
9     $x^L = W^L x^{L-1} + b^L$ 
10    $d = y_{i_{tr}} - x^L$ 
  /* Rückwärts-Modus                                     */
11    $G_{W^L} = W^L$ 
12   Für  $l = L-1, \dots, 1$  tue
13      $G_{W^l} = G_{W^{l+1}} \cdot (\sigma'(z_{i_{tr}}^l) \star W^{l+1})$ 
14     Berechne Zwischenergebnis  $X = (G_{W^l})^T \circ \sigma'(z_{i_{tr}}^l)$ 
15      $\nabla_{W^l} J = \nabla_{W^l} J + \frac{2}{N_{tr}} X (x_{i_{tr}}^{l-1})^T$ 
16      $\nabla_{b^l} J = \nabla_{b^l} J + \frac{2}{N_{tr}} X$ 
Ergebnis : Gradienten  $\nabla_{W^l} J$  und  $\nabla_{b^l} J$ 

```

Satz 11.29 (Backpropagation) Seien N_{tr} die Menge der Datenpunkte, $n := \max n_0, \dots, n_L$ und L die Anzahl der Schichten. Der Backpropagation-Algorithmus berechnet die Gradienten der Verlustfunktion in Bezug auf die Gewichte. Für die Architektur $\mathcal{N}(\sigma; n_0, \dots, n_L)$ können die Gradienten mit dem Aufwand

$$O(N_{tr} n^3 L)$$

berechnet werden.

Beweis Die Berechnung der Gradienten der Verlustfunktion folgt aus den Sätzen 11.22, 11.24, 11.26 und 11.28. Die Aufwandsabschätzung folgt durch Abzählen der Operationen, insbesondere der Matrix-Matrix Multiplikationen. $\square$

Bemerkung 11.30 (Effiziente Implementierungen) Effiziente Implementierungen von neuronalen Netzen wie *PyTorch* [69] oder *Tensorflow* [1] erreichen ihre Leistungsfähigkeit durch extreme Optimierung der grundlegenden Matrixoperationen. Wir haben Algorithmus 11.8 als Schleife über alle Trainingsdatenpaare formuliert. Viel effizienter können die Netzwerke und Ableitungen jedoch ausgerechnet werden, wenn jeweils alle Daten gleichzeitig ausgewertet werden. Dies erfordert jedoch den Umgang mit Tensoren höherer Stufe, d. h. mit multilinearen Abbildungen $X_{i,j,k}$ mit drei oder noch mehr Indizes.

Darüber hinaus nutzen Bibliotheken wie *PyTorch* oder *Tensorflow* spezielle Hardware wie Grafikkarten (GPU's) oder Beschleunigerkarten wie TPU's (Tensor-Processing-Units), die genau für diese Aufgaben optimiert sind. $\blacklozenge$

Mit den Gradienten der Verlustfunktion nach den Gewichten und dem Bias kann das künstliche neuronale Netzwerk *trainiert* werden, d. h., wir können mit dem Gradientenverfahren die optimalen Parameter $(\mathbf{W}, \mathbf{b}) \in \mathcal{P}$ suchen, welche die Verlustfunktion minimieren.

Beispiel 11.31 (Funktionsapproximation mit künstlichen neuronalen Netzen)

Wir betrachten die Funktion

$$f(x) = x(1 + \exp(x)) + 10 \sin(3 + \log(x^2 + 1)), \quad (11.33)$$

die wir bereits bei der Suche nach Nullstellen analysiert haben und versuchen, diese durch ein einfaches künstliches neuronales Netzwerk darzustellen. Wir starten mit der Architektur $\mathcal{N}(\sigma; 1, 8, 1)$, d. h. die Menge der Funktionen

$$f[\mathbf{W}, \mathbf{b}](x) = W_2 \tanh(W_1 x + b_1) + b_2, \quad W_1 \in \mathbb{R}^{8 \times 1}, \quad W_2 \in \mathbb{R}^{1 \times 8}, \quad b_1 \in \mathbb{R}^8, \quad b_2 \in \mathbb{R}.$$

Aus dem Intervall $I = [-10, 2]$ wählen wir $N_{tr} = 20$ zufällige Punkte $x_1, \dots, x_{N_{tr}}$ und zugehörige Werte $y_i = f(x_i)$. In Abb. 11.12 zeigen wir links den Verlauf Verlustfunktion in 10000 Schritten der Optimierung. Rechts geben wir den Verlauf der Funktion $f(x)$

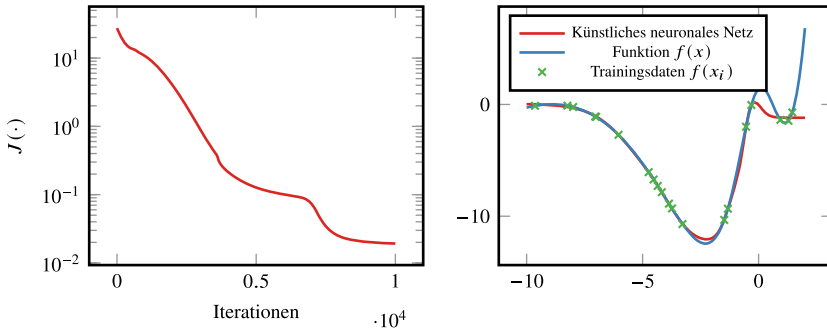


Abb. 11.12 Approximation einer Funktion mit künstlichen neuronalen Netzen. Links ist der Verlauf des Trainings, also das Minimieren der Verlustfunktion mit dem Gradientenverfahren gezeigt. Rechts die Approximationseigenschaft des Netzes sowie die 20 Trainingspunkte

sowie das trainierte neuronale Netzwerk $f[\mathbf{W}, \mathbf{b}](x)$ an. Die meisten Trainingspunkte werden vom Netzwerk sehr gut approximiert. Jenseits der Trainingsdaten, vor allem im Bereich $[1, 2]$ stellt das Netzwerk jedoch keinerlei Approximation an die Funktion $f(x)$ dar. Dies ist auch nicht zu erwarten, da das Netzwerk beim Trainieren ja nicht die Funktion als solche kennengelernt hat, sondern nur einige diskrete Werte.

Verteilen wir mehr Trainingspunkte im Intervall, so verbessert sich die Approximation. In Abb. 11.13 zeigen wir links die Approximation für $N_{tr} = 100$ zufällige Punkte. Das von uns gewählte Netzwerk ist sehr klein und verfügt nur über 25 freie Parameter. In der rechten Abb. 11.13 zeigen wir die Approximation für die gleichen $N_{tr} = 100$ zufälligen Trainingspunkte, jedoch einem weitaus größeren Netzwerk mit der Architektur $\mathcal{N}(\tanh; 1, 16, 16, 16, 1)$. Dieses Netzwerk kann die Funktion in allen Punkten sehr gut darstellen. ◀

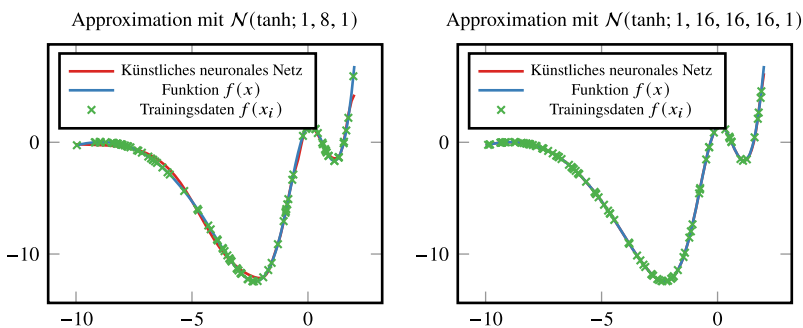


Abb. 11.13 Approximation einer Funktion mit künstlichen neuronalen Netzen. Links: kleines Netzwerk $\mathcal{N}(\tanh; 1, 8, 1)$ mit $N_{tr} = 100$ Trainingsdaten und rechts: großes Netzwerk $\mathcal{N}(\tanh; 1, 16, 16, 16, 1)$ wieder mit $N_{tr} = 100$ Trainingsdaten

11.5.3.2 Überanpassung, Testen und Validieren und Generalisierung

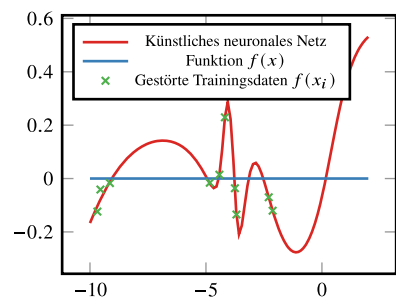
Künstliche Neuronale Netze werden oft mit einer sehr großen Zahl von freien Parametern entworfen. Dies macht sie sehr leistungsfähig und flexibel, die Überparametrisierung birgt jedoch die Gefahr einer *Überanpassung*, *overfitting* genannt: Die Datenpaare werden gut abgebildet $y_i \approx f[\mathbf{W}_{\text{opt}}, \mathbf{b}_{\text{opt}}](x_i)$, in anderen Punkten hat das Netzwerk jedoch nicht den Funktionsverlauf gelernt (siehe Abb. 11.12 rechts). Hierzu betrachten wir die Approximation der Funktion $f(x) = 0$, gehen aber aus, dass zu Trainings-Stützstellen $x_i \in [-10, 2]$ die Funktion nur Fehlerhaft bekannt ist $y_i = 0 + N(0.1)$. Dabei ist $N(0.1)$ eine normalverteilte Zufallszahl mit Standardabweichung 0.1. Abb. 11.14 zeigt die Trainingsdaten und die Approximation mit einem (sehr reichhaltig parametrisierten) Netzwerk.

Es gibt verschiedene Strategien, *overfitting* zu minimieren. Das übliche Vorgehen ist es, die Daten aufzuteilen in *Trainingsdaten* und *Validierungsdaten*: Die *Trainingsdaten* (vielleicht 70 % der Gesamtdaten) dienen dazu, dass künstliche neuronale Netzwerk zu trainieren. D. h., sie sind die Basis zum Auswerten des Gradienten bei der Optimierung.

Während der Optimierung wird die Verlustfunktion nicht nur auf Basis der Trainingsdaten ausgerechnet sondern auch auf Basis der *Validierungsdaten*. Diese werden jedoch nie zur Berechnung des Gradientens beim Trainieren verwendet, so dass das neuronale Netz die Daten nicht kennt. Wir erwarten, dass das neuronale Netzwerk auch auf diesen Validierungsdaten gute Vorhersagen liefert und dass die Verlustfunktion der Validierungsdaten im Laufe der Optimierung abnimmt. Kommt es zu einer Diskrepanz, d. h. erreichen wir einen Punkt, an dem die Verlustfunktion der Trainingsdaten weiter reduziert wird, die Verlustfunktion der Validierungsdaten aber stagniert oder sogar wieder zunimmt, so ist dies ein Anzeichen für *overfitting* und die Optimierung sollte beendet werden.

In Abb. 11.15 zeigen wir links den Verlauf der Verlustfunktion angewendet auf $N_{tr} = 30$ Trainingsdaten und auf $N_{va} = 10$ Validierungsdaten. Auf der rechten Seite zeigen wir einmal das Netzwerk, welches mit 10 000 Schritten trainiert wurde und dann das Netzwerk nach Abbruch der Minimierung bei 2500 Schritten, wo Trainings- und Validierungs-Loss noch gut übereinstimmen. Die Trainingspunkte werden nicht mehr so gut approximiert, aber der gesamte Verlauf des Netzes ist näher an der zugrundeliegenden Funktion $f(x) = 0$.

Abb. 11.14 Overfitting: das Netzwerk approximiert die $N_{tr} = 10$ Trainingsdaten sehr gut, gibt ansonsten aber nicht den Verlauf der Funktion $f(x) = 0$ wieder



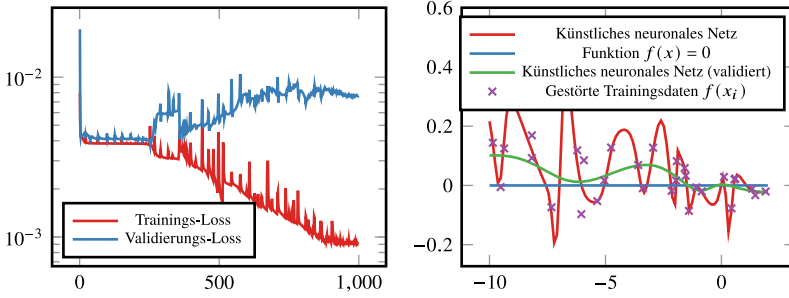


Abb. 11.15 Links: Optimierung und Verlauf der Verlustfunktionen angewendet auf Trainings- und Validierungsdaten. Rechts: Ergebnis der künstlichen Neuronalen Netzwerke nach 10 000 Optimierungsschritten und bei vorzeitigem Abbruch nach 2500 Schritten, bevor Trainings- und Validierungs-Loss auseinanderlaufen

Beispiel 11.32 (Approximation hochdimensionaler Funktionen)

Wir betrachten die Funktion $f : \mathbb{R}^d \rightarrow \mathbb{R}$, gegeben als

$$f(x) = \exp(-\|x\|_2^2)$$

auf dem Hyperwürfel $Q = [-1, 1]^d$. Wir approximieren die Funktion mit einem künstlichen neuronalen Netz $\mathcal{N}$ durch Minimierung der Verlustfunktion

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{N_{tr}} \sum_{i=1}^{N_{tr}} |f(x_i) - f[\mathbf{W}, \mathbf{b}](x_i)|^2$$

für N_{tr} gleichverteilt zufällige Trainingswerte $x_1, \dots, x_{N_{tr}} \in Q$. Die Netzwerk-Architektur ist sehr einfach gehalten und besteht aus L Schichten mit jeweils n künstlichen Neuronen, also

$$\mathcal{N} = \mathcal{N}(\sigma; d, \underbrace{n, \dots, n}_{L \text{ Schichten}}, 1).$$

Das Netzwerk hat somit $nd + (L - 1)n^2 + n = O(Ln^2)$ Gewichte. ◀

Eine weitere Technik zum Vermeiden von *overfitting* und zur Verbesserung des Trainings-Prozesses ist die *Regularisierung* der Verlustfunktion. Im einfachsten Fall bedeutet dies, dass die Verlustfunktion (11.21) um einen quadratischen Term erweitert wird

$$J_\gamma(\mathbf{W}, \mathbf{b}) = \frac{1}{N_{tr}} \sum_{i=1}^{N_{tr}} (f[\mathbf{W}, \mathbf{b}](x_i) - y_i)^2 + \gamma \sum_{l=1}^L \|\mathbf{W}^l\|_F^2 + \|b^l\|_2^2, \quad (11.34)$$

mit dem Regularisierungsparameter $\gamma > 0$. In der mathematischen Optimierung wird diese Methode Tikhonov Regularisierung genannt.

11.6 Exkurs: Eigenwertbestimmung mit künstlichen neuronalen Netzen

Wir werden ein künstliches neuronales Netzwerk entwickeln, welches als Eingabe eine symmetrische Matrix erhält und als Ausgabe alle Eigenwerte der Matrix zurückgeliefert. Als Grundmenge betrachten wir symmetrische Matrizen, die nach folgendem Prinzip aufgebaut sind

$$X_d = \{A \in \mathbb{R}^{d \times d}, A = B + B^T, B \in \mathcal{U}(0, 1)^{d \times d}\}, \quad (11.35)$$

wobei $\mathcal{U}(0, 1)$ gleichverteilte Zahlen im Intervall $[0, 1]$ sind. In Kap. 5 haben wir die Konditionierung des Eigenwertproblems untersucht und schon festgestellt, dass die Eigenwerte stetig von den Matrixeinträgen abhängen, siehe Bemerkung 5.6. Das Problem sollte sich daher gut zur Approximation mit neuronalen Netzen eignen. Auf der anderen Seite ist die Eigenwertbestimmung hinreichend schwer. Wir haben in Kap. 5 verschiedene Algorithmen kennengelernt. Direkte Verfahren eignen sich aufgrund der mangelnden Stabilität nicht und die iterativen Methoden bringen alle einen hohen Aufwand mit sich.

Zunächst erstellen wir die Trainings- und Validierungsdaten. Hierzu wählen wir $N_{tr} \in \mathbb{N}$ und $N_{va} \in \mathbb{N}$ zufällige Matrizen $A_1^{tr}, \dots, A_{N_{tr}}^{tr} \in X_d$ sowie $A_1^{va}, \dots, A_{N_{va}}^{va} \in X_d$ und bestimmen jeweils die Eigenwerte $\Lambda_i^{tr}, \Lambda_i^{va} \in \mathbb{R}^d$. Da die Matrizen symmetrisch sind, sind alle Eigenwerte reell und wir ordnen sie jeweils der Größe nach sortiert an, d. h.

$$\Lambda_i^{tr} = (\lambda_{i,1}^{tr}, \dots, \lambda_{i,d}^{tr}), \text{ mit } \lambda_{i,1}^{tr} \leq \lambda_{i,2}^{tr} \leq \dots \leq \lambda_{i,d}^{tr},$$

sowie

$$\Lambda_i^{va} = (\lambda_{i,1}^{va}, \dots, \lambda_{i,d}^{va}), \text{ mit } \lambda_{i,1}^{va} \leq \lambda_{i,2}^{va} \leq \dots \leq \lambda_{i,d}^{va}.$$

Das künstliche neuronale Netzwerk erhält die Matrixelemente als Eingaben und die Eigenwerte als Ausgabe. Somit gilt $n_0 = d^2$ und $n_L = d$. In den inneren Schichten wählen wir jeweils eine feste Zahl von $n_l = n$ Neuronen. Als Aktivierungsfunktion wird der Tangens hyperbolicus verwendet. Bei zunächst noch freier Anzahl von Schichten ist die Architektur gegeben als

$$\mathcal{N}(\tanh; d^2, \underbrace{n, \dots, n}_{L \text{ Schichten}}, d)$$

Das neuronale Netzwerk hat also $Ln + d$ künstliche Neuronen und die Anzahl der freien Gewichte und Bias beträgt

$$(nd^2 + (L - 1)n^2 + dn) + (Ln + d).$$

Angenommen, durch $\mathbf{A} = (A_1, \dots, A_{N_{tr}})$ und $\mathbf{\Lambda} = (\Lambda_1, \dots, \Lambda_{N_{tr}})$ seien die Trainingsdaten gegeben, dann wählen wir die Verlustfunktion

$$l(\mathbf{A}, \mathbf{\Lambda}) = \frac{1}{N_{tr}} \sum_{i=1}^{N_{tr}} \|f[\mathbf{W}, \mathbf{b}](A_i) - \Lambda_i\|_2^2. \quad (11.36)$$

11.6.1 Training des Netzes

Wir haben uns für einen grundsätzlichen Aufbau des neuronalen Netzes entschieden, müssen nun jedoch noch die Wahl konkretisieren. Insbesondere gilt es, die Anzahl der Schichten L sowie deren Breite n zu bestimmen. Diese Wahl kann oft nur experimentell erfolgen. Wir starten dazu mit einem einfachen Beispiel und $d = 8$, d.h. symmetrischen Matrizen $A \in \mathbb{R}^{8 \times 8}$. Weiter wählen wir $L = 4$ Schichten mit jeweils $n = 64$ künstlichen Neuronen. Zum Training werden $N_{tr} = 5000$ zufällige Matrizen bestimmt. Zur Validierung wählen wir $N_{va} = 1250$ weitere zufällige Matrizen. In Abb. 11.16 links zeigen wir den Verlauf der Verlustfunktion für Trainings- und Validierungsdaten. Hier zeigt sich klar *overfitting*: das Netzwerk gibt auf den Trainingsdaten weitaus bessere Vorhersagen als auf den Validierungsdaten. Gleichzeitig sieht man, dass die Optimierung noch nicht abgeschlossen ist, die Kurve der Trainingsdaten ist nach wie vor fallend.

Obwohl wir nicht davon ausgehen, dass das resultierende Netzwerk gute Vorhersagen liefert, geben wir in Tab. 11.4 für zwei zufällig gewählte Matrizen (die nicht in den Trainingsdaten enthalten sind) die Eigenwerte Λ und die vom Netzwerk approximierten Eigenwerte $\Lambda_{\mathcal{N}}$ jeweils auf drei Stellen gerundet an.

Das Netzwerk trainiert zu spezifisch auf den gegebenen Daten. Eine Möglichkeit, *overfitting* zu vermeiden ist die Hinzunahme eines Regularisierungsterms in die Verlustfunktion (11.36). Die einfachste Form der Regularisierung ist die *Tikhonov Regularisierung*, welche die zu bestimmenden Parameter als quadratische Terme hinzufügt, d.h.

$$l(\mathbf{A}, \mathbf{\Lambda}) = \frac{1}{N_{tr}} \sum_{i=1}^{N_{tr}} \|f[\mathbf{W}, \mathbf{b}](A_i) - \Lambda_i\|_2^2 + \gamma \sum_{l=1}^L (\|\mathbf{W}^l\|_F^2 + \|\mathbf{b}^l\|_2^2), \quad (11.37)$$

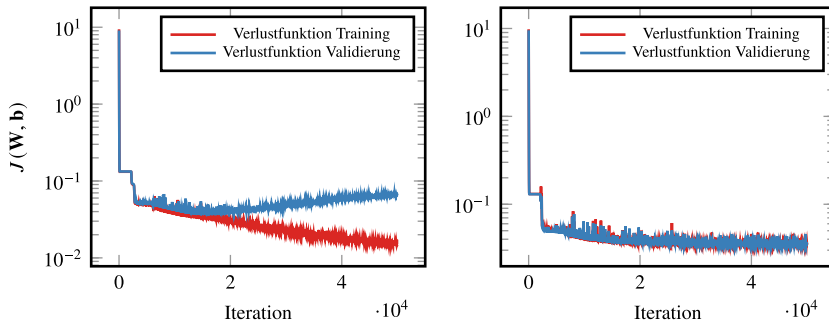


Abb. 11.16 Links: Verlauf der Verlustfunktion für Trainings- und Validierungsdaten zur Eigenwertbestimmung von symmetrischen 8×8 -Matrizen. Es ist klar *overfitting* zu erkennen. Rechts: Training des gleichen Netzes mit *Regularisierung*. Die Verlustfunktion wird um den Term $\gamma \sum_l \|\mathbf{W}^l\|_F^2$ ergänzt

Tab. 11.4 Eigenwerte und Approximation des neuronalen Netzwerks für zwei zufällig gewählte Matrizen, die nicht in den Trainingsdaten enthalten sind. Der relative Fehler ist zwischen Eigenwerten λ_i und Netzwerkapproximation $\lambda_i^{\mathcal{N}}$ ist gegeben als $(\lambda_i - \lambda_i^{\mathcal{N}}) / \max_i |\lambda_i|$

Referenz	−1.945	−1.337	−0.998	−0.016	0.355	1.105	1.444	7.278
Netzwerk	−2.015	−1.040	−0.648	−0.414	−0.187	0.725	2.105	7.218
rel. Fehler	0.010	−0.041	−0.048	0.055	0.074	0.052	−0.091	0.008
Referenz	−1.631	−1.246	−0.222	0.206	0.344	1.130	1.773	8.607
Netzwerk	−1.739	−0.853	−0.361	0.120	0.445	0.955	1.680	8.734
rel. Fehler	0.013	−0.046	0.016	0.010	−0.012	0.020	0.011	−0.015

wobei $\gamma > 0$ ein kleiner Parameter ist. Wir wählen $\gamma = 10^{-4}$ und in Abb. 11.16 rechts ist der Verlauf der beiden Verlustfunktionen gezeigt. Overfitting kann so vermieden werden. Es zeigt sich jedoch auch, dass die Verlustfunktion nicht mehr entsprechend reduziert werden kann. Die Wahl des Parameters γ ist nicht einfach. Bei $\gamma = 10^{-5}$ verliert die Regularisierung bei diesem Beispiel ihre Wirkung. Die Wahl $\gamma = 10^{-3}$ führt zu einem Verlust der Approximationseigenschaft.

Tab. 11.5 zeigt die Ergebnisse für zwei Testmatrizen unter Verwendung von Regularisierung. Alle Eigenwerte können mit einem Fehler von maximal 5 % vorhergesagt werden. Dabei bezieht sich die Normierung im relativen Fehler jeweils auf den betragsgrößten Eigenwert der Matrix.

11.6.2 Einfluss der Netzwerkgröße

Die theoretische Approximationsgüte eines neuronalen Netzes ist der grundlegende Faktor der bestimmt, wie gut ein Problem überhaupt gelöst werden kann. Man nennt dies die *Expressivität des Netzes*. Wir untersuchen zunächst den Einfluss der Anzahl der Neuronen n pro Schicht. D. h., wir bleiben bei der Architektur $\mathcal{N}(\tanh; d^2, n, n, n, d)$ und variieren

Tab. 11.5 Training mit Regularisierung. Eigenwerte und Approximation des neuronalen Netzwerks für zwei zufällig gewählte Matrizen, die nicht in den Trainingsdaten enthalten sind

Referenz	−1.594	−1.106	−0.958	0.337	0.641	1.352	1.494	8.638
Netzwerk	−1.746	−1.074	−0.498	0.035	0.569	1.121	1.758	8.655
rel. Fehler	0.018	−0.004	−0.053	0.035	0.008	0.027	−0.031	−0.002
Referenz	−1.932	−0.958	−0.215	0.557	0.651	1.435	2.037	7.843
Netzwerk	−1.698	−0.936	−0.323	0.261	0.845	1.436	2.088	7.839
rel. Fehler	−0.030	−0.003	0.014	0.038	−0.025	−0.000	−0.007	0.001

die Neuronen pro Schicht. Für $n \in \{2, 4, 8, 16, 32, 64, 128, 256\}$ trainieren wir jeweils ein Netzwerk und wenden es im Anschluss auf 1000 zufällige Matrizen an. Für diese bestimmen wir jeweils die relativen Fehler und bilden die Ergebnisse in Abb. 11.17 links ab. Wir zeigen jeweils den Mittelwert sowie die Standardabweichung der Fehler.

Wie erwartet zeigt sich, dass größere Netzwerke besser in der Lage sind, die Eigenwerte zu approximieren. Die Konvergenz ist allerdings nur langsam. Dies kann an weiteren Einflüssen, wie der Anzahl der Schichten oder der Wahl des Regularisierungsparameters γ liegen, die wir hier nicht angepasst haben. Schließlich halten wir auch die Anzahl der Trainingsdaten fest. Es gilt für alle Fälle $N_{tr} = 10\,000$.

Entsprechend zu diesem Test halten wir nun $n = 64$ fest und variieren die Zahl der Schichten L von $L = 1$ bis $L = 8$. D.h., das kleinste Netz hat die Architektur $\mathcal{N}(\tanh; d^2, n, d)$ und das größte hat 8 innere Schichten. Wir zeigen die Ergebnisse in Abb. 11.17 rechts. Tiefere Netze haben bessere Approximationseigenschaften. Aber auch hier zeigt sich, dass die Verbesserung ab einer gewissen Tiefe stagniert. Dies bedeutet nicht, dass das Netzwerk dann keine bessere Approximationseigenschaft hat, sondern eher, dass wir entweder die Anzahl der Trainingsdaten erhöhen müssen, evtl. den Regularisierungsparameter γ anpassen müssen oder auf andere Weise in den Optimierungsprozess eingreifen sollten. Die Verbesserung des ganzen Aufbaus, also die Wahl des Netzes, der Daten, der Methode zur Optimierung und der Regularisierung wird die *Optimierung der Hyperparameter* genannt. Hier ist meist viel Probieren und Handarbeit gefragt.

11.6.3 Einfluss der Trainingsdaten

Wir haben gesehen, dass es nur bis zu einer bestimmten Grenze Sinn macht, die Komplexität des neuronalen Netzwerkes zu steigern. Ab einer bestimmten Grenze führt dies nicht zu

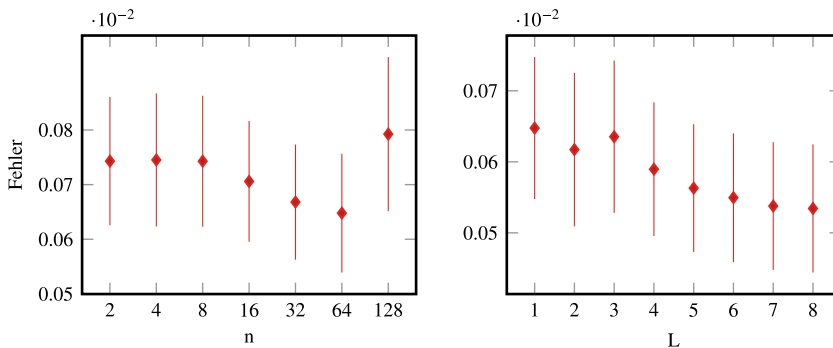


Abb. 11.17 Links: Mittlerer Fehler und Standardabweichung bei wachsender Anzahl von künstlichen Neuronen n pro Schicht. Es gilt stets $L = 4$. Rechts: Mittlerer Fehler und Standardabweichung bei Variation der Anzahl der Schichten L . Hier gilt immer $n = 64$

einer Verbesserung der Anwendung auf Testdaten. Dies liegt daran, dass wir die Anzahl der Trainingsdaten immer gleich belassen haben. In Abb. 11.18 zeigen wir nun die Abhängigkeit der Netzwerkvorhersage für eine wachsende Anzahl von Trainingsdaten. Das Netz selbst hat $L = 4$ Schichten mit jeweils $n = 64$ künstlichen Neuronen (linke Abbildung) und $n = 128$ in der rechten Abbildung.

Es zeigt sich, dass die Approximationsgüte mit steigender Anzahl von künstlichen Neuronen steigt. Aber wieder ist schnell eine Grenze erreicht. Von $N_{tr} = 1000$ bis zu $N_{tr} = 4000$ wird der Fehler schnell kleiner, danach stagnieren die Werte jedoch. Ein weiterer Effekt ist jedoch doch stark verringerte Varianz der Ergebnisse. Dies liegt natürlich daran, dass der Generalisierungsfehler geringer ist. Dieser ist durch den Abstand zwischen einer Validierungsmatrix A und den Testdaten gegeben,

$$\min_{i=1,\dots,N_{train}} \|A - A_i\|^2.$$

In der rechten Abbildung wiederholen wir das Experiment mit doppelter Anzahl von künstlichen Neuronen.

11.6.4 Generalisierung

Bisher haben wir die Netzwerke nur mit Matrizen getestet, die der gleichen X_d entstammen. Insbesondere waren die Einträge der Matrizen bisher alle normalverteilte Zahlen mit $A_{ij} \in [0, 2]$, vergleiche (11.35) (die Einträge von B liegen in $B_{ij} \in [0, 1]$). Wir wenden ein bereits trainiertes Netzwerk zunächst auf Matrizen an, deren Einträge normalverteilt sind

$$Y_d = \{A \in \mathbb{R}^{d \times d}, A = B + B^T, B \in \mathcal{N}(0.5)^{d \times d}\},$$

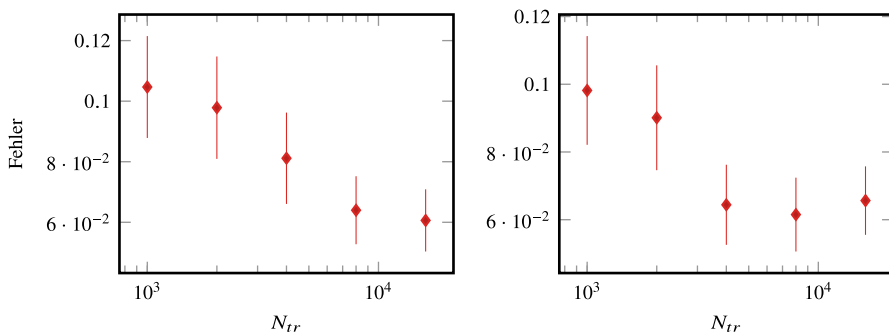


Abb. 11.18 Validierungsfehler (Mittelwert und Standardabweichung) bei steigender Anzahl von Trainingsdaten. Links zeigen wir das Ergebnis für das Netzwerk $\mathcal{N}(\tanh; d^2, 64, 64, 64, 64, d)$ und rechts für ein Netzwerk mit doppelter Anzahl von Neuronen pro Schicht, also $\mathcal{N}(\tanh; d^2, 128, 128, 128, 128, d)$

wobei $\mathcal{N}(0.5)$ eine normalverteilte Zahl mit Standardabweichung 1 und Mittelwert 0.5 ist. Man nennt dies die *Generalisierung* eines künstlichen neuronalen Netzes: *Wie gut ist die Methode auf Daten anzuwenden die beim Training überhaupt nicht vorgekommen sind?*

Wir trainieren hierzu ein Netzwerk mit der Architektur

$$\mathcal{N}(\tanh; 64, 128, 128, 128, 8)$$

mit $N_{train} = 16\,000$ Trainingsmatrizen. Das Training wird nach etwa 40 000 Iterationen abgebrochen, wenn Trainings- und Validierungsfehler beide noch nahe beieinander liegen.

In Tab. 11.6 zeigen wir den mittleren Fehler für die Vorhersage der Eigenwerte von jeweils 1000 zufälligen Matrizen. Zusätzlich geben wir die Standardabweichung der 1000 zufälligen Matrizen an als Maß für die Streuung der Fehler. Wir vergleichen Elemente der Trainingsdaten mit Elementen der Validierungsdaten (also zufälligen Matrizen, die zwar gleichverteilte Einträge haben, aber nicht Teil des Trainings waren) und Generalisierungsdaten aus Y_d . Auf diesem Datensatz liefert das Netzwerk keine brauchbaren Vorhersagen, siehe Tab. 11.6. Die Eigenwerte der Generalisierungsdaten haben einen Vorhersagefehler von durchschnittlich 50 %. Dies ist nicht weiter verwunderlich, da die Matrizen $A \in Y_d$ Einträge haben, die nicht im Intervall $[0, 2]$ liegen müssen, wie es bei allen Trainingsdaten der Fall ist. Normalverteilte Zahlen können größer oder kleiner sein und Einträge dieser Größenordnung hat unser Netzwerk noch nie gesehen. Daher ist auch nicht zu erwarten, dass die Methode gut generalisiert. Wir modifizieren das Beispiel daher etwas: für eine Matrix $A \in Y_d$ bestimmen wir zunächst die betragsgrößte Abweichung vom Mittelwert 1.

$$a_{max} = \max_{i,j} |A_{ij} - 1|.$$

Mit diesem Element wird die Matrix skaliert

$$\tilde{A} = \frac{1}{a_{max}}(A - 1) + 1$$

Tab. 11.6 Generalisierung des künstlichen neuronalen Netzes zur Vorhersage der Eigenwerte einer Matrix. Für 1000 zufällige Matrizen der Trainings- und Testdaten, sowohl für 1000 weitere, diesmal normalverteilte Matrizen, geben wir den mittleren Vorhersagefehler sowie die Standardabweichung der Fehler an

Datensatz	Mittelwert	Standardabweichung
Trainingsdaten	0.058	0.020
Testdaten	0.056	0.021
Generalisierungsdaten	0.537	0.288
Generalisierungsdaten (normiert)	0.042	0.013

und im Anschluss wird $\tilde{A}$ wieder zum Mittelwert 1 verschoben. Für einen Eigenvektor x und Eigenwert λ von A gilt

$$\tilde{A}x = \frac{1}{a_{\max}}(A - 1)x = \left(\frac{\lambda}{a_{\max}} - 1\right)x.$$

Das Netzwerk kann auf $\tilde{A}$ angewendet werden und der resultierende Eigenwert $\tilde{\lambda}$ werden gemäß

$$\lambda = a_{\max}(\tilde{\lambda} + 1)$$

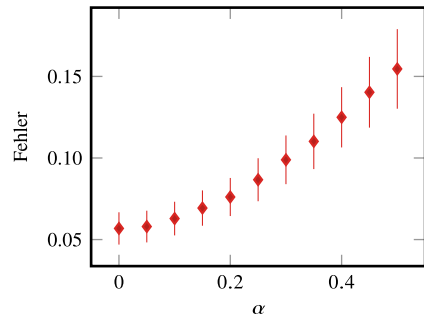
skaliert zu Eigenwerten von A . Die Ergebnisse in der letzten Zeile von Tab. 11.6 zeigen, dass so eine sehr gute Generalisierung auf Matrizen mit normalverteilten Werten erreicht werden kann.

Schließlich untersuchen wir was passiert, wenn wir die Methode auf Matrizen anwenden, die überhaupt nicht mehr symmetrisch sind. Hierzu wählen wir Matrizen der Art

$$Y_{d,\alpha} = \{A \in \mathbb{R}^{d \times d}, A = A_x + \alpha C, A_x \in X_d, C \in \mathcal{N}^{d \times d}\},$$

wobei α ein kleiner Parameter ist. Die eigentlich symmetrische Matrix $A_x \in X_d$ wird mit normalverteilten Zahlen gestört. Die Ergebnisse sind in Abb. 11.19 gezeigt für $\alpha = 0, 0.05, 0.1, \dots, 0.5$. In Tab. 11.7 gegeben wir für $\alpha = 0, 0.2$ und $\alpha = 0.4$ die 8 Eigenwerte für jeweils eine einzelne zufällig gewählte Matrix an.

Abb. 11.19 Generalisierungsfehler für ein das künstliche neuronale Netzwerk, wenn es auf Matrizen mit einer normalverteilten Störung angewendet wird. Der Parameter α gibt die Standardabweichung der Störung an



Tab. 11.7 Eigenwerte für gestörte Matrizen vom Typ $A^T A + \alpha \mathcal{N}$, wobei $\mathcal{N}$ standardnormalverteilte Zahlen sind

	α	λ_1	λ_2	λ_3	λ_4	λ_5	λ_6	λ_7	λ_8
Exakt	0.0	-1.80	-0.72	-0.37	0.15	0.51	1.32	1.99	8.26
Netzwerk	0.0	-1.62	-0.77	-0.38	0.15	0.63	1.31	1.80	8.20
Exakt	0.2	-2.07	-1.33	-0.88	-0.38	0.67	0.84	1.60	7.76
Netzwerk	0.2	-2.08	-1.16	-0.83	-0.34	0.43	0.95	1.54	7.43
Exakt	0.4	-3.34	-1.76	-0.42	0.14	0.70	1.36	2.57	6.59
Netzwerk	0.4	-2.77	-1.57	-0.76	0.24	0.85	1.73	1.89	6.21

Verzeichnis der Exkurse

- 1.6 Exkurs: Steuerfunktion in der Einkommensteuerberechnung 35
- 3.5 Exkurs: LR-Zerlegung auf Parallelrechnern 87
- 3.8 Exkurs: Numerische Approximation des Minimalflächenproblems 121
- 4.6 Exkurs: Astronomie Hauptachsenbestimmung eines Himmelskörpers 170
- 5.6 Exkurs: Eigenschwingungen eines Mehrfach-Federpendels 206
- 5.8 Exkurs: Singulärwertzerlegung zur Bildkompression 236
- 8.5 Exkurs: Nullstellensuche im Komplexen 334
- 8.6 Exkurs: Fast Inverse Square Root 346
- 9.8 Exkurs: Das JPEG-Format zur Komprimierung von Bildern 433
- 9.9 Exkurs: Lösen von Anfangswertproblemen 441
- 10.5 Exkurs: Keplersche Fassregel 484
- 11.4 Exkurs: für Honigverkauf 507

Python-Programme und jupyter-Notebooks

In der zweiten Auflage des Buches wurden zahlreiche Algorithmen als Python-Programme realisiert. Diese finden sich online in Form von jupyter-Notebooks und dienen zum Experimentieren. Auf den Seiten ist die Struktur des Buches nachempfunden und zu den meisten Abschnitten sind Programme hinterlegt:

github.com/sn-code-inside/EinfNumMath-RWW

 Alle Implementierungen im Buch, die auch auf der GitHub Seite zu finden sind, haben wir mit dem Python-Logo hervorgehoben.

Literatur

1. M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Yangqing Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.
2. G. P. Akilov and L. V. Kantorovich. *Functional Analysis in Normed Spaces*. Pergamon Press, 1964.
3. G. S. Almasi and A. Gottlieb. *Highly parallel computing*. Benjamin-Cummings Publishing Co., 2 edition, 1994.
4. H. Amann and J. Escher. *Analysis I*. Birkhäuser, 2006.
5. K. Bach and F. Otto, editors. *Seifenblasen. Eine Forschungsarbeit des Instituts für leichte Flächenwerke über Minimalflächen = Forming bubbles*. Mitteilungen des Instituts für Leichte Flächen-tragwerke. Kämer, Stuttgart, 1980. ISBN 3-7828-2018-5.
6. C. Bär. *Elementare Differentialgeometrie*. De Gruyter, Berlin, Boston, 2 edition, 2010.
7. R. Barrett, M. Berry, T. F. Chan, J. Demmel, J. Donato, J. Dongarra, V. Eijkhout, R. Pozo, C. Romine, and H. van der Vorst. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM, Philadelphia, PA, 1994.
8. U. Becciani, E. Sciacca, M. Bandieramonte, A. Vecchiato, B. Bucciarelli, and M. G. Lattanzi. Solving a very large-scale sparse linear system with a parallel algorithm in the gaia mission. In *2014 International Conference on High Performance Computing Simulation (HPCS)*, pages 104–111, July 2014.
9. P. Benner, A. Cohen, M. Ohlberger, and K. Willcox. *Model Reduction and Approximation: Theory and Algorithms*. SIAM Philadelphia, 2015.
10. J. Bewersdorff. *Algebra für Einsteiger: Von der Gleichungsauflösung zur Galois-Theorie*. Vieweg+Teubner, Wiesbaden, 2007.
11. C. M. Bishop. *Pattern recognition and machine learning*. Springer, 2006.
12. C. M. Bishop and H. Bishop. *Deep Learning. Foundations and Concepts*. Springer Cham, 2024.

13. M. Bonnet. Lecture notes on numerical linear algebra. Méthodes numériques matricielles avancées: Analyse et expérimentation, 2022.
14. Bronstein. Formelsammlung. auf CD.
15. S. L. Brunton and J. N. Kutz. *Daten-driven science and engineering: Machine learning, Dynamical Systems, and Control*. Cambridge University Press, 2019.
16. W. Burkhardt. *Steuerrecht in der Imkerei*. Steuerbuero Burkhardt, 2015.
17. A. Burkov. *The hundred-page machine learning book*. Andriy Burkov, 2019.
18. T. F. Chan. An improved algorithm for computing the singular value decomposition. *ACM Transactions on Mathematical Software*, 8(1):72–83, 1982.
19. F. Cucker and A. G. Corbolan. An alternate proof of the continuity of the roots of a polynomial. *The American Mathematical Monthly*, 96(4):342–345, 1989.
20. G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2:303–314, 1989.
21. W. Dahmen and A. Reusken. *Numerik für Ingenieure und Naturwissenschaftler*. Springer, 2008.
22. J. Demmel, M. Gu, S. Eisenstat, I. Slapničar, K. Veselić, and Z. Drmač. Computing the singular value decomposition with high relative accuracy. *Linear Algebra and its Applications*, 299:21–80, 1999.
23. Bundesministerium der Justiz und Verbraucherschutz. Einkommensteuergesetz. 2002.
24. P. Deufllhard. *Newton Methods for Nonlinear Problems. Affine Invariance and Adaptive Algorithms*, volume 35 of *Computational Mathematics*. Springer, 2011.
25. P. Deufllhard and F. Bornemann. *Gewöhnliche Differentialgleichungen*. De Gruyter, 2008.
26. Duden. *Rechnen und Mathematik: Das Lexikon für Schule und Praxis*. Dudenverlag, 5. ueberarbeitete Auflage, 1994.
27. C. Eck, H. Garcke, and P. Knabner. *Mathematische Modellierung*. Springer, 2008.
28. J. F. Epperson. *An introduction to numerical methods and analysis*. John Wiley & Sons, 2007.
29. D. Etling. *Theoretische Meteorologie*. Springer, 2008.
30. L. C. Evans. *Partial Differential Equations*. Graduate Studies in Mathematics. American Mathematical Society, 2010.
31. J. D. Faires and R. L. Burdon. *Numerische Methoden*. Spektrum Akademischer Verlag, 1994.
32. G. Fischer. *Lineare Algebra*. Vieweg + Teubner, 2010.
33. A. Fog. Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for intel, amd and via cpus. Technical report, 2016. Zugriff am 22.05.2016.
34. E. Freitag and R. Busam. *Funktionentheorie I*. Springer, 2006.
35. R. Freund and R. Hoppe. *Stoer/Bulirsch: Numerische Mathematik I*. Springer, 2007.
36. S. Froeba and A. Wassermann. *Die bedeutendsten Mathematiker*. Marixverlag, 2007.
37. C. Gambella, B. Ghaddar, and J. Naoum-Sawaya. Optimization problems for machine learning: A survey. *European Journal of Operational Research*, 290(3):807–828, 2021.
38. C. Geiger and C. Kanzow. *Numerische Verfahren zur Lösung unrestringierter Optimierungsaufgaben*. Springer, 1999.
39. C. Geiger and C. Kanzow. *Theorie und Numerik restringierter Optimierungsaufgaben*. Springer, 2002.
40. G. Golub and W. Kahan. Calculating the singular values and pseudo-inverse of a matrix. *SIAM Journal of Numerical Analysis*, 2(2):205–224, 1965.
41. G. H. Golub and C. Reinsch. Singular value decomposition and least squares solutions. *Numer. Math.*, 14:403–420, 1970.
42. G. H. Golub and C. F. van Loan. *Matrix computations*. Johns Hopkins University Press, 3rd edition, 1996.
43. I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

44. A. Grama, A. Gupta, G. Karypis, and V. Kumar. *Introduction to Parallel Computing*. Addison-Wesley, 2003. 2nd edition.
45. C. Großmann and H. G. Roos. *Numerische Behandlung partieller Differentialgleichungen*. Teubner, 2005.
46. W. Hackbusch. *Iterative Lösung großer schwachbesetzter Gleichungssysteme*. Number 69 in Leitfäden der angewandten Mathematik und Mechanik. Springer, 1993.
47. G. Hämmerlin and K.-H. Hoffmann. *Numerische Mathematik*. Springer Verlag, 1992.
48. M. Hanke-Bourgeois. *Grundlagen der numerischen Mathematik und des Wissenschaftlichen Rechnens*. Vieweg-Teubner Verlag, 2009.
49. C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.
50. H. Heuser. *Lehrbuch der Analysis Teil 2*. Vieweg+Teubner, 14 edition, 2012.
51. C. F. Higham and D. J. Higham. Deep learning: An introduction for applied mathematicians. *SIAM Review*, 61(4):860–891, 2019.
52. K. Hornik. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2:359–366, 1989.
53. M. Huber, B. Gmeiner, U. Rüde, and B. Wohlmuth. Resilience for massively parallel multigrid solvers. *SIAM Journal on Scientific Computing*, 38(5):S217–S239, 2016.
54. D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proceedings of the I.R.E.*, pages 1098–1101, 1952. Aufgerufen 09/2016.
55. IEEE. IEEE 754-2008: Standard for floating-point arithmetic. Technical report, 2008.
56. W. Keiper, A. Milde, and S. Volkwein. *Reduced-Order Modeling (ROM) for Simulation and Optimization*. Springer Verlag, 2018.
57. J. Kepler. *Nova stereometria doliorum vinariorum*. online by courtesy of Carnegie Mellon University Libraries, 1615.
58. H. Kielhöfer. *Variationsrechnung: Eine Einführung in die Theorie einer unabhängigen Variablen mit Beispielen und Anwendungen*. Vieweg+Teubner Verlag, 2010.
59. M. Koecher. *Klassische elementare Analysis*. Springer, 1987.
60. K. Königsberger. *Analysis I*. Springer, 6 edition, 2004.
61. R. Kress. *Numerical Analysis*. Springer Verlag, 1998.
62. W. Ma, Y. Ao, C. Yang, and S. Williams. Solving a trillion unknowns per second with hpgmg on sunway taihulight. *Cluster Computing*, 23(2):493–507, 2020.
63. G. Maess. *Vorlesungen über numerische Mathematik II, Analysis*. Birkhäuser Verlag, 1988.
64. S. Mandal, A. Ouazzi, and S. Turek. Modified Newton solver for yield stress fluids. In *Proceedings of ENUMATH 2015, the 11th European Conference on Numerical Mathematics and Advanced Applications*, volume 112 of *Lecture Notes in Computational Science and Engineering*. Springer, 2016.
65. C. McEniry. The mathematics behind the fast inverse square root function code. Aufgerufen am 22.05.2016, August 2007.
66. M. A. Nielsen. Neural networks and deep learning, 2018.
67. J. Nocedal and S. J. Wright. *Numerical optimization*. Springer Ser. Oper. Res. Financial Engrg., 2006.
68. B. N. Parlett. *The Symmetric Eigenvalue Problem*. Classics in Applied Mathematics. SIAM, 1998.
69. A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, K. A., E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy,

- B. Steiner, L. Fang, J. Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019.
70. D. Pickenbrock. *Einfuehrung in die Volkswirtschaftslehre und Mikroökonomie*. Physica-Verlag HD, 2008.
 71. R. Rannacher. *Einführung in die Numerische Mathematik*. Heidelberg University Publishing, 2017.
 72. R. Rannacher. *Numerik gewöhnlicher Differentialgleichungen*. Heidelberg University Publishing, 2017.
 73. R. Rannacher. *Numerik partieller Differentialgleichungen*. Heidelberg University Publishing, 2017.
 74. R. Rannacher. *Probleme der Kontinuumsmechanik und ihre numerische Behandlung*. Heidelberg University Publishing, 2017.
 75. T. Rauber and G. Rünger. *Parallele Programmierung*. Springer, 2 edition, 2007.
 76. H.-J. Reinhardt. *Numerik gewöhnlicher Differentialgleichungen. Anfangs- und Randwertprobleme*. Berlin, Boston: De Gruyter, 2008.
 77. C. Reinsch and M. Richter. Singular value decomposition in extended double precision arithmetic. *Numer. Algorithms*, 93(3):1137–1155, 2023.
 78. Y. Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, Philadelphia, PA, 2 edition, 2003.
 79. Y. Saad. *Numerical methods for large eigenvalue problems*, volume 66 of *Classics in Applied Mathematics*. SIAM publications, 2011.
 80. R. Schaback and H. Wendland. *Numerische Mathematik*. Springer, 2005.
 81. J. Schüle. *Paralleles Rechnen*. de Gruyter, 2010.
 82. H. R. Schwarz and N. Köckler. *Numerische Mathematik*. Vieweg-Teubner Verlag, 2011.
 83. G. W. Stewart. *Matrix Algorithms. Volume II: Eigensystems*. SIAM publications, 2001.
 84. T. Strutz. *Bilddatenkompression. Grundlagen, Codierung, Wavelets, JPEG, MPEG, H.264*. Praxiswissen. Vieweg+Teubner Verlag, 2005.
 85. L. N. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM, 1997.
 86. E. E. Tyrtshnikov. *A Brief introduction to Numerical Analysis*. Birkhäuser, 1997.
 87. M. Ulbrich and S. Ulbrich. *Nichtlineare Optimierung*. Mathematik Kompakt. Birkhäuser, 2011.
 88. D. Werner. *Funktionalanalysis*. Springer, 2004.
 89. Wikipedia. Fast inverse square root. https://en.wikipedia.org/wiki/Fast_inverse_square_root, Zuletzt aufgesucht am 24.12.2016.
 90. J. H. Wilkinson. *The algebraic eigenvalue problem*. Numerical Mathematics and Scientific Computation. Oxford Science Publications, 1965.
 91. J. Wloka. *Partielle Differentialgleichungen*. Teubner, Stuttgart, 1982.
 92. W. Zulehner. *Numerische Mathematik: Ein Einführung anhand von Differentialgleichungsproblemen; Band 2: Instationäre Probleme*. Mathematik Kompakt. Birkhaeuser Verlag, 2011.

Stichwortverzeichnis

Symbols

l_1 -Norm, [42](#)

p/q -Formel, [285](#)

A

Abbruchkriterium

Newton, [446](#)

Ableitung

komplexe, [341](#)

Multiindex, [275](#)

Abschneidefehler, [133](#)

Abstiegsrichtung, [489](#)

Abstiegsverfahren, [496](#)

Aktivierungsfunktion, [513](#)

Algorithmus, [11](#)

Definition, [11](#)

Alternante, [415](#)

Anfangsbedingung, [207](#)

Anfangswertaufgabe, [216](#)

Anfangswertproblem, [441](#)

lineares, [444](#)

nichtlineares, [448](#)

Ansatzraum, [241](#)

Approximation, [4](#), [359](#), [512](#)

Approximationssatz von Weierstraß, [276](#)

Approximationstheorie, [276](#)

Armijo-Schrittweitenregel, [491](#)

Arnoldi-Verfahren, [252](#)

Assoziativgesetz, [29](#)

Aufwand, [15](#)

Ausgabeschicht, [515](#)

Auslöschung, [22](#)

B

Backpropagation, [530](#)

Baird-Verfahren, [338](#)

Banachscher Fixpunktsatz, [277](#)

Bandmatrix, [77](#)

Basis, [41](#)

Bernoulli-Polynom, [463](#)

Bernoulli-Zahl, [463](#)

Besetzungsmuster, [77](#)

Bestapproximation, [164](#), [231](#), [522](#)

Bias, [350](#)

Bidiagonalmatrix, [225](#)

Boxregel, [452](#)

C

Cauchy-Riemann-Differentialgleichung, [342](#)

Cauchy-Schwarz-Ungleichung, [43](#)

CG-Verfahren, [242](#)

vorkonditioniertes, [248](#)

Cholesky-Zerlegung, [74](#)

Curse of dimensionality, [521](#)

Cuthill-McKee, [81](#)

D

Darstellungsfehler, [29](#)

Deep neural networks, [515](#)

Defekt, [99](#), [318](#)

eines linearen Gleichungssystems, [99](#)

Defektkorrektur, 99, 100, 318

Diagonaldominanz, 73

strikte, 108

Differentialgleichung, 441

elliptische, 126

partielle, 124, 126

Differenzen, dividierte, 365

Differenzenquotient

der zweiten Ableitung, 386

einseitiger, 384

zentraler, 385

Differenzierbarkeit, komplexe, 341

Diskretisierung, 442

Distributivgesetz, 29, 32

double, 25

double-precision, 25

Double-Well-Potenzial, 304, 480

Durchschnittssteuersatz, 37

E

Eigenvektor, 46

Eigenwert, 46, 173

Eingabeschicht, 515

Einheit, imaginäre, 335

Einheitswurzel, komplexe, 334

Einkommensteuer, 35

Energie

Erhaltung, 206

kinematische, 206

Minimierung, 121

potentielle, 206

Energienorm, 496

Euler-Formel, 335, 424

Euler-Maclaurinsche Summenformel, 464

Euler-Verfahren, implizites, 445

Exkurs

Eigenschwingungen Federpendel, 206

Fast Inverse Square Root, 346

Hauptachsenbestimmung eines Himmelskörpers, 170

JPEG-Format, 433

Keplersche Fassregel, 484

LR-Zerlegung auf Parallelrechnern, 87

Minimalflächenproblem, 121

Newton-Verfahren für Anfangswertprobleme, 441

Nullstellensuche im Komplexen, 334

Preisbildung für Honigverkauf, 507

Steuerfunktion, 35

Exponent, 24

Extrapolation, 136

zum Limes, 389

F

Farbraume, 434

Fast-Fourier-Transformation, 430

Fehler, 18

Fehlerabschätzung, 5, 6

Fehlerquadrate, kleinste, 164, 509

Fehlerverstärkung, 18, 20

fill-ins, 80

Fixkommadarstellung, 24

Fixpunkt

abstoßender, 314

anziehender, 314

Fixpunktiteration, 103

Fließkommaeinheit, 346

float, 25

floating-point processing unit(FPU), 25

FLOPS (floating point operations per second), 31, 87

Fourier-Entwicklung, 396

Fourier-Transformation, schnelle, 430

FPU (floating-point processing unit), 25

Frobenius-Matrix, 57

Frobenius-Norm, 48, 440

Funktion, holomorphe, 341

Funktional, 522

G

Galerkin-Orthogonalität, 241, 245

Galerkin-Verfahren, 241

Gauß-Legendre, 472

Gauß-Quadratur, 470

Gewichte, 477

Gauß-Seidel-Verfahren, 107

Gauß-Tschebyscheff, 480

Generalisierung, 539

generalized minimal residual, 258

Gerschgorin-Kreis, 175

Gewicht, 513

Gewichtsfunktion, 481

Gitterweite, 213

Givens-Rotation, 158

Gleichungssystem, lineares, 51

überbestimmtes, 162, 258
Gleitkommadarstellung, 24
GMRES *siehe* generalized minimal residual
GPU (graphics processing units), 25
Gradientenverfahren, 488, 498
Gram-Schmidt-Verfahren, 45, 148
graphics processing unit (GPU), 25
Gravitationskonstante, 18
Grenzgewinn, 511
Grenzsteuersatz, 36
Größenbestimmung, 18

H

half-precision, 25
Hauptachsenbestimmung, 170
Hessenberg-Matrix, 161, 200
Hidden layer, 515
Hilbert-Matrix, 399
Hilbert-Raum, 43
Honig, 507
Horner-Schema, 11, 13
einfaches, 11
Householder-Transformation, 151, 152
Hutfunktion, 378

I

Imaginärteil, 335
Integration, numerische
Gewichte, 458, 477
Interpolation, 359
Hermiteische, 376
stückweise kubische, 379
trigonometrische, 422
Lagrangesche Darstellung, 362
Newtonsche Darstellung, 364
Intervallschachtelung, 291
Irreduzibilität, 110
Iteration, inverse, 184
mit Shift, 184
Iteration, stationäre, 103

J

Jacobi-Verfahren, 106
JPG-Dateiformat, 433
Julia-Menge, 344

K

Künstliches Neuronales Netzwerk (KNN)
Bias, 513
Keplersche Fassregel, 484
KI (Künstliche Intelligenz), 25
Kolmogoroff Kriterium, 413
Komplexität, 61
Konditionierung, 18
Konditionszahl, 20, 22
einer Matrix, 53
Matrix, 55
Konsistenzfehler, 133
Kontraktion, 277
Konvergenzgeschwindigkeit, 7
Konvergenz, 4
lineare, 17, 313
quadratische, 17
superlinear, 17
superlineare, 313
Konvergenzordnung, 7, 273, 313
Konzept
Approximation, 3, 4
Effizienz, 7, 8
Fehler, 5
Fehlerabschätzung, 5, 6
Konvergenz, 4
Konvergenzgeschwindigkeit, 7
Konvergenzordnung, 7
Numerische Mathematik, 3
Sieben, 3
Stabilität, 9, 10
Kosinus-Transformation, diskrete, 437
Krylow-Raum, 241
künstliche Intelligenz (KI), 25
Künstliches Neuronales Netzwerk (KNN), 360, 512
Aktivierungsfunktion, 513
Gewicht, 513
Rückwärts-Modus, 530
Vorwärts-Modus, 530

L

Lagrange-Interpolation, 361
Nachteile, 374
Lanczos-Verfahren, 257
Landau-Symbol, 16
Laplace-Gleichung, 126
Laplace-Operator, 126

Lastverteilung, 97
Least-Squares-Lösung, 164, 509
Legendre-Polynom, 400
Lernen, maschinelles, 25, 360, 512
LGS (Lineares Gleichungssystem), 51
 variationelle Formulierung, 240
Line-Search, 330, 490
lineare Konvergenz, 313
Lineares Gleichungssystem (LGS)
 stationäre Iterationen, 103
Liniensuche, 330, 490
Links-Vorkonditionierung, 249
Lipschitz-stetig, 277
low-rank approximation, 234

M

magic number, 349, 354
Mantisse, 24
Maschinengenauigkeit, 29
Mathematik, Numerische, 3
Matrix
 Ähnliche, 199
 dünn besetzte, 77
 orthogonale, 140
Matrixnorm, 48
 induzierte, 48
 verträgliche, 48
Maximumsnorm, 42
Methode, direkte, 240
Minimalflächenproblem, 121
Minimierung, 488
Minimierungsproblem, 513
Minimum
 globales, 279
 isoliertes, 279
 lokales, 279
 striktes, 279
Mittelpunktsregel, 452
Mittelwertsatz, 274
Modellierung, 121
Modellmatrix, 78, 118, 129
Moore-Penrose Inverse, 230
Multiindex, 275
multilayer perceptron, 516

N

Nachiteration, 100

Netzwerk, neuronales
 Realisierung, 515
Neville-Schema, 367
Newton trust region, 333
Newton-Cotes-Formeln, 458
Newton-Fraktal, 344
Newton-Globalisierung, 330
Newton-Verfahren, 295
 1D, 294
 Abbruchkriterium, 298
 Defektkorrektur, 318
 Globalisierung, 326
 Komplexwertig, 342
 Startwerte, 446
 Komplexes, 342
Niedrig-Rang-Approximation, 234
Norm, 41
 l_1 , 42
 euklidische, 42
Normäquivalenz, 42
Normalgleichungssystem, 164
Nullstelle, 283
 doppelte, 286
 mehrfache, 283
Nullstellensuche, 283

O

Operation, elementare, 12, 15
Optimierung, 488
 Nichtlineare, 488
Ordnung
 Konvergenz, 313
orthogonal, 43
Orthogonalbasis, 44
Orthonormalbasis, 44
overfitting, 533, 536

P

Parallelisierung, 87
Parallelogrammidentität, 43
Parallelrechner, 88
Pentium
 II, 346
 III, 347
Pivot-Element, 57, 58, 64
Pivot-Matrix, 65
Pivotisierung, 64

- Polynom
 charakteristisches, 173
 trigonometrische, 422
Polynomauswertung, 11
Polynominterpolation, 361
Postprocessing, 391
Potenzmethode, 181
Prä-Hilbert-Raum, 43, 397
Preis-Absatz-Funktion, 507
Preisbildung, 507
Produkt, dyadisches, 151, 152
Projektionsverfahren, 241
Pseudoinverse, 230
Pseudonormallösung, 231
Punkt
 stationärer, 281
 zulässiger, 279
- Q**
QR-Verfahren zur Eigenwertberechnung, 195
QR-Zerlegung, 194
Quadratur, 451, 452
 Gewichte, 452, 458, 477
 numerische, 451
Quake III, 346
- R**
Rückwärts-Modus, 530
Rückwärtseinsetzen, 52, 61
Raum
 euklidischer, 43
 normierter, 42
Rayleigh-Quotient, 174
Realteil, 335
Rechenzeit, 8
Rechnen
 paralleles, 87
 Wissenschaftliches, 2
Rechts-Vorkonditionierung, 249
Referenzwert, 135
Regression, lineare, 508
Regressionsgerade, 508
Regula-Falsi, 313
Relaxation, 113
ReLU-Aktivierung, 513
Remez-Algorithmus, 417
Residuum, 318
Resonanz, 212
RGB-Farbraum, 434
Ritz-Vektor, 256
Ritz-Wert, 256
Romberg-Folge, 466
Romberg-Verfahren, 462
Rundungsfehler, 18
Runges Phänomen, 375
- S**
Sattelpunkt, 281
Satz
 von Kolmogoroff, 413
 von Newton-Kantorovich, 319
 von Rolle, 361, 371
Schema, 11
Schranksatz, 279
Schrittweite, 445
Seifenblase, 121
Sekantenmethode, 309
Sigmoid-Aktivierung, 513
Simpson-Regel, 457
single-precision, 25
Singulärwert, 220
Singulärwertzerlegung, 220
 Approximation, 224
 Numerik, 224
Skalarprodukt, euklidisches, 43
SOR-Verfahren, 113
sparsity pattern, 77
Speicheraufwand, 8
Spektralnrm, 49
Spektralradius, 46
Spektrum, 46
Spitzensteuersatz, 36
Spline, 377, 381
 kubischer, 379
 linearer, 378
 natürlicher kubischer, 380
Stützstelle, 452
Stabilität, 9, 10
Steuerfunktion, 35
Suchweite, 489
SVD, 220
- T**
Taylor-Entwicklung, 275

Tensor Cores, [101](#)
Testraum, [241](#)
Tikhonov Regularisierung, [534](#), [536](#)
Trapezregel, [452](#)
Trennung der Variablen, [444](#)
Tridiagonalmatrix, [77](#)
Tschebyscheff-Approximation, [411](#)
Tschebyscheff-Polynome, [410](#)
Tschebyscheffscher Alternantensatz, [415](#)

U

Überanpassung, [533](#)
Überrelaxation, [114](#)
unitärer Raum, [43](#)
Unterrelaxation, [114](#)

V

Vandermonde-Matrix, [362](#)
Vektorraum
 Dimension, [41](#)
Verfahren
 direktes, [51](#), [285](#)
 iteratives, [51](#)

Verlustfunktion, [522](#)
Vorkonditionierung, [248](#)
Vorschrift, [11](#)
Vorwärts-Modus, [530](#)
Vorwärtseinsetzen, [61](#)

W

Wachstum einer Spezies, [441](#)
Weierstraßscher Approximationssatz, [276](#)
Weinfass, [485](#)

Z

Zahl
 denormalisierte, [27](#)
 komplexe, [334](#)
 konjugiert komplexe, [335](#)
 magische, [349](#), [354](#)
Zeilensummenkriterium
 schwaches, [110](#)
 starkes, [108](#)
Zerlegungsverfahren, [187](#)
Zwischenwertsatz, [273](#)